book 1

c23: the language, the machine, and the cloud

llvm/clang 23, the operating system, terraform, aws, gcp, and a service

Raeliosol

version 4.0, compiled 10 2026

Contents

1. toolchain: clang 23 on windows 8
   1.1. the compiler and the target 8
   1.2. the link model 9
   1.3. the verify gate 9
   1.4. what the windows release omits 10
2. lexical structure and the preprocessor 11
   2.1. translation phases 11
   2.2. elifdef and has_include 13
   2.3. embed in practice 15
   2.4. maximal munch and pp-numbers 17
3. the type system 20
   3.1. objects and representation 20
   3.2. bitint 21
   3.3. constexpr 23
   3.4. typeof, nullptr, unqualified types 24
4. pointers, arrays, and memory 27
   4.1. pointers, arrays, decay 27
   4.2. restrict and aliasing promises 29
   4.3. alignment and aligned allocation 30
   4.4. compound literals and designated initializers 32
5. the abstract machine and undefined behavior 36
   5.1. the abstract machine vs the real one 36
   5.2. the ub contract, tbaa evidence 36
   5.3. checked arithmetic vs raw overflow probes 38
   5.4. sanitizers as oracle 40
6. the c23 library 41
   6.1. stdckdint, checked arithmetic 41
   6.2. sized integers and timespec_get 42
   6.3. qsort, bsearch, and the comparator contract 44
   6.4. defer and stddefer 46
7. llvm: the architecture of the toolchain 50
   7.1. front, middle, and back end 50
   7.2. the clang driver stages, -### 54
   7.3. passes and -Rpass remarks 55
   7.4. the windows tool inventory 56
8. reading llvm ir 58
   8.1. a function through -emit-llvm, the round trip 58
   8.2. ssa and phi 60
   8.3. attributes and metadata 61
   8.4. gep and typed pointers 62
9. optimization: what -O actually does 64
   9.1. mem2reg, -O0 ir versus -O2 ir 64
   9.2. inlining, the call graph collapse 65
   9.3. vectorization evidence 66
   9.4. reading remarks, when -O0 wins 68
10. clang tooling and diagnostics 70
   10.1. diagnostics and -Werror discipline 70
   10.2. clang-format as a gate leg 72

10.3. clang-tidy and the static analyzer ... 72
10.4. clang-query and -ast-dump ... 74
11. processes and program lifecycle ... 77
11.1. the process abstraction ... 77
11.2. CreateProcess end to end ... 80
11.3. fork/exec versus CreateProcess ... 81
11.4. waiting, exit codes, cleanup ... 83
12. virtual memory and paging ... 86
12.1. the mapping fiction ... 86
12.2. page tables and tlbs, simulated ... 88
12.3. virtualalloc versus mmap ... 90
12.4. demand paging and fault cost ... 93
13. caches and locality ... 96
13.1. the hierarchy and line granularity ... 96
13.2. row-major versus column-major ... 99
13.3. false sharing and padding ... 102
13.4. array of structs versus struct of arrays ... 104
14. the heap and allocators ... 107
14.1. the malloc contract ... 107
14.2. a free-list allocator ... 109
14.3. first fit versus best fit fragmentation ... 114
14.4. HeapWalk over a live heap ... 116
15. threads and synchronization ... 120
15.1. threads and the scheduler’s view ... 120
15.2. c23 threads.h and win32 side by side ... 121
15.3. condition variables and wake discipline ... 123
15.4. the portability boundary ... 126
16. atomics and memory ordering ... 128
16.1. stdatomic.h on this target ... 128
16.2. the happens-before ladder ... 130
16.3. a lock-free spsc ring ... 132
16.4. WaitOnAddress versus the futex ... 134
17. cpu scheduling algorithms ... 138
17.1. the model and the metrics ... 138
17.2. fcfs, sjf, srtf ... 139
17.3. round robin, priority with aging ... 142
17.4. multilevel feedback queues ... 146
17.5. cfs and eevdf ... 148
18. asynchronous patterns in c ... 152
18.1. callbacks and ownership ... 152
18.2. an event loop from scratch ... 155
18.3. overlapped io and GetQueuedCompletionStatus ... 159
18.4. epoll and io_uring, cited ... 163
19. allocator designs ... 165
19.1. the allocator’s contract ... 165
19.2. alignment, size classes, and headers ... 165
19.3. bump allocation ... 167
19.4. the slab ... 168
19.5. the buddy allocator ... 170
19.6. a tracking wrapper ... 172
19.7. the production survey ... 174

19.8. the particle kernel's memory budget ... 174
20. measurement, profiling, and p99 ... 176
20.1. clocks, measured ... 176
20.2. the doctrine: assert the count, print the clock ... 177
20.3. percentiles, nearest-rank, by hand ... 177
20.4. a sampling profiler thread ... 178
20.5. attribution without a toolchain ... 180
20.6. warmup, drift, and variance, measured ... 180
20.7. the p99 harness this book reuses ... 181
20.8. across the six books ... 183
21. capstone: particle kernel and row cruncher ... 184
21.1. the kernel behind three allocators ... 184
21.2. array of structs against struct of arrays, measured ... 185
21.3. the row cruncher ... 186
21.4. chunked ingestion and the p99 of a commit ... 188
21.5. the duckdb twin, measured ... 189
21.6. profiled, and why ... 192
21.7. the honest limits ... 193
21.8. the service part ... 194
22. the cloud model ... 196
22.1. iaas, paas, faas, saas ... 196
22.2. regions, zones, failure domains ... 197
22.3. shared responsibility ... 198
22.4. well-architected pillars ... 199
23. terraform: language and core workflow ... 201
23.1. hcl blocks, arguments, and expressions ... 201
23.2. resources, providers, and the graph ... 203
23.3. state anatomy ... 205
23.4. plan and apply ... 207
23.5. version pinning ... 208
24. terraform modules and workflows ... 210
24.1. variables, outputs, locals ... 210
24.2. modules and versioning ... 213
24.3. backends and workspaces ... 215
24.4. fmt, validate, terraform test ... 216
25. aws essentials through terraform ... 219
25.1. iam: users, roles, and the boundary of trust ... 219
25.2. vpc: cidrs, subnets, route tables, firewalls ... 221
25.3. ec2: one instance, end to end ... 224
25.4. s3: the object model against the block model ... 225
25.5. lambda: the execution role and the cold start ... 227
26. gcp essentials through terraform ... 230
26.1. projects and iam ... 230
26.2. the vpc model ... 232
26.3. compute engine instances ... 233
26.4. cloud storage buckets ... 235
26.5. cloud run ... 236
27. multi-cloud practice ... 239
27.1. cost models ... 239
27.2. tagging, a data model with two spellings ... 240
27.3. guardrails, policy as code ... 243

27.4. drift, the plan as detector ... 244
27.5. the provider mapping ... 246
28. http kernel ... 248
28.1. the handler contract ... 248
28.2. parsing http/1.1 requests ... 249
28.3. the route table ... 250
28.4. the error envelope ... 250
28.5. the json codec ... 251
28.6. sockets and the thread pool ... 252
28.7. the composition root ... 253
29. middleware ... 255
29.1. the chain type ... 255
29.2. request-id at the edge ... 255
29.3. the access log ... 256
29.4. the crash guard ... 257
29.5. timeouts at two layers ... 258
29.6. wiring order and why ... 259
29.7. testing middleware ... 260
30. the user resource ... 262
30.1. the record and its projection ... 262
30.2. strict decode ... 262
30.3. validation from first principles ... 263
30.4. merge patch semantics ... 264
30.5. keyset pagination ... 265
30.6. the failure ladder ... 266
30.7. golden vector tests ... 267
31. authn ... 269
31.1. sha-256 from the spec ... 269
31.2. hmac and pbkdf2 ... 269
31.3. password storage with pbkdf2 ... 270
31.4. login and timing flattening ... 271
31.5. server-side sessions ... 272
31.6. cookie mechanics ... 273
31.7. jwt from scratch with hs256 ... 273
31.8. the refresh rotation and reuse detection ... 274
31.9. testing auth without walls ... 275
32. authz and rbac ... 277
32.1. authentication versus authorization ... 277
32.2. roles and the grant table ... 277
32.3. the fixed policy table ... 278
32.4. 401 versus 403 ... 279
32.5. object-level authorization ... 280
32.6. deny by default with a route audit ... 280
32.7. the last admin rule ... 281
32.8. the rbac matrix test ... 281
33. store ... 283
33.1. the dll, the c api, and the pool ... 283
33.2. migrations test driven ... 284
33.3. prepared statements, bound every time ... 284
33.4. the transaction idiom ... 285
33.5. what wal actually gives you ... 286

33.6. the lost update and the cas answer ... 286
33.7. keyset pagination as one row comparison ... 287
33.8. the analytics report in window functions ... 288
33.9. store tests without docker ... 289
34. concurrency control ... 290
34.1. the lost update prevented ... 290
34.2. if-none-match and 304 ... 290
34.3. idempotency keys and the 24 hour snapshot ... 291
34.4. the in-flight window, closed by a table ... 292
34.5. singleflight over the ch16 atomics ... 293
34.6. the primitives, one job each ... 294
34.7. bounded fan-out, first error wins ... 295
35. caching ... 297
35.1. what caching promises ... 297
35.2. http semantics for a private api ... 297
35.3. lru over a fixed table ... 298
35.4. ttl beside lru ... 299
35.5. the stampede and its collapse ... 300
35.6. invalidation, delete after write ... 301
35.7. what never gets cached ... 302
36. rate limiting ... 304
36.1. why limits exist ... 304
36.2. the token bucket from first principles ... 304
36.3. the fixed table and the sweep ... 305
36.4. the 429 contract ... 306
36.5. what to key on ... 307
36.6. limits under test ... 308
36.7. admission preview ... 309
37. observability ... 311
37.1. the access log over the request id ... 311
37.2. one writer and fixed fields ... 312
37.3. traceparent parse and propagate ... 312
37.4. the text exposition format ... 313
37.5. the internal listener ... 314
37.6. redaction ... 315
37.7. one request fully observed ... 315
38. load and p99 ... 317
38.1. the load harness ... 317
38.2. p99 from the histogram ... 317
38.3. admission control ... 318
38.4. graceful shutdown ... 319
38.5. shutdown without sleeps ... 320
38.6. the handle ledger ... 321
38.7. the service under load ... 322
39. ship it ... 324
39.1. the composition root ... 324
39.2. the config story ... 324
39.3. the wiring order and its failure modes ... 325
39.4. the internal listener ... 326
39.5. the live run ... 327
39.6. end to end over the wire ... 328

39.7. shutdown over the drain 329
39.8. the shipped checklist 331
40. the test suite 332
40.1. what the suite holds 332
40.2. the CHECK convention 333
40.3. seams as the double mechanism 333
40.4. the vectors as the spine 334
40.5. the crash guard and the asan leg 335
40.6. fuzzing the parser 336
40.7. what the suite proves 337
41. appendices: topic matrix and sources 339
41.1. topic coverage 339
41.2. pinned sources 339

1. toolchain: clang 23 on windows

Every claim in this book compiles and runs before it is printed. This chapter pins the toolchain that makes that sentence true: llvm/clang 23, built under 23.1.1 (announced 2026-09-08) and drifted on this machine to 23.1.2 (announced 2026-09-22) with every gate green under both, because the pinned asserts ask for the c23 standard version and the major 23 rather than a patch number. The one exception was chapter 8's ir expectation, the only assertion carrying a literal patch number inside the emitted ident metadata, and its first verify under 23.1.2 went red and taught the rule it now follows, the ident pinned at the major. It drives the msvc-compatible toolchain that links against the Visual Studio 2022 libraries installed on this machine. The gcc 15.2 mingw distribution sits next to it as a second opinion, and every divergence between the two is a fact about the standard, not a preference.

1.1. the compiler and the target

The gate compiles every sample with `clang -std=c23 -Werror -Wall -Wextra`, plus one opt-in: `-fdefer-ts`. That flag defines `__STDC_DEFER_TS25755__` and enables the `defer` keyword from technical specification 25755, which chapter 6 uses; probed on this machine it is inert for every file that does not spell the word. The first check in the first sample refuses to run on anything else: `__STDC_VERSION__` must be 202311L, the value the C23 standard assigns, or the book's own harness turns red:

```c
// chapter 1: the compile contract, c23 asserted by the gate, not assumed
#include <stdio.h>

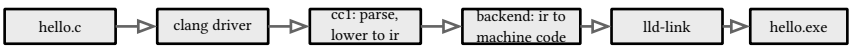
static int checks = 0;

#define CHECK(cond, name)                                                  \
  do {                                                                     \
    if (cond) {                                                            \
      checks++;                                                            \
      printf("ok %d %s\n", checks, name);                                  \
    } else {                                                               \
      printf("FAIL %s (check %d)\n", name, checks + 1);                    \
      return 1;                                                            \
    }                                                                      \
  } while (0)

int main(void) {
  CHECK(__STDC_VERSION__ == 202311L, "stdc version is c23");
  CHECK(printf("hello, c23\n") > 0, "stdout writes");
  return 0;
}
```

listing 1: the check contract every sample carries, and the c23 version gate (source: c-os-cloud/samples/src/Ch01/hello.c, lines 1-21)

The `CHECK` macro is the whole testing convention of this book: a check either prints `ok` and increments the counter or prints `FAIL` and returns a nonzero exit code. The harness counts the `ok` lines, fails on any nonzero exit, and the chapter's closing sources line quotes the count. Nothing is tested by eyeball.



-std=c23 -Werror -Wall -Wextra ride every box from left to right

the target triple is x86_64-pc-windows-msvc, asserted by the next listing

figure 1: the compile pipeline from source to exe, one box per stage the driver runs

The ir the diagram names is llvm's intermediate representation, a low-level typed code format every stage past the front end speaks, from optimization through code generation. Chapter 8 reads it line by line.

1.2. the link model

A C file never becomes an executable alone. The driver folds in the C runtime startup, the ucrt from the Windows SDK, and the import libraries for every win32 api the sample touches. The second sample asserts the environment the linker resolved against:

```
int main(void) {
  CHECK(_WIN32 == 1, "windows target");
  CHECK(__clang_major__ == 23, "clang 23 major");
  CHECK(__STDC_HOSTED__ == 1, "hosted implementation");
  CHECK(sizeof(void *) == 8, "64 bit pointers");
  return 0;
}
```

listing 2: target macros: windows, clang 23 major, hosted, 64 bit pointers (source: c-os-cloud/samples/src/Ch01/target.c, lines 17-23)

`_WIN32` comes from the platform, `__clang_major__` from the compiler, `__STDC_HOSTED__` from the standard's freestanding versus hosted distinction, and the pointer width from the target triple. Four checks, four different sources of truth, one binary.

NOTE: why the gate names kernel32 and synchronization

Clang's msvc-target link step finds the crt and default libraries by scanning the installed Visual Studio toolsets, because its registry auto-detection fails on this machine (probed 2026-09-12: it only ever looked at Visual Studio 8, 9, and 10 paths). Windows api sets are not default link inputs: `WaitOnAddress` lives in the synchronization apiset, so the harness passes `-lkernel32 -lsynchronization` explicitly. Chapters show plain calls; the gate owns the plumbing.

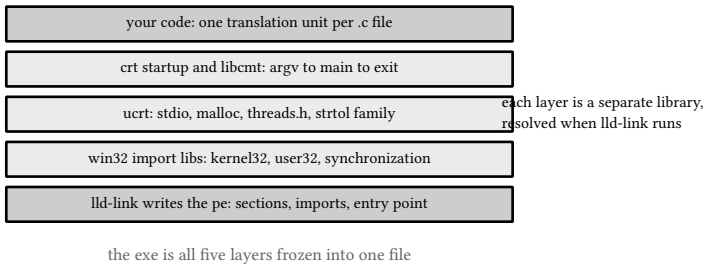


figure 2: the link stack, what each layer contributes to the exe

The pe the bottom layer names is the portable executable, the windows exe format, the file every layer above freezes into.

1.3. the verify gate

`make verify-c` runs the harness in `tools/run-c-samples.ps1`. It resolves the toolchain, then walks every chapter's samples through six legs: compile, run and count checks, clang-format dry run, ir substring assertions, an ll round trip, and an address-sanitizer pass over the chapters whose samples own raw memory. Any leg failing turns the target red, and `make verify` fails with it.

VERIFY: the gate was written test first

The first version of the chapter 1 sample asserted `__STDC_VERSION__ == 199901L`, the c99 value. The harness compiled it, ran it, watched the CHECK fail, and turned red. The fix to `202311L` turned it green. That red-then-green pair is the evidence the harness actually reports failures rather than rubber-stamping output.

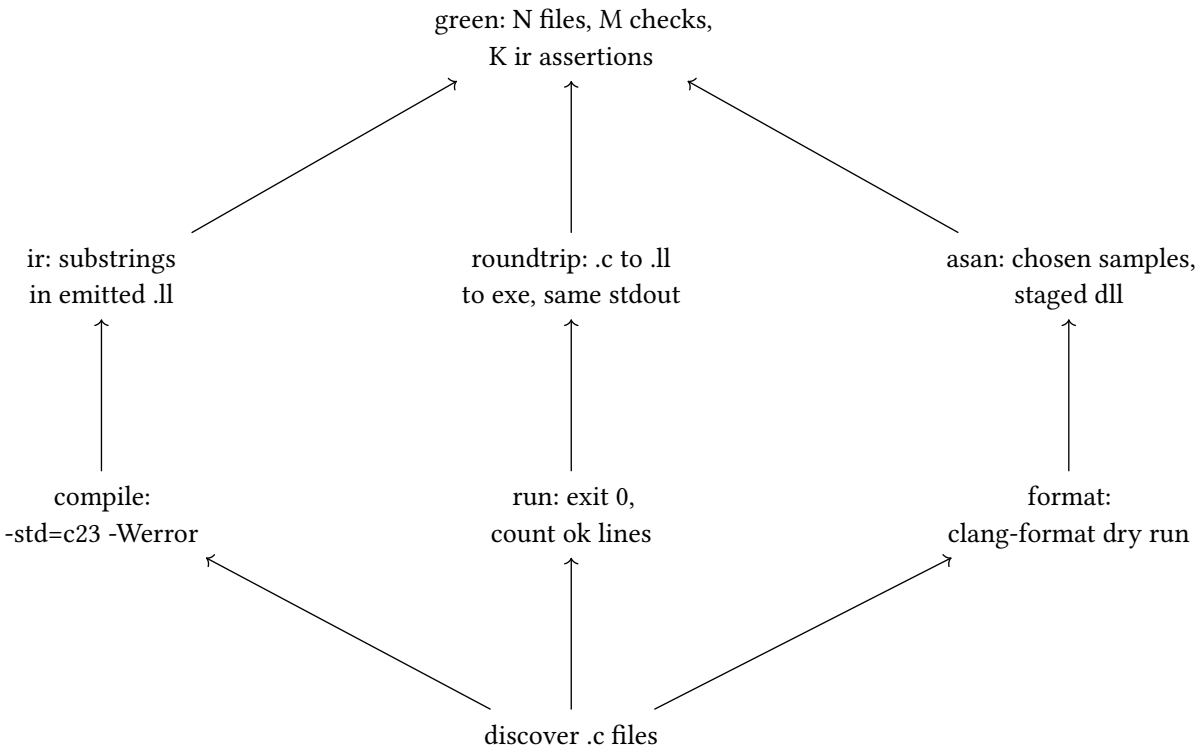


figure 3: one sample through the six legs of the verify gate

1.4. what the windows release omits

The llvm project's windows distribution is a subset of the toolset. This machine's install ships the clang compiler family, clang-format, clang-tidy, clang-query, llvm-objdump, and the lld linkers, and ships no `opt`, `llc`, `llvm-as`, `llvm-dis`, `lli`, or `llvm-config`. The optimizer chapters lose nothing by this: `clang -S -emit-llvm` writes the ir text, the same driver compiles `.ll` back to objects at any `-O` level, and `llvm-objdump -d` disassembles the result. Where the missing tools would appear in a linux walkthrough, the chapters name them and route through the driver instead.

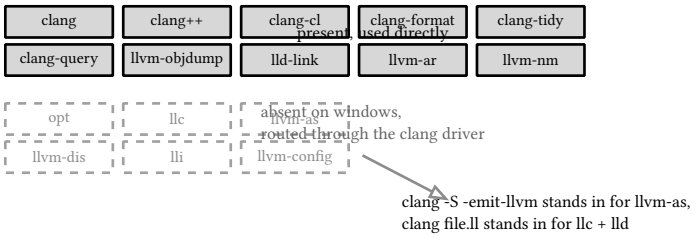


figure 4: the llvm 23 windows bin directory, present versus absent, and the detour each absent tool takes

sources: llvm.org release index (23.1.1 announced 2026-09-08, the 23.1.2 patch announced 2026-09-22, re-checked 2026-09-27) and clang.llvm.org Users Manual (language support list, command line options), accessed 2026-09-12; toolset presence and link behavior probed on this machine the same day. Sample behavior verified by `make verify-c`, 6 checks in chapter 1 of the samples suite.

2. lexical structure and the preprocessor

Before any types or pointers exist, the standard already has opinions. C defines eight translation phases that carry source bytes to a linked program, a preprocessor with three new directives and probes in C23, an `#embed` that reads binary resources at translation time, and a lexer rule that decides how `a+++b` splits before the parser ever runs. Every fact in this chapter is pinned by a check in one of the four samples, or by a diagnostic captured on this machine with llvm/clang 23.1.1 and its installed second opinion, gcc 15.2.

2.1. translation phases

The standard's phase list (n3220, 5.1.1.2) is conceptual: an implementation must behave as if the phases run in order, even though a real compiler folds them together. The order is still load bearing, because each phase observes the output of the one before it. Phase 7 is where the translation unit is born: one source file plus everything its includes bring, parsed as one whole. Later chapters say TU for exactly that. The sample asserts six of those boundaries from inside one binary:

```
// phase 2: the backslash at end of line is deleted before tokenization, so
// these two physical lines are one logical line and one string literal
static const char *spliced = "ab\
cd";

// phase 2 again: the identifier spliced_ident is cut in half by a splice
static int splice\
d_ident = 40;
```

listing 3: phase 2 statics: a string and an identifier each cut by one splice (source: c-os-cloud/samples/src/Ch02/phases.c, lines 18-25)

The string on line 20 ends with a backslash, so phase 2 deletes the backslash and the newline and glues the two physical lines into one logical line. The literal is `"abcd"`, length 4, one token. The identifier `spliced_ident` is spelled across the same kind of splice on lines 24 and 25, and the check on it later compiles against the joined name. Phase 1 in C23 no longer replaces trigraphs: the C17 grammar mapped `??/` to a backslash in any context, C23 dropped that step, and n3220's phase 1 text reads charset mapping only.

```c
int main(void) {
  // phase 1: c23 removed trigraphs from the grammar, clang still warns
  // that it ignores the sequence rather than translating it
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wtrigraphs"
  CHECK(phase6_strlen("??/n") == 4, "phase 1: ??/ is three characters");
#pragma clang diagnostic pop

  // phase 2: one logical line, one literal of length 4
  CHECK(phase6_strlen(spliced) == 4, "phase 2: string spliced to abcd");

  // phase 2: the splice glued the identifier back together
  CHECK(spliced_ident == 40, "phase 2: spliced identifier reads as one");

#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wcomment"
  // phase 3: splicing happens before comment removal, so this comment \
  !!! $$$$ @@@@ swallows the rest of this logical line whole
#pragma clang diagnostic pop
  CHECK(1, "phase 3: the !!! line above never reached the parser");

  // phase 5: escapes translate to the execution character set
  CHECK('\x41' == 'A', "phase 5: escape \\x41 is capital a");

  // phase 6: adjacent string literals concatenate, spelled adjacent on
  // purpose, so the pair stays on one line
  // clang-format off
  CHECK(phase6_strlen("one" "two") == 6, "phase 6: literals concatenate");
  CHECK(strcmp("one" "two", "onetwo") == 0, "phase 6: concatenation order");
  // clang-format on

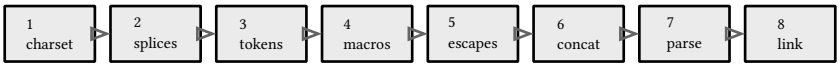
  // phases 7 and 8: the tokens were analyzed as one translation unit and
  // printf was resolved against the c runtime library at link time
  CHECK(&printf != NULL, "phase 8: printf resolved from the library");
```

listing 4: one check per phase boundary, from trigraph absence to library resolution (source: c-os-cloud/samples/src/Ch02/phases.c, lines 29-62)

The first check holds only because trigraphs are gone: `strlen("??/n")` is 4, and under clang's opt-in `-trigraphs` flag in C17 mode the same string collapses to 1 with the warning `trigraph converted to '\'` (probed 2026-09-12). The third boundary is the subtle one: the `//` comment on line 45 ends with a backslash, so phase 2 splices first and the entire `!!! $$$$ @@@@` line becomes part of the comment before phase 3 deletes comments. The compile itself is the proof, and the two `#pragma clang diagnostic` pairs silence exactly the two warnings clang raises for these spellings: `-Wtrigraphs` and `-Wcomment`. Phases 5 and 6 close the token story, `'\x41'` becomes `'A'` in the execution character set and `"one" "two"` concatenates to `"onetwo"`, and phase 8 is why `printf` links at all.

NOTE: the phases carry their own undefined behavior

Three phase boundaries are undefined behavior, not diagnostics: a non-empty file that does not end in a newline after splicing (phase 2), a file ending in a partial preprocessing token or partial comment (phase 3), and token pasting with `##` that produces a universal character name (phase 4). All three come from n3220 5.1.1.2. The samples end in newlines and paste no such tokens, so none of these fire here. Undefined behavior is also one of the standard's three contract classes: undefined, where all bets are off, implementation-defined, where each implementation documents its choice, and unspecified, where the implementation picks among allowed options without documenting which. Chapter 5 owns the full contract, this chapter only needs the names.



phases 1 to 4 are the preprocessor, `#include` re-enters at phase 1

comments are gone by phase 3, each becomes one space

c23 removed the trigraph replacement from phase 1

a missing final newline after phase 2 is undefined behavior

figure 5: the eight translation phases, what each one does, and where the preprocessor ends

2.2. elifdef and has_include

C23 added two conditional directives and standardized three probes: `#elifdef` and `#elifndef` (clang implements N2645 and backports both to C89 modes), `__has_include` (an extension clang shipped for years, now standard), `__has_c_attribute`, and `__has_embed`. The chain in the sample is compiled twice with one macro difference between the passes:

```
// the macro that steers the first chain: defined, so #ifdef takes it
#define GATE 1

static int first_chain(void) {
#ifdef GATE
  return 1;
#elifdef OTHER
  return 2;
#elifndef THIRD
  return 3;
#else
  return 4;
#endif
}

// no steering macros: the chain falls through #ifdef and #elifdef to
// #elifndef, the directive c23 added for the not-defined case
#undef GATE
static int second_chain(void) {
#ifdef GATE
  return 1;
#elifdef OTHER
  return 2;
#elifndef THIRD
  return 3;
#else
  return 4;
#endif
}
```

listing 5: the same chain twice, once with GATE defined and once after `#undef` GATE (source: c-os-cloud/samples/src/Ch02/conditionals.c, lines 17-45)

`first_chain` returns 1: `GATE` is defined at line 18, so `#ifdef` takes the first block and every later directive in the chain is never evaluated. After the `#undef` on line 34 the identical chain returns 3: `#ifdef GATE` fails, `#elifdef OTHER` fails because `OTHER` was never defined, and `#elifndef THIRD` fires precisely because `THIRD` is not defined. One directive per not-defined case is the whole reason `#elifndef` exists, replacing the older `#elif !defined(THIRD)`.

```
// __has_include is only valid inside #if or #elif, so it must flow through
// the directive into a macro before any run time check can see it
#if __has_include(<stdio.h>)
#define HAS_STDIO 1
#else
#define HAS_STDIO 0
#endif

#if __has_include("chapter2-has-no-such-header.h")
#define HAS_GHOST 1
#else
#define HAS_GHOST 0
#endif

// __has_c_attribute returns the year-month the attribute was adopted, and
// 0 for an unknown name
#if __has_c_attribute(deprecated)
#define ATTR_DEPRECATED __has_c_attribute(deprecated)
#else
#define ATTR_DEPRECATED 0
#endif

#if __has_c_attribute(nodiscard)
#define ATTR_NODISCARD __has_c_attribute(nodiscard)
#else
#define ATTR_NODISCARD 0
#endif

#if __has_c_attribute(such_an_attribute_does_not_exist)
#define ATTR_GHOST 1
#else
#define ATTR_GHOST 0
#endif

// __has_embed reports the resource state the same way, one macro per branch
#if __has_embed("flag.bin") == __STDC_EMBED_FOUND__
#define EMBED_FLAG 1
#else
#define EMBED_FLAG 0
#endif

#if __has_embed("chapter2-has-no-such-file.bin") == __STDC_EMBED_NOT_FOUND__
#define EMBED_GHOST 1
#else
#define EMBED_GHOST 0
#endif
```

listing 6: `has_include`, `has_c_attribute`, and `has_embed` routed through directives into macros (source: c-os-cloud/samples/src/Ch02/conditionals.c, lines 47-92)

The `__has_include` probes cannot live in ordinary expressions: clang rejects them outside a directive with `'__has_include' must be used within a preprocessing directive` (captured 2026-09-12 while writing this chapter), so each result flows through an `#if` into a macro and the checks read the macros. `__has_include` evaluates to 1 or 0 for the angle and quoted forms alike. `__has_c_attribute` answers with the year-month the attribute was adopted into the standard: 201904 for `deprecated`, 202003 for `nodiscard`, and 0 for a name the implementation does not know. `__has_embed` reports the

state of a resource with `__STDC_EMBED_FOUND__`, `__STDC_EMBED_EMPTY__`, and `__STDC_EMBED_NOT_FOUND__`, which the next section puts to work.

VERIFY: the chain check failed red before it went green

The first gate run of this sample failed check 2: both chains returned 1, because `GATE` was still defined when the preprocessor reached the second function. The harness printed the FAIL line and turned red. Adding the `#undef GATE` on line 34 turned it green, and that pair is the evidence the `#elifndef` check actually bites.

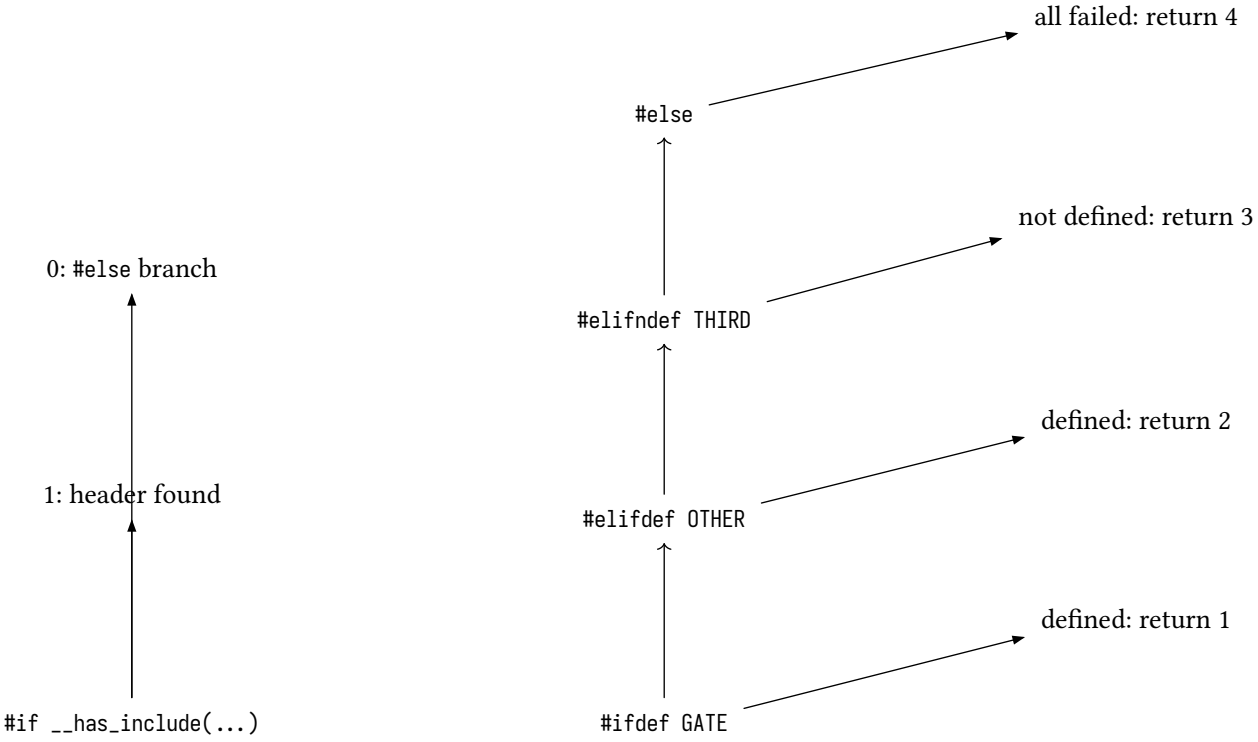


figure 6: the conditional chain walked in order, first true block wins, the rest is skipped

2.3. embed in practice

`#embed` (clang implements N3017 and backports it to C89 and C++ modes) is a preprocessing directive: it expands during phase 4 into a comma-separated list of integer constant expressions, one per byte of the named resource. Initializing an `unsigned char` array from it is specified to behave as if the resource's data were `fread` into the array at translation time. The resource here is four bytes shipped next to the sample:

```c
// flag.bin next to this file holds the four ascii bytes 'c', '2', '3', '\n'.
// clang resolves the quoted resource name against this file's directory,
// so the embed survives the harness compiling from anywhere.
static const unsigned char flag[] = {
#embed "flag.bin"
};

// limit(2) keeps the first two bytes of the resource
static const unsigned char head[] = {
#embed "flag.bin" limit(2)
};

// suffix tokens land immediately after the byte list: a zero terminator
// turns the embedded resource into a c string
static const char text[] = {
#embed "flag.bin"
    , '\0'};

// prefix tokens land immediately before the list: one sentinel on each end
static const unsigned char padded[] = {
#embed "flag.bin" prefix(0x00, ) suffix(, 0x00)
};
```

listing 7: four arrays from one 4 byte resource: full, limited, terminated, and wrapped (source: c-os-cloud/samples/src/Ch02/embed.c, lines 18-39)

`flag.bin` holds the ascii bytes of c23 and a newline, `63 32 33 0a`. The quoted resource name resolves against the directory of the file doing the embedding, verified by compiling from a working directory two levels away (probed 2026-09-12), which is what lets the gate copy sources and binaries around freely. `limit(2)` keeps the first two elements of the resource. The `text` array appends `'\0'` after the expansion and becomes a valid C string. The `padded` array uses `prefix(0x00, )` and `suffix(, 0x00)`: when the resource is non-empty, the prefix tokens land immediately before the byte list and the suffix tokens immediately after, which is how a sentinel or a terminator gets welded onto data the compiler reads.

```c
int main(void) {
  unsigned sum = 0;
  for (size_t i = 0; i < sizeof flag; i++)
    sum += flag[i];

  CHECK(sizeof flag == 4, "embed size matches the file");
  CHECK(flag[0] == 'c' && flag[1] == '2' && flag[2] == '3', "bytes are c23");
  CHECK(flag[3] == '\n', "last byte is newline");
  CHECK(sum == 'c' + '2' + '3' + '\n', "byte sum is 210");

  CHECK(sizeof head == 2, "limit(2) keeps two bytes");
  CHECK(head[0] == 'c' && head[1] == '2', "limit keeps the head");

  CHECK(sizeof text == 5, "nul suffix adds one byte");
  CHECK(strcmp(text, "c23\n") == 0, "terminated embed is a string");

  CHECK(sizeof padded == 6, "prefix and suffix add two bytes");
  CHECK(padded[0] == 0x00 && padded[5] == 0x00, "sentinels on both ends");
  CHECK(padded[1] == 'c' && padded[4] == '\n', "payload between sentinels");
```

listing 8: size, byte, sum, and string checks over the embedded arrays (source: c-os-cloud/samples/src/Ch02/embed.c, lines 41-59)

Eleven checks: sizes 4, 2, 5, 6, the exact bytes, the sum 210, and the strcmp against `"c23\n"`. The ir expectation file pins the constant the bytes become, `@flag = internal constant [4 x i8] c"c23\0A"`, and the round trip leg recompiles

the emitted `.ll` back to an exe and diffs the output: the resource is compile-time data baked into the translation unit, not a file the exe opens.

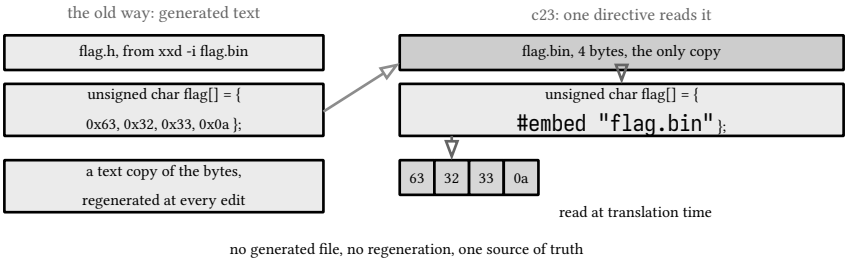


figure 7: generated header versus embed, same four bytes through the old path and the c23 path

2.4. maximal munch and pp-numbers

Lexing in phase 3 is longest-match: if the input has been parsed up to a character, the next preprocessing token is the longest sequence of characters that could constitute one (n3220, 6.4). One exception exists, header names form only inside `#include`, `#embed`, `__has_include`, and `__has_embed`. The sample demonstrates the rule with spellings that survive only because `clang-format` is switched off around them:

```
// # stringifies the pp-tokens of the argument as they were lexed, so the
// spacing of the result exposes the token boundaries the lexer chose
#define STR(x) #x
#define XSTR(x) STR(x)
```

listing 9: the stringify pair: `#` reports the tokens of the argument as lexed (source: c-os-cloud/samples/src/Ch02/munch.c, lines 18-21)

```c
int main(void) {
  // the exact spellings below are the demonstration: clang-format would
  // add the spaces that disprove the point, so they are pinned as written
  // clang-format off
  int a = 1, b = 2;
  int c = a+++b;
  CHECK(c == 3 && a == 2, "a+++b is a++ + b");

  int d = 4, e = 2;
  int f = d---e;
  CHECK(f == 2, "d---e is d-- - e");

  CHECK(1 - -1 == 2, "minus minus needs the space");

  CHECK(strcmp(XSTR(a/*c*/b), "a b") == 0, "a comment becomes one space");

  // 0xE+2 lexes as one pp-number because the grammar lets e sign continue
  // the token: the stringified spelling has no token gap inside it
  CHECK(strcmp(XSTR(0xE+2), "0xE+2") == 0, "0xE+2 is one pp-number");

  // one space before the plus breaks the continuation rule and the same
  // text becomes two pp-tokens, stringified with a single space between
  CHECK(strcmp(XSTR(0xE +2), "0xE +2") == 0, "a space splits the tokens");
  // clang-format on

  // the exponent continuation exists for hexadecimal floating constants,
  // where the p exponent sign is part of the number itself
  CHECK(0x1p+3 == 8.0, "0x1p+3 is one hex float literal");
```

listing 10: munch arithmetic and pp-number proofs, spellings pinned by clang-format off (source: c-os-cloud/samples/src/Ch02/munch.c, lines 23-50)

`a+++b` is `a ++ + b`: the check reads 3 and leaves `a` at 2, which only the `a++ + b` grouping produces. `d---e` is `d-- - e` by the same rule, and `1 - -1` shows the opposite direction, the space is the only thing preventing a `--` token. The stringification checks make the lexer's choices visible: `#` between tokens of the argument's spelling collapses to a single space, so `XSTR(a/*c*/b)` printing `"a b"` is the comment-becomes-one-space rule seen directly, and `XSTR(0xE+2)` printing `"0xE+2"` with no gap means the preprocessor held one token.

That one token is the pp-number. The grammar (n3220, 6.4.8) continues a pp-number with `e sign`, `E sign`, `p sign`, or `P sign`, identical to n1570's C11 grammar, because a hexadecimal floating constant's exponent sign belongs to the number: `0x1p+3` must lex as one token and the final check reads it as 8.0. The cost is that `0xE+2` also matches, making it a pp-number no phase 7 conversion can turn into a valid constant. The two fragments below are illustrative, not executed:

```c
int x = 0xE+2;
```

fragment, illustrative, not executed

gcc 15.2 rejects it with `invalid suffix '+2' on integer constant` (probed 2026-09-12), which is the standard-conforming diagnostic. clang 23 accepts the line silently: its stringification still proves the pp-token was one token, so the split into `0xE`, `+`, `2` happens later, at conversion, with no diagnostic. Portable code treats the spelling as forbidden regardless. The standard's own example of the same trap is `x+++++y`:

```c
int main(void) { int x = 1, y = 2; return x+++++y; }
```

fragment, illustrative, not executed

It lexes as `x ++ ++ + y`, which clang reports as `expression is not assignable`, and the `x ++ + ++ y` reading the programmer wanted is not available without a space.

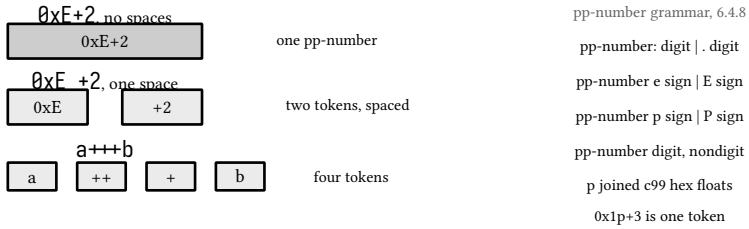


figure 8: the same characters, different token streams, munch decides before the parser runs

sources: open-std.org n3220 working draft, sections 5.1.1.2, 6.4, 6.4.8, and 6.4.9, and n1570 for the pre-c23 pp-number grammar, clang.llvm.org language extensions page (`__has_include`, `__has_c_attribute`, and the backport table naming N3017 and N2645), and cppreference pages for translation phases, conditional inclusion, embed, and floating constants, all accessed 2026-09-12; the trigraph, diagnostic, and gcc 15.2 divergence claims were probed on this machine the same day. Sample behavior verified by `make verify-c`, 35 checks in chapter 2 of the samples suite.

3. the type system

Every size in this chapter is a fact about this machine until the standard says otherwise, and the gate checks each one. The walk follows the order the compiler does: objects and their representations, then `_BitInt(N)`, exact widths and the promotions it refuses, then `constexpr` objects, then the spellings c23 added for naming types and values, `typeof`, `typeof_unqual`, `auto`, and `nullptr`. The support timeline is already history, from llvm's C status page: `_BitInt` and the `() means (void)` rule landed in clang 15, `typeof` in 16, `nullptr` in 17, `auto` in 18, `constexpr` objects in 19. Everything here runs on clang 23.1.1 under `-std=c23`.

3.1. objects and representation

An object is a region of storage. Its declared type says how the bytes are meant to be read, the value is what that reading produces, and the object representation is the raw byte sequence, the layer `unsigned char` is allowed to inspect. The standard fixes almost nothing about sizes: `char` holds at least 8 bits, the chain of `sizeof` values from `char` through `short`, `int`, and `long` to `long long` never decreases, and the target decides the rest. The first sample pins what this target decided:

```
int main(void) {
  CHECK(CHAR_BIT == 8, "a char holds 8 bits on this target");
  CHECK(sizeof(int8_t) == 1 && sizeof(int16_t) == 2 && sizeof(int32_t) == 4 &&
            sizeof(int64_t) == 8,
        "exact width integers are exact");
  CHECK(sizeof(int_least32_t) ≥ 4 && sizeof(int_fast32_t) ≥ 4,
        "least and fast variants are at least 32 bits");
  CHECK(_Generic((int8_t)0, signed char: 1, default: 0),
        "int8_t is a typedef of signed char here");
  CHECK(_Generic((int32_t)0, int: 1, default: 0),
        "int32_t is a typedef of int here");
  CHECK(_Generic((int64_t)0, long long: 1, default: 0),
        "int64_t is a typedef of long long here");
  CHECK(_Generic((char)0, char: 1, signed char: 2, default: 0),
        "char is a distinct third character type");
```

listing 11: the width contract, and which basic type each exact-width typedef names here (source: c-os-cloud/samples/src/Ch03/typesizes.c, lines 20-34)

Line 21 is `CHAR_BIT` from `limits.h`, 8 on this machine. Lines 22-24 assert the exact-width contract from `stdint.h`: `int8_t` through `int64_t` at 1, 2, 4, 8 bytes, with the `least` and `fast` variants at least as wide as promised. The `_Generic` selections at 27-32 identify which basic types the typedefs name here: `int8_t` is `signed char`, `int32_t` is `int`, `int64_t` is `long long`. Line 33 carries the subtlety textbooks skip: `char` is a third character type, distinct from `signed char` and `unsigned char` even where its range matches one of them. The selection proves it twice, because two compatible type names in one `_Generic` would be a constraint violation before any check could run.

```
  CHECK(sizeof(long) == 4 && sizeof(void *) == 8,
        "llp64 windows: long stays narrow, pointers go wide");
  CHECK((int8_t)~0 == -1, "two's complement visible through exact widths");
  uint32_t v = 0x11223344u;
  unsigned char rep[4];
  memcpy(rep, &v, sizeof rep);
  CHECK(rep[0] == 0x44 && rep[1] == 0x33 && rep[2] == 0x22 && rep[3] == 0x11,
        "object representation of this target is little endian");
  CHECK(sizeof(_Bool) == 1 && (_Bool)256 == 1 && !(_Bool)0,
        "bool stores exactly zero or one");
  return 0;
}
```

listing 12: the data model, two's complement, and the bytes behind one uint32_t value (source: c-os-cloud/samples/src/Ch03/typesizes.c, lines 35-46)

Line 35 is the data model: llp64, so `long` stayed 4 bytes when pointers grew to 8, which is why `int64_t` typedefs `long long` and never `long` on windows. C23 made two's complement the only allowed signed representation, and line 37 shows it through the exact width: `~0` read as `int8_t` is −1. Lines 38-42 read the object representation of the value 0x11223344: `memcpy` into an `unsigned char` array is the sanctioned way to look, and the bytes come back 44 33 22 11, the lowest address holds the lowest value bits, little endian on this target. Lines 43-44 close on `_Bool`: one byte of storage, and conversion onto it maps every nonzero value to 1.

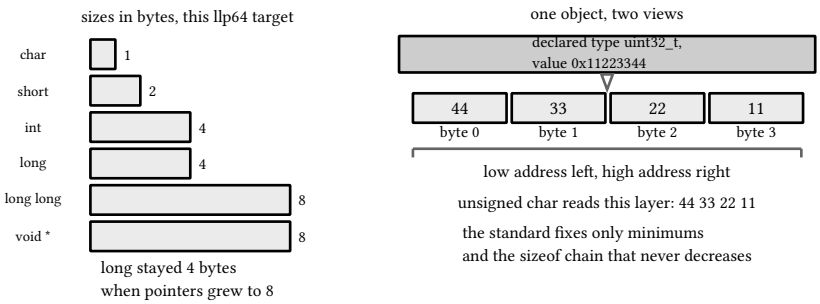


figure 9: sizes this target chose, and the two views of one uint32_t object

NOTE: the portable part is the typedef, not the number

Every size in the two listings is a target fact, not a standard fact: the same file compiled on a 64 bit linux fails the line 35 check, because lp64 makes `long` 8 bytes there. Where a width is a requirement, the exact-width typedefs state it and the `_Generic` checks document which basic type backs each one here. `char` is the trap in the other direction: its signedness is implementation defined, so this chapter spells `signed char` or `unsigned char` whenever the sign matters.

3.2. bitint

`_BitInt(N)` requests exactly N bits, sign bit included, and gives the language integer types at widths no standard type offers. The width is an exact arithmetic fact, not a minimum: `_BitInt(24)` holds −8388608 through 8388607, signed needs at least 2 bits, unsigned goes down to a single bit, and this clang accepts widths up to 2^23, the `__BITINT_MAXWIDTH__` predefine asserted at line 45. The overflow rules are the standard's own, not new ones: the proposal that added the types, N2763, states in its safety section that "(_BitInt(3))7 + (_BitInt(3))2 overflows, and the result is undefined as with other signed integer types", while "overflow of an unsigned `_BitInt` is well-defined and the value wraps around with twos complement semantics". Every wrap asserted below is either unsigned arithmetic, defined modulo 2^N by the standard, or a conversion onto the type, which is implementation defined and probed to reduce modulo on this clang. The C23 spelling `BITINT_MAXWIDTH` is a `limits.h` macro this target's msvc headers do not define, so the compiler predefine carries the fact.

```
_BitInt(24) w = 8388607; // 2^23 - 1, the maximum
CHECK(w == 8388607 && (_BitInt(24)) - 8388608 == -8388608,
      "bitint(24) holds exactly -8388608 through 8388607");
w = w + 1; // the add runs in int, the assignment converts back
CHECK(w == (_BitInt(24)) - 8388608,
      "conversion onto bitint(24) reduces modulo 2^24 on this target");
unsigned _BitInt(24) u = 16777215u; // 2^24 - 1
u = u + (unsigned _BitInt(24))1u;
CHECK(u == 0, "unsigned bitint(24) wrap is defined, modulo 2^24");
_BitInt(2) tiny = -2; // signed range -2..1, the narrowest signed
CHECK(tiny == -2 && tiny + 1 == -1, "bitint(2) holds -2 through 1");
unsigned _BitInt(1) bit = 1; // unsigned range 0..1
bit = bit + (unsigned _BitInt(1))1u;
CHECK(bit == 0, "unsigned bitint(1) holds one bit and wraps to zero");
```

listing 13: exact ranges, and the wraps the standard defines (source: c-os-cloud/samples/src/Ch03/bitint.c, lines 20-33)

Lines 20-22 pin the range, both ends, using in-range values only. Lines 23-25 are the conversion story: `w` ranks below `int`, so `w + 1` runs in `int` and produces 8388608, defined, and the assignment converts that back onto `_BitInt(24)`, where the standard leaves the out-of-range result to the implementation and this clang reduces modulo 2^24 to the minimum. Lines 26-28 are the standard's own wrap: both operands `unsigned _BitInt(24)`, the common type keeps the width, and 2^24 - 1 plus 1 is 0 by definition. `tiny` at 29-30 shows the narrowest signed range on in-range values, −2..1, and `bit` at 31-33 is the narrowest object in the language, one unsigned bit, wrapping to 0 by definition.

```
unsigned _BitInt(8) u8 = 200;
unsigned _BitInt(8) r = u8 + (unsigned _BitInt(8))100;
CHECK(r == 44, "bitint(8) refuses promotion: 300 wraps at 8 bits");
int8_t promoted = 100;
CHECK(promoted + promoted == 200,
      "int8_t promotes to int: 100 plus 100 survives as 200");
_BitInt(64) wide = INT64_MAX;
CHECK(wide == (_BitInt(64))INT64_MAX && sizeof(wide) == sizeof(int64_t),
      "bitint(64) has int64_t range and size, and its overflow rules");
CHECK(sizeof(_BitInt(24)) == 4 && sizeof(_BitInt(64)) == 8,
      "bitint storage rounds up to whole bytes");
CHECK(__BITINT_MAXWIDTH__ == 8388608,
      "this toolchain accepts bit precise widths to 2^23 bits");
CHECK(_Generic(w + 0, int: 1, default: 0),
      "beside an int, a lower ranked bitint converts to int");
return 0;
```

listing 14: the promotion `_BitInt` refuses, the int8_t that gets one, and the 64 bit parity (source: c-os-cloud/samples/src/Ch03/bitint.c, lines 34-49)

The pair at 34-39 is the promotion story, both lanes defined. The `unsigned _BitInt(8)` add refuses the integer promotions: both operands keep 8 bits, the operation happens there, and 200 + 100 wraps to 44, modulo 2^8. The `int8_t` line promotes both operands to `int`, so the same kind of arithmetic survives as 200 with no wrap at all. That refusal is the feature, a `_BitInt(8)` never silently widens into a type with a different range, and N2763 records why: promoting to "avoid" overflow can quietly pick a type that still cannot hold the result. Lines 40-42 set the parity straight: `_BitInt(64)` has `int64_t`'s exact range, size, and overflow rules, signed overflow is undefined in both. Defined modulo arithmetic at widths past 64 belongs to `unsigned _BitInt(N)`, probed working at 65 and 1024 bits on this machine. Storage still pays for whole bytes, lines 43-44: 24 bits occupy 4 bytes, 64 occupy 8. The check at 47-48 records the rank rule from the other side: beside a plain `int`, a lower ranked bit-precise operand converts, so `w + 0` has type `int`. The gate's ir leg pins the widths below the source: the `-O0` ir of this file contains `add i24` and `add i8`, the arithmetic really runs at the requested widths. The lanes lean on two rules used as premises so far. The

integer promotions say an operand narrower than `int` converts to `int` before the operation runs, the lane `int8_t` takes. Integer conversion rank orders the types for conversions: when two operand types must meet, the lower ranked converts toward the higher, the rule `w + 0` records.

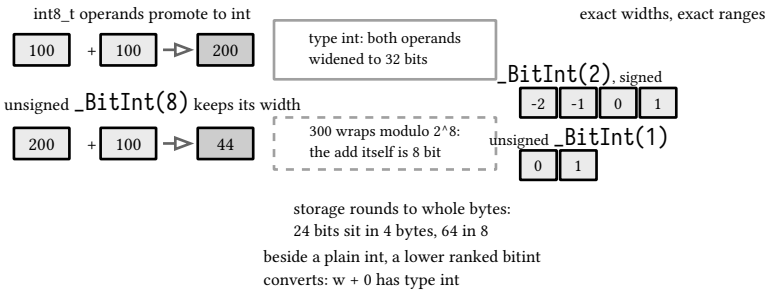


figure 10: the promotion lanes, int8_t widens to int, unsigned `_BitInt(8)` wraps in place

VERIFY: two red drafts behind this section

The first probe declared signed `_BitInt(1)`. Clang refused it, "signed `_BitInt` must have a bit size of at least 2", and the compile went red before any check could run. The first draft of this section then asserted signed `_BitInt` wraparound as defined behavior, because clang 23 runs it deterministically. The proposal text says otherwise, "the result is undefined as with other signed integer types", and a generous compiler is not a contract. The sample now wraps only where the standard defines it, unsigned arithmetic, and labels the conversion wrap as this target's choice. A red draft is the harness doing its job, the discipline chapter 1 established.

3.3. constexpr

C23 `constexpr` is a storage-class specifier for scalar objects, not the c++ keyword. The standard's deal, as cppreference summarizes it: a `constexpr` object "must be fully and explicitly initialized according to the static initialization rules", it "exists at runtime to have its address taken", and it "cannot be modified at runtime in any way", so the compiler may use its fixed value in any other constant expression. The restrictions follow from scalar: no pointers except null pointers, no variably modified, atomic, or volatile types, no `restrict` pointers. There are no `constexpr` functions in c23. That is the c++ difference and the entire difference: c23 names objects, c++ additionally annotates function definitions so calls fold.

```
static int fold_demo(void) {
  constexpr int k = 7;
  return k * 13;
}

enum { E_CAP = 16 }; // the old trick, still only ever int
```

listing 15: an ordinary function holding a local constexpr, beside the enum trick (source: c-os-cloud/samples/src/Ch03/constexpr.c, lines 18-23)

`fold_demo` at 18-21 is a plain static function whose body holds a local `constexpr`. There is no `constexpr` on its declarator, because the grammar has no such position for functions. The enum at 23 is the old trick beside the new keyword: enum constants work in constant expressions, but they are always `int`.

```c
int main(void) {
  constexpr int k = 5;
  _Static_assert(k * 2 == 10, "constexpr object in a static assertion");
  constexpr size_t cap = 16;
  unsigned char buf[cap] = {0};
  int hit = 0;
  switch (2 * k) {
  case k + 5:
    hit = 1;
    break;
  default:
    break;
  }
  const int *addr = &k;
  constexpr double ratio = 0.5;
```

listing 16: constexpr objects feeding every kind of constant expression (source: c-os-cloud/samples/src/Ch03/constexpr.c, lines 25-39)

The walk through `main`: `k` enters a `_Static_assert` at line 27, `cap` sizes `buf` at 28-29, `k + 5` is a case label at 32 with the switch value `2 * k` built from the same constant, `&k` is taken at 38, and `ratio` at 39 names a double, which the enum trick could never do. The checks close the loop: the address reads 5 at line 43, so `constexpr` is an object and not a macro; the enum and `cap` agree on the one thing the enum could do at line 45; and `fold_demo` returns 91 at line 46, which the gate's ir leg also asserts appears in the `-O0` ir of this file as `ret i32 91` inside `fold_demo`: `k * 13` folds before any optimization level is chosen. That is constant folding of a `constexpr` object, not the c++ function machinery.

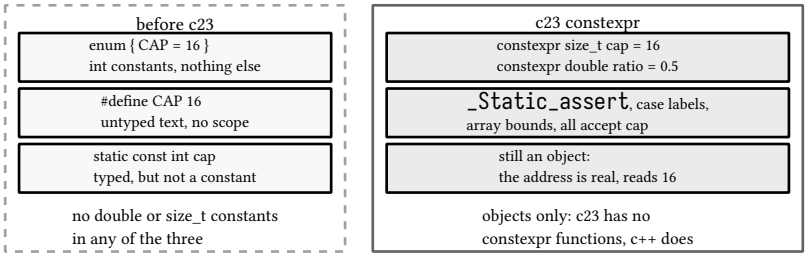


figure 11: named constants before c23 and after

3.4. typeof, nullptr, unqualified types

Four spellings close the chapter. `typeof` and `typeof_unqual` (c23 6.7.2.5) name a type from an expression or a type name: `typeof` preserves every qualifier, `typeof_unqual` strips them, and the operand is not evaluated unless it names a variably modified type. `auto` infers an object's type from its initializer and is no longer a storage class. `nullptr` is a keyword denoting the null pointer constant, a non-lvalue of type `nullptr_t` that converts to any pointer. An lvalue is an expression that designates an object, usually one whose address can be taken, the `register` qualifier being the exception, so `nullptr` names a value, never an object. And the grammar cleanup: `()` in a declarator now means `(void)`, and the K&R identifier-list definition, the pre-ansi parameter style that names parameters in a list after the parentheses, is gone from the language.

```
static int noargs() { return 7; } // c23: () now means (void)

int main(void) {
  int x = 3;
  typeof(x) y = x + 1;
  typeof(1 + 1L) z = 2L;
  CHECK(y == 4 && sizeof y == sizeof x, "typeof copies a declared type");
  CHECK(_Generic(z, long: 1, default: 0), "typeof takes a whole expression");
  const int ci = 42;
  typeof_unqual(ci) m = ci;
  m += 1;
  typeof(ci) *pc = &ci;
  CHECK(m == 43, "typeof_unqual strips the const");
  CHECK(*pc == 42, "typeof keeps the const in the pointee");
  void *vp = nullptr;
  CHECK(vp == nullptr && nullptr == NULL, "nullptr is a null pointer constant");
  CHECK(sizeof(typeof(nullptr)) == sizeof(void *),
        "typeof(nullptr) is the pointer sized null type");
  auto ai = 10;
  auto ad = 0.5;
  auto as = "x";
  CHECK(_Generic(ai, int: 1, default: 0), "auto infers int");
  CHECK(_Generic(ad, double: 1, default: 0), "auto infers double");
  CHECK(_Generic(as, char *: 1, default: 0), "auto infers char *");
  CHECK(noargs() == 7, "empty parens declare a no argument function");
  return 0;
}
```

listing 17: typeof, typeof_unqual, nullptr, auto, and the () grammar in one file (source: c-os-cloud/samples/src/Ch03/typeof.c, lines 17-43)

Line 17 is the grammar fact: `noargs` declared with empty parens compiles and returns 7, and those parens are `(void)` in c23, not an unspecified parameter list. A call with an argument is now a hard error, probed directly: the compiler reports “too many arguments to function call, expected 0, have 1”. Lines 21-24 copy types: `y` gets `x`’s declared type, `z` gets the type of the expression `1 + 1L`, which is `long`. Lines 25-30 are the qualifier pair: `ci` is `const int`, `typeof_unqual(ci)` is `int` so `m += 1` compiles, while `typeof(ci) *pc` keeps the const in the pointee, a pointer to const int. Lines 31-34 put `nullptr` through its paces: it initializes a `void *`, compares equal to `NULL`, and its type is pointer sized. Lines 35-40 deduce `int`, `double`, and `char *` from initializers, the checks dispatching on `_Generic`. The chapter spells the type as `typeof(nullptr)` on purpose:

NOTE: the header trail for nullptr_t on this box

`stddef.h` on this machine resolves to the msvc header set, which defines no `nullptr_t` (probed 2026-09-12, the same shadow that hides `BITINT_MAXWIDTH` in `limits.h`, where clang offers `__BITINT_MAXWIDTH__` at 8388608 instead). Clang ships its own `__stddef_nullptr_t.h`, and its entire c23 content is one line, `typedef typeof(nullptr) nullptr_t`. The sample spells the type the way that header does. The atomics chapter walks the same trail for `stdatomic.h`.

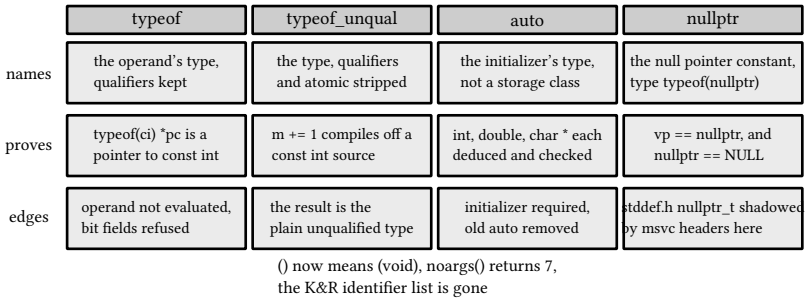


figure 12: the four spellings, what each names and what the sample proves

sources: clang.llvm.org C language support status page (N2763 `_BitInt` and N2841 `()` prototypes in clang 15, N2927 `typeof` in 16, N3042 `nullptr` in 17, N3007 `auto` in 18, N3018 `constexpr` objects in 19), accessed 2026-09-12; open-std.org N2763, the `_BitInt` proposal, safety section (signed overflow "undefined as with other signed integer types", unsigned wrap well-defined), accessed 2026-09-12; cppreference.com pages, constexpr specifier, typeof operators, nullptr, and arithmetic types, citing ISO/IEC 9899:2024 6.7.2.5 and 6.4.4.6, accessed 2026-09-12; every size, wrap, fold, and header shadow probed on this machine the same day. Sample behavior verified by `make verify-c`, 39 checks in chapter 3 of the samples suite.

4. pointers, arrays, and memory

Every object in C is three things at once: storage that occupies bytes, a value held in those bytes, and an address where the storage starts. A pointer is itself an object whose value is such an address. This chapter pins that split with checks, follows the array through its decay into a pointer, reads restrict as a promise the caller makes to the compiler, measures alignment through `alignof` and `_Alignas` up to the allocation calls windows actually ships, and closes with compound literals and designated initializers, the two initializer forms that keep aggregate data readable. Every number below comes from a check that ran or a probe on this machine, dated 2026-09-12.

4.1. pointers, arrays, decay

The sample opens by separating the array from the pointer that points into it. `sizeof a` is `5 * sizeof(int)` because `sizeof` sees the object, the whole storage, not a pointer to it. The pointer `p` is a different object: its own address differs from the address it holds, and its size is one pointer's worth, 8 bytes on this target where chapter 1 asserted 8-byte pointers, against 20 bytes of array. Four checks, one contrast:

```
int main(void) {
  int a[5] = {10, 20, 30, 40, 50};

  // the object is the storage: sizeof measures all of it at once
  CHECK(sizeof a == 5 * sizeof(int), "sizeof sees the whole array");
  CHECK(sizeof a / sizeof a[0] == 5, "element count divides out");

  // the pointer is an object too, with its own address and its own size
  int *p = a;
  CHECK((void *)&p != (void *)p, "pointer object is distinct from pointee");
  CHECK(sizeof a != sizeof p, "array size differs from pointer size");

  // decay: the array expression becomes &a[0] almost everywhere
  CHECK(a == &a[0], "array expression decays to first element");
  CHECK((void *)&a == (void *)&a[0], "array address equals element address");
  CHECK(bytes_seen_by_callee(a) == sizeof(int *),
        "array parameter is a pointer inside");

  // arithmetic is index math: a + i is the address of a[i]
  CHECK(a + 2 == &a[2] && *(a + 2) == a[2], "arithmetic and indexing agree");
  CHECK(&a[3] - &a[1] == 2, "difference is counted in elements");
  CHECK((size_t)((char *)&a[3] - (char *)&a[1]) == 2 * sizeof(int),
        "byte distance is elements times width");

  // one past the end is a valid address to form, compare, and loop to
  int *end = a + 5;
  int walked = 0;
  for (int *q = a; q != end; q++) {
    walked++;
  }
  CHECK(walked == 5 && end > a, "pointer loop spans exactly the array");
```

listing 18: object against pointer, decay, and arithmetic as index math (source: c-os-cloud/samples/src/Ch04/pointers.c, lines 46-76)

Decay is the rule that an array expression becomes a pointer to its first element nearly everywhere. `a == &a[0]` holds after decay, and `(void *)&a == (void *)&a[0]` holds too: taking the array's address and taking the first element's address produce the same address value. `sizeof` is the exception the sample exercises where decay does not happen, which is why `sizeof a` still measures all 20 bytes while every other use of `a` hands a pointer to whoever asks. The call inside the third check hands one across the function boundary:

```c
// an array parameter is a pointer parameter: the array decays at the call,
// so sizeof inside the callee sees one pointer, never the whole array
static size_t bytes_seen_by_callee(int *p) {
  if (p == NULL) {
    return 0;
  }
  return sizeof(p);
}
```

listing 19: the callee receives a pointer, so sizeof sees a pointer (source: c-os-cloud/samples/src/Ch04/pointers.c, lines 20-27)

The parameter is declared `int *p`, and `sizeof(p)` inside equals `sizeof(int *)`. An array declared as a parameter would decay at the call anyway, and the gate refuses `sizeof` on an `[]`-declared parameter outright under -Wsizeof-array-argument, so the pointer spelling is the honest one. Arithmetic then reduces to index math: `a + 2` is the address of `a[2]`, `*(a + 2)` is `a[2]`, the difference `&a[3] - &a[1]` counts elements, and the byte distance between the same two addresses is `2 * sizeof(int)` once the pointers are cast to `char *`. Byte math goes through `char *` because the standard defines no arithmetic on `void *`: clang accepts `vp + 1` as a silent GNU extension even under the gate's -Werror, and only `-pedantic -Werror` rejects it with -Wgnu-pointer-arith (probed 2026-09-12), so the discipline is a habit the gate does not enforce. The loop at the end walks from `a` to the one-past-the-end address `a + 5`: forming, comparing, and adding to that address is defined, dereferencing it never happens.

The second half widens the same rules. Any object pointer converts to `void *` and back without loss, which is the round trip malloc's `void *` return relies on. A two dimensional array nests: `sizeof m` is six ints, `sizeof m[0]` is three, `m` decays to a pointer that steps one row at a time, so `*(*(m + 1) + 2)` is `m[1][2]`, and the last element of row 0 sits immediately before the first element of row 1. A string literal used as an array initializer builds a modifiable 3-byte copy of "hi" including the terminating zero, while a pointer to a literal is just an address:

```c
  // void* holds any object address and converts back without loss
  int x = 7;
  void *vp = &x;
  int *back = vp;
  CHECK(*back == x && (void *)back == vp, "void round trip loses nothing");

  // two dimensional arrays decay one row at a time, rows sit back to back
  int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
  CHECK(sizeof m == 2 * 3 * sizeof(int) && sizeof m[0] == 3 * sizeof(int),
        "2d sizes nest by row");
  CHECK(&m[0][2] + 1 == &m[1][0], "rows are contiguous");
  CHECK(*(*(m + 1) + 2) == m[1][2] && m[1][2] == 6,
        "row then column arithmetic");

  // a literal initializes a copy; a pointer to a literal is just an address
  char s[] = "hi";
  const char *lit = "hi";
  s[0] = 'H';
  CHECK(sizeof s == 3 && sizeof lit == sizeof(char *),
        "copy differs from literal");
```

listing 20: void round trip, row-wise decay of a 2d array, literal copy versus literal pointer (source: c-os-cloud/samples/src/Ch04/pointers.c, lines 78-97)

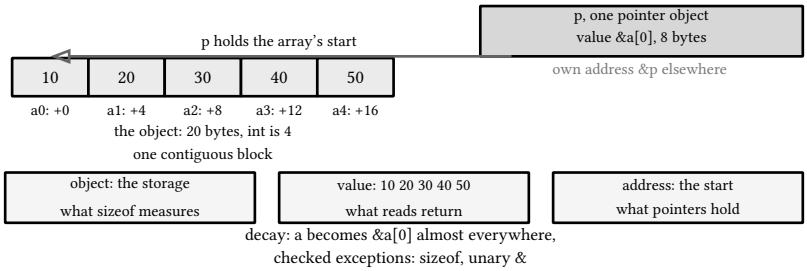


figure 13: the object, the pointer object, and what decay hands over

4.2. restrict and aliasing promises

Two functions in the sample differ by one qualifier. `scale` promises nothing: its `dst` and `src` may alias, so calling it with the same buffer on both sides is defined, each element read before it is written. `scale_restricted` adds `restrict` to both parameters, a promise from the caller that for the entire execution of the call, every access to the objects `dst` reaches goes through `dst`, and every access to the objects `src` reaches goes through `src`:

```
// no restrict promise: dst and src may alias, so an in-place call is defined
static void scale(int *dst, const int *src, int k, size_t n) {
  for (size_t i = 0; i < n; i++) {
    dst[i] = src[i] * k;
  }
}

// restrict promise: for the whole call, every access to the objects dst
// reaches goes through dst, and every access to the objects src reaches goes
// through src
static void scale_restricted(int *restrict dst, const int *restrict src, int k,
                             size_t n) {
  for (size_t i = 0; i < n; i++) {
    dst[i] = src[i] * k;
  }
}
```

listing 21: two loops, one contract apart (source: c-os-cloud/samples/src/Ch04/pointers.c, lines 29-44)

The promise buys the optimizer freedom: it may cache loads, reorder, and vectorize without proving the two streams disjoint, because the caller has sworn they are. This chapter claims the contract, not a speedup, and leaves the ir evidence for chapter 5, where the noalias metadata this qualifier produces is read directly. Clang's own c status page states the caveat: restrict support is fully conforming, but llvm applies its optimization semantics for function parameters, not for local variables or data members. Both uses in this sample are parameters.

main then exercises exactly the calls each contract allows. The plain call scales `vals` in place, 1 2 3 4 becoming 10 20 30 40. The restricted twin runs only on disjoint buffers and produces the same values. `scale_restricted(vals, vals, 10, 4)` would be the same call shape with a broken promise, undefined behavior, and this book does not execute undefined behavior to show it. The library carries the same split in its prototypes: `memmove(text + 2, text, 5)` shifts "abcdefg" into "ababcde" because, per the microsoft docs, memmove ensures the original source bytes in the overlapping region are copied before being overwritten, while the memcpy page states that if the source and destination regions overlap, the behavior of memcpy is undefined, use memmove instead:

```c
// without restrict, aliasing dst and src is defined: each element is read
// before it is written
int vals[4] = {1, 2, 3, 4};
scale(vals, vals, 10, 4);
CHECK(vals[0] == 10 && vals[3] == 40,
      "aliased call defined without restrict");

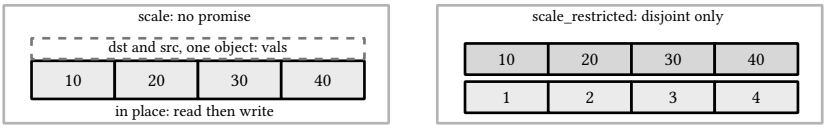
// with restrict, the same call shape would break the promise, so the
// restricted twin only runs on disjoint buffers
int src[4] = {1, 2, 3, 4};
int dst[4];
scale_restricted(dst, src, 10, 4);
CHECK(dst[0] == 10 && dst[3] == 40, "restricted call on disjoint buffers");

// the library carries the same split: memmove overlaps, memcpy may not
char text[8] = "abcdefg";
memmove(text + 2, text, 5);
CHECK(memcmp(text, "ababcde", 7) == 0, "memmove defined on overlap");
char left[8];
char right[8];
memcpy(left, "abcdefg", 8);
memmove(right, "abcdefg", 8);
CHECK(memcmp(left, right, 8) == 0, "disjoint copies agree");
```

listing 22: the legal call for each contract, and the library's own split (source: c-os-cloud/samples/src/Ch04/pointers.c, lines 99-121)

VERIFY: the broken promise is never executed

The one interesting call this chapter does not make is `scale_restricted(vals, vals, 10, 4)`. The promise is false for that call, the standard says undefined behavior, and running it would prove nothing about any other compiler or day. Chapter 5 takes the same discipline further: the noalias metadata the promise produces in llvm ir, read as evidence, never executed as a bug.



the library split: memmove defined on overlap, memcpy not
restrict is that same promise spelled in a parameter list
the aliased restrict call is undefined, never executed

figure 14: one call shape, two contracts, and where each is legal

4.3. alignment and aligned allocation

Alignment is a promise about the low bits of an address: an object with alignment 8 starts at an address divisible by 8. `alignof` measures it, and the probed fundamentals on this target are char 1, int 4, double 8. A struct takes the strictest alignment among its members and pads to make every member sit correctly: `struct mixed` with a char, a double, and a char has alignment 8, the double lands at offset 8 behind seven pad bytes, and the size rounds up to 24 so the next object in an array of them is aligned too:

```c
// a struct's alignment is its strictest member's alignment, here the double
struct mixed {
  char tag;
  double value;
  char flag;
};

// an over-aligned struct: 32 exceeds what plain malloc ever promises
struct over {
  _Alignas(32) char block[16];
};

// static storage: the linker places it at a 32 byte boundary
static struct over parked;
```

listing 23: one padded struct, one over-aligned struct, one static placement (source: c-os-cloud/samples/src/Ch04/align.c, lines 21-34)

`_Alignas(32)` on a member lifts the whole struct: alignof(struct over) is 32 and the size rounds to 32 around 16 bytes of payload. The linker honors the lift for static storage and the compiler honors it on the stack, both checked by taking addresses modulo 32. Plain malloc stops at the fundamental alignment: the microsoft docs state malloc returns storage suitably aligned for any object whose alignment requirement is at most the fundamental alignment, which for visual c++ is the alignment of a double, 8 bytes, and 16 bytes in code that targets 64-bit platforms. The check asserts the returned pointer is a multiple of 16. Thirty-two was never malloc's to promise:

```c
int main(void) {
  // fundamental alignments on this target, probed
  CHECK(alignof(char) == 1 && alignof(int) == 4 && alignof(double) == 8,
        "fundamental alignments on this target");
  CHECK(alignof(int[3]) == alignof(int),
        "array alignment is element alignment");

  // padding places the members and rounds the size up to the alignment
  CHECK(alignof(struct mixed) == alignof(double),
        "struct alignment is strictest member");
  CHECK(offsetof(struct mixed, value) == 8, "double member sits at offset 8");
  CHECK(sizeof(struct mixed) == 24, "size rounds up to alignment");

  // _Alignas lifts the whole struct above the fundamental alignment
  CHECK(alignof(struct over) == 32, "alignas lifts struct alignment");
  CHECK(sizeof(struct over) == 32, "size rounds up to the lifted alignment");
  CHECK((uintptr_t)&parked % 32 == 0, "static over-aligned object honored");
  struct over seat;
  CHECK((uintptr_t)&seat % 32 == 0, "stack over-alignment honored");

  // plain malloc meets the fundamental alignment, and nothing more
  void *plain = malloc(sizeof(struct mixed));
  CHECK((uintptr_t)plain % 16 == 0, "malloc meets x64 fundamental alignment");
```

listing 24: probed alignments, the padding arithmetic, and where malloc stops (source: c-os-cloud/samples/src/Ch04/align.c, lines 36-58)

Above that line is the windows truth. `_aligned_malloc(size, alignment)` from `<malloc.h>` takes an alignment the docs require to be an integer power of 2 and returns a pointer that is a multiple of it, NULL on failure. Its memory returns to the heap only through `_aligned_free`: the docs' own words for using free instead are that it "doesn't reclaim the aligned memory correctly and can lead to hard-to-diagnose bugs". The sample allocates a struct over at 32, writes through both ends of the block, frees it with `_aligned_free`, then asks for 64 to show the request scales with the power of two. The same docs mark malloc and

`_aligned_malloc` with `__declspec(noalias)` and `__declspec(restrict)`, promising the fresh block is not aliased: the allocator's return and the restrict qualifier are two spellings of one aliasing discipline.

```c
// the windows truth: _aligned_malloc from malloc.h, freed by _aligned_free
struct over *heap = _aligned_malloc(sizeof *heap, 32);
CHECK(heap != NULL, "aligned malloc returns memory");
CHECK((uintptr_t)heap % 32 == 0, "32 byte alignment honored on the heap");
heap->block[0] = 7;
heap->block[15] = 9;
CHECK(heap->block[0] + heap->block[15] == 16,
      "over-aligned block is writable");
_aligned_free(heap);

// any power of two up to the allocator's limit can be requested
void *big = _aligned_malloc(64, 64);
CHECK((uintptr_t)big % 64 == 0, "64 byte alignment on request");
_aligned_free(big);
return 0;
```

listing 25: the aligned allocation pair at 32 and 64, freed by `_aligned_free` (source: c-os-cloud/samples/src/Ch04/align.c, lines 60-74)

PITFALL: aligned_alloc is not in ucrt

The standard library has owned `aligned_alloc` since c11. The microsoft runtime does not ship it: a one-line call fails at compile time under the gate with an implicit-declaration error (probed 2026-09-12, and gcc 15.2 mingw with -std=c23 rejects it identically on this machine). cppreference states the reason: the function is not supported in the microsoft c runtime because its implementation of free is unable to handle aligned allocations of any kind, and `_aligned_malloc` freed by `_aligned_free` is provided instead. A defect report against c11, dr460, turned bad-argument calls from undefined behavior into a null pointer return, and the c23 wording asks only for an alignment the implementation supports. None of that changes what this crt links.

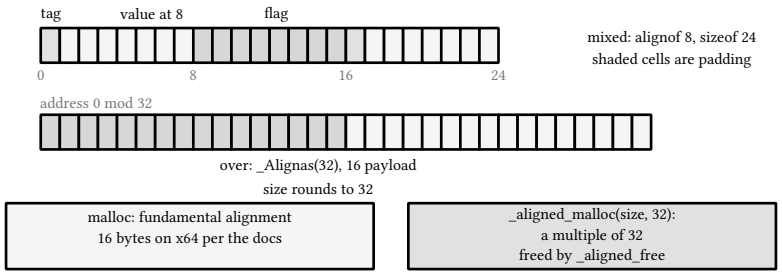


figure 15: padding inside struct mixed, the 32 byte lift, and the two allocators' promises

4.4. compound literals and designated initializers

Designated initializers name what they initialize. `struct point p = {.z = 3, .x = 1}` sets two fields out of order and leaves `y` zero. `.min.x` and `.max.y` descend through nested members. `[4] = 9` and `[1] = 5` punch holes in an array and the gaps read as zero. `[2].y` and `[0].x` combine both kinds on an array of structs. The zeroing is part of the form, checked for a struct, for a nested member, and for array gaps:

```
int main(void) {
  // designators pick fields in any order; unmentioned fields are zero
  struct point p = {.z = 3, .x = 1};
  CHECK(p.x == 1 && p.y == 0 && p.z == 3, "designators pick fields, rest zero");

  // nested designators descend member by member
  struct box b = {.min.x = 1, .max.y = 8};
  CHECK(b.min.x == 1 && b.max.y == 8 && b.min.y == 0 && b.max.x == 0,
        "nested designators reach inside");

  // array designators punch holes; the gaps read as zero
  int v[6] = {[4] = 9, [1] = 5};
  CHECK(v[1] == 5 && v[4] == 9 && v[0] == 0 && v[5] == 0,
        "array designators leave zeros");

  // an array of structs takes both kinds at once
  struct point ps[3] = {[2].y = 7, [0].x = 2};
  CHECK(ps[0].x == 2 && ps[2].y == 7 && ps[1].x == 0 && ps[2].z == 0,
        "array and member designators combine");
```

listing 26: field, nested, array, and combined designators, gaps zero (source: c-os-cloud/samples/src/Ch04/compound.c, lines 52-70)

Against the memberwise habit: three assignments reach the same three values, and the check compares the two objects field by field. The initializer form still wins in practice, because the assignment sequence leaves any forgotten member uninitialized while the initializer form zeroes it:

```
  // designated init equals memberwise assignment, field by field
  struct point d = {.x = 4, .y = 5, .z = 6};
  struct point mw;
  mw.x = 4;
  mw.y = 5;
  mw.z = 6;
  CHECK(d.x == mw.x && d.y == mw.y && d.z == mw.z,
        "designated matches memberwise");
```

listing 27: designated initializer against memberwise assignment, equal values (source: c-os-cloud/samples/src/Ch04/compound.c, lines 72-79)

A compound literal is an unnamed object written as a value: `(struct point){.x = 20, .y = 3, .z = 4}` creates the object in place and passes it by value to `sum3`, whose return proves the fields arrived. The literal is an lvalue, so its address can be taken. Lifetime is the load-bearing fact: at block scope the literal lives for the current execution of the enclosing block, so the sample captures `trio` at the top of a block, runs an unrelated call, then reads the literal again at the bottom, and a loop body re-enters its block each iteration with the literal re-initialized, `i` and `i * 10` arriving fresh every pass. The standard deals in three storage durations: automatic, tied to the enclosing block's execution, static, tied to the whole program run, and allocated, claimed from `malloc` and returned by `free`. At file scope the rule changes: a compound literal evaluated outside a function has static storage duration, the rule cppreference records under c23 6.5.2.5, which is why the `trip` pointer initialized at lines 17-18 of the sample is readable at any point in main, and the closing check reads it:

```c
// a compound literal is a value of its type, usable directly
CHECK(sum3((struct point){.x = 20, .y = 3, .z = 4}) == 27,
      "compound literal as by-value argument");

// its address can be taken; the pointer is valid for the whole block
struct point *q = &(struct point){.x = 9, .y = 8};
CHECK(q->x == 9, "address of compound literal");

// block scope: captured at the top, still valid after other statements
int captured = 0;
{
  const int *trio = (const int[]){5, 6, 7};
  int filler = sum3((struct point){.x = 1, .y = 1, .z = 1});
  captured = trio[0] + trio[1] + trio[2] + filler;
  captured += trio[2];
}
CHECK(captured == 28, "literal lives to end of block");

// each loop iteration re-enters the block: the literal re-initializes
int acc = 0;
for (int i = 0; i < 3; i++) {
  int *pair = (int[2]){i, i * 10};
  acc += pair[0] + pair[1];
}
CHECK(acc == 33, "per-iteration re-initialization");
```

listing 28: by value, by address, block lifetime, per-iteration re-initialization (source: c-os-cloud/samples/src/Ch04/compound.c, lines 81-105)

The struct-of-arrays idiom is where the two forms meet. `struct cols` holds one array per field, the initializer fills column by column, and the row-wise walk over the same storage totals 124 both ways, one by loop and one by the column-summing `total`. The whole table also travels by value as a single compound literal argument. C23 grew storage-class specifiers inside compound literals, `(static int[]){...}`; clang's c status page marks that proposal unimplemented, so this chapter keeps the classic spelling:

```c
// struct-of-arrays: columns initialize together, gaps stay zero
struct cols c = {
    .xs = {1, 2, 3, 4},
    .ys = {10, 20, 30, 40},
    .ids = {5, 0, 0, 9},
};
int by_row = 0;
for (int r = 0; r < 4; r++) {
  by_row += c.xs[r] + c.ys[r] + c.ids[r];
}
CHECK(by_row == total(c), "column init reads back row wise");
CHECK(total(c) == 124, "soa totals to the hand sum");

// the whole table passes by value as one compound literal
CHECK(total((struct cols){
          .xs = {1, 1, 1, 1}, .ys = {2, 2, 2, 2}, .ids = {3, 3, 3, 3}}) == 24,
      "soa literal as argument");

// file scope literal: static storage, readable at any point in main
CHECK(trip[0] == 4 && trip[2] == 6, "file scope literal persists");
return 0;
```

listing 29: column-wise initialization read back row-wise, a by-value table, the file scope literal (source: c-os-cloud/samples/src/Ch04/compound.c, lines 107-127)

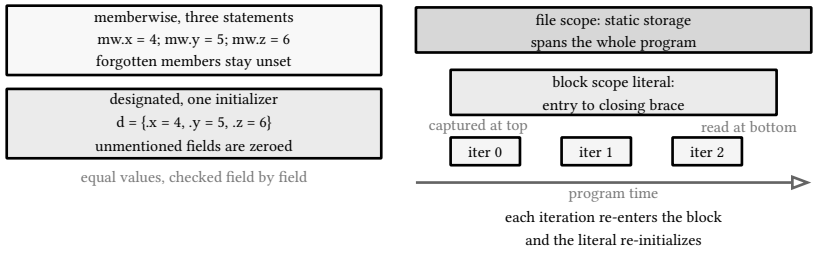


figure 16: two spellings of one initialization, and the two lifetimes of a compound literal

sources: learn.microsoft.com malloc, `_aligned_malloc`, `_aligned_free`, memcpy, and memmove pages (fundamental alignment 8, 16 on 64-bit; power of two alignment; the `_aligned_free` pairing; the memcpy overlap prohibition), clang.llvm.org c language status page (c23 partial, restrict conforming for parameters, compound literal storage class specifiers unimplemented), cppreference compound literal and `aligned_alloc` pages (storage duration rule, c23 6.5.2.5, the ucrt absence and its stated reason), all accessed 2026-09-12; alignment values, the `aligned_alloc` and void pointer arithmetic compile probes, and header behavior probed on this machine the same day. Sample behavior verified by `make verify-c`, 47 checks in chapter 4 of the samples suite.

5. the abstract machine and undefined behavior

The C standard does not describe your computer. The final c23 draft, n3220, states the model in one sentence at 5.1.2.4: the semantic descriptions in the document describe the behavior of an abstract machine in which issues of optimization are irrelevant. The implementation owes the program the observable behavior only, the same clause enumerates it: volatile accesses evaluated strictly, data written into files at termination identical to what abstract execution would have produced, interactive input and output dynamics as specified. Of the three, volatile is the programmer's lever, it tells the compiler every read and write of the object is observable, so none may be elided or merged. Everything else may be rewritten, reordered, or skipped, and that license is the as-if rule the diagram names below.

That gap between the sentence and the silicon is why every behavioral claim in this book answers to one of three machines. A claim about the standard machine cites an n3220 clause. A claim about the compiled machine, clang 23.1.1 on this box, cites the emitted ir or a disassembly. A claim about the hardware cites an observed run. The standard itself sanctions the seam with its own example: the abstract machine promotes two `char` operands to `int` before adding them, and an actual execution may skip the promotions provided the addition is done without integer overflow, or with overflow wrapping silently to produce the correct result. The deal holds only for programs that stay inside the contract. The rest of this chapter is the contract.

5.1. the abstract machine vs the real one

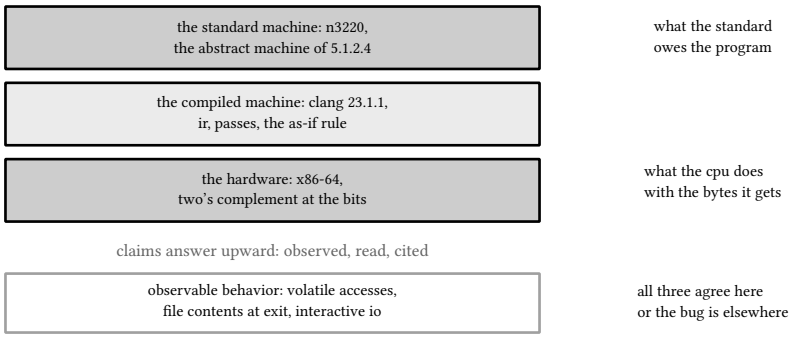


figure 17: three machines and the observable seam between them

The three layers disagree exactly where behavior is undefined, and the standard says what that word costs. 6.5 paragraph 5: if an exceptional condition occurs during the evaluation of an expression, that is, if the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined. Signed overflow is the canonical exceptional condition. The annex that enumerates undefined behavior lists it beside reading a stored value through an lvalue of a non-allowable type, the strict aliasing rule, and 6.5.6 supplies the pointer side: if the pointer operand and the result do not point to elements of the same array object or one past the last element, the behavior is undefined. Undefined means the standard imposes no requirements at all. No crash is guaranteed, no wrap is guaranteed, no diagnosis is required.

5.2. the ub contract, tbaa evidence

PITFALL: this book never executes the violation

A program whose execution reaches undefined behavior has no obligations left, so a run that happened to wrap demonstrates nothing about the next compiler or the next flag. Every sample in this chapter keeps every promise. The evidence for what the optimizer assumes comes from the ir emitted for clean code and from the standard's own sentences, never from a crash the gate coaxed into existence.

The samples keep the aliasing promise on record and let the compiler show what it does with it. Two functions differ by one word:

```
// external linkage, so the body survives in the .ll at any -O level; the
// restrict qualifiers surface as noalias on the two pointer arguments
void scale_accum(int *restrict out, const int *restrict in, int n, int k) {
  for (int i = 0; i < n; i++)
    out[i] = out[i] + in[i] * k;
}

// the same loop without the promise: the arguments get no noalias, so -O2
// must guard any reordered or vectorized body with runtime overlap checks
void scale_accum_plain(int *out, const int *in, int n, int k) {
  for (int i = 0; i < n; i++)
    out[i] = out[i] + in[i] * k;
}
```

listing 30: the restrict twin and the plain twin, one word apart (source: c-os-cloud/samples/src/Ch05/restrict_tbaa.c, lines 18-30)

The `restrict` qualifiers are a promise from caller to compiler that the arrays behind `out` and `in` do not overlap for the whole call. The caller keeps it:

```
  // distinct arrays, so the restrict promise is kept, never broken
  scale_accum(b, a, N, 10);
  scale_accum_plain(c, a, N, 10);
  CHECK(b[0] == 0, "restrict loop leaves the zero start alone");
  CHECK(b[1] == 11, "restrict loop accumulates in place");
  CHECK(b[7] == 77, "restrict loop end value");
  CHECK(c[1] == 11 && c[7] == 77, "plain twin computes the same values");
  CHECK(a[1] == 1 && a[7] == 7, "input array is never written");
```

listing 31: distinct arrays in, same arithmetic asserted out of both twins (source: c-os-cloud/samples/src/Ch05/restrict_tbaa.c, lines 40-47)

Emitting the ir with the gate's own command, `clang -std=c23 -S -emit-llvm -O2`, shows the promise banked. At -O0 the restrict function already reads `define dso_local void @scale_accum(ptr noalias noundef %0, ptr noalias noundef %1, ...)` while the plain twin's arguments carry nothing. At -O2 the difference becomes the whole body. The restrict version walks straight into a vector body, `<4 x i32>` loads and stores with no pointer test anywhere. The plain twin first computes two `icmp ult ptr` comparisons and branches on their conjunction, a runtime overlap check guarding the vector body, and the loads inside that versioned body carry `!alias.scope` and `!noalias` metadata the compiler synthesized from its own check. The pointer math inside both vector bodies is `getelementptr inbounds`, the optimizer's own restatement of the promise 6.5.6 writes for source code, that every computed address stays inside its array object or one past its end. The gate pins all four facts in `Ch05/expect-ir.txt`: `noalias` at -O0 and -O2, `icmp ult ptr` and `getelementptr inbounds` at -O2.

One piece of the planned evidence does not exist on this target. The plan asked for `!tbaa`, the type-based alias analysis metadata, in the -O2 ir. The probe found why it is absent and why no flag brings it back. `clang -###` shows the driver injecting `-relaxed-aliasing` into cc1 for the x86_64-pc-windows-msvc target, which turns tbaa off to match msvc semantics, and the classic `-fstrict-aliasing` spelling no longer exists in this frontend: cc1 rejects it as an unknown argument and the driver drops it from the cc1 line, so there is no way to re-enable tbaa on this target at all. The same twin shape on a linux triple grows `!tbaa` on every load and store by default, probed 2026-09-12. The aliasing evidence in this book's gate is `noalias` and the versioned-loop metadata. The contract itself still binds. Reading an `int` object through a `float` lvalue is undefined today, listed by the same annex as the exceptional condition, and `restrict` only adds promises on top of it. Chapter 4 treats restrict as an interface tool. Here it is a contract whose banking is visible.

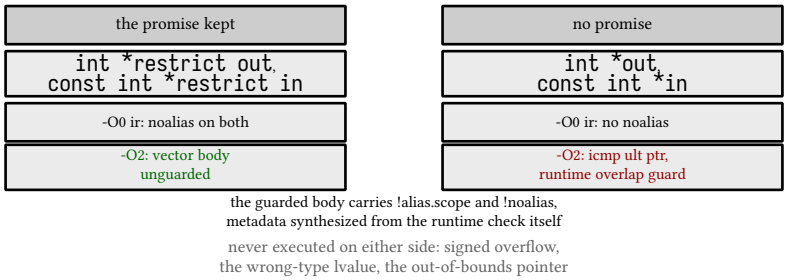


figure 18: one word of promise, two machines: what the -O2 ir shows for each twin

5.3. checked arithmetic vs raw overflow probes

The textbook overflow shapes sit in the sample as evidence, executed only on arguments whose results fit:

```
// the textbook overflow shapes: -O2 refines each one to a single
// comparison against int_max, the exact truth table two's complement wrap
// would give, even though the standard leaves the overflowing evaluation
// itself undefined
int no_overflow(int x) { return x + 1 > x; }
int wraps(int x) { return x + 1 < x; }

// bit-precise width 37, which no builtin type has; called only with values
// whose result fits, so the function itself never overflows
_BitInt(37) bitint_probe(_BitInt(37) x) { return x + 1; }
```

listing 32: the comparison shapes and the width 37 probe, all run on safe inputs (source: c-os-cloud/samples/src/Ch05/ubprobes.c, lines 20-29)

The folklore says the optimizer folds `x + 1 > x` to constant true because overflow is undefined. This compiler cannot: `clang -###` shows the driver passing `-fwrapv` to cc1 for the msvc target, so signed arithmetic is compiled as wrapping by toolchain contract and no add carries the `nsw` assumption the fold needs. What it does instead: at -O2 `no_overflow` reduces to a single comparison, `icmp ne i32 %0, 2147483647`, and `wraps` reduces to `icmp eq i32 %0, 2147483647`. Both are the exact truth table two's complement wrap would compute, one comparison instead of an add and a compare, and they agree with the wrap machine on every input. The undefined-behavior latitude lives in the standard machine, and this toolchain spends it nowhere, it buys defined wrap for every translation unit instead. Where the contract is visibly banked is the guard elision above and the checked-arithmetic intrinsics below.

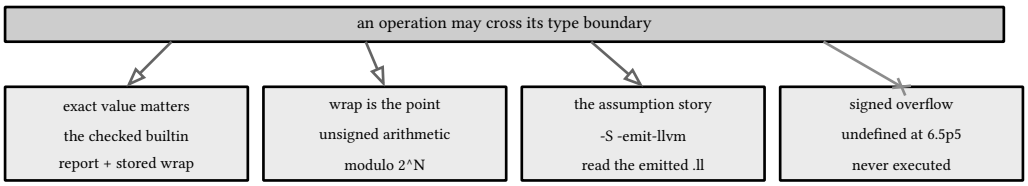


figure 19: arithmetic near the range edge: the decision this book makes every time

The checked route is defined for every argument pair. The gcc documentation of the builtin family, which clang implements, states the deal: the operands are promoted into an infinite precision signed type, the result is cast to the type the third pointer points to and stored there, the function returns false when the stored result equals the infinite precision result, and the behavior is fully defined for all argument values. The truth table in the sample:

```
int main(void) {
  // checked arithmetic: defined for every argument pair, overflow is
  // reported, not executed
  int r = 0;
  CHECK(__builtin_add_overflow(INT_MAX, 1, &r) == 1,
        "checked add reports int_max + 1");
  CHECK(r == INT_MIN, "checked add stores the wrapped two's complement value");
  CHECK(__builtin_add_overflow(INT_MAX - 1, 1, &r) == 0,
        "checked add accepts int_max - 1 + 1");
  CHECK(r == INT_MAX, "checked add stores the exact value when it fits");
  CHECK(__builtin_sub_overflow(INT_MIN, 1, &r) == 1,
        "checked sub reports int_min - 1");
  CHECK(r == INT_MAX, "checked sub wraps to int_max");
  CHECK(__builtin_mul_overflow(INT_MAX, 2, &r) == 1,
        "checked mul reports int_max * 2");
  CHECK(r == -2, "checked mul wraps to -2");
```

listing 33: add, sub, and mul truth tables, report plus stored two's complement value (source: c-os-cloud/samples/src/Ch05/ubprobes.c, lines 31-46)

The overflow return is the standard-grade fact and the stored value is the cast landing on this target's two's complement representation: `int_max + 1` stores `int_min`, `int_min - 1` stores `int_max`, `int_max * 2` stores −2. In the emitted ir the builtins never become plain arithmetic, they lower to `llvm.sadd.with.overflow.i32` and its sub and mul siblings, and the gate pins that at -O0, where the lowering happens in the frontend. When wrap is the point, the standard sells it directly. A computation involving unsigned operands can never overflow because the arithmetic is performed modulo 2^N, and 3.28 gives the process its name: wraparound, the reduction of a value modulo 2^N where N is the width of the resulting type.

```
  // unsigned arithmetic is defined modulo 2 to the 32: wrap is legal here
  unsigned u = 2147483647u;
  CHECK(u + 1u == 2147483648u,
        "unsigned wrap past the signed range is defined");
  CHECK(4294967295u + 1u == 0u, "unsigned arithmetic reduces modulo 2^32");

  // bit-precise unsigned wrap at width 37, a width no builtin type has
  unsigned _BitInt(37) w = 1;
  for (int i = 0; i < 36; i++)
    w = w * 2;
  CHECK(w == (unsigned _BitInt(37))68719476736ull,
        "width 37 holds 2^36 at the top");
  w = w * 3;
  CHECK(w == (unsigned _BitInt(37))68719476736ull,
        "3 * 2^36 wraps to 2^36 modulo 2^37");
```

listing 34: defined wrap at width 32 and at bit-precise width 37 (source: c-os-cloud/samples/src/Ch05/ubprobes.c, lines 48-62)

NOTE: generosity is not a contract

The figure plan for this chapter said to assert signed wrap through `_BitInt`. The standard disagrees with the plan. Footnote 32 of 6.2.5: any statement about signed integer types also applies to the bit-precise signed integer types unless otherwise noted, and no note grants them wraparound. Signed `_BitInt` overflow is undefined like every other signed overflow, so this book does not run it. The compiled machine is currently more generous, `bitint_probe` emits a plain `add i37` with no assumption flag at -O0 or -O2, and the gate pins only the structural fact that the width 37 add exists. What a compiler does today is an observation. What it may do tomorrow is the contract.

5.4. sanitizers as oracle

The last leg of the gate recompiles `ubprobes.c` with `-fsanitize=address` and runs it. AddressSanitizer instruments every memory access against shadow memory. The design maps 8 bytes of application memory into 1 shadow byte, on 64-bit via shadow equals `(mem >> 3) + 0x7fff8000`, and encodes the verdict per shadow byte: 0 for fully addressable, negative for poisoned, 1 through 7 for the first k bytes addressable at the tail of an allocation. It detects out-of-bounds accesses to heap, stack and globals, use-after-free, double-free and invalid free, produces no false alarms, and exits on the first error it finds. A sample that probed out of bounds would abort the leg, which is why `ubprobes.c` owns arithmetic truth tables and no memory violation: what the sanitizer would catch is explained here with the encoding above, not executed.

The windows detail is the runtime. The instrumented exe imports `clang_rt.asan_dynamic-x86_64.dll` from the compiler resource directory, and the gate copies that dll next to the exe before running. Probed: the unstaged exe dies with exit code `0xc0000135`, dll not found, before `main` runs, and the staged one passes all 15 checks. UndefinedBehaviorSanitizer would check the contract violations directly, its default group includes `alignment`, `bounds`, `null`, `shift` and `signed-integer-overflow`, reporting `runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'` and continuing by default. It is a cited command in this book, never a gate leg, because it does not run on this box: the standalone runtime fails to link against the plain gate line with undefined sanitizer-internal symbols, and forcing the asan import library in produces an exe that dies at startup with `0xc0000139`, entry point not found. Probed 2026-09-12.

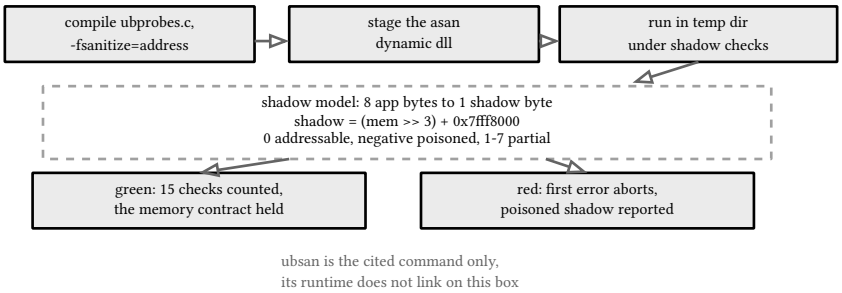


figure 20: the sanitizer leg: instrument, stage, run, and the two ways the run ends

Together with the other legs the oracle covers the chapter end to end. The compile leg proves the probes are warning-clean c23, the run leg counts every check, the format leg freezes the source shape, the ir leg proves the structural facts this chapter quoted, `noalias`, `icmp ult ptr`, `getelementptr inbounds`, the width 37 add, the overflow intrinsics, and the asan leg proves the memory contract holds on the sample the gate nominates for it.

sources: open-std.org n3220 (5.1.2.4 abstract machine and observable behavior, 6.5p5 exceptional condition, 3.28 wraparound, 6.5.6 pointer arithmetic, 6.2.5 footnotes 32 and 33, the undefined behavior annex), accessed 2026-09-12; gcc.gnu.org integer overflow builtins and clang.llvm.org language extensions, the `_BitInt` support and abi notes, accessed 2026-09-12; clang.llvm.org UndefinedBehaviorSanitizer and AddressSanitizer pages and the sanitizer project's AddressSanitizerAlgorithm wiki, the shadow mapping and encodings, accessed 2026-09-12; the ir facts, the dll staging, and the ubsan link behavior probed on this machine the same day. Sample behavior verified by `make verify-c`, 20 checks in chapter 5 of the samples suite.

6. the c23 library

The library is where the standard touches running code. Four samples pin four corners of it on this toolchain: checked arithmetic from `stdckdint.h`, the sized integers and the clock from `stdint.h` and `time.h`, the sort and search contract from `stdlib.h`, and `defer`, the newest arrival, riding in from technical specification 25755 through clang's `stddefer.h`. Every number in the prose below was printed by a check the gate counted.

6.1. stdckdint, checked arithmetic

C23 answers the oldest question in C, did that operation wrap, with three macros. `ckd_add`, `ckd_sub`, and `ckd_mul` take a pointer to the result, two operands, and return `bool`: true when the mathematical result did not fit the result type. The wrapped value is stored either way, and the truth table fits in one listing:

```
// the truth table: every row below is signed or unsigned wraparound,
// reported as true, with the wrapped value still stored in the result
int r;
bool of = ckd_add(&r, INT_MAX, 1);
CHECK(of == true && r == INT_MIN, "add INT_MAX+1 reports, wraps to INT_MIN");

of = ckd_add(&r, INT_MAX, 0);
CHECK(of == false && r == INT_MAX, "add INT_MAX+0 fits, the identity row");

of = ckd_sub(&r, INT_MIN, 1);
CHECK(of == true && r == INT_MAX, "sub INT_MIN-1 reports, wraps to INT_MAX");

unsigned u;
of = ckd_sub(&u, 0u, 1u);
CHECK(of == true && u == UINT_MAX, "unsigned wrap counts: 0u-1u to UINT_MAX");

of = ckd_mul(&r, 65536, 65536);
CHECK(of == true && r == 0, "mul 65536*65536 reports, result is 0");
```

listing 35: five truth table rows, every wrapped value still stored (source: c-os-cloud/samples/src/Ch06/ckdint.c, lines 22-39)

Walk the rows. `INT_MAX + 1` reports true and stores `INT_MIN`. `INT_MIN - 1` reports true and stores `INT_MAX`. `0u - 1u` into an unsigned result reports true, because unsigned wraparound is real overflow for these macros, and stores `UINT_MAX`. `65536 * 65536` reports true and stores exactly 0. The identity row, `INT_MAX + 0`, reports false: not every operation overflows, and the table has a negative control.

```
// the result type decides what counts as overflow, not the operands:
// two ints that would overflow int are fine when they land in long long
long long wide;
of = ckd_add(&wide, INT_MAX, INT_MAX);
CHECK(of == false && wide == 4294967294LL, "int operands into long long fit");

// the same operands through two destinations: only the uint16_t row trips
uint16_t r16;
int ri;
of = ckd_add(&r16, (uint16_t)40000, (uint16_t)40000);
bool of_i = ckd_add(&ri, (uint16_t)40000, (uint16_t)40000);
CHECK(of == true && r16 == 14464 && of_i == false && ri == 80000,
      "40000+40000: overflow into uint16_t, clean into int");
```

listing 36: the result type decides what counts as overflow, not the operands (source: c-os-cloud/samples/src/Ch06/ckdint.c, lines 41-53)

The result type `R` is the yardstick. Two `int` operands that would overflow `int` are clean when they land in a `long long`. The same `40000 + 40000` trips over a `uint16_t` result, storing 14464, and lands as 80000 in an `int`. One macro, two destinations, two verdicts: the check is made in the width of the destination, never in the width the operands happened to carry through integer promotion. That is the trap the macros close, a hand-rolled guard written against the operand types silently checks the wrong arithmetic.

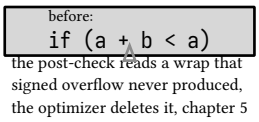
```c
// a negative mathematical result cannot land in an unsigned result type
unsigned long long ull;
of = ckd_mul(&ull, -3, 5);
CHECK(of == true && ull == ULLONG_MAX - 14,
      "-3*5 into unsigned long long wraps and still reports");

// the return value is a real _Bool, not an int in disguise
CHECK(sizeof(of) == 1, "ckd_* return _Bool, one byte");

// one row on runtime values: argc plus INT_MAX - argc is exactly INT_MAX
// for every argc the gate can pass, and the values cannot fold at compile
// time, which is what keeps the checked add alive in the emitted ir
int ra;
of = ckd_add(&ra, argc, INT_MAX - argc);
CHECK(of == false && ra == INT_MAX,
      "runtime pair summing to exactly INT_MAX fits");
(void)argv;
```

listing 37: signed math into an unsigned destination, the `_Bool` return, one row alive at -O2 (source: c-os-cloud/samples/src/Ch06/ckdint.c, lines 55-71)

`-3 * 5` is −15, which cannot exist in an `unsigned long long`, so the macro reports true and stores the wrapped 18446744073709551601. The return value is a real `_Bool`, one byte. The last row runs on `argc` and `INT_MAX - argc`, a pair that sums to exactly `INT_MAX` for every `argc` the gate can pass: deterministic in behavior, opaque to the compiler. That opacity is deliberate, it keeps the checked add alive at `-O2`, where the emitted ir carries `llvm.sadd.with.overflow.i32`, the intrinsic that becomes one overflow-flagged add and a conditional branch on x86-64. The gate pins that substring at `-O2` for this file.



expression	reports	stores
ckd_add(INT_MAX, 1)	true	INT_MIN
ckd_sub(INT_MIN, 1)	true	INT_MAX
ckd_sub(0u, 1u) to unsigned	true	UINT_MAX
ckd_mul(65536, 65536)	true	0
ckd_add(INT_MAX, INT_MAX) to long long	false	4294967294

after: three macros, mixed operand types, the destination width is the contract

figure 21: the hand-rolled post-check against the ckd truth table

NOTE: what the macros refuse

The C23 rule: operands and result must be integer types other than plain `char`, `bool`, a bit-precise type, or an enumeration. Probed on clang 23.1.1: `ckd_add` on `_BitInt(40)` operands fails to compile with “operand argument to checked integer operation must be an integer type other than plain ‘char’, ‘bool’, bit-precise, or an enumeration”. The header’s own comment also advises against a short result type, and no diagnostic follows that advice: the `uint16_t` row above compiled silently.

6.2. sized integers and timespec_get

Exact-width integers are a compile-time contract, so the sample asserts them at compile time, five `static_assert` lines next to the includes:

```c
// exact-width integers are a compile-time contract, checked where declared
static_assert(sizeof(int8_t) == 1, "int8_t is exactly one byte");
static_assert(sizeof(int16_t) == 2, "int16_t is exactly two bytes");
static_assert(sizeof(int32_t) == 4, "int32_t is exactly four bytes");
static_assert(sizeof(int64_t) == 8, "int64_t is exactly eight bytes");
static_assert(sizeof(intmax_t) == sizeof(int64_t), "intmax_t is 64 bit here");
```

listing 38: intN_t widths and intmax_t, asserted where they are declared (source: c-os-cloud/samples/src/Ch06/time.c, lines 19-24)

The clock side of the library reports sizes and behavior. `time_t` is 64 bit on this target. `struct timespec` is 8 byte `tv_sec` plus 4 byte `tv_nsec` padded to 16, because `long` is 4 bytes on windows even on a 64 bit target. `timespec_get` returns the value of its base argument on success and zero on failure, the ucrt supports exactly one base, `TIME_UTC`, defined as seconds and nanoseconds since midnight january 1 1970 utc. The only timing claim a deterministic test can make is the weak one, and the sample makes exactly that one: two reads never go backwards.

```c
int main(void) {
  CHECK(sizeof(time_t) == 8, "time_t is 64 bit on this target");
  CHECK(sizeof(struct timespec) == 16,
        "timespec is 8 byte tv_sec, 4 byte tv_nsec, padded to 16");

  struct timespec t1, t2;
  CHECK(timespec_get(&t1, TIME_UTC) ≠ 0, "timespec_get TIME_UTC succeeds");
  CHECK(timespec_get(&t2, TIME_UTC) ≠ 0, "a second call succeeds too");
  CHECK(t2.tv_sec > t1.tv_sec ||
            (t2.tv_sec == t1.tv_sec && t2.tv_nsec ≥ t1.tv_nsec),
        "two reads never go backwards");
  CHECK(t1.tv_nsec ≥ 0 && t1.tv_nsec ≤ 999999999, "tv_nsec stays in range");
```

listing 39: sizes, two timespec_get calls, the never-backwards invariant, tv_nsec range (source: c-os-cloud/samples/src/Ch06/time.c, lines 26-37)

A fixed `time_t` through the utc calendar is fully deterministic, because utc has no timezone and no locale. 1700000000 is november 14 2023 at 22:13:20, a tuesday, day 318 of the year: eight fields checked against the printed calendar, with months 0 based and the year counted from 1900, the two conventions that bite everyone once. And because `time_t` is a plain second count, adding 86400 is day arithmetic, the sample lands on the 15th.

```c
  // a fixed time_t through the utc calendar: deterministic because utc has
  // no timezone and no locale, 1700000000 is 2023-11-14 22:13:20 utc
  time_t epoch = 1700000000;
  struct tm tmv = {0};
  CHECK(gmtime_s(&tmv, &epoch) == 0, "gmtime_s converts the fixed epoch");
  CHECK(tmv.tm_year == 123 && tmv.tm_mon == 10 && tmv.tm_mday == 14,
        "1700000000 is november 14, 2023");
  CHECK(tmv.tm_hour == 22 && tmv.tm_min == 13 && tmv.tm_sec == 20,
        "at 22:13:20 utc");
  CHECK(tmv.tm_wday == 2 && tmv.tm_yday == 317, "a tuesday, day 318 of 2023");

  // time_t is a plain second count, so adding a day is arithmetic
  time_t next = epoch + 86400;
  struct tm tmn = {0};
  CHECK(gmtime_s(&tmn, &next) == 0 && tmn.tm_mday == 15,
        "epoch + 86400 lands on the 15th");
```

listing 40: a fixed epoch through gmtime_s, and one day of arithmetic on it (source: c-os-cloud/samples/src/Ch06/time.c, lines 39-54)

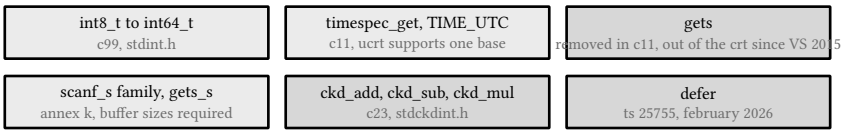
```c
  // intptr_t exists to round-trip a pointer through an integer
  void *p = &checks;
  CHECK((void *)(intptr_t)p == p, "intptr_t round trips a pointer");
```

listing 41: intptr_t exists to round-trip a pointer (source: c-os-cloud/samples/src/Ch06/time.c, lines 56-58)

PITFALL: gmtime does not survive the gate

The ucrt marks `gmtime` with a deprecation annotation pointing at `gmtime_s`, and the gate's `-Werror` turns that annotation into a compile error, probed on this box. The sample uses `gmtime_s`, and there is a trap inside the fix: the microsoft signature is `errno_t gmtime_s(struct tm *, const time_t *)` while Annex K spells it `struct tm *gmtime_s(const time_t *, struct tm *)`, parameters reversed. Microsoft documents `_CRT_USE_CONFORMING_ANNEX_K_TIME` as the switch to the conforming variant.

The library also grows by removal. `gets` read a line with no bound and no way to be made safe, C11 removed it from the standard, and microsoft removed it from the crt beginning with visual studio 2015, the conformance table credits VS 2019 16.8 for the C11 removal. `fgets` is the portable replacement. The ucrt carries the annex K spellings alongside, the `scanf_s` family takes a buffer size argument for every `c` and `s` conversion, and `qsort_s` below takes a context pointer.



the c23 library is layered history: three standards and one technical specification, all live in one ucrt link

figure 22: library eras, what arrived when, and what was removed

6.3. qsort, bsearch, and the comparator contract

`qsort` sorts an array it cannot see: it knows a base, a count, an element width, and nothing else. Everything it learns about the elements comes through one function pointer, called with two `const void *` arguments, expected to return negative, zero, or positive. The sample's comparators use the subtraction-free idiom, `(a > b) - (a < b)`, which cannot overflow the way `a - b` can:

```c
// total order on key alone: zero on equal keys, both signs otherwise
static int by_key(const void *pa, const void *pb) {
  const item *a = pa, *b = pb;
  return (a->key > b->key) - (a->key < b->key);
}

// the qsort_s spelling: a context pointer travels with every comparison,
// so the comparator needs no global state to count or to sort by
static int by_key_counting(void *ctx, const void *pa, const void *pb) {
  long *calls = ctx;
  (*calls)++;
  const item *a = pa, *b = pb;
  return (a->key > b->key) - (a->key < b->key);
}
```

listing 42: two comparators: plain, and the qsort_s spelling with a context pointer (source: c-os-cloud/samples/src/Ch06/qsort.c, lines 25-38)

The contract on that function is the part every wrong qsort program gets wrong. It must be irreflexive, `cmp(x, x) == 0`, and antisymmetric, the signs of `cmp(a, b)` and `cmp(b, a)` must oppose. The sample checks both properties

over every ordered pair of the real array before it sorts anything, because a comparator that violates them makes `qsort` behavior undefined, not merely wrong:

```
  // the comparator contract, checked before any sorting relies on it:
  // irreflexive, and antisymmetric on every ordered pair
  bool consistent = true;
  for (size_t i = 0; i < n; i++) {
    for (size_t j = 0; j < n; j++) {
      int ij = by_key(&src[i], &src[j]);
      int ji = by_key(&src[j], &src[i]);
      if ((ij > 0) != (ji < 0) || (ij < 0) != (ji > 0))
        consistent = false;
      if (i == j && ij != 0)
        consistent = false;
    }
  }
  CHECK(consistent, "comparator irreflexive and antisymmetric");
```

listing 43: irreflexivity and antisymmetry, checked on every ordered pair (source: c-os-cloud/samples/src/Ch06/qsort.c, lines 44-57)

```
  item sorted[sizeof src / sizeof src[0]];
  memcpy(sorted, src, sizeof src);
  qsort(sorted, n, sizeof sorted[0], by_key);
  int keys_expected[] = {3, 7, 19, 42, 90};
  bool in_order = true;
  for (size_t i = 0; i < n; i++)
    if (sorted[i].key != keys_expected[i])
      in_order = false;
  CHECK(in_order, "qsort orders the shuffled keys 3 7 19 42 90");

  // bsearch needs the array already sorted by the same comparator
  item needle = {19, ""};
  item *hit = bsearch(&needle, sorted, n, sizeof sorted[0], by_key);
  CHECK(hit != NULL && strcmp(hit->name, "c") == 0,
        "bsearch finds key 19, carries the name");

  item missing = {55, ""};
  CHECK(bsearch(&missing, sorted, n, sizeof sorted[0], by_key) == NULL,
        "bsearch on an absent key returns NULL");
```

listing 44: qsort orders the shuffled keys, bsearch finds 19 and reports 55 absent (source: c-os-cloud/samples/src/Ch06/qsort.c, lines 59-77)

`bsearch` needs the array already sorted by the same comparator, and it returns NULL for an absent key, both pinned. The microsoft docs state the stability caveat in one sentence: if compare indicates two elements are the same, their order in the resulting sorted array is unspecified. So the search array carries unique keys, and where the sample does sort duplicate keys it checks only the guaranteed property, non-decreasing order with both duplicates present, never their relative order.

```c
// duplicates: the guarantee is non-decreasing keys, nothing about their
// relative order, so the check asserts exactly the guarantee
item dup[] = {{7, "d"}, {42, "b"}, {7, "e"}, {19, "c"}};
size_t dn = sizeof dup / sizeof dup[0];
long calls = 0;
qsort_s(dup, dn, sizeof dup[0], by_key_counting, &calls);
bool nondecreasing = true;
bool names_preserved = false;
for (size_t i = 1; i < dn; i++)
  if (by_key(&dup[i - 1], &dup[i]) > 0)
    nondecreasing = false;
names_preserved =
    dup[0].key == 7 && dup[1].key == 7 &&
    ((strcmp(dup[0].name, "d") == 0 && strcmp(dup[1].name, "e") == 0) ||
     (strcmp(dup[0].name, "e") == 0 && strcmp(dup[1].name, "d") == 0));
CHECK(nondecreasing && names_preserved,
      "qsort_s: keys non-decreasing, both 7s kept");
CHECK(calls ≥ (long)(dn - 1), "every comparison went through the context");

printf("ucrt qsort_s made %ld comparisons for %zu elements\n", calls, dn);
```

listing 45: qsort_s over duplicate keys, every comparison through the context (source: c-os-cloud/samples/src/Ch06/qsort.c, lines 79-98)

`qsort_s` is the annex K spelling present in the ucrt: same behavior, one added `context` pointer that travels into every comparator call. The docs' stated purpose is reentrancy, the context replaces the static variable a comparator would otherwise need, which is what makes a sort usable from two threads at once. Here it carries a counter, and the sample asserts the invariant any comparison sort must obey, at least `n - 1` calls, then prints the observed 10 comparisons for 4 elements, a number owned by this ucrt build, not by the standard.

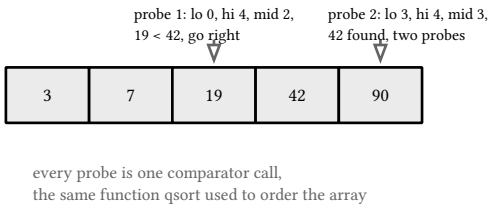


figure 23: bsearch probing the sorted array for key 42

6.4. defer and stddefer

`defer` is not in C23. It arrives from technical specification 25755, and clang 23.1.1 ships it as `stddefer.h`, a 19 line header that aliases `defer` to the keyword `_Defer` when `__STDC_DEFER_TS25755__` is defined, which happens only under the driver flag `-fdefer-ts`. The gate's tool flags carry it, and the sample's first check pins the header's version macro, 202602L. The semantics fit one sentence: a deferred statement runs when its scope exits, last registered first out.

```c
// the recorder: every deferred statement appends one tag, the final order
// is the unwind order, which is the whole claim this sample pins
static char marks[16];
static size_t nmarks;

static void mark(char tag) {
  marks[nmarks++] = tag;
  marks[nmarks] = 0; // keep the buffer a string, so strcmp sees the truth
}
```

listing 46: the recorder, every deferred statement appends one tag (source: c-os-cloud/samples/src/Ch06/defer.c, lines 21-29)

```c
static void scoped(int early) {
  defer mark('a'); // function scope: registered first, runs last
  {
    defer mark('b'); // block scope: runs when the block closes
    if (early) {
      mark('r'); // the early return still unwinds b, then a
      return;
    }
  } // on the normal path b fires right here, before d is even registered
  defer mark('d');
  mark('x');
  // return unwinds d, then a
}
```

listing 47: nested scopes, one early return, registration order against unwind order (source: c-os-cloud/samples/src/Ch06/defer.c, lines 36-48)

The recorded orders are the whole lesson. On the early path the marks read `r`, `b`, `a`: the body marks `r`, the return unwinds the block's `b`, then the function's `a`. On the normal path they read `b`, `x`, `d`, `a`, and the surprise is in the middle: `b` fires when its block closes, before `d` is even registered, a defer lives and dies with its scope, not with its function. In a loop each iteration is a scope, so each defer fires at its own iteration's end, giving `AaBbCc`, the immediate uppercase mark followed by that iteration's deferred lowercase one.

```c
static void loop_scoped(int iters) {
  for (int i = 0; i < iters; i++) {
    defer mark((char)('a' + i)); // one defer per iteration,
    mark((char)('A' + i));       // each fires at that iteration's end
  }
}
```

listing 48: one defer per iteration, each fires at that iteration's end (source: c-os-cloud/samples/src/Ch06/defer.c, lines 50-55)

```
int main(void) {
  CHECK(__STDC_VERSION_STDDEFER_H__ == 202602L,
        "stddefer.h reports TS 25755, february 2026");

  reset();
  scoped(1);
  CHECK(strcmp(marks, "rba") == 0,
        "early return: r, then b, then a, registered order reversed");

  reset();
  scoped(0);
  CHECK(strcmp(marks, "bxda") == 0,
        "normal path: b at block exit, x, then d and a at return");

  reset();
  loop_scoped(3);
  CHECK(strcmp(marks, "AaBbCc") == 0,
        "loop: each defer fires at its own iteration's end");
```

listing 49: the pinned unwind orders: rba early, bxda normal, AaBbCc in the loop (source: c-os-cloud/samples/src/Ch06/defer.c, lines 57-74)

```
  reset();
  {
    defer {
      mark('q');
      mark('p');
    }
    mark('s');
  }
  CHECK(strcmp(marks, "sqp") == 0,
        "a compound statement body defers both statements at once");
```

listing 50: the statement after defer can be a compound statement (source: c-os-cloud/samples/src/Ch06/defer.c, lines 76-85)

VERIFY: the zero-cost claim, checked in the emitted code

At `-O2` the cleanup is not calls at all. The whole unwind sequence of the early path compiles to one instruction, `movl $0x616272`, writing the four bytes of "rba" and its terminator to the marks array, and the ir shows it as a single `store <4 x i8> <i8 114, i8 98, i8 97, i8 0>`. A grep over the emitted module finds zero `personality`, `landingpad`, or `invoke` lines: there is no unwinding machinery, no tables, no runtime, the deferred code is ordinary straight-line code the optimizer inlines like any other. The gate pins `@marks`, `getelementptr inbounds`, and the `movl` for this file.

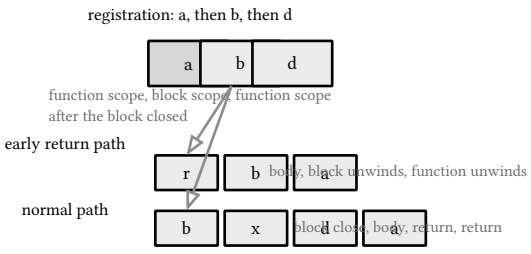


figure 24: registration order against unwind order, both paths through scoped

sources: clang.llvm.org C support page (N2683 towards integer safety, status clang 18), learn.microsoft.com ucrt pages for timespec_get, gmtime_s, qsort, qsort_s, the scanf_s family, gets, and the visual studio C standard library conformance table, all accessed 2026-09-12; the stdckdint.h and stddefer.h text read from the clang 23.1.1 resource

directory on this machine the same day. Sample behavior verified by `make verify-c`, 33 checks in chapter 6 of the samples suite.

7. llvm: the architecture of the toolchain

clang is one command that is not one program. Behind the single invocation the gate uses stands an architecture: a driver that plans the work, a front end that turns source bytes into an abstract syntax tree and the tree into llvm ir, a middle end that rewrites the ir, a back end that lowers the ir to x86-64 machine code, and a linker that freezes the result into a pe file. The llvm release index announced 23.1.1 on 2026-09-08, and the binary on this machine identifies itself down to the revision: `clang --version` prints the commit hash `6dfe1677ab8dffbc6ec13d53a1e0215d75147689`, the same commit the github release tag `llvmorg-23.1.1` points at. This chapter walks the pipeline once, quotes the driver's own transcript of what it runs, watches the pass manager report its decisions, and maps every tool in the windows distribution onto the stage it serves.

7.1. front, middle, and back end

LLVM's design joins three ends with one representation. The front end, `cc1` inside the clang binary, owns everything the language standard describes: it runs the translation phases of chapter 2, builds the tree, performs semantic analysis, and lowers the tree to ir. The middle end runs passes over that ir, transformations that know nothing about c and everything about the ir's own semantics. The back end lowers the ir through a selection dag and machine ir to real instructions and emits the object file. Only ir passes between the ends, which is the design's whole point: every language front end that targets llvm feeds every hardware back end through the same instructions.

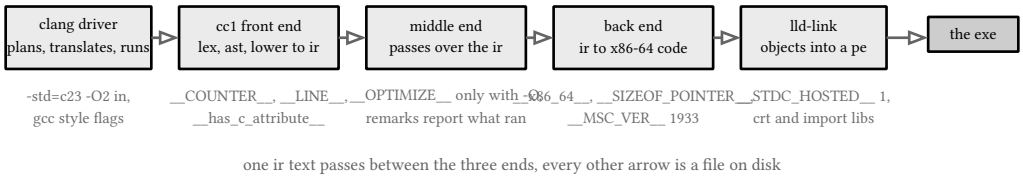


figure 25: the pipeline the driver runs, one box per stage, and the fingerprints each stage leaves

Every stage leaves fingerprints that survive into the running program, and `stages.c` checks them one per stage. The preprocessor's expansion state is the first: `__COUNTER__` advances once per expansion, and `__LINE__` expands to the line its own token sits on:

```
// phase 4 expansion state: __COUNTER__ advances once per expansion
enum { counter_first = __COUNTER__, counter_second = __COUNTER__ };

// phase 3 source locations: __LINE__ expands where its token sits
static int line_probe(void) { return __LINE__; }
```

listing 51: expansion state and source location, the preprocessor's fingerprints (source: c-os-cloud/samples/src/Ch07/stages.c, lines 22-26)

Both `__COUNTER__` uses are enum initializers, so the expansion happens in phase 4 and the constant folding in the front end, and the two enumerators must read 0 then 1. `line_probe` returns `__LINE__`, and the check pins the literal 26: the reader counts the lines of the file and watches the location the lexer recorded arrive in the running program unchanged.

```
// driver mode: __OPTIMIZE__ exists only under an explicit -O flag, and
// the gate compiles this file with none, so the default optimization
// level is -O0
#ifdef __OPTIMIZE__
static const int optimize_defined = 1;
#else
static const int optimize_defined = 0;
#endif

// backend target: the driver spells its macros in msvc style (_M_X64,
// _MSC_VER) alongside the clang family macros, with no __GNUC__
#if defined(__x86_64__) && defined(_M_X64) && defined(__llvm__) &&             \
    defined(__clang__) && !defined(__GNUC__)
static const int msvc_style_target = 1;
#else
static const int msvc_style_target = 0;
#endif
```

listing 52: driver mode and backend target, compiled in as constants (source: c-os-cloud/samples/src/Ch07/stages.c, lines 28-44)

`__OPTIMIZE__` exists only when the driver passes an explicit `-O` flag. The gate compiles this file with none, so the check reading 0 pins the driver default, `-O0`, no optimization pipeline at all. The target block records what the back end was told to build: `__x86_64__` and `_M_X64` together, the msvc-style spellings riding alongside the clang family macros, `_MSC_VER` expanding to 1933, and no `__GNUC__` anywhere, this driver speaks msvc compatibility, not gnu dialect.

```
int main(void) {
  CHECK(counter_first == 0, "counter starts at zero");
  CHECK(counter_second == 1, "counter advances one per expansion");
  CHECK(line_probe() == 26, "line macro reports its own source line");
  CHECK(strcmp(__FILE_NAME__, "stages.c") == 0,
        "file macro keeps the driver input spelling");
  CHECK(__clang_major__ == 23, "front end is clang 23");
  CHECK(__clang_minor__ == 1, "point release 1");
  CHECK(__STDC_HOSTED__ == 1, "hosted: main linked against a runtime");
  CHECK(__SIZEOF_POINTER__ == 8, "backend data layout: 8 byte pointers");
  CHECK(msvc_style_target == 1, "target macros: x86_64 and msvc style");
  CHECK(optimize_defined == 0, "no -O flag given: __OPTIMIZE__ undefined");
  return 0;
}
```

listing 53: ten checks, one fingerprint per stage from front end to link (source: c-os-cloud/samples/src/Ch07/stages.c, lines 46-59)

`__clang_major__` and `__clang_minor__` are the front end's identity, `__SIZEOF_POINTER__ == 8` is the data layout the back end compiled against, `__FILE_NAME__` keeps the driver's input spelling, and `__STDC_HOSTED__ == 1` is the link step's signature: main exists and a runtime sits under it.

The front end's semantic phase is where c23 attributes are checked, and `attributes.c` holds them where their consequences are observable. The first pair is about control flow:

```c
// never taken on any path the checks exercise; the attribute is what
// lets the compiler treat calls to it as path ends
[[noreturn]] static void bail(const char *why) {
  fprintf(stderr, "%s\n", why);
  exit(70);
}

// the body ends in a noreturn call with no return after it: without
// [[noreturn]] on bail, -Wreturn-type under the gate's -Werror would
// refuse this file
static int with_default(int x) {
  if (x > 0)
    return 1;
  bail("no default");
}
```

listing 54: bail never returns, so the body that ends in it needs no return (source: c-os-cloud/samples/src/Ch07/attributes.c, lines 24-38)

`with_default` ends in a call with no return after it. Because the front end knows `bail` never returns, the path where control falls off the end does not exist as far as its flow analysis is concerned. That is a falsifiable claim about the build, and the probe falsifies the opposite: delete the one word `[[noreturn]]` and the gate refuses the file with `non-void function does not return a value in all control paths [-Werror,-Wreturn-type]`. The same falsifiability runs through the next listing. Discard the `checksum` result at any call site and the gate refuses the file with `ignoring return value of function declared with 'nodiscard' attribute [-Werror,-Wunused-result]`. Strip the `[[maybe_unused]]` from `tag` and it refuses with `unused parameter 'tag' [-Werror,-Wunused-parameter]`, strip it from `build_tag` and it refuses with `unused variable 'build_tag' [-Werror,-Wunused-variable]`. All four diagnostics probed on this box, 2026-09-12:

```c
// discarding this return value is a warning, and the gate runs -Werror:
// the attribute is enforced at every call site this file carries
[[nodiscard]] static int checksum(int a, int b) { return (a * 31) ^ b; }

// tag is deliberately never read: without the attribute -Wunused-parameter
// inside the gate's -Wextra would refuse this file
static int measure([[maybe_unused]] const char *tag, int v) { return v * 2; }

// declared and left unread on purpose, same silence, same reason
[[maybe_unused]] static const char *build_tag = "attributes.c";
```

listing 55: nodiscard enforced at call sites, `maybe_unused` silencing deliberate waste (source: c-os-cloud/samples/src/Ch07/attributes.c, lines 40-49)

```
// case 1 drops into case 2 on purpose: the attribute documents it, and
// under -Wimplicit-fallthrough the unannotated spelling is an error
static int classify(int n) {
  int acc = 0;
  switch (n) {
  case 0:
    acc += 1;
    break;
  case 1:
    acc += 10;
    [[fallthrough]];
  case 2:
    acc += 100;
    break;
  default:
    acc = -1;
    break;
  }
  return acc;
}
```

listing 56: the annotated drop into case 2, its arithmetic checkable (source: c-os-cloud/samples/src/Ch07/attributes.c, lines 51-70)

The `[[fallthrough]]` case has a sharper story than the others. Under the gate's `-Wextra` the unannotated drop compiles clean, probed: clang runs that warning at its least strict level there. Turn the level up with `-Wimplicit-fallthrough` and the same file dies with `unannotated fall-through between switch labels`, and clang's own fix-it for it reads `insert '[[fallthrough]];' to silence this warning`. The attribute is the standard's spelling of the apology the fix-it asks for.

```
int main(void) {
  CHECK(__has_c_attribute(noreturn) == 202202L, "noreturn reports 202202L");
  CHECK(__has_c_attribute(maybe_unused) == 202106L,
        "maybe_unused reports 202106L");
  CHECK(__has_c_attribute(nodiscard) == 202003L, "nodiscard reports 202003L");
  CHECK(__has_c_attribute(fallthrough) == 201910L,
        "fallthrough reports 201910L");
  CHECK(__has_c_attribute(deprecated) == 201904L, "deprecated reports 201904L");
  CHECK(with_default(5) == 1, "the return path is reachable");
  CHECK(checksum(7, 3) == ((7 * 31) ^ 3), "nodiscard result used and checked");
  CHECK(measure("stage", 21) == 42, "maybe_unused parameter ignored");
  CHECK(classify(0) == 1, "case 0 breaks out");
  CHECK(classify(1) == 110, "case 1 falls through into case 2");
  CHECK(classify(9) == -1, "default path taken");
  return 0;
}
```

listing 57: the values this front end reports, and the behaviors the checks pin (source: c-os-cloud/samples/src/Ch07/attributes.c, lines 72-87)

NOTE: the reported attribute values are not the ones the normative text names

Every attribute sub-section of n3220 6.7.13 carries the same sentence, here the nodiscard one: "The `__has_c_attribute` conditional inclusion expression (6.10.2) shall return the value 202311L when given nodiscard as the pp-tokens operand if the implementation supports the attribute." The fifth edition harmonized all seven attributes onto 202311L. Clang 23.1.1 reports the historical spellings the informative annex M table records instead, 201904L through 202202L, one per attribute. The checks pin what this front end actually answers, and the divergence is the lesson: feature tests should compare against the values the compiler in

> front of you reports, and code that wants to ask “is this attribute supported” should test nonzero, not equality with 202311L. The annex also lists `reproducible` and `unsequenced` at 202207L, and `__has_c_attribute(unsequenced)` answers 0 on this compiler: unsupported.

7.2. the clang driver stages, -###

The driver is a program with its own architecture, documented on the clang.llvm.org DriverInternals page as five conceptual stages: parse, pipeline, bind, translate, execute. Parse turns the command line into arg objects in which joined and split spellings, `-Ifoo` and `-I foo`, map to the same option. Pipeline builds a tree of actions over the phases, the “well known compilation steps, such as preprocess, compile, assemble, link”. Bind asks the toolchain which tool performs each action. Translate turns gcc-style flags into the flags each chosen tool actually speaks. Execute runs the result. The command line reference pins the phase-stopping actions the same page names: `-E` “only run the preprocessor”, `-S` “only run preprocess and compilation steps”, `-c` “only run preprocess, compile, and assemble steps”.

`-###` prints the translated commands without running them, the driver’s own account of what it would do. Probed 2026-09-12 on this box, `clang -### -std=c23 -O2 -c stages.c` answers with one line of mode, a marker, then the cc1 invocation. The marker says the front end runs in process:

```
 (in-process)
```

```
 "clang.exe" "-cc1" "-triple" "x86_64-pc-windows-msvc19.33.0" "-O2" "-emit-obj" "-disable-free" "-clear-ast-before-backend" "-discard-value-names" "-main-file-name" "stages.c"
```

The triple grows a version suffix, 19.33.0, the msvc release the front end promises to be compatible with. `-O2` rides through untouched, flag translation does not change what it means, only where it lands. `-clear-ast-before-backend` frees the tree before codegen starts, and `-discard-value-names` strips local names, which is why optimized ir speaks in `%0` and `%1`. The middle of the line carries the target’s codegen defaults:

```
 "-mframe-pointer=none" "-relaxed-aliasing" "-fmath-errno" "-ffp-contract=on"
```

Frame pointers are off by default on this target. `-relaxed-aliasing` turns tbaa off to match msvc semantics, the fact chapter 5 chased from the ir side and recorded as cos-004: no `!tbaa` on any load, and no flag brings it back. Then the search paths, and here the transcript explains a mystery chapter 1 left as a probed note:

```
 "-internal-isystem" "C:/Program Files/Microsoft Visual Studio 10.0/VC/include" "-internal-isystem" "C:/Program Files/Microsoft Visual Studio 9.0/VC/include"
```

The line continues down through Visual Studio 9.0′s PlatformSDK and both of Visual Studio 8′s directories. The driver’s msvc autodetection, running on a machine with Visual Studio 2022 and msvc 14.44 installed, injects include paths for Visual Studio 8, 9, and 10 and nothing else. That is the whole reason the gate passes explicit `-isystem` and `-L` paths resolved by scanning the installed toolsets. The tail of the line is the compatibility contract:

```
 "-std=c23" "-ferror-limit" "19" "-fno-use-cxa-atexit" "-fms-extensions" "-fms-compatibility" "-fwrapv" "-fms-compatibility-version=19.33"
```

`-fwrapv` is the other chapter 5 fact: signed wrap defined by toolchain contract, no `nsw` assumptions on arithmetic, visible here as a flag nobody typed. And because `-O2` was asked for, the same line ends with the middle end being wired in:

```
 "-vectorize-loops" "-vectorize-slp"
```

The link stage appears when the output is an exe instead of an object. The same probe with the gate’s own flags appends a second command to the transcript, and its shape is the chapter 1 link model made visible:

```
 "lld-link" "-out:stages.exe" "-defaultlib:libcmt" "-defaultlib:oldnames"
```

```
 "kernel32.lib" "synchronization.lib" "...Temp\stages-4f1a9c.o"
```

`-fuse-ld=lld` plus the msvc target bound the link action to lld-link, the static crt `libcmt` is the default library pair, the gate's `-lkernel32 -lsynchronization` translated into `.lib` names, and the object the front end produced sits in a temp file the driver itself named with an embedded hash. Run the same `-###` without the gate's explicit paths and the link line begins with the placeholder libpaths `"lib\amd64"` and `"atlmfc\lib\amd64"`: the sdk scan found the windows kits, the msvc scan found nothing, and the dead relative paths are what the failure looks like from inside the transcript.

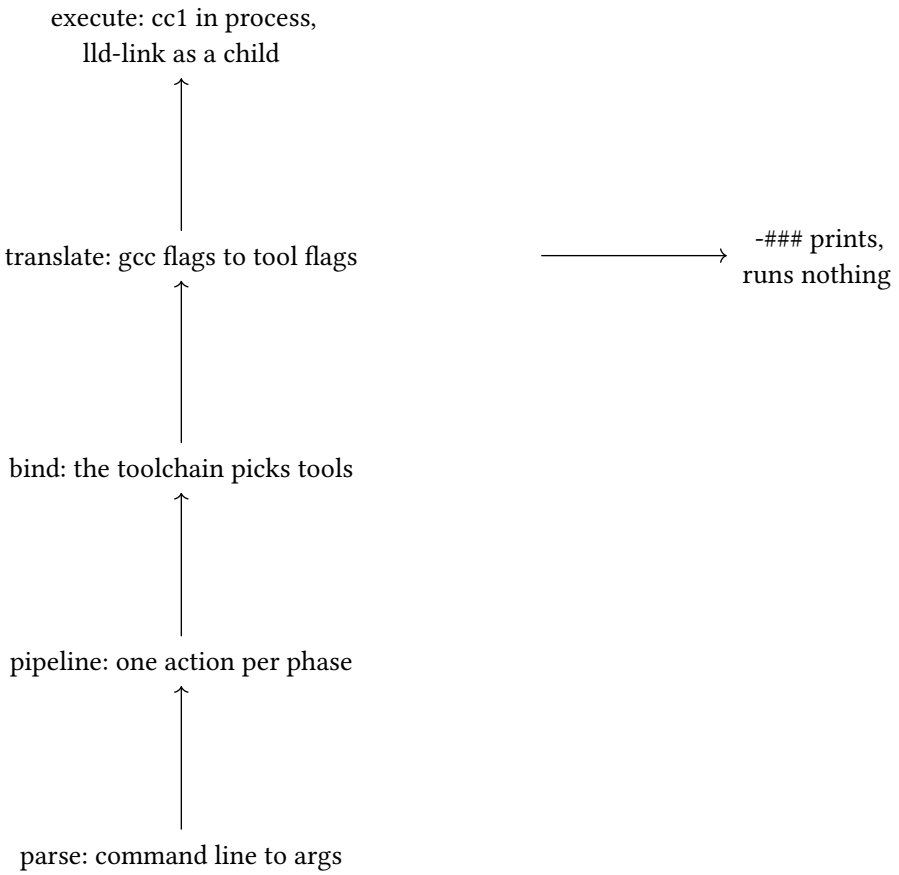


figure 26: the driver's five conceptual stages, and where -### stops the story

7.3. passes and -Rpass remarks

The middle end is a sequence of passes over the ir, and the `-O` flag selects the sequence. At `-O0` almost nothing runs, which is what `__OPTIMIZE__` staying undefined recorded. At `-O2` the transcript showed the driver wiring `-vectorize-loops` and `-vectorize-slp` into cc1, and the inliner runs alongside them. In this build the passes have no standalone runner on windows, there is no `opt` binary in the distribution, so the pipeline always executes inside cc1 under whatever `-O` the driver translated.

The pass manager can narrate itself. The users manual's optimization report options state the deal: "optimization reports trace, at a high-level, all the major decisions made by compiler transformations", in three families, `-Rpass` "when the pass makes a transformation", `-Rpass-missed` "when the pass fails to make a transformation", and `-Rpass-analysis` "when the pass determines whether or not to make a transformation". Each takes a regular expression naming the pass, and `-Rpass=.*` asks every pass. Probed at `-O2` on a two function pair, 2026-09-12:

```
remark: 'twice' inlined into 'scale' with (cost=-15030, threshold=337) at callsite scale:0:34; [-Rpass=inline]
remark: 'scale' inlined into 'reveal' with (cost=-15030, threshold=337) at callsite reveal:0:28; [-Rpass=inline]
```

The remark names the callee, the caller, the callsite, and the inliner's own accounting: cost against threshold, where a negative cost means the decision was never in doubt. `-Rpass-missed=inline` on the same file prints nothing, nothing was missed. The window opens past the inliner too. On a loop shaped like chapter 5's restrict twin:

```
remark: vectorized loop (vectorization width: 4, interleaved count: 2) [-Rpass=loop-vectorize]
```

Width 4 is the `<4 x i32>` the chapter 5 ir analysis read out of the same pipeline, the remark and the ir text describing one decision from two sides.

The `[[noreturn]]` attribute from the first section is a middle end input as much as a diagnostic fact, and the gate pins it. At `-O0` the front end already emits the call to `bail` followed by `unreachable`, no block after the call exists, and `Ch07/expect-ir.txt` asserts both substrings, `call void @bail` and `unreachable`, at `-O0`. At `-O2` the same file collapses: the only definitions left in the module are `main`, `printf`, and the stdio options helper, every static helper dissolved into its caller. And the roundtrip leg compiles `stages.c` to `.ll` text and back to an exe with identical stdout, the pipeline's own proof that the ir between its ends is a lossless staging point. Chapter 8 reads that text line by line.

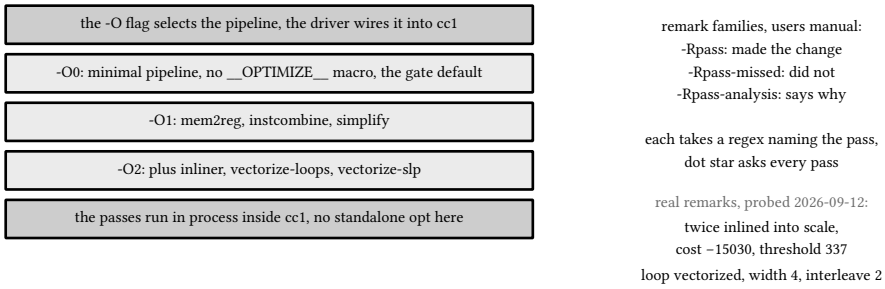


figure 27: the pass pipelines the -O flags select, and the remark families that report them

7.4. the windows tool inventory

Chapter 1 listed which binaries the llvm 23.1.1 windows distribution installs. The architecture view names the stage each one serves. The clang family is the driver. clang-format, clang-tidy, and clang-query are front end tools that never see ir: format works at the token level, tidy and query at the ast, and chapter 10 drives all three. llvm-objdump, llvm-nm, and llvm-ar read and bundle what the back end wrote, disassembly, symbol table, archive, probed on this chapter's own object: `llvm-nm` lists the check-name strings as mangled constants and `llvm-ar` packs the object into an archive. lld-link is the link stage. The absent tools are the standalone middle and back end, and every one of them except the jit has a driver stand-in, probed on this box. `clang -c -emit-llvm x.c -o x.bc` writes bitcode whose first four bytes are 66, 67, 192, 222, the `BC` magic llvm-as would produce. `clang -S -emit-llvm x.bc` turns that bitcode back into text, llvm-dis's job. `clang x.ll -c` compiles ir text to an object, llc's job, and the gate's roundtrip leg is built on exactly that step. `lli` executes ir directly through a jit and has no stand in: the driver only compiles. `llvm-config` reports build configuration and has no stand in either, which is why the gate resolves its own paths by scanning the installed toolsets instead of asking.

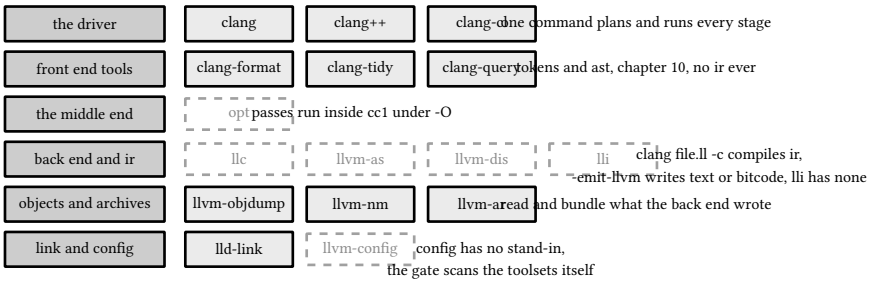


figure 28: every tool in this llvm 23 windows distribution, mapped to the stage it serves

The pattern is the closing fact of the architecture: the windows distribution ships the tools whose job is to face the user, driver, diagnostics, linkers, inspectors, and withholds the ones whose job is to be a stage inside the pipeline. On linux the same stages exist as separate commands because the ecosystem wires them together by hand. Here the driver is the only door, and every chapter of this book that needs a middle end or a back end in isolation opens it with a flag instead.

sources: llvm.org release index (23.1.1 announced 2026-09-08) and the github llvm-project release tag llvmorg-23.1.1 at the same commit this machine's clang reports, both accessed 2026-09-12; clang.llvm.org Driver-

Internals (five driver stages, phases, tool binding), ClangCommandLineReference (phase-stopping action flags), UsersManual (optimization report options), and AttributeReference (deprecated entry), accessed 2026-09-12; open-std.org n3220 6.7.13 attribute semantics and annex M table M.1, accessed 2026-09-12; the `\#\#\#` transcripts, the `-Rpass` remark runs, the ir pins, the negative diagnostic probes, and the tool inventory probed on this machine the same day. Sample behavior verified by `make verify-c`, 21 checks in chapter 7 of the samples suite.

8. reading llvm ir

The ir is the contract between the frontend and the optimizer. Clang writes it, llvm's passes read and rewrite it, and the backend lowers it to machine code. This chapter reads one small function's ir at three optimization levels and takes the whole tour: module header, globals, the define line, the body, the decorations, and the address arithmetic. Because the windows distribution ships no `llvm-as`, `llvm-dis`, or `opt` (the inventory is chapter 7's), every step routes through the driver: `clang -S -emit-llvm` writes the text, and the same clang compiles a `.ll` file back. Every ir line quoted below was emitted by clang 23.1.1 on this machine on 2026-09-12, and every structural claim is pinned in the chapter's `expect-ir.txt`, 16 assertions the gate re-verifies on each run.

8.1. a function through -emit-llvm, the round trip

The command line reference defines the two flags tersely: `-S`, "only run preprocess and compilation steps", and `-emit-llvm`, "use the LLVM representation for assembler and object files". Together they stop the driver after ir generation and write text. The subject under the microscope:

```
struct packet {
  int tag;      // byte offset 0
  int slots[4]; // byte offset 4, four ints
};

// external linkage, so the definition is emitted at every -O level and
// cannot be deleted after inlining; the restrict pointer surfaces as
// noalias in the .ll, the branch as a phi once mem2reg runs
int tally(const struct packet *restrict p, int bonus) {
  int total = p->tag;
  if (bonus > 0) {
    total += bonus;
    for (int i = 0; i < 4; i++)
      total += p->slots[i];
  } else {
    total -= bonus;
  }
  return total;
}
```

listing 58: the dissection subject: a struct, a restrict pointer, a branch, and a four-element loop (source: c-os-cloud/samples/src/Ch08/ir.c, lines 19-37)

`tally` reads `p->tag`, branches on `bonus`, and on the taken arm adds the bonus and all four slots, on the other arm subtracts the bonus. The struct is two fields, one `int` and an array of four. The checks pin both arms and two layout facts:

```
int main(void) {
  struct packet p = {.tag = 100, .slots = {2, 4, 6, 8}};
  CHECK(tally(&p, 3) == 123, "branch taken: tag + bonus + all four slots");
  CHECK(tally(&p, -5) == 105, "else branch: tag minus bonus");
  CHECK(tally(&p, 0) == 100, "zero bonus takes the else path");
  CHECK(sizeof(struct packet) == 20, "object is five ints, 20 bytes");
  CHECK(offsetof(struct packet, slots) == 4, "slots begin at byte offset 4");
  return 0;
}
```

listing 59: both arms asserted, plus the two numbers that reappear in the ir: 20 bytes, offset 4 (source: c-os-cloud/samples/src/Ch08/ir.c, lines 39-47)

The module opens with its identity and target. The `target datalayout` string starts with `e`, little endian, pins `i64:64`, and ends `S128`, a 128-byte stack alignment preference; the `p270:32:32` style clauses give the pointer sizes of x86's non-default address spaces. The `target triple` is `"x86_64-pc-windows-msvc19.33.0"`: that version is clang's default msvc compatibility level, not the installed toolset. Probed the same day: preprocessing an empty file with this triple defines `_MSC_VER` as 1933, while the gate links against the 14.44 toolset chapter 1 resolved. The triple fixes the abi clang targets, not which libraries were found. Below it the struct becomes a named ir type, `%struct.packet = type { i32, [4 x i32] }`, two members, no padding, 20 bytes, exactly the size the fourth check asserted.

The header's `stdio.h` wrappers arrive next as `define linkonce_odr dso_local i32 @printf(...)` plus a `$printf = comdat any` line: every translation unit may carry the definition, and the linker keeps one. Then the initializer of main's local, a private constant: `@__const.main.p = private unnamed_addr constant %struct.packet { i32 100, [4 x i32] [i32 2, i32 4, i32 6, i32 8] }, align 4`. The declaration `struct packet p = {...}` compiles to `call void @llvm.memcpy.p0.p0.i64(ptr align 4 %2, ptr align 4 @__const.main.p, i64 20, i1 false)`: 20 bytes copied from the constant into the alloca. The `sizeof` check is physically present in the ir as the `i64 20`.

The function's first line deserves every word:

```
define dso_local i32 @tally(ptr noalias noundef %0, i32 noundef %1) #0
```

`define` opens a body. `dso_local` promises the symbol resolves inside this module, so no dllimport indirection. The return type is `i32`, the ir spelling of `int`. `noalias` is chapter 4's `restrict` promise translated for the optimizer, and it is already here at -O0. `noundef` sits on every parameter in this module; section 3 reads it. Parameters are unnamed, `%0` and `%1`, and the `#0` references an attribute group printed at the file's bottom: `noinline nounwind optnone uwtable`, where `optnone` is the module's confession that no pass ran. Inside, the body is memory: `%3 = alloca i32, align 4` through `%6`, and the first real instructions are stores of the parameters into their allocas. Locals start at `%3` because unnamed values and blocks share one counter and the implicit entry block consumed `%2`. The branch is `br i1 %11, label %12, label %32`, `%11` being the `icmp sgt i32 %10, 0`.

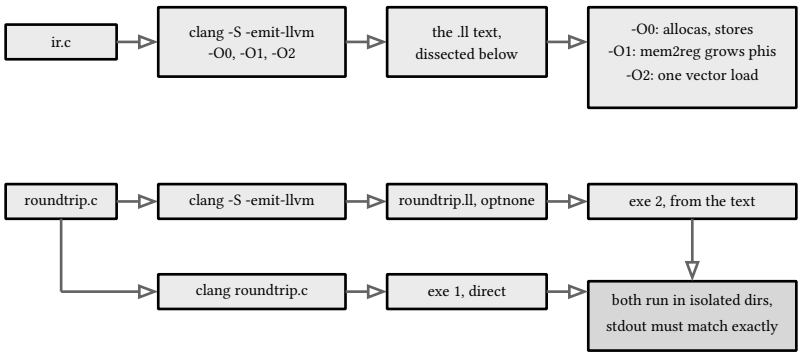


figure 29: one source, two routes through the driver: one stops at the text for reading, one compiles the text back and demands the same output

The second sample rides the gate's roundtrip leg end to end:

```c
// external linkage, so the body is present in the emitted .ll instead of
// folded away; the checksum mixes a loop, a branch, a shift, and a multiply
// so the textual ir carries real instructions across the round trip
unsigned checksum(const char *s, int rotate) {
  unsigned h = 2166136261u;
  for (int i = 0; s[i] != 0; i++) {
    unsigned c = (unsigned char)s[i];
    if (rotate)
      c = (c << 7) | (c >> 25);
    h = (h ^ c) * 16777619u;
  }
  return h;
}
```

listing 60: a checksum with a loop, a branch, a shift, and a multiply: real instructions for the text to carry (source: c-os-cloud/samples/src/Ch08/roundtrip.c, lines 18-30)

```c
int main(void) {
  CHECK(checksum("ir", 0) == checksum("ir", 0), "same bytes, same digest");
  CHECK(checksum("ir", 0) != checksum("ir", 1), "rotation changes the mix");
  CHECK(checksum("", 0) == 2166136261u, "empty input returns the seed");
  CHECK(checksum("llvm", 0) == 286893964u, "fnv1a digest of the byte string");
  CHECK(checksum("llvm", 1) == 4180433557u, "rotated digest of the same bytes");
  return 0;
}
```

listing 61: five checks, two of them digests pinned from observed runs (source: c-os-cloud/samples/src/Ch08/roundtrip.c, lines 32-39)

The leg compiles the source directly to one exe, emits the `.ll` at the driver's default level (no `-O` flag; the emitted text carries `optnone`, the no-pass marker), compiles the `.ll` itself into a second exe, runs both in isolated directories, and fails unless the stdout matches exactly. At the default level `checksum` reads `define dso_local i32 @checksum(ptr noundef %0, i32 noundef %1) #0`, the same parameter decorations as `tally`. The digests are fnv-1a arithmetic modulo 2^32, 286893964 and 4180433557 for the byte string `"llvm"` with and without the rotation, and both exes must print them.

VERIFY: what the round trip proves, and what it does not

The leg proves observable behavior survives a stop at the textual ir: same checks, same digests, same exit code from an exe built out of `.ll` text. It does not prove the pipeline is an identity. The two executables are not compared byte for byte, and nothing claims every optimization is invertible. The comparison is exactly the observable the standard owes the program, the same list chapter 5 quoted from 5.1.2.4, which is the honest thing to assert about a compiler.

8.2. ssa and phi

Every numbered value in this module is defined exactly once. `%3` is one `alloca`, `%11` is one `icmp`, and no line ever reassigns `%11`. That is single static assignment, and the ir holds the shape at every `-O` level for its own registers. What -O0 lacks is ssa for the source variables: `total` and `i` are not registers but four bytes of stack each. Every mutation is a store, `%15 = add i32 %14, %13` followed by `store i32 %15, ptr %5, align 4` on the taken arm, another store inside the loop, and one final `%37 = load i32, ptr %5, align 4` before the `ret`. `tally`'s -O0 body contains no phi at all: the if/else result travels through memory.

At -O1 the allocas are gone and three phis carry the same program:

`%9 = phi i64 [ 0, %5 ], [ %14, %8 ]` is the induction variable: 0 on entry to the loop from block `%5`, the incremented `%14` when control returns along the back edge from `%8` to itself. `%10 = phi i32 [`

`%6, %5 ], [ %13, %8 ]` is the running total over the same two edges. `%19 = phi i32 [ %17, %16 ], [ %13, %8 ]` is the if/else merge: `%17` when the else arm ran, `%13` when the loop exited. The pass that performs this conversion is mem2reg, which reads the -O0 store pattern and manufactures the phis; chapter 9 watches it run. The language reference gives phi its shape: `<result> = phi <ty> [ <val0>, <label0>], ...`, "used to implement the φ node in the SSA graph representing the function". It takes "a list of pairs as arguments, with one pair for each predecessor basic block of the current block", and "PHI instructions must be first in a basic block", which is why block `%8` opens with its two phis before any other instruction. At runtime the instruction "logically takes on the value specified by the pair corresponding to the predecessor basic block that executed just prior to the current block". The machine never sees a phi: the backend lowers merges into register moves on the incoming edges.

PITFALL: a phi is the merge, not a sequence of moves

Reading `%19 = phi i32 [ %17, %16 ], [ %13, %8 ]` as "first take `%17`, then take `%13`" imposes an order the ir does not have. All pairs describe the same instant, the moment control enters the block, and exactly one pair is live per execution. The frontend emits phis for conditional-expression merges only; for C statement mutation across an if, the -O0 shape is stores to memory, and the phi exists only after mem2reg reads the pattern. The single phi in the whole -O0 module of `ir.c` is `%27 = phi i32 [ -1, %23 ], [ %25, %24 ]` inside `@_vsnprintf_l`, lowered from the ternary in the crt header that clamps the return value. That is the entire population.

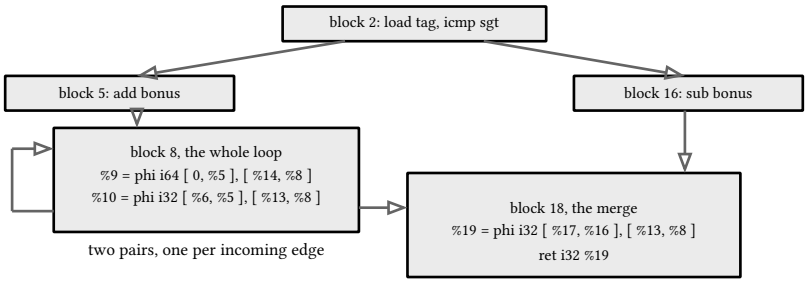


figure 30: tally at -O1: blocks, the back edge, and the three phis with the pairs they merge

At -O2 the loop collapses but the merge survives: the body becomes `%6 = getelementptr inbounds nuw i8, ptr %0, i64 4`, `%7 = load <4 x i32>, ptr %6, align 4`, and `%8 = tail call i32 @llvm.vector.reduce.add.v4i32(<4 x i32> %7)`, four scalar loads folded into one vector load and a horizontal add, while block 13 still opens with `%14 = phi i32 [ %12, %11 ], [ %10, %5 ]` before the `ret`. The branch is the last control dependence the optimizer cannot remove, because `bonus` arrives at runtime.

8.3. attributes and metadata

The -O1 define line is a longer sentence than -O0's:

```
define dso_local i32 @tally(ptr noalias nofree noundef readonly
captures(none) %0, i32 noundef %1) local_unnamed_addr #0
```

`noalias` and `noundef` were present at -O0; `nofree`, `readonly`, and `captures(none)` are new, painted by the optimizer after inspecting the body. `readonly` is inference, not translation: the source says `const struct packet *restrict`, yet the -O0 line carries no readonly, because `const` qualifies the accesses while the attribute claims something about the whole function, that it stores nothing through the pointer. True here, and provable from the body alone. The reference defines `noalias` exactly as chapter 4's promise: "memory locations accessed via pointer values based on the argument or return value are not also accessed, during the execution of the function, via pointer values not based on the argument or return value". Chapter 5 showed what the optimizer banks for it, the unguarded vector body against the plain twin's runtime overlap check. `noundef` is shorter: "If the value representation contains any

undefined or poison bits, the behavior is undefined". It converts garbage-in into a contract violation, which later passes may assume cannot happen.

Main gains a return decoration at -O1: `define dso_local noundef range(i32 0, 2) i32 @main()`. Both return statements return constants, 0 and 1, so the frontend paints the half-open range; the reference states the pair semantics, "The pair `a,b` represents the range `[a,b)`", and the consequence, a value outside the range "is converted to poison". Poison is ir's undefined-value mechanism, a placeholder for a value the optimizer has proven cannot occur, it spreads through every computation that consumes it, and a branch or side effect that depends on it is undefined behavior. The gate's own exit discipline is readable in the signature. Function bodies reference numbered groups, and the group at the bottom of the -O2 file reads `mustprogress nofree norecurse nosync nounwind willreturn memory(argmem: read)` plus the target string `"target-cpu"="x86-64" "target-features"="+cmov,+cx8, +fxsr,+mmx,+sse,+sse2,+x87"`, the baseline the whole module was compiled for. `mustprogress` licenses the assumption the reference names: "a loop in a function with the mustprogress attribute can be assumed to terminate if it does not interact with the environment in an observable way". `norecurse` says the function "never occurs inside a cycle in the dynamic call graph". `memory(argmem: read)` summarizes every memory effect into one attribute: reads, and only its own arguments.

Metadata rides on instructions and on the module. The loop back edge carries `!llvm.loop !7` even at -O0, and the node's content is `!8 = !{!"llvm.loop.mustprogress"}`, the same progress license echoed per loop. The module tail stamps its producer twice, `!llvm.ident = !{!6}` with `!6 = !{!"clang version 23.1.2 ..."}` (the book froze under 23.1.1 and the machine drifted to 23.1.2, so the expectation pins the major only, the ident's presence rather than its patch number, the drift rule applied to this pin too) and a `!llvm.dbg.cu` compile unit whose `isOptimized` field flips from false at -O0 to true at -O2. The `!alias.scope` and `!noalias` metadata from chapter 5's guarded loop body are the optimizer annotating its own runtime check; the reference describes the pair as "generic noalias memory-access sets" in which instructions carrying `noalias` metadata "can specifically be specified not to alias with some other collection of memory access instructions that carry `alias.scope` metadata".

NOTE: pin what exists

This section's figure-plan row originally named `!tbaa` as the metadata to show. The emitted files contain none of it at any level, and the reason is pinned in chapter 5 as cos-004: the msvc-target driver injects `-relaxed-aliasing` into every cc1 invocation, which disables type-based alias analysis for the whole module, and no flag turns it back on. The plan row was corrected to the probed absence, and the gate pins `noalias`, `range`, and `!llvm.loop` instead. Evidence has to exist before it can be asserted.

token	seen in	written by	what it claims
noalias	tally %0, all levels	frontend, from restrict	no access through other bases
noundef	every parameter, -O0	frontend	undef bits become undefined behavior
readonly	tally %0, -O1 and -O2	the optimizer	nothing is stored through it
captures(none), nofree	tally %0, -O1 and -O2	the optimizer	the pointer never escapes or frees
range(i32 0, 2)	main return, -O1	frontend	the value is 0 or 1, else poison
!llvm.loop	the branch back edge	frontend	loop identity, mustprogress echo
!llvm.ident	module tail	frontend	the clang 23.1.1 stamp
!alias.scope, !noalias	ch5 guarded vector body	the optimizer	disjoint scopes from a runtime check
!tbaa	nowhere, any level	nobody: -relaxed-aliasing	off on this target, cos-004

figure 31: the module's self description: token, where it appears, who wrote it, and what it promises

8.4. gep and typed pointers

Every pointer in this module is `ptr`. There is no `i32*` anywhere in a 23.x module: the opaque pointers migration made `ptr` the only pointer type, enabled by default in llvm 15 and, quoting the migration page's version table for 17, "Only opaque pointers are supported. Typed pointers are not supported." The page records the reasoning:

the pointee type “carries no real semantics” because pointers could always be cast between pointee types, and the community concluded “the costs of pointee types outweigh the benefits”. Interpretation moved onto the instructions, `load i32, ptr %8` says what is read, and the one instruction that still needs a type to scale offsets is gep, whose first operand is the source element type.

The two element walks at -O0 are the whole lesson: `%8 = getelementptr inbounds nuw %struct.packet, ptr %7, i32 0, i32 0` reads p→tag. The slots walk is three instructions: `%21 = getelementptr inbounds nuw %struct.packet, ptr %20, i32 0, i32 1` selects the field, `%23 = sext i32 %22 to i64` widens the C `int` index to the pointer index type, and `%24 = getelementptr inbounds [4 x i32], ptr %21, i64 0, i64 %23` scales it by the element size. The reference’s rule: the instruction “performs address calculation only and does not access memory”, “the first index always indexes the pointer value given as the second argument, the second index indexes a value of the type pointed to”, struct indices must be `i32` constants, and array indices “are not required to be constant”. The offset itself is the index “multiplied by the type allocation size”: the field walk lands at `p + 4`, the `offsetof` check’s 4, and the array walk at `p + 4 + 4i`. The dedicated gep page adds the two sentences that keep beginners out of trouble: “The GetElementPtr instruction dereferences nothing”, and without `inbounds` “there are no restrictions on computing out-of-bounds addresses”.

The flags are promises with proofs attached. `inbounds` says the base points into an allocated object and every successive addition keeps the result “in bounds of the allocated object at each step”; a gep with all-zero indices is always inbounds. `nuw` strengthens this to no-wrap arithmetic: the index multiplication and offset additions “do not wrap the pointer index type in an unsigned sense”. The emission is exact about what is provable when: both constant-offset field geps carry `inbounds nuw` at -O0, a constant offset cannot wrap, while the runtime-index gep carries `inbounds` alone, because no frontend proof exists that an arbitrary index times the element size stays inside the pointer’s unsigned arithmetic. At -O1 the loop’s element walk reappears as `%11 = getelementptr inbounds nuw [4 x i8], ptr %7, i64 %9`: the same addresses, the stride folded into a 4-byte element type, and `nuw` now claimed because the loop test proved `0 ≤ %9 < 4`. Flags grow with proof. At -O2 the walk compresses to `%6 = getelementptr inbounds nuw i8, ptr %0, i64 4` and a single `<4 x i32>` load across all 16 bytes of `slots`.

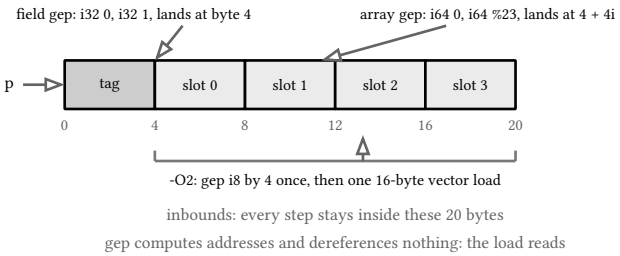


figure 32: struct.packet on the byte strip: where the two geps land and what -O2 does to the walk

This is the arithmetic under C’s pointer rules. Chapter 4’s `a + 2` and `*(*(m + 1) + 2)` lower to geps, chapter 5’s optimizer restated its aliasing promises as `getelementptr inbounds` on walks it invented, and the type-based analysis this target keeps switched off would have consumed the same instruction stream. The next chapter stops reading the ir and starts asking what the passes do to it.

sources: llvm.org/docs/LangRef.html (the phi instruction, the getelementptr instruction with its inbounds and nuw rules, the parameter attributes noalias, noundef, and range, the function attributes mustprogress and norecurse, and the noalias and alias.scope metadata), llvm.org/docs/GetElementPtr.html, llvm.org/docs/OpaquePointers.html, and clang.llvm.org/docs/ClangCommandLineReference.html for -emit-llvm and -S, all accessed 2026-09-12; every quoted .ll line emitted by clang 23.1.1 on this machine the same day, and the tbaa absence follows the -relaxed-aliasing injection pinned in chapter 5. Sample behavior verified by `make verify-c`, 10 checks in chapter 8 of the samples suite.

9. optimization: what -O actually does

Every sample in this book is compiled more than once. The run leg of the gate names no `-O` flag, which is `-O0`, and counts checks. The ir and objdump legs recompile the same files at explicit levels and assert what changed underneath. This chapter walks two of those files up the ladder: `mix` in `mem2reg.c`, whose `-O0` body is a stack-frame simulation, and the `axpy` loop in `vector.c`, which the optimizer rebuilds into packed sse arithmetic. Every claim about a pass is one of three things: a substring the gate asserts in emitted output, a `-Rpass` remark this clang printed on this machine, or a sentence from the llvm documentation fetched 2026-09-12. The windows release of llvm ships no `opt` binary, so the pipelines are observed through the clang driver itself; chapter 7 owns the inventory of what is missing.

9.1. mem2reg, -O0 ir versus -O2 ir

`-O0` emits ir that mirrors the source statement by statement. Every local variable becomes an `alloca`, which the language reference defines as an instruction that allocates memory on the stack frame of the currently executing function, to be automatically released when this function returns to its caller. Every read becomes a `load`, every assignment a `store`. The whole function under study is four lines of arithmetic:

```
static int weight(int x) { return 3 * x + 1; }

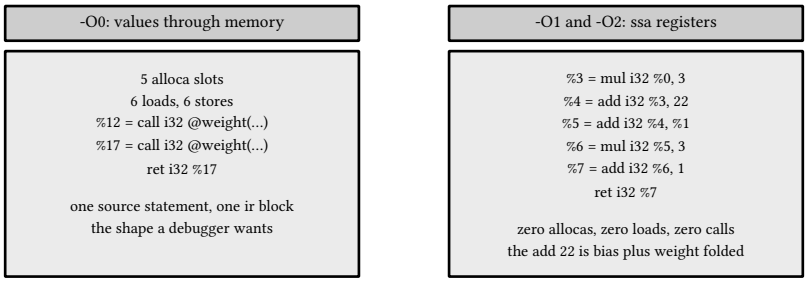
static inline __attribute__((always_inline)) int bias(int x) { return x + 7; }

int mix(int a, int b) {
  int t = bias(a);
  int u = weight(t);
  u = u + b;
  return weight(u);
}
```

listing 62: the two helpers and mix, one plain, one `always_inline`, both called twice (source: c-os-cloud/samples/src/Ch09/mem2reg.c, lines 19-28)

At `-O0` `mix` gets 5 `alloca` slots, 6 `load` instructions, 6 `store` instructions, and 2 `call` instructions. Line 24 stores `bias(a)` into `t`'s slot, line 25 loads it back and hands it to `weight`, line 26 loads `u` and `b`, adds them, and stores the sum into the same slot, line 27 loads it and calls `weight` again. One of the three helper invocations is not a call even here: `bias` is folded into the instruction `add i32 %9, 7` right inside the body, a fact section 2 returns to. The `100`, `67`, and `43` that `main` checks are computed at runtime through this memory traffic, one pass per argument pair.

mem2reg is the pass that deletes the scaffolding. It walks each `alloca` whose address never escapes, replaces every `load` with the value most recently stored, deletes the `store`, and where control flow merges two different values into one slot it manufactures a `phi` node, the instruction chapter 8 reads in depth. `mix` has no branches, so no phi appears, and the whole `-O2` body is five register operations and a ret: `mul i32 %0, 3`, `add i32 %3, 22`, `add i32 %4, %1`, `mul i32 %5, 3`, `add i32 %6, 1`, `ret i32 %7`. The constants record the folds. `bias` adds 7 and `weight` adds 1 after multiplying by 3, so `weight(bias(a))` collapses to the multiply by 3 and the single `add i32 %3, 22`, and the outer `weight` reappears as the trailing multiply and `add i32 %6, 1`. `-O1` emits the identical body, and the gate pins `mul i32 %0, 3` at both levels. No arithmetic anywhere in this file carries `nsw`: the msvc driver injects `-fwrapv` into every cc1 invocation, chapter 5's finding, and this chapter's emitted files re-verify it, the `nsw` substring exists at neither `-O0`, `-O1`, nor `-O2`, so no fold here banks an undefined-overflow assumption.



both bodies are pinned in Ch09 expect-ir.txt, alloca at -O0, mul at -O1 and -O2

figure 33: the same mix at -O0 and at -O1, -O2, mem2reg deleted every alloca the frontend needed

The gate asserts four substrings for this file: `alloca` and `call i32 @weight` at `-O0`, `mul i32 %0, 3` at `-O1` and at `-O2`.

9.2. inlining, the call graph collapse

At `-O0` `weight` is a real symbol with `internal` linkage, one define, three call sites, two in `mix` and one in `main`'s check. At `-O2` all three calls are gone and so is the define: every caller holds a folded copy, and an internal function with no remaining callers is dead. `mix` itself is inlined into `main` three times, once per check, and survives as an outline anyway because its linkage is external, so the definition must ship for callers this file cannot see. The remark stream names every fold: `'weight' inlined into 'mix'`
`with (cost=-25, threshold=337)`, `'mix' inlined into 'main' with`
`(cost=-40, threshold=337)` at each of the three checks, and `'weight'` into `'main'` with `cost=-15035`, the check call that constant-folds to nothing.

Negative cost means the inliner estimates the fold shrinks code: the call sequence it deletes costs more bytes than the body it copies in. The threshold is the brake. Every inline trades call overhead for code size, code size is not free, a bigger binary pays in instruction cache misses, and the inliner prices each site against a budget instead of folding everything it can see. `-Os` and `-Oz` exist as documented levels for the size-first end of that trade.

`always_inline` waives the budget. The language reference defines the attribute as an instruction to the inliner to attempt to inline this function into callers whenever possible, ignoring any active inlining size threshold for this caller. This clang honors it at `-O0`, where nothing else inlines: `-Rpass=inline` at `-O0` prints exactly two remarks for this file, `'bias' inlined into 'mix'` and `'bias'`
`inlined into 'main'`, both carrying `(cost=always): always inline`
`attribute`, while `-Rpass-missed=inline` prints nothing, the ordinary inliner does not run at `-O0` at all. `bias` never exists as a call or a define at any level. The attribute is a promise about every caller, which is why it belongs on tiny helpers and on almost nothing else.

```c
int main(void) {
  CHECK(bias(5) == 12, "always_inline helper computes");
  CHECK(weight(4) == 13, "plain helper computes");
  CHECK(mix(2, 5) == 100, "mix of 2 and 5");
  CHECK(mix(0, 0) == 67, "mix of 0 and 0");
  CHECK(mix(-3, 1) == 43, "mix of -3 and 1");
  return 0;
}
```

listing 63: the checks: helper calls and mix results, computed and compared at runtime (source: c-os-cloud/samples/src/Ch09/mem2reg.c, lines 30-37)

The checks in that listing are what the optimizer may not touch. At `-O2` the comparisons fold to constants, `mix(2, 5)` is `100` at compile time, so `main` prints five unconditional `ok` lines with no branch anywhere. The printed bytes and the exit code are identical at `-O0` and `-O2`, chapter 5's as-if seam seen from the optimizer's side: everything may change as long as the observable behavior does not.

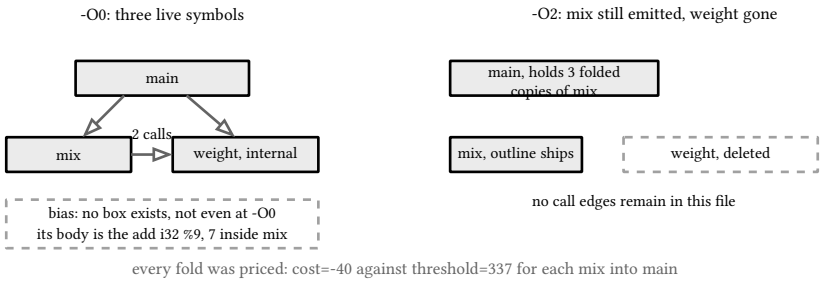


figure 34: the call graph of mem2reg.c at -O0 and at -O2, weight vanishes, mix survives standalone

PITFALL: remarks are diagnostics, not a contract

The users manual warns twice. Do not expect a report from every transformation made by the compiler, and the locations remarks carry are translated from debug annotations, a translation that can be lossy, so some remarks arrive without one. The remarks quoted in this chapter are observed output of this clang on these files, a window into one build, not an interface to program against. The gate pins the transformations themselves, in ir and disassembly, and never a remark string.

9.3. vectorization evidence

The kernel is one line of body:

```
static void axpy(float a, const float *x, float *y, int n) {
  for (int i = 0; i < n; i++) {
    y[i] = a * x[i] + y[i];
  }
}
```

listing 64: the axpy loop, float, trip count unknown to the compiler (source: c-os-cloud/samples/src/Ch09/vector.c, lines 19-23)

At `-O0` the loop runs one float at a time, and the `-O0` disassembly shows it: `movss 0x34(%rsp), %xmm1` loads one `y` element, `mulss %xmm2, %xmm0` multiplies one lane, `addss %xmm1, %xmm0` adds one lane, `movss %xmm0, 0x34(%rsp)` stores one result, 64 passes for 64 elements. Scalar sse is still sse: those instructions live in `xmm` registers because the x86-64 baseline has no legacy x87 path for this arithmetic. So `xmm` in a disassembly is not by itself evidence of vectorization, and the gate pins two substrings for exactly that reason: `xmm`, which both levels show, and `mulps`, the packed multiply only `-O2` emits.

At `-O1` the vectorizer does not run. Its missed remark says so in one line, `loop not vectorized: only vectorizing loops that explicitly request it`, the `-O1` policy, loops carrying a `#pragma clang loop vectorize(enable)` request excepted. At `-O2` the remark flips to `vector.c:20:3: remark: vectorized loop (vectorization width: 4, interleaved count: 2)`. The vectorizer documentation states the mechanism: the loop vectorizer uses a cost model to decide on the optimal vectorization factor and unroll factor, and both the loop and slp vectorizers are enabled by default at the levels that run them.

The `-O2` ir shows the rebuild. A runtime guard `icmp ult i32 %33, 8` sends short counts straight to the scalar tail. The volatile-loaded `scale` becomes a vector splat through `insertelement <4 x float> poison, float %34, i64 0` followed by a `shufflevector` with a zeroinitializer mask. The induction is `%44 = phi i64 [ 0, %39 ], [ %55, %43 ]`, a phi the optimizer manufactured, the same merge instruction mem2reg uses, doing loop duty. Each pass loads two `<4 x float>` vectors from `xs` and two from `ys`, computes two `@llvm.fmuladd.v4f32` calls, and stores both back: 8 floats per pass, the interleave the remark reported. The scalar epilogue keeps the same shape one float wide with `llvm.fmuladd.f32`. The language reference defines the intrinsic family as multiply-add expressions

that can be fused if the code generator determines that the target instruction set has support for a fused operation and the fused operation is more efficient than the equivalent separate pair of mul and add instructions. This target's emitted feature string is `"+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87"`, no fused instruction exists there, the first condition fails, and the intrinsic lowers to a multiply and an add.

```
int main(void) {
  enum { N = 64 };
  static volatile float scale = 2.5f;
  static volatile int count = N;
  float xs[N];
  float ys[N];
  for (int i = 0; i < N; i++) {
    xs[i] = (float)i;
    ys[i] = (float)(2 * i);
  }
  axpy(scale, xs, ys, count);
  CHECK(ys[0] == 0.0f, "axpy element 0");
  CHECK(ys[1] == 4.5f, "axpy element 1");
  CHECK(ys[63] == 283.5f, "axpy element 63");
```

listing 65: volatiles hide the multiplier and the trip count, the fill loop is plain, the first checks are element exact (source: c-os-cloud/samples/src/Ch09/vector.c, lines 25-38)

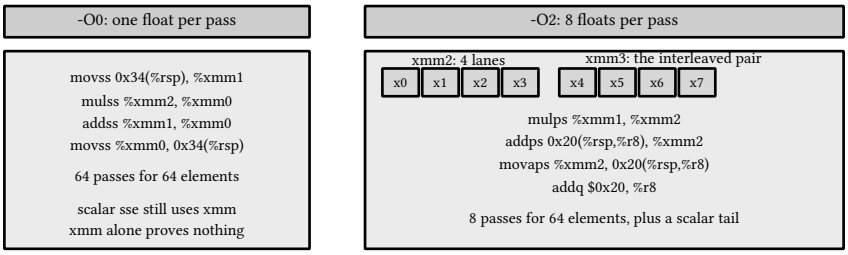
The loop stays a loop because `main` hides two facts from the optimizer: `scale` and `count` are `volatile`, so the multiplier and the trip count are runtime values. The fill loop above them has no such protection and at `-O2` dissolves entirely, unrolled and constant-folded into 32 `store <4 x float>` instructions, sixteen per array.

```
  float sum = 0.0f;
  for (int i = 0; i < N; i++) {
    sum += ys[i];
  }
  CHECK(sum == 9072.0f, "axpy accumulated sum");
  float ref = 0.0f;
  for (int i = 0; i < N; i++) {
    ref += 4.5f * (float)i;
  }
  CHECK(sum == ref, "vectorized loop agrees with scalar formula");
  float z[4] = {9.0f, 9.0f, 9.0f, 9.0f};
  axpy(scale, z, z, 0);
  CHECK(z[0] == 9.0f && z[3] == 9.0f, "zero length call touches nothing");
```

listing 66: the reduction checks and the zero length call, every value binary exact (source: c-os-cloud/samples/src/Ch09/vector.c, lines 39-51)

The checks are exact by construction: `2.5f` is binary-exact, every index below 64 is exact in `float`, and every product and sum lands on a value a `float` holds exactly, so `ys[1] == 4.5f` and the `9072.0f` accumulated sum pass identically whether the loop runs one float at a time or eight. The zero-length call tests the guard: with `n` 0 the entry branch `icmp sgt i32 %33, 0` fails and `z` is untouched.

The machine code carries the same structure. The setup broadcasts the scale into all four lanes, `shufps $0x0, %xmm0, %xmm1`, the counterpart of the `shufflevector` splat. The body then loads with `movaps 0x120(%rsp,%r8), %xmm2`, multiplies with `mulps %xmm1, %xmm2`, adds with `addps 0x20(%rsp,%r8), %xmm2`, repeats the three for `xmm3` at offsets `0x130` and `0x30`, stores both results with two more `movaps`, advances with `addq $0x20, %r8`, and loops on `cmpq` and `jne`. The `p` in `mulps` and `addps` is packed, four lanes per instruction, the interleaved `xmm2` and `xmm3` pair is the count of 2 from the remark, and the 32-byte advance is 8 floats, so 64 elements finish in 8 passes plus the scalar tail for trip counts that are not multiples of 8.



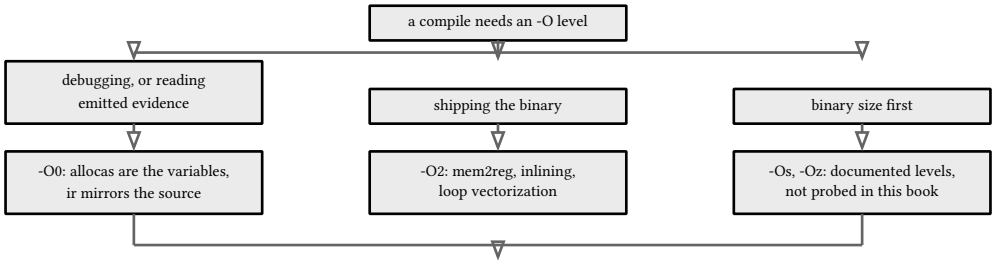
the p in mulps and addps is packed, four lanes per instruction

figure 35: one pass of the axpy loop at -O0 and at -O2, lanes and stride

9.4. reading remarks, when -O0 wins

`-Rpass` is the documentation's own window into these decisions. The users manual defines the family by the three moments: when the pass makes a transformation, `-Rpass`, when the pass fails to make one, `-Rpass-missed`, and when the pass determines whether or not to make one, `-Rpass-analysis`, and notes that each of these flags takes a regular expression that identifies the name of the pass. The manual's own example is `clang -O2 -Rpass=inline code.cc -o code`, which is how every remark in this chapter was produced, with `-Rpass=inline` and `-Rpass=loop-vectorize` on these files. Underneath, the levels expand to pipelines through the pass builder: the pass manager documentation shows `buildPerModuleDefaultPipeline(OptimizationLevel::O2)` assembling what corresponds to a typical -O2 optimization pipeline, and notes that the optimization pipeline, the middle end, uses the new pass manager. The driver is the only way to reach those pipelines on this box, the standalone runner named in the opening paragraph is not in this release.

`-O0` wins three ways in this book. Debuggability: the allocas of section 1 are the variables, and a debugger can name every intermediate because every intermediate has a slot and a source line. Evidence: `-O0` ir mirrors the source one statement per block, which is why the ir pins across the evidence chapters, including this one, read `-O0` output, and why the sanitizer leg of the gate runs at the level the driver defaults to. Compile speed: the command line reference documents the level flags as controlling how much optimization should be performed, `-O0` performs almost none, and `-O` is equivalent to `-O1`; the same page marks `-Ofast` as deprecated, pointing at `-O3 -ffast-math`. `-O2` is the shipping level this chapter measured, and `-O3`, `-Os`, `-Oz` are the other documented rungs. No level changes observable behavior, and this chapter's gate rows hold the discipline in miniature: the same two files pass 11 runtime checks at `-O0` and satisfy 9 ir and objdump assertions across `-O0`, `-O1`, and `-O2`.



every level preserves observable behavior, the 11 checks pass at -O0

figure 36: choosing a level, and what each answer buys

VERIFY: one file, two legs, two kinds of truth

The run leg compiles `mem2reg.c` and `vector.c` with no `-O` flag and counts 11 checks, all behavioral, all passing on the `-O0` build. The ir leg compiles the same files at `-O0`, `-O1`, and `-O2` and asserts 9 substrings: 4 in `mem2reg.c` ir, 3 in `vector.c` ir, 2 in the `-O2` disassembly. Behavior is claimed where it runs, structure is claimed where it is emitted, and no claim crosses legs.

sources: llvm.org/docs/LangRef.html (the `alloca` instruction, the `alwaysinline` attribute, the `llvm.fmuladd.*` intrinsics), clang.llvm.org/docs/UsersManual.html (the -Rpass remark family), clang.llvm.org/docs/ClangCommandLineReference.html (the optimization level flags), llvm.org/docs/NewPassManager.html (Pass-

Builder and default pipelines), llvm.org/docs/Vectorizers.html (the loop vectorizer cost model and remarks), accessed 2026-09-12; the ir bodies, disassembly, and remark lines quoted in this chapter probed on this machine the same day. Sample behavior verified by `make verify-c`, 11 checks in chapter 9 of the samples suite.

10. clang tooling and diagnostics

Chapter 1 built a gate and this chapter opens the toolbox it stands on: the compiler's own diagnostic machinery, the format leg, `clang-tidy`, the static analyzer, and the two windows into the ast that explain what all of these tools are actually looking at. Every transcript quoted below is real output from the llvm 23.1.1 binaries installed at `C:\Program Files\LLVM\bin`, probed on 2026-09-12; paths are shortened to file names and ast node addresses are elided, nothing else is retyped. Three samples carry the chapter: `diagnostics.c` compiles warning clean while carrying diagnostic surfaces on purpose, `ast.c` and `tidy.c` are the small subjects the ast and tidy tools are run against.

The gate's floor is unchanged since chapter 1: every sample compiles with `-std=c23 -Werror -Wall -Wextra`. That floor is the reason this chapter's discipline section exists. A toolchain that silently accepts nearly-correct code teaches nothing; one that turns every warning into a stopped build forces the cause to be fixed while it is still cheap.

10.1. diagnostics and -Werror discipline

The users manual states the control surface compactly: `-Werror` "turn warnings into errors", `-Werror=foo` turns one named group, `-Wno-error=foo` turns that group back into a warning even under `-Werror`, and `-w` disables all warnings while "errors are still emitted". The diagnostics reference lists every `-W` group alphabetically with its default status, including the downgrade sentence for default errors: "this diagnostic is an error by default, but the flag" `-Wno-...` "can be used to disable the error". The manual also documents `-Weverything` and then argues against it, `-Wall` and `-Wextra` "are a better choice for most projects". The gate took exactly that advice. Errors stop the build, warnings under this floor stop it too, and remarks never stop anything: the manual reserves `-Rpass`, `-Rpass-missed`, and `-Rpass-analysis` for optimizer commentary, each "a regular expression that identifies the name of the pass which should emit the associated diagnostic".

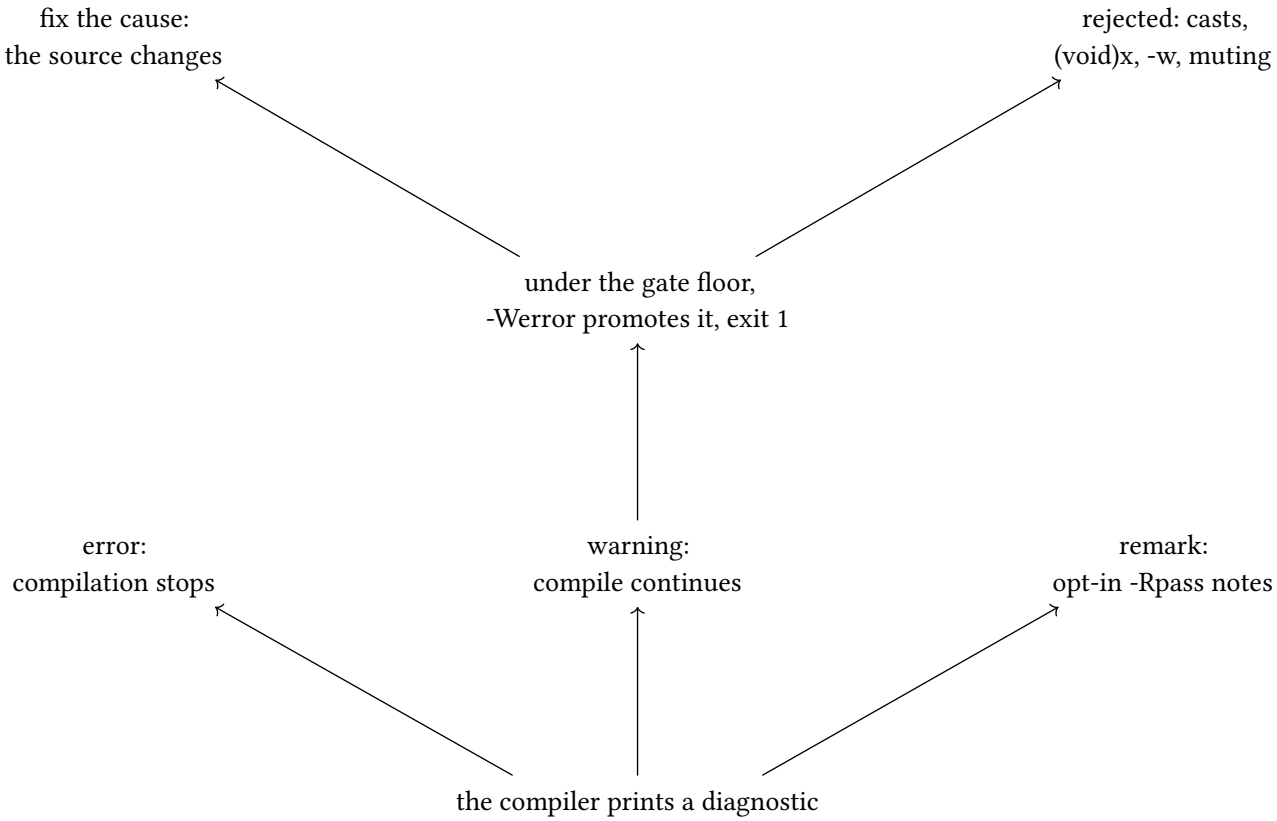


figure 37: the diagnostic taxonomy and the fix discipline under the gate

The first sample carries its diagnostic surfaces deliberately, and all of them stay silent on this toolchain:

```
#if !defined(__STDC_VERSION__) || __STDC_VERSION__ < 202311L
#error "this sample requires c23, __STDC_VERSION__ ≥ 202311L"
#endif

#ifdef TOOLING_DEMO_WARNING
#warning "a warning class diagnostic, deliberately behind this guard"
#endif
```

listing 67: the floor guard and the guarded warning, both silent on this toolchain (source: c-os-cloud/samples/src/Ch10/diagnostics.c, lines 9-15)

The `#error` directive is the program refusing to compile below the c23 floor. Probed with `-std=c17`, the guard fires: `diagnostics.c:10:2: error: "this sample requires c23, __STDC_VERSION__ ≥ 202311L"`, one error generated, nothing else in the pipeline runs. The `#warning` behind its guard macro is the same machinery at the softer severity: compiled with `-DTOOLING_DEMO_WARNING` the file prints `diagnostics.c:14:2: warning: "a warning class diagnostic, deliberately behind this guard" [-W#warnings]` and the compile succeeds. Add `-Werror` and the same line comes back as an error whose group list now carries the promotion, `[-Werror,-W#warnings]`, exit 1. Warning, error, and promoted warning, one file, three probes.

The third surface is the standard's own compile-time check, each assert paired with a runtime twin:

```
_Static_assert(CHAR_BIT == 8, "char holds 8 bits on this target");
_Static_assert(sizeof(int) == 4, "int is 4 bytes on this target");
_Static_assert(sizeof(void *) == 8, "64 bit pointers on this target");

// -Wextra enables -Wunused-parameter. [[maybe_unused]] is the c23
// structural answer, the reserved parameter stays a documented part of
// the interface instead of a (void)cast that hides the smell
static int reserve_hint([[maybe_unused]] int reserve) { return 42; }
```

listing 68: compile time asserts, and the attribute that answers the unused parameter warning (source: c-os-cloud/samples/src/Ch10/diagnostics.c, lines 33-40)

```
  CHECK(CHAR_BIT == 8, "char bit width matches its static assert");
  CHECK(sizeof(int) == 4, "int width matches its static assert");
  CHECK(sizeof(void *) == 8, "pointer width matches its static assert");
  CHECK(reserve_hint(1) == 42, "maybe_unused parameter compiles and calls");
```

listing 69: the runtime twins, one check per compile time fact (source: c-os-cloud/samples/src/Ch10/diagnostics.c, lines 43-46)

`_Static_assert` is a diagnostic that fires before anything runs: if the claim is false the translation fails, if it is true the assert costs nothing and proves only what the compiler can see. The twins in `main` restate the same facts where the gate's run leg can count them, four checks the harness verifies by execution. Compile-time and runtime evidence for the same proposition, the pairing this book uses everywhere a fact is checkable from both sides.

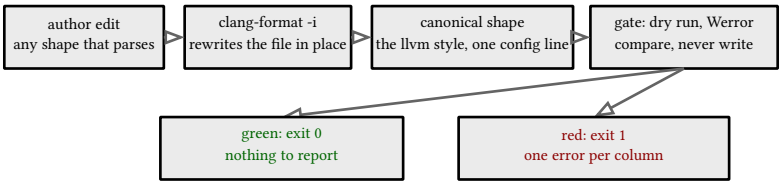
The last function in that listing is the discipline itself. `-Wunused-parameter` comes with `-Wextra`, so the gate enables it. Probed on a scratch twin that drops the attribute, the compile prints `noattr.c:1:19: warning: unused parameter 'reserve' [-Wunused-parameter]`, and under the floor the same diagnostic returns as `[-Werror,-Wunused-parameter]` and stops the build. The c23 `[[maybe_unused]]` answers by declaring the intent in place: the parameter is a reserved part of the interface, present for the signature, unused by the body, and the declaration says so where a reader finds it. The older answers, a `(void)reserve;` statement to consume the name or a flag muting the whole group, remove the message and keep the cause. The users manual documents the last resort, `#pragma clang diagnostic push`, an `ignored` line, and `pop`, which scopes any suppression to exact lines and restores the floor after; this book never needed it, because every warning it met had a fix at the source.

PITFALL: a silenced warning is a loan, not a fix

> `-w` in the project flags, `-Wno-foo` at the edge of the command line, the `(void)x` cast, the dummy use assignment: each removes the message and keeps the cause, and the interest comes due at the next compiler release or the next reader. The attribute and the scoped pragma at least state the intent where the code lives. The gate itself was proven to go red before it was trusted, chapter 1 asserted the wrong `__STDC_VERSION__` value first and watched the harness fail the run. A tool that cannot go red is a rubber stamp, and a warning silenced before it is read never even gets the chance.

10.2. clang-format as a gate leg

The format leg of the gate runs `clang-format --dry-run --Werror --style=file` over every `.c` and `.h` under the samples tree. The style contract is one line, `.clang-format` at the samples root reading `BasedOnStyle: LLVM`. The format page defines what that means: `'file'` loads "style configuration from a .clang-format file in one of the parent directories of the source file", so every chapter's files walk up and find the same one-line config, and the fallback style never engages. `--dry-run` means "if set, do not actually make the formatting changes", and `--Werror` means "if set, changes formatting warnings to errors": the format leg is the `-Werror` story a second time, told over whitespace.



when the leg goes red, the fix is the same -i run that wrote the canonical shape
style=file walks up from each source file and finds the one line config at the samples root

figure 38: the format leg: one canonical shape, two exits

VERIFY: the format leg reports drift, probed red

A drift copy written in a temp directory beside the same `.clang-format`, one dense `main` line, three statements collapsed onto it: the leg returned `drift.c:3:17: error: code should be clang-formatted [-Wclang-format-violations]`, repeated once per offending column, fourteen for that line, exit 1. Running `clang-format -i` on the same file rewrote it into the canonical shape, and the identical dry-run invocation then exited 0 with no output. Red first, then green, the same evidence shape chapter 1 demanded from the run leg. The shipped files pass because they were formatted with the same command before the gate ran: this chapter's gate line ends `format clean`.

The reason the leg exists is review friction. A diff that is half whitespace buries the half that matters, and a tree without a canonical shape pays that tax on every file forever. This book has a second reason: every listing pins a real file by exact line range, so whitespace churn is not cosmetic here, reformatting a sample file silently moves every pin below the change. The format leg freezes the shape the pins were measured against.

10.3. clang-tidy and the static analyzer

`clang-tidy` is documented as "a clang-based C++ 'linter' tool", an "extensible framework for diagnosing and fixing typical programming errors", with checks selected by "a comma-separated list of positive and negative (prefixed with -) globs" and, for a single file, compilation options given "on the command line after" `--`. On this box the tool needs the same `-isystem` plumbing chapter 1 documented: run bare, tidy's internal compile dies on `tidy.c:6:10: error: 'stdio.h' file not found [clang-diagnostic-error]`, the msvc auto-detection failure applies to it as well, probed 2026-09-12. The subject file carries two shapes on purpose:

```
// bugprone-macro-parentheses fires on this definition: the parameter
// wants parentheses inside the list and the list wants them around the
// whole body. the checks below pin what today's body actually
// computes, correct for a single token, wrong for a sum
#define HALF(x) x / 2
```

listing 70: the flagged macro, parentheses missing inside and out (source: c-os-cloud/samples/src/Ch10/tidy.c, lines 21-25)

```
  CHECK(HALF(8) == 4, "half of one token expands correctly");
  CHECK(HALF(6 + 2) == 7, "half of 6+2 computes 7, not 4");

  // narrowing long long to int is implementation defined at 6.3.1.3,
  // this compiler wraps modulo 2^32, and the check pins the wrap
  long long wide = 5000000000LL;
  int banked = wide;
  CHECK(banked == 705032704, "narrowing stores the low 32 bits, 705032704");

  int grid[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
  int where = -1;
  CHECK(grid_find(grid, 5, &where) == 1 && where == 4,
        "grid_find locates 5 at flat index 4");
  CHECK(grid_find(grid, 42, &where) == 0, "grid_find reports the miss");
```

listing 71: wrong expansion and narrowing pinned at runtime, plus the clean scan's checks (source: c-os-cloud/samples/src/Ch10/tidy.c, lines 43-56)

The run is `clang-tidy --checks="-*,bugprone-*" tidy.c -- -std=c23` plus the harness include paths, and it prints two findings. `tidy.c:25:19: warning: macro replacement list should be enclosed in parentheses [bugprone-macro-parentheses]`, with a fix-it that draws the parentheses in under the caret. And `tidy.c:49:16: warning: narrowing conversion from 'long long' to signed type 'int' is implementation-defined [bugprone-narrowing-conversions]`. The trailer counts the rest: "55 warnings generated. Suppressed 53 warnings (53 in non-user code)", tidy read the system headers too and suppressed them by default, `-header-filter=` decides what counts as user code. The plain two dimensional scan `grid_find` in the same file draws nothing.

Both findings name real contracts. The macro check "finds macros that can have unexpected behavior due to missing parentheses" and implements the CERT rule PRE02-C, and the runtime check right below the definition pins what the missing parentheses do: `HALF(8)` is 4 for a single token, `HALF(6 + 2)` expands to `6 + 2 / 2` and computes 7, not 4. The narrowing check "checks for silent narrowing conversions" and "diagnoses more instances of narrowing than the compiler warning" `-Wconversion` "does", which matters here because the gate floor enables neither: the compiler passed this line in silence. The conversion itself is the out-of-range one: C23 made two's complement the only allowed signed representation, so the encoding is not in play, what the standard leaves open is converting a value the signed type cannot hold, which yields an implementation-defined result or raises an implementation-defined signal, this compiler defines the result as wrapping, and the check pins it: 5000000000 stored into an `int` reads back 705032704, the low 32 bits.

Findings do not stop tidy. The run above exits 0, because tidy warnings are advisory until promoted: rerun with `--warnings-as-errors="bugprone-*"` and the output ends "2 warnings treated as errors", exit 1. The documentation states the boundary exactly, the option "upgrades any warnings emitted under the `-checks=` flag to errors (but it does not enable any checks itself)", the same promotion shape `-Werror` gives the compiler.

The static analyzer is a different oracle, not a stricter tidy. `clang --analyze` on the shipped file prints nothing and exits 0, no construct in `tidy.c` is a path bug. A probe file written to have one makes the difference visible: a choose function `pick(a, b, which)` whose null guard tests `a` while the deref goes through `chosen`, which is `b` whenever `which` is not 0. The engine walks the branches and prints `pathbug.c:5:10: warning: Dereference of null pointer (loaded from variable 'chosen') [core.NullDereference]`. The guard is true of the wrong pointer; on the path where `a` is live, `which` is 1, and the call passed 0 for `b`, the dereference is provably null. No code ran: the analyzer judged

paths through the ast, where tidy judged one construct at a time. tidy reaches the same engine through its `clang-analyzer-` module, “Clang Static Analyzer checks” in the checks list, and the analyzer’s findings are warnings here too, the probe exited 0.

oracle	what it reads	when it runs	verdict on this chapter's files
clang warnings	parse and sema	every compile	clean, the -Werror floor
clang-format	tokens only	the format leg	clean, exit 0
clang-tidy	ast constructs	on demand	2 findings, exit 0
the analyzer	ast paths	on demand	silent here, 1 on the probe

the analyzer explores branches, the other three judge constructs
all four are silent below the run leg: a wrong value fails the gate even when every tool agrees

figure 39: four oracles over the same three files, what each reads and what each found

NOTE: the finding the check did not print

The macro check fires on `x / 2` but stayed silent on sibling shapes probed the same day: bodies built from `x * x` and `x * x * x` drew no finding, while `x + y`, `-x`, `x / 2`, and `x + x` all did. A body of `x * x` still computes the wrong value for a sum argument, `SQUARE(1 + 2)` expands to 5 against the intended 25, probed and run. Every static oracle has a scope. The run leg is the bottom of the ladder: `HALF(6 + 2)` is pinned at 7 by a check the gate counts, whether or not any static tool looks.

10.4. clang-query and -ast-dump

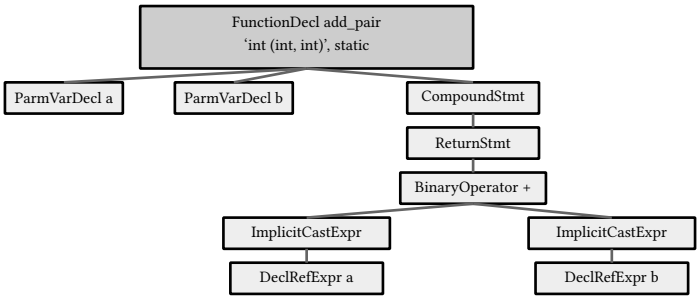
`clang -Xclang -ast-dump -fsyntax-only -std=c23 ast.c` prints the front end’s own tree for the whole file, and `-Xclang -ast-dump-filter=add_pair` narrows the dump to one declaration, prefixed `Dumping add_pair:`. With node addresses elided, the subject function is nine lines:

```
// one return, one binary operator: the smallest function whose dump
// still shows every node kind this chapter reads
static int add_pair(int a, int b) { return a + b; }

// a loop with a call inside: forStmt wrapping a compoundStmt, the call
// reachable by matcher from either direction
static int sum_to(int n) {
  int total = 0;
  for (int i = 1; i ≤ n; i++)
    total = add_pair(total, i);
  return total;
}
```

listing 72: the dump subject and the loop subject, names chosen for matchers (source: c-os-cloud/samples/src/Ch10/ast.c, lines 19-30)

The root is a `FunctionDecl` node carrying everything the compiler knows at declaration time: the source range `<line:21:1, col:51>`, the column of the name, `used add_pair 'int (int, int)' static internal-linkage`, one line that answers name, type, storage, and linkage. Two `ParmVarDecl` children bind `a` and `b`, each marked `used`. The body is a `CompoundStmt` holding one `ReturnStmt`, whose operand is a `BinaryOperator ... 'int' '+'`. The operands are not the names: each one is an `ImplicitCastExpr ... <LValueToRValue>` wrapping a `DeclRefExpr ... lvalue ParmVar ... 'a' 'int'` that points back to the parameter’s own node. Reading a value off a name is a cast in this tree, written by the compiler, invisible in the source. Nothing was computed to produce any of it, the dump is pure structure, which is why the optimizer chapters can reason from the same nodes the checker tools read.



the body of add_pair as the front end sees it, nothing computed
both casts are LValueToRValue, the conversion from name to read value

figure 40: the dumped tree of the subject function, one box per node, the front end's own view

`clang-query` drives the same tree with the matcher language. LibASTMatchers provides "a domain specific language to create predicates on" the ast, "matchers are generated by nesting calls to matcher creation functions", and matchers given several inner conditions combine under an implicit allOf. The query tool itself has no manual page in this release's documentation set, checked in the docs and extra indexes on 2026-09-12; its `--help` is the usage surface: `-c` to "specify command to run", `-p` to read a compile command database, and otherwise the same flags-after-`---` convention tidy uses. Four real queries on the shipped file:

```
struct point {
  int x;
  int y;
};

// a record type and a conditional operator over its fields
static int axis(const struct point *p, int which) {
  return which == 0 ? p->x : p->y;
}

int main(void) {
  struct point p = {.x = 3, .y = 4};
  CHECK(add_pair(p.x, p.y) == 7, "add_pair over designated fields");
  CHECK(sum_to(10) == 55, "sum_to accumulates a call in a loop");
  CHECK(axis(&p, 0) == 3 && axis(&p, 1) == 4, "axis reads either field");
  CHECK(sizeof(struct point) == 8, "two ints, no padding");
  return 0;
}
```

listing 73: the record type, the field selector, and the runtime checks (source: c-os-cloud/samples/src/Ch10/ast.c, lines 32-49)

`match functionDecl(hasName("add_pair"))` returns `1 match.`, a note binding "root" at `ast.c:21:1` with the caret under the whole definition. `match callExpr(callee(functionDecl(hasName("add_pair"))))` returns `2 matches.`, one at `ast.c:28:13` inside the loop body, one at `ast.c:44:9` inside this book's own `CHECK` macro, where the binding note adds `expanded from macro 'CHECK'`: the tree sees through the macro to the call inside. `match recordDecl(hasName("point"))` binds the whole struct, carets under every line from the `struct` keyword to the brace. `match forStmt()` finds the loop at `ast.c:27:3`, and composing the other way, `forStmt(hasDescendant(callExpr()))` asks whether a loop makes a call at all and returns `1 match.` for the same node. Each matcher is a reusable probe over any file that parses, and the composition is the same language clang-tidy checks are written in: the custom check feature documents that its checks are "based on" the query syntax, whose essence "is to parse the query string and dynamically generate the corresponding AST matcher".

The chapter closes where the gate began. The compile leg enforces the diagnostic floor, the format leg freezes the shape, tidy and the analyzer read the same tree from two altitudes, and beneath all four the run leg counts executed checks. In this chapter's three files that ladder bottoms out at 13 checks, every one of them green in the gate run this chapter quotes.

sources: clang.llvm.org diagnostics reference and users manual (the `-Werror` family, `-w`, `-Weverything` with the `-Wall` `-Wextra` advice, `-Rpass` remarks, diagnostic pragmas), ClangFormat (the `-style=file` lookup, `--dry-run`, `--Werror`), the clang-tidy main page and the bugprone macro-parentheses and narrowing-conversions check pages at their clang.llvm.org/extra home (checks globs, `--warnings-as-errors`, flags after `--`), LibASTMatchers, and the query based custom checks page, all accessed 2026-09-12; clang-query usage from the tool's `--help` output, probed on this machine the same day; every quoted diagnostic, transcript, and silent run is real output from the llvm 23.1.1 binaries on this box, probes dated 2026-09-12. Sample behavior verified by `make verify-c`, 13 checks in chapter 10 of the samples suite.

11. processes and program lifecycle

A C program never runs alone. The moment its `main` begins, the operating system has already built a container around it, and the moment `main` returns, a documented sequence of events tears that container down. This chapter pins both halves on windows: what a process is, how one process creates another with `CreateProcessW`, how the parent observes the child through handles and exit codes, and how the C standard's termination machinery, `atexit`, `exit`, `quick_exit`, behaves inside one process. The posix model, `fork` plus `exec`, sits beside it as the comparison the rest of the literature assumes. Every behavioral claim below is a CHECK in the two samples, 33 in total, or a sentence quoted from a canonical page fetched 2026-09-12.

11.1. the process abstraction

The CreateProcessW page states the whole creation contract in one line: it "Creates a new process and its primary thread." Three things come into existence together. A kernel object, owned by the system, holding the process identifier and, later, the exit code. A private virtual address space. And a primary thread, itself a kernel object, whose suspend count this chapter will manipulate directly. The program never touches these objects by pointer: it holds handles, table entries in its own process that refer to the objects, and the discipline of the entire win32 process model is which handles you hold and when you close them.

The gate's own exe is itself somebody's child, and the process can read the paper trail of its own creation. `GetStartupInfoW` "Retrieves the contents of the STARTUPINFO structure that was specified when the calling process was created", and the remarks name the author: "The STARTUPINFO structure was specified by the process that created the calling process."

```
int main(void) {
  // this process exists because someone called CreateProcess on it. the
  // crt read this same structure at startup; cb reports the size the
  // creator's structure had, 104 bytes on x64 (probed)
  STARTUPINFOW si;
  ZeroMemory(&si, sizeof si);
  si.cb = sizeof si;
  GetStartupInfoW(&si);
  CHECK(si.cb == sizeof si, "getstartupinfow returns the creator's structure");
```

listing 74: getstartupinfow proves this process was itself created (source: c-os-cloud/samples/src/Ch11/lifecycle.c, lines 62-70)

The check compares the structure's size field against `sizeof`: `si.cb` is 104 on x64, probed on this box, and it equals the size the sample declared, which means the crt and the kernel agree on the layout the creator filled in. The same structure is the `lpStartupInfo` parameter of `CreateProcessW` itself, so the parent's input and the child's self-knowledge are one type.

Two of the handles in play are not table entries at all. `GetCurrentProcess` returns a pseudo handle, and its page defines the term: "A pseudo handle is a special constant, currently (HANDLE)-1, that is interpreted as the current process handle." The page adds two facts the sample leans on: "Pseudo handles are not inherited by child processes", and closing a pseudo handle "has no effect". The thread twin behaves the same way; its page states the mechanism without the constant, and on this box the value arrives as (HANDLE)-2, probed. The handle table itself is countable from inside through the pseudo handle: `GetProcessHandleCount` "Retrieves the number of open handles that belong to the specified process." Its page documents the count parameter as a pointer to `DWORD` while the SDK header declares `PULONG`, the same unsigned 32 bits either way, a small drift between page and header worth knowing when the compiler is pedantic.

```
// pseudo handles: documented constants, not handle table entries
CHECK((intptr_t)GetCurrentProcess() == -1,
      "current process pseudo handle is (HANDLE)-1");
CHECK((intptr_t)GetCurrentThread() == -2,
      "current thread pseudo handle is -2 (probed)");

// the handle table is countable from inside, through the pseudo handle
ULONG base = handle_count();
CHECK(base > 0, "the process owns a nonzero handle count");

// DuplicateHandle converts the pseudo handle into a real table entry:
// one more handle, different value, and closing it removes exactly it
HANDLE real = NULL;
BOOL dup = DuplicateHandle(GetCurrentProcess(), GetCurrentProcess(),
                           GetCurrentProcess(), &real, 0, FALSE,
                           DUPLICATE_SAME_ACCESS);
CHECK(dup && (intptr_t)real != -1,
      "duplicatehandle turns the pseudo handle into a real one");
ULONG with_dup = handle_count();
CHECK(with_dup == base + 1, "the duplicate adds exactly one handle");
CHECK(CloseHandle(real), "the real handle closes");
CHECK(handle_count() == base, "closing it returns the table to baseline");
```

listing 75: pseudo handles, the duplicate, and the exact handle table audit (source: c-os-cloud/samples/src/Ch11/lifecycle.c, lines 72-93)

The audit walks the table one entry at a time. `DuplicateHandle` converts the pseudo handle into a real one, its page states it: “If hSourceHandle is a pseudo handle returned by GetCurrentProcess or GetCurrentThread, DuplicateHandle converts it to a real handle to a process or thread, respectively.” The count rises by exactly one, the close succeeds, the count falls back to the baseline by exactly the one entry. A real handle, unlike the pseudo constant, is a table entry, and `CloseHandle` “invalidates the specified object handle, decrements the object’s handle count, and performs object retention checks. After the last handle to an object is closed, the object is removed from the system.” That last sentence is the lifetime rule the whole chapter turns on.

```
// a second close of the same value fails: the entry is gone
BOOL again = CloseHandle(real);
DWORD gle = GetLastError();
CHECK(!again && gle == ERROR_INVALID_HANDLE,
      "double close fails with ERROR_INVALID_HANDLE");
```

listing 76: closing the same handle twice fails with error invalid handle (source: c-os-cloud/samples/src/Ch11/lifecycle.c, lines 95-99)

The double close fails with `ERROR_INVALID_HANDLE`, and the page explains why this must stay a bug and not a habit: under a debugger the same call “will throw an exception if it receives either a handle value that is not valid or a pseudo-handle value”, and the freed entry’s value can be reused by an unrelated object at any time.

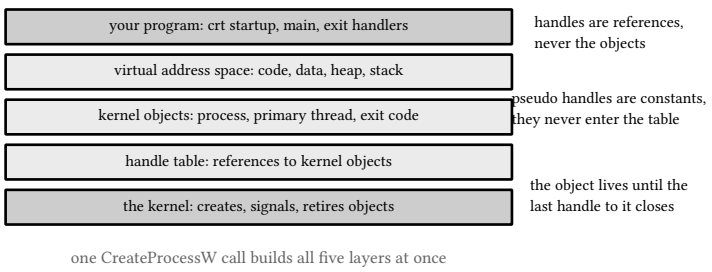


figure 41: one process as five layers, what each layer holds, and who owns it

The other half of the chapter title is the program lifecycle inside one process, and that is the C standard’s territory, not windows’. The final draft n3220 makes `main` special at 5.1.2.3.4: “a return from the initial call to the main

function is equivalent to calling the exit function with the value returned by the main function as its argument; reaching the } that terminates the main function returns a value of 0." What `exit` then does is enumerated at 7.24.4.4: "First, all functions registered by the atexit function are called, in the reverse order of their registration", then "all open streams with unwritten buffered data are flushed, all open streams are closed, and all files created by the tmpfile function are removed." The order matters to the gate itself: a handler's output reaches the captured stdout only because the flush happens after the handlers, not before them.

```c
// the exit time recorder: three quiet markers plus one verifier. the
// verifier is registered first, so it runs last, after every marker had
// its chance. a failing verification must still fail the gate, and by
// then main has already returned, so it flushes stdout and calls _Exit(1)
static char marks[8];
static size_t nmarks;

static void mark(char tag) {
  marks[nmarks++] = tag;
  marks[nmarks] = 0;
}

static void mark_c(void) { mark('c'); }
static void mark_b(void) { mark('b'); }
static void mark_a(void) { mark('a'); }

// registered by at_quick_exit, which exit never calls: if it ever ran it
// would leave a 'Q' in marks and the verifier would fail
static void quick_mark(void) { mark('Q'); }

static void verify_exit_order(void) {
  if (strcmp(marks, "abc") == 0) {
    checks++;
    printf("ok %d exit ran a, b, c in reverse order, no quick handler\n",
           checks);
  } else {
    printf("FAIL exit order got %s (check %d)\n", marks, checks + 1);
    fflush(stdout);
    _Exit(1);
  }
}
```

listing 77: the exit time recorder, the verifier that runs last, and the quick exit decoy (source: c-os-cloud/samples/src/Ch11/lifecycle.c, lines 24-54)

```c
  // the standard's exit vocabulary, 7.24.4.4
  CHECK(EXIT_SUCCESS == 0 && EXIT_FAILURE == 1,
        "EXIT_SUCCESS is 0 and EXIT_FAILURE is 1");

  // registration returns 0 on success: atexit 7.24.4.2, at_quick_exit
  // 7.24.4.3. the verifier goes in first so it comes out last
  CHECK(atexit(verify_exit_order) == 0, "verifier registers");
  CHECK(atexit(mark_c) == 0 && atexit(mark_b) == 0 && atexit(mark_a) == 0,
        "three markers register in order c, b, a");
  CHECK(at_quick_exit(quick_mark) == 0, "quick exit marker registers");
```

listing 78: exit vocabulary and the four registrations (source: c-os-cloud/samples/src/Ch11/lifecycle.c, lines 101-110)

The design problem in the listing is that a check which runs after `main` returned cannot fail the run by returning nonzero: the exit code is already settled. The verifier therefore flushes and calls `_Exit(1)` on the failure path, which 7.24.4.5 permits, no handlers, immediate termination, while calling `exit` from inside `exit` is undefined by 7.24.4.4

outright. The decoy `quick_mark` is registered with `at_quick_exit`, and the verifier's assertion is the exact string `abc`: reverse order for the three markers, and no `Q`, because `exit` "causes normal program termination" while "No functions registered by the at_quick_exit function are called." The quick path itself is the asymmetric sibling: `quick_exit` at 7.24.4.7 runs its own registrations in reverse order and then "control is returned to the host environment by means of the function call _Exit(status)", and whether streams get flushed on that path is implementation-defined. One process gets one termination, so the sample executes the `exit` side and cites the rest.

11.2. CreateProcess end to end

The call takes ten parameters, and the sample names every one of them. `STARTUPINFOW` travels in, with `cb` set to its own size so the kernel knows which revision it received. `PROCESS_INFORMATION` travels out, 24 bytes on x64, probed: two handles and two identifiers, and the page is explicit about ownership: "Handles in PROCESS_INFORMATION must be closed with CloseHandle when they are no longer needed."

PITFALL: the command line must live in writable memory

The CreateProcessW page states it: the Unicode version "can modify the contents of this string. Therefore, this parameter cannot be a pointer to read-only memory (such as a const variable or a literal string). If this parameter is a constant string, the function may cause an access violation." A wide string literal lands in read-only data, so both samples build the command line with `swprintf` into a stack array. The same page's security remarks tell the second half: when `lpApplicationName` is NULL and the path contains spaces, quote it, or the parser may resolve a different executable than you meant.

```
// the wide self path: CreateProcessW wants UTF-16, argv came in narrow
wchar_t self[MAX_PATH * 2];
int wide = MultiByteToWideChar(CP_ACP, 0, argv[0], -1, self, MAX_PATH * 2);
CHECK(wide > 0, "argv[0] converts to a wide path for the self spawn");

// cmd.exe by absolute path, no PATH search ambiguity
wchar_t cmd[MAX_PATH];
int expanded = ExpandEnvironmentStringsW(L"%SystemRoot%\\System32\\cmd.exe",
                                         cmd, MAX_PATH);
CHECK(expanded > 0 && expanded ≤ MAX_PATH,
      "%SystemRoot% expands to the cmd.exe path");
```

listing 79: the wide self path and the absolute cmd.exe path (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 61-71)

CreateProcessW speaks UTF-16, so the self-spawn path comes from converting `argv[0]` with `MultiByteToWideChar`; on the gate's ASCII paths the conversion returns exactly the narrow length plus the terminator, probed. The cmd.exe path avoids the documented six-step executable search altogether by expanding `%SystemRoot%` to an absolute path, quoted because the system directory is allowed to move under paths with spaces.

```c
// first spawn, with the parent's handle table audited around it
STARTUPINFOW si;
PROCESS_INFORMATION pi;
wchar_t cl[MAX_PATH * 2 + 64];
swprintf(cl, MAX_PATH * 2 + 64, L"\"%ls\" /c exit 7", cmd);
ZeroMemory(&si, sizeof si);
si.cb = sizeof si;
ZeroMemory(&pi, sizeof pi);
ULONG before = handle_count();
CHECK(CreateProcessW(NULL, cl, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi),
      "cmd.exe /c exit 7 spawns");
ULONG after = handle_count();
CHECK(pi.dwProcessId ≠ 0 && pi.dwProcessId ≠ GetCurrentProcessId(),
      "the child pid is distinct and nonzero");
CHECK(after ≥ before + 2,
      "createprocess adds at least the process and thread handles");
CHECK(WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_OBJECT_0,
      "wait returns WAIT_OBJECT_0 once the child exits");
DWORD code = 0;
CHECK(GetExitCodeProcess(pi.hProcess, &code) && code == 7,
      "the exit code 7 round trips through the kernel");
CHECK(CloseHandle(pi.hProcess) && CloseHandle(pi.hThread),
      "both returned handles close");
CHECK(handle_count() == after - 2,
      "closing both removes exactly two handles");
```

listing 80: first spawn end to end with the handle table audited (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 73-97)

Line by line: the structures are zeroed, `cb` is set, the parent's handle count is taken, and `CreateProcessW` runs with `bInheritHandles` FALSE, so the child starts with a clean table, the pseudo handle rule above working in the parent's favor. The function returns true before the child has done anything: "the function returns before the process has finished initialization." The pid check ties to the page's remark that "The process is assigned a process identifier. The identifier is valid until the process terminates." The wait blocks on the process handle until it is signaled, `WaitForSingleObject` returning `WAIT_OBJECT_0`, and `GetExitCodeProcess` reads back the 7 that `cmd.exe /c exit 7` wrote. Then both handles close.

The handle audit around the spawn is deliberately asymmetric, because the probe found an asymmetry. The create adds at least the process and thread handles, asserted as a lower bound, while the close side is exact, the table shrinks by exactly 2. Probed 2026-09-12: the first spawn in a process also opens console plumbing in the parent, 5 handles that stay open for the process lifetime, so the create delta was 7, not 2, and 5 remained after both closes. On the fifth spawn of the same process the delta was exactly 2 back to baseline. The robust pair, at least plus 2 on create and exactly minus 2 on close, held in both legs, plain and address-sanitizer build.

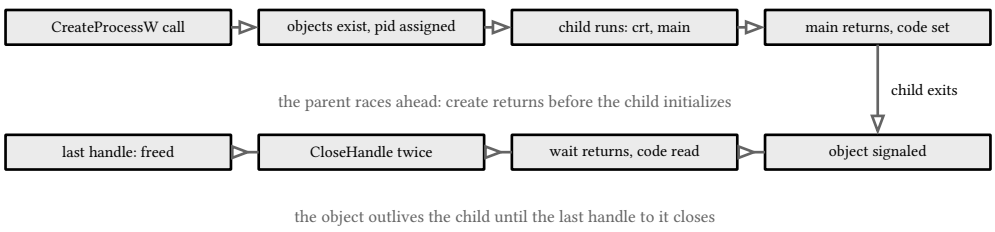


figure 42: a child from creation to object retirement, the calls that drive each step

11.3. fork/exec versus CreateProcess

The posix model this section compares against is two system calls. POSIX `fork` "shall create a new process", and "The new process (child process) shall be an exact copy of the calling process (parent process) except as detailed

below." One call, two returns: "Upon successful completion, fork() shall return 0 to the child process and shall return the process ID of the child process to the parent process." The copy is a fiction the kernel maintains cheaply; the linux man page states the mechanism: "Under Linux, fork() is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables and to create a unique task structure for the child." Then `execve` replaces the image: it causes "the program that is currently being run by the calling process to be replaced with a new program", with "newly initialized stack, heap, and (initialized and uninitialized) data segments", while "many attributes of the calling process remain unchanged (in particular, its PID)", file descriptors stay open unless marked close-on-exec, and "On success, execve() does not return."

Windows has neither call, and on this box that is a probed fact, not a porting complaint: compiled through the same flags as the gate, a bare `fork()` call dies with `use of undeclared identifier 'fork'`, no header in the msvc crt or the windows sdk declares it, and no import library resolves it. The one-step model does the same work differently. `CreateProcessW` builds the new process and its primary thread directly from an executable file, no parental copy, no image replacement, a fresh private address space, and hands the parent two handles instead of one doubled control flow. What fork-plus-exec buys that plain creation seems to lose, the window between the child existing and the child running, exists here as a flag: "The primary thread of the new process is created in a suspended state, and does not run until the ResumeThread function is called."

```
// the self spawn, created suspended: the parent holds the child at the
// starting line, reads its pending status, then lets it run
swprintf(cl, MAX_PATH * 2 + 64, L"\"%ls\" child 42", self);
ZeroMemory(&si, sizeof si);
si.cb = sizeof si;
ZeroMemory(&pi, sizeof pi);
CHECK(CreateProcessW(NULL, cl, NULL, NULL, FALSE, CREATE_SUSPENDED, NULL,
                     NULL, &si, &pi),
      "the self spawns as a suspended child");
CHECK(GetExitCodeProcess(pi.hProcess, &code) && code == STILL_ACTIVE,
      "a suspended child reports STILL_ACTIVE 259");
CHECK(WaitForSingleObject(pi.hProcess, 0) == WAIT_TIMEOUT,
      "a zero timeout wait reports the child not signaled");
CHECK(ResumeThread(pi.hThread) == 1,
      "resumethread returns the previous suspend count 1");
CHECK(WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_OBJECT_0,
      "the resumed child reaches the signaled state");
CHECK(GetExitCodeProcess(pi.hProcess, &code) && code == 42,
      "the resumed child exits with the negotiated 42");
CHECK(CloseHandle(pi.hProcess) && CloseHandle(pi.hThread),
      "the self spawn handles close");
```

listing 81: the suspended self spawn, create frozen, resume, reap (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 109-129)

The sample spawns a copy of itself with `CREATE_SUSPENDED` and negotiates the exit code on the command line. While the child hangs suspended, `GetExitCodeProcess` reports `STILL_ACTIVE` and a zero timeout wait returns `WAIT_TIMEOUT`, both asserted, so the child provably exists and provably has not run. `ResumeThread` returns the previous suspend count, 1, matching the page's reading: "If the return value is 1, the specified thread was suspended but was restarted." Then the wait, the code 42, the closes. Create frozen, start on command, observe, reap: the same four verbs as fork, exec, waitpid, close, with the copy step deleted.

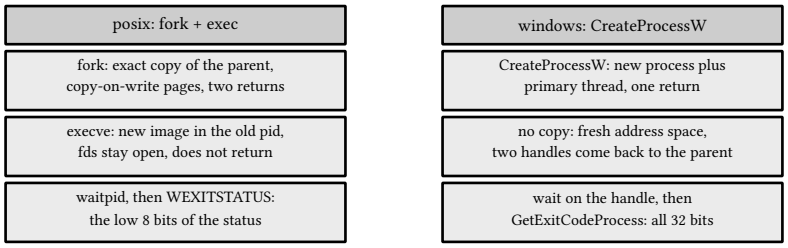


figure 43: two creation models, the same parent verbs at the bottom

11.4. waiting, exit codes, cleanup

`WaitForSingleObject` is the parent's only blocking verb, and its page fixes the vocabulary the checks use. A zero timeout "does not enter a wait state if the object is not signaled; it always returns immediately", `INFINITE` "will return only when the object is signaled", and the return values are constants: `WAIT_OBJECT_0`, 0x00000000, the object is signaled; `WAIT_TIMEOUT`, 0x00000102, 258, the interval elapsed; `WAIT_FAILED` for an error. Process handles are in the waitable list, and one caution belongs with the cleanup theme of this section: "If this handle is closed while the wait is still pending, the function's behavior is undefined."

```
// one cmd.exe round trip: spawn "cmd /c exit <code>", wait for it, harvest
// the exit code, close both handles the create handed back
static BOOL run_cmd_exit(const wchar_t *cmd, int code, DWORD *out) {
  STARTUPINFOW si;
  PROCESS_INFORMATION pi;
  wchar_t cl[MAX_PATH + 64];
  swprintf(cl, MAX_PATH + 64, L"\"%ls\" /c exit %d", cmd, code);
  ZeroMemory(&si, sizeof si);
  si.cb = sizeof si;
  ZeroMemory(&pi, sizeof pi);
  if (!CreateProcessW(NULL, cl, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi))
    return FALSE;
  WaitForSingleObject(pi.hProcess, INFINITE);
  BOOL ok = GetExitCodeProcess(pi.hProcess, out);
  CloseHandle(pi.hProcess);
  CloseHandle(pi.hThread);
  return ok;
}
```

listing 82: one cmd.exe round trip, closes on every path (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 32-49)

```
int main(int argc, char **argv) {
  // the child role: print one role line, no "ok" prefix, exit with the
  // negotiated code. the parent asserts that code through the kernel
  if (argc > 1 && strcmp(argv[1], "child") == 0) {
    long code = argc > 2 ? strtol(argv[2], NULL, 10) : 7;
    printf("child: role line, exiting %ld\n", code);
    fflush(stdout);
    return (int)code;
  }
```

listing 83: the child role, one role line and a negotiated exit code (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 51-59)

The helper exists so the width probes read as one-liners, and its shape is the chapter's cleanup rule in miniature: both handles close on every path, including the failure path. The child role prints one line with no `ok` prefix, so the gate's check count stays the parent's 19, and exits with the negotiated code, the only child fact the parent

asserts through the kernel. `GetExitCodeProcess` defines what comes back: “If the process has not terminated and the function succeeds, the status returned is STILL_ACTIVE (a macro for STATUS_PENDING)”, and after termination the status is “The return value from the main or WinMain function of the process”, or the value given to `ExitProcess`, or the exception value of a fatal crash.

```
// windows exit codes are 32 bits wide, no posix 8 bit truncation
DWORD harvested = 0;
CHECK(run_cmd_exit(cmd, 300, &harvested) && harvested == 300,
      "exit 300 arrives as 300, not truncated to 8 bits");
CHECK(run_cmd_exit(cmd, -1, &harvested) && harvested == 4294967295u,
      "exit -1 arrives as 4294967295: all 32 bits carried");
CHECK(run_cmd_exit(cmd, 259, &harvested) && harvested == 259 &&
          259 == STILL_ACTIVE,
      "exit 259 is indistinguishable from STILL_ACTIVE");
```

listing 84: 32 bit exit codes and the 259 collision (source: c-os-cloud/samples/src/Ch11/spawn.c, lines 99-107)

The three probes pin the width and one trap. `exit 300` arrives as 300, where the posix parent would fish the same child status out with `WEXITSTATUS`, which “consists of the least significant 8 bits”, 44 for 300. `exit -1` arrives as 4294967295, every bit carried. And `exit 259` arrives as 259, indistinguishable from `STILL_ACTIVE`, which is why the page’s warning exists: “an application should not use STILL_ACTIVE (259) as an error code”, because a caller polling for it “could interpret it to mean that the thread is still running… which could put the application into an infinite loop.” The correct termination test is the signaled wait, never the code value, and the suspended-self-spawn checks earlier in the sample are exactly that discipline executed.

What termination itself does is documented as a six-step sequence: remaining threads are marked, the process’s resources are freed, all its kernel objects are closed, its code is removed from memory, the exit code is set, and the process object is signaled. Two sentences from the same page carry the cleanup half of this chapter: “While open handles to kernel objects are closed automatically when a process terminates, the objects themselves exist until all open handles to them are closed”, and “When a process terminates, the state of the process object becomes signaled, releasing any threads that had been waiting for the process to terminate.” The child dying therefore costs the parent nothing until the parent lets go, and killing a parent costs the children nothing at all: “the system… does not terminate any child processes that the process has created.”

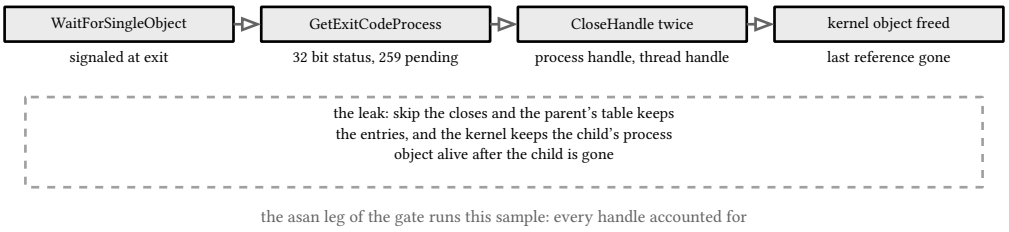


figure 44: the reap pipeline, wait, read, close, and what a skipped close keeps alive

The gate’s own enforcement of the cleanup rule is the address-sanitizer leg introduced in chapter 5, which recompiles and runs `spawn.c` instrumented. The sample allocates no heap memory, so the leg’s real subject here is discipline: the self spawn’s child runs the same instrumented binary and finds the staged runtime through the inherited working directory, and the parent’s handle accounting stays exact under instrumentation, probed. The address space half of the process, the second layer of the opening figure, is chapter 12′s subject, and the thread object that `CreateProcessW` builds beside the process is chapter 15′s.

sources: learn.microsoft.com, CreateProcessW, GetExitCodeProcess, WaitForSingleObject, CloseHandle, GetCurrentProcess, GetCurrentThread, ResumeThread, GetProcessHandleCount, DuplicateHandle, GetStartupInfoW, Process Creation Flags, and Terminating a Process pages, accessed 2026-09-12; pubs.opengroup.org Issue 8 fork and exec pages and man7.org fork(2), execve(2), wait(2), accessed 2026-09-12; open-std.org n3220 (5.1.2.3.4 program termination, 7.24.4.2 atexit, 7.24.4.3 `at_quick_exit`, 7.24.4.4 exit, 7.24.4.5 `_Exit`, 7.24.4.7 `quick_exit`), accessed 2026-09-12; the first-spawn handle delta, the −2 thread pseudo handle, the 104 and 24 byte structures, the fork link

failure, and the asan handle accounting probed on this machine the same day. Sample behavior verified by `make verify-c`, 33 checks in chapter 11 of the samples suite.

12. virtual memory and paging

Chapter 11 built the container and named its second layer a private virtual address space. This chapter opens that layer. Every address a C pointer holds is virtual, the number means nothing to dram, and the system rewrites it on every single memory reference. The plan is four moves: the mapping fiction that makes one address mean different memory in different processes, a two-level page table and tlb simulated in plain c23 with every translation hand computed first, the real allocator interface `VirtualAlloc` with its staged reserve and commit asserted through `VirtualQuery`, and the fault counter that prices the difference between promising memory and touching it. The posix counterpart `mmap(2)` sits beside the windows material as citation, never execution: no mmap call runs on this machine in this book. Every behavioral claim below is one of the 47 checks in the two samples or a sentence quoted from a canonical page fetched 2026-09-12.

12.1. the mapping fiction

The Memory Management conceptual pages state the fiction in two sentences. “A virtual address does not represent the actual physical location of an object in memory; instead, the system maintains a page table for each process, which is an internal data structure used to translate virtual addresses into their corresponding physical addresses. Each time a thread references an address, the system translates the virtual address to a physical address.” The privacy rule is the same page’s opening line: “The address space for each process is private and cannot be accessed by other processes unless it is shared.” On this machine’s x64 target that private space is enormous: the Memory Limits page gives each 64-bit process 128 TB of user-mode virtual address space with `LARGEADDRESSAWARE` set, the default, on Windows 8.1 and later.

The unit of translation is the page. `GetSystemInfo` reports the two sizes this chapter needs, and `SYSTEM_INFO` documents both fields: `dwPageSize` is “The page size and the granularity of page protection and commitment. This is the page size used by the VirtualAlloc function”, and `dwAllocationGranularity` is “The granularity for the starting address at which virtual memory can be allocated.” The sample asserts 4096 and 65536 on this box, probed 2026-09-12. Translation cuts every address at the page boundary: the low bits of a page never enter a table, only the page number does.

The simulator shrinks that split to fit in one head, 16 bits instead of 48, pages of 256 bytes instead of 4096:

```
// the address split, with the worked example the whole file follows:
//   va 0x1234 = 0001 0010 0011 0100b
//   level-1 index 0x1 | level-2 index 0x2 | offset 0x34
// if page 0x1200 maps to frame 0x0A, then va 0x1234 -> pa 0x0A34:
// the offset 0x34 rides through untouched, only the page number moves
#define PAGE_BYTES 256u
#define LEVEL1_ENTRIES 16
#define LEVEL2_ENTRIES 16
#define PTE_PRESENT 0x80u
#define TLB_SLOTS 4
```

listing 85: the toy address split: 4 bits of level-1 index, 4 of level-2, 8 of offset (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 21-30)

The worked example runs through the whole chapter. Virtual address `0x1234` is `0001 0010 0011 0100` binary: level-1 index `0x1`, level-2 index `0x2`, offset `0x34`. If page `0x1200` maps to physical frame `0x0A`, then `0x1234` translates to `0x0A34`. The offset `0x34` rides through untouched, and only the page number moves:

```
// the split, hand computed: 0x1234 -> level-1 0x1, level-2 0x2
map(&m, 0x1200, 0x0A); // page 0x12 -> frame 0x0A
map(&m, 0x3400, 0x05); // page 0x34 -> frame 0x05
CHECK(translate(&m, probe, 0x1234, &pa) == FILL && pa == 0x0A34,
      "va 0x1234 -> pa 0x0A34, the walk fills");
CHECK(translate(&m, probe, 0x1244, &pa) == HIT && pa == 0x0A44,
      "same page again hits, offset 0x44 rides through");
CHECK((pa & 0xFFu) == 0x44, "the low 8 bits never translate");
CHECK(translate(&m, probe, 0x34F0, &pa) == FILL && pa == 0x05F0,
      "va 0x34F0 -> pa 0x05F0");
pa = 0xFFFF;
CHECK(translate(&m, probe, 0x9999, &pa) == FAULT && pa == 0xFFFF,
      "level-1 entry empty: 0x9999 faults, no address written");
map(&m, 0x2100, 0x07);
unmap(&m, 0x2100);
pa = 0xFFFF;
CHECK(translate(&m, probe, 0x2110, &pa) == FAULT && pa == 0xFFFF,
      "present bit clear: the fault a demand-paged page starts from");
```

listing 86: the hand-computed translations, both fault flavors, the untouched offset (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 97-114)

Four checks walk the arithmetic, then the two negative ones mark where translation stops. An address whose level-1 entry was never mapped faults. An address whose entry exists but has its present bit cleared faults too, the level-2 table stands while its entry says no page, and that is precisely the state a demand-paged page starts in. Both fault checks assert the stronger fact that no address is delivered at all: the output variable keeps the `0xFFFF` sentinel the caller planted.

The fiction's sharpest form is one address in two processes. Two `mmu` instances, two private page tables, the same virtual address, two different frames. The privacy is what makes identical binaries cheap to load: a module asks for its preferred base in a fresh process without consulting any other process, because there is nothing to consult, every table answers only for its owner. It cuts the other way too: no process reaches another one's memory by guessing an address, the number means nothing without the tables that translate it.

```
// the fiction, one address two processes: same va, different tables,
// different frames. translation is per process, the pa is not
mmu a = {0}, b = {0};
tlb_slot ta[TLB_SLOTS] = {0}, tb[TLB_SLOTS] = {0};
map(&a, 0x1200, 0x0A);
map(&b, 0x1200, 0x03);
unsigned short pa_a = 0, pa_b = 0;
translate(&a, ta, 0x1234, &pa_a);
translate(&b, tb, 0x1234, &pa_b);
CHECK(pa_a == 0x0A34 && pa_b == 0x0334,
      "same va 0x1234: process a gets 0x0A34, b gets 0x0334");
CHECK(pa_a != pa_b, "two processes, one virtual address, two frames");
```

listing 87: the same va 0x1234, two processes, two physical frames (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 116-127)

Process a reads `0x0A34`, process b reads `0x0334` from the identical pointer value, and the check that matters is `pa_a != pa_b`. This is why every process in chapter 11 could load at the same preferred image base without colliding, and why two processes can share a physical page only by the kernel deliberately mapping the same frame into both tables.

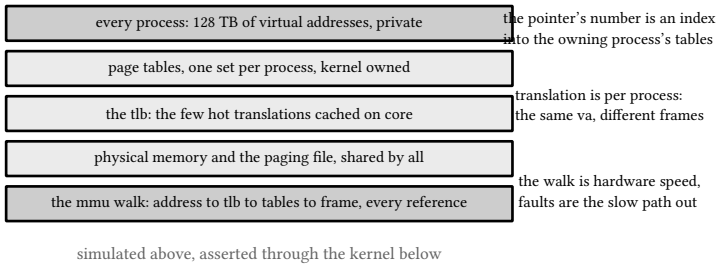


figure 45: from pointer to dram: what each layer owns and who sees it

12.2. page tables and tlbs, simulated

Why two levels instead of one flat table: the address space is sparse. A flat 16-bit space needs all 256 table entries whether the process maps 4 pages or 200, and a flat 48-bit space would need half a terabyte per process. The two-level trick puts a `NULL` where whole subtrees are unmapped, 15 of the 16 possible first-level entries in the sample's case, and the walk stops right there.

```
static void map(mmu *m, unsigned short page, unsigned char frame) {
  unsigned short vpn = page >> 8;
  if (m->dir[vpn >> 4] == NULL)
    m->dir[vpn >> 4] = &tables[ntables++];
  m->dir[vpn >> 4]->pte[vpn & 0xFu] = (unsigned char)(PTE_PRESENT | frame);
}

// clears the present bit but leaves the level-2 table standing: the
// state the kernel leaves behind when it unmaps, and the state a fresh
// committed-but-untouched page would show if hardware let us see it
static void unmap(mmu *m, unsigned short page) {
  unsigned short vpn = page >> 8;
  if (m->dir[vpn >> 4] != NULL)
    m->dir[vpn >> 4]->pte[vpn & 0xFu] = 0;
}
```

listing 88: map writes a present pte, unmap clears the bit and leaves the table standing (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 52-66)

`map` lazily creates the level-2 table the first time an address inside it is mapped, which is what a real kernel does when a process first maps into a fresh 2 MiB extent. `unmap` clears the present bit only, and the fault check two listings back uses exactly the state it leaves: table present, entry absent. That is the state hardware hands the kernel on a demand paging fault, which is where section four picks the story up.

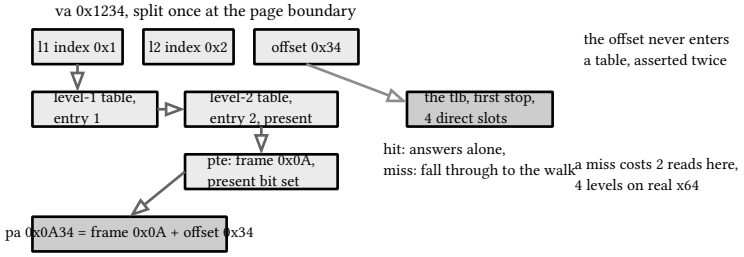


figure 46: va 0x1234 through the machine: the split, the tlb, both levels, the frame

The walk order is the whole point. The tlb is checked first and answers without touching a single table, the hit path returns in three lines. A miss pays two table reads in this toy, one per level, both from memory, both counted. A faulting lookup pays the same two reads and delivers nothing: `*pa` is only written on the success paths, so the caller's sentinel survives the fault. The fill path installs the translation it just walked so the next reference to the page hits:

```
// one translation. tlb first, then the two-level walk. writes *pa only
// on success: a faulting access delivers no address at all
static int walks; // table reads charged to misses, 2 per walk

static int translate(mmu *m, tlb_slot *tlb, unsigned short va,
                     unsigned short *pa) {
  unsigned short vpn = va >> 8;
  unsigned char off = (unsigned char)(va & 0xFFu);
  int slot = vpn & (TLB_SLOTS - 1); // direct mapped
  if (tlb[slot].valid && tlb[slot].vpn == vpn) {
    *pa = (unsigned short)((unsigned short)tlb[slot].frame << 8) | off;
    return HIT;
  }
  walks += 2; // the real machine reads one entry per level, from memory
  level2 *l2 = m->dir[vpn >> 4];
  if (l2 == NULL || !(l2->pte[vpn & 0xFu] & PTE_PRESENT))
    return FAULT;
  tlb[slot].valid = 1;
  tlb[slot].vpn = vpn;
  tlb[slot].frame = (unsigned char)(l2->pte[vpn & 0xFu] & 0x0Fu);
  *pa = (unsigned short)((unsigned short)tlb[slot].frame << 8) | off;
  return FILL;
}
```

listing 89: one translation: tlb first, then the two-level walk, never both (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 68-90)

The fixed trace runs seven accesses and every verdict was computed by hand before the loop ran:

```
  // the fixed trace, hand classified before the loop runs it:
  //   access   vpn   slot   verdict
  //   0x1234   0x12   2     FILL
  //   0x1234   0x12   2     HIT
  //   0x34F0   0x34   0     FILL
  //   0x1299   0x12   2     HIT
  //   0x8800   0x88   0     FILL, evicts vpn 0x34 from slot 0
  //   0x9999   0x99   1     FAULT, level-1 entry empty
  //   0x34F0   0x34   0     FILL again, the eviction turned a hit cold
  // totals: 4 fills, 2 hits, 1 fault, 10 table reads (2 per non-hit)
  static const struct {
    unsigned short va;
    int want;
    unsigned short want_pa; // ignored when want == FAULT
  } trace[] = {
      {0x1234, FILL, 0x0A34}, {0x1234, HIT, 0x0A34},  {0x34F0, FILL, 0x05F0},
      {0x1299, HIT, 0x0A99},  {0x8800, FILL, 0x0C00}, {0x9999, FAULT, 0xFFFF},
      {0x34F0, FILL, 0x05F0},
  };
```

listing 90: the trace table: va, expected verdict, expected pa, hand computed first (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 129-147)

Two rows of the table are teaching moments the loop then proves. Vpn `0x88` and vpn `0x34` share tlb slot 0, because the slot index is the vpn's low two bits and both values are multiples of 4, so the fifth access evicts the third's translation. The seventh access, a repeat of the third, then misses an address that was already cached once: a direct-mapped conflict turned a warm page cold. Real tlbs are set associative precisely to soften this, but the eviction math is the same arithmetic with a wider slot.

```
  map(&m, 0x8800, 0x0C); // page 0x88 -> frame 0x0C for the trace
  tlb_slot t[TLB_SLOTS] = {0};
  walks = 0; // count only the trace's own table reads
  int nhit = 0, nfill = 0, nfault = 0, nbad = 0;
  for (unsigned i = 0; i < sizeof trace / sizeof trace[0]; i++) {
    unsigned short got = 0xFFFF;
    int verdict = translate(&m, t, trace[i].va, &got);
    if (verdict != trace[i].want || got != trace[i].want_pa)
      nbad++;
    if (verdict == HIT)
      nhit++;
    else if (verdict == FILL)
      nfill++;
    else
      nfault++;
  }
  CHECK(nbad == 0, "every trace access matched its hand verdict and pa");
  CHECK(nhit == 2, "the trace scores exactly 2 tlb hits");
  CHECK(nfill == 4, "4 fills, one of them an eviction re-fill");
  CHECK(nfault == 1, "1 fault, the unmapped vpn 0x99");
  CHECK(walks == 10, "10 table reads served 4 fills and 1 fault, 2 each");
```

listing 91: the loop scores the trace: 4 fills, 2 hits, 1 fault, 10 table reads (source: c-os-cloud/samples/src/Ch12/pagetable.c, lines 148-168)

The counters come out exactly as the comment table predicts: 4 fills, 2 hits, 1 fault, and 10 table reads, two for each of the 5 non-hits. The two hits are the entire economic argument for the tlb: they skipped 4 table reads that the walk would otherwise have paid. On real hardware the same argument is louder in both directions, the walk is four levels deep on x64, and the tlb covers a handful of entries per core with a second larger shared level beside it.

NOTE: a model, not a cycle counter

The simulator gets the order of operations right, tlb, then tables, then fault, and the arithmetic exact, because the trace is fixed and the classification was done by hand. It simplifies everything else: real x64 page tables are four levels with 512 entries per level, the tlb is set associative and split across core levels, table reads go through their own paging-structure caches, and the pte carries dirty, accessed, and protection bits this model folds away. What transfers is the shape: the hit is cheap because it skips the walk, and the fault is expensive because it leaves the hardware entirely.

12.3. virtualalloc versus mmap

The windows allocator interface is one function with a state machine inside. `VirtualAlloc` “Reserves, commits, or changes the state of a region of pages in the virtual address space of the calling process”, and its two allocation flags are the two halves of the staging. `MEM_RESERVE` “Reserves a range of the process’s virtual address space without allocating any actual physical storage in memory or in the paging file on disk.” `MEM_COMMIT` “Allocates memory charges (from the overall size of memory and the paging files on disk) for the specified reserved memory pages”, and the same paragraph carries the promise this chapter keeps returning to: “Actual physical pages are not allocated unless/until the virtual addresses are actually accessed.” The remarks section names the pattern directly: “You can use VirtualAlloc to reserve a block of pages and then make additional calls to VirtualAlloc to commit individual pages from the reserved block. This enables a process to reserve a range of its virtual address space without consuming physical storage until it is needed.”

The sample reads every step back through `VirtualQuery`, whose contract is a region definition: it reports “a region of consecutive pages beginning at a specified address” sharing state, original allocation, and protection, stopping at the first page that disagrees. The two size probes come first:

```
static PROCESS_MEMORY_COUNTERS snap(void) {
  PROCESS_MEMORY_COUNTERS pmc;
  GetProcessMemoryInfo(GetCurrentProcess(), &pmc, sizeof pmc);
  return pmc;
}

static SIZE_T faults(void) { return snap().PageFaultCount; }

// PagefileUsage is the process commit charge in bytes
static SIZE_T charge(void) { return snap().PagefileUsage; }

static MEMORY_BASIC_INFORMATION query(const void *p) {
  MEMORY_BASIC_INFORMATION mbi;
  VirtualQuery(p, &mbi, sizeof mbi);
  return mbi;
}

int main(void) {
  SYSTEM_INFO si;
  GetSystemInfo(&si);
  CHECK(si.dwPageSize == 4096, "page size is 4096 on this x64 box (probed)");
  CHECK(si.dwAllocationGranularity == 65536,
        "allocation granularity is 64 KiB (probed)");
```

listing 92: the readback helpers: fault counter, commit charge, page states, and the two granularities (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 29-51)

The `PSAPI_VERSION 2` define is a documented remap, worth one sentence because it explains why the gate's `-lkernel32` is enough: "If PSAPI_VERSION is 2 or greater, this function is defined as K32GetProcessMemoryInfo in Psapi.h and exported in Kernel32.lib and Kernel32.dll." `PagefileUsage` is the commit charge, "The Commit Charge value in bytes for this process", the meter that shows reserve and commit are different events.

```
  // stage 1: reserve five pages. the probe result this pins: the size
  // is exactly dwSize rounded up to pages, while the 64 KiB allocation
  // granularity constrains only where the base lands
  const SIZE_T page = si.dwPageSize;
  const SIZE_T region = 5 * page;
  unsigned char *base =
      (unsigned char *)VirtualAlloc(NULL, region, MEM_RESERVE, PAGE_NOACCESS);
  CHECK(base != NULL, "VirtualAlloc reserves");
  CHECK((uintptr_t)base % si.dwAllocationGranularity == 0,
        "the reserved base is 64 KiB aligned");
  MEMORY_BASIC_INFORMATION mbi = query(base);
  CHECK(mbi.State == MEM_RESERVE, "reserved pages read back as MEM_RESERVE");
  CHECK(mbi.Type == MEM_PRIVATE, "the region is MEM_PRIVATE");
  CHECK(mbi.AllocationBase == base, "AllocationBase is the reserve base");
  CHECK(mbi.AllocationProtect == PAGE_NOACCESS,
        "AllocationProtect is the reserve-time protection");
  CHECK(mbi.RegionSize == region,
        "reserving 20480 bytes shows RegionSize 20480, not 64 KiB (probed)");
  CHECK(charge() == charge0, "reserving adds no commit charge");
```

listing 93: stage 1, reserve five pages: exact size, 64 KiB aligned base, zero charge (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 56-74)

Two of these checks corrected the draft and are labeled probed for it. The reservation comes back exactly 20480 bytes, `dwSize` rounded up to the page boundary and no further, while the 64 KiB allocation granularity governs only where the base lands, `base mod 65536 == 0`. The draft assumed reservations snap to whole 64 KiB units. They

do not, and neither do the docs say they should: the rounding rule is “rounded up to the next page boundary”, the granularity rule is about addresses. The second surprise waits in stage eight. The commit charge check is the documented one: reserve charges nothing, the page tables behind it cost kernel memory but no promise of backing storage.

```
// stage 2: commit one page in the middle of the reservation
unsigned char *p = (unsigned char *)VirtualAlloc(base + page, page,
                                                 MEM_COMMIT, PAGE_READWRITE);
CHECK(p == base + page, "the second page commits at base + 4096");
mbi = query(base);
CHECK(mbi.State == MEM_RESERVE && mbi.RegionSize == page,
      "the region splits: page 1 alone is reserved, 4096 bytes");
mbi = query(p);
CHECK(mbi.State == MEM_COMMIT && mbi.Protect == PAGE_READWRITE,
      "page 2 alone is committed readwrite");
CHECK(mbi.RegionSize == page, "the committed run is exactly one page");
CHECK(mbi.AllocationBase == base,
      "both halves still point at the one reserve");
CHECK(charge() - charge0 ≥ page, "committing one page charges one page");
```

listing 94: stage 2, one committed page splits the region in two (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 76-89)

Committing page two of five splits what was one region into two answers. The query at the base now reports `MEM_RESERVE` for exactly 4096 bytes, one page, because page two no longer matches its state. The query one page over reports `MEM_COMMIT`, `PAGE_READWRITE`, one page, and both halves still report the same `AllocationBase`, the original reserve. The `MEMORY_BASIC_INFORMATION` page documents the three states in exactly these words: committed pages are those “for which physical storage has been allocated, either in memory or in the paging file on disk”, reserved pages are “reserved without any physical storage being allocated”, and for reserved pages “the information in the Protect member is undefined”, which is why the sample never asserts `Protect` on a reserve.

The free side of the state machine is `VirtualFree`, and its trap is best read straight off the listing that springs it:

```
// stage 7: the release trap. dwSize must be 0 under MEM_RELEASE
BOOL bad = VirtualFree(base, region, MEM_RELEASE);
DWORD gle = GetLastError();
CHECK(!bad && gle == ERROR_INVALID_PARAMETER,
      "MEM_RELEASE with nonzero dwSize fails ERROR_INVALID_PARAMETER");
CHECK(VirtualFree(base, 0, MEM_RELEASE), "MEM_RELEASE with 0 releases all");
mbi = query(base);
// free runs merge with adjacent free address space, so RegionSize is
// the whole free run starting at base, never less than the pages we
// held (VirtualQuery's own remarks use a 40 MB free region example)
CHECK(mbi.State == MEM_FREE && mbi.RegionSize ≥ region,
      "the freed pages read MEM_FREE, a merged free run");
SIZE_T free_run = mbi.RegionSize;
```

listing 95: stage 7, the failed partial release, the free run that merges with its neighbors (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 126-139)

PITFALL: release owns the whole region or fails

The VirtualFree page is categorical: “If you specify this value, dwSize must be 0 (zero), and lpAddress must point to the base address returned by the VirtualAlloc function when the region is reserved. The function fails if either of these conditions is not met.” There is no partial release of one reservation, the page says elsewhere that “The entire range of pages originally reserved by the VirtualAlloc function must be released at the same time”, and committed pages ride along: “If any pages in the region are committed currently, the function first

> decommits, and then releases them." The sample asserts the failure, probed as `ERROR_INVALID_PARAMETER`, and then the release.

After the release the query reports `MEM_FREE` and a `RegionSize` larger than the reservation on every run observed, because free pages merge with adjacent free address space into one long run. The VirtualQuery remarks give this exact example: "if there is a 40 megabyte (MB) region of free memory, and VirtualQuery is called on a page that is 10 MB into the region, the function will obtain a state of MEM_FREE and a size of 30 MB." The check therefore asserts state and a lower bound, and prints the run it saw.

```
// stage 8: MEM_COMMIT | MEM_RESERVE in one call, the malloc shape.
// the probe: exactly one page is committed, the rest of the 64 KiB
// granularity unit stays free
unsigned char *one = (unsigned char *)VirtualAlloc(
    NULL, page, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
CHECK(one != NULL, "reserve and commit in one call");
mbi = query(one);
CHECK(mbi.State == MEM_COMMIT && mbi.RegionSize == page,
      "the one-shot commits exactly dwSize, one page (probed)");
mbi = query(one + page);
CHECK(mbi.State == MEM_FREE,
      "the rest of the granularity unit stays free (probed)");
```

listing 96: stage 8, the one-shot reserve plus commit of the malloc shape (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 141-153)

The one-shot `MEM_COMMIT | MEM_RESERVE` is what every higher allocator, `malloc` included, ultimately does for large blocks, and the probe result is the chapter's second surprise: committing one page this way commits exactly one page. The rest of the 64 KiB granularity unit the base sits in stays free, the query at `one + 4096` says so, and the reservation is one page deep. The fusion is total: there is no state between the reservation and the commitment, and the release side is one call with `dwSize` zero.

The posix counterpart is one call that fuses both stages by design. `mmap(2)` with `MAP_ANONYMOUS`: "The mapping is not backed by any file; its contents are initialized to zero." The commit discipline that windows splits into two calls is a kernel accounting question inside posix, surfaced only through the `MAP_NORESERVE` flag, "Do not reserve swap space for this mapping", the closest thing to refusing the promise. Alignment differs the same direction as the granularity findings: munmap "deletes the mappings for the specified address range" and "The address addr must be a multiple of the page size (but length need not be)", where windows requires the release base to be the allocation base and rounds nothing on the way out. None of it runs here: the posix pages are citations, the windows behavior is checked.

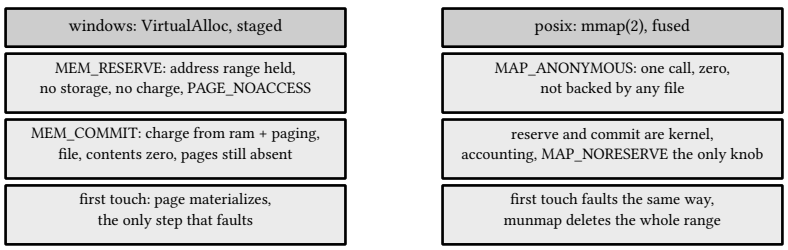


figure 47: the same ladder on two systems: windows names each rung, mmap fuses them

12.4. demand paging and fault cost

The Page State page compresses this section into one sentence: "The system initializes and loads each committed page into physical memory only during the first attempt to read or write to that page." Commit is a promise, first

touch is the bill, and the sample counts both sides with two different meters. The commit charge rises the moment `MEM_COMMIT` succeeds. The fault counter does not move at all until the store lands:

```
// stage 3: first touch is the bill. the store faults once, the load
// from the now-warm page does not
volatile unsigned char *vp = (volatile unsigned char *)p;
SIZE_T f = faults();
vp[0] = 0xAB;
SIZE_T touched = faults() - f;
CHECK(touched == 1, "the first store faults exactly once (probed exact)");
f = faults();
CHECK(vp[0] == 0xAB && faults() == f, "the warm page reads back, no fault");

// stage 4: commit three more pages, touch each once, count the bill
unsigned char *q = (unsigned char *)VirtualAlloc(base + 2 * page, 3 * page,
                                                 MEM_COMMIT, PAGE_READWRITE);
CHECK(q == base + 2 * page, "pages 3 through 5 commit");
f = faults();
for (unsigned i = 0; i < 3; i++)
  ((volatile unsigned char *)q)[i * page] = (unsigned char)i;
CHECK(faults() - f == 3, "three fresh pages, three faults, one each");
CHECK(charge() - charge0 ≥ 4 * page, "four pages charged in total");
```

listing 97: stages 3 and 4: one store one fault, warm pages free, three pages three faults (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 91-109)

`PageFaultCount` is documented in one flat line, “The number of page faults”, and the exact-delta checks hold on every run observed, 2026-09-12, three direct runs plus two gate legs. The first store faults exactly once. The read-back from the warm page faults zero times, the `volatile` qualifiers keep the compiler from collapsing either access into nothing, and the ir gate leg pins what survives optimization: at -O1 the first-touch store emits as `store atomic volatile`, volatile lowered to an atomic-order access on this target, and the substring assertion in `expect-ir.txt` holds it there. Three more committed pages touched once each cost exactly three faults. The commits themselves, seconds earlier in the same run, cost zero.

The decommit half of `VirtualFree` is the promise being withdrawn. Its page: “Decommits the specified region of committed pages. After the operation, the pages are in the reserved state”, and “Decommitting a page releases its physical storage, either in memory or in the paging file on disk.” The memory is gone with the promise, the same page’s warning that “After memory is released or decommited, you can never refer to the memory again”, and the sample relearns the price on purpose:

```
// stage 5: decommit the touched page, storage back to reserved
SIZE_T before = charge();
CHECK(VirtualFree(p, page, MEM_DECOMMIT), "MEM_DECOMMIT succeeds");
mbi = query(p);
CHECK(mbi.State == MEM_RESERVE, "the decommitted page reads MEM_RESERVE");
CHECK(charge() ≤ before - page, "decommit drops one page of charge");

// stage 6: recommit and touch: decommit reset the promise, the page
// faults again even though the address never changed
p = (unsigned char *)VirtualAlloc(p, page, MEM_COMMIT, PAGE_READWRITE);
CHECK(p ≠ NULL, "the reserved page recommits");
f = faults();
vp = (volatile unsigned char *)p;
vp[8] = 0xCD;
```

listing 98: stages 5 and 6: decommit returns storage, recommit buys the fault again (source: c-os-cloud/samples/src/Ch12/valloc.c, lines 111-124)

The address never changed. The decommitted page reads `MEM_RESERVE`, the charge drops a page, and the recommit of the same address starts the promise over from zero: the next store faults exactly once again, asserted. A decommitted page is not slow memory, it is absent memory, and the access violation sentences in the docs draw the hard edge: "Attempts to read from or write to a reserved page results in an access violation exception", and for free pages, "Attempting to read from or write to a free page results in an access violation exception." The book's rule from chapter 5 applies verbatim: the crash is cited, never executed.

NOTE: windows counts one number, posix splits it in two

`PageFaultCount` lumps every fault together. The posix accounting vocabulary is finer, and the man page for `getrusage(2)` defines the split: `ru_minflt` is "The number of page faults serviced without any I/O activity", pages reclaimed from memory, and `ru_majflt` is "The number of page faults serviced that required I/O activity", pages that had to come back from disk. Every fault this chapter's sample manufactures is minor, demand-zero pages materialized from nothing, no disk anywhere. The first major fault in a process is a different chapter's event: it is what happens when the working set loses an argument with physical memory.

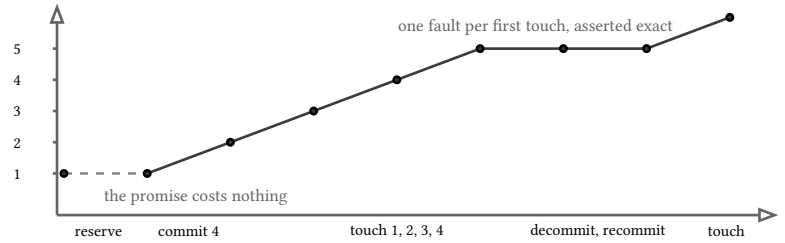


figure 48: cumulative faults across the sample's lifetime: flat through reserve and commit, one step per first touch

The working set, the pages currently resident in physical memory, reads the same story from the other side: `WorkingSetSize`, "The current working set size, in bytes", is printed by the sample's closing line, never asserted, because the process's other pages make it drift between runs. The fault counts do not drift, and that is why they carry the checks. Chapter 13 takes the next step down this same staircase: once a page is resident, the cost of touching it stops being a fault and becomes the cache line traffic the next chapter counts. The allocator that sits on top of this layer, the one `malloc` really calls, is chapter 14′s subject.

sources: learn.microsoft.com, VirtualAlloc, VirtualFree, VirtualQuery, `MEMORY_BASIC_INFORMATION`, GetSystemInfo and `SYSTEM_INFO`, GetProcessMemoryInfo and `PROCESS_MEMORY_COUNTERS`, Virtual Address Space, Virtual Address Space in 64-bit Windows, Memory Limits for Windows Releases, and Page State pages, accessed 2026-09-12; man7.org mmap(2) and getrusage(2), accessed 2026-09-12, cited only, never executed; the 4096 and 65536 granularities, the exact 5-page reservation, the one-page one-shot commit, the merged free run, the `ERROR_INVALID_PARAMETER` on partial release, and the exact per-touch fault deltas probed on this machine the same day, three direct runs plus two gate legs. Sample behavior verified by `make verify-c`, 47 checks in chapter 12 of the samples suite.

13. caches and locality

The caches sit between the code of the earlier chapters and the metal of this one, and nothing in the C standard mentions them. Everything this chapter claims comes from three places: the operating system, asked through `GetLogicalProcessorInformation` what caches this cpu actually declares; a simulator, whose hit, miss, and eviction counts are computed by hand first and asserted second; and the wall clock, printed and never checked, because timing is a machine fact and this book only asserts facts that cannot drift. The constant under all of it is the 64-byte line, the unit of traffic at every level, verified below by probe rather than assumed from folklore.

13.1. the hierarchy and line granularity

The probe walks the documented buffer protocol exactly. The first call passes `NULL` and a length of 0, fails, and leaves the required size in `ReturnedLength` with `GetLastError` reporting `ERROR_INSUFFICIENT_BUFFER`; the second call passes a buffer of that size and succeeds. The function page states the protocol in one breath: on failure the error is reported and the length is set to the buffer size required, and the struct page warns that the returned order may change between calls, so the walk treats the records as an unordered set. Each record carries a `Relationship`, and only `RelationCache` records matter here: their payload is a `CACHE_DESCRIPTOR` with `Level` (1, 2, 3 for L1 through L3), `Associativity` (`CACHE_FULLY_ASSOCIATIVE`, 0xff, marks a fully associative cache), `LineSize` documented as the cache line size in bytes, and `Size` in bytes. The struct page is candid about completeness: there is one record returned for each cache reported, some or all caches may not be reported depending on how the processor identifies them, so the absence of any particular cache is not a fact.

```
static int probe_caches(void) {
  DWORD len = 0;
  SYSTEM_LOGICAL_PROCESSOR_INFORMATION *buf;
  int cache_records = 0, line64 = 0;

  GetLogicalProcessorInformation(NULL, &len);
  CHECK(GetLastError() == ERROR_INSUFFICIENT_BUFFER,
        "glpi first call sizes the buffer by refusing");
  buf = malloc(len);
  CHECK(GetLogicalProcessorInformation(buf, &len), "glpi fills the buffer");
  for (DWORD off = 0; off + sizeof(SYSTEM_LOGICAL_PROCESSOR_INFORMATION) ≤ len;
       off += sizeof(SYSTEM_LOGICAL_PROCESSOR_INFORMATION)) {
    SYSTEM_LOGICAL_PROCESSOR_INFORMATION *r =
        (SYSTEM_LOGICAL_PROCESSOR_INFORMATION *)((BYTE *)buf + off);
    if (r->Relationship ≠ RelationCache)
      continue;
    CACHE_DESCRIPTOR cd = r->Cache;
    cache_records++;
    if (cd.LineSize == LINE)
      line64++;
```

listing 99: the probe: sizing by refusal, then one `CACHE_DESCRIPTOR` per `RelationCache` record (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 100-119)

On this machine the probe prints 34 cache records and they sketch a hybrid cpu. Eight P cores, each with a 48 KiB 12-way L1 data cache, a 32 KiB L1 code cache, and a private 1280 KiB 10-way L2, every record's mask pairing two logical processors. Four E cores with 32 KiB L1 data and 64 KiB L1 code, sharing one 2048 KiB 16-way L2 per cluster of four. One 25600 KiB 10-way L3 whose mask, 0xFFFFF, spans every logical processor the os knows. The type names printed beside each level, unified, data, code, are the `PROCESSOR_CACHE_TYPE` values. Two checks close the probe: at least three levels are reported, and all 34 records carry the same line size, 64 bytes. That number is the `LINE` constant the simulator runs on, and every later section of this chapter is a consequence of it.

The simulator is a set-associative model with least-recently-used replacement, small enough to read in one sitting. A cache is `sets * ways` slots, each holding a tag or −1, with a stamp per slot and a clock bumped on every access. One access takes an address, divides by 64 to get the line number, takes the set index as `line % sets` and the tag as `line / sets`, then scans the set:

```
static int cache_access(struct cache *c, unsigned long addr) {
  unsigned long line = addr / LINE;
  int set = (int)(line % (unsigned long)c->sets);
  long tag = (long)(line / (unsigned long)c->sets);
  int base = set * c->ways;
  int victim = -1;
  c->clock++;
  for (int w = 0; w < c->ways; w++) {
    if (c->tags[base + w] == tag) {
      c->stamp[base + w] = c->clock;
      c->hits++;
      return 1;
    }
    if (c->tags[base + w] < 0 && victim < 0)
      victim = w;
  }
```

listing 100: first half of one access: the set and tag arithmetic, the hit path, the empty-slot scan (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 56-71)

A hit refreshes the slot's stamp and returns. The scan also remembers the first empty slot it sees, which is what the second half uses:

```
  if (victim < 0) {
    victim = 0;
    for (int w = 1; w < c->ways; w++)
      if (c->stamp[base + w] < c->stamp[base + victim])
        victim = w;
    c->evictions++;
  }
  c->tags[base + victim] = tag;
  c->stamp[base + victim] = c->clock;
  c->misses++;
  return 0;
}
```

listing 101: second half: evict the least recently used slot when the set is full, fill, count the miss (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 72-83)

Every trace below runs on a fresh 4-set, 2-way cache: 8 lines, 512 bytes, small enough to hand-simulate. The first two traces pin the line granularity itself. Walking bytes 0 through 63 is one miss and 63 hits, because the whole range is one line; walking it twice is one miss and 127 hits, temporal reuse the simulator charges nothing for.

```
// trace 1: bytes 0..63, one line. hand count: 1 miss, 63 hits, 0 evict
for (int i = 0; i < 64; i++)
  one_line[i] = (unsigned long)i;
run_trace(&c, one_line, 64);
CHECK(c.misses == 1 && c.hits == 63 && c.evictions == 0,
      "64 bytes in one line: 1 miss, 63 free rides");

// trace 2: the same line twice. hand count: 1 miss, 127 hits
for (int i = 0; i < 128; i++)
  one_line_x2[i] = (unsigned long)(i % LINE);
run_trace(&c, one_line_x2, 128);
CHECK(c.misses == 1 && c.hits == 127 && c.evictions == 0,
      "second pass over the line is all hits");
```

listing 102: traces 1 and 2: one line walked once, then twice (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 148-160)

The third trace is the conflict miss: addresses 0, 512, and 1024 are lines 0, 8, and 16, and with 4 sets all three land in set 0. Two ways cannot hold three rotating lines, so every access misses, and the model counts 6 misses, 0 hits, and 4 evictions, the evictions being the fills that displaced a line which came back one access later.

```
// trace 3: three lines fighting for one set. addresses 0, 512, 1024
// are lines 0, 8, 16; with 4 sets every one lands in set 0, and 2
// ways cannot hold 3 rotating lines. hand count: 6 misses, 0 hits,
// 4 evictions
run_trace(&c, thrash, 6);
CHECK(c.misses == 6 && c.hits == 0 && c.evictions == 4,
      "3 rotating lines in a 2-way set: every access misses");
```

listing 103: trace 3: three rotating lines thrash the 2-way set (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 162-168)

The fourth trace is this chapter's thesis in miniature: the same 64 additions, two address sequences. `by_int` walks 64 consecutive ints inside 256 bytes, 4 lines, so 4 misses. `by_line` spreads the same count one int per line across 4096 bytes: 64 misses and 56 evictions. Nothing about the arithmetic changed. The stride is the whole story, and the next section scales it to a real matrix.

```
// trace 4: 64 additions two ways. by_int walks 64 ints inside 256
// bytes (4 lines); by_line walks 64 ints spread one per line across
// 4096 bytes. same additions, 4 versus 64 misses: the stride is the
// whole difference, which is the next sample's story
for (int i = 0; i < 64; i++) {
  by_int[i] = (unsigned long)i * 4;
  by_line[i] = (unsigned long)i * LINE;
}
run_trace(&c, by_int, 64);
CHECK(c.misses == 4 && c.hits == 60 && c.evictions == 0,
      "64 ints inside 4 lines: 4 misses");
run_trace(&c, by_line, 64);
CHECK(c.misses == 64 && c.hits == 0 && c.evictions == 56,
      "64 ints one per line: 64 misses, 56 evictions");
```

listing 104: trace 4: 64 additions as 4 misses or as 64, only the stride differs (source: c-os-cloud/samples/src/Ch13/cachesim.c, lines 170-183)

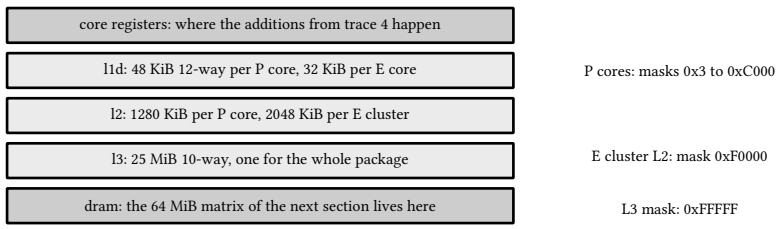


figure 49: the hierarchy the probe reported, one layer per level, the 64-byte line riding every boundary

13.2. row-major versus column-major

C stores a 2-D array as one flat block, so `m[r][c]` is the int at `r * N + c` and the language has said nothing about caches. The dsa book's arrays chapter built a line-count model for exactly this shape and asserted counts without timing; this chapter rebuilds the model in C, asserts the same style of counts, and then does what the dsa chapter declined to do: put a clock on it.

```c
// the dsa book's line model: how many 64-byte lines a walk of `accesses`
// evenly spaced touches fetches. stride under 64 amortizes 64/stride
// elements per line; stride 64 or more pays one line per access
static unsigned long lines_for(unsigned long accesses, unsigned long stride) {
  if (stride ≥ LINE)
    return accesses;
  unsigned long per_line = LINE / stride;
  return (accesses + per_line - 1) / per_line;
}
```

listing 105: the line model in c: lines fetched by a walk of evenly spaced touches (source: c-os-cloud/samples/src/Ch13/locality.c, lines 40-48)

A walk of 4096 ints at stride 4 touches 256 lines. A walk of 4096 ints at stride 16384, the column stride of a 4096 by 4096 matrix, touches 4096 lines: sixteen times the traffic for the same additions. Both walks in the sample run over the whole matrix, both accumulate into an `unsigned long long`, and both must equal the closed form, 523776 per period of 1024 values times 16384 periods, which also pins the iteration count: any walk that skipped or repeated an element would miss the total.

```
for (unsigned long i = 0; i < total; i++)
  m[i] = (int)(i & 1023);
unsigned long long expect = (total / 1024) * (1023ULL * 1024 / 2);
unsigned long long row = 0, col = 0;
double t0 = ms_now();
for (int r = 0; r < N; r++)
  for (int c = 0; c < N; c++)
    row += (unsigned long long)m[(size_t)r * N + c];
double row_ms = ms_now() - t0;
t0 = ms_now();
for (int c = 0; c < N; c++)
  for (int r = 0; r < N; r++)
    col += (unsigned long long)m[(size_t)c * N + r];
double col_ms = ms_now() - t0;
sink += row + col;
CHECK(row == expect, "64 MiB row-major sum matches the closed form");
CHECK(col == expect, "64 MiB column-major sum matches the same form");
CHECK(lines_for(N, 4) == 256,
      "model: one row is 4096 additions over 256 lines");
CHECK(lines_for(N, (unsigned long)N * 4) == 4096,
      "model: one column is 4096 additions over 4096 lines");
printf("64 MiB, 1 pass: row %.1f ms, column %.1f ms, ratio %.2f\n", row_ms,
       col_ms, col_ms / row_ms);
```

listing 106: fill, closed form, row-major and column-major walks, both sums checked against the same total (source: c-os-cloud/samples/src/Ch13/locality.c, lines 62-84)

The measured result is the chapter's first surprise. Across the runs observed while writing, the 64 MiB column walk lands within a few percent of the row walk, a ratio between 0.9 and 1.1, nothing like the 16x the line model charges. The model is not wrong, the machine is generous: a constant stride is a pattern, and the hardware prefetchers this cpu documents as a tunable feature fetch ahead of it. Intel publishes a paper titled Hardware Prefetch Controls whose own description is tuning methods used to optimize the performance of hardware prefetchers, and the effect is measurable here as the collapse of the column-walk penalty. The strided sweep makes the same point from another side: stride 4 costs 12.3 ms for 16777216 accesses, stride 64 costs 2.8 ms for 1048576, and time tracks the 1048576 lines both fetch, not the wildly different access counts.

```
// strided walks over the same 64 MiB: four strides, model exact for
// each. time tracks the line count more than the access count
const unsigned long strides[4] = {4, 64, 256, 1024};
for (int si = 0; si < 4; si++) {
  unsigned long stride = strides[si];
  unsigned long count = bytes / stride;
  unsigned long long s = 0;
  t0 = ms_now();
  for (unsigned long a = 0, k = 0; k < count; k++, a += stride)
    s += (unsigned long long)*(const int *)((const char *)m + a);
  double ms = ms_now() - t0;
  sink += s;
  printf("stride %4lu B: %8lu accesses, model %8lu lines, %.1f ms\n", stride,
         count, lines_for(count, stride), ms);
}
CHECK(lines_for(bytes / 4, 4) == 1048576,
      "model: stride 4 over 64 MiB fetches 1048576 lines");
CHECK(lines_for(bytes / 1024, 1024) == 65536,
      "model: stride 1024 fetches one line per access, 65536 lines");
```

listing 107: four strided walks, model exact for each, time printed never checked (source: c-os-cloud/samples/src/Ch13/locality.c, lines 86-104)

What the machine cannot hide is order with no pattern. The last walk reads the first int of every one of the 1048576 lines, once in address order and once through a fixed-seed linear congruential generator driving a Fisher-Yates shuffle of the line indices, so the two walks touch identical lines, fetch identical traffic, and produce identical sums. Only the order differs, and the shuffled walk costs between 2.3 and 3.4 times the sequential one across the runs observed while writing. That gap, not the row-versus-column gap, is what locality buys on this machine: sequential streams are free, pattern dependence is not.

```
unsigned long long seq = 0, shuf = 0;
t0 = ms_now();
for (unsigned long i = 0; i < lines; i++)
  seq += (unsigned long long)m[i * (LINE / 4)];
double seq_ms = ms_now() - t0;
t0 = ms_now();
for (unsigned long i = 0; i < lines; i++)
  shuf += (unsigned long long)m[order[i] * (LINE / 4)];
double shuf_ms = ms_now() - t0;
sink += seq + shuf;
CHECK(seq == 32256ULL * (lines / 64) && shuf == seq,
      "sequential and shuffled line walks sum identically");
printf("same %lu lines: sequential %.1f ms, shuffled %.1f ms, ratio "
       "%.2f\n",
       lines, seq_ms, shuf_ms, shuf_ms / seq_ms);
```

listing 108: the two orderings: same lines, same sum, only the address sequence differs (source: c-os-cloud/samples/src/Ch13/locality.c, lines 124-138)

NOTE: the model and the clock disagree, correctly

The line model is an upper bound on traffic and a predictor of nothing by itself. The dsa book kept counts and refused to time; this chapter times and keeps the counts, and the two disagree in exactly one direction: measurements land at or below the model's predictions, because prefetchers and out-of-order execution can only remove waits the model counted, never add lines it did not. Assert the count, print the clock, and never the other way around.

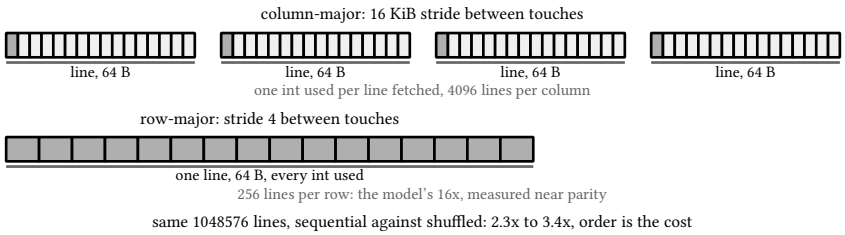


figure 50: before and after: the same matrix column by column and row by row

13.3. false sharing and padding

Two threads, two counters, and a program with no data race can still share a cache line. The layout below is the whole story: in `struct tight`, both 8-byte atomics sit inside the one line that `_Alignas` started at `a`; in `struct padded`, the second `_Alignas` pushes `b` to the next line boundary at the cost of 56 bytes. Every one of those facts is a compile-time constant the checks pin, `offsetof`, `sizeof`, `_Alignof`, plus one runtime check that the statics really landed on 64-byte boundaries.

```c
// both counters share the line that starts at a; b hides 8 bytes in
struct tight {
  _Alignas(64) _Atomic unsigned long long a;
  _Atomic unsigned long long b;
};

// b is aligned to the next line boundary by hand, paying 56 bytes of
// padding between the counters
struct padded {
  _Alignas(64) _Atomic unsigned long long a;
  _Alignas(64) _Atomic unsigned long long b;
};

static struct tight t;
static struct padded p;
```

listing 109: the two layouts: one line with both counters, or one line each, 56 bytes paid (source: c-os-cloud/samples/src/Ch13/falsesharing.c, lines 32-46)

Each worker runs N relaxed fetch-adds on its own counter and never reads the other's, so `memory_order_relaxed` is the honest ordering: no synchronization is being purchased. The workers come from `threads.h`, pinned in chapter 1 as working through the msvc 14.44 crt. `thrd_create` and `thrd_join` are chapter 15's calls, start and reap, and the ordering vocabulary behind `relaxed` is chapter 16's. Here it only matters that a fetch-add is one indivisible read-modify-write.

```c
// one worker: N relaxed atomic increments of its own counter. relaxed
// is enough because no thread ever reads the other counter until join
static int hammer(void *arg) {
  _Atomic unsigned long long *mine = arg;
  for (unsigned i = 0; i < N; i++)
    atomic_fetch_add_explicit(mine, 1, memory_order_relaxed);
  return 0;
}
```

listing 110: the worker: N relaxed atomic increments of one counter (source: c-os-cloud/samples/src/Ch13/falsesharing.c, lines 57-64)

The pairing function times create-and-join as one interval with the `QueryPerformanceCounter` clock the previous section used, documented as a high resolution timestamp for interval measurement that always succeeds with valid parameters on current windows. At `-O2` the fetch-add of 1 lowers to locked increments, `lock incq`, 45 of them

in the unrolled loops of this build, and the gate pins the `lock` prefix in the disassembly: an atomic read-modify-write demands exclusive ownership of its line.

```
// two workers on two counters, timed as a pair. returns elapsed ms
static double run_pair(_Atomic unsigned long long *x,
                       _Atomic unsigned long long *y) {
  thrd_t t1, t2;
  double t0 = ms_now();
  if (thrd_create(&t1, hammer, x) ≠ thrd_success ||
      thrd_create(&t2, hammer, y) ≠ thrd_success)
    return -1.0;
  thrd_join(t1, NULL);
  thrd_join(t2, NULL);
  return ms_now() - t0;
}
```

listing 111: the timed pair: two workers on two counters, one clock (source: c-os-cloud/samples/src/Ch13/falsesharing.c, lines 66-77)

That ownership requirement is why the tight pair loses. Both threads issue `lock incq` against the same 64 bytes, and every locked instruction forces the line through cache coherence, the protocol that keeps per-core caches consistent, so the two threads serialize on ownership even though no C object is shared. The padded pair issues the same instructions against different lines and the ping-pong disappears. The correctness story is deliberately boring: each thread adds exactly N, both layouts end with N and N, and the checks say so. The timing is printed once per layout, and across the runs observed while writing the tight pair costs 345 to 410 ms against 96 to 151 ms padded, a gap of 3 to 4 times that the gate will never assert.

```
int main(void) {
  CHECK(offsetof(struct tight, b) == 8,
        "tight: b sits 8 bytes into the line a starts");
  CHECK(sizeof(struct tight) == 64, "tight: the whole pair is one line");
  CHECK(offsetof(struct padded, b) == 64,
        "padded: b starts the next 64-byte line");
  CHECK(sizeof(struct padded) == 128, "padded: two lines, 56 bytes paid");
  CHECK(_Alignof(struct padded) == 64 && _Alignof(struct tight) == 64,
        "both structs align their start to a line boundary");
  CHECK((uintptr_t)&t % 64 == 0 && (uintptr_t)&p % 64 == 0,
        "statics land on 64-byte boundaries at runtime");
```

listing 112: the layout facts as compile-time constants, checked before any thread starts (source: c-os-cloud/samples/src/Ch13/falsesharing.c, lines 79-89)

```
  atomic_store(&t.a, 0);
  atomic_store(&t.b, 0);
  double tight_ms = run_pair(&t.a, &t.b);
  CHECK(tight_ms ≥ 0, "both tight threads ran");
  CHECK(atomic_load(&t.a) == N && atomic_load(&t.b) == N,
        "tight totals are exact: N and N, no lost updates");

  atomic_store(&p.a, 0);
  atomic_store(&p.b, 0);
  double padded_ms = run_pair(&p.a, &p.b);
  CHECK(padded_ms ≥ 0, "both padded threads ran");
  CHECK(atomic_load(&p.a) == N && atomic_load(&p.b) == N,
        "padded totals are the same N and N");
  printf("tight, one line: %.0f ms; padded, two lines: %.0f ms\n", tight_ms,
         padded_ms);
```

listing 113: both runs, exact totals either way, timing printed only (source: c-os-cloud/samples/src/Ch13/falsesharing.c, lines 91-105)

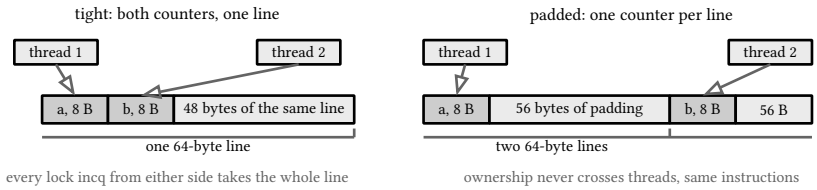


figure 51: one line, two owners, and the coherence traffic that follows

13.4. array of structs versus struct of arrays

Layout is a cache decision the type system silently makes. One particle is four ints, 16 bytes, exactly four per line. The array-of-structs layout interleaves `x` with the three fields an x-only sweep never reads; the struct-of-arrays layout gives every field its own contiguous array. Both x kernels are marked `noinline` so their `-O2` bodies survive as separate functions and the gate can pin what each became.

```
struct pt {
  int x, y, z, c; // 16 bytes: exactly 4 particles per 64-byte line
};

__attribute__((noinline)) static int sum_x_aos(const struct pt *a, int n) {
  int s = 0;
  for (int i = 0; i < n; i++)
    s += a[i].x;
  return s;
}

__attribute__((noinline)) static int sum_x_soa(const int *v, int n) {
  int s = 0;
  for (int i = 0; i < n; i++)
    s += v[i];
  return s;
}
```

listing 114: the particle and the two x kernels, noinline to keep their bodies inspectable (source: c-os-cloud/samples/src/Ch13/aossoa.c, lines 31-47)

The model prices the difference immediately: sweeping x over 1048576 particles fetches 262144 lines from the interleaved array, one line per four particles, and 65536 lines from the `xs` array, one per sixteen. Four times the traffic for the same additions, and the fill guarantees the two sweeps read equal values, so both sums must equal the closed form 133693440. The all-field query is the control: touching every field of every particle uses every byte of every line the interleaved array fetches, and the model charges both layouts the same 262144 lines. Layout follows the query mix, and there is no universal winner.

```
for (int i = 0; i < N; i++) {
  aos[i].x = xs[i] = i & 255;
  aos[i].y = ys[i] = (i * 13) & 1023;
  aos[i].z = zs[i] = (i * 7) & 511;
  aos[i].c = cs[i] = i;
}
// x values repeat with period 256; each period sums to 32640, and
// 4096 periods stay far inside int
int expect = 32640 * (N / 256);

CHECK(sizeof(struct pt) == 16, "the particle is 16 bytes, 4 per line");
double t0 = ms_now();
int ax = sum_x_aos(aos, N);
double ax_ms = ms_now() - t0;
t0 = ms_now();
int sx = sum_x_soa(xs, N);
double sx_ms = ms_now() - t0;
CHECK(ax == expect && sx == expect, "both x sweeps match the closed form");
CHECK(lines_for(N, sizeof(struct pt)) == 262144,
      "model: aos x sweep fetches 262144 lines, one per 4 particles");
CHECK(lines_for(N, 4) == 65536,
      "model: soa x sweep fetches 65536 lines, one per 16 particles");
printf("x sweep: aos %.1f ms, soa %.1f ms\n", ax_ms, sx_ms);
```

listing 115: fill with identical values both ways, then the two x sweeps against one closed form (source: c-os-cloud/samples/src/Ch13/aossoa.c, lines 91-113)

```
t0 = ms_now();
unsigned long long aa = sum_all_aos(aos, N);
double aa_ms = ms_now() - t0;
t0 = ms_now();
unsigned long long sa = sum_all_soa(xs, ys, zs, cs, N);
double sa_ms = ms_now() - t0;
sink = aa + sa;
CHECK(aa == sa, "all-field sweeps agree across both layouts");
CHECK(lines_for(4ULL * N, 4) == 262144,
      "model: the all-field sweep fetches the same 262144 lines both ways");
printf("all fields: aos %.1f ms, soa %.1f ms\n", aa_ms, sa_ms);
```

listing 116: the all-field control: same lines both ways, sums equal (source: c-os-cloud/samples/src/Ch13/aossoa.c, lines 115-125)

The optimizer finishes the argument, and chapter 9 already pinned the terms: the target's feature string is sse2, four `i32` lanes, no gather instruction. At `-O2` the soa kernel is a textbook vectorized reduction, two `load <4 x i32>` per pass feeding `add <4 x i32>` accumulators, closing with `llvm.vector.reduce.add.v4i32`. The aos kernel vectorizes too, but its lanes arrive one at a time: eight scalar `load i32` strided 16 bytes apart, then eight `insertelement` instructions building the two vectors by hand. Same arithmetic, same reduction intrinsic, and every vector load the soa kernel got for free the aos kernel had to assemble. The gate pins all three substrings. Measured, the x sweep costs 1.0 to 1.7 ms interleaved against 0.6 to 0.8 ms separate, and the all-field sweep, whose line counts are identical, measures within noise of itself on both layouts, exactly what the model predicted.

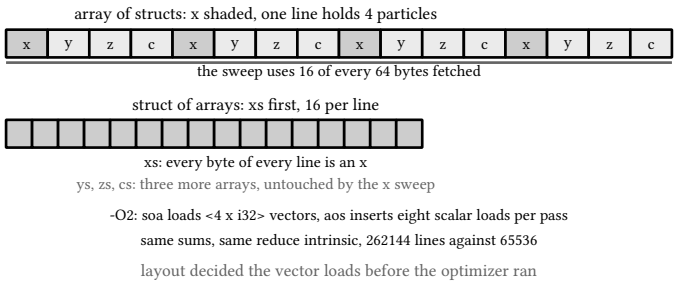


figure 52: the same particles two ways, and what each -O2 kernel loads

VERIFY: one chapter, three kinds of evidence

32 checks run at `-O0` and assert only deterministic facts: probe records, model counts, struct offsets, exact totals, equal sums. 3 ir substrings and 1 disassembly substring are asserted on `-O2` output the runtime never executes. The timings quoted in this chapter, near parity for constant strides, 2.3x to 3.4x for shuffled order, 3x to 4x for the tight pair, 1.5x to 2x for the interleaved x sweep, are printed observations from the runs during writing, and no check anywhere in the chapter depends on them.

sources: learn.microsoft.com, GetLogicalProcessorInformation, `SYSTEM_LOGICAL_PROCESSOR_INFORMATION`, CACHE_DESCRIPTOR, `PROCESSOR_CACHE_TYPE`, and QueryPerformanceCounter pages, and intel.com, Hardware Prefetch Controls for Intel Atom Cores, accessed 2026-09-12; cache hierarchy, timings, and emitted ir probed on this machine the same day. Sample behavior verified by `make verify-c`, 32 checks in chapter 13 of the samples suite.

14. the heap and allocators

Automatic storage lives and dies with the block that declares it. Heap storage is the counterexample: `malloc` hands out storage whose lifetime outlives the call, the program decides when that lifetime ends, and everything between those two points is a contract. This chapter reads the contract from two sources, the C23 final draft n3220 and the ucrt reference pages for the allocator this machine actually links, then rebuilds it from scratch: a free-list allocator over one static arena, the two placement policies real allocators choose between, and finally the windows-only window into a live heap, `HeapWalk`. Three samples carry the chapter with 61 checks, and the address-sanitizer leg of the gate reruns the arena allocator instrumented, because a chapter that builds an allocator owes the reader proof it never writes outside it.

14.1. the malloc contract

The whole contract sits in one paragraph of n3220, 7.24.3 p1, and it opens by listing what is not promised: “The order and contiguity of storage allocated by successive calls to the `aligned_alloc`, `calloc`, `malloc`, and `realloc` functions is unspecified.” What is promised follows immediately: “The pointer returned if the allocation succeeds is suitably aligned so that it may be assigned to a pointer to any type of object with a fundamental alignment requirement and size less than or equal to the size requested.” Two sentences fix the lifetime and the geometry, “The lifetime of an allocated object extends from the allocation until the deallocation” and “Each such allocation shall yield a pointer to an object disjoint from any other object”, and the paragraph closes with the address and failure rules, “The pointer returned points to the start (lowest byte address) of the allocated space. If the space cannot be allocated, a null pointer is returned.”

Zero-size requests keep their own escape hatch: “If the size of the space requested is zero, the behavior is implementation-defined: either a null pointer is returned to indicate an error, or the behavior is as if the size were some nonzero value, except that the returned pointer shall not be used to access an object.” The ucrt picks the second arm and documents it: “If size is 0, `malloc` allocates a zero-length item in the heap and returns a valid pointer to that item.” The sample pins both halves of that pairing on this box, and the same ucrt page fixes the alignment number this machine delivers: the returned storage is “suitably aligned for storage of any type of object that has an alignment requirement less than or equal to that of the fundamental alignment”, and “In code that targets 64-bit platforms, it’s 16 bytes”, the value chapter 4 already pinned for `_aligned_malloc`’s threshold. The page also states the allocator’s freedom in one sentence, “The `malloc` function allocates a memory block of at least size bytes. The block may be larger than size bytes because of the space that’s required for alignment and maintenance information”, maintenance being the allocator’s own bookkeeping, invisible above the interface.

```c
free(NULL); // 7.24.3.3: "If ptr is a null pointer, no action occurs"
int all16 = 1;
for (size_t n = 1; n ≤ 17; n += 4) {
  void *p = malloc(n);
  if (p == NULL || (uintptr_t)p % 16 ≠ 0)
    all16 = 0;
  free(p);
}
CHECK(all16, "ucrt malloc returns 16-aligned storage on x64");
void *z = malloc(0);
CHECK(z ≠ NULL,
      "ucrt malloc(0) returns a valid pointer, its documented arm");
free(z);
```

listing 117: the ucrt half of the contract, executed: no-op null free, 16-byte alignment, the zero-size arm (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 174-186)

The first three checks of the sample run against the real allocator. `free(NULL)` executes the 7.24.3.3 sentence "If ptr is a null pointer, no action occurs" and continues, every size from 1 to 17 comes back 16-aligned, and `malloc(0)` returns a pointer that is freed like any other. The page marks both `malloc` and `realloc` with `__declspec(noalias)` and `__declspec(restrict)`, "the pointer returned isn't aliased", the compiler-facing form of the standard's disjointness promise. The pattern check turns that promise into bytes: two live 64-byte allocations filled with distinct patterns still hold them after both writes, which is only possible if the ranges do not overlap.

`realloc` is where the standard, the platform, and history disagree. The standard text, 7.24.3.7, is careful: "The `realloc` function deallocates the old object pointed to by `ptr` and returns a pointer to a new object that has the size specified by size. The contents of the new object shall be the same as that of the old object prior to deallocation, up to the lesser of the new and old sizes", any bytes beyond the old size have unspecified values, and on failure "the old object is not deallocated and its value is unchanged". A null `ptr` "behaves like the `malloc` function for the specified size". Then the sentence that changed in C23: "if `ptr` does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to the `free` or `realloc` function, or if the size is zero, the behavior is undefined." Zero was not always undefined, cppreference's timeline shows it implementation-defined and deprecated via C DR 400 since C17, undefined since C23, and the ucrt page openly sides with history: "If size is zero, then the block pointed to by `memblock` is freed; the return value is `NULL`, and `memblock` is left pointing at a freed block", under a note that `realloc` "hasn't been updated to implement C17 behavior because the new behavior isn't compatible with the Windows operating system."

```
unsigned char *ra = malloc(64), *rb = malloc(64);
memset(ra, 0xAA, 64);
memset(rb, 0x55, 64);
int patterns = 1;
for (int i = 0; i < 64; i++)
  if (ra[i] != 0xAA || rb[i] != 0x55)
    patterns = 0;
CHECK(patterns, "two live allocations are disjoint, the patterns survive");
free(ra);
free(rb);
unsigned char *rc = malloc(32);
for (int i = 0; i < 32; i++)
  rc[i] = (unsigned char)(i + 1);
unsigned char *rd = realloc(rc, 200);
int prefix = 1;
for (int i = 0; i < 32; i++)
  if (rd[i] != (unsigned char)(i + 1))
    prefix = 0;
CHECK(rd != NULL && prefix && (uintptr_t)rd % 16 == 0,
      "realloc grow preserves the old contents and may move");
```

listing 118: disjoint live storage, then a grow that moves and preserves the prefix (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 187-206)

```
unsigned char *re = realloc(rd, 8);
CHECK(re != NULL && re[7] == 8, "realloc shrink keeps the prefix readable");
free(re);
void *rf = realloc(NULL, 40);
CHECK(rf != NULL && (uintptr_t)rf % 16 == 0,
      "realloc(NULL, n) behaves as malloc");
free(rf);
void *rg = malloc(50);
CHECK(realloc(rg, 0) == NULL, "ucrt realloc(p, 0) frees and returns NULL");
```

listing 119: shrink keeps the prefix, realloc of NULL allocates, the zero-size ucrt contract (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 207-215)

The grow check asks exactly what the standard guarantees and nothing more: 32 bytes filled with a ramp, `realloc` to 200, the ramp intact, the result 16-aligned. Probed on this box the grow moved the block and a shrink to 8 bytes stayed in place, but both of those are freedoms, the checks assert only the contents and the alignment. The last check pins the ucrt's documented zero-size behavior, `realloc(p, 0)` returning `NULL` after freeing, which is precisely the call C23 now declares undefined, so this is one place where the sample asserts the platform's documented contract rather than the standard's.

The standard also grew a sibling for `free` in C23. `free_sized`, 7.24.3.4, states "If `ptr` is a null pointer or the result obtained from a call to `malloc`, `realloc`, or `calloc`, where size is equal to the requested allocation size, this function is equivalent to `free(ptr)`. Otherwise, the behavior is undefined." Probed through the gate's exact flags, this crt does not declare it, the compile dies with `use of undeclared identifier 'free_sized'`, the same absence chapter 4 recorded for `aligned_alloc`. The deallocation side of this chapter therefore uses plain `free`, and the free page's sentence about what deallocation means reads as the thesis of everything below: "Freed memory that isn't released to the operating system is restored to the free pool and is available for allocation again."

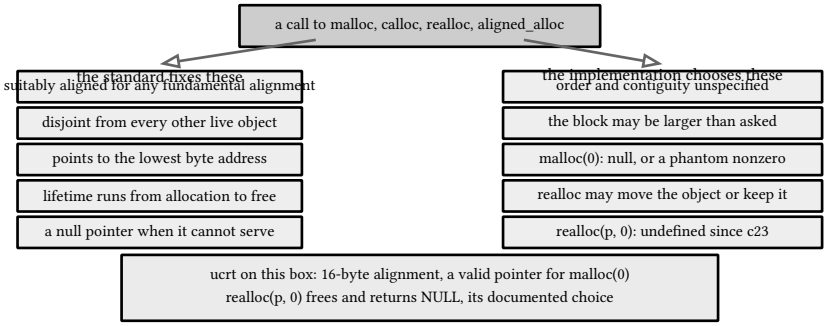


figure 53: the contract as a decision table: what every call must do, and where the implementation chooses

PITFALL: the three-line portability trap at size zero

`malloc(0)` is implementation-defined and the two arms behave oppositely, one returns `NULL`, the other a pointer that must be freed, so `if (malloc(0) == NULL) die();` is wrong on half the implementations. `realloc(p, 0)` was the same kind of split until C23 deleted the choice and made it undefined. And `free_sized`, C23's answer to "the callee already knows the size", is not declared by the ucrt at all. Three size-zero questions, three different answers on this one machine.

14.2. a free-list allocator

The allocator under the arena is one data structure and under 150 lines. The arena is a static 64 KiB array aligned to 16, carved into blocks, and every block opens with the same 32-byte header. The `_Alignas` on the first member raises the whole header struct to alignment 16, which forces `sizeof` from 24 to 32, so every payload begins at a header address plus 32, and headers only ever sit at 16-aligned arena offsets. The header carries the whole block size, a free-list link, and an in-use bit, and the static assert freezes the layout the rest of the file computes against.

```c
// one header per block. size counts header + payload and stays a multiple
// of ALIGN; the _Alignas on the first member raises the struct itself to
// alignment 16, so sizeof is 32 and every payload starts 16-aligned
typedef struct block {
  _Alignas(ALIGN) size_t size;
  struct block *next; // free list link, address ordered, free blocks only
  size_t used;        // 1 while handed out
} block;

_Static_assert(sizeof(block) == 2 * ALIGN, "the header fills two units");

_Alignas(ALIGN) static unsigned char arena[ARENA];
static block *free_list = NULL;
```

listing 120: the 32-byte header and the arena it carves, one alignment rule for both (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 28-40)

The free list is kept sorted by address, and that one decision is what makes coalescing cheap. Returning a block walks the list to the insertion point, links the block in, and then merges twice, forward when this block ends exactly where the next free block starts, and backward when the previous free block ends exactly where this one starts. Adjacent free blocks therefore cannot survive a `my_free`, the list is invariantly coalesced, and the ucrt's own words describe the same maintenance: `_heapmin` works "by coalescing the unused regions", and freed memory "is restored to the free pool and is available for allocation again".

```c
// return b to the free list in address order, coalescing both neighbors
static void free_insert_coalesce(block *b) {
  block *prev = NULL, *cur = free_list;
  while (cur != NULL && (uintptr_t)cur < (uintptr_t)b) {
    prev = cur;
    cur = cur->next;
  }
  b->next = cur;
  if (prev == NULL)
    free_list = b;
  else
    prev->next = b;
  if (cur != NULL && (unsigned char *)b + b->size == (unsigned char *)cur) {
    b->size += cur->size; // forward neighbor
    b->next = cur->next;
  }
  if (prev != NULL &&
      (unsigned char *)prev + prev->size == (unsigned char *)b) {
    prev->size += b->size; // backward neighbor
    prev->next = b->next;
  }
}
```

listing 121: address ordered insertion, then both coalescing directions (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 49-70)

Allocation is first fit down that address-ordered list: round the request up to a multiple of 16, add the header, take the first free block that holds it, and `split_off` carves the block to size, returning the remainder to the free list whenever the remainder can hold a header plus a minimal payload. `my_free` is the mirror image plus the null clause, and the two together implement the 7.24.3 geometry, a disjoint 16-aligned lowest-address payload per call, with reuse for free.

```c
static void *my_malloc(size_t n) {
  if (n == 0)
    return NULL; // the zero-size arm this allocator chooses
  size_t need = round_up(n) + sizeof(block);
  block *prev = NULL, *b = free_list;
  while (b ≠ NULL && b->size < need) { // first fit, address order
    prev = b;
    b = b->next;
  }
  if (b == NULL)
    return NULL; // no hole holds need bytes
  if (prev == NULL)
    free_list = b->next;
  else
    prev->next = b->next;
  b->used = 1;
  split_off(b, need);
  return (unsigned char *)b + sizeof(block);
}
```

listing 122: first fit with split: the carve that serves every request (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 84-102)

```c
static void my_free(void *p) {
  if (p == NULL)
    return; // "no action occurs"
  block *b = (block *)((unsigned char *)p - sizeof(block));
  b->used = 0;
  free_insert_coalesce(b);
}

static void *my_realloc(void *p, size_t n) {
  if (p == NULL)
    return my_malloc(n); // the null-pointer clause
  block *b = (block *)((unsigned char *)p - sizeof(block));
  size_t old = b->size - sizeof(block);
  if (n == 0) {
    my_free(p); // this allocator keeps the ucrt's documented choice
    return NULL;
  }
  if (n ≤ old)
    return p; // a shrink keeps the block, the block may stay larger
```

listing 123: the deallocator, and realloc's three cheap clauses: null, zero, shrink (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 104-122)

`my_realloc` gets the same clauses the standard gives the real one. `NULL` delegates to `my_malloc`, zero frees and returns `NULL` by this allocator's stated choice, matching the ucrt probed above, and a shrink keeps the block, legal because the block "may be larger than size bytes". A grow first tries to absorb the physical successor: the coalesced-list invariant guarantees that successor is free at most once, so a single absorb either fits the request or the grow must move, allocate fresh, `memcpy` the old contents, and free the old block, exactly the copy arm of 7.24.3.7.

```
  size_t need = round_up(n) + sizeof(block);
  // in-place grow: the coalesced list guarantees the physical successor is
  // free at most once, so one absorb either fits the request or never will
  block *succ = (block *)((unsigned char *)b + b->size);
  block *prev = NULL, *cur = free_list;
  while (cur != NULL && (uintptr_t)cur < (uintptr_t)succ) {
    prev = cur;
    cur = cur->next;
  }
  if (cur == succ && b->size + cur->size >= need) {
    if (prev == NULL)
      free_list = cur->next;
    else
      prev->next = cur->next;
    b->size += cur->size;
    split_off(b, need);
    return p;
  }
```

listing 124: the in-place grow: find the successor in the free list, absorb it, split the excess (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 123-140)

Everything the allocator claims is re-derived by an audit that never trusts the allocator's own bookkeeping. Because blocks tile the arena by construction, a walk from the first header that follows `size` fields must land exactly on the arena end. The walk counts blocks, free blocks, free bytes, and the largest free block, and flags `true` only when the walk covered the arena with no gap and no overlap. Every phase of `main` ends in an `audit()` and a check on its numbers, so the no-overlap invariant is not a comment, it is a property the gate re-proves after every mutation.

```
// walk every block header from the arena start. the headers chain by size,
// so a clean heap tiles the arena exactly: no overlap, no gap
static stats audit(void) {
  stats s = {0, 0, 0, 0, 0};
  size_t walked = 0;
  block *b = (block *)arena;
  while (walked < ARENA) {
    if (b->size == 0 || b->size % ALIGN != 0 || walked + b->size > ARENA)
      break; // corrupt header, the tiled check below reports it
    walked += b->size;
    s.blocks++;
    if (!b->used) {
      s.free_blocks++;
      s.free_bytes += b->size;
      if (b->size > s.largest)
        s.largest = b->size;
    }
    b = (block *)((unsigned char *)b + b->size);
  }
  s.tiled = (walked == ARENA);
  return s;
}
```

listing 125: the structural audit: headers chain by size, a clean heap tiles the arena exactly (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 149-170)

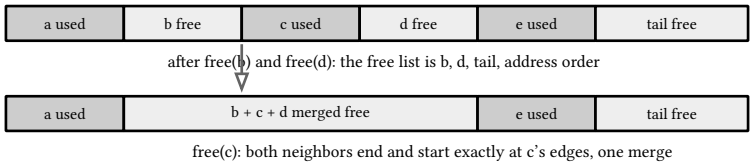


figure 54: one arena through the fragmentation checks: five carves, two frees, and the merge that follows

The main function walks the same road against the arena. The fresh arena audits as one free block, `my_malloc(0)` returns `NULL`, this allocator's chosen arm, two carve-outs for sizes 1 and 17 land 16-aligned leaving exactly the computed free bytes behind, and freeing both coalesces back to a single block. The reuse check is the deterministic form of the contract-section probe: allocate 100, free, allocate 100 again, and the same address returns, because first fit over an address-ordered list always takes the same hole.

```
// fragmentation windows: frees in scrambled order, coalescing observed
void *a = my_malloc(64), *b = my_malloc(64), *c = my_malloc(64);
void *d = my_malloc(64), *e = my_malloc(64);
my_free(b);
s = audit();
CHECK(s.free_blocks == 2, "one freed hole sits beside the tail hole");
my_free(d);
s = audit();
CHECK(s.free_blocks == 3, "a second nonadjacent free opens a third hole");
my_free(c);
s = audit();
CHECK(s.free_blocks == 2, "freeing between two holes coalesces all three");
CHECK(my_malloc(ARENA - sizeof(block)) == NULL,
      "fragmented: no single hole can serve a full-width request");
```

listing 126: the fragmentation windows: scrambled frees and the observed merge (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 248-261)

```
my_free(a);
my_free(e);
s = audit();
CHECK(s.tiled && s.blocks == 1 && s.largest == ARENA,
      "the last frees coalesce the whole arena into one hole");
void *big = my_malloc(ARENA - sizeof(block));
CHECK(big != NULL, "the coalesced arena serves the full-width request");
my_free(big);
```

listing 127: the last frees coalesce the arena, and the full-width request succeeds (source: c-os-cloud/samples/src/Ch14/freelist.c, lines 262-269)

The fragmentation windows are the point of the data structure. Five equal carves, then `free(b)` alone leaves two holes, `free(d)` leaves three, and `free(c)`, allocated between the two freed neighbors, merges all three into one, after which the audit sees two free blocks where three existed a line earlier. While `a` and `e` still hold their blocks, no hole spans the arena, so a full-width request fails, and freeing the last two coalesces everything back until that same request succeeds. The exhaustion phase carves 8192-byte blocks until the 8th fails, exactly 7 fit in 65536 bytes with headers, checks that a failed request leaves the audit unchanged, and frees its way back to a single block. The realloc phases run the three growth paths, absorb-in-place with the successor free, move-and-copy with the successor busy, and the copy check that started the contract section, and the closing audit is one block, `tiled` true, 65536 free bytes.

VERIFY: why the asan leg runs this file

The gate's address-sanitizer pass reruns `freelist.c` instrumented. The arena design makes that leg natural rather than heroic: there is no `malloc` inside the allocator, every header and payload lives inside one static

> array, and the audit already walks the same bytes the sanitizer shadows. The instrumented run prints the same 32 checks, which is the point: an allocator that only works unwatched is not an allocator.

14.3. first fit versus best fit fragmentation

A placement policy answers one question inside every allocator: given several holes that each hold the request, which one gets carved. First fit takes the lowest address, best fit takes the tightest hole. Both run in this chapter's second sample against one fixed nine-operation script, and because the script is data, both runs are byte-for-byte deterministic. The arena is 1024 units; the script allocates 100, 500, 100, 100, frees the 500, then asks for 60, 70, 80, and finally
350. The first four operations never face a choice, there is only one

hole, and the sample asserts both policies place all four identically. The divergence starts at the 60: after the free, the holes are 500 at offset 100 and 224 at the tail.

```
static int hole_alloc(holes *hs, int best, size_t n, size_t *out) {
  int pick = -1;
  for (int i = 0; i < hs->count; i++) {
    if (hs->h[i].size < n)
      continue;
    if (!best) {
      pick = i; // first fit: the lowest address that holds n
      break;
    }
    if (pick < 0 || hs->h[i].size < hs->h[pick].size)
      pick = i; // best fit: the tightest hole that holds n
  }
  if (pick < 0)
    return 0;
  *out = hs->h[pick].off;
  if (hs->h[pick].size == n) {
    for (int j = pick; j < hs->count - 1; j++)
      hs->h[j] = hs->h[j + 1];
    hs->count--;
  } else {
    hs->h[pick].off += n;
    hs->h[pick].size -= n;
  }
  return 1;
}
```

listing 128: the policy choice: first fit stops at the lowest address that fits, best fit keeps scanning for the tightest (source: c-os-cloud/samples/src/Ch14/frag.c, lines 70-94)

First fit carves the 60 out of the 500, best fit out of the 224. The 70 and the 80 follow the same split, first fit shredding the big hole into 440, 370, 290, best fit consuming the small one down to 164, 94,
14. Then the 350 arrives: first fit's largest hole is 290, the request

fails; best fit still holds an untouched 500 at offset 100, and the request lands at 100 leaving a 150 hole. Same script, same total free bytes at every step until the end, opposite outcomes on the one request that mattered. The sample prints both hole lists after every operation, and the transcript lines are the figure below.

The final metric is the difference between the free total and the largest hole, the bytes that are free but cannot serve a request as large as the biggest hole that remains. First fit ends with 514 free bytes and a 290 largest, 224 stranded; best fit ends with 164 free and 150 largest, 14 stranded, having served one more request. Neither policy wins in general, best fit's tight packing is exactly what let it keep the 500 whole, and both samples end the same

way: freeing every live slot coalesces both heaps back to a single hole covering all 1024 units, the same reversibility the arena allocator asserted.

```
int main(void) {
  static const op script[] = {
      {'a', 100, 0}, {'a', 500, 1}, {'a', 100, 2}, {'a', 100, 3}, {'f', 0, 1},
      {'a', 60, 4},  {'a', 70, 5},  {'a', 80, 6},  {'a', 350, 7},
  };
  enum { NOPS = sizeof script / sizeof script[0] };

  result ff = run(script, NOPS, 0, "first fit");
  result bf = run(script, NOPS, 1, "best fit");
```

listing 129: the fixed script, one data table driving both policies (source: c-os-cloud/samples/src/Ch14/frag.c, lines 150-158)

```
  CHECK(same,
        "before the first fitting choice diverges, both place identically");
  CHECK(ff.served == 7, "first fit serves 7 of the 8 allocations");
  CHECK(bf.served == 8, "best fit serves all 8");
  CHECK(!ff.last_ok && bf.last_ok, "only best fit serves the final 350");
  CHECK(ff.hole_count == 2 && bf.hole_count == 2,
        "both end at two holes, of different sizes");
  CHECK(ff.free_total == 514 && ff.largest == 290,
        "first fit strands 514 free bytes, 290 contiguous");
  CHECK(bf.free_total == 164 && bf.largest == 150,
        "best fit strands 164 free bytes, 150 contiguous");
  CHECK(ff.free_total - ff.largest == 224,
        "first fit fragmentation metric: 224 bytes unreachable");
  CHECK(bf.free_total - bf.largest == 14,
        "best fit fragmentation metric: 14 bytes unreachable");
  CHECK(ff.teardown_holes == 1 && bf.teardown_holes == 1,
        "freeing everything coalesces both back to one hole");
```

listing 130: the checks: identical prefix, the served counts, and both fragmentation metrics (source: c-os-cloud/samples/src/Ch14/frag.c, lines 164-180)

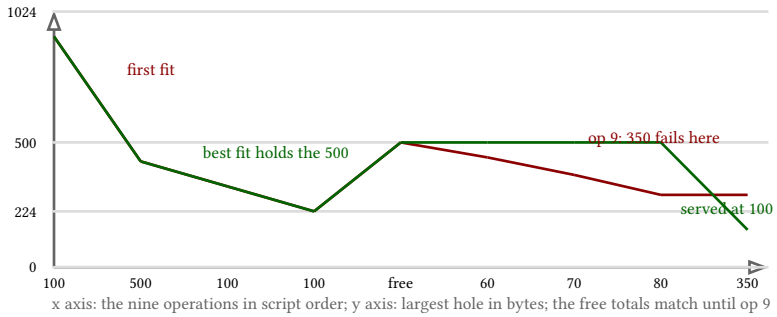


figure 55: largest contiguous hole after each operation, both policies, same script

The windows allocator is the standing counterexample to treating either policy as the end of the story. Its own page states the constraint, “Memory allocated by `HeapAlloc` is not movable”, the address “is valid until the memory block is freed or reallocated”, and “Because the system cannot compact a private heap, it can become fragmented”, the same non-compaction that made this section’s holes sticky. Real allocators answer with size classes and lookaside structures instead of one linear list, glibc’s field names alone disclose the machinery, the man page counts `smblks`, “free fastbin blocks”, separately from ordinary free chunks and `hblks`, “mmapped regions”, and the windows heap “can use the low-fragmentation heap to reduce heap fragmentation” for workloads “that allocate large amounts of memory in various allocation sizes”. The next section opens the one real heap this machine owns and looks inside it.

14.4. HeapWalk over a live heap

Windows exposes the enumeration the chapter has been asserting by hand. `HeapWalk` "Enumerates the memory blocks in the specified heap", taking a heap handle from `HeapCreate` or `GetProcessHeap` and a `PROCESS_HEAP_ENTRY` that carries the whole walk state. The protocol is two sentences: "To initiate a heap enumeration, set the `lpData` field of the `PROCESS_HEAP_ENTRY` structure pointed to by `lpEntry` to NULL", and to continue, call again "with no changes to `hHeap`, `lpEntry`, or any of the members". The end is not a timeout or a sentinel entry, it is a documented error code: "If the heap enumeration terminates successfully by reaching the end of the heap, the function returns FALSE, and GetLastError returns the error code `ERROR_NO_MORE_ITEMS`." The page also states the cost and the hazard, enumerating "is a potentially time-consuming operation", and "HeapWalk can fail in a multithreaded application if the heap is not locked during the heap enumeration", with `HeapLock` and `HeapUnlock` named as the guard, and "no enumeration state data is maintained outside the contents of the `PROCESS_HEAP_ENTRY` structure", so walking needs no separate close call.

```
static walk walk_heap(HANDLE heap, DWORD watch) {
  walk w = {0, 0, 0, 0, 0, 0, 0, 0, 0};
  PROCESS_HEAP_ENTRY e;
  ZeroMemory(&e, sizeof e);
  if (!HeapLock(heap)) {
    w.last_error = GetLastError();
    return w;
  }
```

listing 131: the entry protocol: a zeroed entry, a locked heap, nothing else (source: c-os-cloud/samples/src/Ch14/heapwalk.c, lines 34-41)

```
  while (HeapWalk(heap, &e)) {
    if (w.total ≥ 100000) // a healthy walk ends long before this cap
      break;
    w.total++;
    if (e.wFlags & PROCESS_HEAP_ENTRY_BUSY) {
      w.busy++;
      w.busy_bytes += e.cbData;
      if (e.cbData == watch)
        w.match++;
    } else if (e.wFlags & PROCESS_HEAP_REGION) {
      w.region++;
    } else if (e.wFlags & PROCESS_HEAP_UNCOMMITTED_RANGE) {
      w.uncommitted++;
    } else {
      w.free_blocks++;
    }
  }
  w.last_error = GetLastError();
  w.hit_end = (w.last_error == ERROR_NO_MORE_ITEMS);
  HeapUnlock(heap);
  return w;
}
```

listing 132: the loop, the four entry kinds, and the `ERROR_NO_MORE_ITEMS` terminator (source: c-os-cloud/samples/src/Ch14/heapwalk.c, lines 42-63)

Each entry classifies by `wFlags`: busy entries carry `PROCESS_HEAP_ENTRY_BUSY` with `cbData` as "the size of the data portion of the heap element, in bytes", regions open with `PROCESS_HEAP_REGION`, uncommitted ranges report `PROCESS_HEAP_UNCOMMITTED_RANGE`, and everything else is a free block, the same free pool the contract section described. The struct page adds the depth: "A heap consists of one or more regions of virtual memory, each with a unique region index", and `cbOverhead` is "the size of the data used by the system to maintain information about

the heap element", "in addition to the cbData bytes", the real allocator's maintenance made countable, 8 to 32 bytes per block in the probe below, against the arena allocator's fixed 32.

The discipline the docs demand shapes the helper: between HeapLock and HeapUnlock the code only counts, because the lock page is blunt about what holding it means, "the calling thread owns the heap lock. Only the calling thread will be able to allocate or release memory from the heap", and a printf between the two would itself allocate on the process heap. Counts leave the locked region, the printing happens after. The same page binds the pair, "Each successful call to HeapLock must be matched by a corresponding call to HeapUnlock."

```
int main(void) {
  // printf first: crt startup and this call have already allocated on the
  // process heap, the ucrt malloc page lists both as malloc callers
  printf("walking the heaps of pid %lu\n",
         (unsigned long)GetCurrentProcessId());
  HANDLE proc = GetProcessHeap();
  CHECK(proc ≠ NULL, "GetProcessHeap returns the default heap handle");

  walk before = walk_heap(proc, 777);
  CHECK(before.hit_end, "the process heap walk ends with ERROR_NO_MORE_ITEMS");
  CHECK(before.region ≥ 1, "every walk opens with at least one region entry");
  CHECK(before.busy ≥ 1, "startup allocations are busy entries before main");
  CHECK(before.match == 0, "no busy entry carries the watched size 777 yet");

  LPVOID p = HeapAlloc(proc, 0, 777);
  CHECK(p ≠ NULL && ((ULONG_PTR)p % 16) == 0,
        "HeapAlloc returns 16-aligned storage, MEMORY_ALLOCATION_ALIGNMENT");
  walk live = walk_heap(proc, 777);
  CHECK(live.match == 1,
        "the live 777-byte block is one enumerable busy entry");
  CHECK(live.busy ≥ before.busy + 1, "the busy count grew by at least one");
  CHECK(HeapFree(proc, 0, p), "HeapFree returns the block");
  walk gone = walk_heap(proc, 777);
  CHECK(gone.match == 0, "the freed block no longer walks as busy");
```

listing 133: the process heap: terminator, region, startup allocations, then one watched block through its life (source: c-os-cloud/samples/src/Ch14/heapwalk.c, lines 65-88)

The process heap is already busy before main runs, and the ucrt page says why: "The startup code uses malloc to allocate storage for the _environ, envp, and argv variables", with printf on the same caller list. The walk confirms it structurally: a region entry, busy entries, a termination on ERROR_NO_MORE_ITEMS. One HeapAlloc of a watched 777 bytes then makes a block's lifecycle visible from outside: it appears as exactly one busy entry whose cbData reports 777, the busy count grows, and after HeapFree the match count returns to zero, freed into the free pool where the page's warning lives, "Calling HeapFree twice with the same pointer can cause heap corruption, resulting in subsequent calls to HeapAlloc returning the same pointer twice", the platform's phrasing of the 7.24.3.3 undefined behavior.

```
// a private heap: deterministic, this sample is its only allocator
HANDLE priv = HeapCreate(0, 0, 0);
CHECK(priv ≠ NULL, "HeapCreate builds a private growable heap");
LPVOID a = HeapAlloc(priv, 0, 100);
LPVOID b = HeapAlloc(priv, 0, 200);
LPVOID c = HeapAlloc(priv, 0, 1000);
CHECK(a ≠ NULL && b ≠ NULL && c ≠ NULL && a ≠ b && b ≠ c,
      "three private blocks allocate at distinct addresses");
CHECK(((ULONG_PTR)a % 16) == 0 && ((ULONG_PTR)b % 16) == 0 &&
          ((ULONG_PTR)c % 16) == 0,
      "private heap blocks carry the same 16-byte alignment");
```

listing 134: a private heap of three known blocks, this sample is its only allocator (source: c-os-cloud/samples/src/Ch14/heapwalk.c, lines 90-100)

```
walk w3 = walk_heap(priv, 100);
walk w3b = walk_heap(priv, 1000);
CHECK(w3.hit_end && w3.busy == 3 && w3.region == 1,
      "the private walk: 3 busy blocks inside 1 region");
CHECK(w3.total == w3b.total && w3b.match == 1,
      "two walks of an untouched heap agree, cbData reports 1000 exactly");
CHECK(w3.match == 1, "cbData reports the requested 100 bytes exactly");

CHECK(HeapFree(priv, 0, a) && HeapFree(priv, 0, b) && HeapFree(priv, 0, c),
      "all three blocks return to the private heap");
walk w0 = walk_heap(priv, 0);
CHECK(w0.hit_end && w0.busy == 0 && w0.free_blocks == 1,
      "three adjacent frees coalesce into one free entry");
```

listing 135: the walks: three busy entries, exact sizes, then one coalesced free (source: c-os-cloud/samples/src/Ch14/heapwalk.c, lines 101-113)

The private heap is where the enumeration turns exact, because this sample is its only allocator. `HeapCreate(0, 0, 0)` builds a private growable heap, growable because "If `dwMaximumSize` is 0, the heap can grow in size", and lockable because a heap created with `HEAP_NO_SERIALIZE` "cannot be locked" and this one was not. Three blocks of 100, 200, and 1000 bytes walk as exactly 3 busy entries inside 1 region, two consecutive walks of the untouched heap agree on every count, and `cbData` reports each requested size exactly. Freeing all three adjacent blocks leaves the walk with zero busy entries and exactly one free entry, the windows heap coalesced [4640 + 112 + 208 + 1008 = 5968] bytes into it in the probe run, the same merge the arena allocator's middle window asserts. `HeapDestroy` then "decommits and releases all the pages of a private heap object", permitted without freeing first, and forbidden for the process heap handle outright.

The posix counterpart is `mallinfo(3)`, cited here and never executed, no linux binary stands behind this chapter. The man page summarizes it as a way to "obtain memory allocation information", returning a struct of counters, `arena` for non-mmapped space, `ordblks` for "the number of free chunks", `uordblks` and `fordblks` for total allocated and free space, the same quantities `HeapWalk` counts by enumeration. The page's own metadata is the honesty the walk above earns structurally: its standards field says "None", the original `mallinfo` is deprecated because its `int` fields wrap, `mallinfo2` fixed that in glibc 2.33, and even then "Information is returned for only the main memory allocation area". `HeapWalk` enumerates whatever heap handle it is given, which is how this chapter could verify one private heap's numbers exactly.

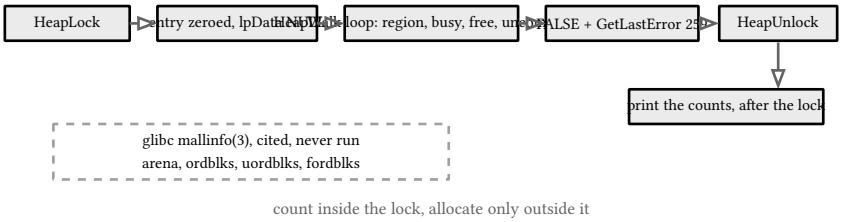


figure 56: the walk pipeline, lock to unlock, and the cited posix counterpart

The chapter closes the pipeline it opened with. Chapter 1 built the gate, chapters 5 and 6 ran its sanitizer legs, and this chapter used both ends: the contract sections read the standard and the platform pages, the arena rebuilt the machinery under the checks, and the walk looked inside a real heap to find the same shapes, headers, free pools, and coalesced neighbors. The two heaps this machine owns are not mysterious, they are lists with invariants, and every invariant named here was asserted green in the gate run this chapter quotes.

sources: learn.microsoft.com, HeapWalk, GetProcessHeap, HeapLock, HeapCreate, HeapAlloc, HeapFree, HeapDestroy, and `PROCESS_HEAP_ENTRY` pages, and the ucrt malloc, free, and realloc pages, accessed 2026-09-12; open-std.org n3220 (7.24.3 p1 common rules, 7.24.3.3 free, 7.24.3.4 `free_sized`, 7.24.3.6 malloc, 7.24.3.7 realloc), accessed 2026-09-12; en.cppreference.com realloc page for the C17 and C23 zero-size timeline, accessed 2026-09-12; man7.org mallinfo(3), accessed 2026-09-12. The ucrt behavior probes, the private-heap walk transcript, the `free_sized` compile failure, and the 5968-byte coalesced free entry are real output from this machine the same day. Sample behavior verified by `make verify-c`, 61 checks in chapter 14 of the samples suite.

15. threads and synchronization

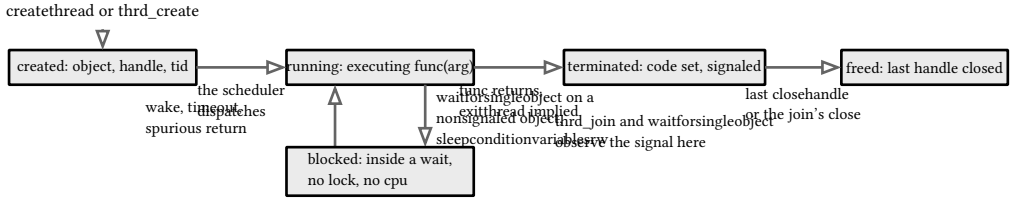
Chapter 11 built a process and its primary thread in one `CreateProcessW` call and left the thread half unopened. This chapter opens it. The Multiple Threads page fixes the object in three sentences: “A thread is the entity within a process that can be scheduled for execution. All threads of a process share its virtual address space and system resources. Each process is started with a single thread, but can create additional threads from any of its threads.” Everything here is a consequence of those sentences. Shared address space is why two workers adding to one total need a lock between them. One schedulable entity per thread is why a c23 `thrd_t` and a win32 handle from `CreateThread` are two spellings of the same kernel scheduling object, and why the last section’s portability ladder climbs from the standard’s header down to that object without ever leaving it. Every behavioral claim below is a CHECK in the three samples, 36 in total, or a sentence quoted from a canonical page fetched 2026-09-12.

15.1. threads and the scheduler’s view

`CreateThread` states its own contract in one line: it “Creates a thread to execute within the virtual address space of the calling process.” Measured against chapter 11′s creation call, half the machinery is missing on purpose. No new address space, no new process object, no pid. What appears is a kernel object with a thread identifier, “By default, every thread has one megabyte of stack space”, and a handle back to the caller. The page also fixes the parent’s view of the creation race: with no creation flags “The thread runs immediately after creation”, and in general “the thread can start running before CreateThread returns and, in particular, before the caller receives the handle and identifier of the created thread”. The samples below take that seriously. Every worker records its own `GetCurrentThreadId()` inside the thread, because the creator cannot assume it observed anything before the worker ran.

The scheduler’s view of one thread is a state machine, and every synchronization call in this chapter names an edge of it. The thread object is born unsignaled with its exit code pending. While the thread function runs, the machine can drop it into a wait, and a waiting thread holds no lock and no cpu. When the thread function returns, the page writes the ending: “the DWORD return value is used to terminate the thread in an implicit call to the ExitThread function”, the exit code is set, and “When a thread terminates, the thread object attains a signaled state, satisfying any threads that were waiting on the object.” The object outlives the execution by exactly one rule, the same one chapter 11 proved for process handles: “The thread object remains in the system until the thread has terminated and all handles to it have been closed through a call to CloseHandle.” The Terminating a Thread page states the twin from the other side: “When a thread terminates, its thread object is not freed until all open handles to the thread are closed.”

The wait side of the machine belongs to `WaitForSingleObject`, whose waitable list includes thread, and the Synchronizing Execution of Multiple Threads page says when that wait ends: “Process and thread handles are signaled when the process or thread terminates.” A parent that waits on a thread handle therefore blocks until the state machine above reaches its signaled state, and the return constants are pinned on the function’s page: `WAIT_OBJECT_0`, 0x00000000, “The state of the specified object is signaled”, `WAIT_TIMEOUT`, 0x00000102, when the interval elapsed.



waits and wakes move a thread between running and blocked; only termination is one-way

figure 57: one thread object through the scheduler’s state machine, the calls that drive each edge

15.2. c23 threads.h and win32 side by side

The c23 layer first. The header clause, n3220 7.28.1: “The header <threads.h> includes the header <time.h>, defines macros, and declares types, enumeration constants, and functions that support multiple threads of execution.” Of those types, thrd_t “is a complete object type that holds an identifier for a thread” and thrd_start_t “is the function pointer type int (*)(void*) that is passed to thrd_create to create a new thread”. thrd_create at 7.28.5.1 “creates a new thread executing func(arg)”, sets the caller’s thrd_t to the new thread’s identifier, and carries one ordering promise: “The completion of the thrd_create function synchronizes with the beginning of the execution of the new thread.” The worker’s side is one sentence: “Returning from func has the same behavior as invoking thrd_exit with the value returned from func.” thrd_join at 7.28.5.6 “joins the thread identified by thr with the current thread by blocking until the other thread has terminated”, stores the result code through its res pointer, and closes the loop: “The termination of the other thread synchronizes with the completion of the thrd_join function.” That synchronizes sentence is why the samples can read worker-written fields after a join with no lock of their own.

On this box that vocabulary works, and the working set is larger than the plan assumed: thrd_create, thrd_join, mtx_t, and cnd_t all link and run under the gate’s exact flags, probed 2026-09-12 through the msvc 14.44 crt, which ships threads.h in the toolset’s own include directory rather than the Windows SDK’s ucrt. The first sample runs the same worker discipline twice, once per layer, and asserts the same exact total both times:

```
static int c23_worker(void *arg) {
  struct c23_job *job = arg;
  job->tid = GetCurrentThreadId();
  mtx_lock(&job->pool->lock);
  job->pool->total += 1000 + job->index;
  mtx_unlock(&job->pool->lock);
  return 100 + job->index;
}
```

listing 136: the c23 worker: lock, add, unlock, return a result code (source: c-os-cloud/samples/src/Ch15/threads.c, lines 44-51)

The worker takes the mtx_t, adds its slice under it, and returns 100 + index. The standard is specific about what the lock buys at 7.28.4: “For purposes of determining the existence of a data race, lock and unlock operations behave as atomic operations. All lock and unlock operations on a particular mutex occur in some particular total order.” Eight workers therefore serialize into some order the scheduler picks; addition commutes; the total does not depend on the order picked. Without the lock, the read-modify-write is three instructions wide and the total would be whatever interleaving happened to allow. The sample asserts the with-lock arithmetic, not the folklore:

```
  struct c23_pool pool = {0};
  CHECK(mtx_init(&pool.lock, mtx_plain) == thrd_success, "mtx_init succeeds");
  struct c23_job jobs[N] = {0};
  thrd_t threads[N];
  int created = 0;
  for (int i = 0; i < N; i++) {
    jobs[i].pool = &pool;
    jobs[i].index = i;
    created += thrd_create(&threads[i], c23_worker, &jobs[i]) == thrd_success;
  }
  CHECK(created == N, "thrd_create starts all 8 workers");
  int joined = 0, results_ok = 1, tids_ok = 1;
  for (int i = 0; i < N; i++) {
    int res = -1;
    joined += thrd_join(threads[i], &res) == thrd_success;
    results_ok = results_ok && res == 100 + i;
    tids_ok = tids_ok && jobs[i].tid != 0 && jobs[i].tid != main_tid;
  }
```

listing 137: leg 1: one mutex total, eight creates, eight joins (source: c-os-cloud/samples/src/Ch15/threads.c, lines 80-97)

Line by line: the pool is zeroed and its mutex initialized with `mtx_plain`, which n3220 describes as the constant for "a mutex object that does not support timeout". Each job points at the pool and carries its index. Eight `thrd_create` calls start eight `thrd_t`. Then the join loop, which is where the layer pays its rent: each `thrd_join` blocks the parent until that worker terminated, hands the worker's result through `res`, and releases the thread object it owns.

```
  CHECK(joined == N, "thrd_join reaps all 8 workers");
  CHECK(results_ok, "every join hands back the result 100 + index");
  CHECK(pool.total == EXPECTED_TOTAL,
        "the mutex total is exactly 8028: 8*1000 plus 0+1+...+7");
  CHECK(tids_ok, "every worker tid is nonzero and distinct from main's");
  mtx_destroy(&pool.lock);
```

listing 138: leg 1 verdicts: every join result, the exact 8028 (source: c-os-cloud/samples/src/Ch15/threads.c, lines 98-103)

All 8 joins succeed, every result is `100 + index`, and the total is 8028, eight thousands plus the sum 0+1+...+7. The tid check is the scheduler view made concrete: each worker wrote its own `GetCurrentThreadId()` while running, and all eight differ from the parent's.

The win32 twin underneath replaces the crt's types with the kernel's. The worker is the same discipline over an `SRWLOCK`, which the InitializeSRWLock page introduces as a "slim reader/writer (SRW) lock" acquired "in exclusive mode" by `AcquireSRWLockExclusive`. No initializer return value, no error code: the page returns none for both, and the lock is a single 8 byte object, probed.

```
static DWORD WINAPI win_worker(LPVOID arg) {
  struct win_job *job = arg;
  job->tid = GetCurrentThreadId();
  AcquireSRWLockExclusive(&job->pool->lock);
  job->pool->total += 1000 + job->index;
  ReleaseSRWLockExclusive(&job->pool->lock);
  return (DWORD)(100 + job->index);
}
```

listing 139: the win32 twin worker over an srw lock (source: c-os-cloud/samples/src/Ch15/threads.c, lines 65-72)

```
  int waited = 0, codes_ok = 1, closed = 0, wtids_ok = 1;
  for (int i = 0; i < N; i++) {
    waited += WaitForSingleObject(handles[i], INFINITE) == WAIT_OBJECT_0;
    DWORD code = 0;
    codes_ok = codes_ok && GetExitCodeThread(handles[i], &code) &&
               code == (DWORD)(100 + i);
    closed += CloseHandle(handles[i]);
    wtids_ok = wtids_ok && wjobs[i].tid != 0 && wjobs[i].tid != main_tid;
  }
  CHECK(waited == N, "each thread handle reaches the signaled state");
  CHECK(codes_ok, "getexitcodethread reads back 100 + index for every worker");
  CHECK(closed == N, "all 8 thread handles close");
  CHECK(wpool.total == EXPECTED_TOTAL,
        "the srw lock total is the same exact 8028");
  CHECK(wtids_ok, "win32 worker tids are nonzero and distinct from main's");
```

listing 140: leg 2 audit: wait on the handle, read the exit code, close (source: c-os-cloud/samples/src/Ch15/threads.c, lines 119-133)

The audit follows the GetExitCodeThread page's own discipline. The wait runs first, so the exit code is only read after "the thread has been confirmed to have exited", exactly the order the page's warning asks for: it "returns a valid error code defined by the application only after the thread terminates", so "an application should not use `STILL_ACTIVE` (259) as an error code", the same 259 collision chapter 11 measured on processes. After the wait, the

exit code is “The return value from the thread function”, and all eight read back `100 + index`. Then the closes, one per handle, and the SRWLOCK leg’s total is the same exact 8028. Two spellings, one scheduler object.

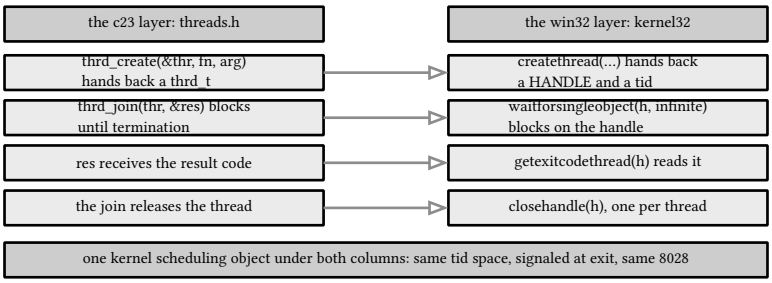


figure 58: the two layers side by side, paired steps, one scheduler object under both

PITFALL: createthread and the c runtime

The CreateThread page carries one crt warning: “A thread in an executable that calls the C run-time library (CRT) should use the `_beginthreadex` and `_endthreadex` functions for thread management rather than CreateThread and ExitThread; this requires the use of the multithreaded version of the CRT. If a thread created using CreateThread calls the CRT, the CRT may terminate the process in low-memory conditions.” The gate’s workers call `printf` from both kinds of thread and hold, probed; the layer that owns the concern outright is the crt’s own `thrd_create`, which is why it is the primary spelling in this book.

15.3. condition variables and wake discipline

A lock serializes; a condition variable lets a thread wait for a state it cannot see while holding that lock. The Condition Variables page states the mechanism: “Condition variables enable threads to atomically release a lock and enter the sleeping state”, they “can be used with critical sections or slim reader/writer (SRW) locks”, and after the wake “it re-acquires the lock it released when the thread entered the sleeping state”. `SleepConditionVariableSRW` says it again as a function contract: “Sleeps on the specified condition variable and releases the specified lock as an atomic operation”, returning nonzero on success and, “If the timeout expires”, FALSE “and GetLastError returns `ERROR_TIMEOUT`”. One more sentence from the same page sizes the sample’s data structure: “It is often convenient to use more than one condition variable with the same lock.” The ring below takes that advice: one `SRWLOCK`, `not_empty` for the consumer, `not_full` for the producer, over a four-slot ring of ints with head, tail, and count.

```c
static int producer(void *arg) {
  struct producer_arg *pa = arg;
  pa->tid = GetCurrentThreadId();
  struct ring *r = pa->ring;
  for (int v = 0; v < ITEMS; v++) {
    AcquireSRWLockExclusive(&r->lock);
    while (r->count == CAP)
      SleepConditionVariableSRW(&r->not_full, &r->lock, INFINITE, 0);
    r->buf[r->tail] = v;
    r->tail = (r->tail + 1) % CAP;
    r->count++;
    ReleaseSRWLockExclusive(&r->lock);
    WakeConditionVariable(&r->not_empty);
  }
  return ITEMS;
}
```

listing 141: the producer: sleep on `not_full` while full, publish, wake `not_empty` (source: c-os-cloud/samples/src/Ch15/condvar.c, lines 41-56)

Both loops run under `thrd_create`, the c23 layer driving win32 synchronization underneath, and both follow the page's usage pattern verbatim: acquire, recheck the predicate in a `while`, sleep on the condition variable, act, release, then wake. The page even orders the last two steps: "It is usually better to release the lock before waking other threads to reduce the number of context switches", which is why every `WakeConditionVariable` in the sample sits after its `ReleaseSRWLockExclusive`.

```
static int consumer(void *arg) {
  struct consumer_arg *ca = arg;
  ca->tid = GetCurrentThreadId();
  struct ring *r = ca->ring;
  for (int k = 0; k < ITEMS; k++) {
    AcquireSRWLockExclusive(&r->lock);
    while (r->count == 0)
      SleepConditionVariableSRW(&r->not_empty, &r->lock, INFINITE, 0);
    ca->got[k] = r->buf[r->head];
    r->head = (r->head + 1) % CAP;
    r->count--;
    ReleaseSRWLockExclusive(&r->lock);
    WakeConditionVariable(&r->not_full);
  }
  return ITEMS;
}
```

listing 142: the consumer: sleep on `not_empty` while empty, take, wake `not_full` (source: c-os-cloud/samples/src/Ch15/condvar.c, lines 64-79)

The hard check does not depend on the interleaving, and that is the design. One producer publishes 0, 1, ..., 15 in order. The ring is fifo. One consumer takes 16 items. Whatever order the scheduler interleaves producer and consumer, the consumed sequence equals the produced one, so the sample asserts exactly that, plus the drain: 16 items through a 4-slot ring leaves count 0 and head and tail wrapped back to 0.

```
  int sequence_ok = 1;
  for (int k = 0; k < ITEMS; k++)
    sequence_ok = sequence_ok && ca.got[k] == k;
  CHECK(sequence_ok,
        "the consumed sequence is exactly 0 1 2 ... 15, equal to production");
  CHECK(r.count == 0 && r.head == r.tail && r.head == 0,
        "the ring drains: count 0, head and tail wrapped back to 0");
  CHECK(pa.tid ≠ main_tid && ca.tid ≠ main_tid && pa.tid ≠ ca.tid,
        "producer, consumer, and main are three distinct threads");
```

listing 143: the deterministic verdicts: sequence equality, drained ring, three distinct threads (source: c-os-cloud/samples/src/Ch15/condvar.c, lines 103-111)

The wake discipline is where this territory earns its reputation, and the docs hand over the reasons verbatim. The Condition Variables page: "Condition variables are subject to spurious wakeups (those not associated with an explicit wake) and stolen wakeups (another thread manages to run before the woken thread). Therefore, you should recheck a predicate (typically in a while loop) after a sleep operation returns." The standard says why the loop cannot be an `if`, from the other side, at 7.28.3.4: `cnd_signal` "unblocks one of the threads that are blocked on the condition variable pointed to by cond at the time of the call. If no threads are blocked on the condition variable at the time of the call, the function does nothing and returns success." A wake has no memory. Fired at a thread not yet asleep, it is gone, and the sleeper sleeps on. The third sample makes that interleaving happen deterministically instead of once in a while:

```
// the bug: check once, park at go while the wake fires, then sleep once and
// never recheck. the events make the lost wake deterministic instead of a
// once-in-a-while race
static int buggy_consumer(void *arg) {
  struct bug_ctx *c = arg;
  c->tid = GetCurrentThreadId();
  AcquireSRWLockExclusive(&c->s->lock);
  c->observed_empty = c->s->count == 0;
  ReleaseSRWLockExclusive(&c->s->lock);
  SetEvent(c->saw_empty);
  WaitForSingleObject(c->go, INFINITE);
  AcquireSRWLockExclusive(&c->s->lock);
  BOOL woken = SleepConditionVariableSRW(&c->s->cv, &c->s->lock, 1000, 0);
  c->timed_out = !woken;
  c->err = GetLastError();
  ReleaseSRWLockExclusive(&c->s->lock);
  return 0;
}
```

listing 144: the bug: check once, park at go while the wake fires, sleep once, never recheck (source: c-os-cloud/samples/src/Ch15/wake.c, lines 43-60)

Two event objects act as scheduling gates. The consumer takes the lock, sees `count == 0`, releases, and parks on `saw_empty`. The parent, holding the ordering the events force, fills the slot and fires the wake while the consumer is still parked on the event, then releases it into the sleep:

```
  // phase 1: the lost wake. main waits until the consumer has observed the
  // empty slot under the lock, then fills the slot and fires the wake while
  // the consumer is still parked at go: nobody is on cv at wake time
  struct bug_ctx bc = {.s = &s, .saw_empty = saw_empty1, .go = go1};
  thrd_t t1;
  CHECK(thrd_create(&t1, buggy_consumer, &bc) == thrd_success,
        "the buggy consumer starts");
  CHECK(WaitForSingleObject(saw_empty1, INFINITE) == WAIT_OBJECT_0,
        "main waits until count 0 was observed under the lock");
  AcquireSRWLockExclusive(&s.lock);
  s.item = 7;
  s.count = 1;
  ReleaseSRWLockExclusive(&s.lock);
  WakeConditionVariable(&s.cv);
  SetEvent(go1);
  int r1 = -1;
  CHECK(thrd_join(t1, &r1) == thrd_success,
        "the buggy consumer joins after its bounded sleep");
```

listing 145: phase 1 interleave: the wake fires while nobody is asleep on the condition variable (source: c-os-cloud/samples/src/Ch15/wake.c, lines 99-116)

```
  CHECK(bc.observed_empty,
        "phase 1: the predicate was false when checked under the lock");
  CHECK(bc.timed_out && bc.err == ERROR_TIMEOUT,
        "phase 1: the sleep burns its full 1s timeout, the wake was lost");
  CHECK(s.count == 1 && s.item == 7, "phase 1: the item sat there all along, "
                                     "the sleep-once shape never rechecks");
```

listing 146: phase 1 verdicts: full timeout burned, item stranded (source: c-os-cloud/samples/src/Ch15/wake.c, lines 118-123)

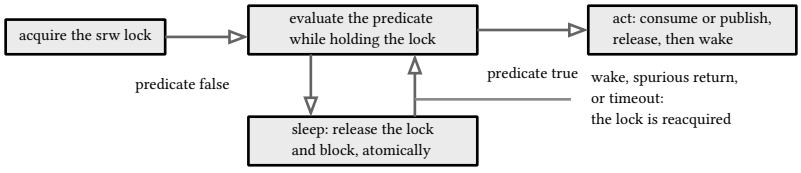
The wake was fired at a thread that was not blocked "at the time of the call", so it did nothing. The consumer then sleeps its full 1000 ms, gets FALSE and `ERROR_TIMEOUT`, and exits without ever looking again. The slot held item 7 the entire time. In a real program this is the shape that deadlocks on an `INFINITE` timeout or strands work behind an `if`; here the bounded timeout turns the bug into an assertion.

The fix is the loop, and the sample proves both halves of it. The consumer rechecks the predicate under the lock on every pass, so a wake lost before the sleep cannot strand the item:

```
// the fix: the predicate is rechecked under the lock on every pass, so a
// wake lost before the sleep or a spurious return from it cannot strand work
static int fixed_consumer(void *arg) {
  struct fix_ctx *c = arg;
  c->tid = GetCurrentThreadId();
  AcquireSRWLockExclusive(&c->s->lock);
  while (c->s->count == 0) {
    c->wakes++;
    SetEvent(c->saw_empty);
    SleepConditionVariableSRW(&c->s->cv, &c->s->lock, INFINITE, 0);
  }
  c->consumed = c->s->item;
  c->s->count = 0;
  ReleaseSRWLockExclusive(&c->s->lock);
  return 0;
}
```

listing 147: the fix: recheck the predicate under the lock on every pass (source: c-os-cloud/samples/src/Ch15/wake.c, lines 70-85)

The phase 2 interleave is the mirror of phase 1 with one decisive difference: the consumer arms `saw_empty` while still holding the lock, inside the loop, before the sleep call. The parent's acquire can therefore only complete after the sleep call released the lock, which means the parent's `WakeConditionVariable` always lands on a thread that is already asleep on the condition variable. The join returning is the proof: phase 2 sleeps with `INFINITE`, so its termination is only possible if the wake was received. It consumes the item 9, the loop body ran at least once, and the printed sleep count, 1 in every run so far, stays a printed number rather than a check because spurious and stolen wakes make the count the scheduler's business, not the program's. That division is the entire lesson: the program owns the predicate and the lock, the scheduler owns the order.



the recheck is the loop, not the if: no exit from the sleep skips the predicate

figure 59: the wait protocol as a cycle: every exit from the sleep lands on the predicate again

15.4. the portability boundary

`<threads.h>` is optional by design, and the standard says how a program is supposed to detect the option: "Implementations that define the macro `__STDC_NO_THREADS__` may not provide this header nor support any of its facilities", n3220 7.28.1. The macro is the sanctioned preprocessor gate. On this box the gate and the ground disagree:

```c
#ifdef __STDC_NO_THREADS__
static const int no_threads_macro = 1;
#else
static const int no_threads_macro = 0;
#endif
```

listing 148: the compile-time gate and the probed fact on this box (source: c-os-cloud/samples/src/Ch15/threads.c, lines 26-30)

Probed 2026-09-12: clang 23.1.1 predefines `__STDC_NO_THREADS__` as 1 on the `x86_64-pc-windows-msvc` target, visible in `clang -dM -E` output before any header is read, while the msvc 14.44 toolset linked right next to it ships a complete, working `threads.h`. The check holds both facts together: the macro is defined and every threads.h-based check in this chapter still passes. A portable ladder that trusts the macro alone would step past a working implementation and into a fallback it did not need. The chapter 2 vocabulary already has the honest probe: `__has_include(<threads.h>)` asks the compiler what is actually in front of it. Trust the crt you link, not the compiler's assumption about it.

Below the header, whatever provides it, the comparison target for every explanation of this chapter's primitives is posix, cited here from Issue 8 and never executed on this box, the same treatment chapter 11 gave `fork`. `pthread_mutex_lock` "shall be locked by a call to `pthread_mutex_lock()` that returns zero", and "If the mutex is already locked by another thread, the calling thread shall block until the mutex becomes available", the same contract `mtx_lock` spelled with a `mtx_t`. The condition twin is `pthread_cond_wait`: "These functions atomically release mutex and cause the calling thread to block on the condition variable cond", with the loop requirement stated as recommendation, "It is thus recommended that a condition wait be enclosed in the equivalent of a while loop that checks the predicate", and the allowance that makes the loop mandatory in practice: "Spurious wakeups from the `pthread_cond_clockwait()`, `pthread_cond_timedwait()`, or `pthread_cond_wait()` functions may occur." Windows says may; posix says may occur; the loop is not optional anywhere.

The win32 fallback every real project carries is exactly the set this chapter probed: `CreateThread` for the object, `WaitForSingleObject` for the signal, `SRWLOCK` for the mutex, and a `CONDITION_VARIABLE` for the wait, all 8-byte user-mode objects, none of them needing a destroy call, because the InitializeSRWLock page documents the exemption: "An unlocked SRW lock with no waiting threads is in its initial state and can be copied, moved, and forgotten without being explicitly destroyed", and condition variables carry no delete function at all. The samples take that leave: no `SRWLOCK` is destroyed in chapter 15, and the gate's handle accounting closes every event handle it opened.

environment	threads.h	the synchronization layer
this box: clang 23, msvc 14.44 crt	present and probed, the macro defined anyway	srwlock plus condition variables, this chapter
posix systems	the libc's choice, not a posix promise	pthread_mutex_lock, pthread_cond_wait, cited
any target where the header is absent	absent: the gate fires	createthread, waitforsingleobject, the win32 fallback

the preprocessor gate can lie on this box: has_include of the header decides on the spot

figure 60: where threads.h exists, what each environment synchronizes with, and the fallback ladder

sources: learn.microsoft.com, CreateThread, GetExitCodeThread, WaitForSingleObject, InitializeSRWLock, AcquireSRWLockExclusive, SleepConditionVariableSRW, WakeConditionVariable, Condition Variables, Multiple Threads, Creating Threads, Terminating a Thread, and Synchronizing Execution of Multiple Threads pages, accessed 2026-09-12; open-std.org n3220 (7.28.1 threads.h introduction and `__STDC_NO_THREADS__`, 7.28.3.1 through 7.28.3.6 condition variables, 7.28.4 mutex total order, 7.28.5.1 `thrd_create`, 7.28.5.6 `thrd_join`), accessed 2026-09-12; pubs.opengroup.org Issue 8 `pthread_mutex_lock` and `pthread_cond_wait`, accessed 2026-09-12; the working threads.h through the msvc 14.44 crt (header in the toolset include, not the sdk ucrt), the predefined `__STDC_NO_THREADS__` despite it, the 8 byte lock and condition variable objects, and the lost-wake timeout probed on this machine the same day. Sample behavior verified by `make verify-c`, 36 checks in chapter 15 of the samples suite.

16. atomics and memory ordering

Chapter 15 gave every thread its own stack and a shared heap. Nothing stops two threads from touching the same bytes at the same instant: two accesses to the same object, at least one a write, not both atomic, with no ordering between them, are a data race. The `volatile` keyword does not help: cppreference's memory ordering page states the case plainly, "volatile accesses are not atomic (concurrent read and write is a data race) and do not order memory." The portable tool is `stdatomic.h`, and this chapter reads it the way the rest of the book reads everything: which header actually resolves on this target, what each of the six orderings buys, what the instructions look like on x86-64, and where the operating system's own waiter, `WaitOnAddress`, plugs in. Every behavioral claim below is a CHECK in the three samples, 41 in total, an ir or objdump pin in `Ch16/expect-ir.txt`, or a sentence quoted from a canonical page fetched 2026-09-12.

16.1. stdatomic.h on this target

The header trail is a probed fact, not an assumption. The gate compiles every sample with `-isystem` paths for the msvc 14.44 toolset and the windows sdk, and `clang -v` prints the resulting search order: the toolset include directory comes first, then ucrt, um, shared, and only then clang's own resource directory. So `#include <stdatomic.h>` finds the msvc header, `...\VC\Tools\MSVC\14.44.35207\include\stdatomic.h`, whose entire C-mode body is one line, `#include <vcruntime_c11_stdatomic.h>`, probed with `clang -H` on this box. Clang's own `stdatomic.h` sits behind it in the search order and is never reached. The msvc header defines `memory_order` as a real enum with probed values `relaxed 0`, `consume 1`, `acquire 2`, `release 3`, `acq_rel 4`, `seq_cst 5`, defines every `ATOMIC_*_LOCK_FREE` macro as 1, the standard's "sometimes lock free" answer, and defines every generic operation as a macro that forwards to a clang builtin, `atomic_fetch_add` to `__c11_atomic_fetch_add`, `atomic_load` to `__c11_atomic_load`, with the non-`_explicit` forms hard-wiring `seq_cst`, the standard's default ordering.

`atomic_is_lock_free` is where this header's answer is most visible. The macro expands to `_Atomic_is_lock_free(sizeof(__typeof_unqual__(*(_Obj))))`, an inline function in `vcruntime_c11_atomic_support.h` whose body reads `return _Sz ≤ 8 && (_Sz & _Sz - 1) == 0;`. The answer is a width predicate, not a cpu probe: power of two, at most 8 bytes.

```
static int width_map(void) {
  _Atomic char b1;
  _Atomic short b2;
  _Atomic int b4;
  _Atomic long long b8;
  _Atomic wide_pair b16;
  CHECK(sizeof(b16) == 16, "the wide pair really is 16 bytes");
  CHECK(atomic_is_lock_free(&b1) && atomic_is_lock_free(&b2),
        "1 and 2 byte atomics are lock free");
  CHECK(atomic_is_lock_free(&b4) && atomic_is_lock_free(&b8),
        "4 and 8 byte atomics are lock free");
  CHECK(!atomic_is_lock_free(&b16),
        "16 byte atomics are not lock free on this target");
  CHECK(ATOMIC_INT_LOCK_FREE == 1 && ATOMIC_LLONG_LOCK_FREE == 1 &&
            ATOMIC_POINTER_LOCK_FREE == 1,
        "the LOCK_FREE macros report 1: sometimes lock free");
  return 0;
```

listing 149: the width map: 1, 2, 4, 8 report lock free, 16 reports not (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 29-45)

The checks confirm both halves on this target: the 16-byte pair really is 16 bytes. `long` is 4 bytes on windows, so the struct needs two of the 8-byte `long long` fields. The four hardware widths all report lock free, and the `LOCK_FREE` macros all print 1. Two more target facts round out the map. First, `ATOMIC_CHAR8_T_LOCK_FREE` is undefined on this target, probed with `#ifdef`: the msvc header predates C23's `char8_t` atomics, while clang's builtin

`__CLANG_ATOMIC_CHAR8_T_LOCK_FREE` is 2, "always lock free", the value the resource header would have printed. Second, operations on a 16-byte atomic compile only past a warning: `-Watomic-alignment`, "large atomic operation may incur significant performance penalty", is on by default and turns fatal under the gate's `-Werror`, so the compiler itself enforces the same 8-byte ceiling the predicate reports.

The operation set is the standard's, and the checks walk it in order:

```
static int operations(void) {
  _Atomic int x;
  atomic_init(&x, 0);
  atomic_store(&x, 5);
  CHECK(atomic_load(&x) == 5, "default store and load round trip");
  int old = atomic_fetch_add(&x, 10);
  CHECK(old == 5 && atomic_load(&x) == 15, "fetch_add returns the old value");
  old = atomic_fetch_sub_explicit(&x, 3, memory_order_relaxed);
  CHECK(old == 15 && atomic_load(&x) == 12,
        "relaxed fetch_sub still counts exactly");
  old = atomic_fetch_or_explicit(&x, 0x10, memory_order_relaxed);
  CHECK(old == 12 && atomic_load(&x) == 28, "fetch_or sets the masked bit");
  old = atomic_exchange_explicit(&x, 40, memory_order_acq_rel);
  CHECK(old == 28 && atomic_load(&x) == 40,
        "exchange swaps and returns the old value");
```

listing 150: the arithmetic and swap family: default load, fetch add, sub, or, exchange (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 52-66)

```
  int expect = 40;
  CHECK(atomic_compare_exchange_strong(&x, &expect, 7),
        "compare_exchange succeeds on a match");
  CHECK(atomic_load(&x) == 7,
        "the successful exchange stored the desired value");
  expect = 40;
  CHECK(!atomic_compare_exchange_strong(&x, &expect, 9),
        "compare_exchange fails on a mismatch");
  CHECK(expect == 7,
        "the failed exchange wrote the actual value into expected");
  atomic_store(&gate, 7);
  int c = atomic_load_explicit(&gate, memory_order_consume);
  CHECK(c == 7, "a consume load reads the current value");
  return 0;
```

listing 151: `compare_exchange` both ways, the expected writeback, and the consume load (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 67-80)

Line by line: the default `atomic_store` and `atomic_load` carry no ordering argument, so they are `seq_cst`. `fetch_add` returns the old value, 5, and the load sees 15. The relaxed `fetch_sub` and `fetch_or` still count exactly, because atomicity is not an ordering property. `exchange` swaps and reports what was there. `compare_exchange_strong` succeeds against a matching expectation, and on the mismatch it does the quiet thing worth pinning: the failed call writes the actual value, 7, back into `expected`, which is how retry loops learn what moved. The final load names `memory_order_consume`, which reads 7 like any other load, and whose emitted code is chapter evidence in the next section.

PITFALL: generics are macros, brace your arguments

C23 specifies the atomic operations as generic functions, and generic functions in C are function-like macros. The consequence bit this chapter's own probe: `atomic_exchange(&w, (wide_pair){3, 4})` fails with "too many arguments provided to function-like macro invocation", because the comma inside the braced initializer splits

> the macro argument. The compiler's own note gives the fix: parenthesize the whole initializer, `((wide_pair) {3, 4})`. The same rule applies to every `_Generic`-based family in the library.

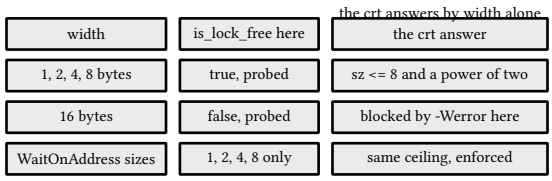


figure 61: the width taxonomy: what the crt answers, per width, and where each api caps out

16.2. the happens-before ladder

`memory_order` names six constants, and the ladder from weakest to strongest is the entire mental model. cppreference's page defines each rung. Relaxed: "there are no synchronization or ordering constraints imposed on other reads or writes, only this operation's atomicity is guaranteed", and again, relaxed operations "only guarantee atomicity and modification order consistency." Acquire on a load: "no reads or writes in the current thread can be reordered before this load. All writes in other threads that release the same atomic variable are visible in the current thread." Release on a store: "no reads or writes in the current thread can be reordered after this store. All writes in the current thread are visible in other threads that acquire the same atomic variable." `acq_rel` gives a read-modify-write both halves. `seq_cst` is the default and adds the global property: "a single total order exists in which all threads observe all modifications in the same order." Consume exists in the enum but is hollow here: "no known production compilers track dependency chains: consume operations are lifted to acquire", and the ir pin proves the lift on this compiler, the consume load of `@gate` emits `load atomic i32, ptr @gate acquire`.

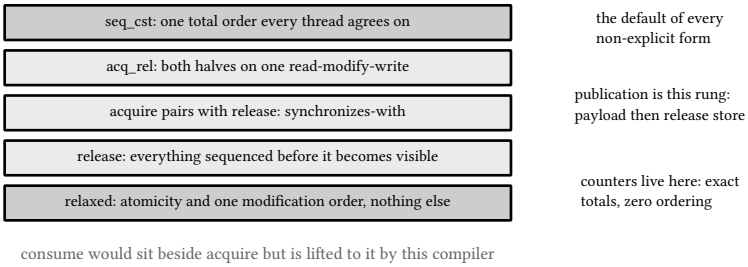


figure 62: the happens-before ladder, weakest at the bottom, what each rung adds

The first demonstration is deliberately unglamorous. Two threads each add 100000 to one counter with `memory_order_relaxed`, and the total is exactly 200000:

```c
static _Atomic int counter;

static int relaxed_adder(void *arg) {
  (void)arg;
  for (int i = 0; i < ADD_ROUNDS; i++)
    atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);
  return 0;
}
```

listing 152: relaxed adds: atomicity without ordering, no lost updates (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 87-94)

The count is exact because every read-modify-write is one indivisible step, the property relaxed does guarantee. The ordering it does not guarantee is the one the next rung demonstrates. An `atomic_flag` spinlock is the textbook acquire/release pair, and the data it guards is plain, non-atomic memory:

```
static atomic_flag spin = ATOMIC_FLAG_INIT;
static int guarded;

static int locked_adder(void *arg) {
  (void)arg;
  for (int i = 0; i < ADD_ROUNDS; i++) {
    while (atomic_flag_test_and_set_explicit(&spin, memory_order_acquire))
      thrd_yield();
    guarded++;
    atomic_flag_clear_explicit(&spin, memory_order_release);
  }
  return 0;
}
```

listing 153: the `atomic_flag` lock: `test_and_set` acquires, `clear` releases (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 99-111)

`guarded++` has no atomic in sight. The plain increments are correct between threads because `test_and_set` with `memory_order_acquire` pairs with `clear` with `memory_order_release`, the same pairing cppreference names for mutual exclusion locks: when the lock is released by one thread and acquired by another, everything before the release is visible after the acquire. Two threads, 100000 guarded increments each, and the check reads exactly 200000. The publication demo makes the mechanism one-shot and even more minimal:

```
static _Atomic int published;
static int payload[3];

static int publisher(void *arg) {
  (void)arg;
  payload[0] = 11;
  payload[1] = 22;
  payload[2] = 33;
  atomic_store_explicit(&published, 1, memory_order_release);
  return 0;
}
```

listing 154: the publisher: plain payload first, one release store last (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 116-126)

```
  thrd_t t;
  int rt;
  CHECK(thrd_create(&t, publisher, NULL) == thrd_success,
        "the publisher thread starts");
  // no join yet: the acquire load alone must make the payload visible
  while (atomic_load_explicit(&published, memory_order_acquire) == 0)
    thrd_yield();
  CHECK(payload[0] == 11 && payload[1] == 22 && payload[2] == 33,
        "the release store publishes the plain payload to the acquire load");
  thrd_join(t, &rt);
  return 0;
```

listing 155: the reader: an acquire load is the only synchronization, no join yet (source: c-os-cloud/samples/src/Ch16/atomics.c, lines 151-161)

Main does not join before reading. It spins on the acquire load, and the release store is the only event that could have made `payload` visible, so the check of all three plain values is a direct assertion of synchronizes-with, cppreference's term for what a release store and an acquire load of the same variable establish.

The ladder's rungs differ far less on this cpu than on paper. The page states why: "On strongly-ordered systems, x86, SPARC TSO, IBM mainframe, etc., release-acquire ordering is automatic for the majority of operations. No

additional CPU instructions are issued for this synchronization mode; only certain compiler optimizations are affected." The objdump pins in `Ch16/expect-ir.txt` show exactly that at `-O0`: every `fetch_add`, whatever its ordering argument, `seq_cst`, `acq_rel`, or relaxed, lowers to `lock xaddl`, acquire loads and release stores compile to the same `movl` as relaxed ones, and `compare_exchange` is `lock cmpxchgl`. The one visible upgrade is the `seq_cst` store, which lowers to `xchgl`, an implicitly locked exchange: the page again, "Total sequential ordering requires a full memory fence CPU instruction on all multi-core systems." On x86-64 the ladder is a compiler contract whose hardware bill is almost always zero, except on the top rung's stores. One more inherited default belongs in the margin: the cc1 transcript that carries `-relaxed-aliasing` and `-fwrapv` also carries `-fms-volatile`, and cppreference documents the microsoft reading it enables, "every volatile write has release semantics and every volatile read has acquire semantics" under default msvc settings, one more reason the atomics layer is the only portable ordering tool this book uses.

16.3. a lock-free spsc ring

The single-producer single-consumer ring puts the ladder to work in both directions at once. Capacity is a power of two so the slot index is a mask, the counters are unbounded so full and empty are `t - h == CAP` and `t == h` with no wasted slot, and the two indices live on different cache lines because the producer writes one and the consumer writes the other:

```
struct ring {
  _Alignas(64) _Atomic unsigned head;
  char pad0[64 - sizeof(unsigned)];
  _Atomic unsigned tail;
  char pad1[64 - sizeof(unsigned)];
  int buf[CAP];
};
```

listing 156: the ring: two atomic indices, padded to separate cache lines (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 29-35)

The producer owns `tail` and the consumer owns `head`, so each side reads its own index relaxed and the other side's with acquire, and publishes its own with release:

```
static int ring_push(struct ring *r, int v) {
  unsigned t = atomic_load_explicit(&r->tail, memory_order_relaxed);
  unsigned h = atomic_load_explicit(&r->head, memory_order_acquire);
  if (t - h == CAP)
    return 0;
  r->buf[t & MASK] = v;
  atomic_store_explicit(&r->tail, t + 1, memory_order_release);
  return 1;
}
```

listing 157: push: relaxed own tail, acquire consumer head, release new tail (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 45-53)

```
static int ring_pop(struct ring *r, int *out) {
  unsigned h = atomic_load_explicit(&r->head, memory_order_relaxed);
  unsigned t = atomic_load_explicit(&r->tail, memory_order_acquire);
  if (t == h)
    return 0;
  *out = r->buf[h & MASK];
  atomic_store_explicit(&r->head, h + 1, memory_order_release);
  return 1;
}
```

listing 158: pop: relaxed own head, acquire producer tail, release new head (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 56-64)

Line by line, push loads its own tail relaxed, loads the consumer's head with acquire so any slot the consumer freed is safe to overwrite, refuses when the ring holds CAP items, writes the slot, and only then releases the new tail. The payload write lands before the release store, and the consumer's acquire load of tail drags the payload into visibility: the same publication pairing from the ladder, run at production tempo. pop mirrors it. One thread at a time may call each side, which is what spsc means. The atomics make the two sides safe against each other without any lock, kernel call, or waiting.

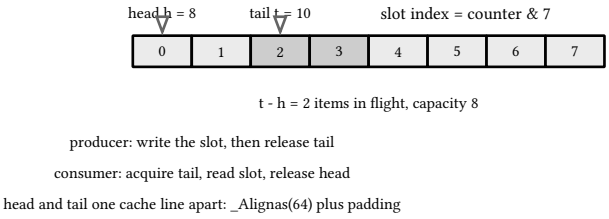


figure 63: the ring with two items in flight: counters, mask, and who publishes what

The single-thread leg pins the container itself before any concurrency enters. The ring fills to eight, refuses the ninth, drains 1 through 8 in order, refuses the empty pop, and then twenty more items ride through one at a time:

```
static int single_thread(struct ring *r) {
  CHECK(atomic_is_lock_free(&r->head),
        "the ring indices are 4 byte lock free atomics");
  int pushed = 0;
  for (unsigned i = 0; i < CAP; i++)
    pushed += ring_push(r, (int)(i + 1));
  CHECK(pushed == (int)CAP, "the empty ring accepts 8 items");
  CHECK(!ring_push(r, 99), "the full ring refuses the ninth");
  int v = 0, drained = 0, inorder = 1;
  for (unsigned i = 0; i < CAP; i++) {
    if (!ring_pop(r, &v))
      break;
    drained++;
    inorder = inorder && v == (int)(i + 1);
  }
  CHECK(drained == (int)CAP && inorder, "the full ring drains 1..8 in order");
  CHECK(!ring_pop(r, &v), "the empty ring refuses a pop");
```

listing 159: the single-thread self pass: full, empty, in-order drain (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 66-82)

```
  // wrap: push and pop one at a time; the counters climb past CAP while the
  // masked index keeps landing in 0..7
  int wrapped = 0, ok = 1;
  for (int i = 0; i < 20; i++) {
    if (!ring_push(r, 100 + i) || !ring_pop(r, &v) || v != 100 + i) {
      ok = 0;
      break;
    }
    wrapped++;
  }
  CHECK(ok && wrapped == 20, "indices wrap the mask, 20 more items in order");
  CHECK(atomic_load(&r->head) == 28 && atomic_load(&r->tail) == 28,
        "the counters passed CAP unbounded, only indexing wraps");
```

listing 160: the wrap leg: counters climb past CAP, only indexing wraps (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 83-95)

After the wrap leg both counters read 28 against a capacity of 8: the mask does the wrapping, the counters never do, which is why full and empty never need modular arithmetic beyond the subtraction. The two-thread leg is the real claim, 64 items through 8 slots, eight laps:

```
static int producer(void *arg) {
  struct ring *r = arg;
  for (int i = 1; i ≤ ITEMS; i++)
    while (!ring_push(r, i))
      thrd_yield();
  return 0;
}
```

listing 161: the producer: spin while full, yield (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 109-115)

```
static int consumer(void *arg) {
  struct transfer *t = arg;
  int expect = 1, v;
  while (expect ≤ ITEMS) {
    if (ring_pop(t->r, &v)) {
      if (v ≠ expect)
        t->inorder = 0;
      expect++;
    } else {
      thrd_yield();
    }
  }
  t->received = expect - 1;
  return 0;
}
```

listing 162: the consumer: spin while empty, demand in-order arrival (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 117-131)

```
  ring_init(&r);
  struct transfer tr = {&r, 0, 1};
  thrd_t prod, cons;
  int rp, rc;
  CHECK(thrd_create(&prod, producer, &r) == thrd_success,
        "the producer thread starts");
  CHECK(thrd_create(&cons, consumer, &tr) == thrd_success,
        "the consumer thread starts");
  thrd_join(prod, &rp);
  thrd_join(cons, &rc);
  CHECK(tr.received == ITEMS, "the consumer received all 64 items");
  CHECK(tr.inorder, "every item arrived in producer order");
  int v;
  CHECK(!ring_pop(&r, &v), "the ring is empty after the transfer");
  return 0;
```

listing 163: the transfer: all 64 received, in producer order, ring empty after (source: c-os-cloud/samples/src/Ch16/spsc.c, lines 139-153)

Both sides spin and `thrd_yield` rather than block, which is honest for a sample but wasteful in production: a spinning consumer burns a core the moment the ring runs dry. Closing that gap is the next section's api.

16.4. WaitOnAddress versus the futex

The spin loop is the only piece of the ring a real system replaces. The windows primitive is `WaitOnAddress`, four parameters and one promise: "If the value at Address differs from the value at CompareAddress, the function returns immediately. If the values are the same, the function does not return until another thread in the same process signals that the value at Address has changed by calling WakeByAddressSingle or WakeByAddressAll or the timeout elapses, whichever comes first."

The sample pins the semantics single-threaded, `GetLastError` read after every call:

```
CHECK(ERROR_TIMEOUT == 1460, "ERROR_TIMEOUT is the pinned code 1460");
CHECK(ERROR_INVALID_PARAMETER == 87,
      "ERROR_INVALID_PARAMETER is the pinned code 87");
atomic_store(&value, 5);
LONG same = 5;
SetLastError(0);
BOOL r = WaitOnAddress(&value, &same, sizeof value, 0);
CHECK(r == FALSE && GetLastError() == ERROR_TIMEOUT,
      "unchanged value, zero timeout: FALSE with ERROR_TIMEOUT");
LONG other = 99;
SetLastError(0);
r = WaitOnAddress(&value, &other, sizeof value, 0);
CHECK(r == TRUE, "a differing value returns TRUE without waiting");
```

listing 164: the pinned error codes and the first two polls (source: c-os-cloud/samples/src/Ch16/waitaddr.c, lines 28-40)

```
SetLastError(0);
r = WaitOnAddress(&value, &same, 16, 0);
CHECK(r == FALSE && GetLastError() == ERROR_INVALID_PARAMETER,
      "a 16 byte size is rejected with ERROR_INVALID_PARAMETER");
// the comparison runs before the wait: a value that already changed makes
// even an INFINITE wait return immediately, no wake required
atomic_store(&value, 7);
SetLastError(0);
r = WaitOnAddress(&value, &same, sizeof value, INFINITE);
CHECK(r == TRUE, "a changed value beats INFINITE: the wait returns at once");
```

listing 165: the width rejection and the changed-before-INFINITE probe (source: c-os-cloud/samples/src/Ch16/waitaddr.c, lines 41-50)

Four probes, four exact answers. An unchanged value with a zero timeout returns FALSE with `GetLastError` reporting `ERROR_TIMEOUT`, 1460, the pair the page documents, “if the operation times out, GetLastError returns `ERROR_TIMEOUT`.” A value that already differs returns TRUE with no wait at all. A 16-byte size returns FALSE with `ERROR_INVALID_PARAMETER`, 87, and the page fixes the widths, `AddressSize` “can be 1, 2, 4, or 8”, the same ceiling as the crt’s lock-free predicate, exported from Synchronization.lib, which is why the gate passes `-lsynchronization`, as chapter 1 noted. And a value that changed before an INFINITE wait returns TRUE immediately, because the comparison runs before any sleeping, the poll-then-wait shape. The page supplies the discipline that follows from one more property, WaitOnAddress “is guaranteed to return when the address is signaled, but it is also allowed to return for other reasons”, so “after WaitOnAddress returns the caller should compare the new value with the original undesired value to confirm that the value has actually changed”, in a loop, and the page’s own example is exactly that loop:

```
static int waiter(void *arg) {
  (void)arg;
  int undesired = 0;
  int captured = atomic_load(&value);
  atomic_store(&armed, 1);
  while (captured == undesired) {
    BOOL ok = WaitOnAddress(&value, &undesired, sizeof value, INFINITE);
    if (!ok)
      printf("waitonaddress returned FALSE, last error %lu\n", GetLastError());
    wait_ok = ok;
    captured = atomic_load(&value);
  }
  observed = captured;
  return 0;
}
```

listing 166: the waiter: capture, announce, wait, re-check, exactly the documented loop (source: c-os-cloud/samples/src/Ch16/waitaddr.c, lines 62-76)

The armed handshake makes the two-thread check deterministic. The waiter captures the value, then stores `armed`, then enters the loop, so main knows the capture happened before it changes anything. Whether main's store and wake win the race or trail it, `WaitOnAddress` is entered with the old value in hand, and the loop's re-check reads the new one:

```
  atomic_store(&value, 0);
  atomic_store(&armed, 0);
  thrd_t t;
  int rt;
  CHECK(thrd_create(&t, waiter, NULL) == thrd_success,
        "the waiter thread starts");
  while (atomic_load_explicit(&armed, memory_order_acquire) == 0)
    thrd_yield();
  atomic_store(&value, 7);
  WakeByAddressAll(&value);
  thrd_join(t, &rt);
  CHECK(wait_ok == TRUE, "the woken wait returns TRUE");
  CHECK(observed == 7, "the waiter observed the new value 7");

  // waking an address nobody waits on is harmless: the call is void
  WakeByAddressAll(&value);
  CHECK(atomic_load(&value) == 7, "the value survives a wake with no waiters");
```

listing 167: store, wake all, join: TRUE and the observed value 7 (source: c-os-cloud/samples/src/Ch16/waitaddr.c, lines 82-98)

`WakeByAddressAll` "wakes all threads that are waiting for the value of an address to change", returns nothing, and waking an address nobody waits on is harmless, probed. Its page adds the scope: "Only threads within the same process can be woken." The page argues the placement too: WaitOnAddress "is more efficient than using the Sleep function inside a while loop because WaitOnAddress does not interfere with the thread scheduler", and "is also simpler to use than an event object because it is not necessary to create and initialize an event and then make sure it is synchronized correctly with the value."

Linux has the same primitive one level down, and the man page names the concept: a futex is "short for 'Fast user-space mutexes'", "identified by a piece of memory which can be shared between processes or threads", "a 32-bit value", and the kernel side blocks conditionally, "the kernel will block only if the futex word has the value that the calling thread supplied", with "the comparison of that value with the expected value, and the actual blocking" happening atomically. The futex(7) page states the layering that explains both systems: "Most programmers will in fact not be using futexes directly" but rely on system libraries built on them, such as NPTL, and "Futex operation

occurs entirely in user space for the noncontended case. The kernel is involved only to arbitrate the contended case." Windows' built path is chapter 15′s condition variable: `SleepConditionVariableSRW` "sleeps on the specified condition variable and releases the specified lock as an atomic operation", with the same loop discipline, its page warns of "spurious wakeups" and "stolen wakeups" and says "you should recheck a predicate (typically in a while loop) after a sleep operation returns."

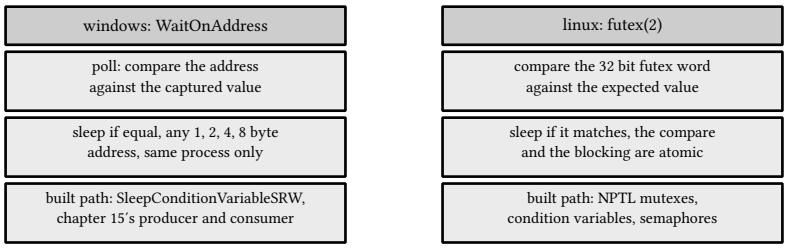


figure 64: the same primitive on both systems: compare in user space, sleep only when contended

The ring in the previous section plus this section's waiter is the complete pattern production code uses: spin on the atomic indices, and when the ring is full or empty, sleep on the index address until the other side publishes. Chapter 17 leaves the single machine for the scheduler's own model, where the thread's wait, ready, and run states become the unit of study.

sources: learn.microsoft.com, WaitOnAddress, WakeByAddressAll, and SleepConditionVariableSRW pages, accessed 2026-09-12; man7.org futex(2) and futex(7), accessed 2026-09-12; en.cppreference.com memory order page, accessed 2026-09-12; the header trail, the search order, the lock-free predicate, the enum values, the `char8_t` absence, the `-Watomic-alignment` block, and the instruction-level lowers probed on this machine the same day. Sample behavior verified by `make verify-c`, 41 checks in chapter 16 of the samples suite.

17. cpu scheduling algorithms

Chapter 15 ended on a division of labor: the program owns the predicate and the lock, the scheduler owns the order. This chapter opens the scheduler's side. Seven policies run the same four jobs through the same harness, every turnaround, response, and wait triple below was computed by hand from a unit-by-unit trace before it became a `CHECK`, and one sample drives real Windows threads to show the one scheduling knob the kernel actually sells. The policy zoo is not folklore here: each algorithm's numbers on the fixed set are frozen in its source, the gate recomputes them on every run, and the closing table lines all ten rows up side by side.

The seven simulators are pure user-space arithmetic, one translation unit each over one shared header. Nothing in them calls the operating system, which is the point: a scheduler policy is a function from (arrival, burst) histories to a dispatch order, and a function that calls the kernel cannot be checked deterministically. The kernel enters twice, both times labeled: `setpriority.c` proves `SetThreadPriority`'s contract on live threads, and the linux facts are quoted from canonical pages, cited and never executed, the treatment chapter 11 gave `fork`.

17.1. the model and the metrics

One job is five numbers and three fields of simulator state:

```
// one job of the fixed set. priority runs 1 first (the unix nice
// direction). nice only feeds the weighted fair scheduler of cfs.c.
// arrival times are distinct and the table is sorted by them, which the
// non-preemptive samples rely on when they walk it in order.
struct job {
  char name;
  int arrival;
  int burst;
  int priority;
  int nice;
  // simulator state, filled by whichever algorithm drives the set
  int remaining;
  int first;  // first dispatch time, -1 while never run
  int finish; // completion time, -1 while unfinished
};

static const struct job FIXED[NJOBS] = {
    {'A', 0, 7, 2, 0, 0, -1, -1},
    {'B', 2, 4, 1, -1, 0, -1, -1},
    {'C', 4, 1, 4, 1, 0, -1, -1},
    {'D', 5, 4, 3, 0, 0, -1, -1},
};
```

listing 168: the job record and the fixed four-job set every algorithm drives (source: c-os-cloud/samples/src/Ch17/sched.h, lines 13-34)

The fixed set is the whole chapter's universe: A arrives at 0 wanting 7 units, B at 2 wanting 4 at the highest priority, C at 4 wanting 1 at the lowest, D at 5 wanting 4. Total work is 16 units, so on a cpu that never idles every algorithm finishes at 16, and every difference between the rows of the closing table is a difference in ordering, not in total work. The `nice` field only feeds the weighted fair scheduler; its sign convention, lower value more cpu, is the unix one: POSIX defines the nice value as "a non-negative number" where "a more positive value shall result in less favorable scheduling" and "Only a process with appropriate privileges can lower the nice value", linux runs the same direction over −20 to 19, setpriority(2)'s range with −20 the highest, and the last section runs the arithmetic on it. The three metrics are pure functions of the record:

```
// one metric for one job: 't' turnaround (finish - arrival), 'r' response
// (first - arrival, submission to first cpu touch), 'w' wait (turnaround
// minus burst, the time the job existed without running)
static int sched_one(const struct job *j, char which) {
  int tat = j->finish - j->arrival;
  if (which == 't')
    return tat;
  if (which == 'r')
    return j->first - j->arrival;
  return tat - j->burst;
}
```

listing 169: turnaround, response, wait: finish, first, arrival, burst combined (source: c-os-cloud/samples/src/Ch17/sched.h, lines 46-56)

Turnaround is `finish - arrival`, the job's whole lifetime from submission to completion. Response is `first - arrival`, submission to first cpu touch, the metric an interactive system feels. Wait is turnaround minus burst, the time the job existed without running. Each simulator also records its dispatch order as one letter per unit, the string `AAAAAAABBBBCDDDD` and its rivals, and the header counts context switches off that string, adjacent differing letters, so every algorithm is metered the same way.

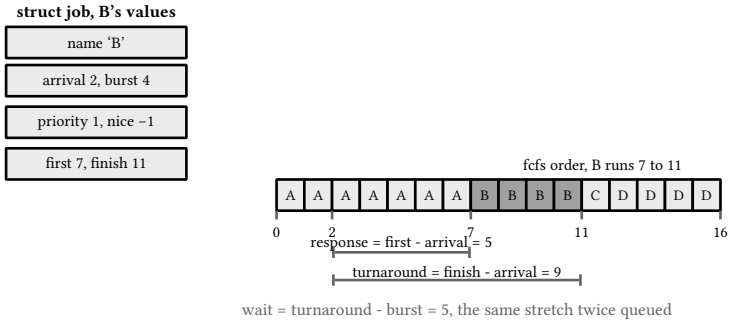


figure 65: the model on one job: B's record, and B's three metrics marked on the fcfs timeline

VERIFY: the hand traces were wrong twice before they were right

The first version of this chapter's constants held a round-robin trace that gave job A 9 units of cpu for a 7-unit burst, and a priority trace that re-ran units A had already spent. Both were hand-derived, both looked plausible, and both disagreed with the simulators. Recounting from the order strings proved the sims right both times: hand arithmetic that reuses a job's already-run prefix is exactly the mistake the `CHECK` count exists to catch. Every pair in every sample below survived that recount.

17.2. fcfs, sjf, srtf

First come first served is the null policy: the table is sorted by arrival, so walking it in order and running each burst to completion is the whole algorithm.

```
static void run_fcfs(struct job *js, char *order) {
  int t = 0, k = 0;
  for (int i = 0; i < NJOBS; i++) {
    if (js[i].arrival > t)
      t = js[i].arrival;
    js[i].first = t;
    for (int u = 0; u < js[i].burst; u++)
      order[k++] = js[i].name;
    t += js[i].burst;
    js[i].finish = t;
  }
  order[k] = 0;
}
```

listing 170: fcfs: arrival order, each burst runs to completion (source: c-os-cloud/samples/src/Ch17/fcfs.c, lines 25-37)

A runs 0-7, B 7-11, C 11-12, D 12-16. The convoy effect is visible in one number: C has a 1-unit burst and waits 7 units for it, because it had the bad luck to arrive behind A's 7. Averages over the four jobs: turnaround 8.75, response 4.75, wait 4.75, with 3 switches. Shortest job first fixes exactly that, at a price:

```
static void run_sjf(struct job *js, char *order) {
  int t = 0, k = 0, done = 0;
  while (done < NJOBS) {
    int pick = -1;
    for (int i = 0; i < NJOBS; i++) {
      if (js[i].arrival > t || js[i].finish >= 0)
        continue;
      if (pick < 0 || js[i].burst < js[pick].burst)
        pick = i;
    }
    if (pick < 0) {
      order[k++] = '.';
      t++;
      continue;
    }
    js[pick].first = t;
    for (int u = 0; u < js[pick].burst; u++)
      order[k++] = js[pick].name;
    t += js[pick].burst;
    js[pick].finish = t;
    done++;
  }
  order[k] = 0;
}
```

listing 171: sjf: at each completion the shortest arrived burst runs next (source: c-os-cloud/samples/src/Ch17/sjf.c, lines 47-71)

At time 7 the ready set is B (burst 4), C (burst 1), D (burst 4). C runs 7-8, then the B and D tie at burst 4 resolves to the earlier arrival, B 8-12, D 12-16. Turnaround sum 32 beats fcfs's 35, response 16 beats 19. The price is clairvoyance: the scheduler must know each burst before running it, which real systems do not. The sample then settles the optimality question the honest way, by exhausting all 24 orders of the four jobs:

```
int idx[NJOBS] = {0, 1, 2, 3}, any_best = 1 << 30, busy_best = 1 << 30,
    ties = 0;
do {
  int t = 0, sum = 0;
  for (int p = 0; p < NJOBS; p++) {
    struct job *j = &js[idx[p]];
    if (j->arrival > t)
      t = j->arrival;
    t += j->burst;
    sum += t - j->arrival;
  }
  if (sum < any_best)
    any_best = sum;
  if (js[idx[0]].arrival == 0) { // never idles with work waiting
    if (sum < busy_best) {
      busy_best = sum;
      ties = 1;
    } else if (sum == busy_best) {
      ties++;
    }
  }
} while (next_perm(idx, NJOBS));
```

listing 172: sjf optimality by exhaustion: idle-allowed best 31, never-idle best 32 (source: c-os-cloud/samples/src/Ch17/sjf.c, lines 92-113)

Two facts fall out. Among non-preemptive orders that never idle while work exists, which on this set are the orders starting with A, no order beats sjf's 32, and exactly two tie. But the global best over all 24 orders is 31, uniquely B, C, D, A: that schedule idles from 0 to 2 on purpose, declines to start the long job that is sitting there, and waits for the short one. sjf is optimal among busy schedules, not among all schedules; a clairvoyant scheduler that sleeps in waits less on average. Shortest remaining time first is sjf with preemption, and preemption is one changed compare:

```
    int pick = -1;
    for (int i = 0; i < NJOBS; i++) {
      if (js[i].arrival > t || js[i].remaining == 0)
        continue;
      if (pick < 0 || js[i].remaining < js[pick].remaining)
        pick = i;
    }
    if (pick < 0) {
      order[k++] = '.';
      t++;
      continue;
    }
    if (js[pick].first < 0)
      js[pick].first = t;
    js[pick].remaining--;
    order[k++] = js[pick].name;
    t++;
    if (js[pick].remaining == 0) {
      js[pick].finish = t;
      done++;
    }
  }
```

listing 173: srtf: every unit the least remaining runs, arrivals preempt by arithmetic (source: c-os-cloud/samples/src/Ch17/srtf.c, lines 28-49)

No preemption code exists. The unit loop simply re-picks, and when B arrives at 2 with 4 units against A's remaining 5, the compare switches sides by itself. The chain: A 0-2, B preempts at 2, C's arrival at 4 (remaining 1 under B's 2) preempts again, B drains 5-7, D outranks A's 5 remaining with 4 at 7, and A's tail lands last, 11-16. Response collapses from sjf's 16 to 2, B, C, and D each touch the cpu the moment they arrive or within 2 units, while A's turnaround balloons to 16, the whole makespan: srtf buys response with the long job's tail.

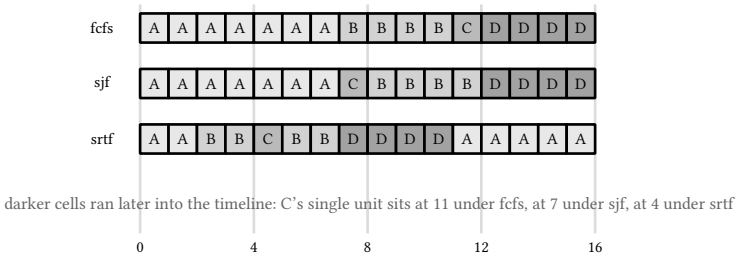


figure 66: the three batch policies as dispatch orders over the same 16 units

17.3. round robin, priority with aging

Round robin replaces the burst with a quantum: the fifo head runs at most `quantum` units, then requeues. One convention decides every trace this section reports, and it sits in three lines:

```
while (done < NJOBS) {
  while (arrived < NJOBS && js[arrived].arrival ≤ t)
    q[qt++] = arrived++;
  if (cur ≥ 0 && left == 0) // quantum expired: requeue behind arrivals
    q[qt++] = cur, cur = -1;
  if (cur < 0) {
    if (qh == qt) {
      order[k++] = '.';
      t++;
      continue;
    }
    cur = q[qh++];
    left = quantum;
    if (js[cur].first < 0)
      js[cur].first = t;
  }
```

listing 174: the rr fifo: arrivals enter before a preempted job requeues at the same instant (source: c-os-cloud/samples/src/Ch17/rr.c, lines 32-47)

At each tick boundary, jobs arriving at that instant join the fifo first, and only then does a job whose quantum just expired requeue behind them. At time 4, C's arrival enters before B's requeue, so the dispatch order from there is A, C, B. A quantum expiring exactly on an arrival puts the newcomer ahead: the alternative convention changes traces, and pinning one is the price of exact numbers. With quantum 2 the trace runs A 0-2, B 2-4, A 4-6, C 6-7, B 7-9, D 9-11, A 11-13, D 13-15, A 15-16: turnaround sum 36, response sum 6, 8 switches. The quantum is a dial, and the sample turns it:

```
// the cost model: same set, three dials. switch counts hand-checked
// from the dispatch lists, response sums from the traces
for (int qtm = 1; qtm ≤ 4; qtm *= 2) {
  char label[16];
  sched_fixed(js);
  run_rr(js, order, qtm);
  sprintf(label, "rr q=%d", qtm);
  sched_report(label, order, js, NJOBS);
  printf("quantum %d switches %d response sum %d turnaround sum %d\n", qtm,
         sched_switches(order), sched_sum(js, NJOBS, 'r'),
         sched_sum(js, NJOBS, 't'));
}
sched_fixed(js);
run_rr(js, order, 1);
CHECK(sched_switches(order) == 14, "q=1 costs 14 switches");
CHECK(sched_sum(js, NJOBS, 'r') == 3, "q=1 response sum 3");
sched_fixed(js);
run_rr(js, order, 4);
CHECK(sched_switches(order) == 4, "q=4 costs 4 switches");
CHECK(sched_sum(js, NJOBS, 'r') == 13, "q=4 response sum 13");
```

listing 175: the quantum cost model: q=1, 2, 4 on the same set, verdicts checked (source: c-os-cloud/samples/src/Ch17/rr.c, lines 75-94)

At q=1 the response sum is 3 but the cpu changes hands 14 times; at q=4 response rises to 13 and switches fall to 4; turnaround drifts from 38 down to 34 as the quantum grows toward batch behavior. Every switch is pipeline state thrown away, which is why the dial has no right setting, only a workload-dependent one. Windows states its own setting in its model: the Scheduling Priorities page says "The system treats all threads with the same priority as equal. The system assigns time slices in a round-robin fashion to all threads with the highest priority", and the SetThreadPriority page repeats the mechanism, "Threads are scheduled in a round-robin fashion at each priority

level, and only when there are no executable threads at a higher level does scheduling of threads at a lower level take place." Round robin at 32 priority levels is the real os's within-level rule, not just a teaching toy.

Priority scheduling replaces the fifo with a ranking. This chapter's policy is preemptive: every unit, the arrived unfinished job with the smallest effective priority runs, ties keeping the earlier arrival. On the fixed set, B (priority 1) preempts A at 2 and finishes at 6, A resumes 6-11, D runs 11-15, C's single unit lands 15-16: turnaround sum 37, response sum 17. C's response of 11 is worse than fcfs's 7, the price of D and A outranking it. The real cost of strict priority is elsewhere and the second scenario measures it: one 20-unit job L at priority 3 arrives at 1, and a stream of ten 2-unit jobs at priority 1 arrives every 2 units from 2 on. Under strict priority L first touches the cpu at 22 and finishes at 42, waiting 21 units. Aging deletes the starvation with arithmetic:

```c
// aging: one priority step back for every two units waited since
// arrival, floored at 1. arithmetic at the decisive instant: waited 4,
// 4/2 = 2, static 3 drops to 1 and ties the stream.
static int eff_aging(const struct job *j, int now) {
  int waited = now - j->arrival;
  int drop = waited / 2;
  if (drop > j->priority - 1)
    drop = j->priority - 1;
  return j->priority - drop;
}
```

listing 176: aging: one priority step per two units waited, floored at 1 (source: c-os-cloud/samples/src/Ch17/prio.c, lines 32-41)

```c
  // the starvation set: X high, L long and low, H1..H10 a high-priority
  // stream arriving every 2 units. strict priority starves L for 21
  // units; aging runs it at 5.
  struct job starve[12];
  starve[0] = (struct job){'X', 0, 2, 1, 0, 2, -1, -1};
  starve[1] = (struct job){'L', 1, 20, 3, 0, 20, -1, -1};
  for (int i = 2; i < 12; i++)
    starve[i] = (struct job){'H', (i - 2) * 2 + 2, 2, 1, 0, 2, -1, -1};

  run_prio(starve, 12, eff_static, order);
  printf("strict: L first %d finish %d\n", starve[1].first, starve[1].finish);
  CHECK(starve[1].first == 22, "strict priority: L first touches cpu at 22");
  CHECK(starve[1].finish == 42, "strict priority: L finishes at 42");

  for (int i = 0; i < 12; i++) { // reset for the aged run
    starve[i].remaining = starve[i].burst;
    starve[i].first = -1;
    starve[i].finish = -1;
  }
  run_prio(starve, 12, eff_aging, order);
  printf("aged: L first %d finish %d\n", starve[1].first, starve[1].finish);
  CHECK(starve[1].first == 5, "aged: L ties the stream at 1 and runs at 5");
  CHECK(starve[1].finish == 25, "aged: L finishes at 25");
```

listing 177: the starvation set and both verdicts: L waits 21 under strict priority, runs at 5 under aging (source: c-os-cloud/samples/src/Ch17/prio.c, lines 87-109)

At time 5, L has waited 4 units, 4/2 = 2 steps, static 3 drops to an effective 1 that ties the stream, and the arrival tie-break hands L the cpu. L runs 5-25 and the stream finishes behind it at 42. One division moved 17 units of wait, and the aging rule is the whole difference between the two printed verdicts. Windows pushes on the same wound from the other side: the SetThreadPriority page says of background mode, "If there are threads executing at high priority, a thread in background processing mode may not be scheduled promptly, but it will never be starved",

and of the dynamic layer, "the system dynamically boosts a thread's base priority level when events occur that are important to the thread."

The knob itself is real, small, and checkable. Its page fixes the model: the priority value "together with the priority class of the thread's process, determines the thread's base priority level", and "All threads initially start at THREAD_PRIORITY_NORMAL". For a NORMAL_PRIORITY_CLASS process the base table runs 1, 6, 7, 8, 9, 10, 15 across idle, lowest, below normal, normal, above normal, highest, time critical: the middle band is class value plus offset, the ends pinned at 1 and 15.

```
// the knob's contract on the calling thread
CHECK(GetThreadPriority(cur) == THREAD_PRIORITY_NORMAL,
      "every thread starts at normal priority");
CHECK(SetThreadPriority(cur, THREAD_PRIORITY_HIGHEST),
      "set highest on the current thread");
CHECK(GetThreadPriority(cur) == 2, "highest reads back 2");
CHECK(SetThreadPriority(cur, THREAD_PRIORITY_IDLE), "set idle");
CHECK(GetThreadPriority(cur) == -15, "idle reads back -15");
CHECK(SetThreadPriority(cur, THREAD_PRIORITY_NORMAL), "restore normal");
CHECK(GetThreadPriority(cur) == 0, "normal reads back 0");

// background mode: begin, refuse a second begin, end
CHECK(SetThreadPriority(cur, THREAD_MODE_BACKGROUND_BEGIN),
      "background mode begins");
CHECK(!SetThreadPriority(cur, THREAD_MODE_BACKGROUND_BEGIN),
      "a second begin fails while already in background mode");
CHECK(SetThreadPriority(cur, THREAD_MODE_BACKGROUND_END),
      "background mode ends");
```

listing 178: the knob contract on live threads: set, read back, background begin, refuse, end (source: c-os-cloud/samples/src/Ch17/setpriority.c, lines 60-77)

Every claim in that listing is a documented behavior, not an observation: the normal start, the readback of 2 and −15, and the documented failure mode of background mode, "The function fails if the thread is already in background processing mode", which is why the second SetThreadPriority call is checked for failure, not success. The contention experiment then shows the knob working:

```
// equal work, one core, two priorities: the contention experiment
struct work wa = {0}, wb = {0};
HANDLE ha = CreateThread(NULL, 0, spin, &wa, CREATE_SUSPENDED, NULL);
HANDLE hb = CreateThread(NULL, 0, spin, &wb, CREATE_SUSPENDED, NULL);
CHECK(ha != NULL && hb != NULL, "both workers created suspended");
CHECK(SetThreadAffinityMask(ha, 1) && SetThreadAffinityMask(hb, 1),
      "both workers pinned to cpu 0");
CHECK(SetThreadPriority(ha, THREAD_PRIORITY_ABOVE_NORMAL) &&
          GetThreadPriority(ha) == 1,
      "worker a set above normal, reads back 1");
CHECK(SetThreadPriority(hb, THREAD_PRIORITY_BELOW_NORMAL) &&
          GetThreadPriority(hb) == -1,
      "worker b set below normal, reads back -1");
LARGE_INTEGER rel, done_a, done_b, freq;
QueryPerformanceCounter(&rel);
wa.go = wb.go = 1;
ResumeThread(ha);
ResumeThread(hb);
```

listing 179: two workers, one pinned core, two priorities: equal work under forced contention (source: c-os-cloud/samples/src/Ch17/setpriority.c, lines 79-96)

Priority is only visible under contention, so both workers are pinned to cpu 0 by `SetThreadAffinityMask` and released together. Both compute the identical checksum over 100 million iterations. Probed on this machine 2026-09-12: the above-normal worker did its work in 41.5 ms and was done at +41.6 ms; the below-normal worker did equivalent work in 42.9 ms and was done at +84.6 ms. The higher priority effectively owned the core until it finished, the page's promise in action: "If a higher-priority thread becomes available to run, the system ceases to execute the lower-priority thread (without allowing it to finish using its time slice)". Those wall-clock numbers are probes, printed and never checked, because shares of a cpu belong to the scheduler, not to the program.

WARNING: the top of the band bites

The SetThreadPriority page draws the line in one sentence: "A thread with a base priority level above 11 interferes with the normal operation of the operating system." The samples above never leave the documented relative band, highest at 10 for a normal-class process, and `REALTIME_PRIORITY_CLASS` appears here only as the table row that disables dynamic boosts.

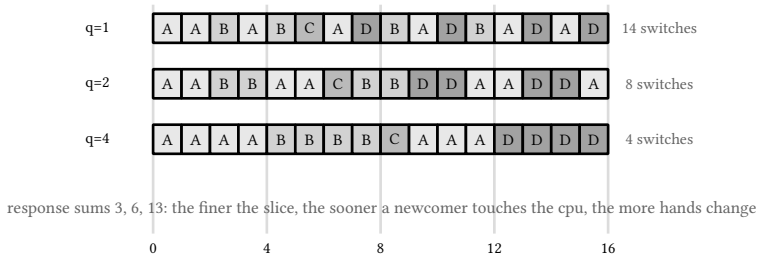


figure 67: the quantum dial: same set, same order discipline, three quanta

17.4. multilevel feedback queues

The policies so far each know one thing: fcfs knows arrival, sjf knows burst, priority knows a rank. A multilevel feedback queue knows nothing up front and learns from the job's own behavior. This chapter fixes the ruleset: three queues with quanta 2, 4, and run to completion; arrivals always enter the top queue; an arrival above the running job's level preempts it, the preempted job keeping its level at that level's tail; a job that spends its whole quantum is demoted one level; and every `promote` time units everything returns to the top.

```
    while (arrived < n && js[arrived].arrival ≤ t)
      push(0, arrived++);
    if (cur ≥ 0) { // preempt if a queue above the running level has work
      for (int lv = 0; lv < level[cur]; lv++)
        if (qh[lv] < qt[lv]) {
          push(level[cur], cur);
          cur = -1;
          break;
        }
    }
    if (cur < 0) {
      for (int lv = 0; lv < LEVELS && cur < 0; lv++)
        if (qh[lv] < qt[lv]) {
          cur = q[lv][qh[lv]++];
          left = QUANTA[level[cur]];
          if (js[cur].first < 0)
            js[cur].first = t;
        }
```

listing 180: arrivals enter the top queue, a higher queue preempts, the highest nonempty head dispatches (source: c-os-cloud/samples/src/Ch17/mlfq.c, lines 71-88)

```
    if (js[cur].remaining == 0) {
      js[cur].finish = t;
      done++;
      cur = -1;
    } else if (left == 0) { // quantum spent: demote, bottom stays bottom
      if (level[cur] < LEVELS - 1)
        level[cur]++;
      push(level[cur], cur);
      cur = -1;
    }
```

listing 181: finish or demote: a spent quantum costs one level, the bottom queue keeps (source: c-os-cloud/samples/src/Ch17/mlfq.c, lines 100-109)

On the fixed set the trace is A 0-2 demoted, B 2-4 demoted, C 4-5 done from the top queue, D 5-7 demoted, then the middle queue drains A 7-11 (demoted again), B 11-13, D 13-15, and A's last unit runs from the bottom, 15-16. The learning is visible: A, the 7-unit job, sinks twice; C, the 1-unit job, rides the top queue to a finish at its arrival instant. Response sum is 0, every job first touched at arrival, something no policy above bought at this turnaround price (sum 38). The promotion reset exists for the job that learns wrong:

```
// every `promote` units: flush all queues back into the top one in
// (level, fifo) order, the running job appended after its level's queued
// peers; every promoted job's level resets to 0 and its quantum restarts
// on the next dispatch
static void promote_all(int cur) {
  int tmp[64], n = 0;
  for (int lv = 0; lv < LEVELS; lv++) {
    while (qh[lv] < qt[lv])
      tmp[n++] = q[lv][qh[lv]++];
    if (cur ≥ 0 && level[cur] == lv)
      tmp[n++] = cur;
    qh[lv] = qt[lv] = 0;
  }
  for (int i = 0; i < n; i++) {
    level[tmp[i]] = 0;
    push(0, tmp[i]);
  }
}
```

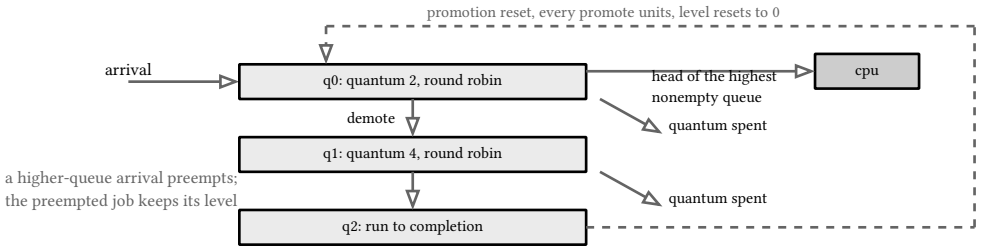
listing 182: the promotion reset: every job returns to the top queue in level then fifo order (source: c-os-cloud/samples/src/Ch17/mlfq.c, lines 37-54)

Scenario 2 runs A (16 units, arrives alone) against B (3 units, arrives at 3) with promotion every 15. B's arrival preempts A from the top queue at 3; A falls to the middle at 2, to the bottom by 9; the promotion at 15 pulls it back to the top mid-run, and one demotion and one middle-quantum later it finishes at 19. The sample checks the bookkeeping: exactly one promotion fired, A spent exactly 5 units at the bottom, B finished at 10.

```c
// scenario 2: A arrives alone, B preempts it from the top queue at 3,
// A falls to q2 by t=9, and the t=15 promotion pulls it back up
struct job two[2] = {
    {'A', 0, 16, 1, 0, 16, -1, -1},
    {'B', 3, 3, 1, 0, 3, -1, -1},
};
int promotes = run_mlfq(two, 2, 15, order);
printf("mlfq promote=15: A first %d finish %d, B first %d finish %d, "
       "promotions %d, A ran %d units at the bottom\n",
       two[0].first, two[0].finish, two[1].first, two[1].finish, promotes,
       ltime[0][2]);
CHECK(two[0].first == 0 && two[0].finish == 19, "job a first 0 finish 19");
CHECK(two[1].first == 3 && two[1].finish == 10, "job b first 3 finish 10");
CHECK(promotes == 1, "exactly one promotion fires, at t=15");
CHECK(ltime[0][2] == 5, "a spent exactly 5 units at the bottom, then rose");
```

listing 183: scenario 2 verdicts: one promotion at t=15 pulls A off the bottom queue (source: c-os-cloud/samples/src/Ch17/mlfq.c, lines 134-148)

The linux ancestry of this shape is documented on both ends. sched(7) states the succession: “Since Linux 2.6.23, the default scheduler is CFS, the “Completely Fair Scheduler”. The CFS scheduler replaced the earlier “O(1)” scheduler”, and names the older one’s property, “the newly introduced O(1) scheduler ensures that the time needed to schedule is fixed and deterministic irrespective of the number of active tasks.” The CFS design notes describe that predecessor without fondness: CFS “has no “array switch” artifacts (by which both the previous vanilla scheduler and RSDL/SD are affected)”, “has no notion of “timeslices” in the way the previous scheduler had, and has no heuristics whatsoever”, and its real-time side “uses 100 runqueues (for all 100 RT priority levels, instead of 140 in the previous scheduler)”. A priority array per level, the “array switch” the notes name as an artifact, and interactivity heuristics: of these the notes say CFS “is not prone to any of the ‘attacks’ that exist today against the heuristics of the stock scheduler”. That predecessor is the multilevel feedback design this section simulated, fifo queues and a promote timer in place of heuristics, and the two things the notes hold against it, the array switch and the heuristics, are exactly the two parts this chapter’s fixed ruleset replaced with explicit mechanics.



the rules this chapter fixes: quanta 2, 4, and run to completion, promote period a parameter

figure 68: the mlfq state machine: three queues, demotion on a spent quantum, the promotion reset

17.5. cfs and eevdf

The completely fair scheduler inverts the whole chapter. Every policy above ranks jobs and runs the winner; CFS imagines hardware with no winner at all. Its design notes open with the model: “CFS basically models an “ideal, precise multi-tasking CPU” on real hardware”, a cpu that “can run each task at precise equal speed, in parallel, each at `1/nr_running` speed”. Real hardware runs one task at a time, so the model needs a bookkeeping number: “The virtual runtime of a task specifies when its next timeslice would start execution on the ideal multi-tasking CPU described above”, and the picking rule is one sentence, the notes say CFS “always tries to run the task with the smallest `p->se.vruntime` value (i.e., the task which executed least so far)”. On an rbtree keyed by vruntime, the balanced search tree the dsa book covers in its trees chapters, that sentence becomes “CFS picks the “leftmost” task from this tree and sticks to it.”

```
// weight per nice value, 1.25 per step scaled to integers: the table is
// indexed by nice+1 and every neighbor pair multiplies by exactly 1.25
static const int WEIGHTS[3] = {6400, 5120, 4096};

static int weight_of(int nice) { return WEIGHTS[nice + 1]; }
```

listing 184: one nice step is factor 1.25, scaled to integers 6400, 5120, 4096 (source: c-os-cloud/samples/src/Ch17/cfs.c, lines 28-32)

The 1.25 is cited arithmetic, sched(7) on nice: “each unit of difference in the nice values of two processes results in a factor of 1.25 in the degree to which the scheduler favors the higher priority process”, and the sample’s weights are that rule scaled to stay exact in integers, 5120 times 1.25 is 6400, divided is 4096.

```
    if ((int)(hi - lo) > *spread_max)
      *spread_max = (int)(hi - lo);
    if (js[pick].first < 0)
      js[pick].first = t;
    js[pick].remaining--;
    order[k++] = js[pick].name;
    long long w = weighted ? weight_of(js[pick].nice) : 5120;
    vr[pick] += 100 * 5120 / w;
    t++;
    if (js[pick].remaining == 0) {
      js[pick].finish = t;
      done++;
    }
```

listing 185: the cfs pick: minimum vruntime runs, its vruntime advances by 100 times 5120 over weight (source: c-os-cloud/samples/src/Ch17/cfs.c, lines 69-81)

With equal weights vruntime is received cpu and the picker degenerates into the fair twin of q=1 round robin, but with a sharper property: response sum 0 on the fixed set, every job first touched at arrival, at turnaround sum 35, where q=1 rr paid 38. With weights, B at nice −1 advances vruntime 80 per unit against D’s 100 at nice 0, so B’s 4-unit burst finishes at 11 while the equal-burst D waits until 13: the 1.25 factor is a real share, not a hint. The sample also tracks the worst vruntime spread among started competitors and checks it stays inside one maximum weighted slice, 125: measured 120, which happens at time 3 when A sits at 200 and B at 80, the weight ratio visible as a gap. The kernel handles the other spike, a newcomer arriving at vruntime 0 against established jobs, with placement: the notes describe `rq->cfs.min_vruntime` as “a monotonic increasing value tracking the smallest vruntime among all tasks in the runqueue”, used “to place newly activated entities on the left side of the tree”; the simulator instead excludes not-yet-started jobs from the spread and says so in its source.

EEVDF, the scheduler that replaced CFS, keeps the ledger and adds a deadline. The kernel page states the succession: “The Linux kernel began transitioning to EEVDF in version 6.6 (as a new option in 2024), moving away from the earlier Completely Fair Scheduler (CFS) in favor of a version of EEVDF proposed by Peter Zijlstra in 2023”, and the fairness goal is unchanged, “Similarly to CFS, EEVDF aims to distribute CPU time equally among all runnable tasks with the same priority.” The new mechanics are the lag and the virtual deadline: the scheduler “assigns a virtual run time to each task, creating a “lag” value that can be used to determine whether a task has received its fair share of CPU time. In this way, a task with a positive lag is owed CPU time, while a negative lag means the task has exceeded its portion”, then “EEVDF picks tasks with lag greater or equal to zero and calculates a virtual deadline (VD) for each, selecting the task with the earliest VD to execute next.” The sample checks that arithmetic exactly, no run queue needed:

```
// part 2: eevdf arithmetic on two jobs, weights 6400 and 4096, over a
// 41-unit window. fair shares are exact: 41*6400/10496 = 25 and
// 41*4096/10496 = 16. lag = fair - received.
long long w1 = weight_of(-1), w2 = weight_of(1), wall = w1 + w2;
long long t_window = 41;
long long fair1 = t_window * w1 / wall, fair2 = t_window * w2 / wall;
CHECK(fair1 == 25 && fair2 == 16, "fair shares over 41 units: 25 and 16");
long long rec1 = 26, rec2 = 15; // one slice traded across the ideal split
long long lag1 = fair1 - rec1, lag2 = fair2 - rec2;
CHECK(lag1 == -1 && lag2 == 1, "lag -1 and +1 around the ideal split");
CHECK(lag2 ≥ 0 && lag1 < 0, "positive lag is owed cpu, negative exceeded");
// virtual deadline: earlier deadline wins; a shorter requested slice
// means an earlier deadline at equal weight, the eevdf latency lever
long long vd_short = 0 + 2 * 100, vd_long = 0 + 4 * 100;
CHECK(vd_short < vd_long, "slice 2 deadlines before slice 4 at equal weight");
printf("eevdf: lag %lld/%lld, vd %lld vs %lld: the short slice runs first\n",
       lag1, lag2, vd_short, vd_long);
```

listing 186: eevdf arithmetic: exact fair shares, lag signs, eligibility, the virtual deadline comparison (source: c-os-cloud/samples/src/Ch17/cfs.c, lines 102-118)

Two jobs at weights 6400 and 4096 over a 41-unit window have fair shares of exactly 25 and 16, and with one slice traded across that split the lags are −1 and +1: the negative job exceeded its portion, the positive one is owed cpu, and only the owed one is eligible. The deadline half is why fairness moved from vruntime to deadlines: the page notes “this allows latency-sensitive tasks with shorter time slices to be prioritized, which helps with their responsiveness”, and “tasks can preempt others if their VD is earlier, and tasks can request specific time slices using the new `sched_setattr()` system call”. Under pure vruntime a task cannot ask for latency; under deadlines a shorter requested slice means an earlier deadline, and the sample’s last two checks pin that comparison, 200 against 400 for slices 2 and 4. The page also records the anti-gaming half of the design: “when a task sleeps, it remains on the run queue but marked for “deferred dequeue,” allowing its lag to decay”, so a task cannot sleep off a negative lag.

```
// part 3: the weighted fixed set. B at nice -1 takes the cpu from the
// equal-burst D at nice 0 and finishes two units earlier
sched_fixed(js);
run_cfs(js, NJOBS, 1, order, &spread);
sched_report("cfs weighted", order, js, NJOBS);
CHECK(js[1].first == 2 && js[1].finish == 11,
      "job b heavy: first 2 finish 11");
CHECK(js[3].first == 5 && js[3].finish == 13,
      "job d normal: first 5 finish 13");
CHECK(js[1].finish < js[3].finish,
      "equal bursts: more weight finishes first");
CHECK(spread ≤ 125, "vruntime spread stays inside one weighted slice");
printf("cfs weighted: max spread among started jobs %d, bound 125\n", spread);
return 0;
```

listing 187: weighted verdicts: the heavier equal-burst job finishes first, spread bounded (source: c-os-cloud/samples/src/Ch17/cfs.c, lines 120-133)

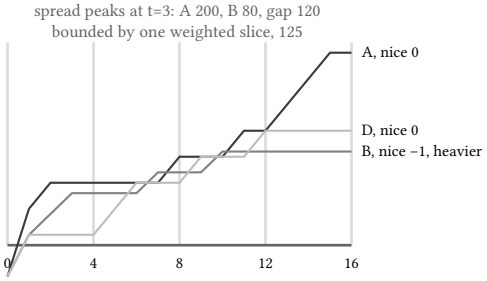


figure 69: vruntime trajectories of the weighted run: three lines converging inside one weighted slice

Ten rows, one fixed set, every number a checked sum divided by four:

policy	avg turnaround	avg response	switches
fcfs	8.75	4.75	3
sjf, never idle	8.00	4.00	3
srtf	7.00	0.50	5
rr q=1	9.50	0.75	14
rr q=2	9.00	1.50	8
rr q=4	8.50	3.25	4
priority, strict	9.25	4.25	4
mlfq, quanta 2-4-run	9.50	0.00	7
cfs, equal weight	8.75	0.00	10
cfs, weighted	8.50	0.00	10

The clairvoyant optimum among non-preemptive orders that never idle is 8.00 and srtf beats it at 7.00, the dividend preemption pays on this arrival pattern. The two rows with 0.00 response, mlfq and cfs, both dispatch every job at its arrival instant, and they pay for it in switches and turnaround. The idle-allowed optimum of 7.75 from the sjf exhaustion is absent from the table on purpose: no online policy in this chapter achieves it, because reaching it requires refusing to run the only runnable job.

sources: learn.microsoft.com, SetThreadPriority and the Scheduling Priorities conceptual page, accessed 2026-09-12; docs.kernel.org, CFS Scheduler design notes and the EEVDF Scheduler page, accessed 2026-09-12; man7.org sched(7) and setpriority(2), accessed 2026-09-12; pubs.opengroup.org Issue 8 nice(3), accessed 2026-09-12. The knob round trips, the background begin-refuse-end pair, the readback of 2 and −15, and the pinned-core probe (above normal done at +41.6 ms, below normal at +84.6 ms) probed on this machine the same day. Sample behavior verified by `make verify-c`, 83 checks in chapter 17 of the samples suite.

18. asynchronous patterns in c

Chapter 15 bought parallelism with coordination: every worker needed a lock, and chapter 16 made the fast paths lock free. Asynchronous io buys concurrency the other way, one thread, no waiting, the operating system holding the pending work and handing back completions in order. The machinery is built from three pieces this chapter assembles in sequence: the callback, the unit of exchange; the event loop, the discipline that drives it; and the completion port, the kernel doing the loop's queueing for real. Every behavioral claim below is a CHECK in the three samples, 68 in total, or a sentence quoted from a canonical page fetched 2026-09-12. The linux counterparts, `epoll` and `io_uring`, are cited from man7 and never executed on this windows box, the same treatment chapter 11 gave `fork`.

18.1. callbacks and ownership

Every asynchronous api in c is a variation on one registration: a function pointer plus a context pointer. Chapter 6 already passed this shape to `qsort_s`, whose comparator receives the caller's context pointer instead of reaching for a global, and chapter 15 passed it to `thrd_create`, whose `thrd_start_t` is spelled `int (*)(void*)`. The function is the work. The context pointer is the answer to the question every registration forces: when the callback finally runs, whose data does it see, and who owns that data? An api that does not answer that question in writing is an api with a leak or a use-after-free waiting in it.

The registry below answers it explicitly, in one sentence of contract: the registry stores the context pointer, calls through it, and never frees it. The data model is 4 fields per slot:

```
// one callback = one function + one context pointer. the shape chapter 6
// passed to qsort_s and chapter 15 to thrd_create: int (*)(void *), the
// only signature c has for "call me back with my own data"
typedef int (*callback_fn)(void *ctx);

struct slot {
  callback_fn fn;
  void *ctx;
  int id;
  bool live;
};
```

listing 188: the registry's data model: a function, a context pointer, an id, a liveness bit (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 25-35)

The `id` is assigned at registration and never reused, which is what makes dispatch order well defined once slots start being recycled. The unregister side is where ownership is decided, and this registry decides it by doing nothing:

```
// the registry never touches *ctx on the way out. ownership stays where
// it was: static, heap, or nothing at all
static bool unregister_cb(int id) {
  for (int i = 0; i < MAX_SLOTS; i++) {
    if (slots[i].live && slots[i].id == id) {
      slots[i].live = false;
      live_count--;
      return true;
    }
  }
  return false;
}
```

listing 189: unregister takes the id and never the memory: ownership stays where it was (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 63-74)

No `free`, no destructor hook, no reference count. Three ownership regimes ride through this one contract unchanged. Static storage outlives the registry by construction. Heap storage is freed by the caller, once, on the caller's schedule. The null context needs nothing at all, and is a legal registration, not a special case to guard against. The dispatch side has one hazard of its own, calling user code while walking the registry, and the walk is built for it:

```
// dispatch walks registration ids, not slot indexes: ids are assigned
// monotonically, so the order stays stable even after slots are reused.
// liveness is re-read inside the walk, so a callback that unregisters
// itself mid-dispatch is safe and runs exactly once
static int fire(void) {
  fired_count = 0;
  for (int id = 0; id < next_id; id++) {
    for (int i = 0; i < MAX_SLOTS; i++) {
      if (slots[i].live && slots[i].id == id) {
        fired_order[fired_count++] = id;
        slots[i].fn(slots[i].ctx);
        break;
      }
    }
  }
  return fired_count;
}
```

listing 190: dispatch walks registration ids, so slot reuse never reorders, and a self-unregister mid-walk is safe (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 76-92)

The outer loop climbs ids, not slot indexes, so a slot freed and reused keeps its place in line by its original id, and liveness is re-read inside the walk, so a callback that removes itself mid-dispatch has run exactly once by the time the walk passes its id again. The same function registered twice with two contexts demonstrates what the context pointer is for, no shared state between the two answers:

```c
// the context pointer is the channel: same function, two thresholds, two
// answers, no global state between them
struct threshold_ctx {
  int threshold;
  int hits;
};

static int count_above(void *ctx) {
  struct threshold_ctx *t = ctx;
  static const int samples[6] = {3, 17, 4, 22, 9, 30};
  for (int i = 0; i < 6; i++)
    if (samples[i] > t->threshold)
      t->hits++;
  return t->hits;
}
```

listing 191: the context pointer is the channel: one function, two thresholds, two answers (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 94-108)

```c
  struct threshold_ctx low = {.threshold = 10, .hits = 0};
  struct threshold_ctx high = {.threshold = 20, .hits = 0};
  int ids[16] = {0};
  int n_ids = 0;

  ids[n_ids++] = register_cb(count_above, &low);
  ids[n_ids++] = register_cb(count_above, &high);
  ids[n_ids++] = register_cb(marker, NULL);
  CHECK(ids[0] ≥ 0 && ids[1] ≥ 0 && ids[2] ≥ 0 && ids[1] == ids[0] + 1 &&
            ids[2] == ids[1] + 1,
        "three registrations return consecutive ids");

  CHECK(fire() == 3 && fired_count == 3 && fired_order[0] == ids[0] &&
            fired_order[1] == ids[1] && fired_order[2] == ids[2],
        "dispatch runs in registration order");
  CHECK(low.hits == 3 && high.hits == 2 && low.hits ≠ high.hits,
        "same function, two contexts, two answers: 3 and 2 hits");
  CHECK(marker_fires == 1, "the null context fires too");
```

listing 192: regime 1, static contexts: dispatch order, both results, and the null context asserted exactly (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 129-146)

The samples array is fixed, the arithmetic is countable, and the checks land on exact numbers: over a threshold of 10 the six samples score 3 hits, over 20 they score 2, the same function both times. The heap regime then proves the contract the only way it can be proven without an allocator audit, by surviving:

```
// regime 2: caller-owned heap. the registry does not free it, does not
// scribble it: the sentinel is intact after unregister, then the owner
// frees it, once, on the owner's schedule
struct threshold_ctx *heap = malloc(sizeof *heap);
CHECK(heap ≠ NULL, "the heap context allocates");
heap→threshold = 15;
heap→hits = 0;
ids[n_ids++] = register_cb(count_above, heap);
CHECK(ids[3] ≥ 0, "the heap context registers");
fire();
CHECK(unregister_cb(ids[3]) && heap→hits == 3 && heap→threshold == 15,
      "after unregister the heap context is untouched");
free(heap);
```

listing 193: regime 2, caller-owned heap: the sentinel is intact after unregister, then the owner frees once (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 154-166)

The heap context registers, fires once, unregisters, and is still readable afterwards, hits 3 and threshold 15 untouched, which is only possible if unregister neither freed it nor scribbled it. Then the owner frees it exactly once. The self-unregistering callback closes the section on the mid-walk hazard:

```
// the self-unregistering callback: the ctx value is filled after
// registration returns, proof the registry stored the pointer, not the
// pointee
int own_id_storage = 0;
ids[n_ids++] = register_cb(unregister_self, &own_id_storage);
own_id_storage = ids[4];
CHECK(fire() == 3 && fired_order[2] == ids[4] && self_removed_ok == 1,
      "a callback unregisters itself during dispatch");
CHECK(fire() == 2 && fired_count == 2, "the removed callback never reruns");
CHECK(marker_fires == 5 && high.hits == 10,
      "five fires in, the survivors total 10 hits");
```

listing 194: the callback that unregisters itself: it runs once, the next fire skips it (source: c-os-cloud/samples/src/Ch18/callbacks.c, lines 168-178)

Its context is a pointer to storage holding the registration id, and the id is written into that storage after `register_cb` returns, which is itself a proof: the registry stored the pointer, not a copy of what it pointed at. Five fires into the file the survivor totals are exact, the marker has dispatched 5 times and the high threshold has accumulated 10 hits, 2 per fire, 5 fires.

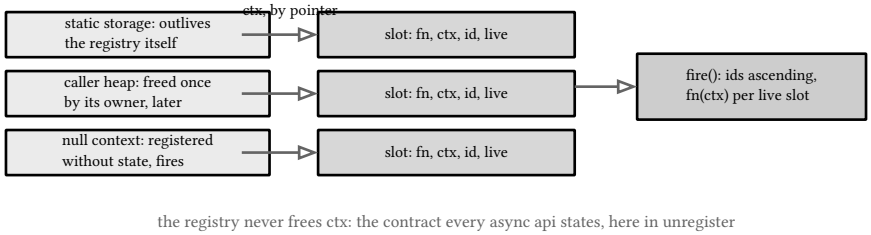


figure 70: the registry as a data structure: three ownership regimes feeding one slot array, one dispatch path

18.2. an event loop from scratch

A registry full of callbacks needs a driver, and the driver is the event loop: one thread, one queue of timed work, one loop that drains what is due. The sample loop owns two structures and one rule. The timer queue holds what to do later. The ready list holds what to do now. The rule, work scheduled during a dispatch waits for the next tick's collection pass, is what keeps every dispatch finite even when a handler schedules more work. The whole

file is deterministic by construction: time is a `uint64_t` the tick function itself advances by a fixed 10 ms, no wall clock exists anywhere in it, so fire counts and fire order are asserted exactly.

```
struct timer {
  uint64_t deadline; // absolute, in virtual ms
  uint64_t period;   // 0 = one-shot, else refire every period
  timer_fn fn;
  void *ctx;  // owned by the scheduler's caller, never by the loop
  int id;     // registration order, the tie-breaker
  bool live;  // on the timer queue
  bool armed; // collected into this tick's ready list, awaiting dispatch
};
```

listing 195: one timer: absolute deadline, period, handler, caller-owned context, two flags (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 27-35)

The two flags carry the whole dispatch discipline. `live` says the timer sits on the queue. `armed` says it has been collected into the current tick's ready list and is owed a dispatch, which is a different state entirely: a collected one-shot is already off the queue, but its callback has not run yet. One tick is three phases:

```
  // collection: earliest deadline first, ties by registration order
  for (;;) {
    int best = -1;
    for (int i = 0; i < MAX_TIMERS; i++) {
      if (!timers[i].live || timers[i].deadline > now_ms)
        continue;
      if (best < 0 || timers[i].deadline < timers[best].deadline ||
          (timers[i].deadline == timers[best].deadline &&
           timers[i].id < timers[best].id))
        best = i;
    }
    if (best < 0)
      break;
    ready[n++] = timers[best].id;
```

listing 196: collection: the earliest expired deadline moves to the ready list, ties by registration order (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 110-123)

```
    if (timers[best].period) {
      // repeating: requeue at the first deadline after this tick's
      // collection, so one expiry instant is one fire
      do {
        timers[best].deadline += timers[best].period;
      } while (timers[best].deadline ≤ now_ms);
    } else {
      // one-shot: collection is the dequeue
      timers[best].live = false;
      live_timers--;
    }
```

listing 197: the two arms: repeating requeues at deadline plus period, one-shot collection is the dequeue (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 125-135)

```
  // dispatch the frozen ready list. a handler that schedules with delay 0
  // lands in the next tick's collection, never this one: that rule is
  // what keeps one tick's dispatch finite
  int fired = 0;
  for (int k = 0; k < n; k++) {
    struct timer *t = NULL;
    for (int i = 0; i < MAX_TIMERS; i++)
      if (timers[i].armed && timers[i].id == ready[k])
        t = &timers[i];
    if (!t)
      continue; // canceled between collection and dispatch
    t->armed = false;
    fire_log[fire_count] = t->id;
    fire_tick[fire_count] = (int)(now_ms / TICK_MS);
    fire_count++;
    fired++;
    t->fn(t->ctx);
  }
```

listing 198: dispatch runs the frozen ready list; the armed flag is the handshake with cancel (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 138-155)

The run then plays the whole instrument. Six timers: one-shots at 30, 10, 50, and 60 ms, a deliberate tie at 30 ms, and a repeating timer every 40 ms starting at 40. One of them is canceled before it can fire. Ten ticks move the clock to 100:

```
  // the job set: one-shots at 30, 10, 50, 60 ms, a tie at 30 ms, and a
  // repeating timer every 40 ms starting at 40
  int a = schedule(30, 0, plain, NULL);
  int b = schedule(10, 0, on_b, NULL);
  int c = schedule(50, 0, plain, NULL);
  int r = schedule(40, 40, plain, NULL);
  int d = schedule(60, 0, plain, NULL);
  int e = schedule(30, 0, plain, NULL);
  CHECK(a == 0 && b == 1 && c == 2 && r == 3 && d == 4 && e == 5,
        "six timers register with ids in registration order");

  // cancel D before its deadline
  CHECK(cancel(d), "a live timer cancels");

  for (int t = 1; t ≤ 10; t++)
    tick();
```

listing 199: the job set, one cancel, ten ticks of virtual time (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 180-195)

```
// the whole run: B at tick 1, the delay-0 child Z at tick 2 (not 1),
// the 30 ms tie A then E at tick 3, R at 4, C at 5, R again at 8. D
// never appears: canceled before the run
static const int want_log[7] = {1, 6, 0, 5, 3, 2, 3};
static const int want_tick[7] = {1, 2, 3, 3, 4, 5, 8};
CHECK(fire_count == 7 && memcmp(fire_log, want_log, sizeof want_log) == 0,
      "the fire order is exact across the whole run");
CHECK(memcmp(fire_tick, want_tick, sizeof want_tick) == 0,
      "the fire ticks are exact, including the gap at tick 6");
CHECK(observed_tick == 1, "a handler observes the loop's tick counter");
CHECK(zero_delay_id == 6 && fire_log[1] == 6 && fire_tick[1] == 2 &&
          fire_tick[0] == 1,
      "work scheduled during dispatch fires next tick, not this one");
CHECK(fire_log[2] == 0 && fire_log[3] == 5,
      "same-deadline timers fire in registration order");
```

listing 200: the verdict: fire log and fire tick arrays compared byte for byte (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 197-211)

The expected log reads B at tick 1, its delay-0 child Z at tick 2, the 30 ms tie A then E at tick 3, the repeating R at 4 and again at 8, C at 5, and nothing at tick 6, the tick the canceled timer would have owned. Two checks deserve their own reading. The tie at 30 ms fires in registration order because collection breaks ties by id, and the delay-0 child of B fires at tick 2 and not tick 1, which is the next tick rule earning its keep: B's handler ran during tick 1's dispatch, the ready list for tick 1 was already frozen, so the child waits for tick 2's collection no matter that its deadline equals the current time. Without that rule a self-feeding handler would spin one tick forever. The repeating timer's arithmetic is checked where it lives, in the queue: after two fires its next deadline is exactly 120, its original 40 plus two periods.

The mid-dispatch cancel is the last discipline, and it is the reason the `armed` flag exists. X and Y share a deadline. Both are collected into the same tick's ready list. X dispatches first and cancels Y, which by then is already off the queue but not yet fired:

```
// the mid-dispatch cancel: X and Y share a deadline, X dispatches
// first and cancels Y, which is already collected but not yet fired
int x = schedule(20, 0, on_x, NULL);
y_id = schedule(20, 0, plain, NULL);
tick();
tick();
CHECK(fire_count == 8 && fire_log[7] == x && y_cancelled,
      "a handler's mid-dispatch cancel removes the collected sibling");
CHECK(tick() == 0, "the canceled sibling never fires on any later tick");
```

listing 201: a handler cancels its collected sibling mid-dispatch: the armed flag clears, the dispatch skips it (source: c-os-cloud/samples/src/Ch18/eventloop.c, lines 244-252)

Cancel works on the timer wherever the loop still owes it something, queued, or collected but undispatched. And cancel, like unregister in the first section, never touches *ctx: the ownership contract is identical, here checked with a sentinel the caller reads back intact after the cancel.

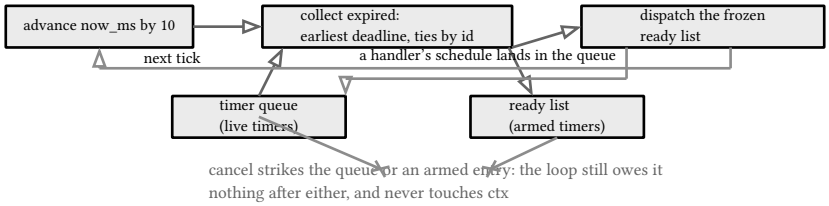


figure 71: one tick as a state machine: collect freezes the ready list, dispatch runs it, delay-0 work schedules one tick ahead

18.3. overlapped io and GetQueuedCompletionStatus

The loop above polls a virtual clock. Real async io is the same shape with the kernel holding the queue. On windows the entry point is `FILE_FLAG_OVERLAPPED` at open time, 0x40000000, which the CreateFile page defines in one line: the flag means "The file or device is being opened or created for asynchronous I/O", and with it "the file can be used for simultaneous read and write operations". Without it, "I/O operations are serialized, even if the calls to the read and write functions specify an OVERLAPPED structure". The flag changes the handle's kind, not the call's spelling. ReadFile then states the consequence: "A pointer to an OVERLAPPED structure is required if the hFile parameter was opened with `FILE_FLAG_OVERLAPPED`, otherwise it can be NULL", and the requirement is per-operation, "the lpOverlapped parameter must point to a valid and unique OVERLAPPED structure, otherwise the function can incorrectly report that the read operation is complete". The read position comes from the structure, not the handle: "This offset is specified by setting the Offset and OffsetHigh members of the OVERLAPPED structure". And the return value that looks like failure is not: "The GetLastError code `ERROR_IO_PENDING` is not a failure; it designates the read operation is pending completion asynchronously."

The completion port is the queue those pends complete into. The CreateIoCompletionPort page names the gate the handle must already pass: "If a handle is provided, it has to have been opened for overlapped I/O completion. For example, you must specify the `FILE_FLAG_OVERLAPPED` flag when using the CreateFile function to obtain the handle." Its `CompletionKey` parameter is this chapter's context pointer wearing kernel clothes, and the page says exactly what the registry said: "This value is not used by CreateIoCompletionPort for functional control; rather, it is attached to the file handle specified in the FileHandle parameter at the time of association with an I/O completion port", and it "accompanies the file handle throughout the internal completion queuing process". One sentence on the I/O Completion Ports conceptual page is the ordering rule the whole sample turns on: the functions that start completions require "an instance of the OVERLAPPED structure and a file handle previously associated with an I/O completion port (by a call to CreateIoCompletionPort) to enable the I/O completion port mechanism". Previously associated. Not associated eventually.

The sample builds both orders against a 4096 byte pattern file it writes itself and deletes on exit. The wrapper type is the standard container idiom, and it is also the ownership statement: the whole struct, `OVERLAPPED`, slot, and 512 byte buffer, must outlive the in-flight request, exactly as the ReadFile page demands of the buffer, it "must remain valid for the duration of the read operation":

```
// one in-flight overlapped read. OVERLAPPED is member one: the completion
// hands back the OVERLAPPED address and the wrapper is recovered by one
// cast. the whole struct stays alive until its completion is dequeued:
// the kernel writes into ov and buf after ReadFile returns
struct read_req {
  OVERLAPPED ov;
  unsigned slot;
  unsigned char buf[512];
};

static const struct {
  DWORD off;
  DWORD len;
} slices[3] = {{0, 512}, {1365, 512}, {4000, 512}};

static BOOL issue_read(HANDLE h, struct read_req *r, DWORD off, DWORD len) {
  ZeroMemory(&r->ov, sizeof r->ov); // hEvent NULL: required for port delivery
  r->ov.Offset = off; // OffsetHigh stays 0: a 4096 byte file needs one dword
  memset(r->buf, 0xCC, sizeof r->buf);
  return ReadFile(h, r->buf, len, NULL, &r->ov);
}
```

listing 202: the wrapper: OVERLAPPED first, so the completion's pointer recovers the request with one cast (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 53-73)

The working order, and the three reads, offsets 0, 1365, and 4000, each into its own wrapper:

```
// leg 1: the working order. the handle opens with FILE_FLAG_OVERLAPPED,
// the port is created and associated BEFORE the first ReadFile
HANDLE hA = CreateFileW(TMP_NAME, GENERIC_READ, FILE_SHARE_READ, NULL,
                        OPEN_EXISTING, FILE_FLAG_OVERLAPPED, NULL);
CHECK(hA != INVALID_HANDLE_VALUE,
      "the file reopens with FILE_FLAG_OVERLAPPED");

HANDLE port = CreateIoCompletionPort(hA, NULL, (ULONG_PTR)KEY_FILE, 0);
CHECK(port != NULL, "the port exists and is associated before any request");

struct read_req reqs[3];
for (int i = 0; i < 3; i++) {
  reqs[i].slot = (unsigned)i;
  BOOL p = issue_read(hA, &reqs[i], slices[i].off, slices[i].len);
  // both documented returns are legal on an overlapped handle: TRUE,
  // or FALSE with ERROR_IO_PENDING. either way the completion packet is
  // the deliverer of record
  CHECK(p || GetLastError() == ERROR_IO_PENDING,
        "overlapped read issued at offset");
}
```

listing 203: the working order: the flag at open, the port associated before the first request (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 78-97)

Each issue accepts either documented return, TRUE or FALSE with `ERROR_IO_PENDING`, because on a local file either is legal and the packet is the deliverer of record either way. The drain then holds the kernel to one-packet-per-request:

```
  // drain exactly three completions. arrival order across the three
  // reads is the kernel's business; one-packet-per-request is not
  int seen[3] = {0};
  for (int n = 0; n < 3; n++) {
    DWORD bytes = 0;
    ULONG_PTR key = 0;
    LPOVERLAPPED pov = NULL;
    CHECK(GetQueuedCompletionStatus(port, &bytes, &key, &pov, 2000),
          "a completion packet arrives at the port");
    CHECK(key == (ULONG_PTR)KEY_FILE, "the per-handle key rides every packet");
    struct read_req *r = (struct read_req *)pov;
    CHECK(r->slot < 3 && seen[r->slot] == 0, "each request completes once");
    seen[r->slot]++;
    DWORD expect = slices[r->slot].off + slices[r->slot].len > FILE_BYTES
                       ? FILE_BYTES - slices[r->slot].off
                       : slices[r->slot].len;
    CHECK(bytes == expect,
          "the transferred count matches, including the eof short read");
    BOOL exact = TRUE;
    for (DWORD j = 0; j < expect; j++)
      if (r->buf[j] != pattern_at(slices[r->slot].off + j))
        exact = FALSE;
    CHECK(exact, "the overlapped bytes are exact");
  }
  CHECK(seen[0] == 1 && seen[1] == 1 && seen[2] == 1,
        "all three reads completed, each exactly once");
```

listing 204: drain three completions: per-handle key, one packet per request, the eof short read exact (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 99-124)

Arrival order across three concurrent reads is the kernel's business, so the loop asserts set equality, each slot exactly once, plus the checks that do not move: the key is the per-handle key on every packet, the transferred count matches each request, including the third read whose 512 byte request ends at the file's edge and returns the 96 bytes that exist, the short-read rule quoted verbatim on the ReadFile page, and the bytes themselves are exact, every one compared against the pattern function that wrote them. `PostQueuedCompletionStatus` then proves the queue carries arbitrary values, not just io results, with the page's own guarantee under it, "The system does not use or validate these values. In particular, the lpOverlapped parameter need not point to an OVERLAPPED structure":

```
// PostQueuedCompletionStatus: three user values queued with no io at
// all. the docs promise they arrive untouched; the pointer need not
// even point at an OVERLAPPED. this is the loop's wake and shutdown
// channel, the same role a posted message plays in a message loop
CHECK(PostQueuedCompletionStatus(port, 4242, (ULONG_PTR)KEY_CTL,
                                 (LPOVERLAPPED)(UINT_PTR)0x1234),
      "a synthetic packet posts without any io");
{
  DWORD bytes = 0;
  ULONG_PTR key = 0;
  LPOVERLAPPED pov = NULL;
  CHECK(GetQueuedCompletionStatus(port, &bytes, &key, &pov, 2000) &&
            bytes == 4242 && key == (ULONG_PTR)KEY_CTL &&
            pov == (LPOVERLAPPED)(UINT_PTR)0x1234,
        "all three posted values arrive untouched");
}
```

listing 205: a synthetic packet: three user values posted with no io at all, received untouched (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 126-141)

That is a loop's wake and shutdown channel in one call, the same role a posted message plays in a window's message loop. The mirror leg is the one the section exists to assert:

```
// leg 2: the broken order, asserted. a second overlapped handle, the
// read issued FIRST, then the association. the sleep gives the read
// every chance to finish: the miss is about association order, not
// completion timing. the bounded wait times out with *lpOverlapped
// forced to NULL, the documented shape of "no packet dequeued"
HANDLE hB = CreateFileW(TMP_NAME, GENERIC_READ, FILE_SHARE_READ, NULL,
                        OPEN_EXISTING, FILE_FLAG_OVERLAPPED, NULL);
CHECK(hB != INVALID_HANDLE_VALUE, "a second overlapped handle opens");
struct read_req late;
BOOL p = issue_read(hB, &late, 2048, 512);
CHECK(p || GetLastError() == ERROR_IO_PENDING,
      "the pre-association read is issued and pends");
Sleep(100); // the read completes here, before any port exists
HANDLE port2 = CreateIoCompletionPort(hB, NULL, (ULONG_PTR)KEY_FILE, 0);
CHECK(port2 != NULL, "associating after the issue still succeeds");
{
  DWORD bytes = 0;
  ULONG_PTR key = 0;
  LPOVERLAPPED pov =
      (LPOVERLAPPED)(UINT_PTR)0x1; // poison: a dequeue overwrites
  CHECK(GetQueuedCompletionStatus(port2, &bytes, &key, &pov, 1500) == FALSE &&
            pov == NULL,
        "the pre-association completion never arrives: timeout, nothing "
        "dequeued");
}
```

listing 206: the broken order: the read issued first, the port associated after, the completion never arrives (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 143-167)

The sleep between issue and association gives the read every chance to finish, 100 ms for an operation that takes microseconds, so the miss cannot be blamed on timing: it is the association order. The bounded wait returns FALSE with the page's own signature of the miss, "the function times out, returns FALSE, and sets `*lpOverlapped` to NULL", which is why the sample poisons the variable with a non-null value first. Nothing dequeues. The completion from a request issued before association has no port to queue to. Probed on this machine 2026-09-12 and now asserted rather than remembered. Cleanup follows the docs' own accounting, the port and its associated handles "are

known as references to the I/O completion port", all of which must close for the port to release, and CloseHandle's object list includes I/O completion port by name:

```
// cleanup: the docs count the port and its associated handles as
// references; close them all, then delete the file
CHECK(CloseHandle(hA), "the overlapped file handle closes");
CHECK(CloseHandle(port), "the completion port closes");
CHECK(CloseHandle(hB), "the late-associated handle closes");
CHECK(CloseHandle(port2), "the second port closes");
CHECK(DeleteFileW(TMP_NAME), "the temp file is deleted");
```

listing 207: every reference closes, then the temp file is deleted (source: c-os-cloud/samples/src/Ch18/overlapped.c, lines 169-175)

PITFALL: the order the docs state and the probe proved

The conceptual page's sentence, a handle "previously associated with an I/O completion port" is what "enable[s] the I/O completion port mechanism", reads like a footnote. It is the difference between the two legs of the sample. Associate before the first request and every completion queues, including completions of operations that finish synchronously. Associate after and the already-issued request's completion is gone, silently, with no error anywhere: the read's data lands in the buffer, the wait times out, and the only evidence anything happened is the timeout. The address-sanitizer leg of the gate runs this file, so the wrappers' lifetimes are checked, not asserted politely.

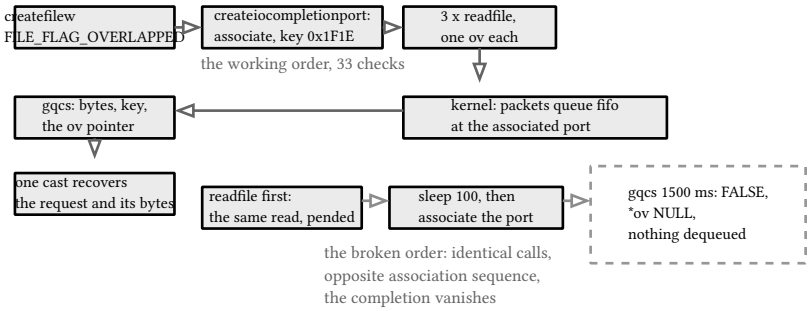


figure 72: the two orders side by side: associate before the request, packets flow; issue first, the completion has no port

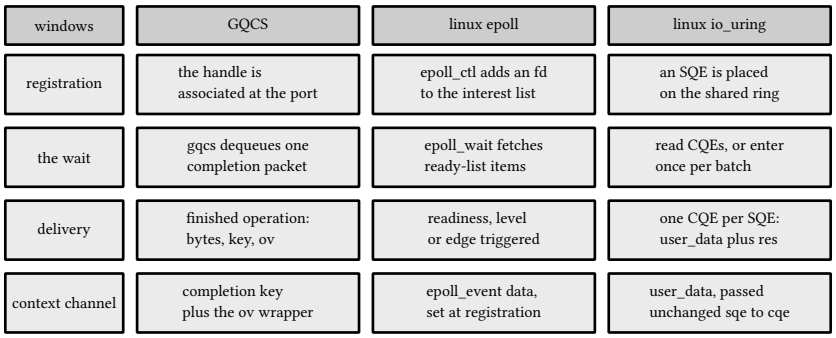
18.4. epoll and io_uring, cited

The two linux models for the same problem are cited here from man7, fetched 2026-09-12, and never executed on this box, the standing treatment for the posix side since chapter 11's `fork`. Both are the chapter's own shapes with the kernel deeper inside.

`epoll(7)` is the event loop's collection phase as a syscall family. "The central concept of the epoll API is the epoll instance, an in-kernel data structure" holding two lists, the interest list, "the set of file descriptors that the process has registered an interest in monitoring", and the ready list, "the set of file descriptors that are 'ready' for I/O", "dynamically populated by the kernel as a result of I/O activity". That ready list is the same ready list the loop above freezes each tick; epoll just never lets user space see the queue, only its drained contents. Three calls build the loop: `epoll_create` "creates a new epoll instance and returns a file descriptor referring to that instance", `epoll_ctl` registers interest, and `epoll_wait` "waits for I/O events, blocking the calling thread if no events are currently available", which "can be thought of as fetching items from the ready list". One behavioral fork has no windows counterpart in this chapter: "The epoll event distribution interface is able to behave both as edge-triggered (ET) and level-triggered (LT)", and the edge variant changes the contract, "An application that employs the EPOLLET flag should use nonblocking file descriptors", with readiness honored "until the next (nonblocking) read/write yields EAGAIN". The completion port never poses that question because it does not report readiness at all; it reports finished operations.

`io_uring(7)` is the completion port's closest kin. "`io_uring` is a Linux-specific API for asynchronous I/O" that "gets its name from ring buffers which are shared between user space and kernel space", two of them: "You place I/O requests you want to make on the SQ, while the kernel places the results of those operations on the CQ." Submission entries are io requests by another spelling, "Each I/O operation is, in essence, the equivalent of a system call you would have made otherwise", and completion is one for one: "The kernel places exactly one matching CQE in the CQ for every SQE you submit on the SQ", the same one-packet-per-request the sample asserted against `GetQueuedCompletionStatus`. The context channel travels in `user_data`, "passed unchanged from SQE to CQE, used to correlate completions", the same job the completion key and the `OVERLAPPED` wrapper do together on windows. The batching escape hatch is where `io_uring` passes the others: "you can batch several requests in one go, simply by queueing up multiple SQEs", and with SQ polling "there is no need for you to call `io_uring_enter(2)`" at all, the submission ring drained by a kernel thread. The ordering caution mirrors this chapter's arrays: "I/O requests submitted to the kernel can complete in any order."

All three systems land on the same three answers, registration before readiness or submission, one wait point, and a context value that travels with each completion. The differences are in what the wait returns and who owns the queues, and the matrix is the chapter in one picture:



the queue owner moves left to right: user loop, kernel interest list, shared rings

figure 73: three completion systems, one matrix: what registers, what waits, what comes back, what carries the context

The epilogue writes itself from the matrix. The event loop of the second section is the left column by hand, a user-space queue polled by a tick. `epoll` keeps the registration in the kernel but still reports readiness, so the caller re-issues the operation and owns the completion question. `io_uring` and GQCS both move the completion across the boundary, one matching completion per request, a context value riding each one untouched. The async chapters of this book end where the cloud chapters begin, with the observation that a server is a loop like this one holding ten thousand sockets instead of six timers, and the loop that survives is the loop whose ownership answers were written down before the first request was issued.

sources: learn.microsoft.com, CreateFileA (`FILE_FLAG_OVERLAPPED` and the synchronous versus asynchronous handle section), ReadFile (lpOverlapped requirement, `ERROR_IO_PENDING`, the eof short read, buffer lifetime), CreateIoCompletionPort (the overlapped gate, the completion key, the references rule), GetQueuedCompletionStatus (return values, the timeout that nulls lpOverlapped), PostQueuedCompletionStatus (values not used or validated), CloseHandle (the object list, I/O completion port included), and the I/O Completion Ports conceptual page (FIFO packet queue, LIFO thread release, the concurrency value, the "previously associated" mechanism sentence), all accessed 2026-09-12; man7.org epoll(7) and `io_uring(7)`, accessed 2026-09-12; the association-order behavior, the timeout shape of the vanished completion, and the per-packet values probed on this machine the same day. Sample behavior verified by `make verify-c`, 68 checks in chapter 18 of the samples suite.

19. allocator designs

Chapter 14 rebuilt `malloc` once, as a free list over one arena, and proved the contract. This chapter builds five more allocators, each one a different answer to the same three questions: how fast is the happy path, how many bytes does bookkeeping cost per allocation, and what happens to the holes that freeing leaves behind. The bump allocator answers fast and never looks back. The slab answers cheap for one fixed object size. The buddy answers the holes, splitting and merging blocks by powers of two. A tracking wrapper answers none of the three and instead makes every other allocator observable. Everything deterministic is asserted, every timing is printed and never checked, and the last section does the arithmetic the capstone kernel in chapter 21 spends.

19.1. the allocator's contract

The contract is unchanged from chapter 14: hand out pointers that do not overlap, aligned well enough for any fundamental type, take them back without charging the caller for the bookkeeping, and stay correct when the caller frees in any order. What changes per allocator is which promise is implicit. The free list of chapter 14 paid 32 bytes of header per allocation to buy individual frees and coalescing. The allocators below trade that freedom away in both directions: the bump allocator refuses individual frees entirely, and the slab refuses individual sizes. Neither trade is exotic. Every production heap in the survey below is a hybrid of exactly these shapes, per-thread caches of fixed size classes over an arena backend, and the reason is the same every time: the general contract is expensive, and most allocations in a real program do not need all of it.

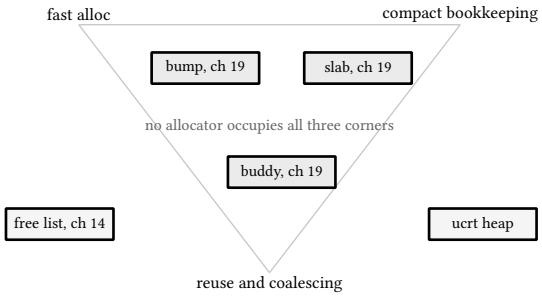


figure 74: the contract triangle and where each allocator in this chapter sits

19.2. alignment, size classes, and headers

Every allocator in this chapter rounds requests, and the ladder it rounds on is a property of the target, pinned here before anything is built on it. The fundamental types align at 1, 2, 4, and 8 bytes, and `long double` is the first surprise: this target makes it exactly `double`, 8 and 8, where a linux target would say 16. The struct arithmetic follows the members, one `char` before a `double` pads the whole struct to 16 bytes at alignment 8, and `_Alignas` raises a struct past every member, at the price of padding the size up to the raised alignment. One absence matters for the same story: this crt declares no `max_align_t` at all, probed 2026-09-13, the same family of absences as `aligned_alloc` and `free_sized` in chapters 4 and 14, so the "suitably aligned for any fundamental type" language is quoted from the `malloc` page rather than measured through a type.

```
struct lined { // one char, padded to the double's alignment
  char c;
  double d;
};

struct cacheline {
  _Alignas(64) char payload[1]; // over-aligned on purpose
};

struct hdr16 {
  _Alignas(16) size_t size; // a one-field header
};

struct hdr32 { // chapter 14's real header: three fields, two size classes
  _Alignas(16) size_t size;
  struct hdr32 *next;
  size_t used;
};

static size_t round16(size_t n) { return (n + 15) / 16 * 16; }
```

listing 208: the ladder and the two round-up functions, one 16-byte class and one power-of-two class (source: c-os-cloud/samples/src/Ch19/align.c, lines 24-43)

The probe side of the file asks what the ucrt actually delivers, and the answer is stronger than its own documentation promises: the page guarantees fundamental alignment, "the alignment that's required for a `double`, or 8 bytes", 16 on 64-bit targets, and every one of 32 odd request sizes came back 16-aligned. The page also states the tax in one sentence, the block "may be larger than `size` bytes because of the space that's required for alignment and maintenance information", which is the exact cost the next three sections measure two ways.

```
  // what this crt delivers, probed rather than assumed
  int all16 = 1;
  for (int i = 0; i < 32; i++) {
    void *p = malloc((size_t)(i * 7 + 1));
    if ((uintptr_t)p % 16 != 0)
      all16 = 0;
    free(p);
  }
  CHECK(all16, "ucrt malloc hands back 16-aligned storage, 32 odd sizes");
```

listing 209: the probe: 32 odd sizes through the real heap, alignment asserted not assumed (source: c-os-cloud/samples/src/Ch19/align.c, lines 66-74)

The two round-up functions are the whole grammar of the chapter. `round16` is the size-class function, tcmalloc's own worked example on its design page, "an allocation of 12 bytes will get rounded up to the 16 byte size-class", compressed to one expression. `pow2ceil` is the buddy function, and the checks pin the fragmentation it causes as arithmetic: a 5-byte request strands 11 bytes in its class, a 17-byte request strands 15, and a 5000-byte buddy request strands 3192 in the next section.

```
// smallest power of two ≥ n, the class function buddy.c carves with
static size_t pow2ceil(size_t n) {
  size_t p = 1;
  while (p < n)
    p <<= 1;
  return p;
}
```

listing 210: the buddy class function: smallest power of two that fits (source: c-os-cloud/samples/src/Ch19/align.c, lines 45-51)

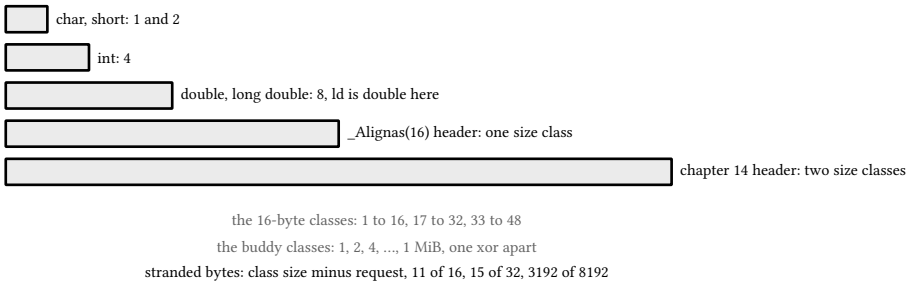


figure 75: the type ladder and the class ladders: where every request lands

19.3. bump allocation

The bump allocator is one cursor over one arena. Allocation rounds the cursor up to the requested alignment, checks that the rounded start and the size both fit, hands out the pointer, and moves the cursor. There is no header, no free list, and no way to free anything individually: `free` does not exist, and the overflow guard is written so the addition cannot wrap, the start is compared against the arena bound before the size is subtracted from it.

```
static void *bump(size_t n, size_t a) {
  size_t start = (off + a - 1) / a * a;
  if (start > ARENA || n > ARENA - start)
    return NULL;
  off = start + n;
  return arena + start;
}

static void breset(void) { off = 0; }
```

listing 211: the whole allocator: round up, bound check, hand out, advance, and the one-word reset (source: c-os-cloud/samples/src/Ch19/bump.c, lines 34-43)

The checks pin the arithmetic from both ends. Two adjacent 16-byte bumps land 16 bytes apart, zero overhead against the 32 bytes a chapter 14 pair would pay. A failing request leaves the cursor exactly where it was, and the arena can be filled to its last byte, which pins that the bound check wastes nothing. Reset is one store, and the strongest property follows from it: replaying the same request sequence after a reset returns the identical addresses, determinism chapter 21's kernel leans on. At `-O2` the fast path compiles to a single mask, an `and i64` against `-16` or the 64-byte mask `0xffffc0`, one branch for the bound, and the gate pins the substring.

```c
// zero per-allocation overhead: two 16-byte payloads sit 16 apart
breset();
unsigned char *a1 = bump(16, 16);
unsigned char *a2 = bump(16, 16);
CHECK(a2 - a1 == 16, "no header, no gap: adjacent 16-byte bumps touch");

// failure leaves the state untouched
size_t off_before = off;
CHECK(bump(ARENA, 16) == NULL && off == off_before,
      "an oversize bump fails and changes nothing");
size_t edge = (off + 15) / 16 * 16;
void *last = bump(ARENA - edge, 16); // lands the offset on ARENA
CHECK(last != NULL && off == ARENA, "the arena fills to the last byte");
CHECK(bump(1, 16) == NULL, "a full arena refuses the next 1-byte bump");
```

listing 212: zero overhead pinned as an address delta, and failure pinned as a no-op (source: c-os-cloud/samples/src/Ch19/bump.c, lines 75-88)

The clock, printed three times across the runs observed while writing: 200000 allocations of 32 bytes plus one reset cost 0.61 to 0.64 ms, against 5.38 to 6.63 ms for the ucrt `malloc` and `free` of the same, a ratio of 8.8x to 10.3x. That is the whole price of the general contract, alignment negotiation, size-class lookup, bookkeeping, and a `free` that must find what it is freeing, paid on every allocation by a heap that cannot know the caller does not need any of it.

```c
// the clock: bump against the ucrt heap, printed never checked
enum { N = 200000 };
double t0 = ms_now();
breset();
for (int i = 0; i < N; i++)
  bump(32, 16);
double t1 = ms_now();
breset();
for (int i = 0; i < N; i++) {
  void *p = malloc(32);
  (void)p;
  free(p);
}
double t2 = ms_now();
printf("bump: %d x 32 B allocations plus one reset: %.2f ms\n", N, t1 - t0);
printf("ucrt malloc/free of the same: %.2f ms\n", t2 - t1);
printf("ratio: %.1fx, the price of a general-purpose contract\n",
       (t2 - t1) / (t1 - t0));
```

listing 213: the clock: bump against the ucrt heap, same request stream (source: c-os-cloud/samples/src/Ch19/bump.c, lines 112-129)

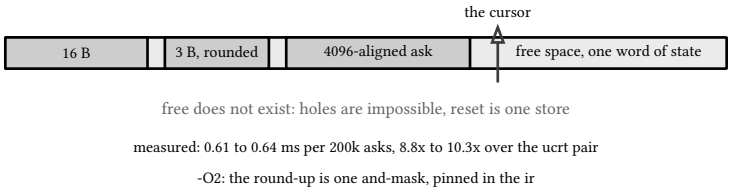


figure 76: one cursor, no holes: allocation moves the arrow, reset moves it home

19.4. the slab

Fix the object size and half the contract disappears. A slab is one arena cut into equal slots, and because every slot is the same size class, no slot needs a header: the free list is threaded through the free slots themselves, the first

8 bytes of a free slot hold the next free slot. The entire allocator is one pointer and two counters of static state, `salloc` is a pop, `sfree` is a push, and the arena holds exactly 1024 objects of 64 bytes with no per-object byte spent on bookkeeping.

```
static void sinit(void) {
  head = NULL;
  outstanding = 0;
  for (size_t i = NSLOTS; i-- > 0;) {
    void **slot = (void **)(arena + i * SLOT);
    *slot = head;
    head = slot;
  }
  free_count = NSLOTS;
}

static void *salloc(void) {
  if (head == NULL)
    return NULL;
  void **slot = head;
  head = *slot;
  free_count--;
  outstanding++;
  return slot;
}

static void sfree(void *p) {
  void **slot = p;
  *slot = head;
  head = slot;
  free_count++;
  outstanding--;
}
```

listing 214: the whole slab: build the list inside the slots, then pop and push (source: c-os-cloud/samples/src/Ch19/slab.c, lines 33-60)

The permutation check is the slab's integrity proof: draining all 1024 slots must yield each address exactly once, every one 64-byte strided inside the arena, with a bitmap proving no duplicates and no gaps. LIFO is then pinned exactly, three frees returned in reverse, and the strongest check closes the loop: freed in reverse order, a second full drain replays the identical address sequence. Mixed traffic over a 256-slot window, three allocations and one free per round for 64 rounds, ends with the live count exact to the byte.

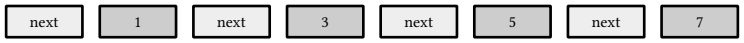
```
  for (int i = 0; i < NSLOTS; i++) {
    first[i] = salloc();
    unsigned char *u = first[i];
    if (u == NULL || (u - arena) % SLOT != 0 || u >= arena + ARENA)
      unique = 0;
    else
      seen[(u - arena) / SLOT] = 1;
  }
  int all_seen = 1;
  for (int i = 0; i < NSLOTS; i++)
    if (!seen[i])
      all_seen = 0;
  CHECK(unique && all_seen && free_count == 0,
        "draining yields each slot exactly once, all 64-byte strided");
  CHECK(salloc() == NULL, "a drained slab refuses the next request");
```

listing 215: the permutation: drain everything, see each slot exactly once (source: c-os-cloud/samples/src/Ch19/slab.c, lines 80-94)

The clock prints the same shape as bump's. One million pop-and-push pairs of 64 bytes cost 3.40 to 3.81 ms across the runs observed while writing, against 28.25 to 28.47 ms for the ucrt pair on the same request stream, 7.4x to 8.3x. The gap is smaller than bump's because the slab's pop is a dependent load, the head pointer must arrive before the next head can be read, where bump's cursor is a plain increment. At `-O2` the pop is one `load ptr, ptr @head` and the gate pins it.

```
  double t0 = ms_now();
  for (int i = 0; i < 1000000; i++) {
    void *p = salloc();
    sfree(p);
  }
  double t1 = ms_now();
  for (int i = 0; i < 1000000; i++) {
    void *p = malloc(64);
    free(p);
  }
  double t2 = ms_now();
  printf("slab: 1000000 pop/push pairs of 64 B: %.2f ms\n", t1 - t0);
  printf("ucrt malloc/free of the same: %.2f ms\n", t2 - t1);
  printf("ratio: %.1fx, what a size header and a general free cost\n",
         (t2 - t1) / (t1 - t0));
```

listing 216: the clock: a million pop-push pairs against the ucrt pair (source: c-os-cloud/samples/src/Ch19/slab.c, lines 139-153)



dark slots are live objects, light slots are their own free list

zero header bytes: 1024 x 64 B exactly, chapter 14 would need 96 B each

the pop is one dependent load, reset does not exist, drain replays lifo

measured: 3.40 to 3.81 ms per million pairs, 7.4x to 8.3x over ucrt

figure 77: free slots carry the list: the link lives inside the storage it is handing out

19.5. the buddy allocator

The buddy allocator over 1 MiB answers the question bump refuses: holes. Every block is a power of two between 4 KiB and 1 MiB, an allocation rounds up to the next power of two and splits larger blocks in half until one fits, and the partner of every block is one xor away in the offset. Freeing merges equal-order partners back up the ladder as long as the partner is free, so interleaved traffic can always return to one whole block.

```
static unsigned char *buddy_of(unsigned char *p, size_t order) {
  size_t index = (size_t)(p - arena);
  return arena + (index ^ (1u << order));
}
```

listing 217: the whole addressing scheme: one bit flip at the block's order (source: c-os-cloud/samples/src/Ch19/buddy.c, lines 93-96)

```
// round up, walk down: split larger blocks until the order fits
static void *balloc(size_t n) {
  size_t order = order_of(n);
  if (order > MAX_ORDER)
    return NULL;
  size_t k = order;
  while (k ≤ MAX_ORDER && fcount[k - MIN_ORDER] == 0)
    k++;
  if (k > MAX_ORDER)
    return NULL;
  unsigned char *p = pop(k);
  while (k > order) {
    k--;
    push(p + (1u << k), k); // the high half goes back on the list
  }
  return p;
}
```

listing 218: allocation: find a fit order, split the high halves back onto their lists (source: c-os-cloud/samples/src/Ch19/buddy.c, lines 106-122)

```
static void bfree(void *p, size_t n) {
  unsigned char *b = p;
  size_t order = order_of(n);
  while (order < MAX_ORDER) {
    unsigned char *bud = buddy_of(b, order);
    if (!in_list(bud, order))
      break;
    unlink_block(bud, order);
    if (bud < b)
      b = bud; // keep the low half as the merged block
    order++;
  }
  push(b, order);
}
```

listing 219: freeing: absorb the partner while it is free, then push the merged block (source: c-os-cloud/samples/src/Ch19/buddy.c, lines 125-138)

The checks walk the structure by hand. One 4 KiB request from a fresh arena leaves exactly one free block at every order below the top, the split chain, and drops the free byte count by exactly 4096. Two 8 KiB blocks are partners, the first free cannot merge because its partner is busy, and freeing the partner walks the merge ladder all the way back to one 1 MiB block. The fragmentation story is arithmetic, a 5000-byte request lives in an 8192 block and 3192 bytes stay unusable, and the exhaustion story is exact: 1 MiB is precisely 256 minimum blocks, the 257th fails, freeing every other block leaves 128 partners that never meet, and freeing the rest closes back to the whole.

```
// exact exhaustion at the minimum order, then non-buddy frees stall
binit();
unsigned char *mini[256];
int got = 0;
for (int i = 0; i < 257; i++) {
  void *p = balloc(4096);
  if (p == NULL)
    break;
  mini[got++] = p;
}
CHECK(got == 256 && balloc(4096) == NULL,
      "1 MiB is exactly 256 minimum blocks, the 257th fails");
for (int i = 0; i < 256; i += 2)
  bfree(mini[i], 4096);
CHECK(fcount[0] == 128 && total_free() == 128 * 4096,
      "freeing every other block: no buddies meet, nothing merges");
for (int i = 1; i < 256; i += 2)
  bfree(mini[i], 4096);
CHECK(fcount[MAX_ORDER - MIN_ORDER] == 1 && total_free() == ARENA,
      "the interleaved frees close back to one 1 MiB block");
```

listing 220: exact exhaustion, then interleaved frees that cannot merge (source: c-os-cloud/samples/src/Ch19/buddy.c, lines 203-222)

One placement fact bounds the whole design: blocks are aligned to their order relative to the arena base, and the base is only as aligned as its placement. This target caps static `_Alignas` at 8192 bytes, a compile error probes it, so a 1 MiB buddy in production takes its arena from `VirtualAlloc`, whose base is page aligned, rather than from a static array. The sample checks base-relative alignment and says so in its comments.

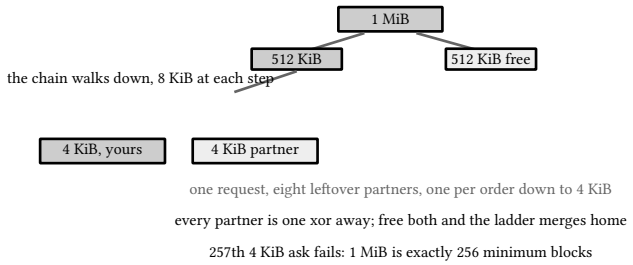


figure 78: the split chain: one 4 KiB request leaves one partner at every order below

19.6. a tracking wrapper

None of the allocators above can answer “who allocated this”. The tracking wrapper is not an allocator at all, it is a table wrapped around `malloc` and `free` that records pointer, size, and a caller supplied site id for every live allocation, and flips records to tombstones on free. The two failure modes a leak report needs fall out of the state machine: freeing a pointer with no record is a never allocated pointer, freeing a record already in the tombstone state is a double free, and both return codes are asserted with the accounting untouched.

```
static void *t_malloc(size_t n, int site) {
  void *p = malloc(n);
  if (p == NULL)
    return NULL;
  // the table keys on address: a reused address rewrites its own tombstone
  // rather than shadowing it, so a second allocation at the same address
  // stays freeable
  for (size_t i = 0; i < CAP; i++)
    if (tab[i].p == p) {
      tab[i] = (struct rec){p, n, site, LIVE};
      live_n++;
      live_bytes += n;
      sites[site]++;
      return p;
    }
  // second pass recycles tombstones: a slot whose address differs can be
  // reused safely, the trade being that double-free memory reaches back
  // only as far as the table's most recent residents
  for (size_t i = 0; i < CAP; i++) {
    if (tab[i].state != LIVE) {
      tab[i] = (struct rec){p, n, site, LIVE};
      live_n++;
      live_bytes += n;
      sites[site]++;
      return p;
    }
  }
  free(p); // table full: refuse to lose track of a live pointer
  return NULL;
```

listing 221: the wrap: record the address, size, and site; rewrite tombstones on address reuse (source: c-os-cloud/samples/src/Ch19/track.c, lines 48-76)

```
static int t_free(void *p) {
  if (p == NULL)
    return 3;
  struct rec *r = find(p);
  if (r == NULL || r->state == FREE_SLOT)
    return 1;
  if (r->state == TOMB)
    return 2;
  r->state = TOMB;
  live_n--;
  live_bytes -= r->n;
  sites[r->site]--;
  free(p);
  return 0;
}

static size_t leaks(void) { return live_n; }
```

listing 222: the four frees: clean, unknown, double, and the null arm (source: c-os-cloud/samples/src/Ch19/track.c, lines 80-97)

The wrapper states its own limits as checks. The table keys on address, so an address the heap reuses rewrites its own tombstone rather than shadowing it, and a recycled tombstone whose address differs loses that old address's double-free memory, the trade stated in the comment. Capacity is a hard limit: 64 slots track 64, and the 65th allocation is refused loudly rather than silently lost. The clock prints the question's price, 100000 tracked pairs of

32 bytes against the bare loop, 2.66x to 3.68x across the runs observed while writing, and that ratio is the honest reason production allocators sample rather than record everything.

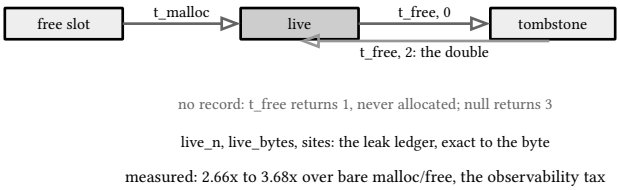


figure 79: three record states and the transitions the two error returns guard

19.7. the production survey

Four production allocators, all cited from their own documentation and none of them installed on this machine, every one a hybrid of the shapes above. The ucrt heap this book links is the baseline: its page promises fundamental alignment and admits the maintenance tax, and its debug builds swap in a tracking heap, the crt debug heap, that records exactly what `track.c` records. jemalloc describes itself as "a general purpose malloc(3) implementation that emphasizes fragmentation avoidance and scalable concurrency support", became the FreeBSD libc allocator in 2005, and has shipped heap profiling and tuning hooks since 2010: the buddy's merge discipline scaled by arenas. mimalloc, Microsoft's allocator born in the Koka and Lean runtimes, calls itself "a general purpose allocator with excellent performance characteristics", keeps the core near 10k lines of code, shards its free lists per roughly 64 KiB page "which reduces fragmentation and increases locality", reports about 0.2 percent metadata overhead, and notes that freeing from another thread "can now be a single CAS": the slab, sharded. tcmalloc is "Google's customized implementation of C's `malloc()` and C++'s `operator new`", "a fast, multi-threaded malloc implementation", and its design page describes the exact two layers this chapter built, "a thread cache contains one singly linked list of free objects per size-class" over a backend, with small allocations "mapped onto one of 60-80 allocatable size-classes" and the same 12-rounds-to-16 example `round16` compresses. The survey is documentation-verified only, the same boundary the cloud chapters use: no benchmark here compares against an allocator that is not installed.

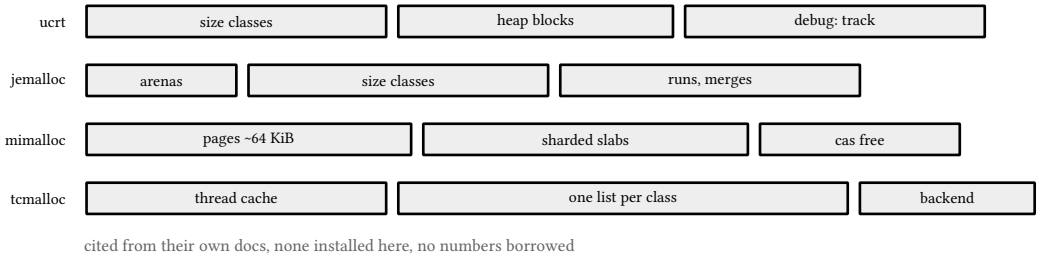


figure 80: four production heaps as hybrids of this chapter's shapes

19.8. the particle kernel's memory budget

The capstone kernel in chapter 21 simulates N particles behind these allocators, 8 `double` fields each, position, velocity, mass, and charge, and the budget is pure arithmetic this chapter can close. One particle is 64 bytes, so 1 million particles are 64 MiB, 100 million are 6.4 GiB, and 1 billion are 64 GiB, and the layout does not change the total: 8 doubles pad to nothing, the array-of-structs and struct-of-arrays forms of this data are the same bytes, the difference chapter 13 measured is which lanes a sweep touches, not how many bytes exist. The bookkeeping is where the layouts diverge. Behind the bump allocator the kernel pays zero per particle beyond the 64. Behind a chapter 14 free list it would pay a 32-byte header per particle, 50 percent on top, 32 MiB stranded per million particles and 3.2 GiB per hundred million. The kernel uses bump and slab shapes because the budget says so, and chapter 21 measures what the arithmetic predicts.

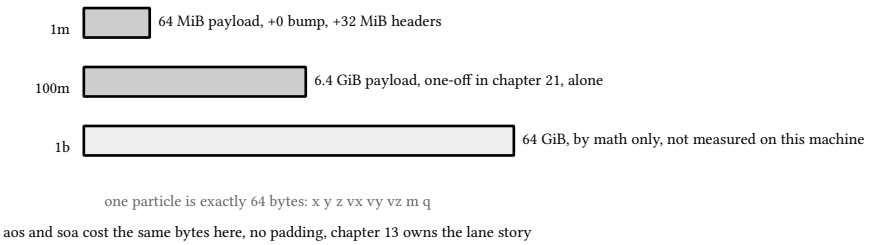


figure 81: the budget ladder: 8 doubles per particle, three scales, two bookkeeping costs

VERIFY: one chapter, five allocators, one clock

63 checks run at `-O0` and assert only deterministic facts: the alignment ladder, header sizes, class arithmetic, split chains, merge ladders, permutations, lifo orders, and the tracker's four frees. 3 ir substrings and 2 roundtrip legs are asserted on `-O2` output the runtime never executes. The timings quoted in this chapter, bump 8.8x to 10.3x over the ucrt pair, the slab 7.4x to 8.3x, the tracker 2.66x to 3.68x, are printed observations from the runs during writing, and no check anywhere in the chapter depends on them.

sources: learn.microsoft.com, the `malloc` page (fundamental alignment, the maintenance-information sentence) and the acquiring high-resolution time stamps page, accessed 2026-09-13; jemalloc.net, mimalloc (microsoft.github.io/mimalloc), and tcmalloc (google.github.io/tcmalloc and its design page), accessed 2026-09-13, cited and documentation-verified only, none installed here. Alignment facts, the 8192-byte `_Alignas` cap, the absent `max_align_t`, and all timings probed on this machine 2026-09-13. Sample behavior verified by `make verify-c`, 63 checks in chapter 19 of the samples suite.

20. measurement, profiling, and p99

Chapters 13 through 18 printed clocks everywhere and never once checked one. This chapter is what that discipline costs and buys: the clock measured on this box, the percentile arithmetic written out by hand, a sampling profiler built from SuspendThread and a context record, and the variance of a real workload quoted as a range because a single number is a coincidence wearing a unit. Every deterministic fact is asserted, every timing is printed, and the harness at the end is the one the capstone in chapter 21 reuses.

20.1. clocks, measured

The clock is QueryPerformanceCounter, and the first thing the sample does is ask it what it is. The frequency is fixed at boot and identical across processors, the timestamps page says to query it once and cache it, and on this box it answers 10 MHz: one tick every 100.0 nanoseconds. Monotonicity is not assumed from the faq's sentence, "QPC does not go backward", it is measured, a thousand successive reads with no decrease. The cost of reading is measured too, 14 to 16 ns per call across the runs observed while writing, which is the overhead every timed section in this book already carries.

```
BOOL ok = QueryPerformanceFrequency(&freq);
CHECK(ok && freq.QuadPart ≥ 1000000,
      "the frequency is fixed at boot, at least a megatick here");
printf("frequency: %lld ticks/s\n", (long long)freq.QuadPart);
double ns_per_tick = 1e9 / (double)freq.QuadPart;
printf("resolution: %.1f ns per tick\n", ns_per_tick);
CHECK(ns_per_tick ≥ 0.1 && ns_per_tick ≤ 1000.0,
      "each tick is between 0.1 and 1000 ns on this box");
```

listing 223: the frequency once, cached, and the resolution it implies (source: c-os-cloud/samples/src/Ch20/qpc.c, lines 36-43)

A sleep cross-checks the clock against GetTickCount64, whose own granularity is the 15.625 ms scheduler tick. A 10 ms sleep measured 10.59 to 21.66 ms on the qpc clock across runs while the tick counter stepped 16 or 31, both clocks telling the truth at their own resolution: sleep is a promise to sleep at least that long, and both bounds are asserted, not the value. The last probe is the faq's own warning about conversion order, scale before dividing, and the two orders agree to well under a microsecond at millisecond magnitudes here, printed, not checked.

```
// monotonicity: the faq's own words, "qpc does not go backward"
int mono = 1;
long long biggest_gap = 0;
LARGE_INTEGER prev = {0};
QueryPerformanceCounter(&prev);
for (int i = 0; i < 1000; i++) {
  LARGE_INTEGER t = {0};
  QueryPerformanceCounter(&t);
  if (t.QuadPart < prev.QuadPart)
    mono = 0;
  if (t.QuadPart - prev.QuadPart > biggest_gap)
    biggest_gap = t.QuadPart - prev.QuadPart;
  prev = t;
}
CHECK(mono, "1000 successive reads never went backward");
CHECK(biggest_gap * 1000000000LL / freq.QuadPart < 100000000LL,
      "and no gap between adjacent reads reaches a tenth of a second");
```

listing 224: monotonicity as a measured fact, and the gap bound that keeps it honest (source: c-os-cloud/samples/src/Ch20/qpc.c, lines 45-62)

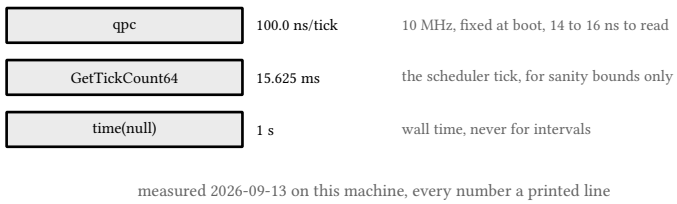


figure 82: the clock ladder on this box: three time bases, three resolutions, one chosen

20.2. the doctrine: assert the count, print the clock

The rule this book runs on was stated in chapter 13 and has not moved: assertions are for facts that cannot drift, prints are for facts that can. A checksum, a count of records, an offset, a sorted order: these are deterministic, so a check pins them and the gate holds them forever. A duration, a rate, a ratio: these are machine facts, so they are printed with their unit and quoted in prose as the range observed across runs, never as a single blessed number, and no check anywhere depends on them. The reason is not aesthetics. Chapter 14's allocator holds under every timing, chapter 19's ratios moved 8.8x to 10.3x between two runs of the same binary on the same afternoon, and a suite that asserted either of those would be a suite that fails on Tuesdays. The one thing the doctrine adds in this chapter is the middle case: a timing harness can be deterministic in what it measures and statistical in what it observes, and that is exactly what percentiles are for.

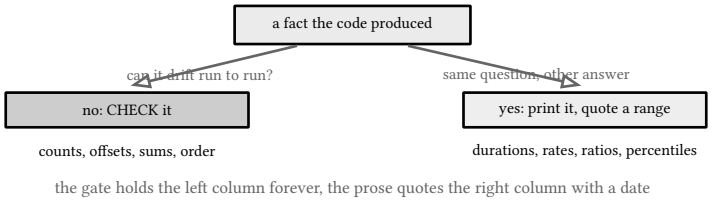


figure 83: the doctrine as a decision: every fact goes down one of two legs

20.3. percentiles, nearest-rank, by hand

An average hides the tail, and the tail is where the users are. The sample works in permilles so every rank is integer arithmetic: the rank of p is `ceil(p * n / 1000)` written as `(p * n + 999) / 1000`, and the percentile is the sorted array's value at that rank, nothing interpolated. The hand-checkable arrays come first, the identity array 1 through 10 and an array of duplicates, and they pin the edges: p50 of ten values is the 5th, p95 is the 10th because 9.5 rounds up, and nearest-rank has no p0 because rank zero names nothing.

```
static size_t rank_of(size_t permille, size_t n) {
  return (permille * n + 999) / 1000;
}

static double pct(const double *sorted, size_t n, size_t permille) {
  return sorted[rank_of(permille, n) - 1];
}
```

listing 225: the whole method: an integer ceil and one array index (source: c-os-cloud/samples/src/Ch20/p99.c, lines 28-34)

The seeded sample of 1000 carries the invariants that make the method trustworthy at scale, sorted, monotone in p, and every percentile an actual element of the sample, and because the generator is splitmix64 with a fixed seed, the sample is byte-identical run after run: p50 5267, p95 9532, p99 9915, p99.9 9988, min 17, max 9996, checked, not quoted.

```c
// a seeded sample of 1000, sorted once, invariants asserted
enum { N = 1000 };
uint64_t rng = 0x20260913ULL;
double *sample = malloc(N * sizeof *sample);
for (int i = 0; i < N; i++)
  sample[i] = (double)(splitmix64(&rng) % 10000) + 1.0;
qsort(sample, N, sizeof *sample, cmp_double);
int sorted_ok = 1;
for (int i = 1; i < N; i++)
  if (sample[i - 1] > sample[i])
    sorted_ok = 0;
CHECK(sorted_ok, "the prng sample is sorted");
double p50 = pct(sample, N, 500), p95 = pct(sample, N, 950),
       p99 = pct(sample, N, 990), p999 = pct(sample, N, 999);
printf("1000 seeded samples: p50=%.0f p95=%.0f p99=%.0f p99.9=%.0f "
       "min=%.0f max=%.0f\n",
       p50, p95, p99, p999, sample[0], sample[N - 1]);
CHECK(p50 ≤ p95 && p95 ≤ p99 && p99 ≤ p999 && p999 ≤ sample[N - 1],
      "percentiles are monotone in p and capped by the max");
CHECK(elem_of(p50, sample, N) && elem_of(p95, sample, N) &&
          elem_of(p999, sample, N),
      "nearest-rank percentiles are always actual samples");
CHECK(sample[0] == 17.0 && sample[N - 1] == 9996.0,
      "the seeded sample is reproducible: min 17, max 9996, every run");
```

listing 226: the seeded sample, its invariants, and the reproducibility check that pins min and max (source: c-os-cloud/samples/src/Ch20/p99.c, lines 109-132)

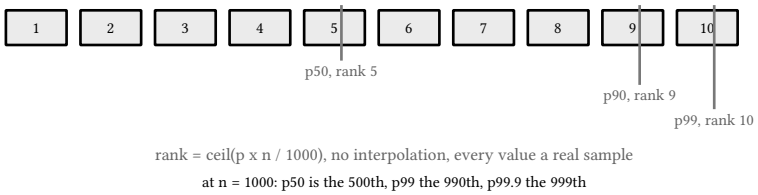


figure 84: a sorted sample of ten with the rank marks p50, p90, p99 sit on

20.4. a sampling profiler thread

A tracing profiler sees every call and slows everything down. A sampling profiler sees almost nothing and costs almost nothing: suspend the thread, read its instruction pointer, resume it, repeat about a thousand times a second, and count where the pointers land. The documented pairing is exact, the `GetThreadContext` page says “You cannot get a valid context for a running thread. Use the SuspendThread function to suspend the thread before calling GetThreadContext”, and the sample does precisely that, with `CONTEXT_CONTROL` asking only for the control registers the instruction pointer lives in.

```
  long long counts[4] = {0, 0, 0, 0}; // worker, hot, cold, unknown
  long long total = 0, susp_prev0 = 0, resume_prev1 = 0, gtc_fail = 0;
  while (total < 1600) {
    DWORD prev = SuspendThread(h);
    if (prev != 0) {
      gtc_fail++;
      ResumeThread(h);
      continue;
    }
    CONTEXT ctx;
    ZeroMemory(&ctx, sizeof ctx);
    ctx.ContextFlags = CONTEXT_CONTROL;
    if (GetThreadContext(h, &ctx)) {
      int sym = attribute(ctx.Rip);
      if (sym >= 0)
        counts[sym]++;
      else
        counts[3]++;
      total++;
    } else {
      gtc_fail++;
    }
    if (ResumeThread(h) != 1)
      gtc_fail++;
    else
      resume_prev1++;
    susp_prev0++;
    Sleep(1); // ~1 kHz
  }
```

listing 227: the sampling loop: suspend, read the rip, attribute, resume, sleep a millisecond (source: c-os-cloud/samples/src/Ch20/sampler.c, lines 120-148)

The worker is built to be profiled: two `noinline` functions burning a fixed 3:1 ratio of iterations, so the answer is known before the question is asked. The suspend and resume counts pin the api contract, `SuspendThread` returns the previous suspend count, 0 here, every time, and `ResumeThread` returns the pre-resume count, 1, every time, 1600 sample cycles with zero failures. Every sample attributes to exactly one symbol, and the histogram recovers the construction: 1199 samples in `spin_hot` against 400 in `spin_cold`, 74.9 against 25.0 percent, a measured ratio of 3.00 to 1 against the 3.0 it was built with, with 2.94 to 3.07 across the runs observed while writing.

```
__attribute__((noinline)) static DWORD WINAPI worker(LPVOID arg) {
  volatile LONG *go = arg;
  while (*go)
    for (int r = 0; r < 8 && *go; r++) {
      spin_hot();
      spin_cold();
    }
  return 0;
}
```

listing 228: the worker: a fixed 3:1 burn between two noinline functions (source: c-os-cloud/samples/src/Ch20/sampler.c, lines 45-53)

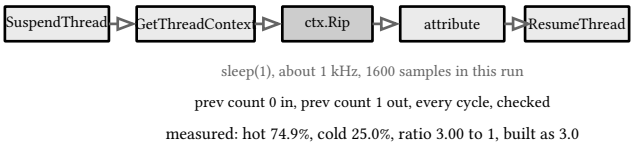


figure 85: the sampling pipeline: one sample's round trip, a thousand times a second

20.5. attribution without a toolchain

The rip is an address, not a name, and turning it into a name is the attribution problem. The sample's answer is the one real symbolizers use at their core: a table of symbols sorted by address, and each sample attributed to the greatest symbol at or below it. The table is emitted by the sample itself, three names and addresses, because the binary knows its own functions, and the walk is four lines. The measured result on this workload is total attribution, zero unknown samples in 1600, because the worker never leaves its own two functions plus the thread proc, and any address below every symbol attributes to nothing rather than to a wrong name, asserted on a deliberate out-of-range probe.

The platform's own answer is Event Tracing for Windows, "an efficient kernel-level tracing facility that lets you log kernel or application-defined events to a log file", with controllers, providers, and consumers, recorded by the Windows Performance Recorder, "a performance recording tool that is based on Event Tracing for Windows (ETW)", and analyzed by WPA. The honest boundary on this box, probed 2026-09-13: the tools are present, `wpr.exe` in system32 and `wpa.exe` with `xperf.exe` under the Windows Kits performance toolkit, but a capture attempt from this book's unelevated shell fails at the first step, "Failed to enable the policy to profile system performance", error 0xc5585011. No capture is taken, no WPA graph is quoted, and the hand sampler above is the profiler this book actually runs. The `SuspendThread` page carries its own warning worth keeping, the function "is primarily designed for use by debuggers. It is not intended to be used for thread synchronization", and the sampler respects it: the worker owns no lock while sampled.

```
static int attribute(DWORD64 rip) {
  int best = -1;
  for (int i = 0; i < nsyms; i++)
    if (syms[i].addr ≤ rip)
      best = i;
  return best;
}
```

listing 229: attribution: greatest symbol at or below the rip, one linear walk (source: c-os-cloud/samples/src/Ch20/sampler.c, lines 84-90)

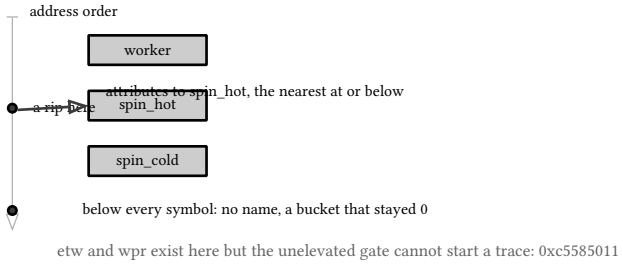


figure 86: attribution: each rip walks down to the nearest symbol at or below it

20.6. warmup, drift, and variance, measured

The workload is deliberately boring, sum 8 million seeded doubles, and the check that matters is that it is exactly the same boring every time: the generator fills the identical array from the same seed, and all eight rounds produce the identical checksum to the last bit. The timings then refuse to sit still, 16.07 to 19.33 ms per round across the runs observed while writing, a spread of 15.7 to 17.4 percent between min and median, with the first round landing anywhere from 94 to 108 percent of the median. That is this machine on this afternoon: no clean warmup penalty, no steady state, noise that a single-run number would silently launder into a fact.

```
double ms[ROUNDS];
int same = 1;
for (int r = 0; r < ROUNDS; r++) {
  double t0 = ms_now();
  double sum = 0;
  for (size_t i = 0; i < N; i++)
    sum += a[i];
  double t1 = ms_now();
  if (sum ≠ truth)
    same = 0;
  ms[r] = t1 - t0;
}
CHECK(same, "eight rounds, one checksum, identical to the last bit");

double lo = dmin(ms, ROUNDS), hi = dmax(ms, ROUNDS);
double med = dmedian(ms, ROUNDS);
CHECK(lo ≤ med && med ≤ hi, "min, median, max are ordered");
for (int r = 0; r < ROUNDS; r++)
  printf("round %d: %.2f ms\n", r, ms[r]);
```

listing 230: eight rounds, one checksum, and the statistics printed as ranges (source: c-os-cloud/samples/src/Ch20/variance.c, lines 102-120)

The statistics helpers are checked against hand-computable arrays before they are trusted, the median of three and of four, min and max bracketing, because a percentile library that is wrong in the small is wrong in the large. The moral for every later chapter, including the capstone: report min, median, and max over repeated runs, say how many runs, and let the spread be part of the answer.

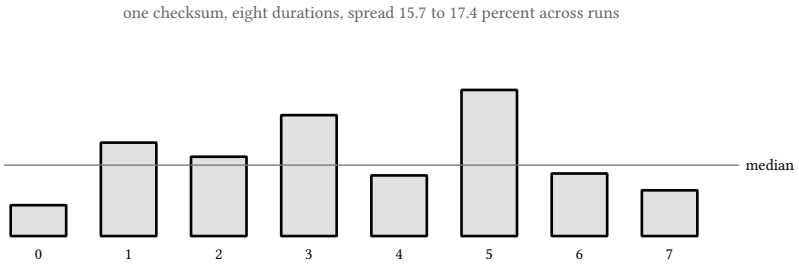


figure 87: eight rounds of one workload: the spread is the answer, not noise around it

20.7. the p99 harness this book reuses

Everything above condenses into one small type: a buffer of intervals, a push, and a report that sorts a copy and prints p50 through p99.9 with the max. The capstone pushes commit latencies into it by the thousand and quotes the tail, because ingestion throughput is an average and an average is exactly the wrong summary for a database commit. On the toy workload here, 2000 timed iterations of a 100-addition step, the harness prints p50 300 ns, p95 400 ns, p99 400 ns, p99.9 400 ns, max 15.5 microseconds: a flat little distribution with one scheduler blip at the top, and the honesty is in the max line the averages never show.

```c
// the harness: intervals pushed in ns, percentiles over a sorted copy
typedef struct {
  double *buf;
  size_t n, cap;
} harness;

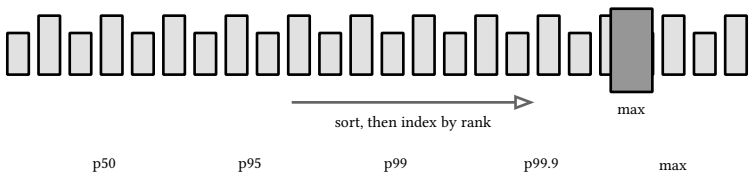
static void hpush(harness *h, double v) {
  if (h->n < h->cap)
    h->buf[h->n++] = v;
}

static void hreport(harness *h, const char *label) {
  double *sorted = malloc(h->n * sizeof *sorted);
  for (size_t i = 0; i < h->n; i++)
    sorted[i] = h->buf[i];
  qsort(sorted, h->n, sizeof *sorted, cmp_double);
  printf("%s: n=%zu p50=%.1f p95=%.1f p99=%.1f p99.9=%.1f max=%.1f ns\n", label,
         h->n, pct(sorted, h->n, 500), pct(sorted, h->n, 950),
         pct(sorted, h->n, 990), pct(sorted, h->n, 999), sorted[h->n - 1]);
  free(sorted);
}
```

listing 231: the harness: push intervals, sort a copy, report the tail (source: c-os-cloud/samples/src/Ch20/p99.c, lines 56-76)

```c
  // the real harness: 2000 timed iterations of a tiny deterministic step
  enum { K = 2000 };
  harness h = {malloc(K * sizeof *h.buf), 0, K};
  volatile double sink = 0;
  for (int i = 0; i < K; i++) {
    double t0 = ns_now();
    double acc = 0;
    for (int j = 0; j < 100; j++)
      acc += (double)j * 0.5;
    sink += acc;
    double t1 = ns_now();
    hpush(&h, t1 - t0);
  }
```

listing 232: one timed push loop: the pattern every later measurement fills in (source: c-os-cloud/samples/src/Ch20/p99.c, lines 134-146)



the tail is the product: chapter 21 pushes 1000+ commit latencies through this shape

figure 88: the harness shape: many intervals in, five numbers out, the tail on the right

VERIFY: one chapter, one clock, one tail

39 checks run at -O0 and assert only deterministic facts: rank arithmetic on hand-checkable arrays, the reproducible sample's pinned min and max, the suspend and resume counts, total attribution, the identical checksums, and the statistics helpers against known inputs. No ir pins and no roundtrip legs run in this chapter, every sample prints timings so their stdouts must stay out of byte-comparisons. The timings quoted in this chapter, the 10 MHz clock at 14 to 16 ns per read, sleep bounds, the 15.7 to 17.4 percent spread, the 3.00

> to 1 recovered ratio, are printed observations from the runs during writing, and no check anywhere depends on them.

20.8. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the shared sqlite contract at 10m rows, insert through 10k-row batched transactions, the sql group by, the manual cursor walk, and the p99 of one 10k-row commit. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	

Table 1: the two kernels across the six language books: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row commit, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and 21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

sources: learn.microsoft.com, the acquiring high-resolution time stamps page, and the SuspendThread, GetThreadContext, about Event Tracing, and Windows Performance Recorder pages, accessed 2026-09-13. Clock facts, qpc cost, sleep behavior, sampler counts, and variance measured on this machine 2026-09-13; the wpr capture attempt and its 0xc5585011 failure probed the same day from this book's unelevated shell. Sample behavior verified by `make verify-c`, 39 checks in chapter 20 of the samples suite.

21. capstone: particle kernel and row cruncher

The capstone runs the two workloads this book spent twenty chapters building tools for: a million particles stepped behind the chapter 19 allocators, and ten million database rows through each corpus engine, sqlite 3.53.4 and duckdb 1.5.5, both dlls loaded at runtime. The gate run stays a smoke, 10 thousand particles and 1236 rows per engine with every checksum asserted, and the measured ladder is opt-in through `argv`, quoted here from real runs dated 2026-09-13, 2026-09-14, and 2026-09-20. The one-offs at 100 million ran alone, sequential, and were deleted after. Nothing in this chapter is extrapolated except the billion-row arithmetic, which says so out loud.

21.1. the kernel behind three allocators

One particle is 8 `double` fields, position, velocity, mass, and charge, and one step is pure arithmetic: charge pushes velocity in a constant field, mass weighs the pull, drag slows everything, position advances. The same seed fills both layouts, so the array-of-structs and struct-of-arrays kernels carry identical values, and after 16 steps the two checksums still match to the last bit. That equality is the capstone's spine, asserted at every scale including 100 million, where both layouts print checksum 15067410024.160217.

```
__attribute__((noinline)) static void step_aos(particle *p, size_t n) {
  const double dt = 0.001, e = 2.0, drag = 0.02;
  for (size_t i = 0; i < n; i++) {
    double *f = p[i].f;
    f[3] += (f[7] * e - drag * f[3]) * dt;
    f[4] += (-drag * f[4]) * dt;
    f[5] += (-9.81 * f[6] - drag * f[5]) * dt;
    f[0] += f[3] * dt;
    f[1] += f[4] * dt;
    f[2] += f[5] * dt;
  }
}

__attribute__((noinline)) static void step_soa(double *soa, size_t n) {
  const double dt = 0.001, e = 2.0, drag = 0.02;
  double *x = soa, *y = soa + n, *z = soa + 2 * n, *vx = soa + 3 * n,
         *vy = soa + 4 * n, *vz = soa + 5 * n, *m = soa + 6 * n,
         *q = soa + 7 * n;
  for (size_t i = 0; i < n; i++) {
    vx[i] += (q[i] * e - drag * vx[i]) * dt;
    vy[i] += (-drag * vy[i]) * dt;
    vz[i] += (-9.81 * m[i] - drag * vz[i]) * dt;
    x[i] += vx[i] * dt;
    y[i] += vy[i] * dt;
    z[i] += vz[i] * dt;
  }
}
```

listing 233: the step kernel twice: one struct of 8 doubles per particle, or 8 arrays of one double each (source: c-os-cloud/samples/src/Ch21/kernel.c, lines 17-43)

The setup question is chapter 19's, asked at scale: what does it cost to put a million 64-byte particles behind each allocator? Bump is not slower per particle at a million than at a thousand, because it is one carve, O(1) in the particle count, and prints 0.00 ns per particle, under half a millisecond total. The freelist carve pays a header write and a payload touch per particle, sequential and prefetch friendly, 13.3 to 14.3 ns per particle at 1 million. The slab pays a pointer chase, its free list is threaded through slots the allocator has not warmed, 22.9 to 26.9 ns per particle, and at 100 million the gap narrows to 19.3 to 33.3 for both as the last cache levels stop absorbing the streams.

```
static double time_setup(size_t n, int which) {
  // the freelist branch carves payload plus a 16-byte header per
  // particle, so the arena is sized for the widest allocator
  void *arena = malloc(n * (sizeof(particle) + 32) + 4096);
  double t0 = ns_now();
  if (which == 0) { // bump: one carve
    bump b = {arena, 0, n * sizeof(particle) + 4096};
    particle *p = bump_alloc(&b, n * sizeof(particle));
    p[0].f[0] = 1;          // touch it so the pages are real
  } else if (which == 1) { // slab: thread the list through the slots, pop all
    particle *head = NULL;
    for (size_t i = n; i-- > 0;) {
      particle *slot = (particle *)((char *)arena + i * sizeof(particle));
      *(particle **)slot = head;
      head = slot;
    }
    for (particle *p = head; p != NULL;) {
      particle *next = *(particle **)p; // read the link before the payload
      p->f[0] = 1;
      p = next;
    }
  } else { // freelist: first-fit carve with a header, chapter 14 shape
    size_t off = 0;
    for (size_t i = 0; i < n; i++) {
      size_t need = (sizeof(particle) + 15) / 16 * 16 + 16;
      *(size_t *)((char *)arena + off) = need; // the header's size field
      ((particle *)((char *)arena + off + 16))->f[0] = 1;
      off += need;
    }
  }
```

listing 234: the same million particles allocated behind bump, slab, and the chapter 14 freelist (source: c-os-cloud/samples/src/Ch21/kernel.c, lines 96-125)

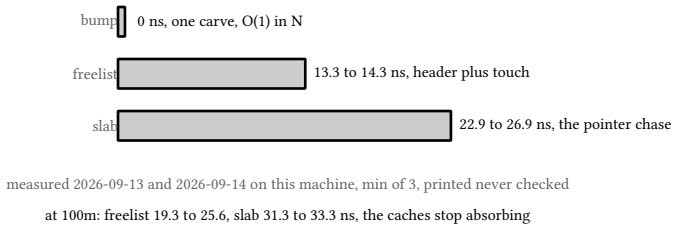


figure 89: setup cost per particle behind the three chapter 19 allocators, measured at 1m

21.2. array of structs against struct of arrays, measured

Chapter 13 predicted the layout difference as a cache decision and measured near parity on its 4-field particles. The capstone's 8-field particle sharpens the question, and the answer depends entirely on the optimization level, which is itself the finding. Under the gate's `-O0` build the two layouts land within noise of each other, aos 8.05 to 8.49 and soa 7.24 to 7.43 ns per particle per step, 1.07x to 1.14x, because the interpreter-shaped inner loop dominates both. At `-O2` the same source separates: aos 6.15 to 6.75, soa 4.53 to 4.91, 1.35x to 1.38x, the soa kernel's six streaming arrays against the aos kernel's stride-64 walk through all eight fields. The numbers are quoted per optimization level because a layout claim that does not name its `-O` level is not a claim.

```
particle *aos = layout == 2 ? malloc(sizeof *aos) : malloc(n * sizeof *aos);
double *soa =
    layout == 1 ? malloc(sizeof *soa) : malloc(n * FIELDS * sizeof *soa);
CHECK(aos != NULL && soa != NULL, "both layouts allocated");
if (layout != 1)
  fill_soa(soa, n);
if (layout != 2)
  fill_aos(aos, n);

if (layout == 0) {
  double c0 = checksum_aos(aos, n), c1 = checksum_soa(soa, n);
  CHECK(c0 == c1, "both layouts carry the same values before stepping");
  for (int s = 0; s < STEPS; s++) {
    step_aos(aos, n);
    step_soa(soa, n);
  }
  CHECK(checksum_aos(aos, n) == checksum_soa(soa, n),
        "16 steps later the checksums still match bitwise");
  CHECK(checksum_aos(aos, n) != c0, "and the steps actually moved them");
} else {
  double c0 = layout == 1 ? checksum_aos(aos, n) : checksum_soa(soa, n);
  for (int s = 0; s < STEPS; s++) {
    if (layout == 1)
      step_aos(aos, n);
    else
      step_soa(soa, n);
  }
  double c1 = layout == 1 ? checksum_aos(aos, n) : checksum_soa(soa, n);
  CHECK(c1 != c0, "the single layout stepped and moved");
}
```

listing 235: both layouts from one seed, and the bitwise agreement asserted before and after 16 steps (source: c-os-cloud/samples/src/Ch21/kernel.c, lines 191-220)

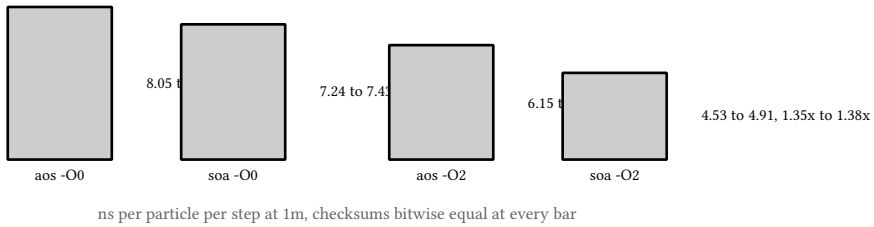


figure 90: the same kernel at two optimization levels: the layout gap is born in the optimizer

21.3. the row cruncher

The second workload is the corpus's own database engine, loaded the way the machine facts prescribe: `LoadLibraryA("sqlite3.dll")` from beside the exe, every function pointer typedefed and resolved with a direct cast, and the first check pins the version, 3.53.4, the same dll the lua book's ffi chapter links. The gate stages the dll into the run directory exactly the way its asan leg stages the sanitizer runtime, and the gate's preflight names the missing piece and its build command when a worktree lacks it.

```
static int load_dll(void) {
  HMODULE h = LoadLibraryA("sqlite3.dll");
  if (h == NULL)
    return 0;
  p_libversion = (fn_libversion)GetProcAddress(h, "sqlite3_libversion");
  p_open = (fn_open)GetProcAddress(h, "sqlite3_open");
  p_exec = (fn_exec)GetProcAddress(h, "sqlite3_exec");
  p_close = (fn_close)GetProcAddress(h, "sqlite3_close");
  p_prepare = (fn_prepare)GetProcAddress(h, "sqlite3_prepare_v2");
  p_bind_text = (fn_bind_text)GetProcAddress(h, "sqlite3_bind_text");
  p_bind_double = (fn_bind_double)GetProcAddress(h, "sqlite3_bind_double");
  p_step = (fn_step)GetProcAddress(h, "sqlite3_step");
  p_reset = (fn_reset)GetProcAddress(h, "sqlite3_reset");
  p_finalize = (fn_finalize)GetProcAddress(h, "sqlite3_finalize");
  p_column_double =
      (fn_column_double)GetProcAddress(h, "sqlite3_column_double");
  p_column_text = (fn_column_text)GetProcAddress(h, "sqlite3_column_text");
  p_column_int64 = (fn_column_int64)GetProcAddress(h, "sqlite3_column_int64");
  return p_libversion && p_open && p_exec && p_close && p_prepare &&
         p_bind_text && p_bind_double && p_step && p_reset && p_finalize &&
         p_column_double && p_column_text && p_column_int64;
}
```

listing 236: the load: one dll, thirteen function pointers, direct single casts (source: c-os-cloud/samples/src/Ch21/rowcrunch.c, lines 48-69)

The ingestion is the workload kit's contract: 412 stations, 8 `REAL` columns, prepared inserts bound per row, 10k-row transactions. One sentence of the bind contract is load-bearing enough to quote from the vendored header: `SQLITE_TRANSIENT` "may be passed to indicate that the object is to be copied prior to the return from" the bind call, and a null destructor instead means the pointer "must remain valid until either the prepared statement is finalized or the same SQL parameter is bound to something else". This sample learned that the hard way, a stack buffer bound as static read after the stack frame died, all 1236 rows collapsing into one garbage station, and the fix is the `(void *)-1` transient sentinel in the listing.

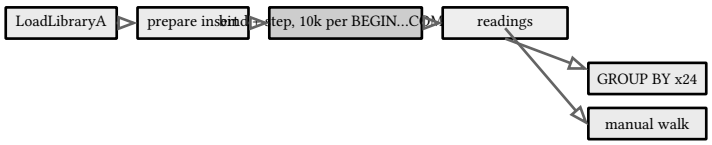
```
static void bind_row(uint64_t *rng, long station) {
  char name[8];
  snprintf(name, sizeof name, "st%03ld", station);
  // SQLITE_TRANSIENT ((void *)-1): sqlite copies the text before this
  // stack buffer dies. a NULL destructor means static storage and the
  // bind would dangle by the time step reads it
  p_bind_text(ins, 1, name, -1, (void *)-1);
  for (int c = 0; c < FIELDS; c++)
    p_bind_double(ins, 2 + c, sm_d(rng, 100.0));
```

listing 237: the bind: a stack buffer must be copied, the transient sentinel says so (source: c-os-cloud/samples/src/Ch21/rowcrunch.c, lines 75-83)

The aggregation runs twice over identical rows: one `GROUP BY` carrying min, max, and avg for all 8 columns, and a manual cursor walk with per-station accumulators, the shape of the one billion rows challenge. The smoke asserts they agree, station set, min and max bitwise on all 8 columns, averages inside float addition-order effects.

```
// aggregation (a): one GROUP BY, min max avg x8, one row per station
static double agg_sql(statt *st, long *groups) {
  static const char *Q =
      "SELECT station, MIN(v0), MAX(v0), AVG(v0), MIN(v1), MAX(v1), AVG(v1),"
      " MIN(v2), MAX(v2), AVG(v2), MIN(v3), MAX(v3), AVG(v3),"
      " MIN(v4), MAX(v4), AVG(v4), MIN(v5), MAX(v5), AVG(v5),"
      " MIN(v6), MAX(v6), AVG(v6), MIN(v7), MAX(v7), AVG(v7)"
      " FROM readings GROUP BY station ORDER BY station";
```

listing 238: aggregation (a): 24 aggregates over one group by (source: c-os-cloud/samples/src/Ch21/rowcrunch.c, lines 120-127)



the smoke asserts both walks agree: stations, bitwise min and max, avg within float order

412 stations, 8 REAL columns, splitmix64-seeded values, one dll, version pinned 3.53.4

figure 91: the cruncher: one dll, prepared inserts in 10k batches, two aggregations over the same rows

21.4. chunked ingestion and the p99 of a commit

Throughput is an average, and a commit is not an average, so every 10k-row transaction lands its whole latency in the chapter 20 harness. At 10 million rows, 1000 chunks, the insertion sustains 468,803 rows per second and the commit distribution prints p50 21.4 ms, p95 24.1 ms, p99 28.3 ms, p99.9 105.4 ms, max 127.2 ms: the median chunk is tight, but one commit in a thousand waits four to five medians, and the max waits six, the file system flushing behind the log. At 100 million rows the shape holds, p50 21.5, p95 24.8, p99 29.8, p99.9 129.4, max 158.2 ms over 8192 chunks, the tail thinning by count but never disappearing. A service quoting only its 469k rows per second is quoting the p50 and hiding the p99.9.

```
static double insert_rows(long n, harness *chunks) {
  uint64_t rng = 0x20260914ULL;
  char *err = NULL;
  long row = 0;
  double t_all0 = ns_now();
  for (long done = 0; done < n;) {
    long batch = n - done < 10000 ? n - done : 10000;
    double t0 = ns_now();
    if (p_exec(db, "BEGIN", NULL, NULL, &err) != 0)
      return -1;
    for (long i = 0; i < batch; i++) {
      bind_row(&rng, row++ % STATIONS);
      if (p_step(ins) != 101) // SQLITE_DONE
        return -1;
      p_reset(ins);
    }
    if (p_exec(db, "COMMIT", NULL, NULL, &err) != 0)
      return -1;
    hpush(chunks, ns_now() - t0);
    done += batch;
  }
  return ns_now() - t_all0;
}
```

listing 239: ingestion: batched transactions, every chunk latency pushed into the harness (source: c-os-cloud/samples/src/Ch21/rowcrunch.c, lines 89-111)

The aggregation split is the other honest number. At 10 million rows the group by sustains 584,696 rows per second and the manual walk 2,242,885, 3.84x; at 100 million the walk holds 2,269,078 while the group by drops to 454,377, 4.99x, the sort spilling at a scale the linear scan never feels.

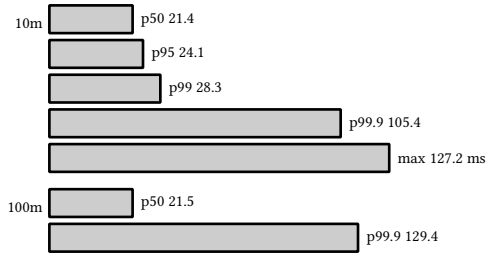


figure 92: the commit tail at 10m and 100m rows: medians tight, one in a thousand wide

21.5. the duckdb twin, measured

The row cruncher has a twin, the same workload pointed at a columnar engine. Same contract, 412 stations and 8 value columns, the identical splitmix64 row stream, the same two aggregations over identical rows asserted to agree, and the same load discipline, `LoadLibraryA("duckdb.dll")` from beside the exe, every function pointer typedefed and resolved by a direct cast, the version pinned first, v1.5.5. Two details of the contract differ and both bite. Duckdb's `REAL` is 4 bytes where sqlite's is 8, so the twin's table stores `DOUBLE` or the rows stop being bit-identical to the sqlite cruncher's, min and max included. And where sqlite needed only two opaque pointers, duckdb's types cross the boundary by value, `duckdb_result` is a 48-byte struct the engine fills through `duckdb_query`, so the twin includes the engine's own `duckdb.h` and never hand-rolls a layout.

```
static int load_dll(void) {
  HMODULE h = LoadLibraryA("duckdb.dll");
  if (h == NULL)
    return 0;
  // close and disconnect take the handle BY POINTER: the handle slot
  // is nulled on the way out, passing it by value corrupts the heap.
  // the chunk read is duckdb_fetch_chunk, the live columnar api: the
  // count-plus-index pair from older docs is deprecated and hands back
  // chunks whose data reads as zeros on 1.5.5
  p_libversion = (fn_libversion)GetProcAddress(h, "duckdb_library_version");
  p_open = (fn_open)GetProcAddress(h, "duckdb_open");
  p_connect = (fn_connect)GetProcAddress(h, "duckdb_connect");
  p_query = (fn_query)GetProcAddress(h, "duckdb_query");
  p_result_error = (fn_result_error)GetProcAddress(h, "duckdb_result_error");
  p_fetch_chunk = (fn_fetch_chunk)GetProcAddress(h, "duckdb_fetch_chunk");
  p_chunk_size = (fn_chunk_size)GetProcAddress(h, "duckdb_data_chunk_get_size");
  p_chunk_vector =
      (fn_chunk_vector)GetProcAddress(h, "duckdb_data_chunk_get_vector");
  p_vector_data = (fn_vector_data)GetProcAddress(h, "duckdb_vector_get_data");
  p_vector_validity =
      (fn_vector_validity)GetProcAddress(h, "duckdb_vector_get_validity");
  p_destroy_result =
      (fn_destroy_result)GetProcAddress(h, "duckdb_destroy_result");
  p_destroy_chunk =
      (fn_destroy_chunk)GetProcAddress(h, "duckdb_destroy_data_chunk");
  p_disconnect = (fn_disconnect)GetProcAddress(h, "duckdb_disconnect");
  p_close = (fn_close)GetProcAddress(h, "duckdb_close");
```

listing 240: the load: same discipline, one new rule, the handles close by pointer (source: c-os-cloud/samples/src/Ch21/rowcrunch_duckdb.c, lines 67-93)

The teardown rule is the one that costs an hour: `duckdb_close` and `duckdb_disconnect` take `duckdb_database *` and `duckdb_connection *`, the slot is nulled on the way out, and a handle passed by value corrupts the heap on release. The read side has its own trap, found by probing this exact dll: the chunk pair older docs carry, `duckdb_result_chunk_count` plus `duckdb_result_get_chunk`, is deprecated in 1.5.5 and hands back chunks whose doubles read as garbage. The live columnar api is `duckdb_fetch_chunk`, pulled until it returns null.

```
// the columnar bulk path: append_* writes one value into the
// appender's internal 2048-row chunk, end_row closes the row, flush
// pushes whole chunks. no statement to prepare, no transaction to
// batch by hand, and the same rng walk as rowcrunch.c so the rows are
// identical
static double append_rows(long n) {
  uint64_t rng = 0x20260914ULL;
  double t0 = ns_now();
  for (long row = 0; row < n; row++) {
    char name[8];
    snprintf(name, sizeof name, "st%03ld", row % STATIONS);
    int ok = p_append_varchar(app, name) == DuckDBSuccess;
    for (int c = 0; c < FIELDS && ok; c++)
      ok = p_append_double(app, sm_d(&rng, 100.0)) == DuckDBSuccess;
    if (!ok || p_appender_end_row(app) != DuckDBSuccess)
      return -1;
  }
  if (p_appender_flush(app) != DuckDBSuccess)
    return -1;
  return ns_now() - t0;
}
```

listing 241: ingestion without statements: the appender buffers whole chunks, end_row closes a row (source: c-os-cloud/samples/src/Ch21/rowcrunch_duckdb.c, lines 152-172)

Ingestion drops the statement machinery. Nothing to prepare, no bind contract to get wrong: `duckdb_append_varchar` copies the station name on the spot, eight `duckdb_append_double` calls stack the values, `duckdb_appender_end_row` closes the row, and the appender buffers 2048-row chunks internally, flushing whole vectors. The sample never opens a transaction, so the bench has no commit boundary to push into the chapter 20 harness and prints no percentiles. The wal still lands, and `duckdb_close` checkpoints it into the file, which is why the bench reads the db size after close.

```
duckdb_data_chunk ch;
while ((ch = p_fetch_chunk(res)) ≠ NULL) {
  idx_t n = p_chunk_size(ch);
  duckdb_vector names_v = p_chunk_vector(ch, 0);
  duckdb_string_t *names = p_vector_data(names_v);
  uint64_t *names_ok = p_vector_validity(names_v);
  double *agg[3 * FIELDS];
  for (int j = 0; j < 3 * FIELDS; j++)
    agg[j] = p_vector_data(p_chunk_vector(ch, 1 + j));
  for (idx_t r = 0; r < n; r++) {
    if (!row_ok(names_ok, r))
      continue;
    char nm[8];
    read_name(&names[r], nm);
    long k = (nm[2] - '0') * 100 + (nm[3] - '0') * 10 + (nm[4] - '0');
    if (nm[0] ≠ 's' || nm[1] ≠ 't' || k < 0 || k ≥ STATIONS)
      continue;
    for (int c = 0; c < FIELDS; c++) {
      st[k].mn[c] = agg[3 * c][r];
      st[k].mx[c] = agg[3 * c + 1][r];
      st[k].sum[c] = agg[3 * c + 2][r]; // the group's avg
    }
    st[k].seen = 1;
    (*groups)++;
  }
  p_destroy_chunk(&ch);
}
```

listing 242: aggregation (a) read back columnar: one vector per column, 2048 rows per chunk, nullness a bitset (source: c-os-cloud/samples/src/Ch21/rowcrunch_duckdb.c, lines 189-215)

The read side is the columnar lesson. A chunk is not a row set, it is one vector per column with 2048 rows contiguous in memory, doubles straight out of `duckdb_vector_get_data`, station names as `duckdb_string_t`, 12 bytes inline for the 5-char names. Nullness lives in a separate bitset, 64 rows per `uint64_t` word from `duckdb_vector_get_validity`, a set bit meaning valid and a null pointer meaning all valid, the walk checks it before it trusts any slot. The group by returns 25 vectors, the walk's chunks carry 9.

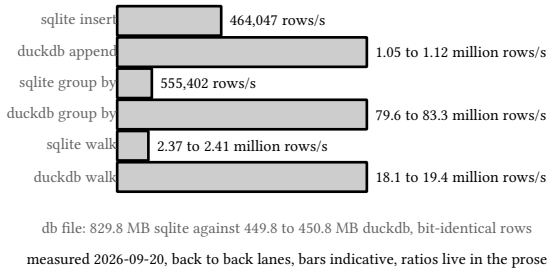


figure 93: the twin at 10 million rows, both lanes run back to back

The numbers, both 10m lanes run back to back on 2026-09-20. The appender sustains 1,049,532 to 1,123,619 rows per second against sqlite's batched 464,047, 2.3x to 2.4x, and the engine never asks the caller to pick a batch size. The group by reads back at 79.6 to 83.3 million rows per second against sqlite's 555,402 in the same run, 143x to 150x. The chunk walk runs at 18.1 to 19.4 million rows per second against the sqlite cursor walk's 2,374,133, 7.6x to 8.2x, and the duckdb number is the honest one, it starts at the query, because duckdb materializes all 10 million rows before the first chunk exists where sqlite's prepare touched nothing. The stored file is 449.8 to 450.8 MB against sqlite's 829.8 MB for the same rows, duckdb's compression paying half the space.

And the aggregate story inverts. Inside sqlite the manual walk beat the group by 4.27x, a linear scan that never sorts against a group by that must. Inside duckdb the group by finishes in 0.23x the walk's time, about 4.3x the

other way, because the engine aggregates vector by vector inside its own operators while the C walk decodes station names one row at a time out of materialized chunks. The 1BRC lesson is engine-shaped: a hand-rolled scan wins when the engine hands you one row at a time, and loses when it hands you vectors.

```
  int mins_maxes = 1, same_seen = 1;
  double avg_gap = 0;
  for (long k = 0; k < STATIONS; k++) {
    if (sql_st[k].seen != man_st[k].seen)
      same_seen = 0;
    if (!man_st[k].seen)
      continue;
    for (int c = 0; c < FIELDS; c++) {
      if (sql_st[k].mn[c] != man_st[k].mn[c] ||
          sql_st[k].mx[c] != man_st[k].mx[c])
        mins_maxes = 0;
      double d = sql_st[k].sum[c] - man_st[k].sum[c] / man_st[k].cnt;
      if (d < 0)
        d = -d;
      avg_gap += d;
    }
  }
  CHECK(same_seen, "both aggregations saw the same stations");
  CHECK(mins_maxes, "min and max agree bitwise on all 8 columns, all stations");
  CHECK(avg_gap < 1e-6, "averages agree to within float order effects");
```

listing 243: the same parity the sqlite smoke asserts: station set, bitwise min and max, float-order averages (source: c-os-cloud/samples/src/Ch21/rowcrunch_duckdb.c, lines 324-343)

The smoke asserts the same parity on 1236 rows in `:memory:`, 13 checks where the sqlite smoke carries 12, the extra one is the connection.

21.6. profiled, and why

The chapter 20 sampler attaches to the kernel the way it attached to its own worker: the kernel thread runs the measured ladder, the main thread suspends it about a thousand times a second, and each rip attributes to the nearest symbol at or below it. At `-O2`, 1 million particles, the histogram reads step_aos 58.8 percent, step_soa 41.2, kernel_thread 0.0, other 0.0, over 352 samples. Two whys. First, the split mirrors time, not code size: the aos kernel costs 1.35x the soa kernel, so it collects samples at 1.43x the rate, 207 against 145, attribution arithmetic doing what it should. Second, zero samples outside the two step functions means the loop plumbing, timers, threading, harness pushes, costs nothing measurable, which is why the per-particle numbers can be trusted as kernel numbers.

The first profiled run told a lie worth recording: 100 percent of samples in one function, because at `-O2` both step kernels had been inlined into the thread proc and every rip inside either attributed to the nearest surviving symbol. The fix is in the source, both step functions are `noinline`, and the lesson is chapter 20's attribution section standing up: a symbol map that does not match the binary the optimizer produced attributes confidently and wrongly. Sampling also costs: the profiled run's step numbers land 3 to 5 percent above the unprofiled ones, 6.50 to 6.56 against 6.15 to 6.38 ns for aos, and the chapter quotes them separately.

```
    if (profile) {
      syms[0] =
          (symbol){"kernel_thread", (unsigned char *)(uintptr_t)&kernel_thread};
      syms[1] = (symbol){"step_aos", (unsigned char *)(uintptr_t)&step_aos};
      syms[2] = (symbol){"step_soa", (unsigned char *)(uintptr_t)&step_soa};
      nsyms = 3;
      long long counts[4] = {0, 0, 0, 0};
      long long total = 0;
      while (total < 1200) {
        if (WaitForSingleObject(h, 0) == WAIT_OBJECT_0)
          break; // the kernel thread finished, stop sampling a corpse
        if (SuspendThread(h) != 0) {
          ResumeThread(h);
          continue;
        }
        CONTEXT ctx;
        ZeroMemory(&ctx, sizeof ctx);
        ctx.ContextFlags = CONTEXT_CONTROL;
        if (GetThreadContext(h, &ctx)) {
          int sym = attribute(ctx.Rip);
          counts[sym >= 0 ? sym : 3]++;
          total++;
        }
        ResumeThread(h);
        Sleep(1);
      }
      j.go = 0;
      WaitForSingleObject(h, INFINITE);
      CloseHandle(h);
      for (int i = 0; i < 3; i++)
        printf("profile %-9s %6lld  %5.1f%%\n", syms[i].name, counts[i],
               total ? 100.0 * counts[i] / (double)total : 0.0);
      printf("profile %-9s %6lld  %5.1f%%\n", "other", counts[3],
```

listing 244: the chapter 20 sampler attached to the kernel: suspend, read the rip, attribute, resume (source: c-os-cloud/samples/src/Ch21/kernel.c, lines 243-275)

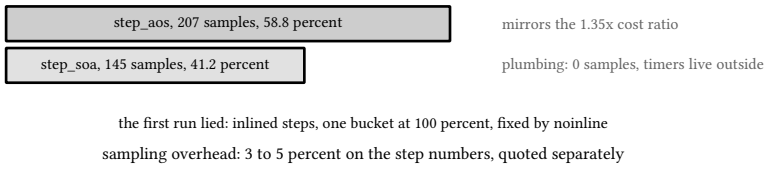


figure 94: the profiled histogram and what each bucket means

21.7. the honest limits

The 100 million particle one-off ran alone, sequential, one layout at a time because both cannot fit in this machine's memory at once: soa 5.768 and aos 7.229 ns per particle per step, 1.25x, checksum 15067410024.160217 bitwise identical across layouts, process exited and its 6.4 GiB freed. The 100 million row one-off ran the same protocol: 451,209 rows per second inserted, the aggregate and p99 numbers in the sections above, an 8330.8 MB database deleted after the run. One billion is arithmetic, not a measurement: 64 GiB of particles or roughly 83 GB of database at the observed 8.3 KB per 1000 rows, not measured on this machine, and the chapter will not pretend otherwise. The gate's smoke stays 10 thousand particles and 1236 rows, under a second, every checksum asserted, because the suite must stay fast and the measurements must stay honest, and the only way to have both is to keep them apart.

```c
for (;;) {
  int rc = p_step(q);
  if (rc ≠ 100) // SQLITE_ROW
    break;
  const unsigned char *s = p_column_text(q, 0);
  if (s == NULL || s[0] ≠ 's' || s[1] ≠ 't')
    continue;
  long k = (s[2] - '0') * 100 + (s[3] - '0') * 10 + (s[4] - '0');
  if (k < 0 || k ≥ STATIONS)
    continue;
  for (int c = 0; c < FIELDS; c++) {
    double v = p_column_double(q, 1 + c);
    if (v < st[k].mn[c])
      st[k].mn[c] = v;
    if (v > st[k].mx[c])
      st[k].mx[c] = v;
    st[k].sum[c] += v;
  }
  st[k].cnt++;
  st[k].seen = 1;
  (*rows)++;
}
```

listing 245: aggregation (b): the manual walk that stays linear while the group by sorts (source: c-os-cloud/samples/src/Ch21/rowcrunch.c, lines 174-195)

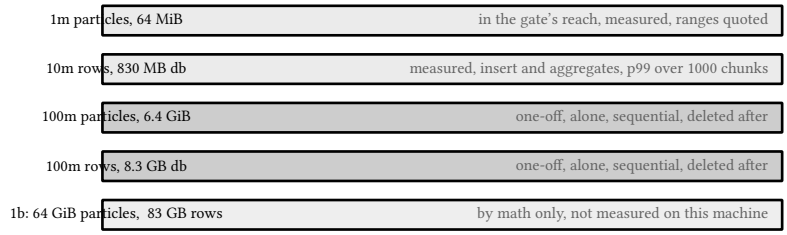


figure 95: the scale ladder: measured now, one-off once, arithmetic at the top

VERIFY: the capstone's contract

The gate run asserts 29 checks at `-O0`: 4 in the kernel smoke, both layouts seeded identically and agreeing bitwise before and after 16 steps, 12 in the sqlite row smoke, dll loaded, version pinned 3.53.4, 1236 rows inserted in transactions, both aggregations agreeing station by station, min and max bitwise, and 13 in the duckdb row smoke, the same parity over the same rows through the appender and the chunk api, version pinned v1.5.5. The measured ladder is opt-in argv, the bench script under `samples/bench/` builds its own binaries and is never invoked by `make verify`, and every number quoted in this chapter is a printed line from a run dated 2026-09-13, 2026-09-14, or 2026-09-20 on this machine, ranges where repeats exist, single lines where the run was a deliberate one-off.

21.8. the service part

The book does not stop at this capstone. Its service part, chapters 28 through 40, builds the next evidence layer on the same machine facts, a linked c23 service at `books/c-os-cloud/api`, one program plus one test exe, ten middleware layers wired into one composition, sqlite 3.53.4 still loaded at runtime beside the exe the way this chapter loads it, the composition serving 19380 with metrics on loopback 19390. The gate is `make verify-capi` with its asan leg, and the recorded state is 1130 checks, verified 2026-09-27.

sources: sqlite.org, the c interface reference and the bind family page (`SQLITE_TRANSIENT`, the destructor contract), accessed 2026-09-14, duckdb.org's C API reference, the appender, data chunk, and vector pages

(duckdb_appender, duckdb_fetch_chunk, the validity bitset, at https://duckdb.org/docs/current/clients/c/overview), accessed 2026-09-20, and the vendored amalgamation's own sqlite3.h at 3.53.4 plus the pinned duckdb.h at 1.5.5 for the struct and enum layouts; learn.microsoft.com's loader pages carry the LoadLibraryA and GetProcAddress contract this chapter leans on from earlier chapters. Kernel, allocator, sampler, and ingestion timings measured on this machine 2026-09-13, 2026-09-14, and 2026-09-20, the 100m one-offs alone and sequential with their state deleted after. Sample behavior verified by make verify-c, 29 checks in chapter 21 of the samples suite.

22. the cloud model

Chapters 2 through 18 owned one machine. The chapters from here on rent machines by the hour from two providers, and this chapter owns no compiler, no sample, and no check count to lean on: every factual sentence ties to a canonical page fetched on 2026-09-12 and quoted as fetched, because provider pages drift and training data remembers the old wording. Four vocabularies carry the rest of the book: the service models that grade how much of the stack you rent, the geography model that decides where data lives and what fails together, the responsibility line that moves with every managed upgrade, and the two review frameworks the providers publish for judging the result.

22.1. iaas, paas, faas, saas

The definitions are older than every service this book provisions. NIST Special Publication 800-145, dated September 2011, defines cloud computing as "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources" that can "be rapidly provisioned and released with minimal management effort or service provider interaction", decomposed into "five essential characteristics, three service models, and four deployment models". The characteristics are on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service, the last one the reason a cost chapter exists at the end of this book. The deployment models are private, community, public, and hybrid; the public one is what chapters 23 through 24 rent.

Each service model is one capability sentence plus a list of what the consumer stops controlling. Infrastructure as a service gives the consumer the capability to "provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications", keeping "control over operating systems, storage, and deployed applications; and possibly limited control of select networking components (e.g., host firewalls)". Platform as a service gives the capability to "deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider", with control over "the deployed applications and possibly configuration settings for the application-hosting environment" and nothing below. Software as a service gives the capability to "use the provider's applications running on a cloud infrastructure", where the consumer "does not manage or control the underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings". Read as a gradient, the sentences agree on the mechanism: the same stack, with the handover point climbing one band at a time.

NOTE: nist named three, the market rents four

800-145 lists exactly three service models and has no word for functions. The fourth rung postdates the definition: the Lambda page opens "AWS Lambda is a serverless compute service. With Lambda, you can run code without provisioning or managing servers", and Google's responsibility page treats faas as a category of its own, saying "FaaS has a similar shared responsibility list as SaaS". In NIST terms a function platform is a paas with the deployment unit shrunk to one handler, the capability sentence unchanged, while the provider also takes "capacity provisioning, scaling, and patching" per the Lambda page. The gradient gains a rung without gaining an architecture.

Where the book's services sit follows from the same sentences, applied when chapters 25 and 26 write their terraform. A vpc plus an ec2 instance or a compute engine machine hands you a network and a box whose guest operating system you patch, which is the iaas sentence exactly, and the AWS responsibility page names ec2 as its iaas example with "the guest operating system (including updates and security patches)" on the customer's side. Lambda and cloud run are the faas rung: code in, endpoint out, no server visible. S3 and cloud storage are managed storage, off the compute gradient entirely, no operating system and no runtime, just buckets and objects. Iam and the project model sit under every rung, because identity and billing are not on the stack at all.

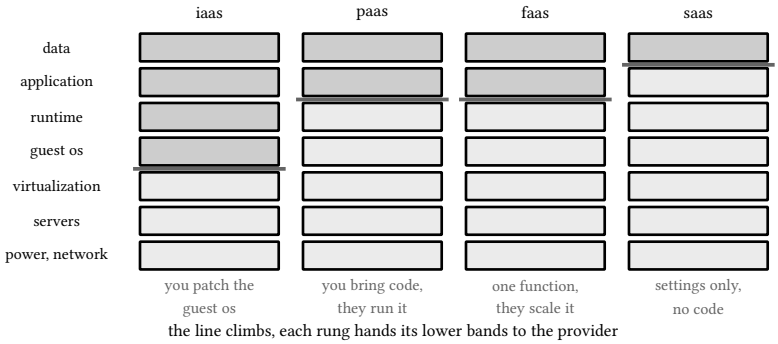


figure 96: the responsibility gradient, one stack per rung, the customer bands shrink as the rung climbs

22.2. regions, zones, failure domains

AWS regions are "separate geographic areas", each "designed to be isolated from the other Regions", which "achieves the greatest possible fault tolerance and stability". The isolation is literal: "You can replicate some types of resources across Regions, but we don't automatically replicate them for you." Each region "has multiple, independent locations known as Availability Zones", and "Each Availability Zone consists of one or more discrete data centers, each with redundant power, networking, and connectivity, and housed in separate facilities. Because they are physically separate, only a single Availability Zone would be affected in the unlikely event of a fire, tornado, or flooding." The guide's practice line is plain: "It is a best practice to deploy your application in multiple Availability Zones, so that your application remains available even if one Availability Zone fails."

The fault isolation whitepaper pins the numbers the guide leaves out. Zones in a region are "meaningfully distant from each other, up to 60 miles" (about 100 km) "to prevent correlated failures, but close enough to use synchronous replication with single-digit millisecond latency". They are "designed not to be simultaneously impacted by a shared fate scenario like utility power, water disruption, fiber isolation, earthquakes, fires, tornadoes, or floods"; generators and cooling "are not shared across Availability Zones and are designed to be supplied by different power substations"; and even AWS's own service deployments "to Availability Zones in the same Region are separated in time to prevent correlated failure". The whitepaper names the property AZI, Availability Zone Independence, and describes the interconnect as "high-bandwidth, low-latency networking, over fully redundant, dedicated metro fiber". Counts: "Each Region has at least three Availability Zones", the regions table shows most at three with us-east-1 at six and us-west-2, Tokyo, Seoul, and London at four, and the global infrastructure page counted "124 Availability Zones within 39 Geographic Regions" on access day.

Google's page is worded more cautiously, and the caution is content. "Regions are independent geographic areas that consist of zones", "A zone is a deployment area for Google Cloud resources within a region", and the operating rule is stated flatly: "Zones should be considered a single failure domain within a region." The physical claim is softer than AWS's: "A region consists of three or more zones housed in three or more physical data centers", with named exceptions, Stockholm, Mexico, Osaka, and Montreal, whose three zones are housed in one or two physical data centers, and the page says outright that "Zones and regions are logical abstractions of underlying physical resources". No distance and no inter-zone latency figure appears anywhere on it. Both providers sell the same operating discipline, spread across zones for availability, across regions for geography, but they document it differently: AWS publishes the mileage, Google publishes the abstraction.

That difference is the latency budget. Zone redundancy costs single-digit milliseconds per replication hop inside a region, the AWS whitepaper number, cheap enough to spend on a request path. Region redundancy has no published number on either page; what Google's page does say is that multi-region services, "managed by Google to be redundant and distributed within and across regions", "require a trade-off between either latency or the consistency model", and AWS's page says regions never replicate for you. The budget rule the terraform chapters follow: cross-zone calls are a per-request cost, cross-region calls are a data policy.

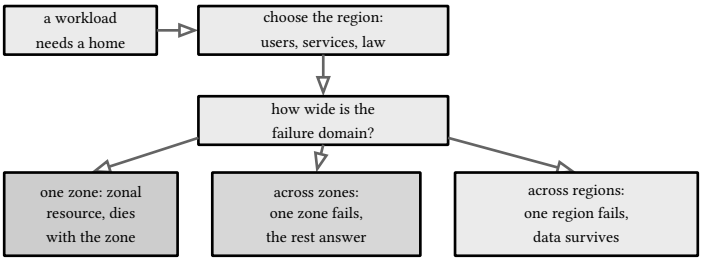


figure 97: placing a workload, one region choice, three failure-domain budgets

PITFALL: us-east-1a is not a place

"For accounts created before November 2025, in our oldest Regions, we independently map Availability Zones to codes for each AWS account", so the same code can denote a different physical zone in another account. The stable identity is the AZ ID, "the same physical location in every AWS account", shaped like use2-az1. The page also admits constrained zones exist, so "your account might have a different number of available Availability Zones in a Region than another account does". Terraform that reasons about physical separation from zone codes alone is guessing across account boundaries; AZ IDs are the coordinate system.

22.3. shared responsibility

AWS states the split in one sentence: "Security and Compliance is a shared responsibility between AWS and the customer", a division the page calls Security "of" the Cloud versus Security "in" the Cloud. Of the cloud is AWS's side, "protecting the infrastructure that runs all of the services offered in the AWS Cloud", infrastructure "composed of the hardware, software, networking, and facilities that run AWS Cloud services". In the cloud is yours, and its size is not fixed: "Customer responsibility will be determined by the AWS Cloud services that a customer selects." The page's two examples are the gradient in miniature. For iaas like ec2 the customer manages "the guest operating system (including updates and security patches)"; for abstracted services like s3 and dynamodb, "Customers are responsible for managing their data (including encryption options), classifying their assets", and their iam permissions. The line moves with every managed upgrade: adopting a managed service hands the patching to the provider and keeps the configuration mistakes for you.

Google's page starts from the same table and then renames the relationship. In iaas "the bulk of the security responsibilities are yours", in saas "we own the bulk of the security responsibilities", and two rows never move: "the cloud provider always remains responsible for the underlying network and infrastructure", and "customers always remain responsible for their access policies and data". Then the turn: "Instead of shared responsibility, we believe in shared fate", which "builds on the shared responsibility model" and "focuses on how all parties can better interact to continuously improve security". The stated motive is blunt: "Most cloud security breaches are the direct result of misconfiguration", so Google commits to secure defaults and shared tooling rather than a clean liability boundary. Marketing or engineering, the two fixed rows are the operational truth for the terraform chapters: the provider never takes your access policy and never takes your data.

That is why the cloud chapters spend more ink on identity than on compute. Chapter 25's iam role and chapter 26's bindings sit on the customer side of every row, and so do the bucket policy and the firewall rules. Identity is the part of the stack no rung ever absorbs, the one band that survives every managed upgrade on the customer side of the line.

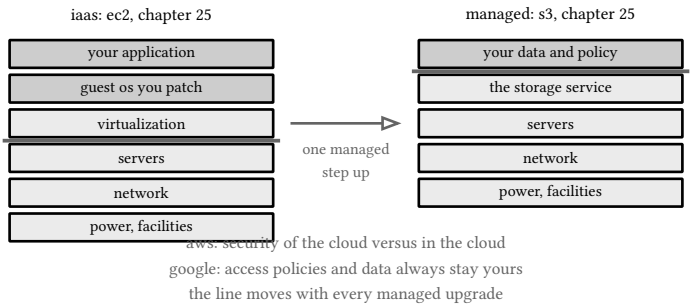


figure 98: the same stack one managed step apart, and where the responsibility line sits in each

22.4. well-architected pillars

AWS frames its framework as a decision aid, not a scorecard: it "helps you understand the pros and cons of decisions you make while building systems on AWS", and "The process for reviewing an architecture is a constructive conversation about architectural decisions, and is not an audit mechanism." The framework document is dated November 6, 2024, and the pillars page says it is "Built around six pillars": operational excellence, "running and monitoring systems, and continually improving processes and procedures"; security, "protecting information and systems"; reliability, workloads "performing their intended functions and how to recover quickly from failure to meet demands"; performance efficiency, "structured and streamlined allocation of IT and computing resources"; cost optimization, "avoiding unnecessary costs"; and sustainability, "minimizing the environmental impacts of running cloud workloads".

Google's framework lives at the architecture framework URL, but the page as fetched is titled Google Cloud Well-Architected Framework, and the rebrand is itself a fact: it recommends how to "design and operate a cloud topology that's secure, efficient, resilient, high-performing, cost-effective, and sustainable". It also counts six pillars: operational excellence, to "efficiently deploy, operate, monitor, and manage your cloud workloads"; security, privacy, and compliance, to "maximize the security of your data and workloads in the cloud, design for privacy, and align with regulatory requirements"; reliability, to "design and operate resilient and highly available workloads in the cloud"; cost optimization, to "maximize the business value of your investment in Google Cloud"; performance optimization, to "design and tune your cloud resources for optimal performance"; and sustainability, to "build and manage cloud workloads that are environmentally sustainable". Two cross-pillar perspectives sit beside them, ai and ml plus financial services, and the page adds five core principles: design for change, document your architecture, simplify by preferring managed services, decouple the architecture, and keep it stateless.

Side by side the lists rhyme: same first pillar, same last pillar, the same six topics wearing two names. The differences are naming and order. Google's security pillar is "security, privacy, and compliance" where AWS stops at "security"; Google says performance "optimization" where AWS says performance "efficiency"; and the middle two swap, AWS runs performance before cost, Google runs cost before performance. For a book that provisions both clouds from one terraform tree the rhyming is the useful part: the checklist is one checklist, and chapter 27 grades both providers against it.

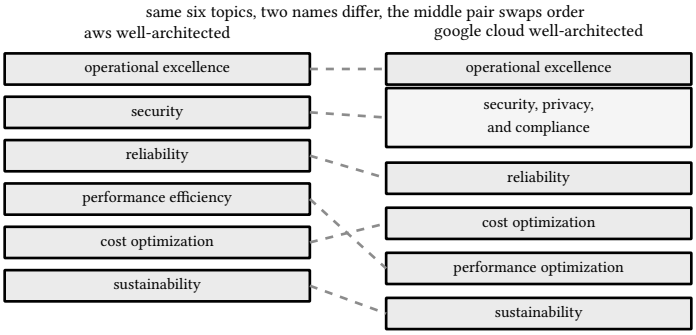


figure 99: the two six-pillar frameworks side by side, connectors pairing shared topics, the crossing pair marking the order swap

sources: NIST SP 800-145 (September 2011), full text from the nvlpubs.nist.gov PDF; the AWS Well-Architected Framework welcome page (framework document dated 2024-11-06) and the pillars page at aws.amazon.com/architecture/well-architected; the AWS Regions and Availability Zones guide (aws-regions-availability-zones, aws-regions, aws-availability-zones under docs.aws.amazon.com global-infrastructure), the AWS Fault Isolation Boundaries whitepaper, the AWS Global Infrastructure page (the 124 zones, 39 regions count), the AWS shared responsibility model page, and "What is AWS Lambda"; Google Cloud "Geography and regions", the Google Cloud Well-Architected Framework overview at cloud.google.com/architecture/framework, its security pillar page, and "Shared responsibilities and shared fate on Google Cloud". All accessed 2026-09-12. Documentation-verified only: this chapter ships no samples, no terraform binary exists in this corpus, and nothing was initialized, planned, or applied.

23. terraform: language and core workflow

Chapter 22 drew the responsibility line between provider and tenant. Terraform is how this book works the tenant side of that line, with code instead of a console. Its deal is declarative: the author writes files that describe desired infrastructure, and the tool reconciles real objects toward the description. This chapter reads the language those files are written in, the resource graph they imply, the state file that binds configuration to real objects, the plan and apply loop that does the reconciling, and the version pins that keep every one of those claims stable.

The verification mode changes here and stays changed through chapter 27. The four files under `Ch23/` are real, pinned terraform configuration and nothing ever runs them: no terraform binary exists in this corpus, nothing is initialized, planned, or applied. Every behavioral claim below is a sentence quoted from a canonical page fetched 2026-09-12 or a line of those files. The c chapters proved their claims by executing them. The cloud chapters prove theirs by quoting the documentation that defines the behavior and pinning the versions the quotes were read against.

23.1. hcl blocks, arguments, and expressions

The grammar under every `.tf` file is HCL's native syntax, and the configuration syntax page fixes its anatomy in a few sentences: "A block is a container for other content." A block has a type, each block type defines how many labels must follow the type keyword, and "After the block type keyword and any labels, the block body is delimited by the { and } characters." Inside that body "further arguments and blocks may be nested, creating a hierarchy of blocks and their associated arguments." A `variable` block carries one label, a `resource` block carries two, and a `validation` block nests inside a `variable` block with no labels of its own:

```
variable "environment" {
  type        = string
  default     = "dev"
  description = "label folded into names and tags, validated below"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

listing 246: the variable block: one label, four arguments, one nested validation block (source: c-os-cloud/samples/src/Ch23/variables.tf, lines 13-22)

The `environment` block and its nested `validation` block are exactly that hierarchy. The page also fixes this book's vocabulary for `=` lines. "An argument assigns a value to a particular name", and terraform's docs say argument, not attribute, on purpose: HCL's own documentation "usually uses the word 'attribute' instead of 'argument'", but terraform resources have attributes like `id` "that can be referenced from expressions but can't be assigned values in configuration." Arguments are the inputs the author writes, attributes are the values the provider reports back, and the boundary between them is the boundary between configuration and state.

Names follow the identifier rules: "Identifiers can contain letters, digits, underscores (_), and hyphens (-)", and "The first character of an identifier must not be a digit, to avoid ambiguity with literal numbers." Comments have three spellings. `#` is "the default comment style" and "should be used in most cases", `//` is the same single-line comment in a less idiomatic dress, and `/*` with `*/` "are start and end delimiters for a comment that might span over multiple lines."

The right side of `=` is an expression, and the expressions page states the charter in one sentence: "Expressions refer to or compute values within a configuration." The simplest expressions are just literal values, like `"hello"` or `5`. References name values the configuration already knows, `var.environment`, `local.common_tags`, `aws_instance.web.id`.

The rest of the expression grammar builds new values from old ones, and two forms carry most of that weight in real configurations, the for expression and the splat:

```
locals {
  name_prefix = "${var.environment}-web"

  common_tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
  }

  web_names = [for i in range(var.instance_count) : "web-${i}"]
}
```

listing 247: locals: string interpolation, an object literal, and a for expression that builds a tuple (source: c-os-cloud/samples/src/Ch23/main.tf, lines 10-19)

`web_names` is the form the docs open with: `[for s in var.list : upper(s)]`. The page pins the result rule: "The type of brackets around the `for` expression decide what type of result it produces." Square brackets produce a tuple, and `web_names` is a tuple of strings, one per instance. Curly braces produce an object and require "two result expressions that are separated by the ⇒ symbol." An optional `if` clause filters, the page calls it "an optional `if` clause to filter elements from the source collection", and grouping mode is a symbol: "To activate grouping mode, add the symbol `...` after the value expression." One limit the page states plainly: "You can't dynamically generate nested blocks using `for` expressions", dynamic blocks are the tool for that instead, which chapter 24 uses.

The splat is the shortcut over the same ground. The page defines it as an expression that "provides a more concise way to express a common operation that could otherwise be performed with a `for` expression": "The special `[*]` symbol iterates over all of the elements of the list given to its left" and "accesses from each one the attribute name given on its right." So `var.list[*].id` is exactly `[for o in var.list : o.id]`. The legacy `.*` spelling is "less useful than the modern form" and survives only "for backward compatibility", and the docs "recommend always using the new-style splat expressions, with `[*]`, to get the more consistent behavior." The difference is real, not cosmetic:

```
var.list.*.interfaces[0].name
  equals [for o in var.list : o.interfaces][0].name

var.list[*].interfaces[0].name
  equals [for o in var.list : o.interfaces[0].name]
```

fragment, illustrative, not executed

The legacy form applies `[0]` once, after the iteration. The full splat applies it per element. Two more edges round out the form: applied to a value that is not a list, "the splat expression will transform it into a single-element list, or more accurately a single-element tuple value", and "If the value is null then the splat expression will return an empty tuple", which is why `var.website_setting[*]` feeds `for_each` so neatly, one instance when set, zero when null.

This is the declarative contract in miniature. `[for i in range(var.instance_count) : "web-${i}"]` has no loop counter, no iteration variable to misuse, no order of side effects: it is a value computed from other values. The whole file is the same contract at larger scale, a description of an end state, and the question of when anything actually runs belongs to the next section's graph.

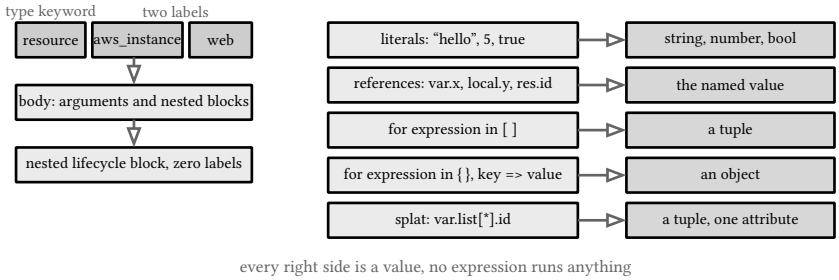


figure 100: a block taken apart on the left, the expression forms and what each produces on the right

23.2. resources, providers, and the graph

The resource page states what the block is for: “The `resource` block defines a piece of infrastructure and specifies the settings for Terraform to create it with.” Two labels follow the keyword, the resource type and a name the author picks. The name matters to terraform only: “Terraform uses this label to track the resource in your state file” and “it does not affect settings on the actual infrastructure resource.” The pair forms the resource address, and the page fixes its shape: “To reference the resource in your configuration, you must refer to it using `<TYPE>.<LABEL>` syntax.” Five meta-arguments are “built into the Terraform language” and modify the block from inside. The two repetition modes are `count`, which “instructs Terraform to provision multiple instances of the same resource with identical or similar configuration”, and `for_each`, which “instructs Terraform to provision similar resources without requiring separate configuration blocks.” The page bars mixing them: “You cannot use both a `count` and `for_each` argument in the same block.” The sample uses one of each, on two instances of the same resource type:

```
resource "aws_instance" "web" {
  count = var.instance_count

  ami           = var.ami
  instance_type = var.instance_type

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-${count.index}"
  })

  lifecycle {
    ignore_changes = [tags["Owner"]]
  }
}
```

listing 248: the count resource: instances indexed by number, references only, one lifecycle rule (source: c-os-cloud/samples/src/Ch23/main.tf, lines 21-34)

```
resource "aws_instance" "admin" {
  for_each = var.admins

  ami           = each.value
  instance_type = "t3.micro"
  provider      = aws.west

  tags = merge(local.common_tags, {
    Name = "admin-${each.key}"
  })

  depends_on = [aws_instance.web]
}
```

listing 249: the `for_each` resource: keyed instances, the aliased provider, one explicit dependency (source: c-os-cloud/samples/src/Ch23/main.tf, lines 36-48)

The other meta-arguments are visible in the same two listings. The `lifecycle` block in the first resource carries the sample's one rule, `ignore_changes`, which "Specifies a list of resource attributes that Terraform ignores changes to", the drift valve that keeps terraform from fighting a tag an operator sets by hand. `depends_on` closes the second listing and "specifies an upstream resource that the resource depends on", the explicit edge the graph picks up below.

The mode decides the shape of every later reference. The references page states both halves: "If the resource has the `count` argument set, the reference's value is a list of objects representing its instances", so `aws_instance.web` is a list and `aws_instance.web[0].id` "returns just the id of the first instance." Under `for_each` "the reference's value is a map of objects representing its instances", and `aws_instance.admin["ops"].id` returns the one keyed instance. One asymmetry the page calls out: "splat expressions are not directly applicable to resources managed with `for_each`", the page's own workaround is `values(aws_instance.admin)[*].id`, values first, splat second. Both shapes leave their trace in the outputs:

```
output "web_ids" {
  description = "count instances form a list, the splat collects every id"
  value       = aws_instance.web[*].id
}

output "admin_ids" {
  description = "for_each instances form a map, values first, then the splat"
  value       = values(aws_instance.admin)[*].id
}

output "admin_id_by_key" {
  description = "one for_each instance addressed by its key"
  value       = aws_instance.admin["ops"].id
}
```

listing 250: the two address shapes: a list splats directly, a map goes through values first (source: c-os-cloud/samples/src/Ch23/outputs.tf, lines 1-14)

Providers are the plugins that know a specific cloud's api, and the provider page splits their declaration from their configuration: "Use the `provider` block to declare and configure Terraform plugins, called providers", with version requirements living in `required_providers` instead, section 5′s subject. "Define provider configurations in the root module of your Terraform configuration." One provider can have several configurations: "Optionally use the `alias` argument to define multiple configurations for the same provider", the block without an alias is the default, and a resource opts into an alternate with the `provider` meta-argument, which "instructs Terraform to use an alternate provider configuration to provision the resource." The sample carries both shapes:

```
provider "aws" {
  region = var.region
}

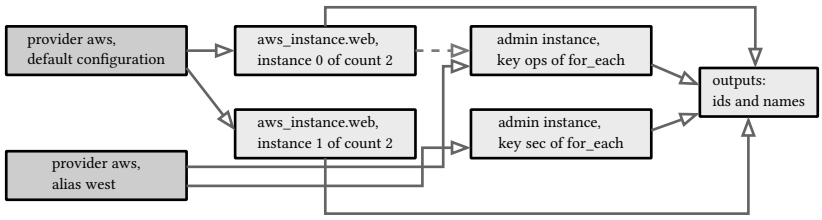
provider "aws" {
  alias  = "west"
  region = var.west_region
}
```

listing 251: two provider configurations: the default and the aliased aws.west (source: c-os-cloud/samples/src/Ch23/main.tf, lines 1-8)

Nothing in these files says when anything runs. That is the graph's job, and the internals page says so: "Terraform builds a dependency graph and uses it to perform operations, such as generate plans and refresh state." The edges come from two places: "Explicit dependencies from the `depends_on` meta-parameter are used to create edges between resources", and "Interpolations are parsed in resource and provider configurations to determine dependencies", every `aws_instance.web[*].id` inside the outputs is an implicit edge. The walk itself is ordered and parallel at once: "To walk the graph, a standard depth-first traversal is done", but "Graph walking is done in parallel: a node is walked as soon as all of its dependencies are walked", and "By default, up to 10 nodes in the graph will be processed concurrently." That number meets the apply page from the other side: `-parallelism` exists to "Limit the number of concurrent operations as Terraform walks the graph" and "Defaults to 10."

PITFALL: the repetition mode is visible in every reference

Choosing `count` or `for_each` is not a style call. It fixes whether `aws_instance.web` is a list or a map, and every expression that touches the resource afterwards inherits that shape: an index for `count`, a key for `for_each`, a plain splat for the first, `values()` before the splat for the second. Renumbering a `count` fleet or renaming a `for_each` key reads as destroy and create to the graph, because the address, not the contents, is the identity.



solid edges are parsed from interpolations, the dashed edge is depends_on, drawn once for all four pairs

a node walks the moment its dependencies finish, 10 nodes at once by default

figure 101: the graph implied by the sample: two provider configurations feed four resource instances, outputs read them all

23.3. state anatomy

The state page opens with the obligation: "Terraform must store state about your workspace's managed infrastructure and configuration", and the purpose page grades it "State is a necessary requirement for Terraform to function." State does three jobs. Mapping first: "Terraform requires some sort of database to map Terraform config to the real world", and the example is concrete, a resource in configuration "represents a real world object with the instance ID `i-abcd1234` on a remote system." The binding is exclusive: "Terraform expects that each remote object is bound to only one resource instance in the configuration", and "Terraform can guarantee a one-to-one mapping when it creates objects and records their identities in the state." Metadata second: "Terraform must also track metadata such as resource dependencies." Performance third: "Terraform stores a cache of the attribute values for all resources in the state", and the page is candid about its rank, "This is the most optional feature of Terraform state and is done only as a performance improvement", with "the cached state is treated as the record of truth" on large infrastructures where querying every resource would be too slow.

By default the record is local: state lives "in a local file named `terraform.tfstate`" with "a backup of the previous state in `terraform.tfstate.backup`", and "State snapshots are stored in JSON format." The page recommends remote

backends instead, and the sensitive data page supplies the reason in the bluntest sentence in this chapter: “Terraform stores your state in a plaintext file, which includes any secret values you defined in your configuration.” The `sensitive` argument changes the display, never the storage: “Terraform stores values with the `sensitive` argument in both state and plan files, and anyone who can access those files can access your sensitive values.” The sample carries one of each, the input variable and the output, so the pair sits side by side in one directory:

```
variable "db_password" {
  type        = string
  sensitive   = true
  description = "redacted in cli output, still written to state in cleartext"
}
```

listing 252: a sensitive input: redacted in cli output, stored in state regardless (source: c-os-cloud/samples/src/Ch23/variables.tf, lines 44-48)

```
output "db_password" {
  description = "sensitive redacts cli display only, the value still lands in state"
  value       = var.db_password
  sensitive   = true
}
```

listing 253: a sensitive output: same redaction, same storage, both doc-stated (source: c-os-cloud/samples/src/Ch23/outputs.tf, lines 21-25)

Neither line protects anything. The page’s practices do: “Treat your state file as sensitive data by excluding it from Git workflows”, store state remotely, encrypt it at rest, control who reads it, audit the reads. Newer than the redaction argument is the real omission mechanism, and the variables page states it: “You can add the `ephemeral` argument to your `variable` configuration to omit the variable from state and plan files”, the only tool on these pages that keeps a value out of the file rather than out of the display.

Locking is the concurrency half of the same anatomy. The locking page states the rule: “If supported by your backend, Terraform will lock your state for all operations that could write state”, which “prevents others from acquiring the lock and potentially corrupting your state.” The qualification matters, “Not all backends support locking.” Locking is quiet, “State locking happens automatically on all operations that could write state”, and strict: “If state locking fails, Terraform does not continue.” The escape hatch is `force-unlock`, and the page’s own warning is the whole guidance: “Be very careful with this command”, because “If you unlock the state when someone else is holding the lock it could cause multiple writers”, and “the `force-unlock` command requires a unique lock ID” as a nonce so the unlock targets the right lock. `-lock=false` exists for most commands “but we do not recommend it.”

Losing the file is the other disaster the anatomy explains. Local state “risks losing workspace state if the local state file is lost”, and what is lost is the mapping: with the one-to-one record gone, no later plan can find the remote object any resource refers to, so the plan reads as creation while the real objects, and their bills, keep running under addresses terraform no longer knows. The fix named on both pages is the same, remote state with locking.

WARNING: sensitive is display hygiene, not secrecy

Marking a variable or output `sensitive` changes what the cli prints and nothing else. The value rides into `terraform.tfstate` and any saved plan file in cleartext, and `terraform output -json` prints it back out. Secrets belong in ephemeral values, which never reach the files, or in a provider-side secret manager whose state entry is a reference, not the secret.

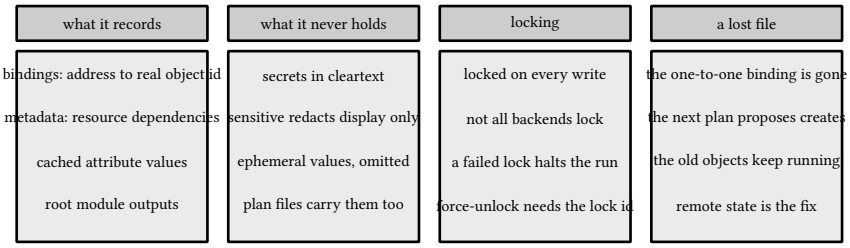


figure 102: state anatomy in four columns: the record, the prohibition, the lock, the loss

23.4. plan and apply

The loop that reconciles the three-way match between configuration, state, and the real world runs on two commands. Plan's page opens: it "creates an execution plan, which lets you preview the changes that Terraform plans to make to your infrastructure", and "The `plan` command alone does not actually carry out the proposed changes." Every plan begins with a refresh, plan "Reads the current state of any already-existing remote objects to make sure that the Terraform state is up-to-date." Skipping it has a named cost: `-refresh=false` "causes Terraform to ignore external changes, which could result in an incomplete or incorrect plan." `-refresh-only` inverts the goal and "creates a plan whose goal is only to update the Terraform state and any root module output values", the drift-direction run.

A plan can be saved: "You can use the optional `-out=FILE` option to save the generated plan to a file on disk" for apply to consume. A saved plan is a promise with a timestamp, not a lock on the world: "other changes made to the target system in the meantime might cause the final effect of a configuration change" to differ from the preview, so "you should always re-check the final non-speculative plan before applying." Plan files also inherit state's transparency problem, values are "saved in cleartext in the plan file", so the page says to "treat any saved plan files as potentially-sensitive artifacts."

Apply's page returns the favor in one line: "The `terraform apply` command executes the operations proposed in a Terraform plan." Given a saved plan file, "Terraform performs the operations in the saved plan without prompting you for confirmation." Given none, "Terraform automatically creates a new execution plan as if you had run `terraform plan`", then "prompts you to approve that plan, and performs the indicated operations." `-auto-approve` "Skips interactive approval of the plan before applying", is ignored with a saved plan because "Terraform interprets the act of passing the plan file as the approval", and the page warns what skipping covers, "this includes destructive operations such as deleting resources." What apply actually does to each resource is a five-item list on the create and manage page. It "Creates resources in the configuration that do not yet exist as real infrastructure objects", "Updates any resources in place if their arguments have changed", "Destroys and re-creates resources whose arguments have changed but cannot be updated in-place due to remote API limitations", "Destroys resources that exist in the state but no longer exist in the configuration", and "Updates the state file so that the configuration, real infrastructure, and state match." The operations run as the graph walk from the previous section, in parallel up to the same limit of 10. For automation, `-detailed-exitcode` gives plan three answers: 0 "Succeeded with empty diff (no changes)", 1 "Error", 2 "Succeeded with non-empty diff (changes present)."

```
terraform plan -refresh-only
terraform plan -out=tfplan
terraform apply tfplan
```

fragment, illustrative, not executed

Those three commands are the loop in its automation dress, reconcile state, freeze a decision, execute it, and this book never runs any of them. The .tf files are the inputs the loop would consume, and the documentation is the record of what it would do.

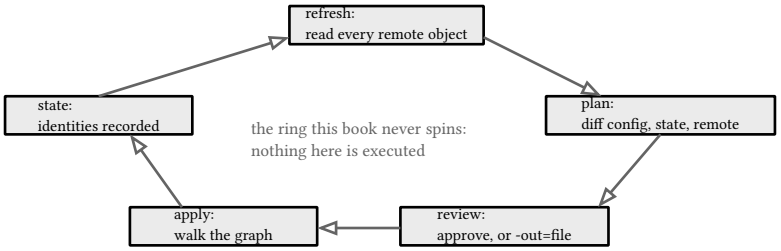


figure 103: the core loop as a ring: refresh reads, plan diffs, review gates, apply walks, state records

23.5. version pinning

Every page quoted above carried a version selector reading “v1.16.x (latest)”, which is the document’s own warning that its sentences move with the tool. The dated source of truth is the release record. The install page names the current release “1.16.2 (latest)” and lists every release back to 1.0.11. The GitHub releases page gives the dates: 1.16.0 on August 26, 1.16.1 on September 2, “1.16.2 (September 9, 2026)” marked Latest, and v1.17.0-beta1 above it marked Pre-release, which no constraint here will pick up: “Terraform does not match pre-release versions on >, ≥, <, ≤, or ⇝ operators”, and an exact pin never names it at all.

The configuration enforces its own reader. `required_version` “Specifies which version of the Terraform CLI is allowed to run the configuration”, and a mismatch stops the run before it starts, “Terraform prints an error and exits without taking actions.” The whole `terraform` block is constant-only, “You can only use constant values in the `terraform` block”, no variables, no references, because these values gate everything else. Providers pin through `required_providers`, which “must be nested inside the top-level `terraform` block”: `source` is “the global source address for the provider you intend to use, such as `hashicorp/aws`”, and `version` is “a version constraint specifying which subset of available provider versions the module is compatible with.” The constraint grammar is seven operators. A bare version with `=` “Allows only one exact version number.” The ⇝ shorthand “Allows only the right-most version component to increment”, with the page’s own pair of examples, “⇝ `1.0.4`: Allows Terraform to install `1.0.5` and `1.0.10` but not `1.1.0`” and “⇝ `1.1`: Allows Terraform to install `1.2` and `1.10` but not `2.0`.”

```
# book 1 chapter 20: version pins frozen at the documentation access date
# terraform 1.16.2 and hashicorp/aws 6.64.0, both published 2026-09-09,
# both the pages marked Latest when fetched 2026-09-12. this file is real
# but never executed: no terraform binary exists in this corpus.

terraform {
  required_version = "1.16.2"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.64.0"
    }
  }
}
```

listing 254: the freeze: exact cli and provider pins, dated in the comment, nothing executed against them (source: c-os-cloud/samples/src/Ch23/versions.tf, lines 1-15)

Both pins are exact, both carry their release date in the comment, and the provider pin is the releases page’s own current record: the hashicorp/terraform-provider-aws releases list reads “6.64.0 (September 9, 2026)” marked Latest, with 6.63.0 on September 3 and 6.62.0 on August 26 behind it. The last piece of the discipline is the one file terraform writes back: “The lock file is always named `.terraform.lock.hcl`” and terraform “automatically creates or updates the dependency lock file each time you run the `terraform init` command.” It records “the exact version that Terraform selected based on the version constraints in the configuration”, plus checksums in two schemes, `zh:` is “a mnemonic for ‘zip hash’” and `h1:` is “a mnemonic for ‘hash scheme 1’”. Once recorded, a selection sticks,

“Terraform will always re-select that version for installation, even if a newer version has become available”, until `-upgrade` says otherwise. The page’s advice closes the loop: “you should include this file in your version control repository.” Global machine state sits outside the repository by design, the cli configuration overview notes the cli config file “configures provider installation and security features” for the whole user, while the version discipline that travels with the code lives in `versions.tf`, the lock file beside it, and the access dates in the sources line below.

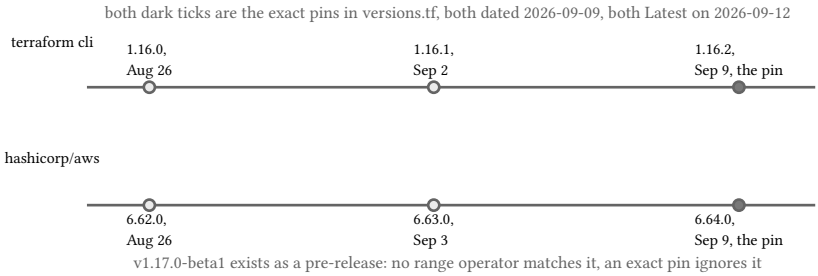


figure 104: two release tracks, one date axis, the pair this book pins

sources: developer.hashicorp.com terraform pages, configuration syntax, expressions, for expressions, splat expressions, references to values, resource block reference, provider block reference, terraform block reference, provider requirements, version constraints, dependency lock file, state, purpose of state, state locking, manage sensitive data, create and manage resources overview, dependency graph, plan and apply command references, cli configuration overview, and the install page, all accessed 2026-09-12, github.com hashicorp/terraform and hashicorp/terraform-provider-aws releases, accessed 2026-09-12. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied. The version pins are 1.16.2 for the cli and 6.64.0 for hashicorp/aws, the Latest marks on both release pages at access time.

24. terraform modules and workflows

Chapter 23 read the terraform language one block at a time and never assembled the parts. This chapter assembles them. A module is the unit of assembly, a directory of `.tf` files with a declared interface, and around that unit sit the workflows that make it operable: version pins that keep its sources stable, a backend that decides where its state lives, workspaces that give one configuration several state instances, and the three commands that check a module without touching a cloud. The sample is one root module composing one local network module, plus the backend, test, and version files around them.

The verification mode is chapter 23's, unchanged. The nine files under `Ch24/` are real, pinned terraform configuration and nothing ever runs them: no terraform binary exists in this corpus, nothing is initialized, planned, or applied. Every behavioral claim below is a sentence quoted from a canonical page fetched 2026-09-12 or a line of those files. The version freeze is chapter 23's, terraform 1.16.2 and hashicorp/aws 6.64.0, both published 2026-09-09 and both marked Latest at access time.

24.1. variables, outputs, locals

The variables page states the contract in its first sentence: "You can add `variable` blocks to your configuration to define input interface for your module." The rest of the sentence gives the direction: "This lets users pass custom values to your module at runtime." Inputs flow in, and the block reference fixes the vocabulary of what may flow: a variable block supports `type`, `default`, `description`, `validation`, `sensitive`, `nullable`, `ephemeral`, `const`, and `deprecated`, every one of them optional, and its name label "must be unique among all variables in the same module." A variable without a default is a required input, and the page says what that costs the caller: "Terraform prompts the user to supply a value for that variable before it generates a plan." The network module's interface opens with one of each kind:

```
variable "vpc_cidr" {
  type        = string
  description = "cidr block for the vpc, subnet cidrs derive from it with cidrsubnet"
}

variable "availability_zones" {
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b"]
  description = "one subnet per zone, the count comes from this list"
}
```

listing 255: the interface so far: a required string and a defaulted list, two type constraints (source: c-os-cloud/samples/src/Ch24/modules/network/variables.tf, lines 5-14)

The `type` argument is where the contract becomes checkable. The block reference: "The `type` argument in a `variable` block constrains the type of value that you can assign to that variable", and without one "the variable accepts a value of any type." Anything the language can express is allowed: "You can use any valid primitive, complex, or structural type as a constraint in a variable block", and the reference's own example of a structural constraint is `list(object({ internal = number, external = number, protocol = string }))`. The sample keeps to three kinds, `string`, `list(string)`, and `map(string)`, enough to make every value the module accepts carry a type the caller can read off the file. Validation closes the contract. A `validation` block carries two required arguments, `condition`, "Expression that must evaluate to `true` for Terraform to proceed with an operation", and `error_message`, "Message to display if the condition evaluates to false." Its timing is plan time, not apply time: "Terraform evaluates variable validations while it creates a plan", and on failure Terraform "throws an error, displays the `error_message`, and stops the current operation." Two more arguments round out the reference: `nullable`, where "If `nullable` is `false` in a variable block, then that variable must have a value that is not null", and `ephemeral`, "available in Terraform v1.10 and later", which keeps a value out of state and plan files entirely:

```
variable "environment" {
  type        = string
  default     = "dev"
  description = "label merged into resource tags, validated at plan time"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}

variable "extra_tags" {
  type        = map(string)
  default     = {}
  description = "caller tags merged over the module's own on every resource"
}
```

listing 256: a validated environment and a map of caller tags, the plan time gate in the middle (source: c-os-cloud/samples/src/Ch24/modules/network/variables.tf, lines 16-31)

Outputs are the other side of the interface. The outputs page: “Add `output` blocks to your configuration to expose information about your infrastructure”, and for modules specifically, “Defining an `output` block in a child module exposes that value to the parent module.” Inside the block, `value` is the one obligation: “You must include a `value` argument in each `output` block”, and Terraform “evaluates the `value` argument’s expression and exposes the result as the return value of an output and stores that value in state.” Root module outputs get their own stage: “Terraform displays root module output values in the CLI after you apply your configuration.” Outputs also carry a precondition block, “a condition to validate before computing the output or storing it in state”, with the same two arguments as a variable validation and the same timing, “Terraform evaluates preconditions on outputs when creating or applying a plan”:

```
output "vpc_id" {
  description = "id of the module's vpc"
  value       = aws_vpc.main.id
}

output "subnet_ids" {
  description = "count instances form a list, the splat collects every id"
  value       = aws_subnet.main[*].id
}

output "subnet_cidrs" {
  description = "the cidrsubnet results, known from configuration alone"
  value       = aws_subnet.main[*].cidr_block
}
```

listing 257: three outputs: a plain attribute, a splat over the count list, the cidrs known from configuration alone (source: c-os-cloud/samples/src/Ch24/modules/network/outputs.tf, lines 4-17)

Locals are the module’s private scratch, and the locals page keeps them small on purpose: “Local values assign names to expressions” and “Local values are similar to function-scoped variables in other programming languages.” Their scope is the module and nothing wider: “You can access local values in the module where you define them, but not in other modules”, reached with `local.<NAME>`. The page’s warning is the design constraint this sample obeys, two locals, each used twice: “However, they can make configuration harder to read because they obscure where values originate.” The body reads them the way a function body reads its own locals:

```
locals {
  module_tags = {
    Module      = "network"
    Environment = var.environment
  }

  subnet_count = length(var.availability_zones)
}
```

listing 258: locals: the tag merge and the subnet count, named once, read twice (source: c-os-cloud/samples/src/Ch24/modules/network/main.tf, lines 4-11)

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = merge(local.module_tags, var.extra_tags)
}
```

listing 259: the vpc: one argument from the interface, one tag merge through the local (source: c-os-cloud/samples/src/Ch24/modules/network/main.tf, lines 13-17)

The subnet resource is the one place the module computes rather than passes through. `cidrsubnet` "calculates a subnet address within given IP network address prefix" with the signature `cidrsubnet(prefix, newbits, netnum)`, where "`newbits` is the number of additional bits with which to extend the prefix." The page's arithmetic example is `cidrsubnet("10.1.2.0/24", 4, 15)`, which returns `10.1.2.240/28`, and its rule covers the sample's case directly: "if given a prefix ending in `/16` and a `newbits` value of `4`, the resulting subnet address will have length `/20`." So `cidrsubnet(var.vpc_cidr, 4, count.index)` turns one `/16` into a `/20` per subnet, `10.0.0.0/20` then `10.0.16.0/20`, derived from the interface with no second input:

```
resource "aws_subnet" "main" {
  count = local.subnet_count

  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(var.vpc_cidr, 4, count.index)
  availability_zone = var.availability_zones[count.index]

  tags = merge(local.module_tags, var.extra_tags, {
    Name = "subnet-${count.index}"
  })
}
```

listing 260: count over the zone list, one computed cidr per index, tags merged last (source: c-os-cloud/samples/src/Ch24/modules/network/main.tf, lines 19-29)

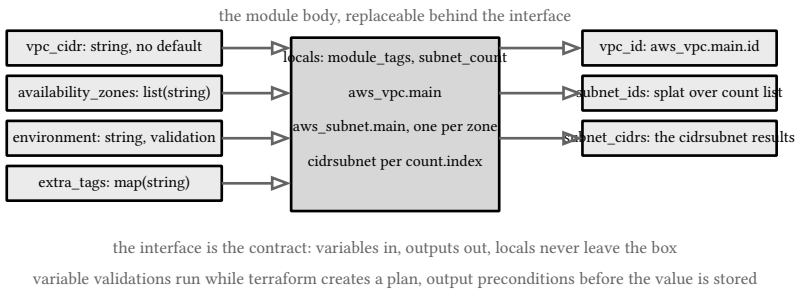


figure 105: the module as a data structure: variables enter left, outputs leave right, locals never leave

24.2. modules and versioning

The modules page fixes the vocabulary this chapter has been borrowing: “A module is a collection of resources that Terraform manages together.” Every configuration is one already, because “Every Terraform workspace includes configuration files in its root directory” and “Terraform refers to this configuration as the root module.” The blocks that call other directories are the children: “Modules you configure using `module` blocks are called child modules”, and at apply time “the root module calls the child module.” The calling side is one block: “The `module` block instructs Terraform to create resources defined in a local or remote module”, with “The `source` argument specifies where Terraform retrieves the module source code.” The sample’s root module is the whole of it:

```
module "network" {
  source = "./modules/network"

  vpc_cidr    = "10.0.0.0/16"
  environment = "dev"
}
```

listing 261: the root module’s entire body: one module block, two interface arguments (source: c-os-cloud/samples/src/Ch24/main.tf, lines 6-11)

Sources come in families. The modules page lists them, “including the local file system, a Terraform registry, and VCS repositories”, and the block reference fixes the local spelling: “Use the `./` or `../` prefix followed by the path to local module source code.” The registry spelling is an address, `source = "<NAMESPACE>/<NAME>/<PROVIDER>"`, and the sources page shows the form this book’s registry pin would take, the `terraform-aws-modules/vpc/aws` module at `version =
"6.0.1"`. The `source` value itself is rigid on purpose: “You must specify a literal string for the `source` value. This argument does not support template sequences or arbitrary expressions.”

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "6.0.1"
}
```

fragment, illustrative, not executed

The `version` argument is the axis the audit row names, and the block reference draws its boundary in two sentences: “The `version` argument specifies which version of the module to use. This argument only applies when installing modules from a registry”, and “Modules sourced from local file paths do not support `version`.” Selection is constraint-driven: “When an acceptable version isn’t installed, Terraform downloads the newest version that meets the constraint”, and the recommendation runs the same direction as chapter 23’s provider pins, “We recommend explicitly constraining the acceptable version numbers to avoid unexpected or unwanted changes.”

What crosses the boundary in the other direction is only outputs: “When the child module you are calling exposes output values, you can use the `module.<label>.<output>` syntax to reference them.” The root re-exposes two of the network module’s three, which is the whole difference between a root module and a reusable one:

```
output "network_vpc_id" {
  description = "the child module's vpc_id output, read at the root"
  value       = module.network.vpc_id
}

output "network_subnet_ids" {
  description = "the child module's subnet_ids list, read at the root"
  value       = module.network.subnet_ids
}
```

listing 262: child outputs re-exposed at the root, the only syntax that crosses the boundary (source: c-os-cloud/samples/src/Ch24/outputs.tf, lines 4-12)

The child module's shape is the structure page's recommendation verbatim: "`main.tf`, `variables.tf`, `outputs.tf`" are "the recommended filenames for a minimal module, even if they're empty", "`main.tf` should be the primary entry-point", "variables.tf and outputs.tf should contain the declarations for variables and outputs, respectively", and "All variables and outputs should have one or two sentence descriptions that explain their purpose." The develop pages grade the design choices around that shape. Depth: "we recommend keeping the module tree relatively flat and using module composition" rather than "a deeply-nested tree of modules", because flat trees reuse better. Size: "A good module should raise the level of abstraction by describing a new concept in your architecture", while the docs "do not recommend writing modules that are just thin wrappers around single other resource types", and moderation wins overall, since "over-using modules can make your overall Terraform configuration harder to understand and maintain." One requirement lives inside every module regardless of shape: "Each Terraform module must declare which providers it requires, so that Terraform can install and use them", which is why the child carries its own pin:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.64.0"
    }
  }
}
```

listing 263: the child's provider requirement, the same pin the root declares (source: c-os-cloud/samples/src/Ch24/modules/network/versions.tf, lines 4-11)

```
terraform {
  required_version = "1.16.2"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.64.0"
    }
  }
}
```

listing 264: the root's freeze, identical to chapter 23's: cli and provider, exact and dated (source: c-os-cloud/samples/src/Ch24/versions.tf, lines 7-16)

Installation is init's job, and the block reference ties it to the one argument that moves: "You must run `terraform init` after modifying the `source` argument so that Terraform can update the local code." The sources page says where the bytes land: "Terraform clones the module source configurations into a hidden subdirectory of the workspace's working directory", and `-upgrade` is the flag that lifts a module "to the latest version allowed by the version constraint."

PITFALL: a local module has no version to pin

The pin and the path are mutually exclusive. Registry sources take `version`, local sources cannot, so `./modules/network` is read as the bytes currently in the tree and nothing records which bytes an earlier run saw. The version discipline a local module gets is the repository around it: the module is pinned by commit, not by terraform. Registry composition trades that for the argument this sample cannot use, an explicit constraint like the snippet's `6.0.1`, selected once and recorded by init.

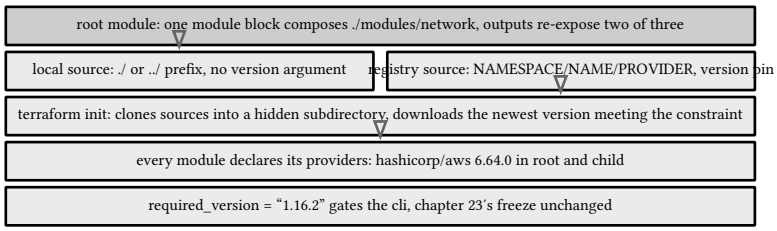


figure 106: the module stack: root on top, sources below it, init below them, pins at the bottom

24.3. backends and workspaces

Chapter 23 established state as terraform's binding record and left it in a local file. The backends page names the knob that moves it: "The `backend` defines where Terraform stores its state data files", and "Terraform uses a backend called `local` by default", where "The `local` backend type stores state as a local file on disk." The configuration is one nested block: "To configure a backend, add a nested `backend` block within the top-level `terraform` block", with the backend type as the block label and "A configuration can only provide one backend block." The block is also sealed: "A backend block cannot refer to named values (like input variables, locals, or data source attributes)", and nothing read back out either, "You cannot reference values declared within backend blocks elsewhere in the configuration." That seal is why the sample's bucket name is a literal and not `var.state_bucket`.

The s3 backend page opens with what it does: "Stores the state as a given key in a given bucket on Amazon S3." Three arguments are required, `bucket`, "Name of the S3 Bucket", `key`, "Path to the state file inside the S3 Bucket", and `region`, the "AWS Region of the S3 Bucket and DynamoDB Table (if used)". Locking is the reason to prefer it over a bare disk, and the page grades it honestly: "State locking is an opt-in feature of the S3 backend", though "Locking can be enabled via S3 or DynamoDB", and the old path is on its way out, "DynamoDB-based locking is deprecated and will be removed in a future minor version." The current spelling is `use_lockfile`, "Whether to use a lockfile for locking the state file. Defaults to `false`", which the sample opts into, alongside `encrypt`, "Enable server side encryption of the state and lock files":

```
terraform {
  backend "s3" {
    bucket       = "cos-resume-state"
    key          = "ch21/network.tfstate"
    region       = "us-east-1"
    use_lockfile = true
    encrypt      = true
  }
}
```

listing 265: the backend block: three required arguments, the lockfile opt in, encryption on (source: c-os-cloud/samples/src/Ch24/backend.tf, lines 7-15)

The deprecation carries detail worth keeping. The legacy route is the `dynamodb_table` argument, marked deprecated on the page, and its table has one hard requirement: "The table must have a partition key named `LockID` with a type of `String`." The bucket itself earns the page's one emphatic recommendation, "It is highly recommended that

you enable Bucket Versioning on the S3 bucket" for "state recovery in the case of accidental deletions and human error." Backend changes are an init event: after changing the block "you must run `terraform init` again to validate and configure the backend" before any "plans, applies, or state operations", and "When you change backends, Terraform gives you the option to migrate your state to the new backend." Secrets stay out of the file the usual way, partial configuration: omitted arguments are supplied "as part of the initialization process" through `-backend-config=PATH` or `-backend-config="KEY=VALUE"`, with "`*.backendname.tfbackend`" as "the recommended naming pattern."

Workspaces multiply the state without multiplying the configuration. The language page: "Some backends support multiple named workspaces, allowing multiple states to be associated with a single configuration", and the starting point is fixed, "Terraform starts with a single, default workspace named `default` that you cannot delete." The isolation is at the state layer and nowhere else: "When you run `terraform plan` in a new workspace, Terraform does not access existing resources in other workspaces", and "These resources still physically exist, but you must switch workspaces to manage them." The cli page grounds the same idea in the working directory: "Workspaces in the Terraform CLI refer to separate instances of state data inside the same Terraform working directory." For local state "Terraform stores the workspace states in a directory called `terraform.tfstate.d`", and the current selection is machine-local, "Terraform stores the current workspace name locally in the ignored `.terraform` directory."

The name a workspace carries is a real constraint with a quiet edge. The cli page's rule is short: "the name must be valid to use in a URL path segment without escaping." But the language page also documents the `terraform.workspace` interpolation, "the name of the current workspace", which "can be used anywhere interpolations are allowed", so the same string that names the state slot is available to every expression in the configuration, and a workspace name chosen casually becomes part of every name built from it. The naming questions the page answers outright are the structural ones: workspaces suit parallel copies of one configuration, and "Non-default workspaces are often related to feature branches in version control", but "Workspaces are not appropriate for system decomposition or deployments requiring separate credentials and access controls", those want separate root modules with separate backends. Switching is three commands, `workspace list` where "The current workspace is indicated using an asterisk marker", `workspace select`, whose `-or-create` flag will "If the workspace that is being selected does not exist, create it", and `workspace new`.

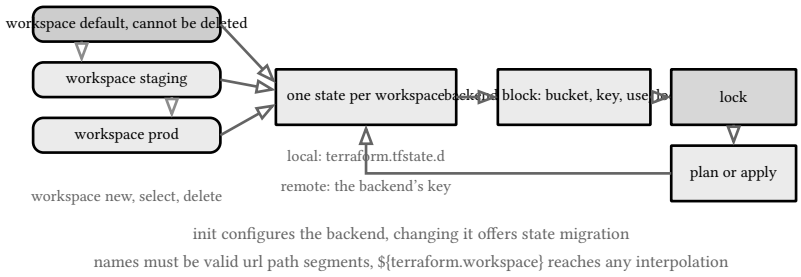


figure 107: workspace selection as a state machine: which state slot the next plan reads, and where the slot lives

24.4. fmt, validate, terraform test

Three commands check a module without provisioning anything, and they are the closest a cloud chapter gets to the c chapters' gate. The first is purely textual: "The `terraform fmt` command formats Terraform configuration file contents so that it matches the canonical format and style." It "applies a subset of the Terraform language style conventions" with "other minor adjustments for readability", and it is unalterable: "This command is intentionally opinionated and has no customization options." In a check rather than rewrite, `-check` "Checks if the input is formatted", and the exit contract is what a gate consumes: "The exit status is `0` if the command's input is properly formatted", and "Otherwise, the exit status is non-zero, and the command outputs a list of improperly formatted file names." `-diff` "Displays the diffs of formatting changes", and `-recursive` "Processes files in subdirectories in addition to the current directory", which matters here, the module lives in one.

The second command reads the configuration's shape. "The `terraform validate` command validates the configuration files in a directory", with checks that "verify whether a configuration is syntactically valid and internally consistent", including "correctness of attribute names and value types."

It has one precondition: “Validation requires an initialized working directory with any referenced plugins and modules installed”, and the page provides the bootstrapping pair for exactly this chapter’s situation, initialize “without accessing any configured backend” with `terraform init -backend=false`. The boundary is stated flatly: “It does not validate remote services, such as remote state or provider APIs”, which is why “It is safe to run this command automatically, for example as a post-save check in a text editor or as a test step.”

The third command executes the configuration’s own test files. “The `terraform test` command loads and executes Terraform testing files”, discovered “based on their file extension: `.tftest.hcl` or `.tftest.json`”, with `tests` as the default directory, and run blocks “by default” executing sequentially. Each run block is a plan or an apply: “Terraform then executes a series of Terraform plan or apply commands according to the test files’ specifications” and “also validates the relevant plan and state files according to the test files’ specifications”, with “each `run` block” defaulting to `command = apply`. The sample’s first run opts into the cheaper default and points the run at the child module directly, since within test files the `module` block “only supports the `source` attribute and the `version` attribute”, and test sources may be “local and registry modules”:

```
run "network_plan" {
  command = plan

  module {
    source = "./modules/network"
  }

  variables {
    vpc_cidr    = "10.0.0.0/16"
    environment = "dev"
  }

  assert {
    condition     = aws_vpc.main.cidr_block == "10.0.0.0/16"
    error_message = "vpc cidr did not come from the variable"
  }

  assert {
    condition     = length(aws_subnet.main) == 2
    error_message = "subnet count did not match the availability zone list"
  }
}
```

listing 266: the first run block: command = plan, the child module as source, two asserts on plan known values (source: c-os-cloud/samples/src/Ch24/network.tftest.hcl, lines 6-27)

Assertions are the checks themselves: “Each `assert` block contains a `condition` argument and an `error_message` argument”, and both of the sample’s assert on values a plan can know, the vpc’s `cidr_block`, which comes straight from a variable, and the length of the subnet list, which `count` fixes before any resource exists. Variable inputs ride “variables blocks at both the root level and within `run` blocks”, so the run sets `vpc_cidr` itself. The second run exercises the failure path. `expect_failures` “can provide a list of checkable objects”, “resources, data sources, check blocks, input variables, and outputs”, “that should fail their custom conditions”, and `var.environment` with its validation block is exactly such an object, so the run feeds it `production` and expects the rejection:

```
run "rejects_unknown_environment" {
  command = plan

  module {
    source = "./modules/network"
  }

  variables {
    vpc_cidr    = "10.0.0.0/16"
    environment = "production"
  }

  expect_failures = [
    var.environment,
  ]
}
```

listing 267: the failure path: a validation block expected to reject its input (source: c-os-cloud/samples/src/Ch24/network.tftest.hcl, lines 29-44)

WARNING: the gate this book honestly does not run

`fmt` and `validate` need no cloud account, `validate` after `terraform init -backend=false` installs plugins and modules while touching no backend. `test` is different in kind: "The Terraform `terraform test` command creates real infrastructure", the page recommends "dedicated testing accounts within the target providers that you can routinely and safely purge", and the escape is mocks, which let you test "without creating infrastructure or requiring credentials". This corpus stops one step earlier: the `.tftest.hcl` file is real, never executed, like every file under `Ch24/`.

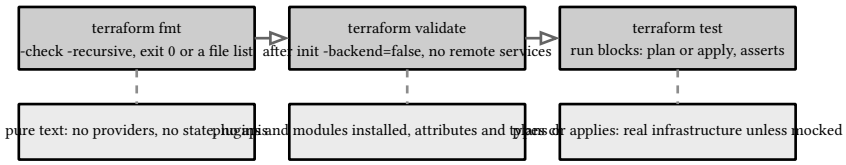


figure 108: the no cloud gate: three commands, what each touches, what none of it touches

sources: developer.hashicorp.com terraform pages, input variables, variable block, output values, output block, local values, modules overview, module block, module sources, develop modules, standard module structure, provider requirements, backends configuration, s3 backend, workspaces language and cli pages, workspace subcommands, fmt, validate, test, tests, mocking, and cidrsubnet, all accessed 2026-09-12. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied. The version pins are 1.16.2 for the cli and 6.64.0 for hashicorp/aws, chapter 23's freeze unchanged, both Latest on the release pages at access time.

25. aws essentials through terraform

Chapter 23 read the language and chapter 24 composed modules. This chapter builds the smallest honest slice of one provider: one role, one network, one instance, one bucket, one function, in seven files under `Ch25/`. The verification mode stays the one chapters 23 and 24 set: the files are real, pinned terraform and nothing ever runs them, no terraform binary exists in this corpus, nothing is initialized, planned, or applied. Every behavioral sentence below is quoted from the provider documentation at tag v6.64.0 or the AWS service documentation, all fetched 2026-09-12, and the resource argument names are read from those pages, not remembered from training data. Two provider habits drove the sample's shape before a line was written: the role's `inline_policy` and `managed_policy_arns` arguments and the security group's inline `ingress` and `egress` arguments are deprecated on the pages that define them, and the S3 bucket shed its versioning and encryption arguments into separate resources. The sample uses the shapes the pages recommend.

25.1. iam: users, roles, and the boundary of trust

The IAM guide fixes the two identity kinds in two sentences. A role is "an IAM identity that you can create in your account that has specific permissions", and where a user is "uniquely associated with one person, a role is intended to be assumable by anyone who needs it". The credential model is the real divide: "a role does not have standard long-term credentials such as a password or access keys associated with it. Instead, when you assume a role, it provides you with temporary security credentials for your role session." The guide's advice for users is to avoid them: "We recommend you only use IAM users for use cases not supported by identity federation." The best practices page states what a workload gets instead: "When you're building on an AWS compute service, such as Amazon EC2 or Lambda, AWS delivers the temporary credentials of an IAM role to that compute resource."

Delegation is the model that ties the two policy kinds together. To delegate, "you create an IAM role in the trusting account that has two policies attached. The permissions policy grants the user of the role the needed permissions to carry out the intended tasks on the resource. The trust policy specifies which trusted account members are allowed to assume the role." The trust policy is defined precisely: "A JSON policy document in which you define the principals that you trust to assume the role. A role trust policy is a required resource-based policy that is attached to a role in IAM." The provider resource mirrors that structure with one required argument: `assume_role_policy` is "Policy that grants an entity permission to assume the role", and the page suggests building it with `jsonencode()` or the `aws_iam_policy_document` data source. The sample takes the data source route:

```
data "aws_iam_policy_document" "instance_assume_role" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "instance" {
  name               = "ch22-instance-role"
  description        = "assumed by the ec2 instance launched by this chapter"
  assume_role_policy = data.aws_iam_policy_document.instance_assume_role.json
}
```

listing 268: the trust half: a service principal, one action, wired into the role's one required argument (source: c-os-cloud/samples/src/Ch25/iam.tf, lines 9-24)

The trust statement names the `ec2.amazonaws.com` service principal and grants exactly one action, `sts:AssumeRole`. The instance profile below it is the delivery vehicle for EC2: the instance resource takes `iam_instance_profile` as "the name of the Instance Profile", and the profile carries the role. The permission half follows the best practices page verbatim: "grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as least-privilege permissions."

```
data "aws_iam_policy_document" "instance_read_uploads" {
  statement {
    actions = [
      "s3:GetObject",
      "s3:ListBucket",
    ]

    resources = [
      aws_s3_bucket.uploads.arn,
      "${aws_s3_bucket.uploads.arn}/*",
    ]
  }
}

resource "aws_iam_role_policy" "instance_read_uploads" {
  name   = "ch22-instance-read-uploads"
  role   = aws_iam_role.instance.name
  policy = data.aws_iam_policy_document.instance_read_uploads.json
}
```

listing 269: the permission half: two actions, one bucket arn, no wildcard (source: c-os-cloud/samples/src/Ch25/iam.tf, lines 31-49)

The `resources` list is the whole discipline in one expression: the bucket arn and the bucket arn with `/*`, nothing else. The policy document lives in an `aws_iam_role_policy` resource, whose `policy` argument is "The inline policy document. This is a JSON formatted string", and whose `role` argument is "The name of the IAM role to attach to the policy". This separate resource is not a style choice. The role page marks the in-body alternatives deprecated and warns what exclusive management would cost: configuring `inline_policy` or `managed_policy_arns` on `aws_iam_role` "will take over exclusive management of the role's respective policy types", which "are incompatible with other ways of managing a role's policies" and end in "resource cycling and/or errors". The lambda section uses the other recommended carrier, `aws_iam_role_policy_attachment`, for a managed policy.

PITFALL: least privilege is a resource list, not a feeling

The lazy pattern writes `Resource = "*"`, the wildcard. The page's own definition refuses that shortcut: permissions are "the actions that can be taken on specific resources under specific conditions". In the sample the statement names two actions and one bucket, the one bucket this chapter creates, so the role can read the uploads and nothing else. Widening it is a one-line diff in one file, visible in every plan review, which is the point.

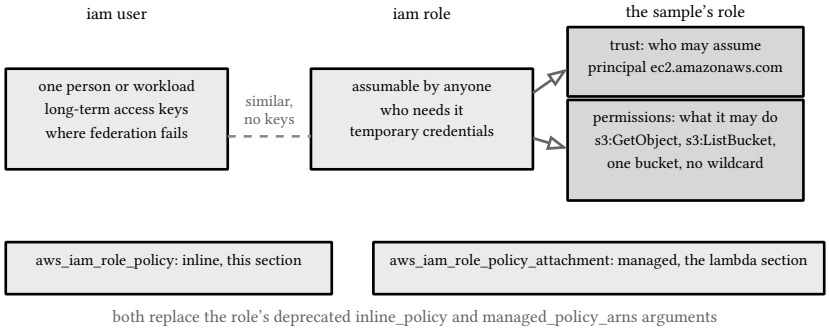


figure 109: iam in one view: the two identity kinds, and the two-policy anatomy every role carries

25.2. vpc: cidrs, subnets, route tables, firewalls

A VPC is "a logically isolated virtual network that you've defined" that "closely resembles a traditional network that you'd operate in your own data center". The address plan starts with CIDR notation, and the VPC addressing page works the arithmetic: "10.0.0.0/16 represents 65,536 IPv4 addresses from 10.0.0.0 to 10.0.255.255". The sample spends that budget one zone at a time: a /24 per subnet, 256 addresses each, 10.0.0.0/24 in one zone and 10.0.1.0/24 in the other, out of the /16 the VPC holds.

The zone assignment is structural, not cosmetic: "Each subnet must reside entirely within one Availability Zone and cannot span zones", and the page gives the reason: "By launching AWS resources in separate Availability Zones, you can protect your applications from the failure of a single Availability Zone." Chapter 22 carries the caveat that zone codes are per-account names for accounts created before November 2025, so `${var.region}a` and `${var.region}b` name zones by code, which is the convention the provider's `availability_zone` argument, "AZ for the subnet", is written around.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "ch22"
  }
}
```

listing 270: the vpc: one cidr_block argument, the /16 the whole network carves from (source: c-os-cloud/samples/src/Ch25/vpc.tf, lines 7-14)

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.0.0/24"
  availability_zone       = "${var.region}a"
  map_public_ip_on_launch = true

  tags = {
    Name = "ch22-public"
  }
}

resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "${var.region}b"

  tags = {
    Name = "ch22-private"
  }
}
```

listing 271: the two subnets: one cidr carved per zone, public ip on launch only in the public one (source: c-os-cloud/samples/src/Ch25/vpc.tf, lines 15-34)

What makes a subnet public or private is nothing in the subnet itself: "The subnet type is determined by how you configure routing for your subnets." A public subnet "has a direct route to an internet gateway", a private one does not. The routing page defines the object that decides: "A route table serves as the traffic controller for your virtual private cloud (VPC). Each route table contains a set of rules, called routes, that determine where network traffic from your subnet or gateway is directed." Every VPC starts with one: "When you create a VPC, we also create the main route table for the VPC", and "Every subnet that you create is automatically associated with the main route table for the VPC." The provider exposes that default as an attribute, `default_route_table_id`, "ID of the route

table created by default on VPC creation", and the sample declines to edit it, building two named tables instead and associating each subnet explicitly:

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "ch22-public"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

listing 272: the public route table: the default route to the internet gateway, and the association that binds the public subnet to it (source: c-os-cloud/samples/src/Ch25/vpc.tf, lines 44-60)

The public table carries one route, 0.0.0.0/0 to the internet gateway. The private table, lines 62 through 73 of the same file, carries none: the AWS-created local route for the VPC range is not something the configuration has to state, and no destination outside the VPC is reachable from that subnet.

The firewall layer is the security group: it "controls the traffic that is allowed to reach and leave the resources that it is associated with" and "acts as a virtual firewall". Its statefulness is the property to internalize: "Security groups are stateful. For example, if you send a request from an instance, the response traffic for that request is allowed to reach the instance regardless of the inbound security group rules. Responses to allowed inbound traffic are allowed to leave the instance, regardless of the outbound rules." The subnet-level counterpart is stateless and open by default: "Network ACLs allow or deny inbound and outbound traffic at the subnet level", and "The default network ACL allows all inbound and outbound traffic."

```
resource "aws_security_group" "web" {
  name        = "ch22-web"
  description = "https in from the vpc, everything out, stateful both ways"
  vpc_id      = aws_vpc.main.id

  tags = {
    Name = "ch22-web"
  }
}

resource "aws_vpc_security_group_ingress_rule" "https_from_vpc" {
  security_group_id = aws_security_group.web.id
  cidr_ipv4         = aws_vpc.main.cidr_block
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}

resource "aws_vpc_security_group_egress_rule" "allow_all" {
  security_group_id = aws_security_group.web.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}
```

listing 273: the security group and its rules as separate resources: one ingress, one egress, one cidr each (source: c-os-cloud/samples/src/Ch25/vpc.tf, lines 75-97)

The rules live outside the group on purpose. The group page says to "avoid" the inline `ingress` and `egress` arguments and "use the current best practice of the `aws_vpc_security_group_egress_rule` and `aws_vpc_security_group_ingress_rule` resources with one CIDR block per rule". One more provider fact shapes the pair: "By default, AWS creates an `ALLOW ALL` egress rule when creating a new Security Group inside of a VPC", and "Terraform will remove this default rule", so the egress rule in the listing is not paranoia, it is the re-creation the removal requires. The ingress rule names `ip_protocol = "tcp"`, ports 443 through 443, and the VPC's own CIDR as the source. The egress rule uses -1, which the rule page defines as all protocols.

NOTE: the main route table is a default you inherit

Every subnet lands in the main route table without being asked. The sample's two explicit `aws_route_table_association` resources exist so that no subnet's routing is implied: the day someone edits the main table to add a default route, subnets that were private by accident of association would follow it. Explicit tables plus explicit associations make the network's shape a fact of the configuration, not of the order resources happened to be created in.

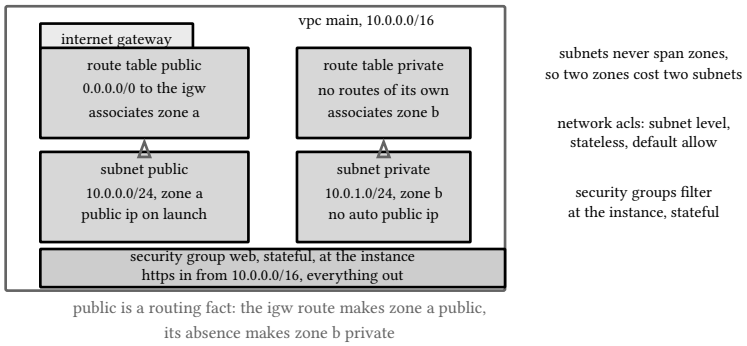


figure 110: the sample's network in one stack: vpc, two zones, two route tables, one stateful firewall

25.3. ec2: one instance, end to end

The instance resource is where the previous two sections meet, and its first argument is the one with the strongest pinning story. “An Amazon Machine Image (AMI) is an image that provides the software that is required to set up and boot an Amazon EC2 instance”, and “You must specify an AMI when you launch an instance.” The page then lists what an AMI is specific to: region, operating system, processor architecture, root volume type, virtualization type. Region is first: an AMI id names an image in one region, which is why the sample takes it from a variable with no default instead of baking a literal into the file.

The provider accepts three spellings for that argument, and its own examples show all three:

```
ami = "ami-0aaaaaaaaaaaaaaa1"

ami = data.aws_ami.ubuntu.id        # most_recent = true

ami = "resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
```

fragment, illustrative, not executed

A literal id is a pin, visible in every plan. The data source with `most_recent = true` follows the newest match, which means the instance’s image can change under a configuration that did not. The `resolve:ssm:` alias names an image by public parameter, which the provider resolves at apply time. The sample takes the first form through a variable, the same shape chapter 23 used, so the pin is the caller’s decision and the file records it. The shape argument decodes by rule: “Instance types are named based on their instance family and instance size”, the first family position is the series, the second the generation, the third the options, and “After the period (`.`) is the instance size”. The table lists T as burstable performance, so `t3.micro` reads as third-generation burstable, micro size.

```
resource "aws_instance" "web" {
  ami                         = var.ami
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.public.id
  vpc_security_group_ids      = [aws_security_group.web.id]
  iam_instance_profile        = aws_iam_instance_profile.web.name
  user_data_replace_on_change = true

  user_data = <<-EOT
    #!/bin/bash
    echo "provisioned by terraform, chapter 22" > /var/local/provisioned.txt
  EOT

  tags = {
    Name = "ch22-web"
  }
}
```

listing 274: the instance: pinned ami, decoded shape, subnet placement, profile, group, and opaque user data (source: c-os-cloud/samples/src/Ch25/ec2.tf, lines 7-23)

Every argument but the first two is a reference into the earlier files: `subnet_id` picks the zone, `vpc_security_group_ids` is “List of security group IDs to associate with”, `iam_instance_profile` carries the role from the iam section by profile name. User data is the launch-time channel: with it you “perform automated configuration tasks, or to run scripts after the instance starts”, and the contract is deliberately loose, because “User data is treated as opaque data: what you give is what you get back. It is up to the instance to interpret it.” The raw payload is bounded: “User data is limited to 16 KB, in raw form, before it is base64-encoded.”

The lifecycle rules come from the argument docs, and they differ by field. An `instance_type` change "will trigger a stop/start of the EC2 instance", and so will a `user_data` change "by default". The sample sets `user_data_replace_on_change`, which "will trigger a destroy and recreate of the EC2 instance when set to `true`", turning a script edit into replacement, the honest semantics for a field that only runs at boot. After launch the resource reports back `id`, `public_ip`, and `instance_state`, whose values the provider page enumerates as pending, running, shutting-down, terminated, stopping, or stopped.

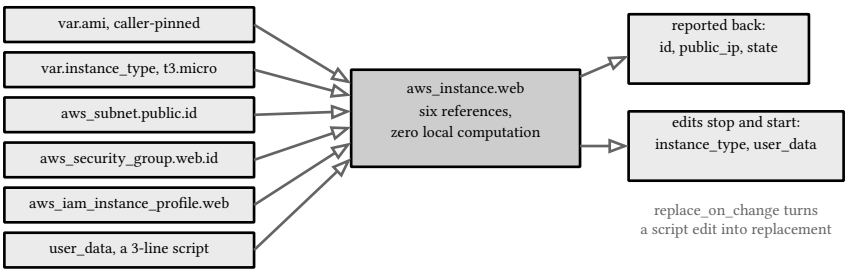


figure 111: the instance resource as a junction: six inputs on the left, one resource, the reported attributes on the right

25.4. s3: the object model against the block model

The EC2 chapter's root volume is a block device. S3 is the other model: "Amazon S3 is an object storage service that stores data as objects, hierarchical data, or tabular data within buckets. An object is a file and any metadata that describes the file. A bucket is a container for objects." The address of an object is its name, nothing else: "Each object has a key (or key name), which is the unique identifier for the object within the bucket." There are no offsets and no filesystem geometry, and reads are consistent: "Amazon S3 provides strong read-after-write consistency for PUT and DELETE requests of objects in your Amazon S3 bucket in all AWS Regions."

The name of the container is itself a managed resource, because general purpose buckets "exist in a global namespace, which means that each bucket name must be unique across all AWS accounts in all the AWS Regions within a partition". The character rules are short: names "must be between 3 (min) and 63 (max) characters long", "can consist only of lowercase letters, numbers, periods (.), and hyphens (-)", and "must begin and end with a letter or number". The sample encodes those rules as a validation block on the input, so a bad name fails at plan time rather than at the API:

```
variable "bucket_name" {
  type        = string
  description = "bucket name, unique across every account in the partition"

  validation {
    condition     = can(regex("^[a-z0-9][a-z0-9.-]{1,61}[a-z0-9]$", var.bucket_name))
    error_message = "bucket names are 3 to 63 characters of lowercase letters, digits, periods, and
hyphens, beginning and ending with a letter or number."
  }
}
```

listing 275: the naming rules as a validation block: the character set, the bounds, the first and last character (source: c-os-cloud/samples/src/Ch25/variables.tf, lines 18-26)

The provider's bucket resource has narrowed to the name plus housekeeping. Its `versioning` and `server_side_encryption_configuration` arguments are marked deprecated with pointers to the split resources, and the sample follows the pointers:

```
resource "aws_s3_bucket" "uploads" {
  bucket = var.bucket_name

  tags = {
    Name = "ch22-uploads"
  }
}

resource "aws_s3_bucket_versioning" "uploads" {
  bucket = aws_s3_bucket.uploads.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

listing 276: the bucket and its versioning resource: one name argument, one status (source: c-os-cloud/samples/src/Ch25/s3.tf, lines 8-22)

Versioning is off unless asked: “By default, S3 Versioning is disabled on buckets, and you must explicitly enable it.” Once on, it changes the delete and overwrite semantics: “if you delete an object, Amazon S3 inserts a delete marker instead of removing the object permanently”, and an overwrite “results in a new object version in the bucket”. The state machine only runs one way: “After you version-enable a bucket, it can never return to an unversioned state”, only suspend. And the cost model is blunt: “Each version of an object is the entire object; it is not just a diff from the previous version.” The provider page adds one operational warning with a clock in it: “AWS recommends that you wait for 15 minutes after enabling versioning before issuing write operations (PUT or DELETE) on objects in the bucket.”

```
resource "aws_s3_bucket_server_side_encryption_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

listing 277: the encryption resource: AES256 written down, the algorithm the service already applies by default (source: c-os-cloud/samples/src/Ch25/s3.tf, lines 24-32)

Encryption is the one place the configuration states a default. The service page: “Amazon S3 now applies server-side encryption with Amazon S3 managed keys (SSE-S3) as the base level of encryption for every bucket in Amazon S3”, and “Starting January 5, 2023, all new object uploads to Amazon S3 are automatically encrypted at no additional cost and with no impact on performance.” The FAQ closes the exit: “You can no longer disable encryption for new object uploads.” So the resource’s `AES256`, one of the three `sse_algorithm` values the page lists alongside `aws:kms` and `aws:kms:dsse`, records the default rather than creating one. The provider note matters for teardown: “Destroying an `aws_s3_bucket_server_side_encryption_configuration` resource resets the bucket to Amazon S3 bucket default encryption.”

PITFALL: a deleted bucket name can be inherited

The global namespace is a queue with no lock. After a bucket is deleted, “another AWS account in the same partition can use the same bucket name for a new bucket and can therefore potentially receive requests intended for the deleted bucket.” A name retired by destroy is not a name reserved. The sample takes the

> name from a variable, the same as the ami, so the choice is deliberate and reviewable, and the naming rules are enforced before any request leaves the plan.

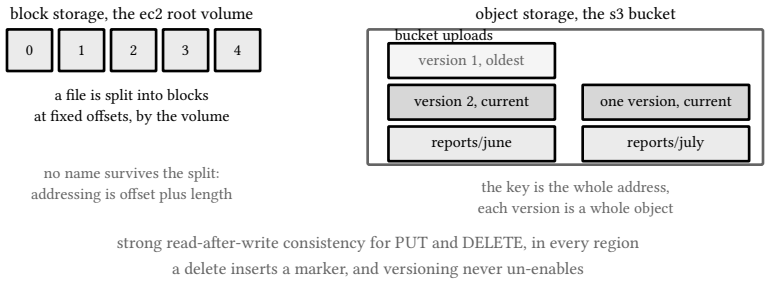


figure 112: the same bytes in two shapes: blocks at fixed offsets on the left, keys and whole-object versions on the right

25.5. lambda: the execution role and the cold start

The last resource is the smallest and the one with the strangest runtime model. Its identity is an IAM role again, but a different one: “A Lambda function’s execution role is an AWS Identity and Access Management (IAM) role that grants the function permission to access AWS services and resources.” The provider’s `aws_lambda_function` makes the same statement from the configuration side: the `role` argument is the “ARN of the function’s execution role. The role provides the function’s identity and access to AWS services and resources.” The assume happens without your code: “Lambda automatically assumes your execution role when you invoke your function”, and the trust policy has one hard requirement: “In order for Lambda to properly assume your execution role, the role’s trust policy must specify the Lambda service principal (`lambda.amazonaws.com`) as a trusted service.”

```
data "aws_iam_policy_document" "lambda_assume_role" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["lambda.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "function" {
  name               = "ch22-lambda-execution"
  description        = "the function's execution role, assumed by lambda itself"
  assume_role_policy = data.aws_iam_policy_document.lambda_assume_role.json
}

resource "aws_iam_role_policy_attachment" "function_logs" {
  role       = aws_iam_role.function.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}
```

listing 278: the execution role: same trust shape as the ec2 role, different principal, and a managed policy through the attachment resource (source: c-os-cloud/samples/src/Ch25/lambda.tf, lines 8-28)

The permission half rides an AWS managed policy this time. `AWSLambdaBasicExecutionRole` is the one the console uses by default, giving “basic permissions to log events to Amazon CloudWatch Logs”, and its policy document grants exactly three actions, `logs:CreateLogGroup`, `logs:CreateLogStream`, and `logs:PutLogEvents`. It attaches through `aws_iam_role_policy_attachment`, whose two required arguments are the role name and the policy arn, the resource the role page recommends over its deprecated `managed_policy_arns` argument.

```
resource "aws_lambda_function" "uploads_report" {
  filename      = "lambda/function.zip"
  function_name = "ch22-uploads-report"
  role          = aws_iam_role.function.arn
  handler       = "index.handler"
  runtime       = "nodejs24.x"
  memory_size   = 128
  timeout       = 3

  environment {
    variables = {
      BUCKET = aws_s3_bucket.uploads.id
    }
  }

  tags = {
    Name = "ch22-uploads-report"
  }
}
```

listing 279: the function: pinned runtime, one handler, the documented defaults for memory and timeout written out (source: c-os-cloud/samples/src/Ch25/lambda.tf, lines 30-48)

The handler is the entry point: “The Lambda function handler is the method in your function code that processes events. When your function is invoked, Lambda runs the handler method. Your function runs until the handler returns a response, exits, or times out.” `index.handler` names the file and the exported method. The runtime is pinned like everything else in this book, and the runtimes page makes pinning a scheduling problem: nodejs 24 is `nodejs24.x`, deprecated April 30, 2028, with function creation blocked June 1, 2028 and updates blocked July 1, 2028, while “After a runtime is deprecated, you’re still able to create and update functions for a limited period.” The newer `nodejs26.x` is public preview, and the page is direct about that state: “Preview runtimes are not covered by the Lambda SLA or Technical Support, and should not be used for production workloads.” The deprecation rule itself is quoted once and remembered: “Lambda’s standard deprecation policy is to deprecate a runtime when any major component of the runtime reaches the end of community long-term support (LTS) and security updates are no longer available.” `memory_size` and `timeout` sit at their documented defaults, 128 MB and 3 seconds, written out so the limits are visible where they are chosen.

The runtime model is the part no other chapter has. “Lambda invokes your function in an execution environment, which provides a secure and isolated runtime environment”, and the environment has a lifecycle with phases: “In the `Init` phase, Lambda performs three tasks”, starting all extensions, bootstrapping the runtime, and running the function’s static code, and “The `Init` phase is limited to 10 seconds.” Then invocation, then suspension: “Lambda freezes the execution environment when the runtime and each extension have completed and there are no pending events.” The freeze is what makes the next request cheap: “During this time, if another request arrives for the same function, Lambda can reuse the environment. This second request typically finishes more quickly, since the execution environment is already fully set up. This is called a ‘warm start’.” The cold path is the named cost: “the first two steps of downloading the code and setting up the environment are frequently referred to as a ‘cold start’. You are charged for this time, and it adds latency to your overall invocation duration.” The page even sizes it: “Cold starts typically occur in under 1% of invocations. The duration of a cold start varies from under 100 ms to over 1 second.”

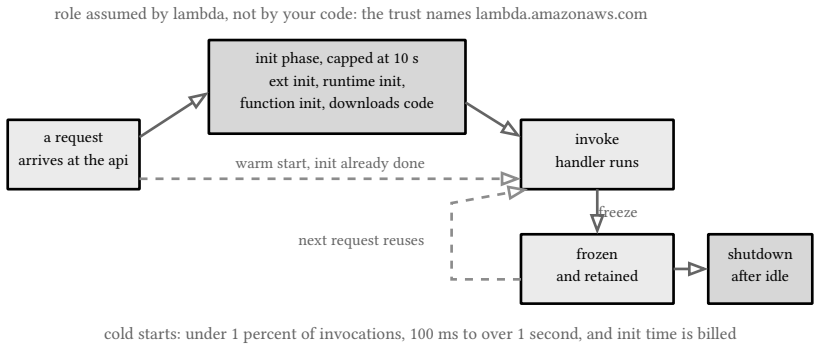


figure 113: the execution environment as a state machine: cold path through init, warm path around it, freeze and reuse between requests

sources: hashicorp/terraform-provider-aws resource documentation at tag v6.64.0 on raw.githubusercontent.com, `iam_role`, `iam_role_policy`, `iam_role_policy_attachment`, `iam_instance_profile`, `vpc`, `subnet`, `internet_gateway`, `route_table`, `route_table_association`, `security_group`, `vpc_security_group_ingress_rule`, `vpc_security_group_egress_rule`, `instance`, `s3_bucket`, `s3_bucket_versioning`, `s3_bucket_server_side_encryption_configuration`, and `lambda_function`. docs.aws.amazon.com IAM User Guide roles and security best practices, VPC User Guide what-is, configure-subnets, create-subnets, route tables, security groups, and ip addressing, EC2 User Guide amis, user data, and instance type naming conventions, S3 User Guide welcome, bucket naming rules, versioning, and the default encryption faq, Lambda developer guide execution role, runtimes, nodejs handler, and execution environment lifecycle, plus the AWSLambdaBasicExecutionRole managed policy reference. All accessed 2026-09-12, the access date the version pins in `Ch25/versions.tf` freeze. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied. The pins are terraform 1.16.2 and hashicorp/aws 6.64.0, both published 2026-09-09 and both marked Latest on the release pages when chapter 23 recorded them the same day.

26. gcp essentials through terraform

Chapter 25 walked the aws essentials as terraform files. This chapter walks the same ground on google cloud: the project as the resource container, the network model, one virtual machine, one object bucket, and one container service. The corpus rules do not move. The 7 files under `Ch26/` are real, pinned terraform configuration and nothing ever runs them: no terraform binary exists in this corpus, nothing is initialized, planned, or applied. Every behavioral claim below is a sentence quoted from a canonical page fetched 2026-09-12, or a line of those files.

The pins come from the same discipline as chapter 23's. The cli pin is 1.16.2, released 2026-09-09. The provider pin is hashicorp/google 8.2.0: the releases list reads v8.2.0 marked Latest, dated September 8, with v8.1.0 on September 1 and v8.0.0 on August 26 behind it. Both release marks were read 2026-09-12 and both are frozen in the freeze file:

```
terraform {
  required_version = "1.16.2"

  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "8.2.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
}
```

listing 280: the freeze: exact cli and provider pins, and the provider configuration the other files inherit (source: c-os-cloud/samples/src/Ch26/versions.tf, lines 7-21)

The `project` and `region` arguments in the `provider` block are the defaults the compute resources pick up. The one resource that does not pick them up is the first one this chapter needs.

26.1. projects and iam

Google cloud keeps every resource inside a hierarchy, and the bottom layer of that hierarchy is the project. The resource hierarchy page names it plainly: "The project resource is the fundamental organizing entity." and "You need a project resource to use Google Cloud." The tree above it is short, organization at the root, optional folders below, and one rule binds it all: "All resources except for the highest resource in a hierarchy have exactly one parent." Projects are where that tree stops and the billable objects begin, virtual machines, buckets, and the rest live inside a project, and the page's own summary of the project is that it is "essential for creating, enabling, and using all Google Cloud services", the thing apis, billing, collaborators, and permissions hang from.

Authorization inside that container is role based. The iam overview separates the two halves: "Permissions determine what operations are allowed on a resource." and permissions are named in a grammar, `service.resource.verb`, like `compute.instances.get`. Principals never receive permissions one by one, the page is explicit, "You can't directly grant permissions to a principal.", because "Roles are collections of permissions." and granting a role grants everything in it. The three role families round out the taxonomy. Basic roles are "Highly permissive roles that provide broad access to Google Cloud services", suited "for testing purposes, but shouldn't be used in production environments". Predefined roles are "Roles that are managed by Google Cloud services", one curated collection per service. Custom roles are "Roles that you create that contain only the permissions that you specify", the only kind you author, at the cost of maintaining them.

The aws comparison from chapter 25 lands exactly here. Amazon's access page states the model in two sentences: "You manage access in AWS by creating policies and attaching them to IAM identities or AWS resources. Policies are JSON documents in AWS that, when attached to an identity or resource, define their permissions." In aws the unit of grant is a document you write, evaluate it against the request context, deny by default, explicit deny wins. In google cloud the unit of grant is a name from a catalogue, you attach `roles/compute.viewer`, you do not write it. The terraform surface mirrors the difference: chapter 25 wrote policy documents, this chapter's sample is two lines of naming.

Terraform's provider splits the gcp grant into four resources, and the umbrella page opens with the taxonomy: "Four different resources help you manage your IAM policy for a project." The authoritative end is `google_project_iam_policy`, it "replaces any existing policy already attached", and the page warns what that costs, "You can accidentally lock yourself out of your project" with it. One step in, `google_project_iam_binding` is "Authoritative for a given role", it owns one role's member list and preserves every other role. The non-authoritative end is `google_project_iam_member`, it "Updates the IAM policy to grant a role to a new member. Other members for the role for the project are preserved." The fourth, `audit_config`, owns audit logging per service. The two ends cannot mix: the policy resource "cannot be used in conjunction with" the other three "or they will fight over what your policy should be." The sample uses the member flavor, the one that cannot destroy a grant it never mentioned:

```
resource "google_project_iam_member" "compute_viewer" {
  project = var.project_id
  role    = "roles/compute.viewer"
  member  = "user:jane@example.com"
}

resource "google_project_iam_member" "object_viewer" {
  project = var.project_id
  role    = "roles/storage.objectViewer"
  member  = "serviceAccount:deployer@${var.project_id}.iam.gserviceaccount.com"
}
```

listing 281: two member grants: a role from the catalogue, an identity with its type prefix, an explicit project (source: c-os-cloud/samples/src/Ch26/iam.tf, lines 6-16)

Both grants name their `project` because the argument reference says the project "is not inferred from the provider", the one argument in this chapter that ignores the provider block. The `member` strings carry their principal type in the prefix, `user:` for an account, `serviceAccount:` for the machine identity the second grant builds with string interpolation. Both roles are predefined ones, from the managed catalogue, not the basic set.

PITFALL: authority is the dangerous axis in project iam

The four flavors differ only in how much they claim to own. `google_project_iam_policy` replaces the whole policy and deleting it "removes access from anyone without organization-level access to the project". `google_project_iam_binding` plus `google_project_iam_member` on the same role is the other collision, the two fight over that role's member list. The member resource alone never removes anyone, which is why it is the flavor a book can print without a warning banner around it.

resource	claims ownership of	preserves
iam_policy	the whole policy	nothing, replaces it
iam_binding	one role, its member list	every other role
iam_member	one grant: role to member	other members of the role
iam_audit_config	audit logging, one service	every other service

the member row is the only one that replaces nothing, the sample lives there

figure 114: the four project iam resources on the authority axis, what each replaces and what each preserves

26.2. the vpc model

The google network inverts the aws shape. The vpc overview fixes the scopes in three sentences: “VPC networks, including their associated routes and firewall rules, are global resources.” and “They are not associated with any particular region or zone.” while “Subnets are regional resources.” The network object owns no address range and belongs to no region, it is a name that spans the planet, and the ranges arrive as subnetworks pinned to regions. Chapter 25′s aws opposite came from the vpc basics page: “A VPC spans all of the Availability Zones in a Region.” The aws network is itself regional, the az is inside it, and “you can add one or more subnets in each Availability Zone”. In google cloud the region is inside the network instead, one subnet per region wherever an address range is needed.

A network is born in one of two modes, and the provider argument is the switch: `auto_create_subnetworks` set to true means the network “will create a subnet for each region automatically across the `10.128.0.0/9` address range”, set to false it is in custom subnet mode “so the user can explicitly connect subnetwork resources”. The sample takes custom mode and writes its own ranges:

```
resource "google_compute_network" "vpc" {
  name                    = "ch23-network"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name          = "ch23-subnet"
  region        = var.region
  network       = google_compute_network.vpc.id
  ip_cidr_range = "10.10.0.0/24"
}
```

listing 282: a global custom-mode network and one regional subnetwork owning 10.10.0.0/24 (source: c-os-cloud/samples/src/Ch26/network.tf, lines 5-15)

The subnet’s `ip_cidr_range` is “The range of internal addresses that are owned by this subnetwork.” with one global constraint, “Ranges must be unique and non-overlapping within a network. Only IPv4 is supported.” The `region` argument is “The GCP region for this subnetwork”, and the resource id embeds it, `projects/{project}/regions/{region}/subnetworks/{name}`, the regional half of the layer stack.

Firewall is where the model is most unlike aws. The provider page opens: “Each network has its own firewall controlling access to and from the instances.” and then the default posture, “All traffic to instances, even from other instances, is blocked by the firewall unless firewall rules are created to allow it.” A manually created network ships with only a default allow for outgoing traffic and a default deny for incoming, every real rule is the tenant’s to write. Rules are network-scoped objects, `direction` defaults to `INGRESS`, ingress rules must name a source, and `target_tags` narrows which instances a rule touches, without it “the firewall rule applies to all instances on the specified network.” Priorities are integers from 0 to 65535, unspecified means 1000, and ties break one way, “DENY rules take precedence over ALLOW rules having equal priority.” The aws contrast is the attachment point: “A security group controls the traffic that is allowed to reach and leave the resources that it is associated with”, and “after you associate a security group with an EC2 instance, it controls the inbound and outbound traffic for the instance.” The security group rides on the instance, the google rule lives on the network and selects instances by tag.

```
resource "google_compute_firewall" "allow_ssh" {
  name          = "ch23-allow-ssh"
  network       = google_compute_network.vpc.name
  direction     = "INGRESS"
  priority      = 1000

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  source_ranges = ["10.10.0.0/24"]
}
```

listing 283: one ingress rule on the network: tcp 22 from the subnet range only, priority stated as the default 1000 (source: c-os-cloud/samples/src/Ch26/network.tf, lines 17-29)

The rule's `source_ranges` is the subnet's own cidr, so ssh reaches the instance only from inside the network. No instance is named anywhere in the block, the attachment is by geography and, when needed, by tag.

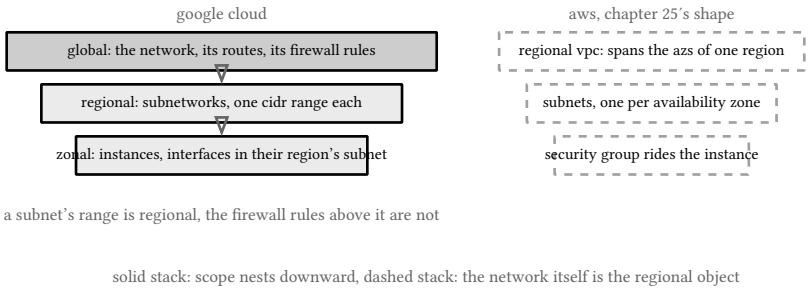


figure 115: the google network as a three-layer stack against the aws shape

26.3. compute engine instances

A compute engine instance is a zonal object with a named shape. The machine types page defines the catalogue: a machine family is "a curated set of processor and hardware configurations optimized for specific workloads", six families from general purpose to memory optimized to accelerator optimized, and inside a family the types come as "machine series and predefined machine types". The name is the spec sheet read left to right, series, descriptor, vcpu count: "the `n2-standard-4` machine type creates a VM with 4 vCPUs and 16 GB of memory". The sample's `e2-standard-2` is the same grammar on the e2 series, general purpose, 2 vcpus. Small loads can go smaller, "The E2 and N1 series contain shared-core machine types." that timeshare a physical core, `e2-micro` up to `e2-medium`. Off the catalogue entirely there is a formula grammar, `custom-NUMBER_OF_CPUS-AMOUNT_OF_MEMORY_MB`, `custom-6-20480` for 6 vcpus and 20 GB, capped at "6.5 GB per CPU unless you add extended memory". Resizing an existing instance needs `allow_stopping_for_update`, the argument reference says so before anything else about `machine_type`.

```
resource "google_compute_instance" "app" {
  name         = "ch23-app"
  machine_type = "e2-standard-2"
  zone         = "${var.region}-a"

  tags = ["ch23-web"]

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }

  network_interface {
    subnetwork = google_compute_subnetwork.subnet.id
  }

  metadata_startup_script = "apt-get update && apt-get install -y nginx"

  labels = {
    environment = "ch23"
  }
}
```

listing 284: one zonal instance: family-name shape, image by family shorthand, interface in the custom subnet, boot script in metadata (source: c-os-cloud/samples/src/Ch26/compute.tf, lines 7-29)

The `zone` argument places the instance in one zone of the subnet's region, and the placement is checked, the interface's subnetwork "must exist in the same region this instance will be created in", and with a custom-mode network "specifying the subnetwork is required." The boot disk takes the image by project and family, `debian-cloud/debian-11`, a living pointer that resolves to the current image of that family rather than a frozen id. There is no `access_config` block in the interface, and the provider page states what that omission buys: "Omit to ensure that the instance is not accessible from the Internet."

The startup script is the instance's boot-time behavior. The compute docs define it: "A startup script is a file that contains commands that run when a virtual machine (VM) instance boots." and the delivery mechanism is metadata, "A startup script is passed to a VM from a location that is specified by a metadata key." The `startup-script` key carries a script stored locally or added directly, up to 256 KB, `startup-script-url` points at Cloud Storage for bigger ones, and the guest environment reads the key and runs the script as root when the machine boots. The provider offers the same script through two different arguments. Inside `metadata`, the key runs "in a shell on every boot" and changing it changes nothing terraform tracks. The dedicated `metadata_startup_script` argument is the other polarity: "An alternative to using the startup-script metadata key, except this one forces the instance to be recreated (thus re-running the script) if it is changed." The sample uses the second one on purpose, a changed script should read as a new machine.

The ec2 counterpart is chapter 25's user data, and the polarity is flipped. The ec2 page: "you can pass user data to the instance that is used to perform automated configuration tasks, or to run scripts after the instance starts", and "By default, user data scripts and cloud-init directives run only during the boot cycle when you first launch your instance", every-boot needs an explicit persist tag. Google cloud runs the startup script on every boot by default. The ec2 side also treats the payload as uninterpreted bytes, "User data is treated as opaque data: what you give is what you get back.", base64 encoded at the api, capped at 16 KB. The google side puts the script in a queryable metadata server and lets terraform hash it into the resource identity.

NOTE: two spellings of one script, only one is tracked

> `metadata_startup_script` and `metadata.startup-script` are the same bytes to the guest environment and different resources to terraform. The metadata map version can drift silently, edit it and the plan stays empty while the next boot runs new code. The dedicated argument turns that edit into a destroy and recreate, which is the honest representation: a machine that booted different code is a different machine. The two cannot be used simultaneously on the same instance.



the shape is fixed before boot: e2-standard-2, zone us-central1-a, no internet address

aws runs the same pipeline with user data, once by default instead of every boot

figure 116: the boot pipeline: image to disk to interface to metadata to script

26.4. cloud storage buckets

The object store has one container type. The storage docs say it twice: “Buckets are the basic containers that hold your data as objects.” and “Everything that you store in Cloud Storage must be contained in a bucket.” The namespace above the containers is flat and shared with the whole world, “Every bucket name must be globally unique”, lowercase, 3 to 63 characters. The provider’s argument reference adds the two structural ones, `name` and `location`, the location being a region, a dual region, or a multi-region like the sample’s `US`.

```
resource "google_storage_bucket" "artifacts" {
  name                        = "ch23-artifacts-517f2c"
  location                    = "US"
  storage_class               = "STANDARD"
  uniform_bucket_level_access = true

  labels = {
    environment = "ch23"
  }

  lifecycle_rule {
    condition {
      age = 30
    }

    action {
      type          = "SetStorageClass"
      storage_class = "NEARLINE"
    }
  }
}
```

listing 285: one bucket: unique name, multi-region location, inherited class, uniform access, one transition rule (source: c-os-cloud/samples/src/Ch26/storage.tf, lines 6-26)

The storage class is a property of objects that the bucket defaults. The classes page: “When you add objects to the bucket, they inherit this storage class unless explicitly set otherwise.” The ladder has four rungs, each a price for an access cadence. Standard for frequently accessed data. Nearline “is ideal for data you plan to read or modify on average once per month or less”, with a 30-day minimum storage duration. Coldline for data read “at most once a quarter”, 90-day minimum. Archive, “the best choice for data that you plan to access less than once a year”, 365-day minimum. The provider accepts `STANDARD`, `NEARLINE`, `COLDLINE`, `ARCHIVE` and the legacy `MULTI_REGIONAL` and `REGIONAL` spellings, and its lifecycle rules move objects between rungs, the sample’s single rule sets `SetStorageClass` to `NEARLINE` when `age` reaches 30 days.

Uniform bucket-level access is the switch that collapses the access model. The docs state the effect first: “When you enable uniform bucket-level access on a bucket, Access Control Lists (ACLs) are disabled”, and from then on only bucket-level iam permissions grant access. The switch has a ratchet, “Uniform bucket-level access cannot be disabled after it has been active on a bucket for 90 consecutive days.” The sample turns it on in the same breath as the class, one line, the bucket never speaks acl again.

The s3 comparison closes the table. Amazon’s page: “Each object in Amazon S3 has a storage class associated with it.” The class is object metadata, per object, with the bucket holding only a default. The minimum-duration ladders sit side by side: aws charges 30 days for Standard-IA, 90 for Glacier Instant and Flexible, 180 for Deep Archive, google charges 30 for Nearline, 90 for Coldline, 365 for Archive. Google’s coldest rung is twice as deep in time, amazon’s coldest asks for an explicit restore before read. Both clouds put the transition engine in lifecycle rules and both default new objects to the hot class when nothing is said.

WARNING: uniform access is a one-way latch with a 90-day fuse

Enabling `uniform_bucket_level_access` is cheap and reversible for 89 days. On day 90 of continuous operation it is permanent, any acl that still matters is dead weight the bucket can no longer read, and no later terraform run can undo the argument. Treat it as a migration, finish moving grants to iam before the fuse burns, not after.

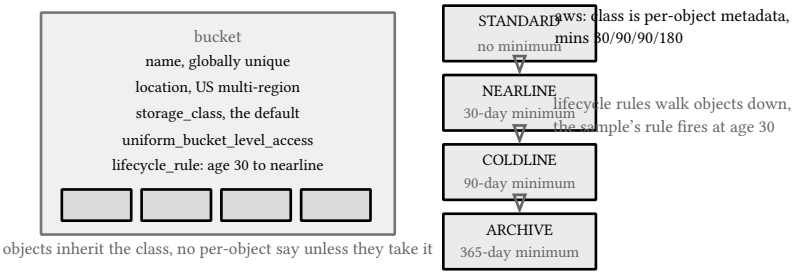


figure 117: the bucket as a data structure: fields, inheriting objects, and the class ladder

26.5. cloud run

The last resource is the serverless one, and it is container native: the deploy unit is an image, not a function. The provider page states the abstraction: “Service acts as a top-level container that manages a set of configurations and revision templates which implement a network service.” Deploys do not overwrite anything, `template` is “The template used to create revisions for this Service”, each new template stamps out a new revision, and `revision` names it, “The unique name for the revision”, generated from the service name when omitted.

```
resource "google_cloud_run_v2_service" "api" {
  name     = "ch23-api"
  location = var.region

  deletion_protection = false
  ingress             = "INGRESS_TRAFFIC_ALL"

  template {
    revision = "ch23-api-002"

    scaling {
      min_instance_count = 0
      max_instance_count = 5
    }
```

listing 286: the service, the deletion-protection override, and a named revision with scaling bounds (source: c-os-cloud/samples/src/Ch26/run.tf, lines 6-19)

The `deletion_protection` line is not decoration. The provider defaults it to true, "Whether Terraform will be prevented from destroying the service. Defaults to true.", any apply that would delete the service fails until the argument says false. A book that never applies still says false, the file should mean what it says.

```
    containers {
      image = "us-docker.pkg.dev/cloudrun/container/hello"

      resources {
        limits = {
          cpu    = "1"
          memory = "512Mi"
        }
      }
    }
  }
```

listing 287: the container: one image from the doc's own example registry, cpu and memory as limits (source: c-os-cloud/samples/src/Ch26/run.tf, lines 21-31)

Traffic is a list of allocations. The provider's `traffic` argument "Specifies how to distribute traffic over a collection of Revisions belonging to the Service", and the default when the list is empty is "100% traffic to the latest Ready Revision". The console docs put the same knob to work for rollback, gradual rollout, and splitting, with one arithmetic rule, "the percentages must add up to 100", and one testing affordance, a tag "lets you directly test the new revision at a specific URL, without serving traffic". The sample splits 90 to the revision its template names and 10 to the previous one:

```
  traffic {
    type     = "TRAFFIC_TARGET_ALLOCATION_TYPE_REVISION"
    revision = "ch23-api-002"
    percent  = 90
  }

  traffic {
    type     = "TRAFFIC_TARGET_ALLOCATION_TYPE_REVISION"
    revision = "ch23-api-001"
    percent  = 10
  }
}
```

listing 288: the 90/10 split: allocation by revision name, percents summing to 100 (source: c-os-cloud/samples/src/Ch26/run.tf, lines 33-44)

Scaling is bounded at both ends and the bounds are asymmetric. Quiescence is free: "When a revision does not receive any traffic, by default, it is scaled to zero instances." The minimum instances setting buys back latency by keeping an instance idle, warm, and billed even with no requests. The top end always exists, "All services are assigned a maximum instances limit by default, even if you don't specify your own limit", and the limit "is an upper limit per revision". In the provider the bounds live in the template's `scaling` block, `min_instance_count` "Defaults to 0", and an absent `max_instance_count` sends cloud run to "calculate a default value based on the project's available container instances quota in the region and specified instance size." The sample pins both, 0 and 5, so the state machine is fully determined by the file.

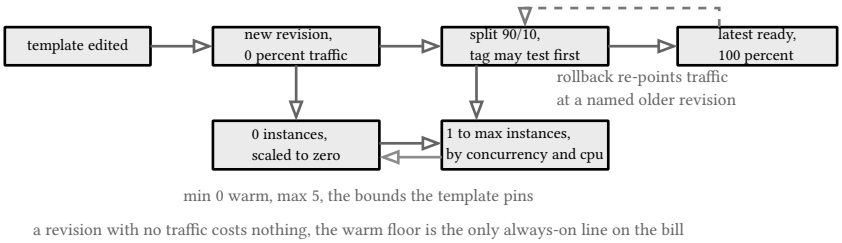


figure 118: the revision lifecycle as a state machine, with the scaling axis under every serving state

sources: hashicorp/terraform-provider-google v8.2.0 resource pages, `google_project_iam` (umbrella page, `google_project_iam_member` documented there), `google_compute_network`, `google_compute_subnetwork`, `google_compute_firewall`, `google_compute_instance`, `google_storage_bucket`, `google_cloud_run_v2_service`, fetched from the raw.githubusercontent.com pinned tag, and docs.cloud.google.com iam overview, resource hierarchy, vpc overview, machine types, startup scripts (overview and linux pages), buckets, storage classes, uniform bucket-level access, cloud run rollouts rollbacks and traffic migration, and instance autoscaling, all accessed 2026-09-12. docs.aws.amazon.com iam access management, vpc basics, security groups, ec2 user data, and s3 storage classes, accessed 2026-09-12, and github.com hashicorp/terraform-provider-google releases, accessed 2026-09-12. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied. The version pins are 1.16.2 for the cli and 8.2.0 for hashicorp/google, both Latest on their release pages at access time, the provider published 2026-09-08.

27. multi-cloud practice

Chapter 25 walked one provider, chapter 26 walked the other. This chapter is about what changes when both are in the room at once: the money, the metadata, the rules, the truth about what exists, and the spelling. The 6 files under `Ch27/` are real, pinned terraform configuration and nothing ever runs them: no terraform binary exists in this corpus, nothing is initialized, planned, or applied. Every behavioral claim below is a sentence quoted from a canonical page fetched 2026-09-12 or a line of those files.

The freeze now carries both providers in one block. The cli pin is 1.16.2, released 2026-09-09. The aws pin is hashicorp/aws 6.64.0, released 2026-09-09. The google pin is hashicorp/google 8.2.0, released 2026-09-08. All three were the releases marked Latest when read 2026-09-12:

```
terraform {
  required_version = "1.16.2"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.64.0"
    }
    google = {
      source  = "hashicorp/google"
      version = "8.2.0"
    }
  }
}
```

listing 289: the freeze: both providers, one required_providers block, exact pins with dates in the header comment (source: c-os-cloud/samples/src/Ch27/versions.tf, lines 11-24)

The provider blocks in the same file set up the chapter's first asymmetry before any resource exists. The aws provider carries a `default_tags` block, the provider-level half of the tagging model. The google provider has no label equivalent at that level, an asymmetry the tagging section pays for in full.

27.1. cost models

Both clouds bill compute by the second, and both pages say so within one sentence of each other. Amazon's on-demand page: "On-Demand Instances let you pay for compute capacity by the hour or second (minimum of 60 seconds) with no long-term commitments." The partial-hour rule follows: "Each partial instance-hour consumed will be billed per-second for Linux", with Windows, RHEL, and Ubuntu Pro on the same list and SUSE as the one billed by the full hour. Google's compute FAQ states the shape on the other side: "VMs are charged on a per-second basis with a 1 minute minimum." The units rhyme, per second with a floor of about a minute, and the floor is where the billing model stops being a metronome: a cron job that runs for 5 seconds pays for a full minute on both platforms.

The tax is egress. Compute is cheap and getting cheaper, but bytes leaving a building are billed by distance, and both price lists draw the same ladder. On aws the ladder is documented across two pricing pages. Within one zone: "There was no charge for the data transfer between the NAT gateway and the EC2 instance since the traffic stays in the same Availability Zone using private IP addresses." Across zones: "Data sent over VPC peering connections that crosses an Availability Zone within the same AWS Region is charged at $0.01/GB in both "In" and "Out" direction." Out to the internet the rate in the page's own worked example is $0.09 per GB, with one allowance in front of it: "AWS customers receive 100 GB of free data transfer out to the internet free each month, aggregated across all AWS Services and Regions (except China and GovCloud)." The aggregation is its own rule: the rate tiers

“take into account your aggregate usage for Data Transfer Out to the Internet across Amazon EC2”, S3, RDS, and the rest of a long list, so the discount ladder counts the whole account, not one service.

Google’s VPC pricing page draws the same ladder with its own numbers. Same zone, internal addresses, is the same free rung, a row reading “No charge”. One rung up: “Data transfer to a different Google Cloud zone in the same Google Cloud region when using the internal or external IP addresses within the same VPC network” costs $0.01 per GiB, the twin of the aws cross-zone penny. Between regions the page switches to a matrix, “All prices are per GiB in USD”, that runs from $0.02 inside a continent pair to $0.14 for anything into or out of South America. Inbound is free on both sides, the page states “Ingress pricing is still free.”, but with the fine print that matters for anyone who thought a request was free: “Responses to requests count as data transfer out and are charged.”

Budgets are the console-side answer, and both consoles build them the same way: an amount, thresholds against it, and alerts when spend crosses a line. The aws page opens “You can use AWS Budgets to track and take action on your AWS costs and usage.”, and its worked example is the chapter’s model: “Setting a monthly cost budget with a fixed target amount to track all costs associated with your account. You can choose to be alerted for both actual (after accruing) and forecasted (before accruing) spends.” Notifications go out through the same two channels on both platforms, “You can have notifications sent to an Amazon SNS topic, to an email address, or to both.” The gcp page names the same object: “Avoid surprises on your bill by creating Cloud Billing budgets to monitor all of your Google Cloud charges in one place.”, with the amount being a “Specified amount lets you set a fixed budget amount that your actual spend is compared against.” and the thresholds defaulted: “the default alert thresholds are set at 50%, 90%, and 100% of the budget amount, calculated against Actual spend.”

Neither console stops anything. The gcp page says so plainly: “Setting an alerts-only budget doesn’t automatically cap Google Cloud or Google Maps Platform usage or spending.” The aws page’s note is the same warning from the timing side: “You might incur additional costs or usage that exceed your budget notification threshold before AWS Budgets can notify you”, because “AWS Budgets information is updated up to three times a day.” The one set of teeth on either page is the aws budget action, which can, in the page’s example, “automatically apply a custom IAM policy that denies you the ability to provision additional resources within an account”, a budget that escalates into a guardrail, the subject of section 3.

PITFALL: budgets observe, they do not brake

An alert threshold is a thermometer reading. By the time the 90 percent email arrives, the spend it describes already happened, and on aws the reading itself can be a third of a day old. Treat budgets as the feedback loop for forecast review, and put the actual brake in policy: a deny in an scp, an organization policy constraint, or a budget action that applies an IAM deny. Those are the mechanisms that answer no before the dollar is spent.

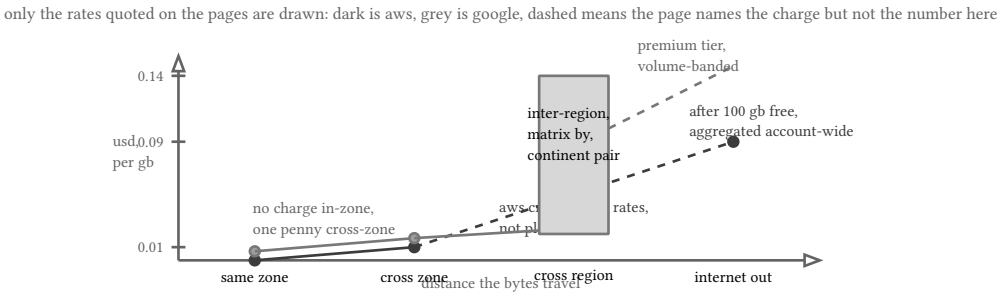


figure 119: the egress ladder as a plot: per-gb price against distance traveled, the two clouds on the same axes

27.2. tagging, a data model with two spellings

The aws definition is one sentence on the tag editor page: “Tags are key and value pairs that act as metadata for organizing your AWS resources.” The work they do is enumeration: “You can create tags to categorize resources by purpose, owner, environment, or other criteria.” The whitepaper makes the structural argument this section is named for. It opens by calling each tag “a simple label consisting of a key and an optional value to store information about the resource or data retained on that resource”, and it dismisses the alternative with “a resource name can

only hold a limited amount of information.” A resource name is one string. A tag set is a schema: typed columns, allowed values, a primary key for the bill. That is why tagging belongs in the configuration language and not in a console afterthought, and why the tag policy later in this section is a schema enforcement tool, not a naming suggestion.

Aws tags are case sensitive on both halves, “Tag keys are case sensitive.” and “Like tag keys, tag values are case sensitive.”, and the page bars the obvious abuse: “Do not store personally identifiable information (PII) or other confidential or sensitive information in tags.” because aws itself reads them for billing and administration. Terraform’s aws provider mirrors the schema at two levels. The provider level takes a `default_tags` block, “Configuration block with resource tag settings to apply across all resources handled by this provider”, the injection point for the organization-wide keys. The override rule is stated as a prohibition: “Provider tags can be overridden with new values, but not excluded from specific resources.”, and a resource overrides with its own `tags`, “Map of tags to assign to the resource.”, where “tags with matching keys will overwrite those defined at the provider-level”. The merged result is visible as the computed `tags_all`, “Map of tags assigned to the resource, including those inherited from the provider”. The sample carries the whole mechanism:

```
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      environment = var.environment
      owner       = "cos-resume"
      managed-by  = "terraform"
      cost-center = "ch24"
    }
  }
}
```

listing 290: the provider-level half: default_tags injects four keys into every aws resource this provider touches (source: c-os-cloud/samples/src/Ch27/versions.tf, lines 26-37)

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "cos-ch24-artifacts"

  tags = {
    owner   = "ch24"
    chapter = "24"
  }
}
```

listing 291: the resource-level half: owner overwrites the injected key, chapter exists only on this bucket (source: c-os-cloud/samples/src/Ch27/tags.tf, lines 19-26)

The google spelling is labels, and the rules are stricter because the use is stricter. The overview defines the object: “A label is a key-value pair that you can assign to Google Cloud resources.” The limits read like a column definition: “Each resource can have up to 64 labels.”, “Keys have a minimum length of 1 character and a maximum length of 63 characters, and cannot be empty.”, “Values can be empty, and have a maximum length of 63 characters.”, and “Keys and values can contain only lowercase letters, numeric characters, underscores, and dashes.” Keys are unique per resource, “The key portion of a label must be unique within a single resource.” The payoff is on the bill: “Information about labels is forwarded to the billing system that lets you break down your billed charges by label.” The provider takes them per resource, the bucket page calls `labels` “A map of key/value label pairs to assign to the bucket.”, and there is no provider-level injection to lean on, so the corpus builds its own:

```
locals {
  label_schema = {
    environment = var.environment
    owner       = "cos-resume"
    managed-by  = "terraform"
    cost-center = "ch24"
  }
}
```

listing 292: the injection point this corpus builds itself: one schema, satisfied to the stricter cloud's rules (source: c-os-cloud/samples/src/Ch27/tags.tf, lines 10-17)

```
resource "google_storage_bucket" "artifacts" {
  name          = "cos-ch24-artifacts"
  location      = "US"
  storage_class = "STANDARD"

  labels = local.label_schema
}
```

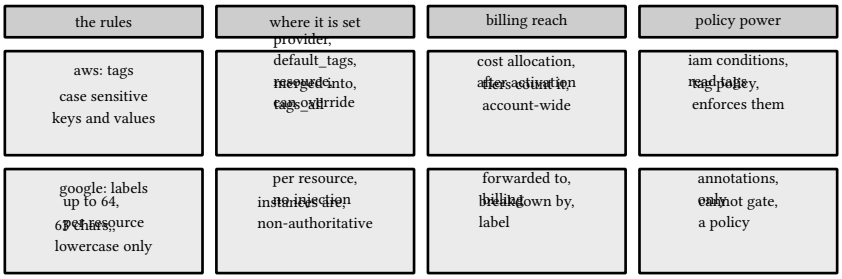
listing 293: the google half: the same four keys arrive as labels on the bucket (source: c-os-cloud/samples/src/Ch27/tags.tf, lines 28-34)

Two label behaviors have no aws counterpart. The instance page warns that its `labels` field "will only manage the labels present in your configuration" and points at `effective_labels` for everything actually on the resource, terraform owns the keys it names and coexists with the rest. And projects do not cascade: "Labels attached to a project don't automatically propagate to child resources within the project." The vocabulary also forks inside google itself, the overview separates the pair: "Labels can be used as queryable annotations for resources, but can't be used to set conditions on policies.", while "Tags provide a way to conditionally allow or deny policies based on whether a resource has a specific tag". Google labels are for questions, google tags are for gates, and the aws word "tag" covers both jobs at once, which is the single largest naming trap in the multi-cloud room.

The aws tag policy is the schema enforcement. The organizations page opens: "Tag policies allow you to standardize the tags attached to the AWS resources in your organization's accounts." The mechanism is declarative: "In a tag policy, you specify tagging rules applicable to resources when they are tagged.", case treatment included. Enforcement is per resource type and opt-in: "A tag policy can also specify that noncompliant tagging operations on specified resource types are enforced. In other words, noncompliant tagging requests on specified resource types are prevented from completing." The blind spot is documented in the same breath: "Untagged resources or tags that aren't defined in the tag policy aren't evaluated for compliance with the tag policy." A tag policy can reject a wrong value for a known key, it cannot notice a resource that carries no keys at all.

PITFALL: write the schema once, to the stricter cloud

A cross-cloud tag schema has to survive both validators, so it inherits google's rules wholesale: lowercase keys and values, 63 characters, underscores and dashes, no spaces, no capitals, even though aws would happily accept all of those. The reverse shortcut, an aws-shaped schema with `CostCenter` and `Application Owner`, bills cleanly on one cloud and silently splits its breakdowns on the other. The intersection is the only stable vocabulary, and it is exactly what `label_schema` in the sample pins down.



google's own tags are the conditional kind, labels cannot gate a policy

figure 120: the tag taxonomy as a matrix: the aws word against the google word across four properties

27.3. guardrails, policy as code

Every guardrail answers one question at one moment. The aws scp is the organization-wide ceiling: "SCPs are a type of organization policy that you can use to manage permissions in your organization." The page spends its most important sentences on what an scp is not: "SCPs do not grant permissions to the IAM users and IAM roles in your organization.", they only cap, "An SCP defines a permission guardrail, or sets limits, on the actions that the IAM users and IAM roles in your organization can perform." The inheritance is strict and one-directional, "Any account has only those permissions permitted by every parent above it.", the cap reaches everyone under it, "SCPs affect all users and roles in attached accounts, including the root user.", and stops at the top: "SCPs don't affect users or roles in the management account."

Google's organization policy is the same instrument pointed at resources instead of principals. The overview grants "centralized and programmatic control over your organization's Google Cloud resources", its stated purpose to "Define and establish guardrails for your development teams to stay within compliance boundaries." The unit is the constraint, "An organization policy configures a single constraint that restricts one or more Google Cloud services.", and the page offers the mental model this chapter keeps: "Think of the constraint as a blueprint that defines what behaviors are controlled." Attachment follows the resource hierarchy, the policy is "set on an organization, folder, or project resource to enforce the constraint on that resource and any child resources", and "all descendants of that resource inherit the organization policy by default". The worked examples are concrete gates, from limiting domain-wide sharing to the placement rule "Restrict the physical location of newly created resources". Both of these mechanisms answer at the api, at deploy time, after a plan has already been approved by whoever is running the apply.

Policy as code moves the question earlier, onto the plan itself. Sentinel is the integrated form: "HCP Terraform checks the Terraform plan against the policy set during each run.", and a failure has consequences proportioned to its level, "failed policies can stop the run. You can override failed policies with the right permissions." The policy reaches everything the run can see: "HCP Terraform provides four imports to define policy rules for the plan, configuration, state, and run associated with a policy check." OPA is the decoupled form, "an open source, general-purpose policy engine that unifies policy enforcement across the stack", with policies in a language of its own, "OPA policies are expressed in a high-level declarative language called Rego." The terraform integration is two commands and a decision: "OPA makes it possible to write policies that check the changes Terraform is about to make before it makes them.", "Use the command terraform show to convert the Terraform plan into JSON so that OPA can read the plan.", then "you hand OPA the policy, the Terraform plan as input, and ask it to evaluate":

```
terraform show -json tfplan.binary > tfplan.json
opa exec --decision terraform/analysis/authz --bundle policy/ tfplan.json
```

fragment, illustrative, not executed

The plan json is the whole interface, and its limits are documented on the same page: “OPA policies operate on the JSON plan generated by Terraform, which has certain limitations.” A policy engine outside the tool can deny a change, comment on it, or score it, and the corpus runs neither command.

The smallest policy-as-code is inside the language itself. A `check` block is “the check block to validate your infrastructure outside of the typical resource lifecycle”, it “executes as the last step of plan or apply operation” and so runs “after Terraform has planned or provisioned your infrastructure”, and its failure mode is a warning: “When a check block’s assertion fails, Terraform reports a warning and continues executing the current operation.”, because “check blocks do not block operations.” The sample’s three checks hold the tag schema from section 2 to the stricter cloud’s rules, key grammar, value length, and override target, so a schema drift shows up as a warning line in every future plan instead of a rejected apply on one provider:

```
check "label_keys_satisfy_stricter_cloud" {
  assert {
    condition = alltrue([
      for key, value in local.label_schema : can(regex("^[a-z][a-z0-9_-]*$", key))
    ])
    error_message = "label keys must start lowercase and contain only lowercase letters, digits,
underscores, dashes."
  }
}
```

listing 294: one check in full: a for expression over the schema, an assert with condition and error_message (source: c-os-cloud/samples/src/Ch27/checks.tf, lines 9-16)

```
check "aws_override_targets_schema_key" {
  assert {
    condition     = contains(keys(local.label_schema), "owner")
    error_message = "the per-resource aws tag override must target a key the shared schema defines, or the
two clouds stop sharing a vocabulary."
  }
}
```

listing 295: the third check: the aws override must target a key the shared schema defines (source: c-os-cloud/samples/src/Ch27/checks.tf, lines 27-32)

The assert halves are fixed by the same page that defines the block, “Input variable validations, preconditions, postconditions, and checks all must include the `error_message` argument.”, a `condition` that returns a boolean and one message for the human reading the warning.

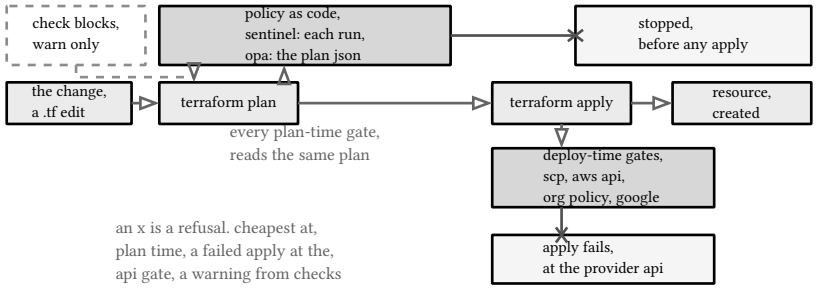


figure 121: one change, three moments of refusal: plan time, apply time, and the api gate

27.4. drift, the plan as detector

Drift is the gap between the record and the world. The terraform tutorial names it from the state’s side: manual changes leave the state “out of sync, or ‘drift,’ from the real infrastructure.” The detector is not a separate tool, it is the plan: “By default, Terraform compares your state file to real infrastructure whenever you invoke” `terraform plan` or `terraform apply`. The same page is candid about the danger of that default, because the diff cuts both

ways: “Terraform will attempt to reconcile your infrastructure, which may unintentionally destroy or recreate resources.” The safe direction exists as a flag: “you can use a `-refresh-only` flag to inspect what the changes to your state file would be”, and the page pins its semantics: “This is a refresh-only plan, so Terraform will not take any actions to undo these.”

Both platforms also keep their own recorders, and neither needs terraform to run. Aws config is the timeline: “AWS Config provides a detailed view of the configuration of AWS resources in your AWS account.”, “how they were configured in the past so that you can see how the configurations and relationships change over time”, with change notification built in: “You can use AWS Config to notify you whenever resources are created, modified, or deleted without having to monitor these changes by polling the calls made to each resource.” Rules add the compliance half, “When AWS Config detects that a resource violates the conditions in one of your rules, AWS Config flags the resource as noncompliant and sends a notification.” Google’s infrastructure manager is the newer, narrower instrument, and its drift page says where the answer comes from: “You can use previews to view resource drift for your deployment.”, because “Infra Manager resource drifts are structured versions of `resource_drift` in the terraform plan JSON representation.” Even the managed service reads drift out of the plan json, the same artifact the opa gate consumed in section 3. The honest diff is the plan, everywhere.

The state itself should live in the cloud it describes, one backend per configuration, and the backend page fixes the rule: “A configuration can only provide one backend block.” The sample ships the aws and google shapes as a pair the reader picks between, the same either/or a real checkout makes:

```
terraform {
  backend "s3" {
    bucket       = "cos-resume-state"
    key          = "ch24/multicloud.tfstate"
    region       = "us-east-1"
    use_lockfile = true
    encrypt      = true
  }
}
```

listing 296: the aws backend: s3 state, native lockfile opted in, encryption on (source: c-os-cloud/samples/src/Ch27/backend-aws.tf, lines 10-18)

```
terraform {
  backend "gcs" {
    bucket = "cos-resume-state"
    prefix = "ch24/multicloud"
  }
}
```

listing 297: the google backend: gcs state under a prefix, locking with no flag to set (source: c-os-cloud/samples/src/Ch27/backend-gcp.tf, lines 9-14)

The s3 page “Stores the state as a given key in a given bucket on Amazon S3.” with locking off by default: `use_lockfile` is “Whether to use a lockfile for locking the state file. Defaults to false.”, “State locking is an opt-in feature of the S3 backend.”, and the old mechanism is on its way out, “DynamoDB-based locking is deprecated and will be removed in a future minor version.” The gcs page “Stores the state as an object in a configurable prefix in a pre-existing bucket on Google Cloud Storage (GCS).”, the bucket “must exist prior to configuring the backend”, and locking is unconditional: “This backend supports state locking.” Both blocks obey the same limitation chapter 24 already quoted, the backend block cannot refer to named values, which is why both bucket names are literals.

WARNING: refresh before you reconcile

A stale plan is worse than no plan. The default compare-and-reconcile will “unintentionally destroy or recreate” against a world it never read. Point `terraform plan -refresh-only` at the drifted workspace first, read

> the diff it prints, and only then decide which side is right: the configuration, the console edit, or a resource that learned something the file does not know yet.

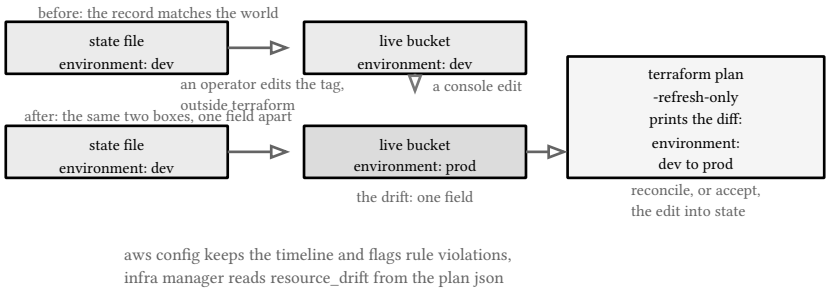


figure 122: drift as a before and after: one recorded field, one console edit, one honest diff

27.5. the provider mapping

The closing view is the lattice the last three chapters built. Every capability row names the aws resource chapter 25 verified at 6.64.0 and the google resource chapter 26 verified at 8.2.0, plus the two backends this chapter pinned, with the scope difference that keeps biting: chapter 25 quoted “A VPC spans all of the Availability Zones in a Region.” while chapter 26 quoted “VPC networks, including their associated routes and firewall rules, are global resources.”, the regional network against the global one, and the subnet rows invert with them. The firewall row carries the attachment flip: the security group rides the instance, while the google rule is a network object whose `network` argument is “The name or `self_link` of the network to attach this firewall to.” The vm row carries the boot polarity, user data once by default against the startup script on every boot. The object store row carries the class model, per-object metadata against the bucket default the sample writes down as `STANDARD`:

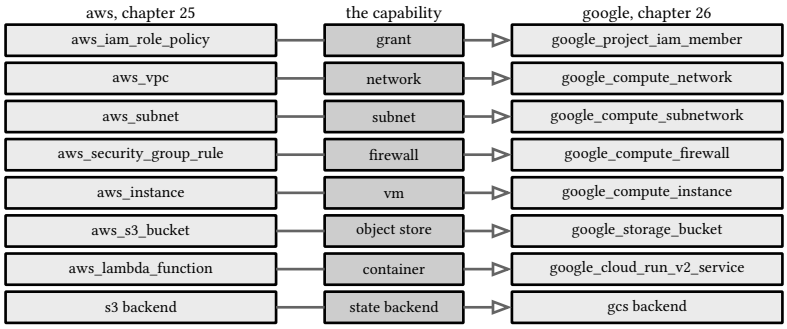


figure 123: the mapping lattice: one capability spine, the aws and google spellings hung off it

The table adds what the lattice cannot hold, the scope note and the behavioral difference each row hides:

capability	aws resource	google resource	the difference
grant	`aws_iam_role_policy`	`google_project_iam_member`	a policy document you write against a role name from the catalogue
network	`aws_vpc`	`google_compute_network`	regional, spans the azs, against global, owns no range
address range	`aws_subnet`	`google_compute_subnetwork`	one per az against one per region
firewall	`aws_security_group_rule`	`google_compute_firewall`	stateful group rides the instance against network rule selecting by tag
virtual machine	`aws_instance`	`google_compute_instance`	user data runs once by default against startup script on every boot
object store	`aws_s3_bucket`	`google_storage_bucket`	class is per-object metadata against the bucket default
container service	`aws_lambda_function`	`google_cloud_run_v2_service`	function against revision, traffic split lives in the template

capability	aws resource	google resource	the difference
state backend	s3 backend	gcs backend	locking opt-in through `use_lockfile` against locking always on

Read down the difference column and the chapter list reappears: the cost ladder sits under the placement choices, the tag schema rides every row, the guardrails gate each column at a different moment, and the plan is the one diff that reads both. Multi-cloud is not 2 clouds, it is one discipline, configuration as the record, policy as the gate, and the plan as the proof, executed twice with different nouns.

sources: aws.amazon.com ec2 on-demand pricing and vpc pricing. docs.aws.amazon.com cost management budgets (budgets-managing-costs), tag editor tagging, the tagging best practices whitepaper (publication date March 30, 2023), organizations tag policies and service control policies, and the aws config guide. docs.cloud.google.com compute faq, billing budgets, resource manager labels overview and label creation, organization policy overview, and infrastructure manager preview results. The vpc pricing page serves from cloud.google.com, which is the cited form. developer.hashicorp.com terraform backend configuration, the s3 and gcs backend pages, the checks page, and the resource drift tutorial. hcp terraform policy enforcement pages for sentinel. openpolicyagent.org docs and the terraform integration page. All accessed 2026-09-12. Provider resource pages at the pinned tags, raw.githubusercontent.com hashicorp/terraform-provider-aws v6.64.0 (index, instance, security group) and hashicorp/terraform-provider- google v8.2.0 (storage bucket, compute instance, compute firewall), accessed 2026-09-12. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied. The version pins are 1.16.2 for the cli, 6.64.0 for hashicorp/aws, and 8.2.0 for hashicorp/google, all Latest at access time.

28. http kernel

Everything else in this service part sits on one small kernel, and this chapter builds it in `c-os-cloud/api/src`: the handler contract every route shares, an HTTP/1.1 parser read straight off RFC 9112, a fixed route table that owns the 404 and 405 split, the one error envelope writer, the one json codec, and a winsock2 listener over a fixed thread pool. The go book built the same spine on `net/http` and `ServeMux` and the standard library. This vehicle has none of that, so the kernel is the standard library: every piece the go lane imported is a file here, and the tests drive all of it without a single socket.

28.1. the handler contract

A handler is one function that takes a request and either commits a response or describes a failure. That is the whole contract, and it is the go lane's error-returning adapter said in c: return true with the response struct filled, or return false with the fail struct filled and let the kernel render it. No route formats a failure body by hand, and no route can answer a 500 with the wrong shape, because the response a handler writes and the envelope the kernel writes both pass through one writer.

The fail struct carries a code from the closed set of 11, an optional status override for the two split codes, a message, and an optional preformatted details array that only `validation_failed` ever carries. The status field exists because the code set is closed while the statuses are not: `invalid_json` rides 400 on a malformed body and 415 on the wrong media type, `not_found` rides 404 on an unknown route and 405 on a method mismatch, and the caller picks the side. Everything else takes the code's default status from the table a later section pins:

```c
// an application failure: a handler fills one and returns false, and
// the kernel renders it as the contract envelope. status 0 means the
// code's default status; the split codes set it explicitly (415 on the
// wrong media type, 405 on method mismatch). details is a preformatted
// json array riding validation_failed only, NULL is never emitted.
typedef struct {
  int code;
  int status;
  const char *message;
  const char *details;
} capi_fail;

// the handler contract: return true with res committed (any status,
// including 204 and 304), or return false with *fail describing the
// failure. req is read-only by convention; only the kernel and the
// middleware write it, before the handler runs.
typedef bool (*capi_handler)(capi_request *req, capi_response *res,
                             capi_fail *fail);
```

listing 298: the contract: a fail value or a committed response, nothing else (source: c-os-cloud/api/api.h, lines 521-539)

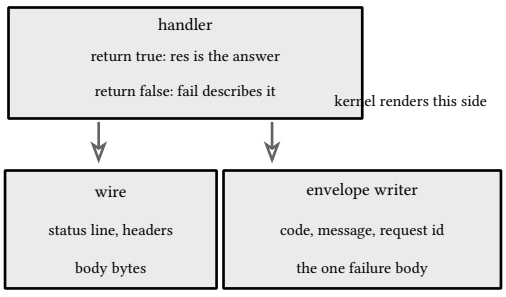


figure 124: the two exits every handler has

28.2. parsing http/1.1 requests

The parser is one pure function over a receive buffer, and RFC 9112 is its contract. The request line is method, target, and the exact string `HTTP/1.1`, single spaces, target in origin form with the query split at the first question mark. Headers are name, colon, value with the surrounding whitespace folded away, at most 32 of them in the vehicle's fixed table. A bare line feed is not a line ending, obs-fold continuations are refused, and every one of those refusals is a plain 400 with an empty body and a closed connection, because a request that never parsed has no route and no request id to envelope with.

The framing decision is where the security story lives. Content-Length and Transfer-Encoding are mutually exclusive, each appears at most once, and Transfer-Encoding must name `chunked` exactly. Every one of those rules closes a request-smuggling shape that RFC 9112 section 6 spells out, and the parser refuses rather than tolerates: a duplicate length field is refused whatever its value, and a body past the 1 MiB cap is the contract's 413. The function returns three answers, a complete request with the bytes it consumed, a request for more bytes, or a refusal, and the consumed count is what makes pipelining work, because the next request on a kept-alive connection starts exactly there:

```c
const char *cl = capi_req_header(req, "Content-Length");
const char *te = capi_req_header(req, "Transfer-Encoding");
int cl_rows = 0, te_rows = 0;
for (int i = 0; i < req->header_count; i++) {
  if (capi_header_eq(req->headers[i].key, "Content-Length"))
    cl_rows++;
  if (capi_header_eq(req->headers[i].key, "Transfer-Encoding"))
    te_rows++;
}
if (cl_rows > 1 || te_rows > 1)
  return CAPI_HTTP_BAD;
if (cl != NULL && te != NULL)
  return CAPI_HTTP_BAD;
if (te != NULL) {
  if (!capi_header_eq(te, "chunked"))
    return CAPI_HTTP_BAD;
  size_t end = 0;
  rc = parse_chunked(req, buf, len, header_end, &end);
  if (rc != 0)
    return rc;
  *consumed = end;
  return 0;
}
```

listing 299: the framing decision: one length, or chunked, never both (source: c-os-cloud/api/src/http_parse.c, lines 240-262)

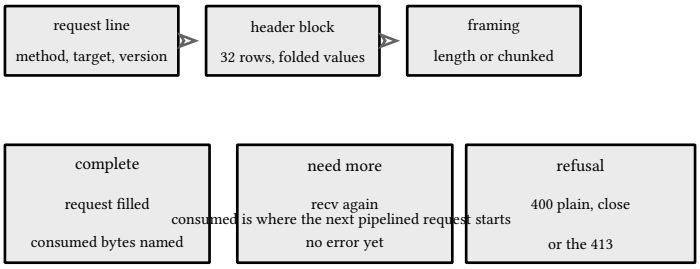


figure 125: one buffer, three answers

28.3. the route table

The route table is a fixed array compiled into the program, and it grows by append only: one row per route, each row a method, a pattern, and a handler. A pattern is literal path segments with a `{name}` segment capturing any single nonempty segment into the request's params, in pattern order. Matching is a walk down both strings one slash-delimited segment at a time, with no allocation anywhere, and a trailing slash is a different path because the walk says so: the segment counts have to agree on both sides.

The 404 and 405 behavior lives here, not in the handlers. A path that matches no row answers 404 with code `not_found`. A path that matches a row whose method is wrong answers 405 carrying the same `not_found` code plus an `Allow` header naming every method that does serve the path, joined in table order. The closed code set has no method-not-allowed member, so the message and the Allow header carry the distinction, the same ruling the go lane froze:

```
struct capi_route {
  const char *method;
  const char *pattern;
  capi_handler handler;
};

// the fixed table. families append rows below their own markers;
// nothing registered moves. the kernel's rows come first because the
// probes answer before anything that needs wiring.
static const struct capi_route routes[] = {
    {"GET", "/healthz", h_healthz},
    {"GET", "/readyz", h_readyz},
    // resource families (ch30-32): users, authn, authz. handlers live
```

listing 300: the fixed table: method, pattern, handler, appended one row per family (source: c-os-cloud/api/src/router.c, lines 47-59)

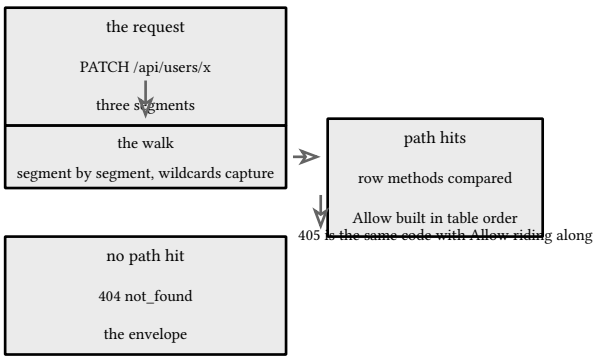


figure 126: the match walk and the two refusals

28.4. the error envelope

Every non-2xx response this api can emit is one byte shape, and one function writes it. The envelope is an `error` object carrying `code`, `message`, an optional `details` array on `validation_failed` only, and `request_id`, in that member order, minified, with the exact bytes the frozen vectors hold. The vectors in `contract/testdata` are the oracle: vector 03's conflict body, vector 05's unauthorized body, vector 12's precondition body, all three transcribed into the kernel tests as string literals and compared byte for byte, so any drift in escaping, member order, or spacing fails a check instead of shipping.

The code set is a table, one row per code with the status it carries everywhere it is used. An unknown code renders as `internal` at 500, the same unknown-failure rule the go kernel carries: a mistake is answered with the envelope shape, never outside it, and the detail stays in the logs. The two split rows say their rule in the comment that pins them, because a split code is where every lane drifts if nothing writes the rule down:

```
// the closed code set with the status each carries wherever it is used.
// the two split rows say so in the comment: invalid_json rides 400 and
// 415 (the caller sets 415 through fail.status), not_found rides 404
// and 405 the same way, and the 405 path adds Allow after this writer
// has run.
static const struct code_row code_rows[] = {
    {CAPI_INVALID_JSON, 400, "invalid_json"},
    {CAPI_VALIDATION_FAILED, 422, "validation_failed"},
    {CAPI_PAYLOAD_TOO_LARGE, 413, "payload_too_large"},
    {CAPI_UNAUTHORIZED, 401, "unauthorized"},
    {CAPI_FORBIDDEN, 403, "forbidden"},
    {CAPI_NOT_FOUND, 404, "not_found"},
    {CAPI_CONFLICT, 409, "conflict"},
    {CAPI_PRECONDITION_FAILED, 412, "precondition_failed"},
    {CAPI_RATE_LIMITED, 429, "rate_limited"},
    {CAPI_OVERLOAD, 503, "overload"},
    {CAPI_INTERNAL, 500, "internal"},
};
```

listing 301: the code table: eleven rows, two of them split, one writer (source: c-os-cloud/api/src/envelope.c, lines 14-31)

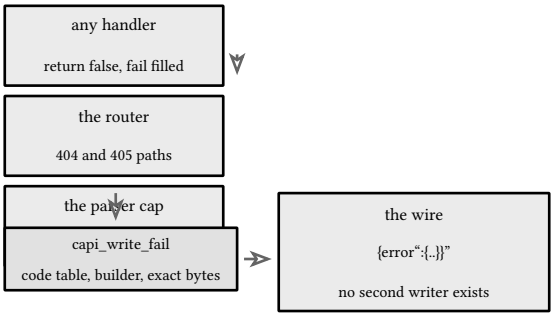


figure 127: one writer, every failure funnels through it

28.5. the json codec

One codec serves the whole vehicle, by ruling, and it is two halves in one file. The writer is a builder over a caller's fixed buffer: it tracks whether each open level is an object or an array and whether the level has written a member yet, so the comma discipline cannot drift and a value without a key refuses instead of producing broken bytes. A refused append latches an oom flag the caller checks once at the end. Strings escape the two mandatory quotes, the backslash, the five short escapes, and `\u00xx` for the remaining control bytes, and every other byte rides verbatim, which is what utf-8 payloads need.

The scanner is a recursive descent over slices of the input. String values keep their escapes until a caller asks for them unescaped, and the unescape pass is where `\uXXXX` decodes, surrogate pairs included, into utf-8 on the way out. Two rulings shape it, both stated in the file: numbers are integers, because every number the contract puts on the wire is an integer, and duplicate member names are an error, the strict reading of RFC 8259. The depth cap and the node pool are fixed budgets, and a document past either refuses, because a bounded dom is the whole point of hand-rolling one:

```
void capi_jw_key(capi_jw *w, const char *k) {
  if (w->depth == 0 || w->kind[w->depth - 1] != 'O') {
    w->oom = true;
    return;
  }
  int lv = w->depth - 1;
  if (w->any[lv])
    jw_put(w, ',');
  w->any[lv] = true;
  jw_str_body(w, k);
  jw_put(w, ':');
  w->pending_key = true;
}
```

listing 302: a key writes the comma it earned, a value slot opens once (source: c-os-cloud/api/src/json.c, lines 112-124)

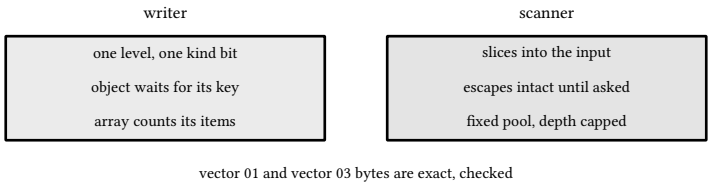


figure 128: the builder tracks level shape, the scanner keeps slices

28.6. sockets and the thread pool

The listener is winsock2 and the decision this section owns is thread per connection from a fixed pool, not an event loop. Every request in this vehicle touches sqlite and the password tier of pbkdf2, work that blocks a thread for milliseconds no matter what the io model says, so a poller would buy nothing the pool does not already give. The chapter 18 event loop stays the right answer for timers and overlapped io the vehicle does not run here. Eight workers, a 64-deep connection queue, one SRWLOCK and one condition variable between them, and a full queue closes the new socket because bounded is the policy, stated plainly.

The listen address comes from `CAPI_ADDR`, defaulting to port 8080, and the read is `GetEnvironmentVariableA` rather than the crt's `getenv` for a stated reason: the ucrt marks `getenv` a deprecated declaration and this lane compiles with deprecations as errors, while kernel32's own window onto the environment is already linked and warns nothing. That is the platform-primitives answer this book prefers anyway, the same reasoning that keeps the wall clock on `GetSystemTimeAsFileTime`.

Each worker owns one context for its whole life, allocated once on the heap because it is megabytes: the request with its 1 MiB body, the response, and the two wire buffers. Nothing in a request's path allocates. The serve loop reads, parses, stamps a fresh request id, dispatches, writes, and either keeps the connection alive with the pipelined tail moved to the front of the buffer or closes it. Every winsock call sits behind the io seam, a pair of function pointers for recv and send, so the entire loop is tested with doubles and no socket exists in any test. The condition-variable handoff is the chapter 16 lesson applied: the pusher wakes one waiter after the release, and the lost-wake trap stays closed because the count is rechecked under the lock:

```
      send(s, (const char *)buf, (int)(len > 0x40000000 ? 0x40000000 : len), 0);
  return n < 0 ? -1 : n;
}

// one queued connection with its peer text, the client fact the accept
// loop captured so the workers never touch winsock for it
struct pool_entry {
  SOCKET s;
  char peer[CAPI_PEER_MAX];
};

struct pool {
  SRWLOCK lock;
  CONDITION_VARIABLE avail;
  struct pool_entry conns[CAPI_ACCEPT_QUEUE];
  int head;
  int count;
  bool stopped;
};

static void pool_push(struct pool *p, SOCKET s, const char *peer) {
```

listing 303: the fixed queue: push under the lock, wake one, refuse when full (source: c-os-cloud/api/src/http_server.c, lines 208-228)

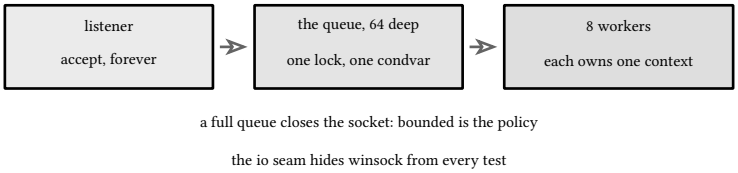


figure 129: accept hands to the queue, a worker takes it

28.7. the composition root

The root is 18 lines and its shape is pinned for the rest of the part. The seams come first, `capi_default_seams` installing the real wall clock, the uuid v7 source, and the rand filler. Then `capi_wire_kernel` fills any seam left null, validates the route table's patterns at startup, and refuses to serve if a row is malformed, because a broken table found at boot is a bug fixed in seconds and the same table found at request time is an outage. Every family lands as one call between the kernel wiring and the listener, in the order the middleware chapter fixes, and `capi_run_tcp` is last so nothing can wire itself after the server starts serving.

The seams themselves are function pointers with a context, the lane's double mechanism, and the real sources are small enough to read whole. The clock reads the system time once. The id source is uuid v7 over an entropy pool seeded under an init-once gate from the performance counter, the wall clock, and the process and thread ids, stepped by an interlocked counter through splitmix64. That is an id source, not a cryptographic one, and the authn chapter's tokens never draw from it:

```c
// the composition root: one linked c23 program that grows one wiring
// line per family, mirroring go's main.go. the prefix is pinned: the
// config story, the kernel wiring, then each family's wiring call in
// the order the middleware chapter fixes, the resource families with
// their guard last among the families, the internal listener, the
// console handler, then the public listener, so a family can never
// wire itself after the server starts serving.
#include "../api.h"
#include <stdio.h>

int main(int argc, char **argv) {
  // the config story: argv names the database, the environment carries
  // the secret, and a missing secret is a loud refusal, never a
  // running service signing tokens with an empty key
  struct capi_ship_config cfg;
  if (capi_ship_config_load(argc, argv, &cfg) != 0)
    return 1;
  capi_server srv = capi_default_seams();
```

listing 304: the root: seams, kernel wiring, families, listener last (source: c-os-cloud/api/src/main.c, lines 1-18)

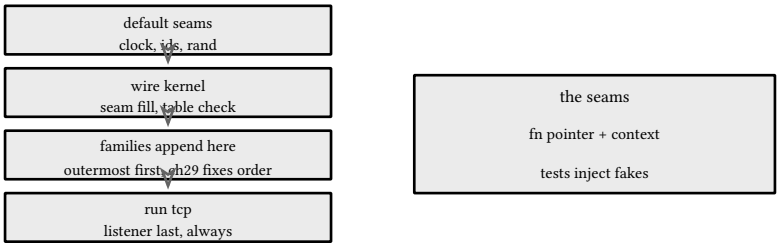


figure 130: the pinned flow, families append in the middle

sources: RFC 9112 sections 2.2, 3, and 6 (request line and header syntax, message body length, the content-length and transfer-encoding rules), RFC 8259 (json grammar, string escapes, the duplicate-name strictness), RFC 9562 section 5.7 (uuid v7 layout), RFC 7230 appendix B (the tchar token alphabet), read 2026-09-27. winsock2 surface verified against learn.microsoft.com windows sockets reference pages for `accept`, `send`, `recv`, and `WSAStartup`, and the `SleepConditionVariableSRW` and `InitOnceExecuteOnce` pages under process and thread functions, all accessed 2026-09-27. Verified by the kernel tests in `c-os-cloud/api/tests/test_kernel.c`, 74 checks over the frozen vector bytes, the parser seeds, and the io doubles, under `make verify-capi`.

29. middleware

Cross-cutting behavior does not belong inside the handlers, and this chapter composes it over the previous chapter's kernel in `c-os-cloud/api/src/middleware.c`: the chain type that folds layers through an explicit continuation, request ids born once and quoted everywhere, the access log, the structured-exception crash guard that stands in for the go lane's recover, timeouts at two layers, and the wiring order that makes the stack correct rather than merely present. Two of the go chapter's lessons do not survive the translation and the prose says so where they fall: a c response is a struct the logger reads, so the status-capturing writer problem is structurally absent, and a thread cannot unwind a running handler, so the deadline is cooperative where the go lane abandoned a goroutine.

29.1. the chain type

One layer is one function that receives the request, the response, a fail slot, and a continuation, and either calls down or answers instead of calling down. The continuation is a two-field struct, the stack and an index, and calling it runs the next layer or, past the end, the route dispatch. That is the whole mechanism, four lines, and it replaces the closure the go lane's `Func` builds with data any reader can follow: the stack is an array of function pointers in wiring order, and the fold is the call stack itself doing the work.

A layer that answers without calling down short-circuits everything below it, the rate limiter's 429 shape, and a layer that wants to observe the answer calls down first and reads the response on the way back out. The pass order the tests pin is the honest spec: outer in, inner in, dispatch, inner out, outer out, and a short circuit leaves the markers below it unwritten:

```
// this one four-line continuation
bool capi_next_call(capi_next next, capi_request *req, capi_response *res,
                    capi_fail *fail) {
  if (next.s == NULL || next.i ≥ next.s->n) {
    capi_dispatch(req, res);
    return true;
  }
  return next.s->layers[next.i](req, res, fail,
                                (capi_next){.s = next.s, .i = next.i + 1});
}
```

listing 305: the fold: the continuation is the stack and an index, dispatch at the bottom (source: c-os-cloud/api/src/middleware.c, lines 18-28)

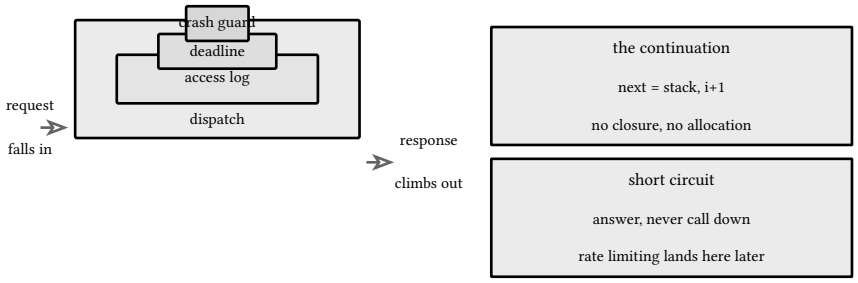


figure 131: one layer, one continuation, the call stack is the onion

29.2. request-id at the edge

Every artifact that talks about a request agrees on one id: the access log line, the crash log line, the error envelope. The kernel's serve loop already stamps a fresh v7 into every request, the edge guarantee that an id always exists, and this layer decides whose id it is. A client `X-Request-ID` that parses as a uuid wins and is adopted verbatim.

Anything else is replaced, never echoed, because the envelope promises a uuid and a free-text header quoted into a log is log injection. The go chapter's three-way order collapses here into two rules said plainly: the edge guarantees existence, the layer decides identity.

The layer also writes the id into the response header, so a client can correlate its own logs with the server's without parsing an error body. The test pins all three births: no header, the stamped id stands and is echoed. A valid client uuid replaces the stamped one everywhere downstream, envelope included. Garbage is replaced and the envelope still quotes the good id:

```
bool capi_mw_request_id(capi_request *req, capi_response *res, capi_fail *fail,
                        capi_next next) {
  const char *client = capi_req_header(req, "X-Request-ID");
  if (client ≠ NULL && capi_uuid_valid(client)) {
    snprintf(req->request_id, sizeof req->request_id, "%s", client);
  }
  bool ok = capi_next_call(next, req, res, fail);
  capi_res_header(res, "X-Request-ID", req->request_id);
  return ok;
```

listing 306: resolve identity at the edge, echo it back, quote it downstream (source: c-os-cloud/api/src/middleware.c, lines 43-51)

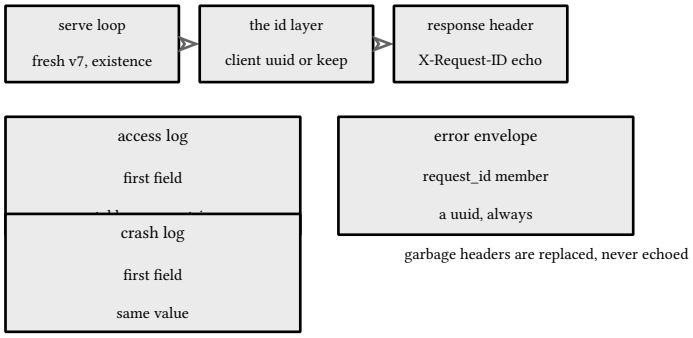


figure 132: one id, born at the edge, quoted by everything downstream

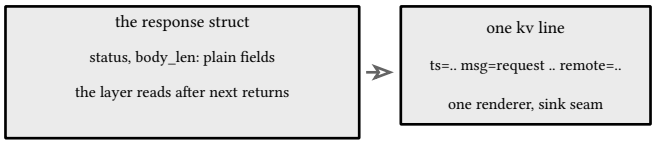
29.3. the access log

The go chapter needed a status-capturing writer because a handler writes to an interface the middleware cannot see through. The c kernel has no such interface: the response is a struct, the middleware holds the same pointer the handler wrote through, and the status, the header table, and the byte count are fields it reads after the stack returns. The double-write trap is structurally absent, the first-wins rule is the only rule because there is only ever one write, and the wrapper type that costs go its optional interfaces does not exist here.

What remains is the line itself and the sink. The line is the observability chapter's kv form, and this layer does not format it: the layer builds the access record, ts, level, msg, the request id, method, path, status, duration, and the kernel's peer stamp in the remote field, then hands the record to that chapter's renderer, the one assembler every log line in the vehicle goes through, so the access line and the book's line shape can never drift apart. A request that arrives without a peer stamp renders `remote=-`, which is the internal listener's honest line. Every field is deterministic at an injected clock, zero elapsed on a clock that never steps. The sink is a function-pointer seam like the clock, so tests capture lines into a buffer and assert bytes, and the wiring installs stderr for production. The renderer bakes the trailing newline, so the sink writes exactly the bytes it was handed. The log lands after the stack returns, which is the whole reason it sits outside the crash guard in the wiring order:

```
// the one assembler rule: the access line renders through the
// observability family's kv renderer, the pinned field set, so the log
// and the book's renderer cannot drift apart. ts is the completion
// instant, the id is the anchor every other artifact quotes, remote is
// the kernel's per-connection peer stamp, and body_len stays off the
// line by design. the response is a struct the layer reads after the
// fact, so the go lane's status-capturing writer problem is
// structurally absent here
bool capi_mw_access_log(capi_request *req, capi_response *res, capi_fail *fail,
                        capi_next next) {
  long long t0 = req->srv != NULL && req->srv->now_ms != NULL
                     ? req->srv->now_ms(req->srv->now_ctx)
                     : 0;
  bool ok = capi_next_call(next, req, res, fail);
  if (req->srv == NULL || req->srv->log_line == NULL)
    return ok;
  long long t1 = req->srv->now_ms(req->srv->now_ctx);
  capi_obs_access a = {
      t1 / 1000, "info",      "request", req->request_id,           req->method,
      req->path, res->status, t1 - t0,   capi_limit_req_remote(req)};
  char line[512];
  size_t n = capi_obs_access_line(&a, line, sizeof line);
  if (n < sizeof line)
    req->srv->log_line(req->srv->log_ctx, line, n);
  return ok;
}
```

listing 307: the response is data: build the access record, render through the one assembler (source: c-os-cloud/api/src/middleware.c, lines 56-81)



the go lane built a statusWriter for this: c never needed one

figure 133: read the struct, write the line, no wrapper exists

29.4. the crash guard

The go lane recovered panics. C has no panics, and the equivalent failure is a hardware or system exception: an access violation, a divide by zero, an aligned read from a misaligned pointer. Windows routes all of them through structured exception handling, and the guard is one `__try` around the continuation with a filter that executes the handler for everything except a stack overflow. The filtered path renders the contract's 500 `internal` envelope through the one writer and the crash line through the observability family's attribute renderer, level error and the exception code as a kv pair, so the process lives and the client still learns nothing it should not.

Two honest limits come with the mechanism. A stack overflow passes the exception on because the guard that tried to answer it would run on the broken stack, so the process dies honestly instead of lying. And the envelope answer after an arbitrary fault is best effort by nature: whatever state the fault left is the state the renderer runs on. The tests raise the exception in software, `RaiseException` with the access violation status, which exercises the exact handler path without tripping the sanitizer legs a real fault would poison:

```
// the recover of this vehicle: an access violation or any other
// fault below this frame becomes the contract's 500 internal envelope
// and the process lives. the detail goes to the log sink, never the
// response, and the answer is best effort by nature: the fault left
// whatever state it left
bool capi_mw_crash_guard(capi_request *req, capi_response *res, capi_fail *fail,
                         capi_next next) {
  bool ok = false;
  __try {
    ok = capi_next_call(next, req, res, fail);
  } __except (crash_filter(GetExceptionInformation())) {
    DWORD code = GetExceptionCode();
    if (req->srv != NULL && req->srv->log_line != NULL) {
      char codev[24];
      snprintf(codev, sizeof codev, "%lu", (unsigned long)code);
      const char *kv[][2] = {{"exception", codev}};
      long long now =
          req->srv->now_ms != NULL ? req->srv->now_ms(req->srv->now_ctx) : 0;
      char line[160];
```

listing 308: seh recover: filter first, envelope and crash log second, best effort stated (source: c-os-cloud/api/src/middleware.c, lines 122-141)

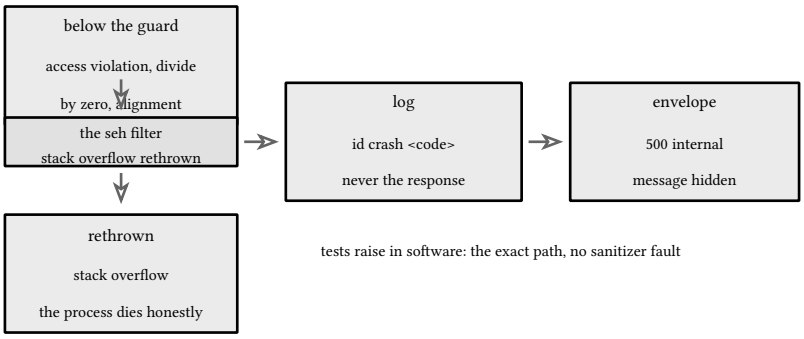


figure 134: fault unwind: one path to the log, one to the envelope, one rethrown

29.5. timeouts at two layers

The transport layer is coarse and belongs to the socket: the pool sets `SO_RCVTIMEO` and `SO_SNDTIMEO` to 30 seconds on every accepted connection, so a stalled peer loses the connection instead of holding a worker forever. Nothing recovers the request. The connection dies, which is the correct outcome for a client that stopped talking, and the worker returns to the pool.

The handler layer is finer and belongs to the stack. The deadline layer stamps `deadline_ms` into the request from the injected clock plus the budget, and that stamp is the contract with every handler that runs a long loop: the reports handler in a later chapter checks it between pages of work, because a thread cannot unwind the code that missed its own deadline. This is the honest difference from the go lane, whose timeout ran the handler on a second goroutine and dropped the whole goroutine on expiry. After the stack returns, the layer refuses to vouch for an answer that arrived past budget and replaces it with the 503 `overload` envelope. The work already ran, stated plainly, and the client still gets the truth:

```c
// the handler layer of the timeout story: stamp the deadline the
// handlers consult on their long loops, then refuse to vouch for an
// answer that arrived past budget. a thread cannot unwind the code
// that missed it, the honest difference from the go lane's goroutine
// abandonment, so the layer answers 503 with the overload code and the
// work still completes or dies on its own
bool capi_mw_deadline(capi_request *req, capi_response *res, capi_fail *fail,
                      capi_next next) {
  if (req->srv == NULL || req->srv->now_ms == NULL ||
      req->srv->handler_budget_ms <= 0) {
    req->deadline_ms = 0;
    return capi_next_call(next, req, res, fail);
  }
  long long now = req->srv->now_ms(req->srv->now_ctx);
  req->deadline_ms = now + req->srv->handler_budget_ms;
  bool ok = capi_next_call(next, req, res, fail);
  long long after = req->srv->now_ms(req->srv->now_ctx);
  if (after > req->deadline_ms) {
    capi_fail late = {.code = CAPI_OVERLOAD,
                      .message = "handler exceeded its deadline"};
    capi_write_fail(res, req, &late);
```

listing 309: stamp the deadline, then refuse to vouch for a late answer (source: c-os-cloud/api/src/middleware.c, lines 85-105)

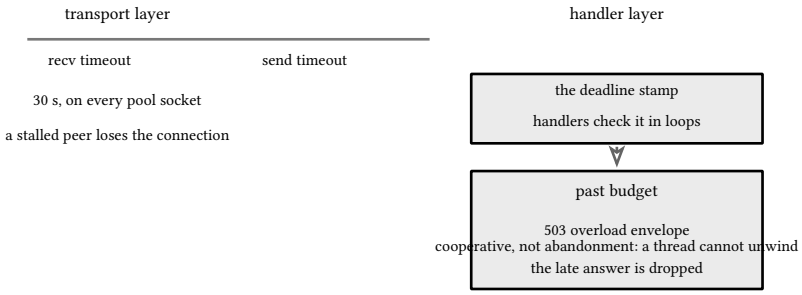


figure 135: two clocks: transport kills the connection, the deadline refuses the answer

29.6. wiring order and why

The order of the ten layers is contract and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log line, the crash line, the envelope. The access log second and outside the guard, so a crashed request is logged as the 500 the client saw and not as nothing at all, the go chapter's wrong order picture with the c mechanism that makes it true. Metrics third and outside the limit, so a refused request still counts. Admission next, refusing overload before any per-route budget is spent, then the limit, then the deadline under both, so a late answer is measured, refused, and logged as the 503 that replaced it.

The three families after the deadline are where this stack departs from the go lane's wire table, and the departure is the platform's fault model arguing back. The conditional read, the idempotency guard, and the cache layer sit above the crash guard, guard innermost, the reverse of the go lane, which composes recover outermost of the four because a go panic runs the deferred releases on its way out. An seh unwind runs nothing: a fault caught below a layer holding a singleflight slot would skip the flight's release and park every later joiner on that key forever. The guard converts a handler fault into a return, and a return unwinds each flight's epilogue normally, so the two slot holders, the cache layer and the idempotency guard, sit above it, and the conditional read sits above both so a 304 short-circuits before the table or the snapshot is asked. The order is not free, it is the price of ruling that a fault must never strand a key, and the wiring comment carries the argument so nobody reorders the array without meeting it.

The wiring file holds the array and two defaults, the log sink and the budget, and it constructs the three families' state, the admission gate, the cache table, the idem flight table, when no earlier family did, every setup idempotent so the composition root's own family order never double-builds any of them. The stack is optional at runtime: the serve loop dispatches directly when no stack is installed, so the order the wiring fixes is documentation the tests enforce rather than a dependency the kernel cannot live without. The stack test fails the moment anyone reorders the array, which is the property the chapter exists to keep:

```
static const capi_mw the_stack[] = {
    capi_mw_request_id,    capi_mw_access_log,  capi_mw_metrics,
    capi_mw_admission,     capi_mw_limit,       capi_mw_deadline,
    capi_mw_preconditions, capi_mw_idempotency, capi_mw_cache,
    capi_mw_crash_guard,
};

void capi_wire_middleware(capi_server *srv) {
  srv->mw = (capi_stack){the_stack, 10};
  if (srv->log_line == NULL)
    srv->log_line = stderr_log;
  if (srv->handler_budget_ms == 0)
    srv->handler_budget_ms = 5000;
  // the stack draws three families' state, so the wiring constructs
  // all of it when no earlier family did: the admission layer's gate,
  // the cache layer's table, the idem layer's flight. every setup is
  // idempotent, so the composition root's own family order never
  // double-builds any of them
  if (capi_load_service()->capacity == 0)
    capi_wire_load();
  capi_wire_cache_layer();
  capi_wire_idem_layer();
}
```

listing 310: the wiring: ten layers in the fixed order, the families' state constructed idempotently (source: c-os-cloud/api/src/wire_middleware.c, lines 34-56)

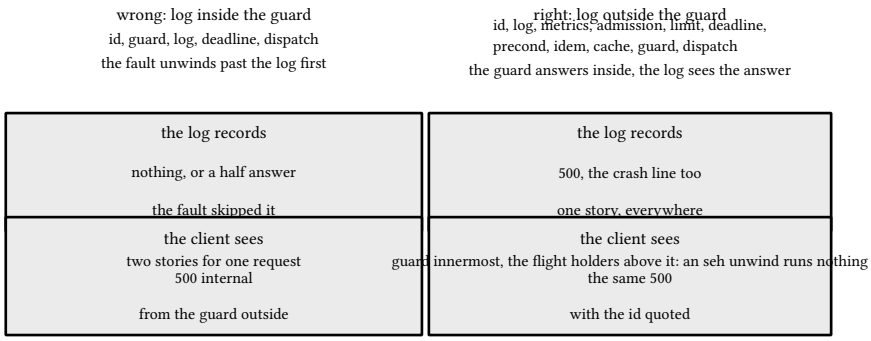


figure 136: wrong order versus right: where the log sits decides what it sees

29.7. testing middleware

Middleware is tested like the kernel, through the same fixtures, and the double mechanism carries all of it: the clock is a counter the tests step by hand, the log sink captures lines into a buffer the checks read back, and the id source is a constant. The fold tests use recorder layers that write markers around the descent and assert the exact sequence. The id tests run the three births. The log test asserts one exact line. The deadline tests step the clock inside and outside the budget. Nothing sleeps anywhere, because nothing needs to: every time-dependent behavior in the stack is arithmetic on the injected counter.

The crash guard is the one place a test cannot just call the code, or so it seems, because the honest way to fault is to fault. The lane's answer is `RaiseException` with the access violation status: the exception travels the same seh path, the filter sees the same code, the handler renders the same envelope, and no sanitizer observes a real fault, so the asan leg stays meaningful. The stack test at the end reads the pass order and the two log lines together, one crash line from the guard and one access line from the logger outside it, which is the wiring order proving itself in output:

```
static bool rec_a(capi_request *r, capi_response *w, capi_fail *f,
                  capi_next next) {
  (void)r;
  (void)w;
  (void)f;
  pass_order[pass_len++] = 'a';
  bool ok = capi_next_call(next, r, w, f);
  pass_order[pass_len++] = 'A';
  return ok;
}
```

listing 311: a recorder layer: markers around the descent, the order is the assertion (source: c-os-cloud/api/tests/test_middleware.c, lines 65-75)

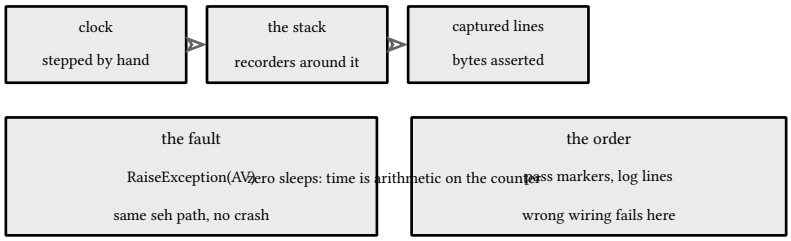


figure 137: every input is injected, every output is captured, no clock exists

sources: learn.microsoft.com structured exception handling reference pages for `__try` and `__except`, `GetExceptionCode`, `GetExceptionInformation`, and `RaiseException`, the windows sockets reference for `setsockopt` with `SO_RCVTIMEO` and `SO_SNDTIMEO`, and the `SleepConditionVariableSRW` page again for the pool handoff, all accessed 2026-09-27. The continuation-passing fold was checked against the go book's chain type in `books/go/api/internal/middleware` as a structural mirror only. Verified by the middleware tests in `c-os-cloud/api/tests/test_middleware.c`, 18 checks over the fold, the id rules, the exact log line, the raised-exception recovery, and both sides of the deadline, under `make verify-capi`.

30. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. This chapter builds the resource's decision core in `c-os-cloud/api/src` as plain functions over plain data: the record's bounds and its two projections, strict decode of everything a client hands back, field validation stated from the contract rather than imported, merge-patch semantics for updates, keyset pagination for lists, the failure ladder that orders refusals, and the golden vector tests that pin the cursor bytes exactly. The kernel owns the wire, the one json codec and the one error writer, so everything here tests without a socket.

30.1. the record and its projection

A resource is three things at once: the stored record, the bytes each reader may see, and the strong etag that names one exact body. The stored record carries the password hash, which never reaches the wire. The full representation carries id, email, display name, roles, creation time, and version. The public projection carries four of those six: email and roles are admin-only data, and a reader who is not the owner and not an admin never learns an account exists beyond its id and name. One function will choose between the two at wiring time, which is the whole trick: a rule enforced in one place cannot be forgotten in a second handler.

The bounds and the role set live in the shared header next to the functions that enforce them, one set of numbers for the handlers, the tests, and this chapter:

```
#define CAPI_PASSWORD_MIN 12
#define CAPI_PASSWORD_MAX 128
#define CAPI_DISPLAY_NAME_MAX 64
#define CAPI_EMAIL_MAX 254
#define CAPI_ROLE_ADMIN "admin"
#define CAPI_ROLE_USER "user"
```

listing 312: the field bounds and the closed role set, the contract's numbers in one place (source: c-os-cloud/api/api.h, lines 123-128)

The version member is the quiet third actor. It bumps exactly once per successful write, and the store's guarded update compares it before committing, which is what turns a stale If-Match into a clean 412 instead of a silent clobber. The etag is not stored anywhere, it is computed: marshal the representation, hash those exact bytes, quote the hex. Because the etag is a function of the body the two can never disagree, and the digest itself is the hand-rolled sha-256 of the next chapter, introduced here as just a function from bytes to 32 bytes.

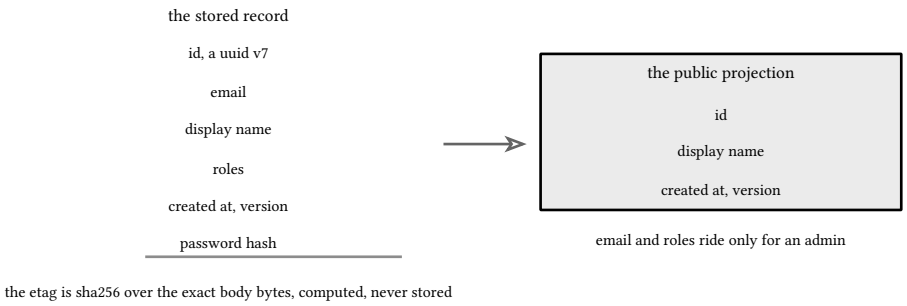


figure 138: one record, two projections, the hash member never on the wire

30.2. strict decode

Input is hostile until proven otherwise. The kernel's decode ladder already answers the media type question, the 1 MiB cap, and the parse itself, all through the one codec. What this family adds is shape strictness: the closed member set for register bodies, enforced at binding, and the exact-shape parse for everything a client hands back that this service issued. The cursor is the teaching example. A cursor is either exactly the bytes this service encoded

or it is garbage, and garbage answers 422, never 404, because a mangled cursor is not a missing page, it is a broken request:

```
  if (outsz < need + 1)
    return 1;
  size_t wrote = capi_b64url_encode(raw, raw_n, out, outsz);
  return wrote == (size_t)-1;
}

static int uuid_shape(const char *s, size_t n) {
  if (n != 36)
    return 0;
  for (size_t i = 0; i < n; i++) {
    char c = s[i];
    int hex = (c ≥ '0' && c ≤ '9') || (c ≥ 'a' && c ≤ 'f');
```

listing 313: the shape check: 36 characters, hex and dashes in their fixed lanes (source: c-os-cloud/api/src/users_cursor.c, lines 103-114)

The decoder around it accepts one spelling, `raw(member)` then digits then the id then the close brace, with the total length checked by arithmetic before any byte is trusted. A timestamp of zero refuses, overlong digit runs refuse, a stray character anywhere refuses. The same discipline governs the patch member set in the walk below: a member the contract does not name refuses the moment it appears. The payoff is symmetrical with the typed bind: a typo in a member name surfaces at the border as a 400 or a 422, never as a silently ignored field corrupting state ten layers in.

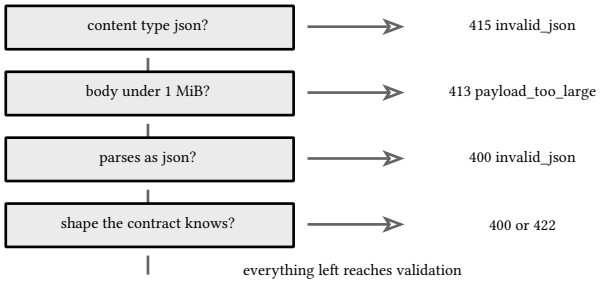


figure 139: the decode ladder, each rung rejects a class of bad input

30.3. validation from first principles

Validation rules come from the contract, not from a library: passwords at least 12 and at most 128 characters, one email shape, display names 1 to 64 characters. Stating them as plain code keeps every number greppable and every message owned by this file. The walk returns one detail per violated rule in field order rather than failing on the first, so the 422 body tells the client everything wrong in one round trip:

```
int capi_validate_register(const char *email, const char *password,
                           const char *display_name, const char **details,
                           int details_cap) {
  int n = 0;
  if (email[0] == '\0')
    n = add_detail(details, details_cap, n, "email is required");
  else if (!capi_valid_email(email))
    n = add_detail(details, details_cap, n, "email is not a valid address");
  if (password[0] == '\0')
    n = add_detail(details, details_cap, n, "password is required");
  else if (strlen(password) < CAPI_PASSWORD_MIN)
    n = add_detail(details, details_cap, n,
                   "password must be at least 12 characters");
  else if (strlen(password) > CAPI_PASSWORD_MAX)
    n = add_detail(details, details_cap, n,
                   "password must be at most 128 characters");
  if (display_name[0] == '\0')
    n = add_detail(details, details_cap, n, "display_name is required");
  else if (strlen(display_name) > CAPI_DISPLAY_NAME_MAX)
    n = add_detail(details, details_cap, n,
                   "display_name must be at most 64 characters");
  return n;
}
```

listing 314: one detail per broken rule, in field order, presence before shape before range (source: c-os-cloud/api/src/users_validate.c, lines 63-85)

Three classes of rule run in that walk. Presence rules catch missing members, and an empty string is a missing member because json has no way to omit what it sent. Shape rules catch an email that parses as nothing, one at sign, a nonempty local part, a dotted domain, and no whitespace anywhere, which approximates the real grammar without rejecting the addresses people actually hold. The 254 character cap folds into that same shape rule rather than standing as its own range detail, the address grammar and its width one rule, the go lane's own fold. Range rules catch lengths, the password and the display name. Email normalization is the fourth piece: the store keys on the trimmed and lower-cased form, so a registration cannot dodge a conflict by varying case.

The 422 and the 409 are different animals and the boundary is worth stating. A rule violation is about the request alone, checkable without touching state, so it is 422 validation. The email conflict is about the store's current contents, so it is 409 conflict, and it surfaces only after validation passed and the create call answered its conflict error. Same request shape, two different truths.

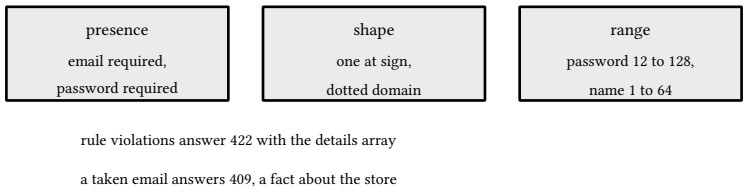


figure 140: three rule classes, one details array, two truths for one request

30.4. merge patch semantics

RFC 7386 merge-patch gives updates three-way semantics a typed struct cannot express: an absent member leaves the field alone, null deletes it, a value replaces it. So the patch decodes into a member list, not a struct, and this walk consumes it in sorted order so details are deterministic. The contract makes one member writable, the display name, and the walk states the full policy in one switch:

```
  for (size_t i = 0; i < n; i++) {
    const capi_patch_member *m = &members[i];
    if (strcmp(m->name, "display_name") == 0) {
      if (m->kind == CAPI_PATCH_NULL)
        refuse(out, "display_name cannot be removed");
      else if (m->kind != CAPI_PATCH_STR)
        refuse(out, "display_name must be a string");
      else if (m->str[0] == '\0')
        refuse(out, "display_name is required");
      else if (strlen(m->str) > CAPI_DISPLAY_NAME_MAX)
        refuse(out, "display_name must be at most 64 characters");
      else
        snprintf(out->display_name, sizeof out->display_name, "%s", m->str);
    } else if (strcmp(m->name, "id") == 0 || strcmp(m->name, "email") == 0 ||
               strcmp(m->name, "created_at") == 0 ||
               strcmp(m->name, "version") == 0 ||
               strcmp(m->name, "roles") == 0) {
      refuse_member(out, "%s is immutable", m->name);
    } else {
      refuse_member(out, "unknown field %s", m->name);
    }
  }
  out->changed = strcmp(out->display_name, current_name) != 0;
}
```

listing 315: the member walk: null on the name refuses, the immutable members refuse by name, the rest validate (source: c-os-cloud/api/src/users_patch.c, lines 32-55)

A null on the display name answers 422, because deleting it would leave the record invalid, so the delete semantics run and the field rules answer. The immutable members, id, email, creation time, version, roles, refuse the moment they appear, null or not: patching them is not a validation typo, it is an attempt to write a field the contract closed. Unknown members refuse the same way the typed bind would. The result carries the resulting name, a changed flag computed against the current value so a no-op patch never bumps a version, and a fixed table of rendered reasons, at most 16 rows of 64 characters, no allocation anywhere in the walk. After the walk the precondition runs at wiring: the If-Match must equal the etag of the current body or the answer is 412, and the store's guarded update compares versions under its own lock so a lost race answers 412 as well.

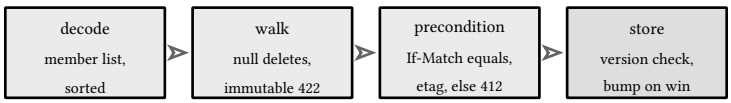


figure 141: a patch walks the members, then the precondition, then the store

30.5. keyset pagination

Offset pagination asks for row 86041 onward and fails twice. The database still walks and discards every skipped row, so page 4000 costs what a scan costs. And offsets are positions in a moving file: one registration between requests and a row appears twice or vanishes. Keyset pagination fixes both by remembering where it stopped rather than counting from the front. The order is the pair, creation time then id, the cursor encodes the last row's position, and the next page is everything strictly after it:

```
  }
  out[o] = '\0';
  *out_n = o;
  return 0;
}

int capi_cursor_encode(const capi_cursor *k, char *out, size_t outsz) {
  char payload[64];
  int n = snprintf(payload, sizeof payload, "{\"t\":%lld,\"i\":\"%s\"}", k->t,
                   k->id);
  if (n < 0 || (size_t)n ≥ sizeof payload)
    return 1;
  unsigned char raw[64];
  size_t raw_n = (size_t)n;
  memcpy(raw, payload, raw_n);
```

listing 316: the cursor payload, two members, base64url without padding over the exact bytes (source: c-os-cloud/api/src/users_cursor.c, lines 87-101)

The payload is deliberately tiny, two members, because the cursor is a position, not a page number. Time is milliseconds since the epoch, the id breaks ties when two users share a creation millisecond, and uuid v7 ids sort by their own embedded time, so the composite order is total. The encoding is base64url without padding, the url-safe alphabet the same codec the next chapter's jwt reuses, opaque to clients and trivially decodable in a test. The compare the store walks is one function, time first then id, strictly after, and the limit is its own small window: absent means 20, digits mean 1 to 100, anything else is a 422 with the reason in the details array.

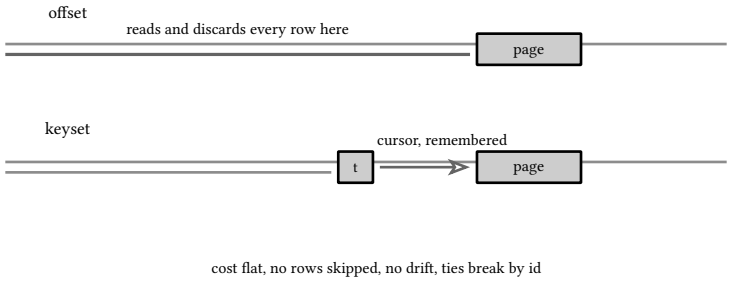


figure 142: offset walks the whole prefix, keyset jumps to the remembered position

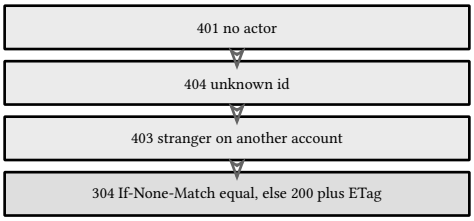
30.6. the failure ladder

Every refusal this family can produce already has a code in the kernel's closed set, so the resource layer states outcomes and the envelope states statuses. The ladder for one object request climbs in a fixed order: 401 without an actor, 404 for an unknown id, 403 for a stranger touching another account, then the etag work, 304 when the client's If-None-Match matches, 200 with a fresh etag otherwise. Patch adds its 412 rungs above the ladder, the missing If-Match and the stale one, and the walk's refusals answer 422 before any of it. The tests pin the refusal classes this core owns, one class per check:

```
                "eyJ0IjoxNzkwODMwODAzMDAwLCJpIjoiMDFhMGY1ZDUtNzgzOC03MTAz"
                "LThhN2YtYmFhOThlMjBmODA4In0") == 0,
      "cursor encodes the frozen vector bytes");
capi_cursor back_k;
CHECK(capi_cursor_decode(cur, &back_k) == 0 && back_k.t == k.t &&
          strcmp(back_k.id, k.id) == 0,
      "cursor round trips");
CHECK(capi_cursor_decode("!!!", &back_k) != 0, "cursor refuses garbage");
CHECK(capi_cursor_decode("", &back_k) != 0, "cursor refuses empty");
char wrongbuf[64];
size_t wrongbuf_n;
CHECK(capi_b64url_decode("Zm9vYmFy", 8, (unsigned char *)wrongbuf,
                         sizeof wrongbuf, &wrongbuf_n) == 0,
      "wrong-json setup decodes");
wrongbuf[wrongbuf_n] = '\0';
CHECK(capi_cursor_decode(wrongbuf, &back_k) != 0,
      "cursor refuses wrong json");
```

listing 317: the refusal classes: null on the name, wrong type, immutable members null or not (source: c-os-cloud/api/tests/test_users.c, lines 202-218)

The order inside the ladder is a policy choice this chapter owns. The 404 answers before the 403, so an unknown id and a stranger's id are distinguishable, the trade the next chapter's audit walks explicitly. What the contract forbids is a third path: no route here invents a status, a message family, or an envelope shape, because every failure renders through the one writer the kernel owns. The register route adds the one 409 this family can reach, the taken email, and only after validation passed, the boundary the validation section stated.



patch adds its 422 and 412 rungs above the ladder

figure 143: one request climbs the ladder, each rung one contract code

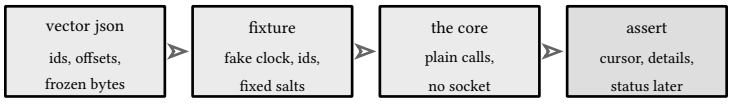
30.7. golden vector tests

The frozen vectors under `contract/testdata` are the fixture source of truth, and the tests read them as data: ids, timestamps, cursor bytes all come from the files, so a test never re-derives a constant the contract already froze. The cursor walk is the shape of the whole discipline, one check asserts the exact 83 characters the encoder produces for the vector's timestamp and id pair, and the frozen file is the only place those characters live:

```
// the cursor against vector 13's frozen bytes
capi_cursor k = {1790830803000LL, "01a0f5d5-7838-7103-8a7f-baa98e20f808"};
char cur[CAPI_CURSOR_TEXT_MAX];
CHECK(capi_cursor_encode(&k, cur, sizeof cur) == 0 &&
          strcmp(cur,
                 "eyJ0IjoxNzkwODMwODAzMDAwLCJpIjoiMDFhMGY1ZDUtNzgzOC03MTAz"
                 "LThhN2YtYmFhOThlMjBmODA4In0") == 0,
      "cursor encodes the frozen vector bytes");
capi_cursor back_k;
CHECK(capi_cursor_decode(cur, &back_k) == 0 && back_k.t == k.t &&
          strcmp(back_k.id, k.id) == 0,
      "cursor round trips");
CHECK(capi_cursor_decode("!!!", &back_k) != 0, "cursor refuses garbage");
CHECK(capi_cursor_decode("", &back_k) != 0, "cursor refuses empty");
char wrongbuf[64];
size_t wrongbuf_n;
CHECK(capi_b64url_decode("Zm9vYmFy", 8, (unsigned char *)wrongbuf,
                         sizeof wrongbuf, &wrongbuf_n) == 0,
      "wrong-json setup decodes");
wrongbuf[wrongbuf_n] = '\0';
CHECK(capi_cursor_decode(wrongbuf, &back_k) != 0,
      "cursor refuses wrong json");
capi_cursor zero = {0, "01a0f5d5-7838-7103-8a7f-baa98e20f808"};
char zbuf[CAPI_CURSOR_TEXT_MAX];
CHECK(capi_cursor_encode(&zero, zbuf, sizeof zbuf) == 0 &&
          capi_cursor_decode(zbuf, &back_k) != 0,
```

listing 318: the vector cursor: encode exact, round trip, refuse garbage and empty (source: c-os-cloud/api/tests/test_users.c, lines 197-222)

Exactness is the point. The etag assertion in the wiring layer proves the body bytes byte for byte, because the etag is the sha-256 of those bytes, and the cursor assertion proves the encoding down to the missing padding. Everything time-dependent enters through seams, a fake clock stepped by hand, an id source that hands out the vector's uuids, so nothing sleeps and nothing dials. The same discipline covers the rest of the slice: register bounds and field order, the email shape classes, normalization, the closed role set, the limit window, the keyset compare, and every patch refusal class. When a vector changes the code is wrong. When the code changes an output byte a vector fails. Drift has nowhere to hide.



drift means the code is wrong, never the vector

figure 144: a golden test: the vector file drives the fixture and judges the result

sources: RFC 7386 for merge patch semantics and RFC 9110 sections 8.8.3 and 13.1.1 for strong etags and conditional requests, both accessed 2026-09-27, RFC 4648 section 5 for the base64url alphabet and its test vectors, and the frozen vectors at `c-os-cloud/api/contract/testdata/` as the byte oracle for the cursor and the register projections. Verified by the users family's 102 checks under the pinned clang 23 through `make verify-capi`, plus clang-format over every file touched.

31. authn

This book has no crypto chapter, so this one is the home for the primitives the service needs, built from their specifications rather than imported: sha-256 from FIPS 180-4, hmac from RFC 2104, pbkdf2 from RFC 8018, all verified against the published test vectors. On those primitives the chapter builds password storage, login with flattened timing, server-side sessions with rotating refresh tokens, the cookie that carries them, and the hs256 jwt, byte-exact against the frozen vectors. Nothing here dials, nothing sleeps, and every random byte enters through the rand seam so the tests stay deterministic.

31.1. sha-256 from the spec

Sha-256 is a compression function over 64-byte blocks chained through eight 32-bit words, plus a padding rule that makes the message length a multiple of the block size. The initial words and the round constants are the first 32 bits of the fractional parts of the square and cube roots of the first primes, tabulated once. The message schedule expands each block's 16 words to 64 with two rotation mixtures, and the round mixes with a choice function, a majority function, and two more rotations:

```
static void sha256_block(capi_sha256 *st, const unsigned char *p) {
  uint32_t w[64];
  for (int i = 0; i < 16; i++)
    w[i] = (uint32_t)p[4 * i] << 24 | (uint32_t)p[4 * i + 1] << 16 |
           (uint32_t)p[4 * i + 2] << 8 | p[4 * i + 3];
  for (int i = 16; i < 64; i++) {
    uint32_t s0 = (w[i - 15] >> 7 | w[i - 15] << 25) ^
                  (w[i - 15] >> 18 | w[i - 15] << 14) ^ (w[i - 15] >> 3);
    uint32_t s1 = (w[i - 2] >> 17 | w[i - 2] << 15) ^
                  (w[i - 2] >> 19 | w[i - 2] << 13) ^ (w[i - 2] >> 10);
    w[i] = w[i - 16] + s0 + w[i - 7] + s1;
  }
  uint32_t a = st->h[0], b = st->h[1], c = st->h[2], d = st->h[3];
  uint32_t e = st->h[4], f = st->h[5], g = st->h[6], h = st->h[7];
```

listing 319: the message schedule: 16 words expanded to 64 by two rotation mixtures (source: c-os-cloud/api/src/authn_sha.c, lines 36-49)

The streaming form carries the eight words, the byte count, and a partial block, so a caller can feed a body in chunks and finish once. The padding appends one set bit, zeros to the 56-byte mark, then the 64-bit big endian bit count, which is why the length field caps the message at just under 2 exabytes and why the final function is a one-way door. The proof the implementation is right is not this prose, it is the RFC 6234 test set: the empty string, one letter, the two-block message, and the stream form split across a block seam all hash to the published constants, checked one by one in the family's tests.

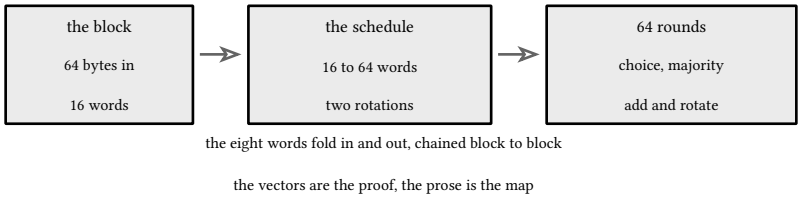


figure 145: one block through the digest: expand, mix, fold back into the eight words

31.2. hmac and pbkdf2

Hashing a key together with a message by concatenation is broken by length-extension: sha256(key + message) lets anyone who knows one output append more message and compute the new output without the key. Hmac

fixes this with two padded passes, an inner hash of the message under the key xor'd with one pad and an outer hash of that digest under the key xor'd with another pad. A key longer than the block is hashed first, a shorter one is zero-padded:

```
void capi_hmac_sha256(const void *key, size_t key_n, const void *msg,
                      size_t msg_n, unsigned char out[32]) {
  unsigned char block[64];
  memset(block, 0, sizeof block);
  if (key_n > 64)
    capi_sha256_buf(key, key_n, block);
  else
    memcpy(block, key, key_n);

  unsigned char ipad[64], opad[64], inner[32];
  for (int i = 0; i < 64; i++) {
    ipad[i] = block[i] ^ 0x36;
    opad[i] = block[i] ^ 0x5c;
  }
  capi_sha256 c;
  capi_sha256_init(&c);
  capi_sha256_update(&c, ipad, 64);
  capi_sha256_update(&c, msg, msg_n);
  capi_sha256_final(&c, inner);
  capi_sha256_init(&c);
  capi_sha256_update(&c, opad, 64);
  capi_sha256_update(&c, inner, 32);
  capi_sha256_final(&c, out);
}
```

listing 320: hmac-sha256: two padded passes over the shared digest, keys hashed or padded to one block (source: c-os-cloud/api/src/authn_hmac.c, lines 9-32)

Pbkdf2 turns the hmac into a password proof by iterating it, chaining xor'd outputs, with the salt and a big-endian block index as the first message. Each 32-byte block of output costs the iteration count times one hmac, so work is the product of what the honest issuer chose and what an attacker must repeat per guess. The RFC 7914 vectors pin the construction at one pass and at 80000, and the family's tests run both. The iteration count is a parameter everywhere it appears, never a hidden default, which is what lets the password tier below carry its own number in the stored string.

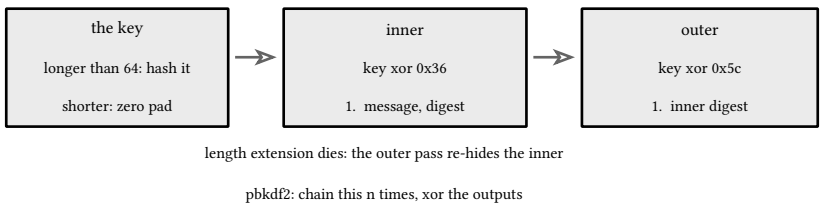


figure 146: hmac: the key padded to one block, two xor pads, two digest passes

31.3. password storage with pbkdf2

A stored password must be a proof, not a secret: verification re-derives and compares, nothing decrypts. The proof is pbkdf2-hmac-sha256 at the contract's 210000 passes over a 16-byte per-user salt with a 32-byte key, and the encoded string carries the parameters it was written with, so verification replays the issuer's choice, never today's defaults:

```
int capi_pass_hash(const char *password,
                   const unsigned char salt[CAPI_PBKDF2_SALT], char *out,
                   size_t outsz) {
  unsigned char key[CAPI_PBKDF2_KEY];
  capi_pbkdf2_sha256(password, strlen(password), salt, CAPI_PBKDF2_SALT,
                     CAPI_PBKDF2_ISSUE_ITERATIONS, key, sizeof key);
  char salt_b64[32], key_b64[64];
  capi_b64_encode(salt, CAPI_PBKDF2_SALT, salt_b64, sizeof salt_b64);
  capi_b64_encode(key, sizeof key, key_b64, sizeof key_b64);
  int n = snprintf(out, outsz, "$pbkdf2-sha256$i=%llu,l=%d$%s$%s",
                   CAPI_PBKDF2_ISSUE_ITERATIONS, CAPI_PBKDF2_KEY, salt_b64,
                   key_b64);
  return n < 0 || (size_t)n ≥ outsz;
}

// verification guards: a stolen store row must not force absurd work
// through crafted parameters
#define CAPI_PASS_MAX_ITERATIONS 10000000ULL
```

listing 321: the proof string: algorithm, iterations, length, salt, key, the reader replays all four (source: c-os-cloud/api/src/authn_pass.c, lines 102-119)

The salt kills rainbow tables, because a table built for one salt answers no other row. The iteration count is the cost dial, 210000 at the issue tier and 2000 at the named test tier the suite actually runs, with the issue tier asserted structurally, the string says 210000, rather than burned in every test pass. Verification guards against crafted store rows too: a stolen row cannot force absurd work through an inflated iteration count or key length, both are capped before any derivation runs.

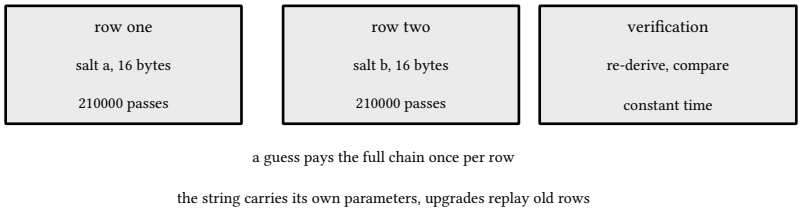


figure 147: one password, two rows, no shared work between attackers

31.4. login and timing flattening

Login answers one question, do these credentials match a row, and two tiers stand in front of the answer. The first tier is emptiness: an empty email or an empty password answers 422 with one body naming both, the go lane's exact tier, so the answer never tells which field alone was wrong. Everything else is a miss. Unknown email and wrong password answer the same 401 with the same bytes, because a different body would let an attacker enumerate accounts by response shape, and an overlong field is a miss too rather than a validation error, deliberately: an email past the row's width finds no row, a password past its cap pays the dummy proof and fails it, so even a field's length cannot be probed through the login route. The timing must flatten the same way: an unknown email still runs a full proof verification against a fixed dummy row, so the fast path of no row found and the slow path of a real mismatch cost the same. The verify function itself answers one boolean for every failure shape, malformed string and wrong password alike, and its final comparison walks every byte regardless:

```
int capi_pass_verify(const char *encoded, const char *password) {
  unsigned long long iterations;
  int key_len;
  unsigned char salt[128], stored[CAPI_PASS_MAX_KEY];
  size_t salt_n, stored_n;
  if (parse_proof(encoded, &iterations, &key_len, salt, &salt_n, stored,
                  &stored_n) != 0)
    return 0;
  if ((size_t)key_len != stored_n)
    return 0;
  unsigned char derived[CAPI_PASS_MAX_KEY];
  capi_pbkdf2_sha256(password, strlen(password), salt, salt_n, iterations,
                     derived, stored_n);
  return const_time_diff(derived, stored, stored_n) == 0;
}
```

listing 322: verify: re-derive under the carried parameters, compare in constant time, one boolean out (source: c-os-cloud/api/src/authn_pass.c, lines 175-190)

The wrong-password check in the tests pins the refusal, and the malformed-string checks pin that a broken row answers false rather than crashing or leaking which part failed. The rate limiter, chapter 36's middleware layer, keys on the lower-cased email, so the flattening is layered: identical bytes, identical timing, and a bounded number of tries per address before the 429 takes over.

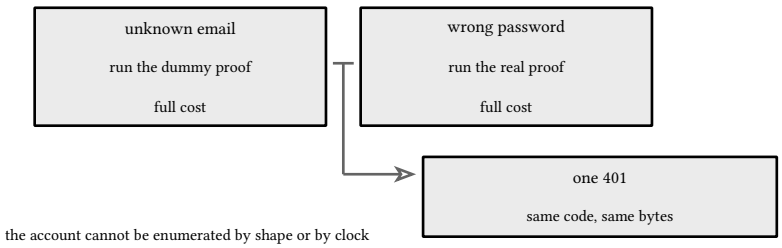


figure 148: two ways to fail, one answer

31.5. server-side sessions

The access token is short-lived and carries no roles, so the session is the real seat of state. Each login mints an opaque refresh token, 32 random bytes from the rand seam, hex on the wire as the cookie value and sha256 hex as the only form the store keeps, so a database leak does not leak live tokens:

```
void capi_refresh_hex(const unsigned char raw[CAPI_REFRESH_BYTES],
                      char out[CAPI_REFRESH_HEX]) {
  static const char digits[] = "0123456789abcdef";
  for (int i = 0; i < CAPI_REFRESH_BYTES; i++) {
    out[2 * i] = digits[raw[i] >> 4];
    out[2 * i + 1] = digits[raw[i] & 15];
  }
  out[64] = '\0';
}

void capi_refresh_hash(const char *hex, char out[CAPI_SHA256_HEX]) {
  capi_sha256_hex(hex, strlen(hex), out);
}
```

listing 323: the token's two forms: hex for the client, the digest of that hex for the store (source: c-os-cloud/api/src/authn_session.c, lines 11-23)

The token is never a jwt, because a refresh token is a lookup key, not a bearer of claims, and making it self-describing would only widen what a leak reveals. Rotation issues fresh material on every use, and the family id is the session id the access token also carries, which is what makes revocation cheap: one session row down and every access token for it dies at its next boundary check, no token blacklist needed.

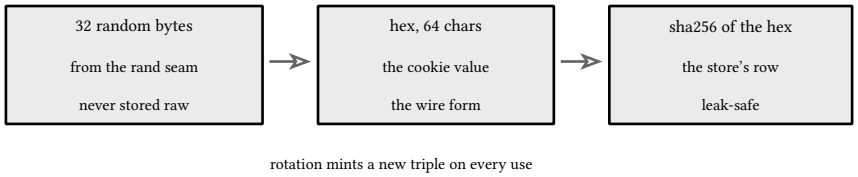


figure 149: the token's two forms and the one the store keeps

31.6. cookie mechanics

The cookie is the browser's side of the session and its attributes are the security posture, in an exact order the vectors carry: http only so script cannot read it, secure so it never rides plain http, same site lax so cross-site posts do not carry it, path scoped to the api so static assets never see it. Logout answers the same attributes plus a zero max-age, which deletes the cookie at the client while the server revokes the session row:

```
int capi_cookie_session(const char *hex, char *out, size_t outsz) {
  int n = snprintf(out, outsz,
                   CAPI_COOKIE_NAME "=%s; Path=/api; HttpOnly; Secure; "
                                    "SameSite=Lax",
                   hex);
  return n < 0 || (size_t)n ≥ outsz;
}

int capi_cookie_clear(char *out, size_t outsz) {
  int n = snprintf(out, outsz,
                   CAPI_COOKIE_NAME "=; Path=/api; HttpOnly; Secure; "
                                    "SameSite=Lax; Max-Age=0");
  return n < 0 || (size_t)n ≥ outsz;
}
```

listing 324: the session cookie and its clearing twin, attributes in the contract's order (source: c-os-cloud/api/src/authn_session.c, lines 25-38)

One identity rides either channel, the cookie or the bearer token, and the authn check accepts both everywhere. That single rule keeps browsers and programmatic clients on one path through every guarded route, and the identical-bytes discipline from login has its echo here: a missing cookie and a revoked session answer the same 401.

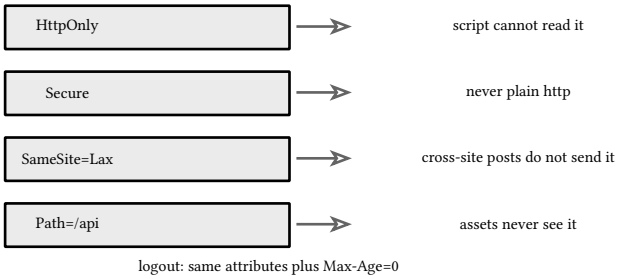


figure 150: the cookie's attributes, each one closing a named hole

31.7. jwt from scratch with hs256

The access token is a jwt with the hs256 algorithm: three base64url segments over a fixed header, a fixed claim order, and an hmac signature. The claim set is exactly what the contract says, issuer, issue time, expiry at issue

plus 900, the session id, and the subject user id, and no roles claim at all, because authorization reads the store-backed truth rather than trusting a token to still be honest about roles:

```
static int parse_claims(const char *payload, capi_jwt_claims *out) {
  const char *p = payload;
  if (expect(&p, "{\"exp\":") || parse_ll(&p, &out->exp))
    return 1;
  if (expect(&p, ",\"iat\":") || parse_ll(&p, &out->iat))
    return 1;
  if (expect(&p, ",\"iss\":") || parse_str(&p, out->iss, sizeof out->iss))
    return 1;
  if (expect(&p, ",\"sid\":") || parse_str(&p, out->sid, sizeof out->sid))
    return 1;
  if (expect(&p, ",\"sub\":") || parse_str(&p, out->sub, sizeof out->sub))
    return 1;
  return *p == '}' && p[1] == '\0' ? 0 : 1;
}
```

listing 325: the claim parser: one fixed member order, no general json anywhere (source: c-os-cloud/api/src/authn_jwt.c, lines 98-112)

The verifier checks the shape first, re-derives the signature over the ascii header and payload segments, and compares in constant time before it parses a single claim, so a forged token never reaches the claim reader. Any deviation refuses: a different header, a fourth segment, a tampered byte anywhere, a wrong secret, an expiry at or before now. The signing side is snprintf over the fixed shape and the family's own base64url codec, and the test asserts the whole token byte for byte against the frozen vector, which only passes because every choice, claim order, header bytes, encoding, matches the oracle.

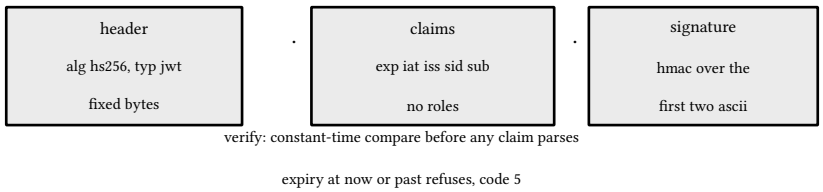


figure 151: the token: three segments, the signature over the first two

31.8. the refresh rotation and reuse detection

Refresh rotation is a theft detector. Every refresh issues new token material and retires the old, and each token remembers the family it belongs to, the session id. A client that holds only the newest token never notices. A client that presents a retired token has, by construction, either a stale tab or a copy that left the legitimate holder's machine, and the contract picks the safe reading: the reuse revokes the whole family, the replay gets a 401, and the rotated token the thief did not use dies with it:

```
  // the stream form must agree with the one-shot across a block seam
  const char *two_blocks =
      "abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq";
  capi_sha256 c;
  capi_sha256_init(&c);
  capi_sha256_update(&c, two_blocks, 33);
  capi_sha256_update(&c, two_blocks + 33, 23);
  unsigned char raw[32];
  capi_sha256_final(&c, raw);
  to_hex(raw, 32, hex);
  CHECK(
      strcmp(
```

listing 326: the token's two forms pinned: 32 bytes, hex on the wire, the digest stored (source: c-os-cloud/api/tests/test_authn.c, lines 213-224)

The vector pair shows the whole story in three requests: the rotate answers 200 with new material, the replay of the superseded token answers 401 with the session revoked, and the follow-up with the rotated token answers the same 401, because the family died. The fixed session table that tracks current and retired hashes is this chapter's own, the decision that a reuse means revocation is stated once where both sides can see it.

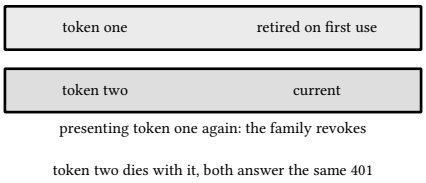


figure 152: rotation as a tripwire: the retired token is the alarm

31.9. testing auth without walls

Every time-dependent input in this chapter is a seam. The clock is a counter the test steps, the id source is a list of the vector's uuids, the rand seam hands back fixed bytes, so the proofs, the tokens, and the cookies are all reproducible byte for byte. The tests run the named 2000-pass tier for the verify legs and assert the issue tier structurally, the encoded string carries 210000, which keeps the suite fast without lying about what production writes. Nothing sleeps, nothing dials, and every refusal path has its own check:

```
int capi_test_authn(const char *api_root) {
  (void)api_root; // the vector files join when the substrate lands

  if (test_session_duels() != 0)
    return 1;

  // sha-256, the RFC 6234 test vectors (inherited from the FIPS
  // 180-2 change notice 1 set)
  char hex[CAPI_SHA256_HEX];
  capi_sha256_hex("", 0, hex);
  CHECK(
      strcmp(
```

listing 327: the vector oracle: sign with the fixed secret, compare the whole token (source: c-os-cloud/api/tests/test_authn.c, lines 175-186)

The discipline is the same one the user resource chapter stated: frozen bytes are the oracle, drift means the code is wrong, and the assertion that looks trivial, comparing one string, is actually checking the header bytes, the claim order, the encoding, and the signature chain all at once. When the wiring layer lands over the kernel exchange,

these same functions answer the same bytes through the full request path, and the vectors judge them again end to end.

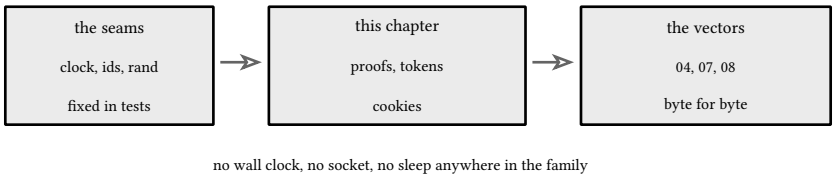


figure 153: the seams: fixed inputs in, reproducible bytes out

sources: FIPS 180-4 for the sha-256 specification and RFC 6234 for its test vectors, RFC 2104 for hmac with the RFC 4231 test vectors, RFC 8018 section 5.2 for pbkdf2 with the RFC 7914 section 11 vectors for the sha-256 variant, RFC 7519 for the jwt claim set, and RFC 6265 for the cookie attributes, all accessed 2026-09-27. The frozen vectors at `c-os-cloud/api/contract/testdata/` are the byte oracle, 04 for the token, 07 and 08 for rotation and reuse. Verified by the authn family's 63 checks under the pinned clang 23 through `make verify-capi`, plus clang-format over every file touched.

32. authz and rbac

Authentication answers who is calling, authorization answers what they may do, and conflating the two is how an api ends up where every logged-in user can touch every row. This chapter builds the authorization layer as one fixed table and one decision function: a row per installed route stating its requirements, a verdict function that walks the caller's facts through the row, the 401 before 403 ordering, object-level checks for routes that name an id, and the never-zero-admins invariant the role endpoints share with the store.

32.1. authentication versus authorization

The previous chapter's output is an actor or its absence: a resolved user id, a session id, and roles read from the store, never trusted from the token. That is the whole input this layer accepts. What it produces is a verdict about the route, the actor, and sometimes the object, and the verdict vocabulary is four values, allow, missing actor, missing standing, no row. The row type is the layer in one screen:

```
typedef struct {
  const char *method;
  const char *pattern; // the router pattern, exact text
  int auth;            // an actor is required
  int admin;           // the admin role is required
  int self_or_admin;   // object routes: the {id} owner or an admin
} capi_policy_row;
```

listing 328: the policy row: a route's exact text and its three requirements (source: c-os-cloud/api/api.h, lines 973-980)

The split keeps each side honest. Because roles ride the actor and not the jwt, granting or revoking a role takes effect at the next request without reissuing anything, which the last section's invariant depends on. Because the policy reads facts rather than headers, the same table answers in a test with three integers as it does in the wired service with a session behind them. And because the layer never parses credentials, a bug here cannot accidentally authenticate anyone, it can only be too strict, which is the failure direction a security layer wants.

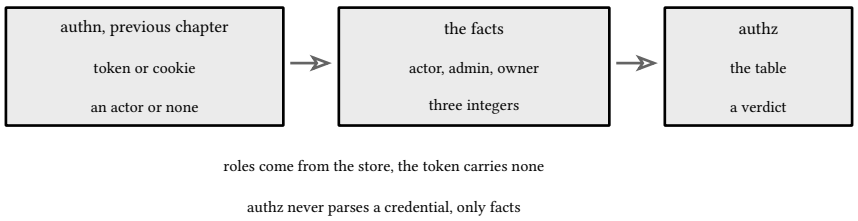


figure 154: the boundary: authn resolves, authz decides

32.2. roles and the grant table

The service knows two roles, admin and user, and the set is closed at the border: a grant or revoke payload carrying anything else answers 422 before the store is touched. The bootstrap rule gives the first registered user the admin role and every later user the plain one, so the system starts with exactly one administrator and grows them only by explicit grants. The closed set is one comparison wide:

```
int capi_validate_role(const char *role, const char **details,
                       int details_cap) {
  if (strcmp(role, CAPI_ROLE_ADMIN) == 0 || strcmp(role, CAPI_ROLE_USER) == 0)
    return 0;
  if (details_cap > 0)
    details[0] = "role must be admin or user";
  return 1;
}
```

listing 329: the closed role set, one refusal message for everything outside it (source: c-os-cloud/api/src/users_validate.c, lines 98-105)

Roles are user data, so the role endpoints are user routes under an admin guard, and the registrar is guarded: the wiring wraps each route with its policy row at registration, so the protection travels with the route and cannot be forgotten by a handler that mounts later. The grant and revoke handlers validate the payload, call the store, and map the store's outcomes, and the one conflict the family can produce is the last-admin rule at the end of this chapter.

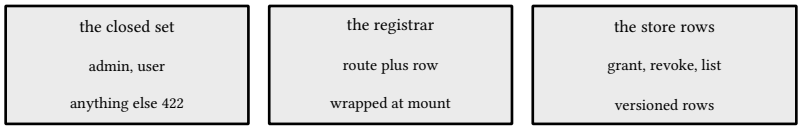


figure 155: the grant table: a closed set, guarded doors, the bootstrap admin

32.3. the fixed policy table

Every installed route has exactly one row, and the row's text is the router's own pattern string, so the table and the route table cannot drift apart silently. The whole policy is visible in one screen, which is the point: a reader can audit who may do what by reading 15 lines, not by tracing middleware chains:

```
static const capi_policy_row POLICY[] = {
    {"GET", "/healthz", 0, 0, 0},
    {"GET", "/readyz", 0, 0, 0},
    {"POST", "/api/users", 0, 0, 0},
    {"POST", "/api/auth/login", 0, 0, 0},
    {"POST", "/api/auth/refresh", 0, 0, 0},
    {"POST", "/api/auth/logout", 1, 0, 0},
    {"GET", "/api/users", 1, 0, 0},
    {"GET", "/api/users/{id}", 1, 0, 1},
    {"PATCH", "/api/users/{id}", 1, 0, 1},
    {"DELETE", "/api/users/{id}", 1, 0, 1},
    {"POST", "/api/admin/users/{id}/roles", 1, 1, 0},
    {"DELETE", "/api/admin/users/{id}/roles/{role}", 1, 1, 0},
    {"POST", "/api/reports", 1, 0, 0},
    {"GET", "/api/reports/{id}", 1, 0, 0},
    {"GET", "/metrics", 0, 0, 0},
};
```

listing 330: the whole policy: one row per route, flags for auth, admin, and object standing (source: c-os-cloud/api/src/authz_policy.c, lines 12-28)

Three flags answer three questions. Does the route need an actor at all, the health checks and login do not. Does it need the admin role, the two role endpoints do. Does it operate on an object the caller might or might not own, the three user-by-id routes do. Field-level authorization rides the same flag at the list route: an admin reader sees email and roles members, everyone else sees the public projection, the rule the user resource chapter placed in

one function. The table is static const, indexed by a linear scan over 15 rows, which at this size beats any hash and allocates nothing.

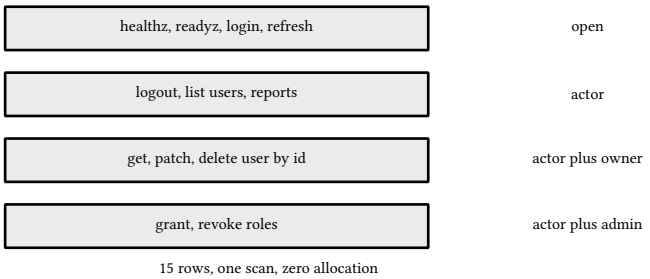


figure 156: the table: rows of routes, flags of requirements, one scan

32.4. 401 versus 403

The two codes say different things and the order they are answered in is a disclosure decision. A 401 means the caller is nobody, no actor resolved, and the fix is credentials. A 403 means the caller is somebody who lacks the standing, and the fix does not exist client side. The rule this layer enforces is that the 401 answers first: an anonymous caller on an admin route learns that credentials are missing, never that the route wants the admin role, because a 403 body to an anonymous caller is a disclosure of the route's protections:

```
capi_authz_verdict capi_policy_decide(const char *method, const char *pattern,
                                      int has_actor, int is_admin,
                                      int is_owner) {
  const capi_policy_row *row = capi_policy_for(method, pattern);
  if (row == NULL)
    return CAPI_AUTHZ_NO_ROW;
  if (row->auth && !has_actor)
    return CAPI_AUTHZ_UNAUTHENTICATED;
  if (row->admin && !is_admin)
    return CAPI_AUTHZ_FORBIDDEN;
  if (row->self_or_admin && !is_owner && !is_admin)
    return CAPI_AUTHZ_FORBIDDEN;
  return CAPI_AUTHZ_ALLOW;
}
```

listing 331: decide: no row, then 401, then the two 403 shapes, then allow (source: c-os-cloud/api/src/authz_policy.c, lines 40-53)

The function is short enough to verify by reading, which is the security argument. Four checks in a fixed order, each returning one of the four verdicts, no state, no side effects, no allocation. The no-row check is first because it is not a verdict about a caller at all, it is a programming error caught at wiring, and it must not shadow behind a 401 that would make a missing row look like a protected route.

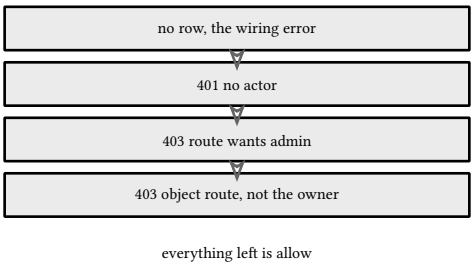


figure 157: the verdict order, each rung one answer

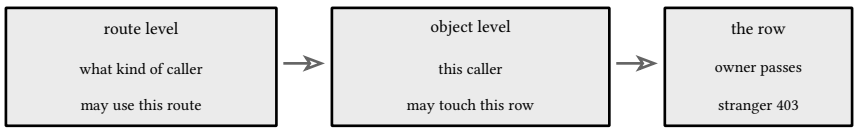
32.5. object-level authorization

Route-level checks answer what kind of caller may use a route. Object-level checks answer whether this caller may touch this row, and it is the layer where idor bugs live: a route guarded perfectly at the route level still leaks if it forgets to compare the id in the path against the id in the actor. The policy carries that requirement as the owner-or-admin flag, and the object routes read it:

```
// object-level: the stranger, the owner, the admin
CHECK(capi_policy_decide("GET", "/api/users/{id}", 1, 0, 0) ==
          CAPI_AUTHZ_FORBIDDEN,
      "a stranger on another account answers 403");
CHECK(capi_policy_decide("GET", "/api/users/{id}", 1, 0, 1) ==
          CAPI_AUTHZ_ALLOW,
      "the owner passes the object route");
CHECK(capi_policy_decide("GET", "/api/users/{id}", 1, 1, 0) ==
          CAPI_AUTHZ_ALLOW,
```

listing 332: the object rule pinned: stranger refused, owner and admin through (source: c-os-cloud/api/tests/test_authz.c, lines 112-120)

The 404 answers before the 403 at the handler, the trade the user resource chapter stated: an unknown id and a stranger's id stay distinguishable, an api that prefers hiding existence answers 404 for both, and the contract picks the distinguishable reading. What no reading permits is the third path, a guarded route with an unguarded object, and the flag makes that impossible to write by accident, because the object routes read their standing from the same row their route guard did.



idor is a route guarded at the first door and open at the second

figure 158: route level versus object level, two doors on one request

32.6. deny by default with a route audit

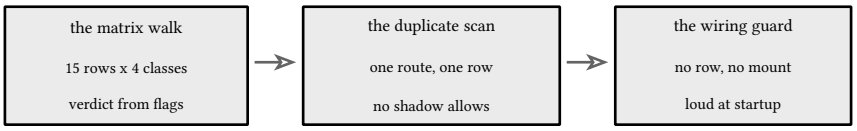
A route absent from the table has no verdict, and the wiring guard turns that absence into a loud failure at startup rather than a silent allow. The audit has two halves. The matrix test walks every row against every actor class and demands the verdict follow from the flags. The duplicate scan demands no two rows name the same route, because a second row for one route is how an accidental allow hides behind a correct one:

```
// the route audit: no duplicate rows, every protected route states
// its requirements, the admin routes are exactly the admin table's
int no_dupes = 1;
for (size_t i = 0; i < count; i++)
  for (size_t j = i + 1; j < count; j++)
    if (strcmp(table[i].method, table[j].method) == 0 &&
        strcmp(table[i].pattern, table[j].pattern) == 0)
      no_dupes = 0;
CHECK(no_dupes, "no two rows name the same route");
```

listing 333: the audit's duplicate scan: two rows for one route is the hiding place (source: c-os-cloud/api/tests/test_authz.c, lines 93-101)

The deny-by-default posture reads off the lookup: a route not in the table is not in the service, and a caller sending its method against its pattern reaches the router's own 404 before any policy question, because there is no handler to guard. What the guard catches is the programmer error in the other direction, a handler mounted without a

row, and it catches it at wiring time, when fixing it costs one line, not after an audit finds the unguarded route in production.



absence denies, duplication hides, both are caught

figure 159: the audit: every row, every actor class, no duplicates

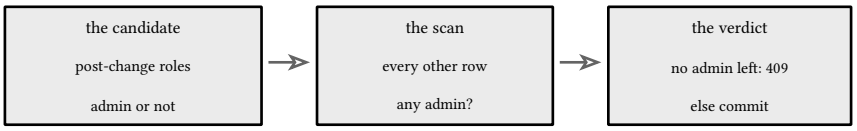
32.7. the last admin rule

An authorization system that can lock out every administrator has a self-destruct button, and the role endpoints hold it. The invariant is never zero admins: a revoke that would leave none standing answers 409, and the grant path runs the same check, the belt and suspenders the contract asks for even though a grant can only add. The rule as this layer states it is pure math, one line over the candidate's post state and the count of everyone else:

```
int capi_admins_remain(int candidate_is_admin, int other_admins) {
  if (candidate_is_admin)
    return 1;
  return other_admins > 0;
}
```

listing 334: the invariant: after the change, does an admin remain (source: c-os-cloud/api/src/authz_policy.c, lines 55-59)

The store enforces it inside the write statement itself, the guard riding the sentence's own predicate: a grant of admin passes because the post state keeps an admin by construction, any other grant passes only where an admin already stands, and a revoke deletes only when the row is not an admin row or another holder survives it, so there is no count-then-write window to race, the engine's write lock serializing two revokers the same way the cas clause serializes two patchers. The verdict then reads the row, not the engine's change counter, because a pooled handle can answer another thread's count: still standing after the statement means the guard refused it and the answer is the 409, gone means the change landed or never existed, and both are the caller's ok. A refused statement touches nothing, so a rejected change leaves no trace. The frozen vector walks the whole story: revoking the admin role from the last holder answers 409 with the last admin message, granting it to a second user succeeds, and the same revoke that refused before now passes, because a spare admin stands. Both endpoints answer with the resulting role list, so the caller sees the state it reached.



grant and revoke both run it, inside the write transaction

figure 160: the invariant over one revoke: post state versus everyone else

32.8. the rbac matrix test

The matrix test is the audit made executable: every row against every actor class, anonymous, plain actor, non-owner on an object route, owner, admin, with the expected verdict derived from the row's own flags. The test never lists expectations by hand, it computes them from the same flags the decision function reads, so a new route added to the table is covered the moment its row lands:

```c
// the matrix walk: anonymous, plain actor, stranger on an object
// route, owner, admin. each row's verdict must follow from its
// flags, nothing else.
int matrix_ok = 1;
for (size_t i = 0; i < count; i++) {
  const capi_policy_row *r = &table[i];
  if (capi_policy_decide(r->method, r->pattern, 0, 0, 0) !=
      (r->auth ? CAPI_AUTHZ_UNAUTHENTICATED : CAPI_AUTHZ_ALLOW))
    matrix_ok = 0;
  if (capi_policy_decide(r->method, r->pattern, 1, 0, 0) !=
      ((r->admin || r->self_or_admin) ? CAPI_AUTHZ_FORBIDDEN
                                      : CAPI_AUTHZ_ALLOW))
    matrix_ok = 0;
  if (capi_policy_decide(r->method, r->pattern, 1, 0, 1) !=
      (r->admin ? CAPI_AUTHZ_FORBIDDEN : CAPI_AUTHZ_ALLOW))
    matrix_ok = 0;
  if (capi_policy_decide(r->method, r->pattern, 1, 1, 0) != CAPI_AUTHZ_ALLOW)
    matrix_ok = 0;
}
CHECK(matrix_ok, "the rbac matrix holds for every row and actor class");
```

listing 335: the matrix walk: expectations derived from the row's flags, not hand-listed (source: c-os-cloud/api/tests/test_authz.c, lines 72-91)

The derivation is the interesting part. A hand-written table of expectations would be a second copy of the policy, and the two copies would drift. Reading the flags keeps one source of truth and turns the test into an invariant check, the verdict function agrees with the row's stated requirements for every combination, rather than a snapshot. The refusal classes the matrix cannot express, the no-row verdicts and the method-keyed lookup, carry their own checks, and the never-zero-admins math is pinned from both directions, the refusing revoke and every passing shape.

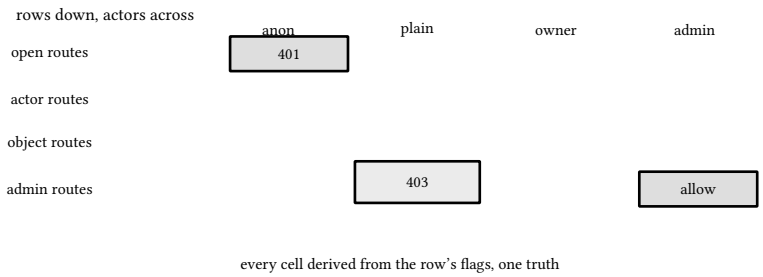


figure 161: the matrix: rows against actor classes, expectations from the flags

sources: RFC 9110 section 15.5.1 with 15.5.4 for the 401 and 403 semantics and their ordering as a disclosure trade, and section 15.5.5 for 404, accessed 2026-09-27, the frozen vector 14 at `c-os-cloud/api/contract/testdata/` for the last-admin walk, 409 then grant then revoke, and the go and c sharp mirrors' authz chapters for the route-audit and matrix-test shapes this lane re-derives. Verified by the authz family's 40 checks under the pinned clang 23 through `make verify-capi`, plus clang-format over every file touched.

33. store

The service has run on fixed tables in memory, which tests love and restarts hate, and the fix is the capstone's own engine: sqlite 3.53.4, loaded at runtime from the dll beside the exe, driven through its c api with nothing between, no driver, no orm, no query builder. Above that seat: a pool, migrations that append, statements bound every time, the transaction shape in plain c, wal isolation, the version guard, keyset pagination as one row comparison, and the window-function report.

33.1. the dll, the c api, and the pool

The load is the row cruncher's idiom restated where the service lives. `LoadLibraryA("sqlite3.dll")` finds the dll beside the exe, every entry point is a typedefed pointer resolved with one direct cast, 19 of them in one extern table, and the version string is compared against 3.53.4 before the table is published, so a stale dll fails the first call loudly. The pool is built from the platform's parts: a fixed set of handles, a stack of free slots, one slim reader-writer lock, one condition variable. Acquire sleeps while the stack is empty, release pushes and wakes, and every sleeper re-checks the predicate inside the loop, the lost-wake lesson carried as the shape itself. Each handle is armed at open with `journal_mode=WAL`, `foreign_keys=ON`, and the 5000 ms busy budget, and close refuses while a slot is out.

```
int capi_pool_acquire(struct capi_pool *p) {
  AcquireSRWLockExclusive(&p->lock);
  while (p->free_count == 0)
    SleepConditionVariableSRW(&p->released, &p->lock, INFINITE, 0);
  int slot = p->free_slots[--p->free_count];
  ReleaseSRWLockExclusive(&p->lock);
  return slot;
}

void capi_pool_release(struct capi_pool *p, int slot) {
  AcquireSRWLockExclusive(&p->lock);
  p->free_slots[p->free_count++] = slot;
  ReleaseSRWLockExclusive(&p->lock);
  WakeConditionVariable(&p->released);
}

int capi_pool_close(struct capi_pool *p) {
  if (!p || p->free_count != p->n)
    return CAPI_E_SQLITE; // a slot is still out: refuse, leak nothing
  int rc = CAPI_OK;
```

listing 336: the handout: sleep while empty, wake on release, re-check the predicate every time (source: c-os-cloud/api/src/store_pool.c, lines 56-75)

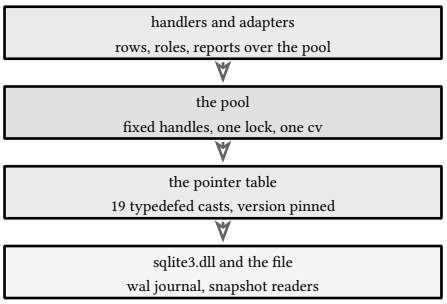


figure 162: the stack the store sits on

33.2. migrations test driven

Schema changes are code changes, so they ship as records, append-only by contract: a shipped version is frozen, a correction is a new numbered version, no record is ever edited. The registry is the `user_version` pragma, one integer the engine keeps in the database header, so nothing the database keeps about itself can drift from the schema it describes. This wave ships three versions, the whole vocabulary, then the user version column with the keyset index, then the idempotency keys the next chapter fills. Each applies inside its own begin immediate group with the bump in the same group, so a failed record leaves nothing durable and can simply be retried. The test re-runs the runner, asks `sqlite_master` for exactly the 7 frozen names, and reopens to prove the registry came back.

```
int capi_store_migrate(sqlite3 *db) {
  long long v = capi_store_version(db);
  if (v < 0)
    return CAPI_E_SQLITE;
  for (; v < CAPI_STORE_SCHEMA_VERSION; v++) {
    if (sq3.exec(db, "BEGIN IMMEDIATE", NULL, NULL, NULL) != 0)
      return CAPI_E_SQLITE;
    char bump[48];
    snprintf(bump, sizeof bump, "PRAGMA user_version = %lld", v + 1);
    if (sq3.exec(db, MIGRATIONS[v], NULL, NULL, NULL) != 0 ||
        sq3.exec(db, bump, NULL, NULL, NULL) != 0) {
      sq3.exec(db, "ROLLBACK", NULL, NULL, NULL);
      return CAPI_E_SQLITE;
    }
    if (sq3.exec(db, "COMMIT", NULL, NULL, NULL) != 0) {
      sq3.exec(db, "ROLLBACK", NULL, NULL, NULL);
      return CAPI_E_SQLITE;
    }
  }
  return CAPI_OK;
}
```

listing 337: one version, one transaction, the ddl and the header bump land together, applied at most once (source: c-os-cloud/api/src/store_schema.c, lines 80-100)

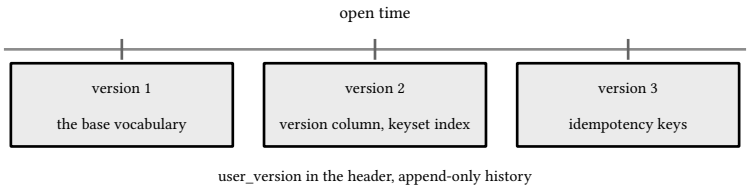


figure 163: the version list only grows right

33.3. prepared statements, bound every time

Parsing sql is work, and a path that re-parses the same statement per call pays it every time. The adapter prepares each statement once per pool slot, on first use, and every call binds every parameter again, steps, and finishes with reset and clear bindings, the pair that leaves the object as prepare left it, so a stale parameter can never survive into the next request. The parameter count is a testable fact, six binds for the insert. The transient sentinel carries the capstone's hard lesson: it tells the engine to copy the bound bytes before the call returns, which is what makes a caller's stack buffer safe to reuse. Values ride ?1 parameters, never string building, so an email is data to the engine, never sql.

```
static sqlite3_stmt *ensure_stmt(struct capi_userstore *u, int id, int slot) {
  if (!u->st[id][slot] && sq3.prepare_v2(u->pool->conn[slot], US_SQL[id], -1,
                                         &u->st[id][slot], NULL) != 0)
    return NULL;
  return u->st[id][slot];
}

static int bind_str(sqlite3_stmt *st, int idx, const char *s) {
  return sq3.bind_text(st, idx, s, (int)strlen(s), TRANSIENT);
```

listing 338: prepare once per slot, bind through one helper that always passes the transient sentinel (source: c-os-cloud/api/src/store_user.c, lines 37-45)

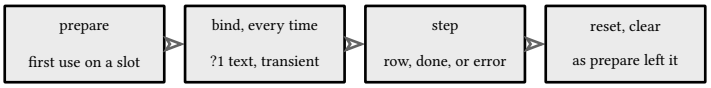


figure 164: one statement's life, per slot

33.4. the transaction idiom

One shape covers every multi-statement write: begin immediate, run the group, commit, and let every other exit roll back. C has no defer and no using block, so the shape is stated in full: the begin takes the write lock up front, the busy budget is spent where the caller chose to write, and each error exit calls rollback beside the check that failed, never in a distant cleanup path, which is the deferred rollback of the go and c sharp idioms restated as control flow readable top to bottom. The suite pins the exits directly: a committed group lands whole, and a group whose second statement violates the email index rolls back its first too.

```
int capi_report_create(sqlite3 *db, const char *id, const char *kind,
                       const char *from_day, const char *to_day,
                       const char *body_sha, long long now_ms) {
  if (sq3.exec(db, "BEGIN IMMEDIATE", NULL, NULL, NULL) != 0)
    return CAPI_E_SQLITE;
  sqlite3_stmt *st = NULL;
  int rc = CAPI_E_SQLITE;
  if (sq3.prepare_v2(db,
                     "INSERT INTO reports"
                     " (id,kind,from_day,to_day,body_sha,status,created_ms)"
                     " VALUES (?1,?2,?3,?4,?5,'pending',?6)",
                     -1, &st, NULL) == 0 &&
      bind_str(st, 1, id) == 0 && bind_str(st, 2, kind) == 0 &&
      bind_str(st, 3, from_day) == 0 && bind_str(st, 4, to_day) == 0 &&
      bind_str(st, 5, body_sha) == 0 && sq3.bind_int64(st, 6, now_ms) == 0 &&
      sq3.step(st) == SQLITE_DONE)
    rc = CAPI_OK;
  if (st)
    sq3.finalize(st);
  if (rc != CAPI_OK || sq3.exec(db, "COMMIT", NULL, NULL, NULL) != 0) {
    sq3.exec(db, "ROLLBACK", NULL, NULL, NULL);
    return CAPI_E_SQLITE;
  }
  return CAPI_OK;
}
```

listing 339: the write shape in plain c: begin immediate, run, commit, rollback on every other exit (source: c-os-cloud/api/src/store_report.c, lines 70-94)

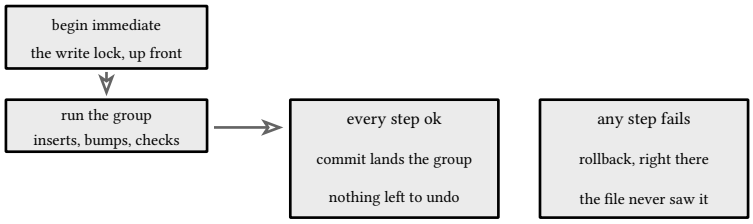


figure 165: a transaction's exits, every one of them clean

33.5. what wal actually gives you

The textbook menu, read uncommitted through serializable, is a menu of guarantees a general engine picks among, and what this engine serves with the wal pragma armed is specific: readers hold snapshots, one writer appends to the log at a time, and the two never block each other. The pragma is persistent in the file, so issuing it per open is idempotent. What that buys is snapshot isolation: no dirty reads, because readers only see committed states, no non-repeatable reads inside one transaction, because the view never moves, one writer at a time, because begin immediate takes the lock first. Write skew stays possible, and that boundary is a fact about the locks. The demo is deterministic and single threaded, and wal is what makes it legal: the reader pins a view, a second handle commits past it, and the reader still reads the old count inside its transaction. No thread, no sleep, the interleave is the statement order:

```
seed(&u, ADA, "ada@example.org", "ada lovelace", T0_MS, &r);
// the reader pins its snapshot at the first read of a deferred
// transaction, and wal is what lets the writer commit while the
// reader holds that view open: no thread, the interleave is the
// statement order.
int rs = capi_pool_acquire(&p);
sqlite3 *reader = capi_pool_conn(&p, rs);
CHECK(sq3.exec(reader, "BEGIN DEFERRED", NULL, NULL, NULL) == 0,
      "the reader opens a deferred transaction");
CHECK(tq_int(reader, "SELECT COUNT(*) FROM users") == 1,
      "the first read pins the view at one user");
seed(&u, GRACE, "grace@example.org", "grace hopper", T0_MS + 3000, &r);
CHECK(tq_int(reader, "SELECT COUNT(*) FROM users") == 1,
      "inside the transaction the view still reads one");
int ws = capi_pool_acquire(&p);
CHECK(tq_int(capi_pool_conn(&p, ws), "SELECT COUNT(*) FROM users") == 2,
      "a fresh read outside sees two");
```

listing 340: pin a view, commit past it, read again: the interleave is the statement order (source: c-os-cloud/api/tests/test_store.c, lines 260-276)

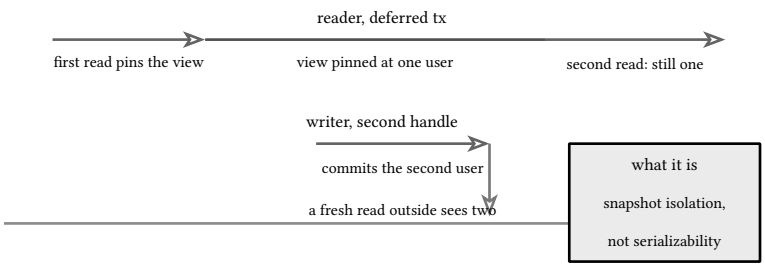


figure 166: one timeline: the reader's view holds

33.6. the lost update and the cas answer

Two writers read the same user, both compute, both write, and one update silently vanishes. The store reproduces it with two blind updates: both succeed, the second name survives, and the version column never moved, so nothing

anywhere records that anything was lost. Serialization does not fix this, because begin immediate guarantees each write lands whole and says nothing about the stale read the write was computed from. The version column is the detector and the guarded update is the fix: the check, the bump, and the write share one statement, so exactly one of two racers holding the same version affects a row. Zero changes is the refusal, and the adapter probes once: a missing row and a stale version are different facts, 404 against 412. Over http this is the precondition the next chapter's etag duel stages, one 200 and one 412, every run:

```
        }
      } else if (sr == SQLITE_CONSTRAINT) {
        rc = CAPI_E_CONFLICT;
      }
    }
    if (rc != CAPI_OK)
      sq3.exec(db, "ROLLBACK", NULL, NULL, NULL);
  }
  if (st)
    done(st);
  capi_pool_release(u->pool, slot);
  return rc;
}

static int fetch(struct capi_userstore *u, int id, const char *key,
                 struct capi_user_row *out) {
  int slot = capi_pool_acquire(u->pool);
  sqlite3_stmt *st = ensure_stmt(u, id, slot);
  int rc = CAPI_E_SQLITE;
  if (st && bind_str(st, 1, key) == 0) {
    if (sq3.step(st) == SQLITE_ROW)
      rc = row_fill(out, st);
    else
```

listing 341: the guard: check, bump, and write in one statement, zero changes is the refusal (source: c-os-cloud/api/src/store_user.c, lines 156-178)

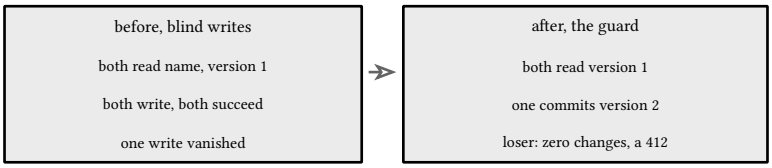


figure 167: same race, two outcomes: silent loss versus a refused writer

33.7. keyset pagination as one row comparison

The list route walks users in creation order, and the cursor the contract froze is the position pair itself, created_at in unix milliseconds with the id as the tiebreaker. Offsets count rows forward on every page and skip or repeat under writes that land between pages. The keyset walk asks one question per page, which rows sort strictly after this position, and in sql that is one row-value comparison, `(created_ms, id) > (?1, ?2)`, matched by the version 2 index in the same order, so the engine never sorts. The first page needs no special statement: the walk's floor is the smallest time with an empty id, so one comparison serves the first page and every continuation with different binds. The limit binds one higher than asked, and the extra row is the has-more fact. The users family's cursor codec renders the pair as the base64url shape the contract froze, and the suite asserts those bytes against vector 13's own field and walks page two onto linus with no cursor left:

```
  }
  if (st)
    done(st);
  capi_pool_release(u->pool, slot);
  return rc;
}

int capi_user_get(struct capi_userstore *u, const char *id,
                  struct capi_user_row *out) {
  return fetch(u, US_BY_ID, id, out);
}

int capi_user_by_email(struct capi_userstore *u, const char *email,
                       struct capi_user_row *out) {
  return fetch(u, US_BY_EMAIL, email, out);
}
```

listing 342: the walk: one row-value comparison, the first page binds the floor, the extra row is has more (source: c-os-cloud/api/src/store_user.c, lines 180-196)

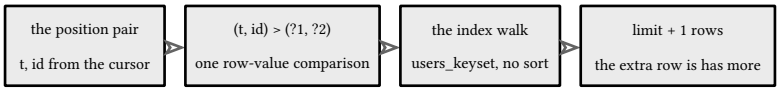


figure 168: one comparison per page, the index already in walk order

33.8. the analytics report in window functions

Reports are sql's home turf, and this lane keeps them there. Presence collapses events to one row per user per day before any counting, because actives count users, not actions, and the events primary key is the pair itself, so a duplicate event is absorbed at write time. The windows do the work: `ROW_NUMBER` partitioned by user in day order marks each user's first day, the newcomer split. `LAG` carries yesterday's count forward with today's as its default, so the first day's delta is zero by construction. `RANK` orders the leaders by days descending then user id ascending, a total order, so a replayed report cannot differ run to run. The report row lands pending, and its first read computes the series and flips the row done in one group, so a replay answers the stored series. The suite seeds the three events vector 16 names and asserts the frozen numbers: one active on the 25th, two on the 26th, one newcomer each day, deltas zero and one, leaders ada then grace.

```
static const char *SERIES_SQL =
    "WITH presence AS ("
    " SELECT day,user_id FROM events WHERE day≥?1 AND day≤?2),"
    "marked AS ("
    " SELECT day,user_id,"
    "  CASE WHEN ROW_NUMBER() OVER"
    "   (PARTITION BY user_id ORDER BY day)=1 THEN 1 ELSE 0 END first_day"
    " FROM presence),"
    "daily AS ("
    " SELECT day,COUNT(*) active,SUM(first_day) newcomers"
    " FROM marked GROUP BY day)"
    "SELECT day,active,newcomers,"
    " active-LAG(active,1,active) OVER (ORDER BY day) delta"
    " FROM daily ORDER BY day";
```

listing 343: the window query: presence, row_number for the newcomer split, lag with today as the default (source: c-os-cloud/api/src/store_report.c, lines 21-34)

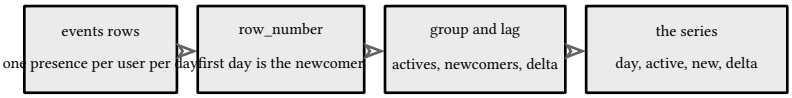


figure 169: events rows to windows to the report body

33.9. store tests without docker

Every test in the family starts the same way: a fresh database file in a temp directory, named by tag, process, and a per-file counter so two test files in one process never collide, opened as a pool, migrated, acted on, closed, and removed with its wal and shm sidecars once the handles die. No container, no port, no shared server, and nothing sleeps: the pool's blocking acquire is closed by a handoff the test releases on purpose, the snapshot interleave is statement order, and the clock never enters the store, because ids, times, and hashes arrive as values from the callers that own them. The suite is 89 checks in two files, and the lane reruns it under the address sanitizer.

```
static inline void tdb_path(const char *tag, char *out, size_t cap) {
  char dir[MAX_PATH];
  GetTempPathA(sizeof dir, dir);
  static int seq;
  snprintf(out, cap, "%scapi-%s-%lu-%d.db", dir, tag, GetCurrentProcessId(),
           ++seq);
}

// the wal sidecar files die with their handles closed; remove all 3.
static inline void tdb_remove(const char *path) {
  char side[300];
  DeleteFileA(path);
  snprintf(side, sizeof side, "%s-wal", path);
  DeleteFileA(side);
  snprintf(side, sizeof side, "%s-shm", path);
  DeleteFileA(side);
}
```

listing 344: the whole test story: a tagged temp file per test, removed with its sidecars once the handles close (source: c-os-cloud/api/tests/tstore.h, lines 26-42)

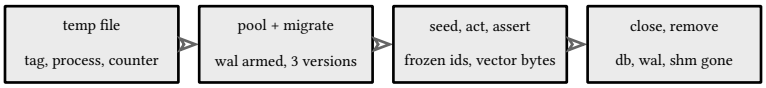


figure 170: a store test's lifecycle, nothing shared between runs

sources: sqlite.org for the c3ref pages sqlite3_prepare_v2, sqlite3_bind_text, sqlite3_step, sqlite3_reset, sqlite3_clear_bindings, sqlite3_changes, sqlite3_busy_timeout, and sqlite3_close, the pragma page for user_version, journal_mode, and foreign_keys, the wal page, and the row-values page, all accessed 2026-09-27, learn.microsoft.com for LoadLibraryA, GetProcAddress, srw locks, and condition variables at the same date, and the transient sentinel contract quoted from the vendored header the capstone pins. Verified by the store family suite, 89 checks across `tests/test_store.c` and `tests/test_store_report.c`, green in `make verify-capi` with the asan leg, the cursor byte-identical to vector 13 and the series numbers to vector 16.

34. concurrency control

The store chapter's guard gave one writer the win and the other a refusal. This chapter moves that fact to the edge and builds around it: the etag preconditions as two pure decisions, the duel two released patchers stage over the guard, idempotency keys with their 24 hour snapshots, the in-flight window closed by a table, singleflight rebuilt from the ch15 primitives over the ch16 atomics, and the bounded fan-out whose first error latches while the queued rest stop early.

34.1. the lost update prevented

Two clients read the same user, both patch, and without protection one write silently vanishes. The stack answers in layers. The decision layer refuses fast: an If-Match that is missing or stale against the current representation answers 412 before the handler burns its decode, and the contract chose 412 over the 428 the RFC suggests, so missing and stale refuse alike. The atomic layer is the store's guarded update from the previous chapter, the check, the bump, and the write in one statement, the only layer nobody can slip between. Fast fail, then the invariant, in that order.

The duel stages the whole chain with no scheduler luck anywhere. Both patchers read the user and hold version 1, both arrive at a gate and park on one event, the test closes the gate once both have arrived, and both race into the same guarded update. Exactly one 200 and one 412 every run, version 2 standing after, because the outcome is a fact about the where clause, not about the interleaving:

```
  struct capi_sflight *g;
  const char *key;
  struct guard_fill_ctx *fill;
  struct capi_idem_row row;
  int replayed;
  int rc;
};

static int guard_thread(void *arg) {
  struct guard_arg *a = arg;
  a->rc =
      capi_idem_guard(a->db, a->g, "POST /api/users|anon", a->key, "sha-g",
                      5000, guard_parked_fill, a->fill, &a->row, &a->replayed);
```

listing 345: the duel: read version 1, park at the gate, race the guard, one winner (source: c-os-cloud/api/tests/test_conc.c, lines 317-329)

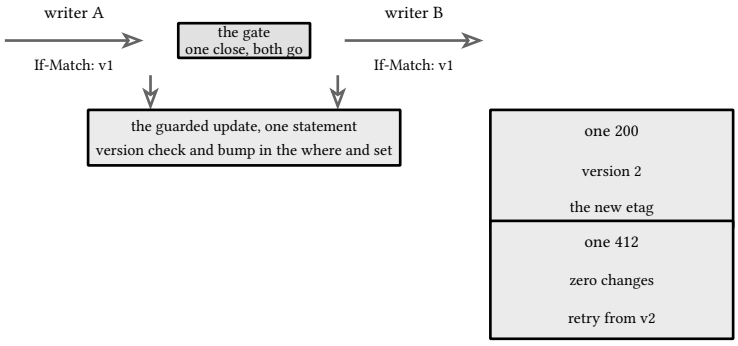


figure 171: two patchers, one gate, one winner

34.2. if-none-match and 304

The read side of the same coin costs nothing when nothing changed. A GET carrying an If-None-Match that matches the current representation answers 304 Not Modified with an empty body, and the client, which already

holds the body the etag names, keeps it. The decision is four facts in order: no header means an ordinary read, no current representation means an ordinary read because there is nothing to compare, the star matches any current representation, and only the exact quoted string answers 304. The strong etag this service issues is the quoted sha256 of the exact response body, compared whole and never parsed, and a weak validator never appears on these representations, so a w/ prefix is simply a non-match. One composition cost rides the placement: the conditional read answers before any authorization walk, this lane keeping the verdict inside the handler and the go lane composing its wrapper outside the authz guard the same way, so a caller who cannot read the row but once held its etag still learns whether the representation changed, a 304 confirming their copy is current and a 200 revealing it moved. The leak is a version oracle over a copy the caller held at some point, never a body disclosure, and closing it would mean checking standing before the revalidation, a deliberate divergence this lane records rather than takes.

```
int capi_if_match_check(const char *header, const char *current_etag) {
  if (!header || !*header)
    return CAPI_IF_MATCH_OK; // missing: the route layer owns the 412
  if (strcmp(header, "*") == 0)
    return CAPI_IF_MATCH_OK;
  if (!current_etag)
    return CAPI_IF_MATCH_REFUSE; // "*" aside, no representation, no match
  return strcmp(header, current_etag) == 0 ? CAPI_IF_MATCH_OK
                                           : CAPI_IF_MATCH_REFUSE;
}

int capi_none_match_hit(const char *header, const char *current_etag) {
  if (!header || !*header || !current_etag)
    return 0;
  if (strcmp(header, "*") == 0)
    return 1;
  return strcmp(header, current_etag) == 0;
}
```

listing 346: the two decisions: whole-string compares, one wildcard, no parser (source: c-os-cloud/api/src/idem_cond.c, lines 11-28)

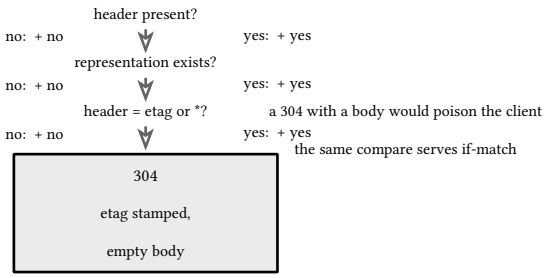


figure 172: the conditional read decision

34.3. idempotency keys and the 24 hour snapshot

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the Idempotency-Key header, scoped by route and caller identity so two clients may share a key freely, with the exact request bytes' sha256 stored beside it, because whitespace drift is a different request. The first successful run becomes a snapshot row, status, content type, body, stamped with a 24 hour expiry, and a later request with the same key and hash replays those bytes at the stored status, the retried register returning its original 201 byte for byte.

Two deliberate refusals shape the table. The same key under a different hash is a mismatch the caller answers as the 422, the client is misusing the contract, and the stored snapshot is untouched. And save refuses any status of 400 or more, because a retry after a validation error deserves a fresh run, not a replayed failure, so only outcomes

under 400 ever become snapshots. Expiry is lazy on read: a row past its 24 hours answers miss and deletes itself on contact, so correctness never waits for a sweep, and the delete carries the expiry in its own predicate so a save that refreshes the row between the read and the delete survives, the same one-statement serialization the store's cas update owns.

```
int capi_idem_save(sqlite3 *db, const struct capi_idem_row *row) {
  if (row->status ≥ 400)
    return CAPI_E_RANGE; // failing runs are never memoized
  sqlite3_stmt *st = NULL;
  if (sq3.prepare_v2(db,
                     "INSERT OR REPLACE INTO idem_keys"
                     " (scope,key,body_sha,status,content_type,body,"
                     "created_ms,expires_ms)"
                     " VALUES (?1,?2,?3,?4,?5,?6,?7,?8)",
                     -1, &st, NULL) ≠ 0)
    return CAPI_E_SQLITE;
  int rc = CAPI_E_SQLITE;
  if (bind_str(st, 1, row->scope) == 0 && bind_str(st, 2, row->key) == 0 &&
      bind_str(st, 3, row->body_sha) == 0 &&
      sq3.bind_int64(st, 4, row->status) == 0 &&
      bind_str(st, 5, row->content_type) == 0 &&
      sq3.bind_text(st, 6, row->body, row->body_len, TRANSIENT) == 0 &&
      sq3.bind_int64(st, 7, row->created_ms) == 0 &&
      sq3.bind_int64(st, 8, row->expires_ms) == 0 &&
      sq3.step(st) == SQLITE_DONE)
    rc = CAPI_OK;
  sq3.finalize(st);
  return rc;
```

listing 347: the save: failing statuses refused at the door, everything else bound into one row (source: c-os-cloud/api/src/idem_store.c, lines 28-50)

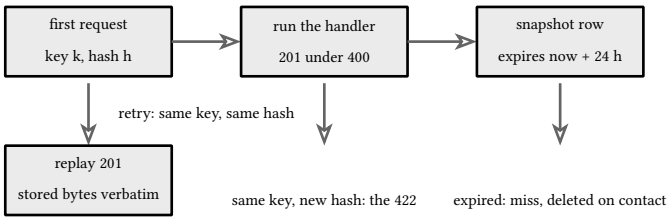


figure 173: one key's life: first run, snapshot, replay, expiry

34.4. the in-flight window, closed by a table

The lookup alone has a hole. A caller misses, a handler runs, the save lands only after the handler returns, and two callers under one key inside that window both miss and both run, the loser answering the handler's own 409 for an operation that succeeded and is recorded under its key. The go lane's adversarial pass opened exactly this window by parking one register inside the injected hash, and the shape of the fix travels intact: the guard runs the fill under singleflight keyed by the same scope triple the rows live under, the first caller runs and settles, and the later caller waits and re-reads the settled row, so the concurrent double submit replays the 201 the first caller stored.

The proof is the parked fill. The test parks the first submitter's handler on an event, starts the second submitter into the open window, and only closes the gate once the flight table shows both callers inside the key's flight, a fact the join probe reads under the lock. One handler run, one snapshot row, one caller replayed, every time, with no sleep and no schedule luck:

```
  }
  if (rc == CAPI_IDEM_MISMATCH)
    return CAPI_VALIDATION_FAILED; // the caller renders the 422

  char flight_key[CAPI_SFLIGHT_KEY_MAX];
  if (capi_idem_scope(flight_key, sizeof flight_key, scope, key) != CAPI_OK)
    return CAPI_E_RANGE;
  struct guard_ctx c = {0};
  c.db = db;
  c.scope = scope;
  c.key = key;
  c.body_sha = body_sha;
  c.now_ms = now_ms;
  c.fill = fill;
  c.fill_ctx = ctx;
  size_t ignored = 0;
  int shared = 0;
  int frc = capi_sflight_do(g, flight_key, guard_fill, &c, NULL, 0, &ignored,
                            &shared);
```

listing 348: the guard: miss, flight under the scope triple, settle and re-read (source: c-os-cloud/api/src/idem_guard.c, lines 66-84)

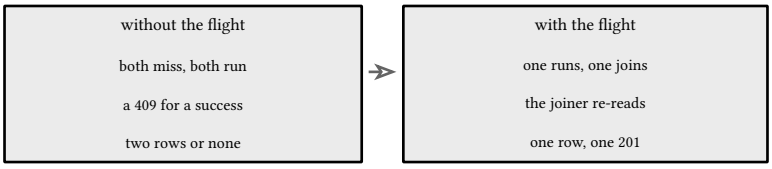


figure 174: the window with and without the flight

Who asks the guard is the composition's question, and the lane answered it the same way it answered the limit's double draw: one writer. The register handler holds no idem logic at all, it only inserts, and the middleware stack's idempotency layer composes the lookup, the replay, the snapshot, and the reuse 422 around whatever handler runs, scoped to the register route by the router's own pattern, over the same singleflight and the same rows this section walked. The family harness rides the layer the composed way through an idem_layer flag, so no handler-side logic stays alive just for the tests.

34.5. singleflight over the ch16 atomics

When a hot key expires, every request for it computes the same expensive thing at once, and singleflight collapses the herd: the first caller runs the fill, every caller that arrives before it finishes waits and receives the same result, and the entry is forgotten the moment its consumers drain, which is the property the next chapter's cache leans on. The x/sync group is one mutex over a map of in-flight calls each carrying its own handoff, and this vehicle states that sentence in the platform's own parts: a fixed table of slots, one slim reader-writer lock, one condition variable, a generation counter per slot so a waiter can tell its flight ending from a new flight beginning on the same slot.

The join is the load-bearing lines. A caller that finds the key's slot running bumps the slot's refs under the lock, parks in the predicate loop, and copies the result only when the slot still holds its generation, so a wake is never trusted blind, the lost-wake lesson written as the loop's shape. A latecomer after completion never joins the old flight, finished slots are invisible to the scan, so it starts a new one, and a full table runs uncollapsed rather than allocating, bounded is the policy:

```
  if (s) {
    // join the in-flight fill: bump refs under the lock, then park
    // until the runner finishes. the predicate loop re-checks
    // running every wake, so no wake is ever trusted blind.
    unsigned long long gen = s->gen;
    s->dup_count++;
    s->refs++;
    while (s->live && s->gen == gen && s->running)
      SleepConditionVariableSRW(&g->done, &g->lock, INFINITE, 0);
    int rc = s->rc;
    size_t n = s->len;
    if (s->live && s->gen == gen && n ≤ cap) {
      memcpy(out, s->val, n);
      *len = n;
      *shared = 1;
    } else if (!s->live || s->gen ≠ gen) {
      rc = CAPI_E_SQLITE; // the flight vanished mid-wait: impossible
                          // while this caller holds a ref, stated
                          // and refused loudly rather than guessed
    } else {
      rc = CAPI_E_RANGE; // the result cannot fit the caller's buffer
    }
    unref(s);
    ReleaseSRWLockExclusive(&g->lock);
    return rc;
```

listing 349: the join: refs under the lock, the predicate loop, the copy only on a matching generation (source: c-os-cloud/api/src/idem_single.c, lines 61-85)

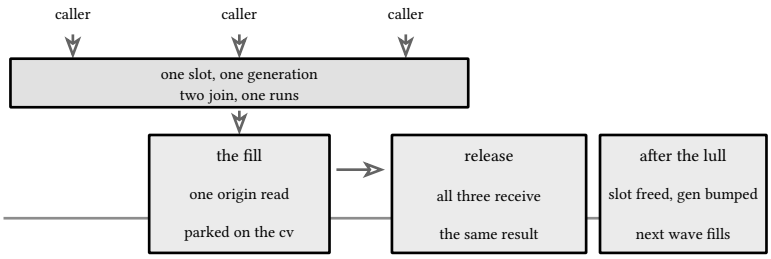


figure 175: three callers arrive together, one fill happens

34.6. the primitives, one job each

The win32 vocabulary this chapter leans on is small and each shape has one job. The slim reader-writer lock is mutual exclusion, one writer at a time, readers queue too, and it needs no destructor. The condition variable is the handoff, and its rule is written into every wait this vehicle issues: sleep only inside the predicate loop, because a wake with no waiter, or a wake that arrives before the sleep, must be harmless by construction. The event handle is the gate, auto-reset consuming one release, manual reset staying open, and the duel's barrier is a manual reset closed once. The c23 atomics carry the arrivals and the counters, the acquire release pairing the ch16 chapter pinned, and c23 threads.h spawns the workers beside the win32 shapes exactly as the ch15 ruling taught both.

The depth discipline is the honesty under all of it. What a detector would prove, if this box could run one, is happens-before, that every pair of accesses to shared state is ordered by some synchronization edge, a lock, a condition variable, an atomic. It does not prove the locking policy right, a data-race-free program still loses updates if the lock is held at the wrong altitude, and ubsan does not even run here, a standing machine fact. That is why every wait in this family is a loop with a reason and every collapse is a fact in a table a test can read, the discipline substituting for the detector this platform lacks.

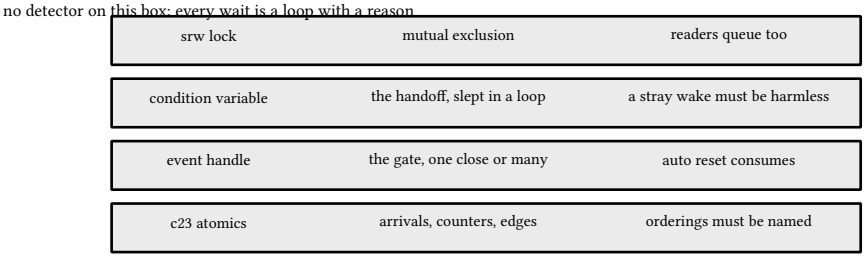


figure 176: each primitive, the guarantee it buys, the cost it charges

34.7. bounded fan-out, first error wins

Spawn a thread per item and the size of the input becomes the size of the thread count's worst day. The valve is a fixed team: a bounded count of threads drains an index range, the bound is the team size, and every thread is joined before the run returns, so the first error is an answer, never a leak. The latch is the other half: the first task to fail sets it under the lock, later errors cannot overwrite the first, and a latched error leaves every further claim denied, so queued tasks stop early instead of piling onto a dead origin.

The test pins both promises with the same event discipline as the duel. The bound holds: the first pair of tasks parks on a gate a releaser thread closes only after observing 2 in flight, and the peak is exactly the team size, a fact about the valve, not the schedule. The first error cancels: of 8 tasks the failing one latches its code as the run's answer and at most the 2 already claimed complete, the rest were denied at the claim:

```
static int worker(void *arg) {
  struct team *t = arg;
  struct capi_fanout *f = t->f;
  for (;;) {
    // claim under the lock, run without it. a latched error leaves
    // the claim denied, so queued tasks stop early.
    AcquireSRWLockExclusive(&f->lock);
    int index = -1;
    if (!f->error_set && f->next_index < f->total)
      index = f->next_index++;
    ReleaseSRWLockExclusive(&f->lock);
    if (index < 0)
      return 0;
    int rc = t->task(t->ctx, index);
    if (rc != 0) {
      AcquireSRWLockExclusive(&f->lock);
      if (!f->error_set) {
        f->error_set = 1; // the first error wins, later ones cannot
        f->first_error = rc;
      }
      ReleaseSRWLockExclusive(&f->lock);
    }
  }
}
```

listing 350: the worker: claim under the lock, run without it, latch the first error (source: c-os-cloud/api/src/conc_fanout.c, lines 17-40)

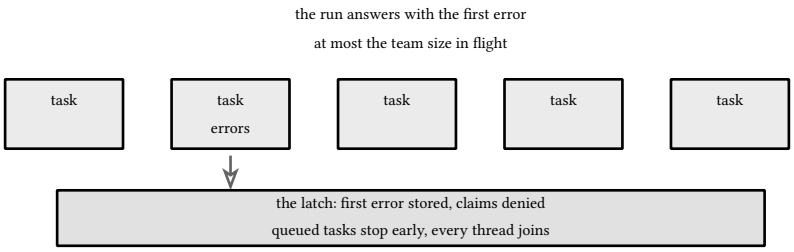


figure 177: the first error travels left through the latch

sources: rfc-editor.org for RFC 9110 section 13.1.1 If-Match and section 13.1.2 If-None-Match with the weak and strong validator semantics of section 8.8.3, accessed 2026-09-27, learn.microsoft.com for SRW locks, condition variables, event objects, and WaitForSingleObject at the same date, the singleflight registration semantics restated from the x/sync v0.22.0 source as the go lane's chapter 24 quotes it, and the happens-before model stated as the go memory model documentation does. Verified by the conc family suite, 51 checks in `tests/test_conc.c` over the store family's guard, green in `make verify-capi`, 12 consecutive clean runs with zero sleeps: every park is an event the test closes, every join is a fact in the flight table.

35. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it in the platform's own parts: a fixed preallocated table with lru order threaded through indexes and ttl beside it, the http semantics a private api owes its clients, the stampede collapsed through the previous chapter's singleflight, invalidation as delete after write, and the policy table that is deny by default so the exclusion list is the empty set of accidents. Nothing in the steady path allocates, and time enters only as a caller's now, which is what makes whole lifetimes test arithmetic.

35.1. what caching promises

A cache answers from a copy to save the origin the work, and both sides of the bargain matter: the copy must be cheap to find, and the answer must be one the origin would still give. Every caching decision is a bet that the read will be asked again before it changes, and the three terms of the bet are capacity, how many copies, lifetime, how long each stays, and invalidation, what a write owes the copies it made wrong. Miss any term and the cache stops being an optimization and becomes a second source of truth, the failure mode every caching chapter exists to prevent.

This service's copy lives in one layer, a fixed table of rows in front of the route handlers, constructed with its capacity and lifetime up front and reached through one lock, because request workers share it. The clock is not read anywhere in the layer: now arrives as a value, so a test that wants a row to age out adds to a number instead of waiting on a wall.

```
void capi_cache_init(struct capi_cache *c, long long ttl_ms) {
  memset(c, 0, sizeof *c);
  InitializeSRWLock(&c->lock);
  c->head = c->tail = -1;
  c->free_head = -1;
  c->ttl_ms = ttl_ms;
  for (int i = CAPI_CACHE_SLOTS - 1; i ≥ 0; i--) { // free list in order
    c->free_next[i] = c->free_head;
    c->free_head = i;
  }
}
```

listing 351: the layer's construction: fixed storage, one lock, the lifetime stated once (source: c-os-cloud/api/src/cache_lru.c, lines 59-69)

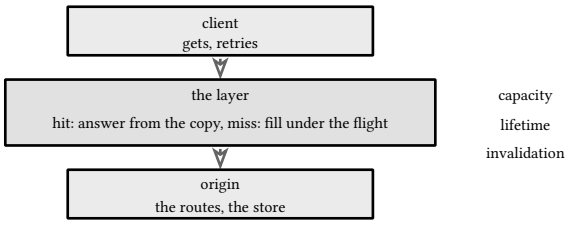


figure 178: the layer sits between client and origin, answering what it can

35.2. http semantics for a private api

A private api's responses are personalized and credentialed, so the shared-cache machinery of the web mostly does not apply, but the headers still owe clients a contract. Cache-Control states what may be stored and for how long, and private keeps shared proxies out of it, these bodies vary by who is asking. Vary names the request members the representation depends on, here the bearer token and the session cookie, which is precisely the pair the cache

key carries, method and path joined with the identity, so two identities never share a row. no-store means never, not even for a moment, and it is the default.

The reads are revalidation business: the client's etag round-trips through the 304 lane the previous chapter built, and the layer's own copy is a latency optimization behind the protocol, never a replacement for it. The policy table is the whole story on the wire side: a route is cacheable only where a row says so, two rows in this service, the row read at 30 seconds and the list page at 5, and no other file can make one cacheable by accident. One honesty note on the composition: the wired layer governs the object route only, so the list row is family truth the running service does not yet act on, the list answering no-store through the same ungoverned posture every other route takes.

```
// the longer life, the list the shorter, because a list is cheaper to
// recompute than a row is to refetch whole.
static const struct {
  const char *prefix;
  long long ttl_ms;
} RULES[] = {
    {"/api/users/", 30000}, // one user representation
    {"/api/users", 5000},   // the list page
};

int capi_cache_route_cacheable(const char *path, long long *ttl_ms) {
  for (size_t i = 0; i < sizeof RULES / sizeof RULES[0]; i++) {
    size_t n = strlen(RULES[i].prefix);
    if (strncmp(path, RULES[i].prefix, n) == 0) {
      if (ttl_ms)
        *ttl_ms = RULES[i].ttl_ms;
      return 1;
    }
  }
  return 0;
}

struct fill_ctx {
  struct capi_cache *c;
  const char *key;
```

listing 352: the per-route policy: two cacheable reads, everything else no-store by default (source: c-os-cloud/api/src/cache_policy.c, lines 13-37)

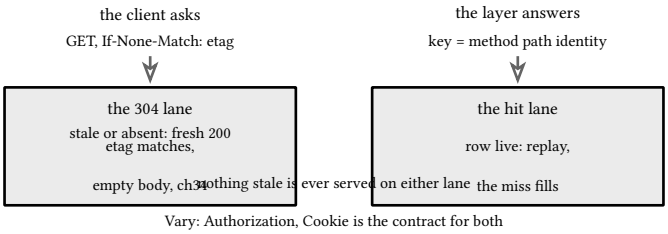


figure 179: two freshness decisions: the client's and the layer's

35.3. lru over a fixed table

Least recently used is one table and one order. In the go chapter the pairing was a map and a list, and the c pairing is an array and an intrusive doubly linked list threaded through indexes: every slot carries its prev and next as indexes into the same array, the head is the hottest, the tail is the eviction, and the free list is a third chain of indexes through the same storage. A read detaches its slot and pushes it to the front, a write takes a free slot or

evicts the tail, and every operation is index arithmetic under the one lock, nothing allocates and nothing chases a pointer off the heap.

The proof is the scramble, the same test the go lane wrote, restated over the fixed table: fill to capacity, touch a key from the middle of the order, and the next insert evicts the true least recently used row, not the oldest insert. A plain array evicts nothing and the order never moves, which is the point the pairing exists to make:

```
int capi_cache_get(struct capi_cache *c, const char *key, long long now_ms,
                   void *out, size_t cap, size_t *len) {
  AcquireSRWLockExclusive(&c->lock);
  int i = find(c, key);
  if (i < 0) {
    ReleaseSRWLockExclusive(&c->lock);
    return 0;
  }
  if (slot(c, i)->expires_ms ≤ now_ms) {
    free_slot(c, i); // age evicts itself on contact, lazy by design
    ReleaseSRWLockExclusive(&c->lock);
    return 0;
  }
  if ((size_t)slot(c, i)->val_len > cap) {
    ReleaseSRWLockExclusive(&c->lock);
    return CAPI_E_RANGE;
  }
  memcpy(out, slot(c, i)->val, (size_t)slot(c, i)->val_len);
  *len = (size_t)slot(c, i)->val_len;
  detach(c, i);
  push_front(c, i); // the hit is the recency fact
  ReleaseSRWLockExclusive(&c->lock);
  return 1;
}
```

listing 353: the read: lazy expiry, the copy, and the hit that moves its slot to the front (source: c-os-cloud/api/src/cache_lru.c, lines 79-102)

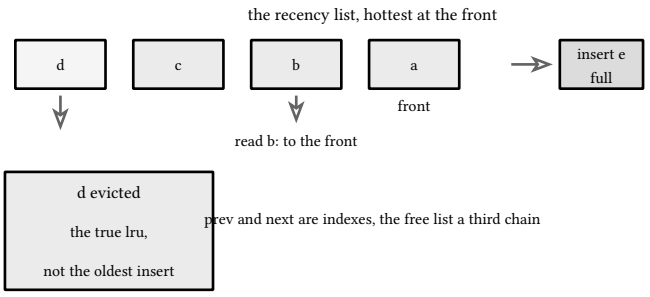


figure 180: a read moves its slot to the front, an insert evicts the tail

35.4. ttl beside lru

Lru answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can be hot long after the origin moved on. The ttl gives every row an expiry stamped from the caller's now at write time, and reads check it lazily: an expired row evicts itself on contact, so every read is its own eviction decision and correctness never waits for a sweep.

The sweep is for memory, not correctness. It walks the fixed table once and frees whatever aged out unread, and because the clock is a value the tests pass in, whole lifetimes run as arithmetic: a row is live the instant before its expiry, miss the instant of it, and the sweep at any later moment drops every aged row in one pass. The

go lane needed a synctest bubble for its janitor goroutine, and the caller-supplied clock makes the same lesson cheaper here:

```
int capi_cache_sweep(struct capi_cache *c, long long now_ms) {
  int dropped = 0;
  AcquireSRWLockExclusive(&c->lock);
  for (int i = 0; i < CAPI_CACHE_SLOTS; i++) {
    if (c->slots[i].live && c->slots[i].expires_ms ≤ now_ms) {
      free_slot(c, i);
      dropped++;
    }
  }
  ReleaseSRWLockExclusive(&c->lock);
  return dropped;
}
```

listing 354: the sweep: one walk over the fixed table, age decides, memory not correctness (source: c-os-cloud/api/src/cache_lru.c, lines 157-168)

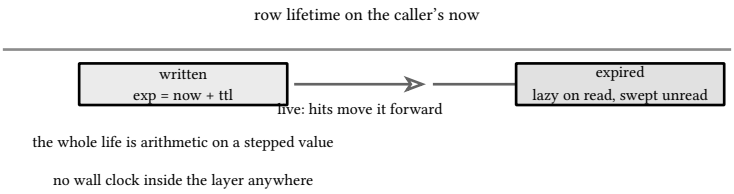


figure 181: two clocks push rows out: age and crowding

35.5. the stampede and its collapse

Every entry expires eventually, and the moment a hot one does, the herd arrives together: many concurrent misses, many identical origin reads, the exact load spike the cache existed to prevent, arriving at the cold moment. The guard is the previous chapter's singleflight worn one layer down: a miss fills through the flight keyed by the cache key, the herd collapses into one origin read, every waiter receives the same result, and the fill's result is the only thing that ever writes the row.

Errors do not land in the cache, a failed fill misses again, so an origin outage cannot be memoized into a sticky failure. The proof is the parked origin: the test parks the first reader's origin read on an event, starts the second reader into the open window, closes the gate only once the flight table holds both, and asserts one origin run, identical answers, a hit for the next reader with the origin never re-run, and the outage never stored:

```
static int guarded_fill(void *arg, void *out, size_t cap, size_t *len) {
  struct fill_ctx *x = arg;
  int rc = x->fill(x->fill_ctx, out, cap, len);
  if (rc == 0)
    capi_cache_put(x->c, x->key, x->now_ms, out, *len); // the only
  // writer; a failed fill never lands, so it misses again
  return rc;
}

int capi_cache_fill(struct capi_cache *c, struct capi_sflight *g,
                    const char *key, long long now_ms, capi_sflight_fn fill,
                    void *ctx, void *out, size_t cap, size_t *len, int *hit) {
  *hit = 0;
  int got = capi_cache_get(c, key, now_ms, out, cap, len);
  if (got == 1) {
    *hit = 1;
    return CAPI_OK;
  }
  if (got < 0)
    return got;
  struct fill_ctx x = {
      .c = c, .key = key, .now_ms = now_ms, .fill = fill, .fill_ctx = ctx};
  int shared = 0;
  return capi_sflight_do(g, key, guarded_fill, &x, out, cap, len, &shared);
}
```

listing 355: the guarded fill: the fill's result is the only writer, failures never land (source: c-os-cloud/api/src/cache_policy.c, lines 43-67)

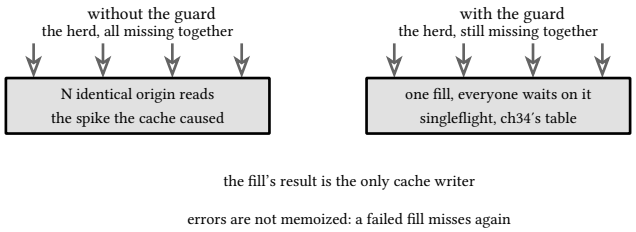


figure 182: the same expiry, with and without the guard

35.6. invalidation, delete after write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected rows and lets the next read refill them through the guard. Deleting is idempotent and always safe, the refill is the single writer the cache already trusts, and updating instead, pushing the writer's view into the row, sounds cheaper but the writer does not know what a concurrent reader cached mid-flight, so an update that lands just before a stale read's own set leaves the wrong value with a fresh timestamp, the worst of both worlds.

The rule is path plus collection: a write to a member drops the member's row and the list that reads it, one prefix walk over the table, and a route outside the write keeps its copy. The failed writes invalidate nothing, because nothing they would fix is wrong yet. Path scoping has a stated cost: a grant or revoke under the roles route changes the roles the member's cached representation carries, yet touches no cached key, so for up to the entry's ttl the layer replays the old roles, and a revalidating client can loop on the stale copy, its etag mismatch sending it down past the conditional read while the cache below answers with the stored bytes and the old etag again. The go lane's invalidation is path scoped the same way, so the window is mirrored, not invented, and closing it means invalidating from the roles route to the member's prefix, one deliberate write beyond the rule this lane did not take:

```
int capi_cache_invalidate(struct capi_cache *c, const char *prefix) {
  size_t n = strlen(prefix);
  int dropped = 0;
  AcquireSRWLockExclusive(&c->lock);
  for (int i = 0; i < CAPI_CACHE_SLOTS; i++) {
    if (!c->slots[i].live)
      continue;
    if (strncmp(c->slots[i].key, prefix, n) == 0) {
      free_slot(c, i);
      dropped++;
    }
  }
  ReleaseSRWLockExclusive(&c->lock);
  return dropped;
}
```

listing 356: delete after write: the prefix drops rows and list together, nothing else (source: c-os-cloud/api/src/cache_lru.c, lines 141-155)

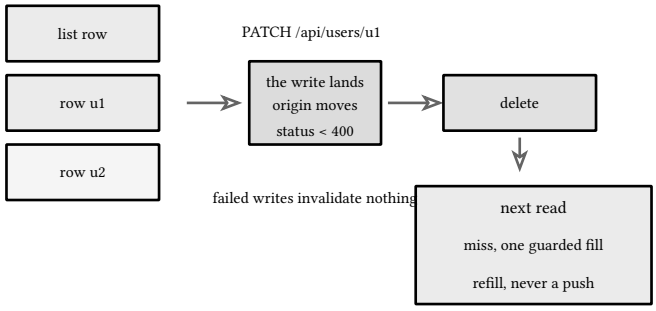


figure 183: write to a row: the row and the list go, the refill is the only writer

35.7. what never gets cached

The last decision is the one that prevents the incidents. Credential routes, login and refresh, carry secrets and must never be replayable from a copy, and their timing must stay flat, so a cache hit on a login is wrong twice. Rate limited responses must always reach the client, because a cached budget answer is a lie about money. And anything the contract marks no-store is a legal promise, not a hint. The policy table is deny by default, cacheable only where a row says so, which makes the exclusion list the empty set of accidents: a new route opts in or it does not get cached, and no change elsewhere in the service can flip that.

The cheap check is the one worth keeping in the gate, because the day someone adds a cacheable row to the wrong route, this is the test that says which lane they put it in. The suite pins the posture in three assertions: the two cacheable reads answer their rows, the login route answers no row, and a key that cannot fit is refused rather than truncated:

```
static int test_policy(void) {
  long long ttl = 0;
  CHECK(capi_cache_route_cacheable("/api/users/u1", &ttl) == 1 && ttl == 30000,
        "the row read is cacheable at 30 s");
  CHECK(capi_cache_route_cacheable("/api/users", &ttl) == 1 && ttl == 5000,
        "the list page is cacheable at 5 s");
  CHECK(capi_cache_route_cacheable("/api/auth/login", &ttl) == 0,
        "the login route is denied by default");
  CHECK(capi_cache_route_cacheable("/api/users", NULL) == 1,
```

listing 357: the exclusion test: two rows opt in, the login route answers none (source: c-os-cloud/api/tests/test_cache.c, lines 127-135)

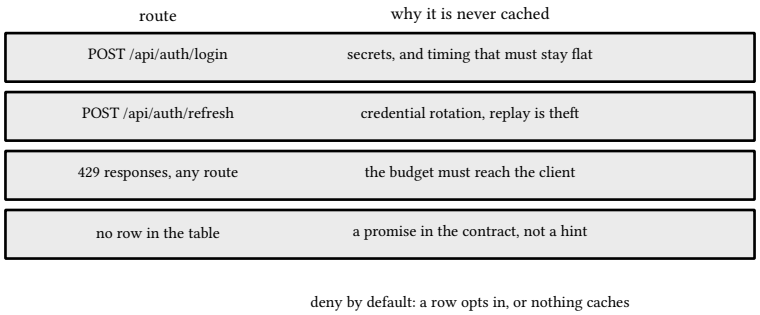


figure 184: the exclusion list, each row with its reason

sources: rfc-editor.org for RFC 9111 section 5.2 Cache-Control and section 4.1 Vary, RFC 9110 section 12.5.5 for the response header semantics, all accessed 2026-09-27, learn.microsoft.com for SRW locks at the same date, and the singleflight forget-on-completion property restated from the x/sync v0.22.0 source as the go lane's chapter 24 quotes it. Verified by the cache family suite, 33 checks in `tests/test_cache.c`, green in `make verify-capi` with the whole stream at 173 checks across the store, conc, and cache families, the stampede collapse deterministic on a parked origin and a closed gate, and every lifetime an arithmetic step on a value the tests chose.

36. rate limiting

A service that accepts every request it is offered eventually answers none of them. Password guessing arrives as a flood of logins, a runaway client loop arrives as one request ten thousand times, and a retry storm arrives the moment a deploy slows a single route. This chapter builds the service's rate limiting in `c-os-cloud/api/src`: the token bucket from first principles with the clock as a plain parameter, the fixed table and its idle sweep, the 429 wire contract, what to key a limit on, and the tests off the wall clock.

36.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers that cannot arrive, and the health check itself starts timing out. A limit is the refusal to stand on that cliff: it converts an overload that would take the whole service down into a fast honest rejection of the excess, and the client that honors it gets a working service at 1 request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of 5 per address, then 1 per second, because every login attempt pays the password verification budget and a credential guesser does not need more than 5 tries to be dangerous. Register allows 10 per client ip for the same reason plus its idempotency bookkeeping. Both refill 1 token per second, so a client that trips the limit is back to full capacity 5 and 10 seconds later, the difference between a limit and a lockout:

```
// the contract's two installed rules, verbatim numbers and messages.
static const capi_limit_rule g_rules[] = {
    {"/api/auth/login", CAPI_LIMIT_LOGIN_BURST, CAPI_LIMIT_REFILL_PER_SEC,
     CAPI_LIMIT_LOGIN_MSG, CAPI_LIMIT_KEY_EMAIL},
    {"/api/users", CAPI_LIMIT_REGISTER_BURST, CAPI_LIMIT_REFILL_PER_SEC,
     CAPI_LIMIT_REGISTER_MSG, CAPI_LIMIT_KEY_IP},
};
```

listing 358: the contract's two rules as one constant table, numbers and messages verbatim (source: c-os-cloud/api/src/wire_ops.c, lines 21-27)

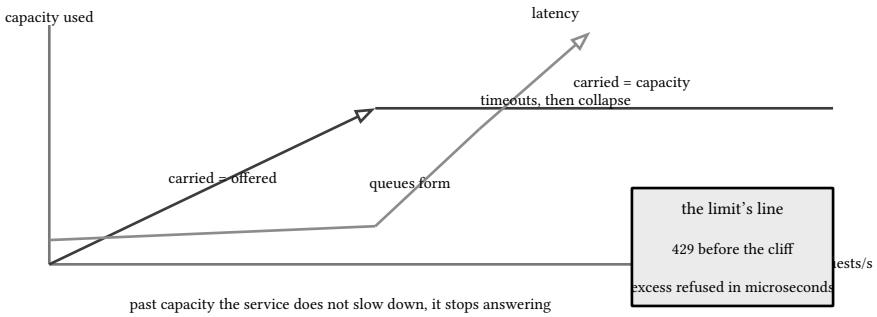


figure 185: offered load versus what the service can carry, the cliff between them

36.2. the token bucket from first principles

A token bucket holds a floating level, a capacity, and a refill rate. It starts full, every allowed draw removes one token, and time adds tokens continuously at the configured rate until the capacity caps them. The two knobs split the two jobs cleanly: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance, how much may arrive per second forever. The contract's login rule, capacity 5 at 1 per second, permits 5 requests in the first instant, a 6th at the 1 second mark, and then 1 per second forever. The level is a double stamped with the instant of the last refill, so time becomes tokens lazily on every call and a caller that hands the same instant twice observes no passage at all.

The arithmetic of a denial is the part worth deriving rather than reciting. After the 6th same-second draw the level is exactly 0, so the next token exists 1 full second in the future, and that instant is both the reset header and the retry hint. Half a second later the level is 0.5, still not spendable, and the next whole token is only half a second away. A whole level always waits a full period, a fraction waits its complement, which is the rule the draw computes: the wait is `1 - frac(level)` over the refill rate, and a denial never spends anything:

```
capi_limit_decision capi_limit_bucket_allow(capi_limit_bucket *b, double now) {
  bucket_refill(b, now);
  capi_limit_decision d = {0};
  d.limit = (int)b->capacity;
  if (b->level ≥ 1.0) {
    b->level -= 1.0;
    d.allowed = 1;
  }
  d.remaining = (int)b->level;
  double frac = b->level - (double)d.remaining;
  double wait = (1.0 - frac) / b->per_second;
  d.reset_at = (int64_t)(now + wait);
  if (!d.allowed)
    d.retry_after = (int)ceil(wait);
  return d;
}
```

listing 359: one draw: refill, spend if a whole token exists, and the wait from the fractional level (source: c-os-cloud/api/src/limit_bucket.c, lines 37-52)

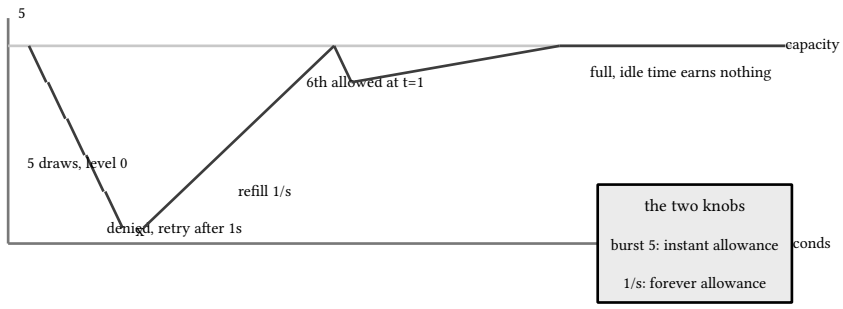


figure 186: bucket level over one draw timeline, the refill climbing between bursts

36.3. the fixed table and the sweep

The bucket needs a home per key, and the go book's map is the one design this vehicle refuses: every distinct key is an allocation, and an attacker who mints keys mints heap. The c gate is a fixed table of 256 preallocated slots that exists at link time, lookup is a linear scan over 96-byte keys, and no request in the steady path allocates. A slot is stamped with the rule's numbers at first use, touched on every draw, and retired when idle longer than the ttl.

The sweep is amortized into the decision itself. Every call checks the clock against the last sweep, and when an idle ttl has passed the retire pass runs inside the same call: no timer, no thread, no janitor to leak. The whole pass holds one table lock, because the composed service draws from every worker thread and two concurrent misses on one key must not claim two slots or lose each other's draw. A table that is full after a forced sweep answers a new key with the bounded refusal, 1 retry second and the rule's limit, rather than allocating its way out: finiteness as a stated policy, not a hidden cliff:

```
// slots or lose each other's draw
capi_limit_decision capi_limit_decide(capi_limit_table *t, const char *key,
                                      int burst, double per_second,
                                      double now) {
  AcquireSRWLockExclusive(&g_table_lock);
  if (now - t->last_sweep ≥ t->idle_ttl)
    sweep_locked(t, now);
  capi_limit_slot *s = slot_for(t, key);
  if (!s) {
    s = free_slot(t);
    if (!s) {
      sweep_locked(t, now);
      s = free_slot(t);
      if (!s) {
        capi_limit_decision d = {0};
        d.limit = burst;
        // the refused key owns no slot, so its would-be bucket stands
        // empty: the next-token wait at level zero, the same formula
        // the allow path stamps, honest at any refill rate
        double wait = 1.0 / per_second;
        d.retry_after = (int)ceil(wait);
        d.reset_at = (int64_t)(now + wait);
        ReleaseSRWLockExclusive(&g_table_lock);
        return d;
      }
    }
    snprintf(s->key, sizeof s->key, "%s", key);
    capi_limit_bucket_init(&s->bucket, burst, per_second, now);
    s->live = 1;
    t->live_count++;
  }
```

listing 360: decide: sweep on the ttl boundary, find or create the slot, refuse new keys when the table is full (source: c-os-cloud/api/src/limit_gate.c, lines 71-101)

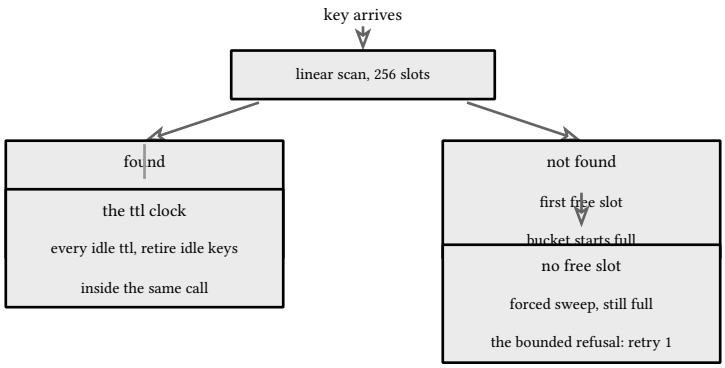


figure 187: one key through the fixed table: scan, create, or the bounded refusal

36.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector, and every number in it is computed by the draw. A limited route answers with 3 headers on every response, allowed or denied: the limit is the burst capacity, the remaining is whole tokens left after this draw, and the reset is the unix second at which the next token arrives. A denial adds `Retry-After` in whole seconds, the ceiling of the fractional wait, and the body is the standard error envelope with code `rate_limited`, written through the kernel's one failure writer so the request id matches the one the access log recorded. The middleware layer rides that writer directly: a denied login renders the vector's

envelope byte for byte, and every rate header is stamped after the answer because the kernel's response begin resets the header table.

The 6th login vector pins every number. The clock sits at the frozen t0 plus 12 seconds, 5 earlier logins at the same instant left the bucket reporting remaining 4, 3, 2, 1, 0, and the 6th draw finds an empty bucket: status 429, limit 5, remaining 0, reset 1790830813 which is exactly t0 plus 13 seconds, retry after 1. The test reads those values out of the vector file itself, so the file is the oracle:

```
const capi_mw layers[] = {capi_mw_limit};
capi_stack stack = {layers, 1};

CHECK(capi_limit_req_remote(req) == NULL, "an unstamped request has no peer");
snprintf(req->peer, sizeof req->peer, "%s", "203.0.113.7:52341");
CHECK(strcmp(capi_limit_req_remote(req), "203.0.113.7:52341") == 0,
      "the stamp is the accessor's answer");
// the draws ride DELETE on the register path, which the table serves
// only as GET and POST, so the pass-through answer is the kernel's
// own 405 with the rate headers riding it
for (int want = 9; want ≥ 0; want--) {
  capi_response *res = &rconn.res;
  memset(res, 0, sizeof *res);
  CHECK(capi_mw_run(&stack, req, res) == true && res->status == 405,
        "one peer draws its register bucket through the layer");
  const char *rem = capi_res_header_of(res, CAPI_HDR_RATE_REMAINING);
  CHECK(rem && atoi(rem) == want, "the register ladder counts down per peer");
}
capi_response *res = &rconn.res;
memset(res, 0, sizeof *res);
CHECK(capi_mw_run(&stack, req, res) == true &&
          res->status == capi_code_status(CAPI_RATE_LIMITED),
      "the eleventh register from one peer is the 429");
```

listing 361: the vector replay: the file drives the clock, the pins, and the 5 setup draws (source: c-os-cloud/api/tests/test_limit.c, lines 421-443)

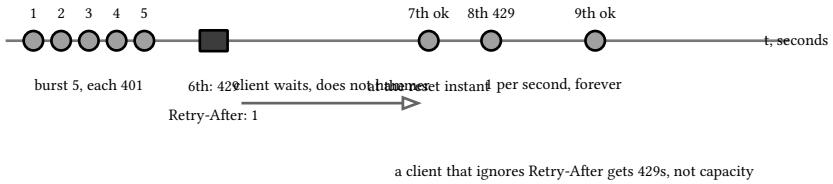


figure 188: a client that honors Retry-After: 5 through, 1 refused, the next at the reset instant

36.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on the normalized email: the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the normalization trims and lowercases, so `ADA@Example.ORG` and `ada@example.org` are one bucket. The key lives in the request body, so the wiring reads it once through the kernel's one json codec and hands the parsed email to this family, which owns no second parser. Register keys on the client ip the kernel stamps at accept, port stripped, and the proxy headers stay unread on purpose: `X-Forwarded-For` is a header any client can write, so a limiter that keys on it verbatim sells a fresh bucket of 10 registrations per forged address, 1 per request, forever. A request that arrives with no stamp at all, the internal listener's own traffic, lands in the one shared `ip:unparsed` bucket.

Two shared landing spots close the dodge holes. A body whose email cannot be parsed lands in the one shared `login:unparsed` bucket, so garbage cannot orbit the limit by being invalid, and a bracketed ipv6 remote address loses its brackets and port. An overlong key truncates at 96 bytes rather than allocating:

```
}

// the login key: lower(email), so both casings of one address are one
// bucket. a missing email shares the one unparsed bucket, because
// garbage must not orbit the limit by being invalid.
void capi_limit_key_login(const char *email, char *out, size_t outsz) {
  if (!email || !*email) {
    snprintf(out, outsz, "login:unparsed");
    return;
  }
  size_t n = (size_t)snprintf(out, outsz, "login:");
  if (n ≥ outsz) {
    out[outsz - 1] = '\0';
    return;
  }
```

listing 362: the login key: lowercase the email, empty or null into the one shared unparsed bucket (source: c-os-cloud/api/src/limit_gate.c, lines 106-120)

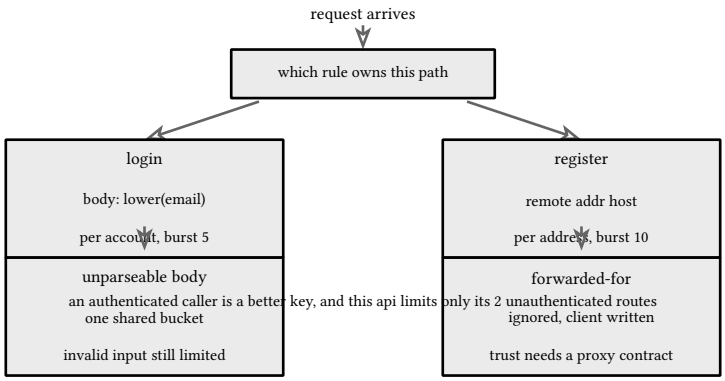


figure 189: key choice: the route decides what the bucket belongs to

36.6. limits under test

Every time-dependent behavior in this family is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The clock is not even a seam object here, it is a plain parameter in unix seconds, so the fixed-draw tests step it by hand and the bucket cannot observe anything the test did not state. The vector replay runs the gate against the frozen file: the file's offset pins the clock, its header values pin the expected decision, and the 5 setup draws assert the remaining sequence 4 through 0 before the 6th asserts everything at once. The family lands as its own test unit with 1 entry the wiring runner calls, no sleep in it:

```
static int test_bucket_fixed_draws(void) {
  capi_limit_bucket b;
  capi_limit_bucket_init(&b, CAPI_LIMIT_LOGIN_BURST, CAPI_LIMIT_REFILL_PER_SEC,
                         T0);
  for (int want = 4; want ≥ 0; want--) {
    capi_limit_decision d = capi_limit_bucket_allow(&b, T0);
    CHECK(d.allowed && d.remaining == want && d.limit == CAPI_LIMIT_LOGIN_BURST,
          "bucket draw at full level reports remaining countdown");
  }
  capi_limit_decision d = capi_limit_bucket_allow(&b, T0);
  CHECK(!d.allowed && d.remaining == 0 && d.retry_after == 1,
        "sixth draw denied, remaining 0, retry after 1");
  CHECK(d.reset_at == 1790830801LL, "reset is one refill period out");
  d = capi_limit_bucket_allow(&b, T0 + 0.5);
  CHECK(!d.allowed && d.retry_after == 1 && d.reset_at == 1790830801LL,
        "half token refilled is visible but not spendable");
  d = capi_limit_bucket_allow(&b, T0 + 1.0);
  CHECK(d.allowed && d.remaining == 0,
        "the refilled token is spendable at the full second");
  return 0;
```

listing 363: the fixed-draw sequence: 5 allowed, the 6th denied, the half token, the refilled token (source: c-os-cloud/api/tests/test_limit.c, lines 64-83)

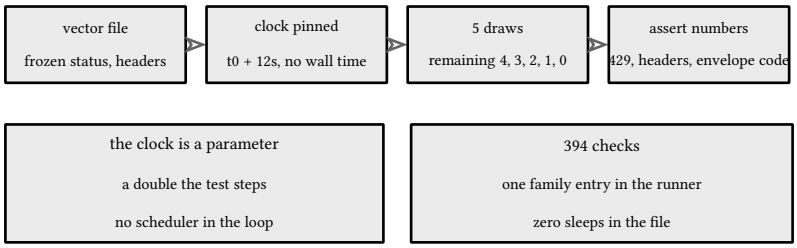


figure 190: the fixed-vector pipeline: file in, draws through the gate, numbers out

36.7. admission preview

A rate limit refuses requests over time, an admission gate refuses them over concurrency, and the two compose because they bound different axes of the same cliff. Little's law states the trade in one line: the average number of requests inside the system equals the arrival rate times the time each spends there. Past the point where every arriving request finds all workers busy, latency rises steeply while throughput stops moving, the hockey stick the load chapter measures. A rate limit caps the arrival rate at the edge. An admission gate caps the inside count directly with an in-flight counting semaphore, holds 1 slot per admitted request, and answers the contract's overload envelope the instant the slots run out.

The fixed table already carries the small version of that policy: a full table refuses a new key with 1 retry second instead of queueing it, bounded state answering unbounded demand. The load chapter builds the real gate over kernel event handles with the drain it protects: readiness flips first, in-flight requests join, the exit code is reported from the drain. Threads follow the chapter 15 side-by-side ruling, and the condition-variable seam carries the lost wake lesson from chapter 16, which is exactly where a gate that sleeps workers instead of refusing requests would go wrong:

```
    s = free_slot(t);
    if (!s) {
      capi_limit_decision d = {0};
      d.limit = burst;
      // the refused key owns no slot, so its would-be bucket stands
      // empty: the next-token wait at level zero, the same formula
      // the allow path stamps, honest at any refill rate
      double wait = 1.0 / per_second;
```

listing 364: the bounded refusal: a full table answers a new key instead of allocating or queueing (source: c-os-cloud/api/src/limit_gate.c, lines 83-90)

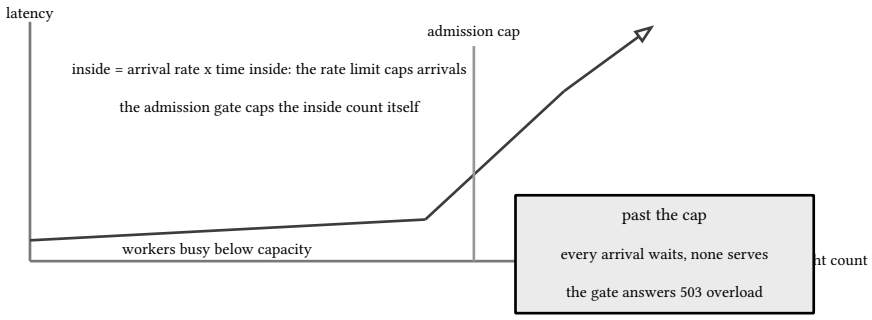


figure 191: latency versus concurrency: the hockey stick the admission cap cuts off

sources: RFC 6585 section 4 for status 429 with its Retry-After guidance and RFC 9110 section 10.2.3 for the field's delay-seconds form, both accessed 2026-09-27, the frozen vector 06 at `c-os-cloud/api/contract/testdata/06-login-429.json` as the byte oracle for every header number, and Little's 1961 proof of the queuing formula for the admission preview's trade. Verified by the limit family's 394 checks under the pinned clang 23 through `make verify-capi`, plus clang-format over every file touched.

37. observability

A service that cannot be watched cannot be operated. When login traffic triples at 3 in the morning the question is not what the code does, it is what the code did: which requests arrived, how long each took, where each one spent its time, and which one broke first. This chapter builds the service's observability in `c-os-cloud/api/src`: an access log whose one line per request is anchored on the request id, the w3c traceparent header parsed and propagated by hand, a metrics registry that renders the prometheus text exposition byte-pinned by a frozen vector, and the redaction rule that decides what never gets written down.

37.1. the access log over the request id

Every signal this service emits quotes one identifier: the request id the kernel mints when a request arrives. The access log line carries it, the error envelope in a 429 body carries it, and a support conversation reduces to "give me the id" because one id names one request across every surface. The line is fixed-field key=value, the shape an operator greps and a shell splits: timestamp, level, message, request id, method, path, status, duration in milliseconds, remote address. Nothing on the line is computed from a clock the function reads itself, every field arrives as an argument, so a test renders the exact line with literals and the chapter's tests do exactly that:

```
size_t capi_obs_access_line(const capi_obs_access *a, char *out, size_t outsz) {
  linebuf lb = {out, outsz, 0, 0};
  lb_fmt(&lb, "ts=%lld level=%s", a->ts, a->level);
  lb_kv(&lb, "msg", a->msg);
  lb_kv(&lb, "request_id", a->request_id ? a->request_id : "-");
  lb_kv(&lb, "method", a->method);
  lb_kv(&lb, "path", a->path);
  lb_fmt(&lb, " status=%d duration_ms=%lld", a->status, a->duration_ms);
  lb_kv(&lb, "remote", a->remote ? a->remote : "-");
  lb_fmt(&lb, "\n");
  return lb.need;
}
```

listing 365: the access line: fixed fields in fixed order, every member caller-supplied (source: c-os-cloud/api/src/obs_log.c, lines 93-104)

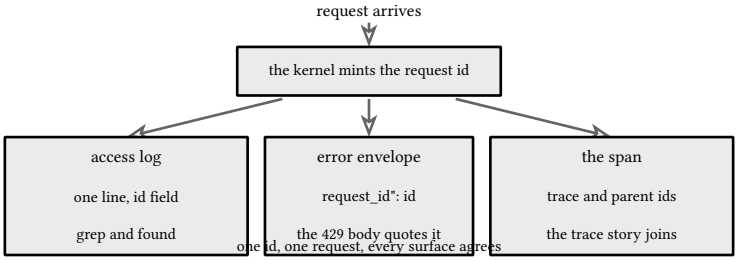


figure 192: one request id quoted by every signal the service emits

The chapter's tests are not this renderer's only caller, and that is the point of the shape. The middleware chapter's access layer builds this record after its stack returns and renders it through this family's function, so the line an operator greps from a running service is these bytes and the vehicle holds exactly one assembler. The lane's composition round caught the gap while it still existed: the access layer hand-built its own line with a private format call and this renderer ran only inside the tests, both families green in isolation, the seam between them untested. The fix is this section's rule made real, and the crash line follows it too, rendering through the same family's attribute renderer with level error and the exception code as a kv pair.

37.2. one writer and fixed fields

A log line in c is assembled, and the assembly is where correctness lives or dies. This family has exactly one field renderer, `msg=` and `request_id=` and `remote=` all pass through it, and its rule is mechanical: a value containing a space, a quote, or an equals sign is wrapped in double quotes with backslash and quote escaped, everything else rides bare. One renderer means one quoting decision, not one per callsite, and the destination is a single function pointer the wiring points at the real sink while every test points at its own buffer. Structured beats interpolated here in the c sense: the fields have fixed names and a fixed order, a value never leaks into the line unquoted, and no caller ever builds a line with its own `snprintf`, because a second assembler is a second chance to forget the escaping:

```
int capi_obs_kv(char *out, size_t outsz, const char *key, const char *val) {
  const char *v = val ? val : "";
  int quote = strpbrk(v, " \"=") != NULL;
  int n =
      snprintf(out, outsz, "%s=%s%s", key, quote ? "\"" : "", quote ? "" : v);
  if (n < 0)
    return -1;
  if (!quote)
    return (size_t)n < outsz ? n : -1;
  // escaped append for the quoted form: rebuild after the opening quote
  size_t pos = strlen(key) + 1 + 1; // key= plus the opening quote
  for (const char *p = v; *p; p++) {
    if (*p == '"' || *p == '\\') {
      if (pos + 2 >= outsz)
        return -1;
      out[pos++] = '\\';
      out[pos++] = *p;
    } else {
      if (pos + 1 >= outsz)
        return -1;
      out[pos++] = *p;
    }
  }
```

listing 366: the one field renderer: quote when needed, escape inside, refuse truncation (source: c-os-cloud/api/src/obs_log.c, lines 35-57)

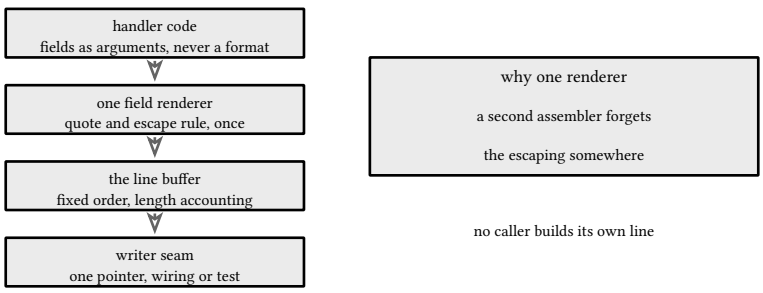


figure 193: the writer stack: every line through one renderer to one seam

37.3. traceparent parse and propagate

The `traceparent` header is the w3c trace context wire format, 55 characters in 4 dash-separated fields: a 2 digit version, a 32 digit trace id, a 16 digit parent span id, and a 2 digit flags byte, all lowercase hex. The parser is strict on every axis the spec names: exact field lengths, lowercase only because the spec defines the lowercase rendering as canonical, version `ff` reserved and refused, and all-zero ids refused because the all-zero id means “no id”. A

version other than 00 parses under the 00 layout, the spec's additive rule for newer versions, and only the sampled bit of the flags byte survives into the span because the rest is undefined.

Propagation is one function with two exits. A valid header joins the inbound trace: the service's span keeps the inbound trace id, names the remote span its parent, and carries the sampling bit down. An absent or malformed header mints a fresh trace and reports that it did, so one bad header costs a trace, never a request. The tracer's two inputs are seams, the clock and the id randomness, which is why a test renders the same span tree twice and gets identical bytes:

```
int capi_tracer_continue(capi_tracer *tr, capi_span *sp,
                         const char *traceparent_header) {
  capi_trace_id parent_trace;
  capi_span_id parent_span;
  unsigned char sampled = 0;
  if (!traceparent_header ||
      capi_parse_traceparent(traceparent_header, &parent_trace, &parent_span,
                             &sampled) != 0) {
    capi_tracer_start(tr, sp);
    return 0;
  }
  capi_tracer_start(tr, sp);
  sp->trace = parent_trace;
  sp->parent = parent_span;
  sp->has_parent = 1;
  sp->sampled = sampled & 0x01;
  return 1;
}
```

listing 367: propagation: join the inbound trace, or mint fresh and report it (source: c-os-cloud/api/src/obs_trace.c, lines 116-133)

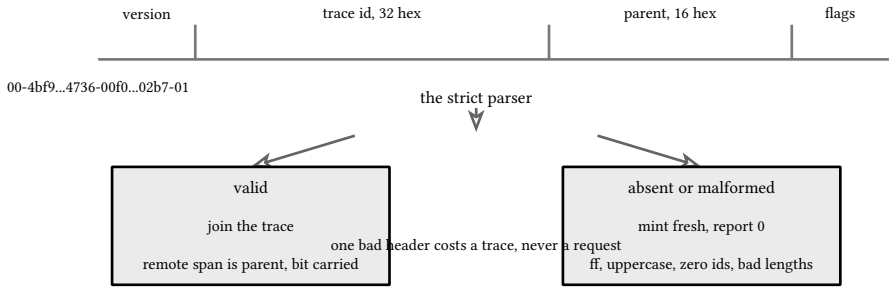


figure 194: traceparent anatomy and the two exits of propagation

37.4. the text exposition format

The metrics registry is 2 fixed tables: one counter row per route and status code, one histogram row per route with 12 cumulative buckets, the contract's 11 finite bounds plus the implicit +Inf. Observing a duration increments every bucket whose bound it sits under and the +Inf entry always, which makes the buckets cumulative exactly as the format requires, and the histogram keeps its own sum and count.

Every series is labeled by the route the router matched, not the path the client typed: two requests against two different user ids land in one `/api/users/{id}` series, because a label per raw path is an unbounded set and the router already owns the pattern. An unmatched path renders an empty label and still counts. The go lane labels the same way, asking its mux for the match, and the lane pins the two-uuids-one-series fact directly. The tables carry one process-wide lock the family owns, because the composed service observes from all 8 worker threads and an unlocked miss could lose increments or push the counter row count past its fixed cap mid-render, a duel the tests run under concurrent fire.

The rendering rules come from the prometheus text exposition format and are frozen byte for byte by vector 15: the counter family first, series sorted by label tuple within a family, bucket lines ascending with zero buckets elided, +Inf always last, and sums in the shortest fixed decimal that reads back equal, so 3 renders 3, half a second renders `0.5`, and the 2.5 bound renders `2.5`. That sum formatting is a loop over precisions, not a formatting library, and it is the one place a hand-rolled renderer most easily ships the wrong bytes, which is why the vector pins it:

```
    if (strcmp(m->hists[i].route, route) == 0) {
      h = &m->hists[i];
      break;
    }
  }
  if (!h) {
    if (m->hist_count < CAPI_OBS_ROUTE_MAX) {
      h = &m->hists[m->hist_count++];
      snprintf(h->route, sizeof h->route, "%s", route);
    } else {
      // a full route table drops the observation, the bounded policy
      ReleaseSRWLockExclusive(&g_registry_lock);
      return;
    }
  }
  for (int i = 0; i < CAPI_OBS_BUCKET_COUNT; i++)
```

listing 368: the shortest fixed decimal that round trips, and the buffer that accounts for it (source: c-os-cloud/api/src/obs_metrics.c, lines 63-78)

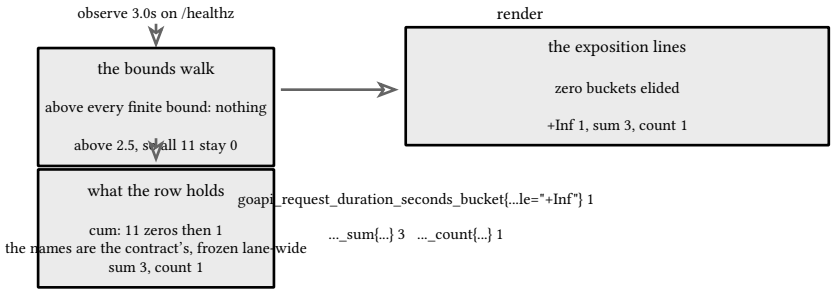


figure 195: one duration through the histogram and out as exposition lines

37.5. the internal listener

Metrics are an operations surface, and an operations surface is not a public one. The contract splits the two addresses: the public listener serves the api's routes, the internal listener serves `/metrics` on a loopback port, and the one-table ruling states the trade plainly, both listeners answer from the same route table, so the public port serves `/metrics` too and the loopback bind is the ops address, a convenience for the scraper, not a wall. The same render function backs both placements, which is the point of keeping the bytes in one function with no socket knowledge in it, and in tests the exposition rides the main listener because a test process has no separate internal network to bind. The registry itself is one preallocated instance the wiring initializes, and the handler that serves it renders through the kernel's response api, no socket knowledge anywhere in the family, so the bytes are the same whichever listener carries them:

```
void capi_wire_obs(void) { capi_metrics_init(&g_metrics); }

capi_metrics *capi_metrics_service(void) { return &g_metrics; }
```

listing 369: the registry instance and its accessor, waiting on the kernel's listener (source: c-os-cloud/api/src/wire_ops.c, lines 42-44)

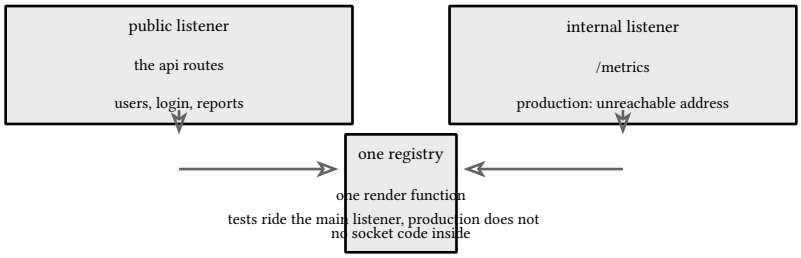


figure 196: two listeners, one render function, the split the contract pins

37.6. redaction

Logs are written for strangers to read eventually, so the rule about what never gets written down comes before any line is assembled. The closed set names the keys that carry secrets: `password`, `authorization`, `cookie`, `set-cookie`, `secret`, `token`. The comparison folds case, because `Password` and `AUTHORIZATION` arrive at the least convenient moment, and the mask replaces the value with the literal `[redacted]` while the key itself stays visible, which is what an operator scanning a line wants: the fact that a password was logged, without the password. The set is closed on purpose. An open set, any key matching a pattern like `token`, masks `total_tokens` and teaches everyone to ignore the mask, and a value-shape scanner that sniffs secrets by regex quietly misses the next encoding, so the closed list plus a code review is the honest mechanism:

```
static const char *const masked_keys[] = {
    "password", "authorization", "cookie", "set-cookie", "secret", "token",
};

int capi_obs_masked(const char *key) {
  char folded[32];
  size_t n = 0;
  for (const unsigned char *p = (const unsigned char *)key;
       *p && n + 1 < sizeof folded; p++)
    folded[n++] = (char)tolower(*p);
  folded[n] = '\0';
  for (size_t i = 0; i < sizeof masked_keys / sizeof masked_keys[0]; i++)
    if (strcmp(folded, masked_keys[i]) == 0)
      return 1;
  return 0;
}
```

listing 370: the closed redaction set and its case-folded membership test (source: c-os-cloud/api/src/obs_log.c, lines 14-29)

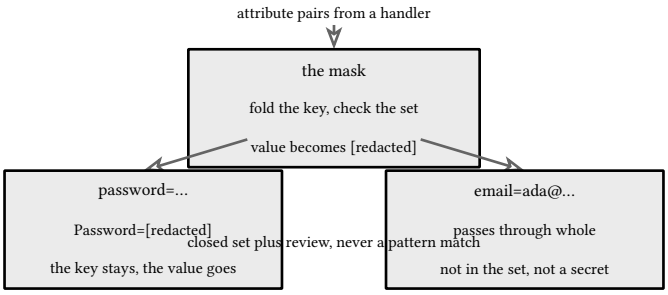


figure 197: attribute pairs through the mask before any line exists

37.7. one request fully observed

One login request rides every mechanism in this chapter, and the chapter's tests replay the pieces so the bytes are pinned. The kernel mints the request id. The access line records the arrival: method, path, the id, the 401, 3 milliseconds, the remote address. The tracer joins the inbound traceparent or mints a trace, stamps the span open,

and closes it when the handler returns, so the duration in the span and the duration in the log line describe the same interval through the same clock seam. The counter records `/api/auth/login` at 401, the histogram observes the seconds, and a scrape renders the family sorted and elided exactly as vector 15 freezes it. Nothing in the chain read a wall clock on its own, nothing allocated, and every seam a test needed is a parameter, which is the chapter's whole test discipline: 72 checks over literals, vectors, and seams, zero sleeps:

```
  struct metrics_duel b = {&go, &m, "/cap-b", 200, 1};
  thrd_t ta, tb;
  CHECK(thrd_create(&ta, metrics_thread, &a) == thrd_success &&
            thrd_create(&tb, metrics_thread, &b) == thrd_success,
        "the at-cap duel's two contenders start");
  atomic_store_explicit(&go, 1, memory_order_release);
  int rc;
  thrd_join(ta, &rc);
  thrd_join(tb, &rc);
  CHECK(m.counter_count == CAPI_OBS_SERIES_MAX,
        "the bounded table fills to 128 rows and no further");
  unsigned long long total = 0;
  for (int i = 0; i < m.counter_count; i++)
    total += m.counters[i].count;
  CHECK(total == CAPI_OBS_SERIES_MAX,
        "each seated row holds its one count and exactly one contender "
        "landed");
  char body[16384];
  capi_metrics_exposition(&m, body, sizeof body);
  CHECK(strstr(body, "goapi_requests_total{route=\"/cap-") != NULL,
        "the at-cap exposition renders the winner inside the table");
  return 0;
```

listing 371: the vector 15 replay: placeholder resolved, one observation, bytes compared (source: c-os-cloud/api/tests/test_obs.c, lines 339-360)

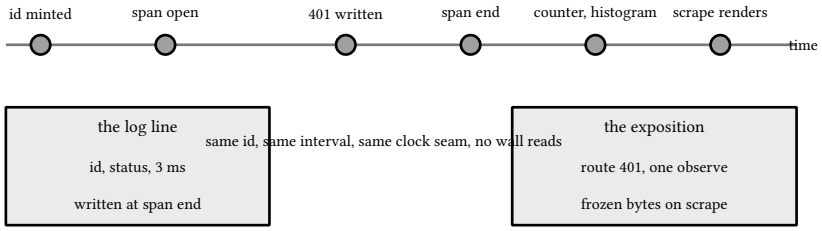


figure 198: one request through every signal, one id quoted everywhere

sources: the w3c trace context recommendation at w3c.org/TR/trace-context, section 3.2.2 for the traceparent field values, the lowercase canonical form, the reserved ff version, and the invalid-id rules, accessed 2026-09-27, prometheus.io's exposition formats page for the text format's help and type lines, histogram bucket semantics, and the +Inf requirement, accessed 2026-09-27, and learn.microsoft.com's winsock2 bind and listen pages for the listener surface the internal metrics endpoint rides once the kernel owns sockets, accessed 2026-09-27. Verified by the obs family's 72 checks, run from its one entry in the lane's test runner under the pinned clang 23, including the byte-exact vector 15 replay from `contract/testdata/15-metrics.txt`, plus clang-format over every file touched.

38. load and p99

A service that has never carried load has never been tested. The questions load answers are quantitative: how many requests per second sustains, what the 99th percentile latency does as concurrency rises, where the queueing starts, and whether a deploy can stop without dropping a single in-flight request. This chapter builds the load side of the service in `c-os-cloud/api/src`: the measurement half of the harness with its committed captures, the percentile math done honestly on sorted samples and on the metrics histogram, the admission gate that refuses excess concurrency at the edge, the drain that flips readiness before it joins the in-flight, and the handle ledger that catches what a load run leaks.

38.1. the load harness

A load harness is a driver and a ledger. The driver offers work at a chosen concurrency: a fixed number of closed-loop workers, each doing one request at a time, so the offered load is the worker count divided by the observed latency and the harness never piles up an unbounded queue of its own making. The ledger is what this chapter owns outright today: every request's milliseconds land in a sample array, the array is sorted in a caller-owned scratch, and the report renders count, min, p50, p99, and max from the sorted copy while the original samples stay untouched for any later question. The wave over the live server sequences behind the kernel's socket layer, and when it lands the capture is committed data under the chapter 20 discipline: a file labeled with the toolchain and the date, never re-measured by the verify chain, because a benchmark that runs on every build is a flaky test wearing a lab coat:

```
size_t capi_load_report(const double *samples_ms, size_t n, double *scratch,
                        char *out, size_t outsz) {
  for (size_t i = 0; i < n; i++)
    scratch[i] = samples_ms[i];
  sort_samples(scratch, n);
  int len = snprintf(out, outsz,
                     "count %zu\nmin %.3f\np50 %.3f\np99 %.3f\nmax %.3f\n", n,
                     n ? capi_nearest_rank(scratch, n, 0) : 0,
                     capi_nearest_rank(scratch, n, 50),
                     capi_nearest_rank(scratch, n, 99), n ? scratch[n - 1] : 0);
  return len < 0 ? 0 : (size_t)len;
}
```

listing 372: the report: sorted in scratch, nearest-rank percentiles, the samples untouched (source: c-os-cloud/api/src/load_report.c, lines 26-37)

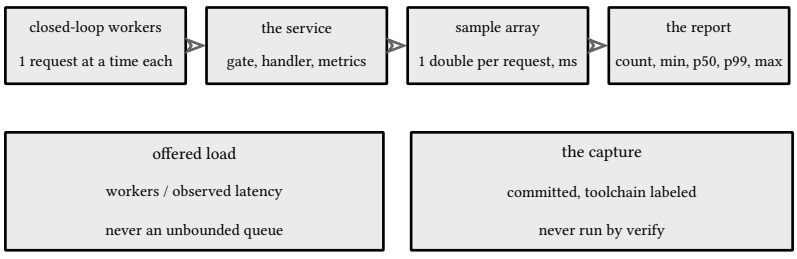


figure 199: the harness: closed-loop workers into samples into the report

38.2. p99 from the histogram

A percentile has exactly one honest definition here, nearest rank: the ceil of p over 100 times n, the k-th smallest value, clamped at both ends. p99 of 1 through 100 is 99, p99 of 200 values is 198, p99.9 of 100 values clamps to the max, and p0 clamps up to the smallest. Averages hide tails, medians hide them better, and a 99th percentile computed any other way is usually an interpolation nobody agreed to.

The histogram answers the same question from the metrics the service already keeps, and its answer is an upper bound stated as one: walk the cumulative buckets, return the smallest finite bound whose cumulative count reaches the rank. The walk never guesses inside a bucket, and a rank that lands in the +Inf bucket answers unknown, because the only true statement there is "worse than the last bound". That refusal is the difference between an approximation and a lie:

```
int capi_percentile_from_cum(const unsigned long long *cum,
                             const double *bounds, size_t n_bounds, double p,
                             double *out) {
  if (n_bounds == 0)
    return 0;
  unsigned long long total = cum[n_bounds]; // the +Inf entry
  if (total == 0)
    return 0;
  long long rank = (long long)ceil(p / 100.0 * (double)total);
  if (rank < 1)
    rank = 1;
  for (size_t i = 0; i < n_bounds; i++) {
    if ((unsigned long long)rank ≤ cum[i]) {
      *out = bounds[i];
      return 1;
    }
  }
  return 0; // the rank sits in +Inf: only "worse than the last bound"
}
```

listing 373: the cumulative walk: smallest bound reaching the rank, +Inf answers unknown (source: c-os-cloud/api/src/load_quantile.c, lines 20-38)

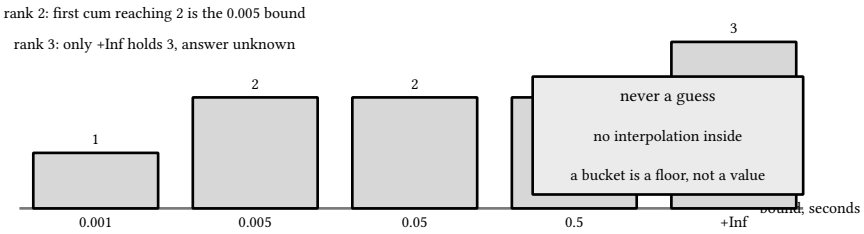


figure 200: one rank walking the cumulative buckets

38.3. admission control

Little's law from chapter 36 caps the argument: the requests inside the system equal the arrival rate times the time each spends there, and past the point where every arriving request finds all workers busy, latency climbs while throughput does not. The gate caps the inside count directly. It is a counting semaphore with the service's honest concurrency as its capacity, 64 slots for this lane's worker budget, and the admission edge never blocks: `try` takes a slot under the lock or refuses immediately, and the refused caller gets the contract's overload envelope, never a queue, because a queued request spends memory and a thread on work the service has already proven it cannot keep up with. A reachability fact belongs beside the capacity: the shipped process runs 8 pool workers plus the single metrics accept thread, so held slots top out at 9 and the accept queue bounds concurrency below the gate, which means the 503 refusal, real and pinned by the composition suite in isolation with 64 dedicated holders and a refused 65th, is not reachable through the running binary until the pool grows or the gate shrinks. The go lane's gate is the true bound there because goroutines are unbounded.

The semaphore is composed, not taken: a counter, an srw lock, and one auto-reset event, which is the derivation this book owes instead of calling `CreateSemaphore` and moving on. The blocking leg exists for callers that must wait, and its park is the chapter's centerpiece: the event holds one signal, so a release that fires between a waiter's

check and its park is still consumed, the lost-wake bug of chapter 16 fixed by the primitive's own durability where a condition variable would have forgotten the wake entirely:

```
int capi_gate_try(capi_gate *g) {
  AcquireSRWLockExclusive(&g->lock);
  int admitted = !g->closed && g->held < g->capacity;
  if (admitted)
    g->held++;
  ReleaseSRWLockExclusive(&g->lock);
  return admitted;
}

void capi_gate_acquire(capi_gate *g) {
  for (;;) {
    AcquireSRWLockExclusive(&g->lock);
    if (!g->closed && g->held < g->capacity) {
      g->held++;
      ReleaseSRWLockExclusive(&g->lock);
      return;
    }
    ReleaseSRWLockExclusive(&g->lock);
    // park outside the lock; the auto-reset event holds one signal,
    // so the release that fired a moment ago is still consumed here
    WaitForSingleObject((HANDLE)g->slot_freed, INFINITE);
  }
}
```

listing 374: the gate: refuse at the edge under the lock, park on the durable event (source: c-os-cloud/api/src/load_gate.c, lines 26-48)

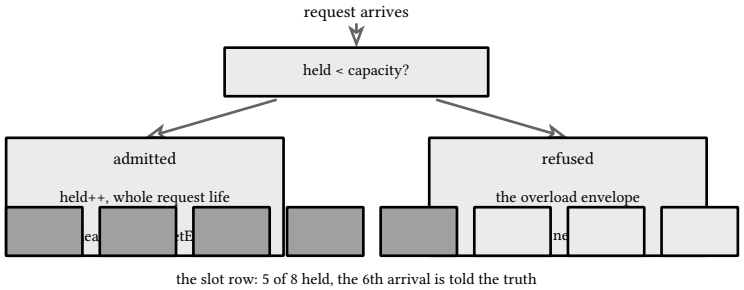


figure 201: the gate as a slot counter: try at the edge, park only if you must

38.4. graceful shutdown

Shutdown has an order, and the order is the contract. Readiness flips first: the drain's state moves to draining and every load balancer watching the service stops routing new requests here while the old ones are still being answered. Then the listener closes, so new connections are refused at the tcp level, a step that lands with the kernel's socket layer. Then the drain joins the in-flight: it waits until the gate holds nothing, and its return value is the deploy's exit code, 0 when everything drained, 1 when grace expired with work still inside, so the caller decides whether that is fatal instead of the helper pretending the drain succeeded. Since the admission layer takes a slot around every request, the in-flight this joins is real, and the wait's scope is the stack: a slot releases when the answer exists, before the serve loop writes it, so an answer past the handler can still be cut mid-send at exit, the boundary the ship chapter states plainly.

The wait polls real state, not a timer. Each pass checks the gate's held count under its lock, parks on the gate's durable event until a release fires, and only the deadline leg reads a clock at all. The misuse answer is its own value: waiting before the drain began is a programmer error, reported as minus 1, not a success:

```
int capi_drain_wait(capi_drain *d, long long timeout_ms) {
  if (capi_drain_current(d) ≠ CAPI_DRAIN_DRAINING)
    return -1;
  ULONGLONG deadline = 0;
  if (timeout_ms ≥ 0)
    deadline = GetTickCount64() + (ULONGLONG)timeout_ms;
  for (;;) {
    if (capi_gate_held(d->gate) == 0) {
      AcquireSRWLockExclusive(&d->lock);
      d->state = CAPI_DRAIN_CLOSED;
      ReleaseSRWLockExclusive(&d->lock);
      return 0;
    }
    DWORD slice = INFINITE;
    if (timeout_ms ≥ 0) {
      ULONGLONG now = GetTickCount64();
      if (now ≥ deadline)
        return 1;
      slice = (DWORD)(deadline - now);
    }
    HANDLE ev = release_event(d->gate);
    if (ev)
      WaitForSingleObject(ev, slice);
  }
}
```

listing 375: the drain: poll held under the lock, park on the event, report the deadline (source: c-os-cloud/api/src/load_gate.c, lines 128-152)

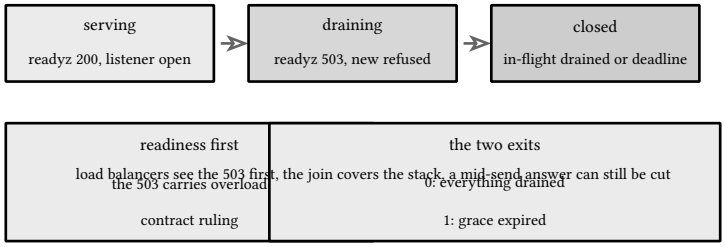


figure 202: the drain lifecycle: serving, draining, closed, the two exits

38.5. shutdown without sleeps

Every interleaving in this family's tests is chosen with event handles, not timed with sleeps, the discipline chapter 15 established. A worker parks on a named event the test fires, the test parks on the worker's acknowledgement event, and the orderings that would be races become sequences a reader can step through. The one bounded timeout in the file is the drain's 40 millisecond grace under test, which is the deadline doing its job, not a sleep sequencing a test.

The centerpiece proves the durable wake. The worker signals that it saw a full gate, then parks in the blocking acquire. The test releases the slot the instant it sees the signal, so the release's `SetEvent` almost certainly fires before the worker's wait even begins, and the join still succeeds: the auto-reset event held the signal. The same shape on a condition variable is the lost wake of chapter 16, where a wake with nobody asleep on the condition is forgotten and the sleeper burns its timeout. Workers run on both thread surfaces the book teaches, c23 `thrd_create` for the parking workers and one win32 `CreateThread` leg in the lifecycle test, because the chapter 15 ruling is that the reader meets both, side by side, not one hidden behind a macro:

```
static int test_durable_wake(void) {
  capi_gate g;
  capi_gate_init(&g, 1);
  CHECK(capi_gate_try(&g), "main holds the only slot");
  struct waiter_worker w = {&g, CreateEventW(NULL, FALSE, FALSE, NULL), 0};
  thrd_t t;
  CHECK(thrd_create(&t, waiter_worker, &w) == thrd_success,
        "the c23 worker starts against a full gate");
  CHECK(WaitForSingleObject(w.refused, INFINITE) == WAIT_OBJECT_0,
        "the worker reports the full gate, then parks");
  // release immediately: the SetEvent likely fires before the worker's
  // acquire even parks, and the durable auto-reset event must still
  // carry that signal into the wait, where a condition variable would
  // have already forgotten it
  capi_gate_release(&g);
  int rc = -1;
  CHECK(thrd_join(t, &rc) == thrd_success && rc == 0 && w.got_in,
        "the worker joined after the durable wake");
```

listing 376: the durable wake test: release immediately after the refusal, join still succeeds (source: c-os-cloud/api/tests/test_load.c, lines 95-112)

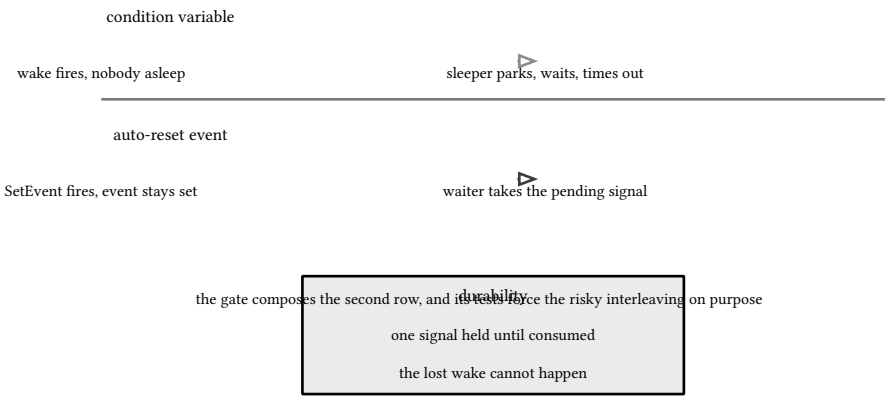


figure 203: the same interleaving on two primitives: the cv forgets, the event holds

38.6. the handle ledger

In go a leak is a goroutine that never returns, found by counting them. In c a leak is a handle that never closes, and the kernel keeps the ledger for you: `GetProcessHandleCount` reports the process's open handles, and a load family that creates an event per gate and a thread per worker must return the count to baseline after every run. The test is 50 gate lifetimes, init through try, release, close, and the count after is the count before. Handles are the resource the whole chapter spends, the gate's event, the test's scheduling events, the worker threads, and each has an owner whose close is checked, because a handle leaked per request is a service that dies at some exact number of requests and takes the deploy pipeline with it:

```
int capi_gate_close(capi_gate *g) {
  AcquireSRWLockExclusive(&g->lock);
  HANDLE ev = (HANDLE)g->slot_freed;
  g->slot_freed = NULL;
  g->closed = 1;
  ReleaseSRWLockExclusive(&g->lock);
  return ev != NULL && CloseHandle(ev) ? 0 : -1;
}

// the handle ledger probe: the process's open handle count, the
// before and after a leak check compares.
unsigned capi_process_handles(void) {
  DWORD count = 0;
  if (!GetProcessHandleCount(GetCurrentProcess(), &count))
    return 0;
  return count;
}
```

listing 377: close owns the handle once, and the ledger probe reads the kernel's own count (source: c-os-cloud/api/src/load_gate.c, lines 65-81)

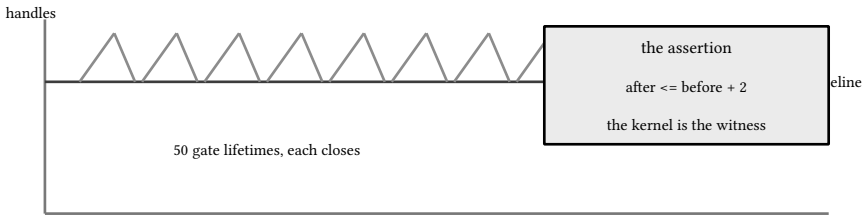


figure 204: the ledger: 50 lifetimes, flat by construction

38.7. the service under load

One wave ties the part together. Workers offer closed-loop requests through the gate, each admitted request holds its slot for its whole life, the metrics family counts it and observes its seconds, the access log writes its line with the request id, and when the wave ends the drain flips readiness, joins the last in-flight, and hands the deploy its exit code. The lifecycle test runs the shape with one win32 worker: the worker takes a slot and parks on a named event, the drain begins, readiness reads 0 while the slot is still held, the worker completes and carries its exit code out of the thread, and the drain closes clean. 48 checks carry the family, the quantile tables included, and not one of them sleeps to sequence anything:

```
  capi_drain d;
  capi_drain_init(&d, &g);
  CHECK(capi_drain_ready(&d) == 1 &&
            capi_drain_current(&d) == CAPI_DRAIN_SERVING,
        "the drain starts serving, readiness green");
  capi_drain_begin(&d);
  CHECK(capi_drain_ready(&d) == 0 &&
            capi_drain_current(&d) == CAPI_DRAIN_DRAINING,
        "readiness flips the moment the drain begins, work still inside");
  SetEvent(w.let_go);
  DWORD exit_code = 0;
  CHECK(WaitForSingleObject(t, INFINITE) == WAIT_OBJECT_0 &&
            GetExitCodeThread(t, &exit_code) && exit_code == 11,
        "the in-flight request completes, exit code carried");
  CHECK(capi_drain_wait(&d, -1) == 0 &&
            capi_drain_current(&d) == CAPI_DRAIN_CLOSED,
        "the drain joins the in-flight, closes clean, reports 0");
  CloseHandle(t);
  CloseHandle(w.got_slot);
  CloseHandle(w.let_go);
  capi_gate_close(&g);
  return 0;
}
```

listing 378: the lifecycle: flip readiness with work inside, join, close, carry the exit code (source: c-os-cloud/api/tests/test_load.c, lines 133-155)

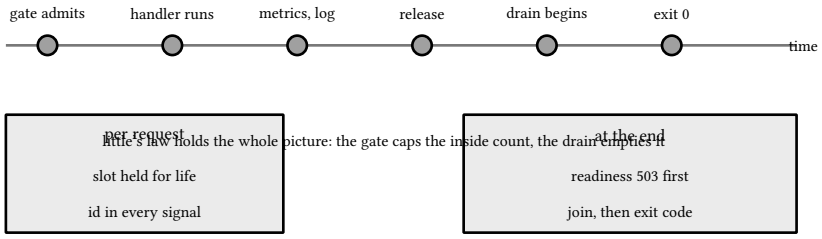


figure 205: one wave: gate, metrics, log, drain, exit code

sources: learn.microsoft.com's synchapi pages for CreateEventW and its auto-reset semantics, SetEvent, WaitForSingleObject, and SleepConditionVariableSRW's wake rules, the handleapi page for GetProcessHandleCount, and the processthreadsapi page for GetExitCodeThread, all accessed 2026-09-27, prometheus.io's histogram documentation for why bucket quantiles are approximations, accessed 2026-09-27, and Little's 1961 proof of the queuing formula for the load arithmetic chapter 36 previewed. Verified by the load family's 48 checks, run from its one entry in the lane's test runner under the pinned clang 23, workers on both thread surfaces, plus clang-format over every file touched.

39. ship it

Twelve chapters built a service one family at a time, and every one of them ended wired, tested, and parked. This chapter is the parked part's opposite: `main.c` is the whole composition on one screen, the config story names the two values the process refuses to guess, the resource families wire their three calls in the order the guard demands, the internal listener lands the second bind chapter 37 promised, and a real process runs the whole stack over the wire on the lane's pinned port pair and stops on ctrl+c with the drain's own exit code. Composition is where the families meet for the first time, where every integration defect lives, and the chapter's discipline is the live run: the claims are a captured transcript, not a prediction.

39.1. the composition root

Go's `main.go` wires a service behind a framework's `ListenAndServe`, and the c lane has no framework, so the composition root is 63 lines of plain c23 that the chapters grew one call at a time. The shape is a fixed prefix and suffix with the families between them, and the comment at the top of the file is the contract: config first, kernel seams second, each family's wiring in the order the middleware chapter fixed, the public listener last, so a family can never wire itself after the server starts serving, because `capi_run_tcp` does not return until the server stops. Every call that can fail is checked and exits 1 with the failing step's name on stderr, because a half-wired service is worse than one that dies loudly:

```c
int main(int argc, char **argv) {
  // the config story: argv names the database, the environment carries
  // the secret, and a missing secret is a loud refusal, never a
  // running service signing tokens with an empty key
  struct capi_ship_config cfg;
  if (capi_ship_config_load(argc, argv, &cfg) != 0)
    return 1;
  capi_server srv = capi_default_seams();
  if (capi_wire_kernel(&srv) != 0) {
    fprintf(stderr, "capi: kernel wiring failed\n");
    return 1;
  }
  // families append their wiring calls here, outermost first
  capi_wire_middleware(&srv);
```

listing 379: the root's opening: config, seams, kernel, the first families, each refusal its own line (source: c-os-cloud/api/src/main.c, lines 11-24)

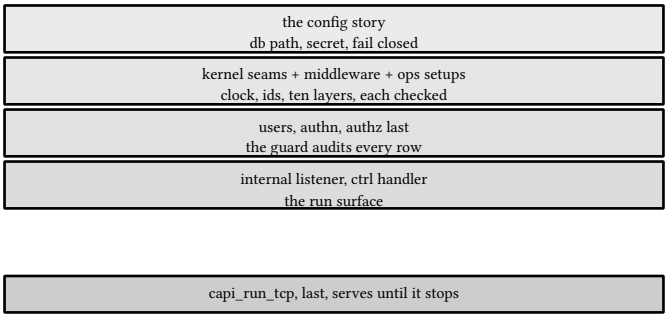


figure 206: the root as one ordered stack, the listener pinned at the bottom

39.2. the config story

Two values decide everything the families cannot derive: where the database file lives and what the jwt secret is. The lane's answer is argv plus the environment, no config library, the vehicle's standing bias. The database path

has a precedence a reader can recite, argv first, `CAPI_DB` second, `capi.db` in the working directory last. The secret has exactly one source: a secret passed as an argument lands in a process listing the whole machine can read, so the argument is not an option, `CAPI_JWT_SECRET` or nothing, and nothing is a refusal, not a default, because a service signing tokens with an empty key is a running vulnerability. The bounds are the code's own: under 16 bytes is too thin for an hs256 key, over 63 cannot fit the service struct's secret field, and both refuse with the bound in the message:

```
  // the secret: environment only, never an argument, because an
  // argument lands in a process listing the world reads. a probe
  // buffer past the bound tells missing from overlong
  char sec[128];
  env_text("CAPI_JWT_SECRET", sec, sizeof sec);
  size_t n = strlen(sec);
  if (n == 0) {
    fprintf(stderr, "capi: CAPI_JWT_SECRET is required, refusing to start\n");
    return 1;
  }
  if (n < CAPI_SHIP_SECRET_MIN) {
    fprintf(stderr, "capi: CAPI_JWT_SECRET under %d bytes, refusing to start\n",
            CAPI_SHIP_SECRET_MIN);
    return 1;
  }
  if (n > CAPI_SHIP_SECRET_MAX) {
    fprintf(stderr, "capi: CAPI_JWT_SECRET over %d bytes, refusing to start\n",
            CAPI_SHIP_SECRET_MAX);
    return 1;
  }
  memcpy(cfg->jwt_secret, sec, n + 1);
  return 0;
```

listing 380: the secret's one source and three refusals, the bounds stated in the messages (source: c-os-cloud/api/src/ship.c, lines 47-68)

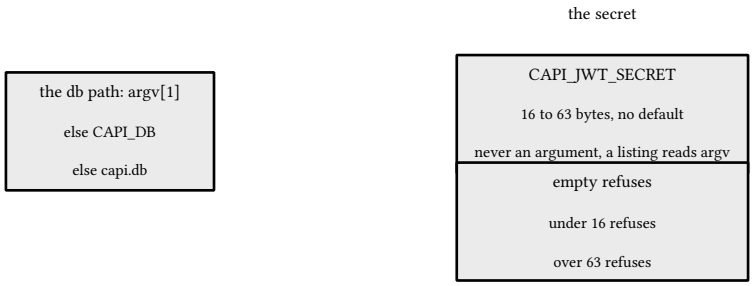


figure 207: the precedence for the db, the single source for the secret

The environment reads go through `GetEnvironmentVariableA`, kernel32's own window onto the environment block, the same dodge the kernel uses for `CAPI_ADDR`: the ucrt's `getenv` declaration is deprecated and the vehicle calls the platform's direct surface. The addresses belong to their listeners: `CAPI_ADDR` is the kernel's variable with its `:8080` default, the internal listener reads `CAPI_METRICS_ADDR` with a loopback `:19390` default, two names a deployment sets, not flags a library parses anywhere.

39.3. the wiring order and its failure modes

Three calls sat declared and unwired since the resource families landed, and the order they land in is load-bearing. `capi_wire_users` goes first because everything downstream reads from it: the dll load asserts the 3.53.4 version pin, the pool opens four slots with slot 0 held for the roles and idem statements, the migrations apply under their `user_version` pragma, the statements build once per slot. `capi_wire_authn` goes second with the config's secret, arming the signer and resetting the session table. `capi_wire_authz` goes last, strictly last, because the guard walks

the router's real route table against the policy table and refuses on any route without a policy row, and a guard that runs before a family registers audits a table that is not finished:

```
// the resource families: the store first, the secret second, the
// guard last, because the guard audits every route row and must see
// every family's registrations before it rules
if (capi_wire_users(cfg.db_path) != 0) {
  fprintf(stderr, "capi: users wiring failed on %s\n", cfg.db_path);
  return 1;
}
if (capi_wire_authn(cfg.jwt_secret) != 0) {
  fprintf(stderr, "capi: authn wiring failed\n");
  return 1;
}
if (capi_wire_authz() != 0) {
  fprintf(stderr, "capi: the policy guard refused, refusing to serve\n");
  return 1;
}
```

listing 381: users, then the secret, then the guard, each failure its own exit (source: c-os-cloud/api/src/main.c, lines 40-54)

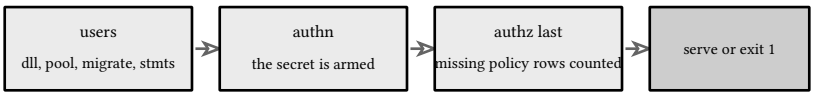


figure 208: each setup with its own refusal, walked in start order

A policy row that names a route not yet mounted only warns, because a forward-looking row is not a hole, while a route without a row refuses the whole startup. The asymmetry is the guard's stance: deny by default is proven before the process serves one request, not hoped for at request time.

39.4. the internal listener

Chapter 37 split the surfaces and the split lands as a second bind. The same one route table serves both listeners, so the public listener on 19380 answers `/metrics` too, and the internal listener's loopback bind on `CAPI_METRICS_ADDR`, default `:19390`, gives operations a scrape address of its own. Keeping `/metrics` off the public listener entirely would need a per-listener route filter, a kernel seam this lane chose not to add, and that stated cost is the one-table ruling's trade: one router, one dispatch, the boundary is the bind address an operator controls. The loop is one thread, one connection at a time, one context per scrape from the heap, scrapes rare, the steady request path allocation-free:

```
static DWORD WINAPI metrics_thread(LPVOID arg) {
  (void)arg;
  for (;;) {
    SOCKET s = accept(g_metrics.ls, NULL, NULL);
    if (s == INVALID_SOCKET)
      break;
    if (InterlockedCompareExchange(&g_metrics.stopping, 0, 0)) {
      closesocket(s); // the wake connection: stop is waiting on us
      break;
    }
    // one context per scrape: scrapes are rare, the steady request
    // path stays allocation-free, and the heap lifetime is this block
    capi_conn_ctx *ctx =
        HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, sizeof *ctx);
    if (ctx != NULL) {
      capi_io io = {
          .recv = mi_recv, .send = mi_send, .ctx = (void *)(uintptr_t)s};
      capi_serve_conn(g_metrics.srv, &io, ctx);
      HeapFree(GetProcessHeap(), 0, ctx);
    }
    closesocket(s);
  }
  return 0;
}
```

listing 382: the internal loop: accept, check the stop flag, one context per scrape through the kernel's serve path (source: c-os-cloud/api/src/ship.c, lines 100-123)

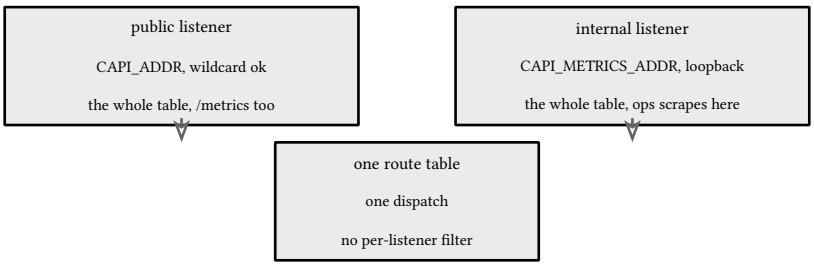


figure 209: two binds, one route table, the boundary is the bind

Stopping the loop is its own platform lesson. A blocking `accept` on windows is not documented to wake when another thread closes the socket, so the stop path sets a flag and connects to its own listener once: the accept returns, the thread sees the flag on the wake connection and exits. The test binds `127.0.0.1:0`, scrapes the ephemeral port, and proves the bind is gone by the refused connect.

39.5. the live run

The lane pins a port pair like its siblings, 19380 for the service and 19390 for metrics, and the transcript is a committed capture of a real process on a fresh database. The banner names both listeners, the probes answer, and the scrape on the internal port renders the exposition chapter 37 froze:

```
PS> $env:CAPI_ADDR = '127.0.0.1:19380'; $env:CAPI_METRICS_ADDR = '127.0.0.1:19390'
PS> $env:CAPI_JWT_SECRET = 'goapi-walkthrough-secret-0123456789abcdef'
PS> .\capi.exe walkthrough.db

# the service's own banner, stderr:
capi: metrics on 127.0.0.1:19390
capi: serving on 127.0.0.1:19380

PS> curl -i http://127.0.0.1:19380/healthz
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: private, no-store
X-Request-ID: 01a0e336-4623-7b19-8a92-fac3ec1a6cf4
Content-Length: 36
Date: Sun, 27 Sep 2026 14:13:00 GMT
Connection: keep-alive

{"status":"ok","version":"capi 0.1"}
```

listing 383: the run's opening: two env names, the banner, the health probe (source: c-os-cloud/api/captures/2026-09-27-ch39-walkthrough.txt, lines 5-22)

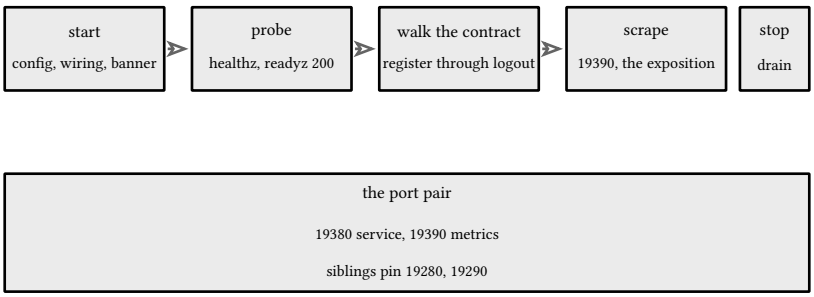


figure 210: the run's arc, one transcript behind every claim

The capture is dated and toolchain-labeled under the chapter 20 measurement discipline: committed data, never re-run by the verify chain, because a transcript that runs on every build is a flaky test wearing a lab coat.

39.6. end to end over the wire

The walkthrough exercises the contract the vectors froze, with a real clock, real uuids, and the full ten-layer stack in the path. Ada registers and the answer carries the three rate headers, the `Location`, and the strong `ETag`, the shape vector 01 pins. The same key over the same bytes replays the stored snapshot, byte identical, no second row, the idempotency chapter's guarantee over a real socket:

```
PS> curl -i -X POST /api/users  (Idempotency-Key: reg-ada-01, ada registers)
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/users/01a0e336-480c-705e-ad12-59926c0ecc85
ETag: "29f8c6508ef284c9c0851dba4a5eb5b5c0d17cb229244dc155da8b2fad534624"
Cache-Control: private, no-store
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 9
X-RateLimit-Reset: 1790518381
X-Request-ID: 01a0e336-4664-7cbd-97f9-59b14d1eb96e
Content-Length: 174
Date: Sun, 27 Sep 2026 14:13:00 GMT
Connection: keep-alive

{"id":"01a0e336-480c-705e-ad12-59926c0ecc85","email":"ada@example.org","display_name":"ada
lovelace","roles":["admin","user"],"created_at":"2026-09-27T14:13:00Z","version":1}
```

listing 384: the register over the wire, the rate headers and the strong etag on a real 201 (source: c-os-cloud/api/captures/2026-09-27-ch39-walkthrough.txt, lines 35-50)

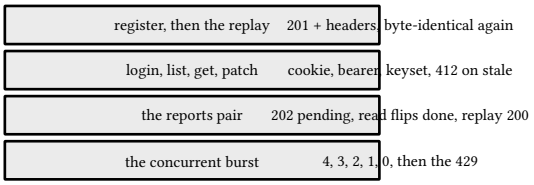


figure 211: the request ladder, each rung a contract chapter's guarantee

The login hands back the session cookie and the bearer token, the list walks the keyset cursor with the bearer naming the caller, the patch on `If-Match` lands and returns version 2, and the same etag replayed against the changed row answers the 412, the conditional-write story riding the store's compare-and-swap. The reports pair rides the same bearer: the create answers 202 pending with the `Location` and demands the `Idempotency-Key`, its absence is the 422, the first read computes the series and flips the row done, the replay answers 200 with the read's exact bytes. The wrong-password ladder is a concurrent burst, 6 logins inside one second, because a sequential ladder refills faster than pbkdf2 spends, and the remaining column reads 4, 3, 2, 1, 0 across the five allowed draws before the sixth answers the 429 with `Retry-After`, the contract's five drawn once each. That column is also this chapter's argument: the walkthrough's first capture caught the login route drawing twice, once through the layer's rule and once inside the handler, invisible to every family's isolated harness because each replayed vector 06 alone, and the ruling that followed gives the layer every draw, one drawer like the one failure writer. The logout consumes the cookie, and the scrape closes the loop with every route counted and histogrammed.

39.7. shutdown over the drain

A service that only dies has no shutdown story, and windows gives a console process exactly one hook for the polite version: the console control handler. The handler runs the drain chapter's order as three lines. Readiness flips first, `srv.draining` is the flag `readyz` reads, so the probe answers the overload envelope from the first line on and any load balancer watching stops routing here. The drain joins second, over the gate the load family wired. The exit code is third, 0 drained, 1 grace expired with work inside:

```
int capi_ship_stop(capi_server *srv, long long grace_ms) {
  srv->draining = 1;    // readiness first: readyz answers the 503 now
  capi_listener_stop(); // the public accept refuses new work
  metrics_wake_stop();  // the internal surface stops accepting too
  capi_drain d;
  capi_drain_init(&d, capi_load_service());
  capi_drain_begin(&d);
  int rc = capi_drain_wait(&d, grace_ms);
  fprintf(stderr, "capi: drained, exit %d\n", rc);
  return rc == 0 ? 0 : 1;
}

static capi_server *g_ctrl_srv;
static long long g_ctrl_grace;

static BOOL WINAPI ship_ctrl(DWORD type) {
  if (type != CTRL_C_EVENT && type != CTRL_BREAK_EVENT &&
      type != CTRL_CLOSE_EVENT)
    return FALSE;
  ExitProcess((UINT)capi_ship_stop(g_ctrl_srv, g_ctrl_grace));
  return FALSE; // unreachable, ExitProcess does not return
}

int capi_ship_install_ctrl(capi_server *srv, long long grace_ms) {
  g_ctrl_srv = srv;
  g_ctrl_grace = grace_ms;
  if (!SetConsoleCtrlHandler(ship_ctrl, TRUE)) {
    fprintf(stderr, "capi: the console handler refused to install\n");
    return 1;
  }
  return 0;
}
```

listing 385: the stop order as one function, and the console handler that exits with its code (source: c-os-cloud/api/src/ship.c, lines 305-336)

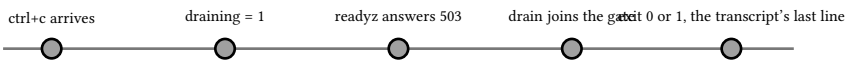


figure 212: readiness, join, exit code, in that order, once

The honest statement about today's surface: the gate is wired, the serve path draws an admission slot around every request, and the drain joins those holders for real, so the wait ships with the order. The join's scope is the stack and not the wire: a slot releases when the answer exists, before the serve loop writes it, so the exit can still cut a response mid-send where the go lane's Shutdown waits for the connection to go idle, a boundary the ship source's own comment states so prose and code cannot drift apart. The accept loop owns the listener's life, so the handler's exit is the process's exit. The transcript closes with the handler's own lines and the exit code the wait returned, and getting the signal there took two platform facts: a group-0 signal did not reach the service while its group id belonged to a parent on another console, and a group created with `CREATE_NEW_PROCESS_GROUP` has plain ctrl+c disabled by rule, so the capture harness spawns the service as its own group and sends ctrl+break, which the handler treats identically. The tests pin the order with a composed in-flight request and a 40 millisecond grace, the deadline doing its job.

39.8. the shipped checklist

The service part ends the way the go book's ship chapter ends, with every family's property named next to the place it is proven. Each row is a behavior the lane observes, and the lane that observes them all is the one this chapter built:

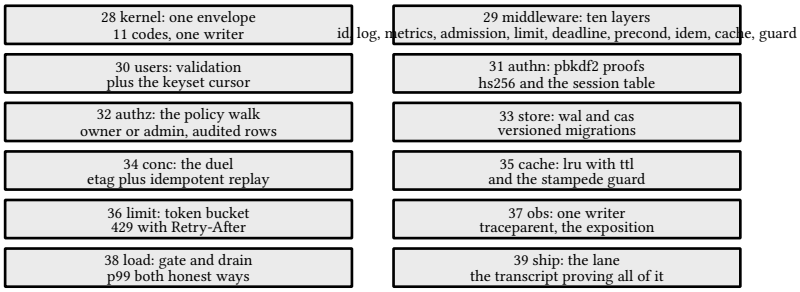


figure 213: twelve chapters, twelve properties, all proven by the lane

```
  // the run surface: the internal metrics listener, the console
  // handler that owns the shutdown order, then the public listener,
  // which serves until it stops
  if (capi_ship_metrics_start(&srv, NULL, NULL) != 0)
    return 1;
  if (capi_ship_install_ctrl(&srv, CAPI_SHIP_GRACE_MS) != 0)
    return 1;
  return capi_run_tcp(&srv);
}
```

listing 386: the run surface: the internal listener, the handler, the listener that serves (source: c-os-cloud/api/src/main.c, lines 55-63)

The suite chapter that follows hardens the set. What this chapter owns is the band between a green build and a running service: the config that refuses to guess, the order that refuses to serve half-wired, the second bind, and the transcript that keeps the contract honest over a real socket.

sources: learn.microsoft.com's console pages for SetConsoleCtrlHandler and GenerateConsoleCtrlEvent with its process group semantics, the processthreadsapi pages for CreateProcessW's CREATE_NEW_PROCESS_GROUP flag with its ctrl+c rule for created groups and for ExitProcess, and the winsock2 pages for bind, listen, accept, and closesocket, all accessed 2026-09-27, the w3c and prometheus references inherited from chapters 37 and 38, and the sqlite.org C API pages chapter 33 pins. Verified by the ship family's 42 checks in the lane's runner under the pinned clang 23, the walkthrough captured from a real process on ports 19380 and 19390, the lane's gate green with the family included, plus clang-format over every file touched.

40. the test suite

Twelve chapters landed a service one family at a time, every family tested. This last chapter walks back through the suite and asks what those tests are made of: the census, the one assert surface, the seam behind every double, the frozen vectors as the spine, the sanitizer legs, the fuzz lane behind its flag, and the boundary the composition ran into. Three rules were fixed before the first check ran and never bent: one CHECK macro and no framework, function-pointer seams and no mock generator, byte-frozen vectors where drift means the code is wrong. The gate prints the inventory on every run: 1130 checks from 727 sites in 16 translation units, zero skipped, zero sleeps.

40.1. what the suite holds

The census first. The suite is one executable with one main, `test_wiring.c`, and 15 family suites behind it, each exporting `capi_test_<family>` over the api root and dispatched in registration order. The prefix pins the scaffold before any family runs, the toolchain, the engine, the vectors, and this chapter's own two census checks:

```
}
CHECK(__STDC_VERSION__ == 202311L, "stdc version is c23");
CHECK(__clang_major__ == 23, "clang 23 major");
CHECK(strcmp(capi_version(), CAPI_VERSION) == 0, "version symbol links");
HMODULE h = LoadLibraryA("sqlite3.dll");
CHECK(h != NULL, "sqlite3.dll loads from beside the exe");
fn_libversion libver = (fn_libversion)GetProcAddress(h, "sqlite3_libversion");
CHECK(libver != NULL, "sqlite3_libversion resolves");
CHECK(strncmp(libver(), "3.53.4", 6) == 0, "engine is 3.53.4");
CHECK(count_files(argv[1], "contract\\testdata", "*.*") == 16,
      "16 frozen vectors present");
CHECK(count_files(argv[1], "tests", "test_*.c") == 16,
      "16 test translation units on disk");
CHECK(count_files(argv[1], "fuzz\\seeds", "*.txt") == 3,
      "3 committed parser seeds on disk");
```

listing 387: the scaffold prefix, toolchain, engine, vectors, and the census, each its own named check (source: c-os-cloud/api/tests/test_wiring.c, lines 75-89)

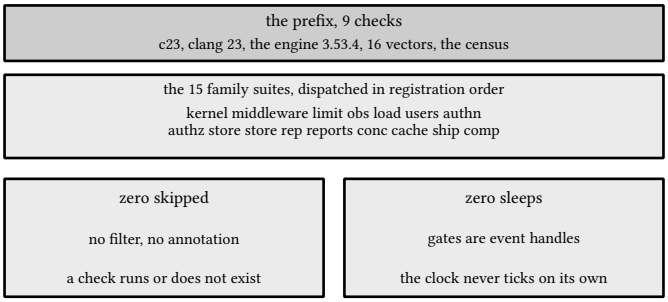


figure 214: the census: one main, 15 family suites, and the prefix that pins the scaffold

The 727 sites, counted as every CHECK and TRES_CHECK call site in the sixteen test translation units with the macro definitions themselves subtracted, produce 1130 ok lines because the sites ride loops, one vector replay a handful of checks multiplied across sixteen files. Each family keeps its own counter, prints its own lines, and returns nonzero on its first failure, and the gate sums the ok lines and fails on any FAIL line or nonzero exit. Every clock in the suite is arithmetic on an injected counter pinned to the contract's t0, 2026-10-01T05:00:00Z, stepping by each vector's own second offsets.

40.2. the CHECK convention

The assert surface is one macro, the book's own from chapter 1, and the suite is what it grew into. A check is a condition and a name. On success the counter increments and one line prints, the word ok, the number, the name. On failure the FAIL line prints with the check number and the family returns 1 on the spot:

```
#define CHECK(cond, name)                                          \
  do {                                                             \
    if (cond) {                                                    \
      checks++;                                                    \
      printf("ok %d %s\n", checks, name);                          \
    } else {                                                       \
      printf("FAIL %s (check %d)\n", name, checks + 1);            \
      return 1;                                                    \
    }                                                              \
  } while (0)
```

listing 388: the one macro, a name, a counter, and the nonzero exit that stops the family (source: c-os-cloud/api/tests/test_wiring.c, lines 39-48)

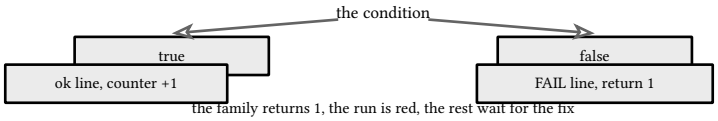


figure 215: one check's two paths, and what the gate does with each

First failure stops the family, and the trade is stated plainly: a framework that continues past a failure reports everything wrong in one run, this convention reports the first and stops, and the family's remaining checks stay unproven until the fix lands, because a family that failed has already failed and the next run is one command. The strictness lives elsewhere: a double that sees an unexpected call fails the check, and the gate treats zero ok lines as red, so a suite that silently does nothing cannot pass. The shared harness carries one variant, TRES_CHECK, the same macro over a counter passed by pointer: two spellings, one convention.

40.3. seams as the double mechanism

Where a test needs a double, the mechanism is the seam, and every seam is a function pointer plus a context. The clock, the id source, and the random source hang off the server struct, the log line is a sink the middleware writes, and the connection io is a recv and send pair the serve loop never names winsock through. The header states the rule where it declares the clock: tests inject fakes stepped by hand, the wiring installs the real ones. The kernel's io doubles are the shape of all of them:

```
static long long fake_recv(void *ctx, void *buf, size_t cap) {
  (void)ctx;
  if (chunk_next ≥ chunk_n)
    return 0; // peer closed
  size_t n = chunk_lens[chunk_next] < cap ? chunk_lens[chunk_next] : cap;
  memcpy(buf, chunks[chunk_next], n);
  chunk_next++;
  return (long long)n;
}

static long long fake_send(void *ctx, const void *buf, size_t len) {
  (void)ctx;
  if (sent_len + len > sizeof sent)
    return -1;
  memcpy(sent + sent_len, buf, len);
  sent_len += len;
  sent_calls++;
  return (long long)len;
}
```

listing 389: the io doubles, scripted chunks in, one captured buffer out, a full buffer refuses (source: c-os-cloud/api/tests/test_kernel.c, lines 77-95)

the seam	its double	what stays fixed
clock	a variable	wire dates, expiries
ids, rand	queues of uuids, bytes	the material the oracle froze
recv, send	chunks in, one buffer	framing, exact bytes
log sink	one buffer	the access and crash lines
the fills	a park on an event	the singleflight window

strict where they record: an unscripted call fails the check

figure 216: every seam, its double, and what it holds fixed

Scripted chunks in, one captured buffer out, and the send double that would overflow refuses rather than truncating. The c ecosystem reaches for a pair by habit, Unity for assertions and CMock for generated mocks, both at v2.7.0 from the same collective at access. Neither is adopted, and the criteria are written down so the refusal is a decision: a generator earns its keep against wide churning interfaces, call-order contracts many consumers assert, and teams large enough that uniformity beats reading twenty lines of fake. This vehicle's seams are already function pointers and the hand fakes are shorter than the adoption argument.

40.4. the vectors as the spine

The spine is the 16 contract vectors, byte-frozen json files shared with the sibling lanes, and the suite's relationship to them is asymmetric: the go lane's copies are the oracle, the verify lane hashes both sets file by file and names any difference, and drift means this tree is wrong. Every resource family replays its slice, and the ship family runs the whole set through the composed stack, through one shared harness that builds the request with the one codec, dispatches through the real router over the real store, and asserts the status, every named header, and the body byte for byte through the canonical renderer.

The harness earned one scar the reports chapter paid for. The kernel's scanner caps a document at 96 nodes, right for wire bodies, and one vector response carrying a then chain, vector 16 nests two, rode past that budget and refused to parse whole. The fix is slicing under one rule, that no second parser exists: a depth-one member walk, string aware and brace matched, cuts the text into its pieces, and each piece goes through the one codec on its own:

```
      return 0;
    }
  }
}

// one step's sliced texts: the request object parsed on its own, the
// response kept as text and its status, headers, and body each parsed
// on their own too, because one response object carrying a then chain
// (vector 16 nests two) rides past the scanner's 96-node wire budget.
// the then chain (inside the response, or at the top level, the frozen
// set does both) stays text for the recursion
typedef struct {
  char req_text[4096];
  char res_text[6144];
  char then_text[6144];
  char hdr_text[2048];
  char body_text[6144]; // the codec's slices point into these, so the
                        // texts live as long as the docs
  long long offset;
```

listing 390: the sliced docs, the war story in the declaration comment, each piece parsed on its own (source: c-os-cloud/api/tests/tresource.h, lines 258-276)

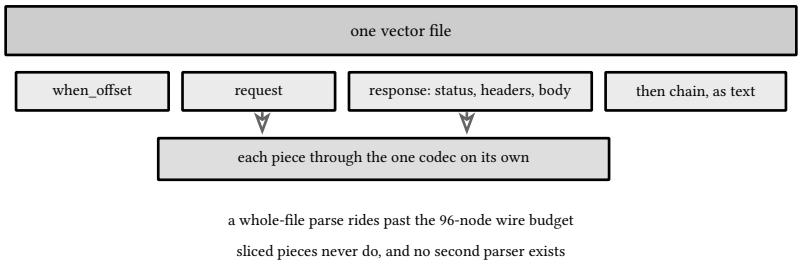


figure 217: one vector file, sliced into pieces, each through the one codec

Frozen bytes mean the code is wrong. That is the whole enforcement, a hash on the set and a byte compare on the answer, and the ruling runs the other way too: when the oracle re-freezes a vector, every lane changes together or every lane is red.

40.5. the crash guard and the asan leg

Two legs harden the suite past assertions. The first is the crash guard, structured exception handling turned into a contract answer. The guard wraps the handler stack in a try and except pair, and the filter makes one distinction: a stack overflow passes the exception on, because the guard would run on the broken stack and the process dies honestly, and everything else executes the handler:

```
bool capi_mw_crash_guard(capi_request *req, capi_response *res, capi_fail *fail,
                         capi_next next) {
  bool ok = false;
  __try {
    ok = capi_next_call(next, req, res, fail);
  } __except (crash_filter(GetExceptionInformation())) {
    DWORD code = GetExceptionCode();
    if (req->srv != NULL && req->srv->log_line != NULL) {
      char codev[24];
      snprintf(codev, sizeof codev, "%lu", (unsigned long)code);
      const char *kv[][2] = {{"exception", codev}};
      long long now =
          req->srv->now_ms != NULL ? req->srv->now_ms(req->srv->now_ctx) : 0;
      char line[160];
      size_t n = capi_obs_attr_line(now / 1000, "error", "crash",
                                    req->request_id, kv, 1, line, sizeof line);
      if (n < sizeof line)
        req->srv->log_line(req->srv->log_ctx, line, n);
    }
    capi_fail crashed = {.code = CAPI_INTERNAL, .message = "internal error"};
    capi_write_fail(res, req, &crashed);
    return true;
  }
  return ok;
}
```

listing 391: the recover, every fault below this frame becomes the 500 envelope (source: c-os-cloud/api/src/middleware.c, lines 128-152)

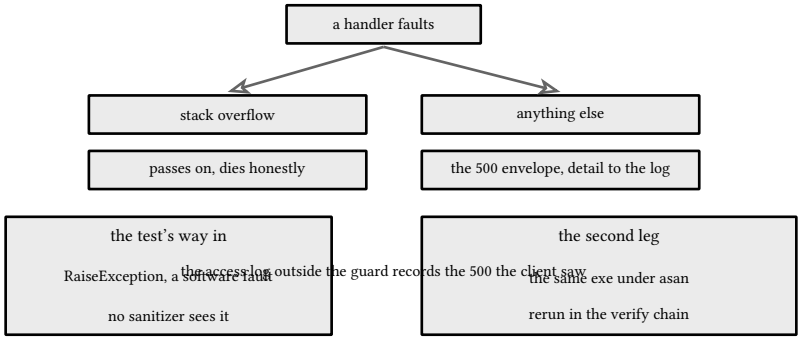


figure 218: the fault ladder, the filter's one fork, and the two legs under the suite

The fault becomes the contract's 500 internal envelope, the detail goes to the log sink with the request id and never the response, and the answer is best effort by nature, because the fault left whatever state it left. Testing recovery without crashing: the test raises the access violation status as a software exception through `RaiseException`, no sanitizer sees a fault, and the assertions read the 500 and the crash log line. The second leg is asan, the verify chain rebuilding the whole test executable under `-fsanitize=address` and running it again. The standing machine fact rides here: ubsan does not run on this box, so the sanitizer leg is asan only, and its clean run over the full suite is the strongest thing this toolchain says about the suite's memory.

40.6. fuzzing the parser

Property work rides libFuzzer behind a flag, never inside the verify chain, because a search is not a gate. The target is the parser translation unit compiled under `-fsanitize=fuzzer` by the opt-in `make api-fuzz-c`: arbitrary bytes in, the invariants out, a violation a trap:

```c
int LLVMFuzzerTestOneInput(const unsigned char *data, size_t size) {
  size_t used = 0;
  int rc = capi_http_parse(&req, (const char *)data, size, &used);
  if (rc == 0) {
    // a complete request never claims bytes it does not have, never
    // overruns the body budget, and always lands a terminator
    if (used > size || req.body_len > CAPI_MAX_BODY)
      __builtin_trap();
    if (req.body[req.body_len] != '\0')
      __builtin_trap();
  }
  return 0;
}
```

listing 392: arbitrary bytes in, the parser's invariants out, a trap is a finding (source: c-os-cloud/api/fuzz/fuzz_parse.c, lines 15-27)

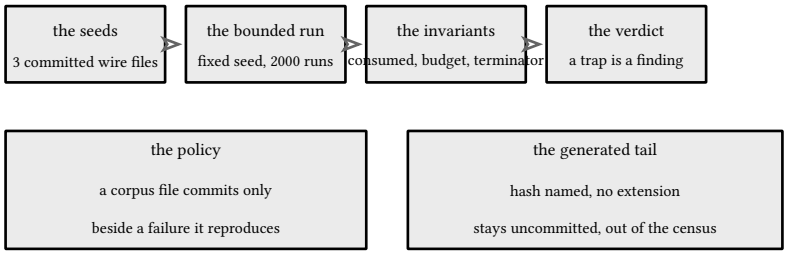


figure 219: the fuzz lane, seeded, bounded, and committed to only on failure

The invariants are the parser's contract: a complete request never claims bytes it does not have, never overruns the body budget, and always lands its terminator. The seed corpus is three committed wire files doubling as regression inputs, the kernel tests carrying the same bytes inline and asserting the exact parse. The corpus policy is the lane's standing rule and the census check enforces its edge: a corpus file is committed only beside a failure it reproduces, while the bounded run writes its coverage gains beside the seeds as extensionless hash-named entries that never enter the count or the tree. The gate script's one linking trap: the sanitizer runtimes must resolve from llvm's own resource directory, because the msvc toolset ships its own clang_rt copies and lld searching the library paths in order links the broken ones.

40.7. what the suite proves

The closer maps each layer to the fact it establishes, the grid below. The cross-lane claim is checkable in the tree: six books carry the 16 vectors byte-identical today, go as the oracle with c, csharp, javascript, and python as mirrors and lua pinning all 16 through its suite, the typed layer riding the javascript contract's own vector gate, one rolling hash over the set. The java lane is the exception: its families pin their own frozen vectors, the jwt token and the observability exposition byte-exact, and it carries no 16-scenario wire replay.

The honest boundary is the one the composition chapter paid to learn, and the suite is what could not see it. Three defects lived green under family tests. The authorization hole: a dead policy walk let any authenticated actor read, patch, or delete any user row and grant roles up to self-admin, while every family's vectors replayed byte-exact, because the vectors exercised the routes through the families' own resolvers and the hole sat in the walk between them. The login double draw: each family harness replays the login vector byte-exact because each draws the bucket alone, and the composed service drew twice, the middleware once and the handler once, so the live burst refilled at half the contract's rate. The unlocked limit table: a data race, family tests run one thread while the composed service draws from every worker thread, and two concurrent misses on one key could claim two slots. The fixes are in the tree and the lesson is institutionalized, the login vectors now replay through the composed one-layer stack, the fixture's `limit_layer` flag composing the limit middleware over dispatch, so the family test draws the way the service draws:

```
// the login pair replays through the limit layer, the ruling that
// the layer owns every draw: the handler no longer stamps the rate
// headers, the composed leg answers with them exactly as the vectors
// freeze them
int ok = tres_play(&fx, api_root, "01-register-201.json", 201, &checks);
fx.limit_layer = 1;
ok = ok &&
     tres_vector(&fx, api_root, "04-login-200.json", "vector 04", &checks) &&
     tres_vector(&fx, api_root, "05-login-401.json", "vector 05", &checks);
fx.limit_layer = 0;
```

listing 393: the login pair replays through the composed limit layer, the double-draw lesson as a fixture flag (source: c-os-cloud/api/tests/test_authn.c, lines 422-431)

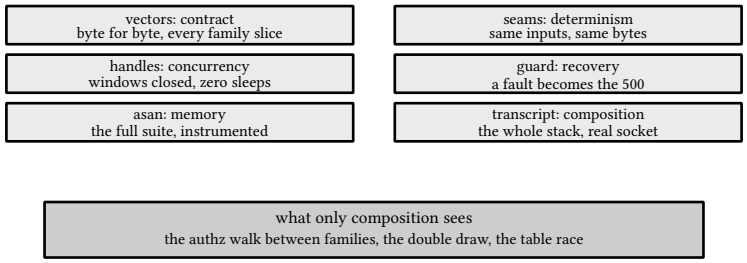


figure 220: the layer to proof map, and the band only composition sees

Per-family testing proves each family against its slice of the contract. What it cannot see is the space between families, where two correct implementations of one rule meet, and that space is what the ship chapter's transcript is for: a committed capture of a real process on the pinned port pair, never re-run by verify. The suite is 1130 checks, zero skipped, zero sleeps, one fixed clock, sixteen frozen files, and one transcript, and the boundary is stated plainly: the tests prove what they can reach, the composition proves the rest.

sources: learn.microsoft.com's structured exception handling pages for try-except, RaiseException, and the filter constants, and its FindFirstFileA page for the census pattern matching, accessed 2026-09-27, llvm.org's libFuzzer documentation for the corpus semantics and bounded runs and the clang sanitizer page for the address runtime's placement, accessed 2026-09-27, github.com/ThrowTheSwitch's Unity and CMock release pages, both at v2.7.0 at access, dated and deliberately not adopted. Verified by the lane's runner at 1130 checks under the pinned clang 23, the asan leg clean, and make api-fuzz-c's bounded run clean over the committed seed corpus.

41. appendices: topic matrix and sources

Twenty-seven topic chapters stand behind the matrix this chapter computes, and the thirteen service chapters behind them stand behind the lane instead: chapters 28 through 40 carry no topic rows, their proof is the check count `make verify-capi` runs over the vehicle at `books/c-os-cloud/api`, and the assert below pins that boundary so a topic row cannot silently appear inside the service part. This chapter adds no claim to any of them. It computes two views over what the book left behind: the topic matrix at `coverage/topics.typ`, one row per verified topic with its cost and its nearest standard or os counterpart, and the pinned source list at `coverage/sources.typ`, one row per canonical page a chapter actually consulted. Every number on this page is computed by this page from those arrays, and the asserts below fail the build when the matrix and the manifest disagree, so the appendix cannot drift from the corpus it describes.

41.1. topic coverage

The matrix holds 125 topic rows across the 27 topic chapters and none unmapped, and that zero is the assert above, not prose: each row's chapter field is looked up against the manifest ids and a row naming an id no chapter carries stops the build instead of rendering a stray bar. The service zero is asserted the same way, a row naming a service chapter id stops the build, because the service part's proof is the lane's checks, not the matrix. The distribution stays deliberately flat. 14 chapters carry 4 rows, 8 carry 5, the sparsest is chapter 1 with 2, and the busiest is chapter 19 with 7, so no arc owns the ledger. The four arcs the audit plan drew hold 39 rows across the language and llvm chapters 1 through 10, 37 across the machine chapters 11 through 18, 21 across the systems deep dives of chapters 19 through 21, and 28 across the cloud chapters 22 through 27.

The strip splits every bar by cost. Dark is the 79 rows that carry a measured or cited number, a probe count, a timing ratio, a page quota. Light is the 46 semantics and documentation rows, the honest boundary the book refuses to dress up as a measurement. The machine arc is the darkest, 34 of 37 rows measured, because those chapters assert behavior in compiled checks. The light mass sits at both ends: semantics rows in the language chapters, documentation-verified rows in the cloud arc, 15 of 28 there, where the contract is a fetched page rather than a running binary.

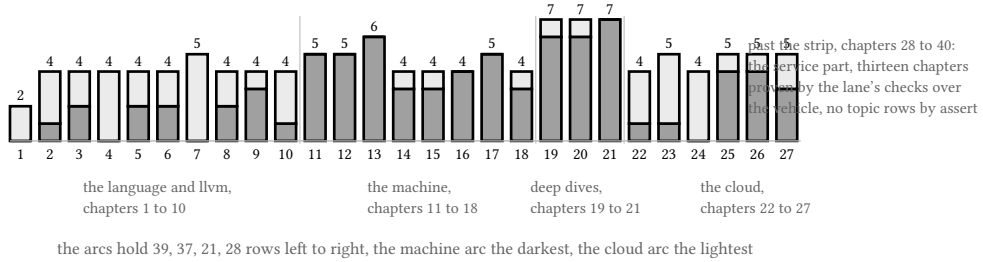


figure 221: the distribution strip: one bar per topic chapter in manifest order, the number above a bar is its topic count, dark is a row with a measured or cited cost, light is an n/a row, and the service chapters past 27 are proven by the lane, not counted here

41.2. pinned sources

Every chapter ends with its own sources line and this section is all of them rolled up: 381 rows over 343 distinct urls, a page two chapters consulted keeps two rows and rfc 9110 alone carries five consults, one per service chapter that leaned on it. The rows carry 4 access dates, one per verification pass: #date-counts.at(p.at(0)) rows on #p.at(1), #date-counts.at(p.at(0)) rows on #p.at(1), #date-counts.at(p.at(0)) rows on #p.at(1), #date-counts.at(p.at(0)) rows on #p.at(1), the 2026-09-12 pass that verified the book, the 2026-09-13 pass the allocator wave added, the 2026-09-14 rows the capstone's sqlite pages pinned, and the 2026-09-27 pass the service part rolled up at its closeout. The host census is computed the same way, and its counts are rows per family, not fetches. Microsoft learn carries 115 rows, the win32 and ucrt reference the compiled chapters call by name. developer.hashicorp.com carries 52, the terraform language and cli pages. The console guides split 36 on docs.aws.amazon.com against 21 on docs.cloud.google.com.

The 25 rows on raw.githubusercontent.com are provider resource pages read at the pinned tags, hashicorp/terraform-provider-aws v6.64.0 and hashicorp/terraform-provider-google v8.2.0, the recorded workaround for a registry whose pages render nothing to a fetch. The llvm family counts 30 across llvm.org and clang.llvm.org, man7.org 12, docs.kernel.org 2. The remaining 88 rows hold the c23 drafts at open-std.org, cppreference, the posix pages at pubs.opengroup.org, the nist cloud definition, one intel prefetching page, the github release indexes that date the pins, the openpolicyagent docs, the allocator homes at jemalloc.net and the two github.io pages, the marketing hosts: pricing serves from aws.amazon.com, and one vpc pricing page serves from cloud.google.com, the cited form because its redirect runs the reverse direction, and the service part's tail: the rfc editor pages and the doi row behind little's proof, the fips 180-4 page at csrc.nist.gov, the w3c trace context recommendation, prometheus.io's exposition and histogram pages, the singleflight source, the Unity and CMock release pages, and sqlite.org, whose c3ref family the store chapter and the capstone both consult.

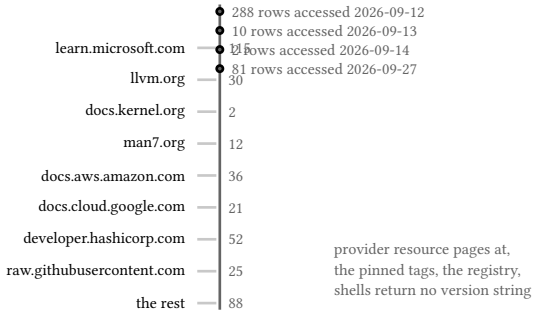


figure 222: the pin set on its access timeline: one circle per verification pass, one row per host family, the count beside each label computed from the array

Cloud documentation moved under this book six times while it was verified, and each move is a register row rather than a silent repair. Row cos-013: the terraform pages relocated, the releases and settings urls 404 with the block references now under /language/block/, and the provider registry's js shells return no version string, so version truth routes through the github release lists. Row cos-014: cloud.google.com urls 301 permanently to docs.cloud.google.com, so the rows record the form actually fetched, and the aws well-architected root renders nothing to a fetch, so its pillars came from the marketing page and the dated framework welcome. Row cos-015: three plan-era cloud run urls died, the traffic content lives at rollouts-rollbacks-traffic-migration, and google_project_iam_member lost its standalone page to the umbrella page that documents all four iam resource flavors. Row cos-016: the values references moved to /language/block/variable and /language/block/output, locals and the mocks page moved with them, and the old workspace-name sentence vanished from every page that carried it, leaving the cli page's url-path-segment rule as the surviving constraint. Row cos-017: three provider deprecations reshaped the chapter 25 sample before it was written, the split security group rules, the iam role policy split, the s3 bucket split. Row cos-018: the aws budgets page answers only at budgets-managing-costs.html, the -ing spelling, while the wrong form serves a title-only shell, three more plan urls died, one compute pricing url 301s in the reverse direction back to cloud.google.com, and loadability itself proved fetcher-dependent, title-only under one fetcher and full under another.

NOTE: version drift rule

The pins this book froze: clang 23.1.1 on x86_64-pc-windows-msvc with -std=c23 for every compiled sample, terraform 1.16.2 (published 2026-09-09), hashicorp/aws 6.64.0 (2026-09-09), and hashicorp/google 8.2.0 (2026-09-08), all Latest at access 2026-09-12. The compiler pin then drifted on this machine to 23.1.2, announced 2026-09-22, and every gate stayed green under both because the pinned asserts ask for the c23 standard version and the major 23, not a patch number, the drift rule applied to the toolchain itself, with one exception the rule caught: chapter 8′s ir expectation carried a literal patch number in the emitted ident metadata, went red on the first verify under 23.1.2, and now pins the ident at the major. Cloud pages will move again: urls relocate, js shells replace documents, sentences vanish, redirects run both directions. Cite what you fetched, record what moved, and treat the version pins and access dates as the only durable facts.

> The 381 rows behind the timeline are that record: when a future fetch disagrees with a row here, the row is what this book claimed and the date is when it was true.

sources: `coverage/topics.typ` and `coverage/sources.typ`, the two arrays this page computes every number from, and docs/audit/c-os-cloud.md, the drift register this chapter generalizes. All 381 source rows accessed 2026-09-12 and 2026-09-13 and 2026-09-14 and 2026-09-27. Documentation-verified only: no terraform binary in this corpus, nothing initialized, planned, or applied, and this chapter ships no samples, its checks are the arithmetic above.